

COA (Computer Organization and Architecture)

Generated 9/27/2026, 10:14:52 PM • 65 messages • Source: Shared ChatGPT link
<https://chatgpt.com/share/6ab947ca-5408-83ee-8489-580957bdacad>

USER

Computer Organization and Architecture

ASSISTANT

Absolutely. We'll do **Computer Organization and Architecture (COA)** in the same complete GATE-focused style as your other subjects.

For COA, I'll cover it from **zero !' GATE level !' placement/interview level**, using simple English, handwritten-note-friendly structure, shortcuts, common traps, numericals, and PYQ-style questions.

The standard GATE CS COA syllabus broadly includes:

- Machine instructions and addressing modes
- ALU, datapath and control unit
- Instruction pipelining and pipeline hazards
- Memory hierarchy
- Cache memory
- Main memory and secondary storage
- I/O interface
- Interrupt and DMA concepts

Send me the **exact GATE COA syllabus copy you are using**, and I'll convert it into a complete chapter-by-chapter roadmap and then start **Chapter 1**.

USER

Instruction set and addressing modes. Design of arithmetic and logic unit (ALU). Design of control unit – hardwired and microprogrammed. Memory interfacing and hierarchy: performance, cache memory mapping. I/O interface (interrupt and DMA). Instruction pipelining, pipeline hazards. give me a prompt based on it

This is the syllabus copy of entire gate coa syllabus so here i dont have much to see hours of youtube content please give me atoz syllabus content for gate

ASSISTANT

Yes. Your syllabus is enough to build the entire **GATE Computer Organization and Architecture course** inside ChatGPT.

I'll give you a reusable master prompt designed specifically so that you **do not need to depend on hours of YouTube lectures** and can make complete notes directly from the chat.

:::writing{variant="document" id="47318" title="GATE Computer Organization and Architecture — Complete Master Teacher Prompt"}

GATE COMPUTER ORGANIZATION AND ARCHITECTURE — COMPLETE MASTER TEACHER PROMPT

I am preparing for **GATE Computer Science** and I have limited time.

I do **NOT** have the flexibility to watch tens or hundreds of hours of YouTubelectures.

I want to learn **Computer Organization and Architecture (COA)** mainly from this chat and make my own handwritten/typed notes from your explanations.

Act as my:

- GATE COA subject expert
- personal teacher
- concept explainer
- notes maker
- PYQ-oriented mentor
- numerical problem-solving trainer
- revision coach
- shortcut and memory-trick creator
- misconception detector
- exam strategy mentor

MY CURRENT SUBJECT

Computer Organization and Architecture (COA)

OFFICIAL GATE SYLLABUS

Computer Organization and Architecture:

Instruction set and addressing modes. Design of arithmetic and logic unit (ALU). Design of control unit – hardwired and microprogrammed. Memory interfacing and hierarchy: performance, cache memory mapping. I/O interface (interrupt and DMA). Instruction pipelining, pipeline hazards.

PRIMARY GOAL

Convert this entire official syllabus into a **complete A-to-Z GATE preparation course inside this chat**.

Assume that I may have weak basics.

Start from the fundamentals required to understand the topic, but eventually take me to the level required for solving difficult GATE questions.

I should ideally be able to prepare this subject primarily from your explanations without depending on long external video courses.

Do NOT just explain the syllabus headings briefly.

Expand every syllabus heading into all the important subtopics, concepts, formulas, numericals, shortcuts, traps and question patterns that GATE can test.

VERY IMPORTANT RULE

Do NOT skip a concept merely because it is not explicitly written word-for-word in the syllabus if it is logically required to understand or solve questions from the listed GATE syllabus.

However, clearly classify topics as:

[GATE CORE] Directly important for GATE.

[GATE SUPPORT] Needed to understand GATE Core topics properly.

[PLACEMENT/INTERVIEW] Useful for technical interviews or practical CS understanding but not a major GATE priority.

[LOW PRIORITY] Useful knowledge, but not worth spending much time on for GATE.

This helps me avoid wasting time.

PHASE 1 — CREATE THE COMPLETE ROADMAP FIRST

Before teaching Chapter 1, build a complete chapter-by-chapter roadmap.

Break COA into logical modules.

At minimum verify coverage of the following areas.

MODULE 0 — REQUIRED FOUNDATIONS

Teach only the foundations necessary for later COA topics.

Include where needed:

- Bits and bytes
- Binary numbers
- Hexadecimal numbers
- Signed and unsigned representation
- 1's complement
- 2's complement
- Range of signed/unsigned numbers
- Binary addition and subtraction
- Overflow
- Basic Boolean operations
- Registers
- Clock cycle
- Word length
- Basic CPU terminology
- Memory terminology
- Instruction basics

Do not turn this into an unnecessarily huge Digital Logic course.

Teach only what COA requires.

MODULE 1 — BASIC COMPUTER ORGANIZATION

Explain:

- Basic functional units of a computer
- Processor
- ALU
- Control unit
- Registers
- Memory
- Input/output
- System buses
- Address bus
- Data bus
- Control bus
- CPU-memory interaction
- Stored-program concept
- Von Neumann architecture basics
- Instruction execution overview
- Fetch-decode-execute cycle
- Clock and CPU cycle
- Register Transfer concepts where required

Explain the role of common registers if relevant:

- PC
- IR
- MAR
- MDR/MBR
- General purpose registers
- Accumulator
- Stack pointer

Clearly indicate how much of this is GATE-relevant.

MODULE 2 — INSTRUCTION SET ARCHITECTURE

Teach thoroughly:

Instruction Basics

- What an instruction is
- Opcode
- Operand
- Instruction format
- Instruction length
- Number of operands
- Instruction fields

Instruction Types

Include concepts such as:

- Data transfer
- Arithmetic
- Logical
- Shift
- Branch
- Jump
- Procedure/function instructions
- Load/store

Number of Address Instructions

Explain:

- 0-address instructions
- 1-address instructions
- 2-address instructions
- 3-address instructions

Show expression evaluation questions.

Example type:

$$X = (A + B) \times (C - D)$$

Show how the same expression may be executed using different instruction formats.

Explain:

- Stack organization
- Accumulator organization
- General-purpose register organization

Include GATE-style comparisons involving:

- Number of instructions
- Instruction size
- Memory accesses
- Code size

MODULE 3 — ADDRESSING MODES

Teach all GATE-relevant addressing modes.

Include:

- Immediate addressing
- Direct/absolute addressing
- Indirect addressing
- Register addressing
- Register indirect addressing
- Indexed addressing
- Base addressing

- Base-index addressing if relevant
- Relative/PC-relative addressing
- Auto-increment
- Auto-decrement
- Stack addressing
- Implied addressing

For each addressing mode explain:

1. Meaning
2. Effective address formula
3. Diagram/intuitive interpretation
4. Example
5. Number of memory accesses
6. Common GATE traps

Create a table containing:

Mode | Address field meaning | Effective address | Operand location | Memory accesses

Teach effective-address numericals very carefully.

MODULE 4 — ALU DESIGN

Teach the design and working of an Arithmetic and Logic Unit.

Start from basics.

Include:

Arithmetic Operations

- Binary addition
- Binary subtraction
- Increment
- Decrement
- Addition using 2's complement
- Carry
- Borrow
- Overflow

Adders

Where required for GATE:

- Half adder
- Full adder
- Ripple Carry Adder
- Carry propagation
- Carry Look Ahead Adder intuition
- Generate and propagate concepts if relevant

Explain speed comparison.

Logical Operations

- AND
- OR
- XOR
- NOT
- Basic ALU selection mechanism

Shift Operations

Where required:

- Logical shift
- Arithmetic shift
- Left/right shifts

ALU Structure

Explain:

- Arithmetic circuit
- Logic circuit
- Multiplexer-based selection
- Control inputs
- Flags/status bits

Include:

- Carry flag
- Zero flag
- Sign/negative flag
- Overflow flag

Teach GATE-style ALU numericals and conceptual questions.

MODULE 5 — CONTROL UNIT

Start with:

- Why the CPU needs a control unit
- Control signals
- Micro-operations
- Register transfer
- Instruction execution steps
- Timing signals

Then teach the two designs separately.

HARDWIRED CONTROL UNIT

Explain:

- Basic structure
- Control signal generation
- Instruction decoder
- Timing generator
- Control logic
- State-based operation

Advantages and disadvantages.

Explain why it is generally:

- Faster
 - Less flexible
 - Harder to modify
-

MICROPROGRAMMED CONTROL UNIT

Explain from zero:

- Control memory
- Microinstruction
- Microprogram
- Control word
- Control address register
- Microprogram sequencer

Explain:

- Horizontal microprogramming
- Vertical microprogramming

Compare:

- Instruction width
- Parallelism
- Control memory requirement
- Speed
- Complexity

Teach GATE questions involving microinstruction format if relevant.

HARDWIRED VS MICROPROGRAMMED

Create a comparison table containing:

- Speed
 - Flexibility
 - Hardware complexity
 - Modification
-

- Cost
- Typical usage

Include GATE traps.

MODULE 6 — MEMORY SYSTEM FUNDAMENTALS

Before cache, teach:

- Memory hierarchy
- Registers
- Cache
- Main memory
- Secondary memory
- Why memory hierarchy exists
- Locality of reference

Explain:

Locality

- Temporal locality
- Spatial locality

Give intuitive examples.

Explain important terminology:

- Memory word
- Address
- Byte-addressable memory
- Word-addressable memory
- Address space
- Memory capacity

Teach questions like:

- Number of address bits required
 - Number of memory locations
 - Word size
 - Capacity calculation
-

MODULE 7 — MEMORY PERFORMANCE

Teach:

- Memory access time
- Memory cycle time
- Hit
- Miss
- Hit ratio

- Miss ratio
- Hit time
- Miss penalty

Teach **Average Memory Access Time (AMAT)**.

Derive formulas rather than making me memorize blindly.

Include different variants of the formula used in GATE questions.

Teach multilevel cache AMAT where relevant.

Include numerical questions.

MODULE 8 — MEMORY INTERFACING

Explain memory interfacing thoroughly but in an exam-oriented way.

Include:

- Address lines
- Data lines
- Chip select
- Memory chips
- Memory expansion

Teach:

Increasing Memory Capacity

- Word expansion
- Bit expansion
- Both together

Questions such as:

“Using smaller memory chips, construct a larger memory.”

Teach how to calculate:

- Number of chips required
- Address lines
- Decoder inputs
- Chip-select logic

Include memory address range questions where relevant.

MODULE 9 — CACHE MEMORY FUNDAMENTALS

Teach cache from zero.

Explain:

- Why cache is needed
- Cache line/block
- CPU block

- Main-memory block
- Cache hit
- Cache miss
- Cache mapping

Important terms:

- Block size
- Cache size
- Number of cache lines
- Address fields
- Tag
- Index/set
- Block/word offset

Teach address decomposition carefully.

MODULE 10 — DIRECT MAPPED CACHE

Teach completely:

- Mapping formula
- Cache line selection
- Tag
- Index
- Offset

Teach:

Cache line = Main memory block number mod Number of cache lines

Explain address breakdown.

Include numericals involving:

- Cache size
- Main-memory size
- Block size
- Tag bits
- Index bits
- Offset bits

Show both:

- Byte-addressable memory
- Word-addressable memory

Explain common traps.

MODULE 11 — FULLY ASSOCIATIVE CACHE

Explain:

- Mapping

- Tag comparison
- No fixed index
- Address fields
- Hardware requirement
- Advantages
- Disadvantages

Teach GATE numericals involving tag bits.

MODULE 12 — SET ASSOCIATIVE CACHE

Teach:

- k-way set associative cache
- Number of sets
- Mapping formula

Set number = Block number mod Number of sets

Teach address fields:

- Tag
- Set/index
- Offset

Cover:

- 2-way
- 4-way
- general k-way

Include numerical questions.

MODULE 13 — CACHE REPLACEMENT AND WRITE POLICIES

Where GATE-relevant, explain:

Replacement Policies

- LRU
- FIFO
- Random

Teach replacement numericals.

Write Policies

- Write-through
- Write-back
- Write allocate
- No-write allocate

Explain dirty bit where needed.

Compare memory traffic.

MODULE 14 — CACHE PERFORMANCE NUMERICALS

Create a dedicated numerical chapter.

Include:

- Hit ratio calculations
- Effective access time
- AMAT
- Cache + memory access
- Multi-level cache
- Instruction/data memory accesses if relevant
- Write-policy effects
- CPI impact caused by cache misses

Teach both:

- Direct method
- Shortcut method

Include common GATE traps.

MODULE 15 — I/O ORGANIZATION

Explain:

- Input/output devices
- I/O controller/module
- CPU-I/O communication
- I/O interface
- Status registers
- Data registers
- Control registers

Where relevant teach:

- Memory-mapped I/O
- Isolated/port-mapped I/O

Compare them.

MODULE 16 — PROGRAMMED I/O AND INTERRUPTS

First explain programmed I/O/polling.

Then teach interrupts thoroughly.

Include:

- What an interrupt is
 - Why interrupts are required
-

- Interrupt request
- Interrupt acknowledgement
- Interrupt Service Routine (ISR)
- Context switching basics
- Returning from interrupt

Where relevant:

- Maskable interrupt
- Non-maskable interrupt
- Vectored interrupt
- Non-vectored interrupt
- Interrupt priority
- Nested interrupts

Explain polling vs interrupt-driven I/O.

Include GATE-style numerical questions if applicable.

MODULE 17 — DMA

Teach Direct Memory Access from zero.

Explain:

- Why DMA is needed
- DMA controller
- CPU involvement
- Memory transfer
- Bus control

DMA operation sequence.

Include:

- Burst/block mode
- Cycle stealing
- Transparent DMA if relevant

Compare:

Programmed I/O vs Interrupt-driven I/O vs DMA

Explain CPU overhead.

Teach numerical questions involving DMA where GATE can ask them.

MODULE 18 — INSTRUCTION PIPELINING BASICS

Teach pipelining very thoroughly.

Start with non-pipelined execution.

Then explain:

- Pipeline concept

- Pipeline stages
- Typical instruction pipeline

For example:

- IF
- ID
- EX
- MEM
- WB

Explain:

- Latency
- Throughput
- Pipeline clock cycle
- Pipeline fill time
- Pipeline completion time

Derive formulas.

Teach:

For k stages and n instructions:

Non-pipelined execution time

vs

Pipelined execution time.

Teach speedup.

Explain ideal maximum speedup.

MODULE 19 — PIPELINE PERFORMANCE

Teach numerical concepts:

- Clock period
- Stage delay
- Pipeline register overhead if relevant
- Number of stages
- Number of instructions
- Execution time
- Throughput
- Latency
- Speedup
- Efficiency

Teach formulas with derivation.

Explain unequal pipeline stages.

Teach GATE-style questions where stage delays are given.

MODULE 20 — PIPELINE HAZARDS

Teach all three major hazards.

STRUCTURAL HAZARD

Explain:

- Why it occurs
 - Resource conflict
 - Example
 - Solution
-

DATA HAZARDS

Teach:

- RAW — Read After Write
- WAR — Write After Read
- WAW — Write After Write

Explain which hazards occur in typical pipelines.

Teach dependency detection.

CONTROL HAZARDS

Explain:

- Branch instructions
- Branch penalty
- Pipeline flushing
- Stall

Where relevant mention:

- Branch prediction
- Delayed branching

Keep low-priority advanced material clearly labelled.

MODULE 21 — DATA HAZARD SOLUTIONS

Explain:

- Pipeline stall
- Bubble
- NOP
- Operand forwarding/bypassing
- Instruction scheduling

Teach dependency timing tables.

Show how many stalls are required:

- Without forwarding
- With forwarding

Include load-use hazard.

This chapter should contain many GATE-style questions.

MODULE 22 — CONTROL HAZARD NUMERICALS

Teach:

- Branch penalty
- Branch frequency
- Prediction accuracy
- Number of wasted cycles
- CPI impact

Where relevant derive:

Effective CPI = Ideal CPI + stall cycles per instruction

Include numerical examples.

MODULE 23 — CPI, CPU PERFORMANCE AND RELATED CONCEPTS

Although not explicitly written separately in the syllabus, teach all performance concepts required to solve pipeline/cache questions.

Explain:

- CPU execution time
- Clock rate
- Clock cycle time
- Instruction count
- CPI
- IPC if relevant

Core formula:

CPU Time = Instruction Count × CPI × Clock Cycle Time

or

CPU Time = (Instruction Count × CPI) / Clock Rate

Teach:

- Average CPI
- Different instruction classes
- Stall contribution
- Speedup

Where appropriate introduce Amdahl's Law only if useful for GATE COA.

MODULE 24 — MIXED COA NUMERICALS

Create mixed practice involving multiple concepts.

For example:

- Cache + CPI
- Pipeline + branch penalty
- Cache misses + execution time
- Instruction count + CPI + clock rate
- Addressing mode + memory access
- Memory hierarchy + effective access time

These should resemble real GATE conceptual/numerical combinations.

MODULE 25 — COMPLETE COA REVISION

At the end, create:

1. Formula Sheet

All important formulas in one place.

2. Cache Cheat Sheet

Direct Associative Set-associative

3. Pipeline Cheat Sheet

Execution time Speedup Throughput Hazards Stalls

4. Addressing Mode Cheat Sheet

Mode !' EA formula.

5. Control Unit Comparison

Hardwired vs Microprogrammed.

6. I/O Comparison

Polling vs Interrupt vs DMA.

7. Top GATE Traps

List the common mistakes students make.

8. Last-Day Revision Notes

A highly compressed revision guide.

HOW TO TEACH EVERY TOPIC

For every important concept, follow approximately this structure:

1. INTUITION

Explain the idea in very simple language.

2. WHY DO WE NEED IT?

Explain the problem it solves.

3. FORMAL CONCEPT

Give the technically correct definition.

4. DIAGRAM

Use simple ASCII diagrams whenever they help.

Example:

CPU !' Cache !' Main Memory !' Secondary Storage

5. FORMULA

Introduce formulas only after explaining the intuition.

6. DERIVATION

Derive important formulas instead of expecting blind memorization.

7. SIMPLE EXAMPLE

Use a small example.

8. GATE-LEVEL EXAMPLE

Solve a realistic exam problem.

9. SHORTCUT

Show the fastest safe exam method.

10. COMMON TRAP

Explain the mistake GATE expects students to make.

11. MEMORY TRICK

Give a short mnemonic when useful.

12. EXAM APPLICATION

Explain exactly how GATE asks this concept.

13. PRACTICE

Give a few questions.

NUMERICAL PROBLEM RULE

For every important numerical category, teach:

Method 1 — Full conceptual method

Then:

Method 2 — Fast GATE exam method

Also explain when the shortcut should NOT be used.

Never directly throw formulas at me without explaining what each variable means.

GATE QUESTION TYPES

Train me for:

- MCQ — single correct
- MSQ — multiple correct
- NAT — numerical answer type

For conceptual questions explain how to eliminate wrong options.

For NAT questions show:

- Units
 - Approximation
 - Rounding
 - Common arithmetic mistakes
-

FOUR LEVEL LEARNING SYSTEM

Mark questions as:

L0 — Foundation

Very simple understanding.

L1 — Standard

Basic GATE-level application.

L2 — Strong GATE

Multi-step or tricky application.

L3 — Advanced GATE

Difficult conceptual or combined problem.

Do not make everything L3.

Build difficulty gradually.

PYQ-ORIENTED TEACHING

For every chapter identify:

- Frequently tested concepts
 - Common GATE patterns
 - Typical numerical patterns
-

- Common conceptual traps
- High-yield formulas

If you know a historical GATE-type question, explain the concept and question pattern.

Do not fabricate exact year/question-number claims if uncertain.

MEMORY TRICKS

Create useful memory tricks.

Examples:

Cache address

T-I-O

Tag !' Index !' Offset

For set associative:

T-S-O

Tag !' Set !' Offset

Data Hazards

RAW = Real dependency

Read waits for an earlier Write.

Memory hierarchy

Faster !' Smaller !' Costlier

Registers !' Cache !' RAM !' Storage

Use similar mnemonics throughout the course.

NOTE-MAKING STYLE

My goal is to make handwritten/typed notes.

Therefore:

- Use clear headings
- Use compact tables
- Highlight formulas
- Keep definitions concise
- Separate examples
- Separate traps
- Give one-line revision points
- Avoid unnecessary long paragraphs
- Do not remove conceptual depth merely to make the answer shorter

Use simple English.

If a technical term is necessary, explain it immediately.

CHAPTER END FORMAT

At the end of every chapter give:

CHAPTER SNAPSHOT

5–15 key points.

MUST REMEMBER

Most important concepts/formulas.

GATE TRAPS

Common mistakes.

MEMORY TRICKS

Mnemonics.

PRACTICE SET

Include MCQ/MSQ/NAT questions.

Do NOT show the answers immediately unless I ask or unless answers are placed after enough spacing so I can attempt them first.

COMPLETION STATUS

Show:

- Chapters completed
 - Chapters remaining
 - Approximate percentage of COA completed
-

UNIT REVISION TESTS

After every few related chapters give a mixed mini-test.

Example groups:

Test 1

Instruction Set + Addressing Modes

Test 2

ALU + Control Unit

Test 3

Memory + Cache

Test 4

I/O + Interrupt + DMA

Test 5

Pipelining + Hazards

Then give a final complete COA mock revision set.

HIGH-YIELD PRIORITY

Whenever relevant, mark:

Ø=~~V~~**VERY HIGH GATE PRIORITY**

+~~H~~**HIGH PRIORITY**

' **STANDARD PRIORITY**

%~~E~~**LOW PRIORITY**

Based on common GATE-style importance.

Do not let priority labels cause you to completely omit syllabus content.

PLACEMENT / INTERVIEW EXTENSION

My primary goal here is GATE.

However, after completing the GATE portion of a topic, briefly mention practical/interview relevance when useful.

Examples:

- CPU cache behaviour
- Locality
- CPU pipeline
- Registers
- Memory hierarchy
- Interrupts

Do not let placement content dominate the GATE course.

THINGS YOU MUST NOT DO

Do NOT:

- Skip chapters because they look basic
 - Give only definitions
 - Dump formulas without explanation
 - Assume I already understand architecture
 - Make every answer extremely short
 - Teach irrelevant advanced computer architecture for many hours
 - Send me to YouTube instead of explaining the concept
 - Say “refer to a standard textbook” instead of teaching it
-

- Fabricate GATE PYQ facts
 - Jump randomly between chapters
-

IMPORTANT COMPLETION RULE

Maintain a syllabus checklist throughout the course.

Whenever I type:

NEXT

continue from exactly where we stopped.

Do NOT restart the course.

Do NOT repeat previous chapters unless revision is required.

Before declaring the course complete, compare everything taught against the original syllabus:

Instruction set and addressing modes. Design of arithmetic and logic unit (ALU). Design of control unit – hardwired and microprogrammed. Memory interfacing and hierarchy: performance, cache memory mapping. I/O interface (interrupt and DMA). Instruction pipelining, pipeline hazards.

Check every phrase and make sure nothing important has been missed.

Then explicitly tell me:

GATE COA SYLLABUS COVERAGE: COMPLETE

only when it is genuinely complete.

START NOW

First give me:

1. Complete COA syllabus decomposition
2. Chapter-by-chapter roadmap
3. Priority of each chapter
4. Estimated relative GATE importance
5. Prerequisite map
6. Complete syllabus checklist
7. Recommended learning order

Then immediately begin:

CHAPTER 0 — COA FOUNDATIONS

Teach from absolute basics but remain GATE-oriented.

I will type **NEXT** whenever I want the next section. :::

This version is deliberately designed to prevent two common problems: **missing hidden prerequisite topics** and **spending too much time on material outside GATE**.

You can paste this into a fresh chat. After that, just keep writing “**NEXT**”, and the course should progress chapter by chapter until the entire COA syllabus is checked off.

GATE COMPUTER ORGANIZATION AND ARCHITECTURE — COMPLETE MASTER TEACHER PROMPT

I am preparing for **GATE Computer Science** and I have limited time.

I do **NOT** have the flexibility to watch tens or hundreds of hours of YouTubelectures.

I want to learn **Computer Organization and Architecture (COA)** mainly from this chat and make my own handwritten/typed notes from your explanations.

Act as my:

- GATE COA subject expert
- personal teacher
- concept explainer
- notes maker
- PYQ-oriented mentor
- numerical problem-solving trainer
- revision coach
- shortcut and memory-trick creator
- misconception detector
- exam strategy mentor

MY CURRENT SUBJECT

Computer Organization and Architecture (COA)

OFFICIAL GATE SYLLABUS

Computer Organization and Architecture:

Instruction set and addressing modes. Design of arithmetic and logic unit (ALU). Design of control unit – hardwired and microprogrammed. Memory interfacing and hierarchy: performance, cache memory mapping. I/O interface (interrupt and DMA). Instruction pipelining, pipeline hazards.

PRIMARY GOAL

Convert this entire official syllabus into a **complete A-to-Z GATE preparation course inside this chat**.

Assume that I may have weak basics.

Start from the fundamentals required to understand the topic, but eventually take me to the level required for solving difficult GATE questions.

I should ideally be able to prepare this subject primarily from your explanations without depending on long external video courses.

Do NOT just explain the syllabus headings briefly.

Expand every syllabus heading into all the important subtopics, concepts, formulas, numericals, shortcuts, traps and

question patterns that GATE can test.

VERY IMPORTANT RULE

Do NOT skip a concept merely because it is not explicitly written word-for-word in the syllabus if it is logically required to understand or solve questions from the listed GATE syllabus.

However, clearly classify topics as:

[GATE CORE] Directly important for GATE.

[GATE SUPPORT] Needed to understand GATE Core topics properly.

[PLACEMENT/INTERVIEW] Useful for technical interviews or practical CS understanding but not a major GATE priority.

[LOW PRIORITY] Useful knowledge, but not worth spending much time on for GATE.

This helps me avoid wasting time.

PHASE 1 — CREATE THE COMPLETE ROADMAP FIRST

Before teaching Chapter 1, build a complete chapter-by-chapter roadmap.

Break COA into logical modules.

At minimum verify coverage of the following areas.

MODULE 0 — REQUIRED FOUNDATIONS

Teach only the foundations necessary for later COA topics.

Include where needed:

- Bits and bytes
- Binary numbers
- Hexadecimal numbers
- Signed and unsigned representation
- 1's complement
- 2's complement
- Range of signed/unsigned numbers
- Binary addition and subtraction
- Overflow
- Basic Boolean operations
- Registers
- Clock cycle
- Word length
- Basic CPU terminology
- Memory terminology
- Instruction basics

Do not turn this into an unnecessarily huge Digital Logic course.

Teach only what COA requires.

MODULE 1 — BASIC COMPUTER ORGANIZATION

Explain:

- Basic functional units of a computer
- Processor
- ALU
- Control unit
- Registers
- Memory
- Input/output
- System buses
- Address bus
- Data bus
- Control bus
- CPU-memory interaction
- Stored-program concept
- Von Neumann architecture basics
- Instruction execution overview
- Fetch-decode-execute cycle
- Clock and CPU cycle
- Register Transfer concepts where required

Explain the role of common registers if relevant:

- PC
- IR
- MAR
- MDR/MBR
- General purpose registers
- Accumulator
- Stack pointer

Clearly indicate how much of this is GATE-relevant.

MODULE 2 — INSTRUCTION SET ARCHITECTURE

Teach thoroughly:

Instruction Basics

- What an instruction is
- Opcode
- Operand
- Instruction format

- Instruction length
- Number of operands
- Instruction fields

Instruction Types

Include concepts such as:

- Data transfer
- Arithmetic
- Logical
- Shift
- Branch
- Jump
- Procedure/function instructions
- Load/store

Number of Address Instructions

Explain:

- 0-address instructions
- 1-address instructions
- 2-address instructions
- 3-address instructions

Show expression evaluation questions.

Example type:

$$X = (A + B) \times (C - D)$$

Show how the same expression may be executed using different instruction formats.

Explain:

- Stack organization
- Accumulator organization
- General-purpose register organization

Include GATE-style comparisons involving:

- Number of instructions
- Instruction size
- Memory accesses
- Code size

MODULE 3 — ADDRESSING MODES

Teach all GATE-relevant addressing modes.

Include:

- Immediate addressing
- Direct/absolute addressing

- Indirect addressing
- Register addressing
- Register indirect addressing
- Indexed addressing
- Base addressing
- Base-index addressing if relevant
- Relative/PC-relative addressing
- Auto-increment
- Auto-decrement
- Stack addressing
- Implied addressing

For each addressing mode explain:

1. Meaning
2. Effective address formula
3. Diagram/intuitive interpretation
4. Example
5. Number of memory accesses
6. Common GATE traps

Create a table containing:

Mode | Address field meaning | Effective address | Operand location | Memory accesses

Teach effective-address numericals very carefully.

MODULE 4 — ALU DESIGN

Teach the design and working of an Arithmetic and Logic Unit.

Start from basics.

Include:

Arithmetic Operations

- Binary addition
- Binary subtraction
- Increment
- Decrement
- Addition using 2's complement
- Carry
- Borrow
- Overflow

Adders

Where required for GATE:

- Half adder

- Full adder
- Ripple Carry Adder
- Carry propagation
- Carry Look Ahead Adder intuition
- Generate and propagate concepts if relevant

Explain speed comparison.

Logical Operations

- AND
- OR
- XOR
- NOT
- Basic ALU selection mechanism

Shift Operations

Where required:

- Logical shift
- Arithmetic shift
- Left/right shifts

ALU Structure

Explain:

- Arithmetic circuit
- Logic circuit
- Multiplexer-based selection
- Control inputs
- Flags/status bits

Include:

- Carry flag
- Zero flag
- Sign/negative flag
- Overflow flag

Teach GATE-style ALU numericals and conceptual questions.

MODULE 5 — CONTROL UNIT

Start with:

- Why the CPU needs a control unit
- Control signals
- Micro-operations
- Register transfer

- Instruction execution steps
- Timing signals

Then teach the two designs separately.

HARDWIRED CONTROL UNIT

Explain:

- Basic structure
- Control signal generation
- Instruction decoder
- Timing generator
- Control logic
- State-based operation

Advantages and disadvantages.

Explain why it is generally:

- Faster
 - Less flexible
 - Harder to modify
-

MICROPROGRAMMED CONTROL UNIT

Explain from zero:

- Control memory
- Microinstruction
- Microprogram
- Control word
- Control address register
- Microprogram sequencer

Explain:

- Horizontal microprogramming
- Vertical microprogramming

Compare:

- Instruction width
- Parallelism
- Control memory requirement
- Speed
- Complexity

Teach GATE questions involving microinstruction format if relevant.

HARDWIRED VS MICROPROGRAMMED

Create a comparison table containing:

- Speed
- Flexibility
- Hardware complexity
- Modification
- Cost
- Typical usage

Include GATE traps.

MODULE 6 — MEMORY SYSTEM FUNDAMENTALS

Before cache, teach:

- Memory hierarchy
- Registers
- Cache
- Main memory
- Secondary memory
- Why memory hierarchy exists
- Locality of reference

Explain:

Locality

- Temporal locality
- Spatial locality

Give intuitive examples.

Explain important terminology:

- Memory word
- Address
- Byte-addressable memory
- Word-addressable memory
- Address space
- Memory capacity

Teach questions like:

- Number of address bits required
 - Number of memory locations
 - Word size
 - Capacity calculation
-

MODULE 7 — MEMORY PERFORMANCE

Teach:

- Memory access time

- Memory cycle time
- Hit
- Miss
- Hit ratio
- Miss ratio
- Hit time
- Miss penalty

Teach **Average Memory Access Time (AMAT)**.

Derive formulas rather than making me memorize blindly.

Include different variants of the formula used in GATE questions.

Teach multilevel cache AMAT where relevant.

Include numerical questions.

MODULE 8 — MEMORY INTERFACING

Explain memory interfacing thoroughly but in an exam-oriented way.

Include:

- Address lines
- Data lines
- Chip select
- Memory chips
- Memory expansion

Teach:

Increasing Memory Capacity

- Word expansion
- Bit expansion
- Both together

Questions such as:

“Using smaller memory chips, construct a larger memory.”

Teach how to calculate:

- Number of chips required
- Address lines
- Decoder inputs
- Chip-select logic

Include memory address range questions where relevant.

MODULE 9 — CACHE MEMORY FUNDAMENTALS

Teach cache from zero.

Explain:

- Why cache is needed
- Cache line/block
- CPU block
- Main-memory block
- Cache hit
- Cache miss
- Cache mapping

Important terms:

- Block size
- Cache size
- Number of cache lines
- Address fields
- Tag
- Index/set
- Block/word offset

Teach address decomposition carefully.

MODULE 10 — DIRECT MAPPED CACHE

Teach completely:

- Mapping formula
- Cache line selection
- Tag
- Index
- Offset

Teach:

Cache line = Main memory block number mod Number of cache lines

Explain address breakdown.

Include numericals involving:

- Cache size
- Main-memory size
- Block size
- Tag bits
- Index bits
- Offset bits

Show both:

- Byte-addressable memory
- Word-addressable memory

Explain common traps.

MODULE 11 — FULLY ASSOCIATIVE CACHE

Explain:

- Mapping
- Tag comparison
- No fixed index
- Address fields
- Hardware requirement
- Advantages
- Disadvantages

Teach GATE numericals involving tag bits.

MODULE 12 — SET ASSOCIATIVE CACHE

Teach:

- k-way set associative cache
- Number of sets
- Mapping formula

Set number = Block number mod Number of sets

Teach address fields:

- Tag
- Set/index
- Offset

Cover:

- 2-way
- 4-way
- general k-way

Include numerical questions.

MODULE 13 — CACHE REPLACEMENT AND WRITE POLICIES

Where GATE-relevant, explain:

Replacement Policies

- LRU
- FIFO
- Random

Teach replacement numericals.

Write Policies

- Write-through
- Write-back
- Write allocate
- No-write allocate

Explain dirty bit where needed.

Compare memory traffic.

MODULE 14 — CACHE PERFORMANCE NUMERICALS

Create a dedicated numerical chapter.

Include:

- Hit ratio calculations
- Effective access time
- AMAT
- Cache + memory access
- Multi-level cache
- Instruction/data memory accesses if relevant
- Write-policy effects
- CPI impact caused by cache misses

Teach both:

- Direct method
- Shortcut method

Include common GATE traps.

MODULE 15 — I/O ORGANIZATION

Explain:

- Input/output devices
- I/O controller/module
- CPU-I/O communication
- I/O interface
- Status registers
- Data registers
- Control registers

Where relevant teach:

- Memory-mapped I/O
- Isolated/port-mapped I/O

Compare them.

MODULE 16 — PROGRAMMED I/O AND INTERRUPTS

First explain programmed I/O/polling.

Then teach interrupts thoroughly.

Include:

- What an interrupt is
- Why interrupts are required
- Interrupt request
- Interrupt acknowledgement
- Interrupt Service Routine (ISR)
- Context switching basics
- Returning from interrupt

Where relevant:

- Maskable interrupt
- Non-maskable interrupt
- Vectored interrupt
- Non-vectored interrupt
- Interrupt priority
- Nested interrupts

Explain polling vs interrupt-driven I/O.

Include GATE-style numerical questions if applicable.

MODULE 17 — DMA

Teach Direct Memory Access from zero.

Explain:

- Why DMA is needed
- DMA controller
- CPU involvement
- Memory transfer
- Bus control

DMA operation sequence.

Include:

- Burst/block mode
- Cycle stealing
- Transparent DMA if relevant

Compare:

Programmed I/O vs Interrupt-driven I/O vs DMA

Explain CPU overhead.

Teach numerical questions involving DMA where GATE can ask them.

MODULE 18 — INSTRUCTION PIPELINING BASICS

Teach pipelining very thoroughly.

Start with non-pipelined execution.

Then explain:

- Pipeline concept
- Pipeline stages
- Typical instruction pipeline

For example:

- IF
- ID
- EX
- MEM
- WB

Explain:

- Latency
- Throughput
- Pipeline clock cycle
- Pipeline fill time
- Pipeline completion time

Derive formulas.

Teach:

For k stages and n instructions:

Non-pipelined execution time

vs

Pipelined execution time.

Teach speedup.

Explain ideal maximum speedup.

MODULE 19 — PIPELINE PERFORMANCE

Teach numerical concepts:

- Clock period
- Stage delay
- Pipeline register overhead if relevant
- Number of stages
- Number of instructions
- Execution time
- Throughput
- Latency

- Speedup
- Efficiency

Teach formulas with derivation.

Explain unequal pipeline stages.

Teach GATE-style questions where stage delays are given.

MODULE 20 — PIPELINE HAZARDS

Teach all three major hazards.

STRUCTURAL HAZARD

Explain:

- Why it occurs
 - Resource conflict
 - Example
 - Solution
-

DATA HAZARDS

Teach:

- RAW — Read After Write
- WAR — Write After Read
- WAW — Write After Write

Explain which hazards occur in typical pipelines.

Teach dependency detection.

CONTROL HAZARDS

Explain:

- Branch instructions
- Branch penalty
- Pipeline flushing
- Stall

Where relevant mention:

- Branch prediction
- Delayed branching

Keep low-priority advanced material clearly labelled.

MODULE 21 — DATA HAZARD SOLUTIONS

Explain:

- Pipeline stall
- Bubble
- NOP
- Operand forwarding/bypassing
- Instruction scheduling

Teach dependency timing tables.

Show how many stalls are required:

- Without forwarding
- With forwarding

Include load-use hazard.

This chapter should contain many GATE-style questions.

MODULE 22 — CONTROL HAZARD NUMERICALS

Teach:

- Branch penalty
- Branch frequency
- Prediction accuracy
- Number of wasted cycles
- CPI impact

Where relevant derive:

Effective CPI = Ideal CPI + stall cycles per instruction

Include numerical examples.

MODULE 23 — CPI, CPU PERFORMANCE AND RELATED CONCEPTS

Although not explicitly written separately in the syllabus, teach all performance concepts required to solve pipeline/cache questions.

Explain:

- CPU execution time
- Clock rate
- Clock cycle time
- Instruction count
- CPI
- IPC if relevant

Core formula:

CPU Time = Instruction Count × CPI × Clock Cycle Time

or

CPU Time = (Instruction Count × CPI) / Clock Rate

Teach:

- Average CPI
- Different instruction classes
- Stall contribution
- Speedup

Where appropriate introduce Amdahl's Law only if useful for GATE COA.

MODULE 24 — MIXED COA NUMERICALS

Create mixed practice involving multiple concepts.

For example:

- Cache + CPI
- Pipeline + branch penalty
- Cache misses + execution time
- Instruction count + CPI + clock rate
- Addressing mode + memory access
- Memory hierarchy + effective access time

These should resemble real GATE conceptual/numerical combinations.

MODULE 25 — COMPLETE COA REVISION

At the end, create:

1. Formula Sheet

All important formulas in one place.

2. Cache Cheat Sheet

Direct Associative Set-associative

3. Pipeline Cheat Sheet

Execution time Speedup Throughput Hazards Stalls

4. Addressing Mode Cheat Sheet

Mode !' EA formula.

5. Control Unit Comparison

Hardwired vs Microprogrammed.

6. I/O Comparison

Polling vs Interrupt vs DMA.

7. Top GATE Traps

List the common mistakes students make.

8. Last-Day Revision Notes

A highly compressed revision guide.

HOW TO TEACH EVERY TOPIC

For every important concept, follow approximately this structure:

1. INTUITION

Explain the idea in very simple language.

2. WHY DO WE NEED IT?

Explain the problem it solves.

3. FORMAL CONCEPT

Give the technically correct definition.

4. DIAGRAM

Use simple ASCII diagrams whenever they help.

Example:

CPU !' Cache !' Main Memory !' Secondary Storage

5. FORMULA

Introduce formulas only after explaining the intuition.

6. DERIVATION

Derive important formulas instead of expecting blind memorization.

7. SIMPLE EXAMPLE

Use a small example.

8. GATE-LEVEL EXAMPLE

Solve a realistic exam problem.

9. SHORTCUT

Show the fastest safe exam method.

10. COMMON TRAP

Explain the mistake GATE expects students to make.

11. MEMORY TRICK

Give a short mnemonic when useful.

12. EXAM APPLICATION

Explain exactly how GATE asks this concept.

13. PRACTICE

Give a few questions.

NUMERICAL PROBLEM RULE

For every important numerical category, teach:

Method 1 — Full conceptual method

Then:

Method 2 — Fast GATE exam method

Also explain when the shortcut should NOT be used.

Never directly throw formulas at me without explaining what each variable means.

GATE QUESTION TYPES

Train me for:

- MCQ — single correct
- MSQ — multiple correct
- NAT — numerical answer type

For conceptual questions explain how to eliminate wrong options.

For NAT questions show:

- Units
 - Approximation
 - Rounding
 - Common arithmetic mistakes
-

FOUR LEVEL LEARNING SYSTEM

Mark questions as:

L0 — Foundation

Very simple understanding.

L1 — Standard

Basic GATE-level application.

L2 — Strong GATE

Multi-step or tricky application.

L3 — Advanced GATE

Difficult conceptual or combined problem.

Do not make everything L3.

Build difficulty gradually.

PYQ-ORIENTED TEACHING

For every chapter identify:

- Frequently tested concepts
- Common GATE patterns
- Typical numerical patterns
- Common conceptual traps
- High-yield formulas

If you know a historical GATE-type question, explain the concept and question pattern.

Do not fabricate exact year/question-number claims if uncertain.

MEMORY TRICKS

Create useful memory tricks.

Examples:

Cache address

T-I-O

Tag !' Index !' Offset

For set associative:

T-S-O

Tag !' Set !' Offset

Data Hazards

RAW = Real dependency

Read waits for an earlier Write.

Memory hierarchy

Faster !' Smaller !' Costlier

Registers !' Cache !' RAM !' Storage

Use similar mnemonics throughout the course.

NOTE-MAKING STYLE

My goal is to make handwritten/typed notes.

Therefore:

- Use clear headings
- Use compact tables
- Highlight formulas
- Keep definitions concise

- Separate examples
- Separate traps
- Give one-line revision points
- Avoid unnecessary long paragraphs
- Do not remove conceptual depth merely to make the answer shorter

Use simple English.

If a technical term is necessary, explain it immediately.

CHAPTER END FORMAT

At the end of every chapter give:

CHAPTER SNAPSHOT

5–15 key points.

MUST REMEMBER

Most important concepts/formulas.

GATE TRAPS

Common mistakes.

MEMORY TRICKS

Mnemonics.

PRACTICE SET

Include MCQ/MSQ/NAT questions.

Do NOT show the answers immediately unless I ask or unless answers are placed after enough spacing so I can attempt them first.

COMPLETION STATUS

Show:

- Chapters completed
 - Chapters remaining
 - Approximate percentage of COA completed
-

UNIT REVISION TESTS

After every few related chapters give a mixed mini-test.

Example groups:

Test 1

Test 2

ALU + Control Unit

Test 3

Memory + Cache

Test 4

I/O + Interrupt + DMA

Test 5

Pipelining + Hazards

Then give a final complete COA mock revision set.

HIGH-YIELD PRIORITY

Whenever relevant, mark:

Ø=~~V~~**VERY HIGH GATE PRIORITY**

+~~H~~**HIGH PRIORITY**

' **STANDARD PRIORITY**

%~~E~~**LOW PRIORITY**

Based on common GATE-style importance.

Do not let priority labels cause you to completely omit syllabus content.

PLACEMENT / INTERVIEW EXTENSION

My primary goal here is GATE.

However, after completing the GATE portion of a topic, briefly mention practical/interview relevance when useful.

Examples:

- CPU cache behaviour
- Locality
- CPU pipeline
- Registers
- Memory hierarchy
- Interrupts

Do not let placement content dominate the GATE course.

THINGS YOU MUST NOT DO

Do NOT:

- Skip chapters because they look basic
-

- Give only definitions
 - Dump formulas without explanation
 - Assume I already understand architecture
 - Make every answer extremely short
 - Teach irrelevant advanced computer architecture for many hours
 - Send me to YouTube instead of explaining the concept
 - Say “refer to a standard textbook” instead of teaching it
 - Fabricate GATE PYQ facts
 - Jump randomly between chapters
-

IMPORTANT COMPLETION RULE

Maintain a syllabus checklist throughout the course.

Whenever I type:

NEXT

continue from exactly where we stopped.

Do NOT restart the course.

Do NOT repeat previous chapters unless revision is required.

Before declaring the course complete, compare everything taught against the original syllabus:

Instruction set and addressing modes. Design of arithmetic and logic unit (ALU). Design of control unit – hardwired and microprogrammed. Memory interfacing and hierarchy: performance, cache memory mapping. I/O interface (interrupt and DMA). Instruction pipelining, pipeline hazards.

Check every phrase and make sure nothing important has been missed.

Then explicitly tell me:

GATE COA SYLLABUS COVERAGE: COMPLETE

only when it is genuinely complete.

START NOW

First give me:

1. Complete COA syllabus decomposition
2. Chapter-by-chapter roadmap
3. Priority of each chapter
4. Estimated relative GATE importance
5. Prerequisite map
6. Complete syllabus checklist
7. Recommended learning order

Then immediately begin:

CHAPTER 0 — COA FOUNDATIONS

Teach from absolute basics but remain GATE-oriented.

I will type **NEXT** whenever I want the next section.

ASSISTANT

I'll lock the full COA roadmap first, then start **Chapter 0** immediately. I'll keep it GATE-first, note-friendly, and I'll mark what is core vs support so you don't waste time.

ASSISTANT

GATE COA — MASTER ROADMAP + CHAPTER 0

Your COA course is now started.

I will follow the syllabus you gave as the **source of truth** and keep a running completion tracker. I will also include prerequisite concepts only where they help solve GATE questions.

PART A — COMPLETE COA SYLLABUS DECOMPOSITION

Your official syllabus is:

Instruction set and addressing modes. Design of arithmetic and logic unit (ALU). Design of control unit – hardwired and microprogrammed. Memory interfacing and hierarchy: performance, cache memory mapping. I/O interface (interrupt and DMA). Instruction pipelining, pipeline hazards.

We can convert it into these 26 learning modules.

#	Module	Classification	Priority
0	COA Foundations	[GATE SUPPORT]	+P High
1	Basic Computer Organization	[GATE SUPPORT]	+P High
2	Instruction Set Architecture	[GATE CORE]	Ø=Ý% Very High
3	Addressing Modes	[GATE CORE]	Ø=Ý% Very High
4	ALU Design	[GATE CORE]	Ø=Ý% Very High
5	Control Unit	[GATE CORE]	+P High
6	Memory Fundamentals	[GATE CORE]	Ø=Ý% Very High
7	Memory Performance	[GATE CORE]	Ø=Ý% Very High
8	Memory Interfacing	[GATE CORE]	+P High
9	Cache Fundamentals	[GATE CORE]	Ø=Ý% Very High
10	Direct-Mapped Cache	[GATE CORE]	Ø=Ý% Very High
11	Fully Associative Cache	[GATE CORE]	Ø=Ý% Very High

#	Module	Classification	Priority
12	Set-Associative Cache	[GATE CORE]	Ø=Ý% Very High
13	Cache Replacement + Write Policies	[GATE CORE]	+P High
14	Cache Performance Numericals	[GATE CORE]	Ø=Ý% Very High
15	I/O Organization	[GATE SUPPORT/CORE]	+P High
16	Polling + Interrupts	[GATE CORE]	Ø=Ý% Very High
17	DMA	[GATE CORE]	Ø=Ý% Very High
18	Pipelining Basics	[GATE CORE]	Ø=Ý% Very High
19	Pipeline Performance	[GATE CORE]	Ø=Ý% Very High
20	Pipeline Hazards	[GATE CORE]	Ø=Ý% Very High
21	Data Hazard Solutions	[GATE CORE]	Ø=Ý% Very High
22	Control Hazard Numericals	[GATE CORE]	+P High
23	CPU Performance + CPI	[GATE SUPPORT/CORE]	Ø=Ý% Very High
24	Mixed COA Numericals	[GATE CORE]	Ø=Ý% Very High
25	Final COA Revision	Revision	Ø=Ý% Essential

Important: these are study priorities, not guaranteed GATE marks. The exact distribution can change from paper to paper.

PART B — RECOMMENDED LEARNING ORDER

We will follow this dependency chain:

TEXT

```

Foundations
! "
Basic CPU Organization
! "
Instruction Set
! "
Addressing Modes
! "
ALU
! "
Control Unit
! "
Memory Fundamentals
! "
Memory Performance
! "
Memory Interfacing
! "
Cache
! "
I/O
! "
```

```
Interrupts
    !"
DMA
    !"
CPU Performance
    !"
Pipelining
    !"
Hazards
    !"
Mixed Numericals
    !"
Final Revision
```

This order is deliberate.

For example, trying to understand cache address bits before understanding:

- powers of 2,
- addresses,
- bytes,
- words,

usually creates unnecessary confusion.

PART C — PREREQUISITE MAP

For Instruction Set

You need:

```
TEXT
Binary basics
!' Registers
!' CPU basics
!' Instruction basics
```

For ALU

You need:

```
TEXT
Binary numbers
!' Signed numbers
!' 2's complement
!' Binary addition
!' Overflow
!' Boolean operations
```

For Cache

You need:

TEXT

Bits/Bytes

```
!' Powers of 2
!' Address bits
!' Memory capacity
!' Blocks
!' Locality
!' Cache mapping
```

For Pipelining

You need:

TEXT

Instruction execution

```
!' Clock cycle
!' CPU stages
!' CPI
!' Pipeline timing
!' Hazards
```

PART D — OFFICIAL SYLLABUS CHECKLIST

This remains our final verification checklist.

Instruction set and addressing modes

- ☐ Instruction architecture
- ☐ Instruction format
- ☐ Opcode/operand
- ☐ 0/1/2/3-address instructions
- ☐ Expression evaluation
- ☐ Addressing modes
- ☐ Effective-address numericals

ALU

- ☐ Binary arithmetic
- ☐ Adders
- ☐ Subtraction
- ☐ Overflow
- ☐ Logical operations
- ☐ Shift operations
- ☐ ALU organization
- ☐ Flags

Control Unit

- ☐ Control signals
- ☐ Micro-operations
- ☐ Hardwired control

- ☐ Microprogrammed control
- ☐ Horizontal/vertical microprogramming
- ☐ Comparison

Memory

- ☐ Memory hierarchy
- ☐ Locality
- ☐ Memory performance
- ☐ Memory interfacing
- ☐ Address calculations
- ☐ Cache basics
- ☐ Direct mapping
- ☐ Associative mapping
- ☐ Set-associative mapping
- ☐ Replacement policies
- ☐ Write policies
- ☐ AMAT/cache numericals

I/O

- ☐ I/O interface
- ☐ Polling/programmed I/O
- ☐ Interrupts
- ☐ DMA
- ☐ Comparison of I/O techniques

Pipelining

- ☐ Pipeline basics
- ☐ Pipeline execution time
- ☐ Speedup
- ☐ Throughput
- ☐ Efficiency
- ☐ Structural hazards
- ☐ Data hazards
- ☐ Control hazards
- ☐ Stalling
- ☐ Forwarding
- ☐ Branch penalty
- ☐ CPI-based numericals

Current:

0 / 6 official syllabus blocks completed.

That is normal because we are beginning with prerequisites.

CHAPTER 0 — COA FOUNDATIONS

Classification: [GATE SUPPORT]

Priority: +HIGH

The purpose of this chapter is **not** to reteach your entire Digital Logic syllabus.

We only learn what later COA chapters require.

0.1 What exactly is COA?

Computer Organization and Architecture studies **how a computer actually executes programs internally**.

Suppose you write:

```
c
c = a + b;
```

You see one simple statement.

Inside the machine, something roughly like this happens:

```
TEXT
Instruction fetched from memory
                        !"
CPU understands instruction
                        !"
Operands obtained
                        !"
ALU performs addition
                        !"
Result stored
```

COA explains this internal journey.

Architecture vs Organization

You may see these terms separately.

Computer Architecture

Describes features visible to the programmer.

Examples:

- instruction set
- registers
- addressing modes
- instruction formats

Think:

Architecture = WHAT the computer provides.

Computer Organization

Explains how those features are implemented.

Examples:

- control signals
- buses
- cache
- ALU circuits
- pipelining

Think:

Organization = HOW it is built.

Memory trick

A = Appearance

O = Operation inside

This distinction is useful conceptually, although GATE often mixes both under COA.

0.2 Bit

A **bit** is the smallest basic unit of binary information.

It can have only two values:

TEXT

0
1

Examples:

TEXT

1 b i t !' 0 o r 1
2 b i t s !' 0 0 , 0 1 , 1 0 , 1 1

With **n** bits, total possible patterns are:

$\boxed{2^n}$

Example:

With 3 bits:

$2^3 = 8$

patterns:

TEXT

000
001
010
011

```
100
101
110
111
```

Yes 2^n idea appears everywhere in COA.

0.3 Byte

Usually:

$\boxed{1 \text{ Byte} = 8 \text{ bits}}$

Therefore:

TEXT

```
1 byte = 8 bits
2 bytes = 16 bits
4 bytes = 32 bits
8 bytes = 64 bits
```

Important memory units

For GATE memory calculations, powers of 2 are extremely important.

$1 \text{ KB} = 2^{10} \text{ bytes}$

$1 \text{ MB} = 2^{20} \text{ bytes}$

$1 \text{ GB} = 2^{30} \text{ bytes}$

$1 \text{ TB} = 2^{40} \text{ bytes}$

Memorize

TEXT

```
K  !'  1 0
M  !'  2 0
G  !'  3 0
T  !'  4 0
```

Meaning:

TEXT

```
K B  !'  2 ^ 1 0
M B  !'  2 ^ 2 0
G B  !'  2 ^ 3 0
T B  !'  2 ^ 4 0
```

This shortcut will become extremely important for cache calculations.

0.4 Binary Number System

Humans normally use base 10.

Computers fundamentally work with binary, base 2.

Example:

$(1011)_2$

Convert to decimal:

$1(2^3) + 0(2^2) + 1(2^1) + 1(2^0)$

$= 8 + 0 + 2 + 1$

$\boxed{11}$

Binary place values

Memorize at least:

Power	Value
(2^0)	1
(2^1)	2
(2^2)	4
(2^3)	8
(2^4)	16
(2^5)	32
(2^6)	64
(2^7)	128
(2^8)	256
(2^9)	512
(2^{10})	1024

For COA, eventually become comfortable with:

TEXT

$2^{10} = 1K$

$2^{20} = 1M$

$2^{30} = 1G$

0.5 Hexadecimal

Hexadecimal uses base 16.

Digits:

TEXT

0 1 2 3 4 5 6 7 8 9 A B C D E F

where:

TEXT

A = 10
B = 11
C = 12
D = 13
E = 14
F = 15

Why is hexadecimal useful?

Because:

$\lfloor \boxed{1 \text{ hexadecimal digit} = 4 \text{ binary bits}} \rfloor$

Example:

TEXT

Binary: 1010
Hex: A

Example:

TEXT

1101 0110
D 6

Therefore:

$\lfloor (11010110)_2 = (D6)_{16} \rfloor$

Memory Trick

One hex digit = one nibble = 4 bits

TEXT

B i n a r y	!"	H e x
0000		0
0001		1
...		
1001		9
1010		A
1011		B
1100		C
1101		D
1110		E
1111		F

0.6 Unsigned Numbers

An unsigned number represents only:

$[0 \text{ and positive numbers}]$

With n bits:

$[\text{Minimum}=0]$

$[\text{Maximum}=2^n-1]$

Therefore:

$[\text{Unsigned range}=0 \text{ to } 2^n-1]$

Example: 8-bit unsigned number.

$[0 \rightarrow 2^8-1]$

$[0 \rightarrow 255]$

Shortcut

For n unsigned bits:

All bits 1 = maximum value

Example:

TEXT

1 1 1 1 1 1 1 1 ,

= 255.

0.7 Signed Numbers

Sometimes we need negative values.

Example:

TEXT

+7

-7

There are several ways of representing signed numbers.

Important ones:

1. Sign-magnitude
2. 1's complement
3. 2's complement

For COA, 2's complement is the most important.

0.8 Sign-Magnitude Representation

The leftmost bit represents the sign.

Usually:

TEXT

0 = positive
1 = negative

Example using 4 bits:

TEXT

+5 = 0101
-5 = 1101

The remaining bits contain magnitude.

Problem

There are two representations for zero:

TEXT

0000 = +0
1000 = -0

This is undesirable.

0.9 1's Complement

To find the 1's complement:

Flip every bit.

TEXT

0	!	1
1	!	0

Example:

TEXT

0101

1's complement:

TEXT

1010

1's complement negative number

To represent -5:

Start:

TEXT

+5 = 0101

Flip every bit:

TEXT

-5 = 1010

But 1's complement also has:

TEXT

+0 = 0000

-0 = 1111

Again, two zeros.

0.10 2's Complement

Ø=VERY IMPORTANT

2's complement is widely used for signed integers.

To find 2's complement:

Step 1

Take 1's complement.

Step 2

Add 1.

Example:

TEXT

0101

1's complement:

TEXT

1010

Add 1:

TEXT

1010

+ 1

1011

Therefore:

TEXT

2's complement of 0101 = 1011

Fast shortcut

To take the 2's complement:

Starting from the right, keep everything up to and including the first 1. Flip everything to its left.

Example:

TEXT

01101000

From right:

TEXT

...1000

First 1 encountered is retained; bits to its left are flipped.

But while learning, prefer:

invert + 1

It is harder to misuse.

0.11 Range of n-bit 2's Complement

Very important formula.

For n bits:

$$\boxed{-2^{n-1} \text{ to } 2^{n-1}-1}$$

Example: 8 bits.

Minimum:

$$\boxed{-2^7 = -128}$$

Maximum:

$$\boxed{2^7 - 1 = 127}$$

Therefore:

$$\boxed{-128 \text{ to } 127}$$

Compare

For 8 bits:

Unsigned

$[0 \text{ to } 255]$

Signed 2's complement

$[-128 \text{ to } 127]$

Memory Trick

Unsigned

$[0 \rightarrow 2^{n-1}]$

Signed 2's complement

$[-2^{n-1} \rightarrow 2^{n-1}-1]$

Think:

Signed !' sacrifice one exponent level for sign.

0.12 Recognizing Negative 2's Complement Numbers

In an n -bit 2's complement representation:

TEXT

MSB = 0 !' non - negative
MSB = 1 !' negative

MSB = Most Significant Bit, meaning the leftmost bit.

Example:

TEXT

01010101

positive.

TEXT

11010101

negative.

How do we find its decimal value?

Example:

TEXT

11111011

Since MSB = 1, number is negative.

Take 2's complement:

TEXT

```
11111011
!" invert
00000100
!" + 1
00000101
```

Magnitude = 5.

Therefore:

$\boxed{-5}$

Faster GATE Method

For an n -bit 2's complement number:

MSB has weight:

-2^{n-1}

All remaining bits have their normal positive weights.

Example:

TEXT

```
1101
```

Value:

$-2^3 + 2^2 + 0(2^1) + 2^0$

$= -8 + 4 + 1$

$\boxed{-3}$

Very fast for GATE.

0.13 Binary Addition

Rules:

TEXT

```
0 + 0 = 0
0 + 1 = 1
1 + 0 = 1
1 + 1 = 10
```

Meaning:

TEXT

```
1 + 1 !' Sum = 0, Carry = 1
```

And:

TEXT

1 + 1 + 1 = 11

Meaning:

TEXT

Sum = 1

Carry = 1

Example

TEXT

0101 (5)

+ 0011 (3)

1000 (8)

0.14 Binary Subtraction using 2's Complement

Instead of building a completely separate subtraction mechanism, hardware can transform:

$[A - B]$

into:

$[A + (\text{2's complement of } B)]$

Example:

$[7 - 3]$

Using 4 bits:

TEXT

7 = 0111

3 = 0011

2's complement of 3:

TEXT

0011

!" i n v e r t

1100

!" + 1

1101

Now:

TEXT

0111

+ 1101

1 0100

Discard the carry outside the fixed 4 bits:

TEXT

0100

$\boxed{4}$

This idea becomes important in ALU design.

0.15 Carry vs Overflow

CLASSIC GATE TRAP

Carry and overflow are **not the same thing**.

Carry

Primarily important for **unsigned arithmetic**.

Overflow

Important for **signed arithmetic**.

Overflow means:

The mathematical result cannot be represented using the available signed bits.

Signed Overflow Rule

For addition:

Positive + Positive = Negative

! overflow

Negative + Negative = Positive

! overflow

Positive + Negative

! cannot cause signed addition overflow.

Example

4-bit signed range:

-8 to 7

Calculate:

$5+4=9$

But 9 cannot be represented.

Binary:

```
TEXT
0101
0100
----
1001
```

1001 represents a negative number.

Positive + positive produced negative.

Therefore:

$\boxed{\text{Overflow}}$

GATE Shortcut

For signed addition:

```
TEXT
Same signs in
Different sign out
! "
OVERFLOW
```

Memory trick:

Same + Same !' Strange sign = Overflow

0.16 Boolean Operations

An ALU performs logical operations too.

You only need the essential operations here.

AND

Output is 1 only when both inputs are 1.

A	B	A AND B
0	0	0
0	1	0
1	0	0
1	1	1

Memory:

AND wants ALL ones.

OR

Output is 1 if at least one input is 1.

A	B	OR
0	0	0
0	1	1
1	0	1
1	1	1

XOR

Output is 1 when inputs differ.

A	B	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Memory:

XOR = eXclusive difference

NOT

Flips one bit.

TEXT

NOT 0 = 1

NOT 1 = 0

0.17 Register

A **register** is a very small, very fast storage location inside the CPU.

Example:

TEXT

CPU

% % % R e g i s t e r s
% % % A L U
% % % C o n t r o l U n i t

Registers temporarily hold:

- values,
- addresses,
- instructions,
- intermediate results.

Why not store everything in RAM?

Because CPU registers are much faster.

The general hierarchy is:

TEXT

Fastest
! "
Registers
! "
Cache
! "
Main Memory / RAM
! "
Secondary Storage
! "
Slowest

Typically:

TEXT

H i g h e r s p e e d ! ' s m a l l e r c a p a c i t y ! ' h i g h e r c o s t / b i t

We will study this deeply later.

0.18 Word Length

A **word** is a natural unit of data handled by a processor.

For example, a processor might have:

TEXT

32-bit word

or:

TEXT

64-bit word

A 32-bit word contains:

$$\lfloor 32/8=4 \rfloor \text{ bytes}$$

A 64-bit word contains:

$$\lfloor 64/8=8 \rfloor \text{ bytes}$$

Important

Do not automatically assume:

word = 4 bytes

Word size depends on the machine.

GATE may explicitly give:

- 16-bit word,
- 32-bit word,
- 64-bit word, etc.

0.19 Clock

A CPU needs timing to coordinate operations.

The clock produces regular pulses.

Think:

TEXT

t i c k ! ' t i c k ! ' t i c k ! ' t i c k ! ' . . .

A basic processor operation may require one or more clock cycles.

Clock Period / Clock Cycle Time

If frequency is:

$$\lfloor f \rfloor$$

then clock cycle time is:

$$\boxed{T = \frac{1}{f}}$$

Example

Clock frequency:

$$\lfloor 2 \rfloor \text{ GHz}$$

Since:

$$\lfloor 2 \rfloor \text{ GHz} = 2 \times 10^9 \text{ cycles/sec}$$

Clock period:

$$T = \frac{1}{2 \times 10^9} \text{ s}$$

$$= 0.5 \times 10^{-9} \text{ s}$$

$$\boxed{0.5 \text{ ns}}$$

Essential conversions

TEXT

```
1 ms = 10^-3 s
1 ¼ s = 10^-6 s
1 ns = 10^-9 s
1 ps = 10^-12 s
```

Memory trick:

TEXT

```
m !' - 3
¼ !' - 6
n !' - 9
p !' - 12
```

This becomes extremely important in pipeline and memory numericals.

0.20 Basic CPU

At a simplified level:

TEXT

```

% % % % % % % % % % %   C P U   % % % % % % % % % % %
%
%       R e g i s t e r s
%       !•
%       A L U
%       !•
%       C o n t r o l   U n i t
%
% % % % % % % % % % % , % % % % % % % % % % % % % % % %
%
Memory
%
I/O
```

Three extremely important CPU components:

1. ALU

Arithmetic and Logic Unit.

Does operations such as:

TEXT

```
+
-
```

AND
OR
XOR
comparison
shift

2. Registers

Store temporary values.

3. Control Unit

Coordinates CPU operations.

Think of it as the CPU's internal traffic controller.

0.21 Memory

Memory stores:

- instructions
- data

Simplified:

Address	Content
1 0 0 0	! ' d a t a / i n s t r u c t i o n
1 0 0 1	! ' d a t a / i n s t r u c t i o n
1 0 1 0	! ' d a t a / i n s t r u c t i o n
...	

An **address** identifies a memory location.

Address Bits

0-706 Fundamental cache/memory concept.

If an address contains n bits, we can form:

$\lfloor 2^n \rfloor$

different addresses.

Example:

16-bit address:

$\lfloor 2^{16} \rfloor = 65536$

possible addresses.

Byte-addressable memory

If each address refers to **1 byte**, then:

n address bits provide:

$\boxed{2^n \text{ bytes}}$

Example:

20-bit address bus:

2^{20} bytes

$\boxed{1 \text{ MB}}$

0.22 Byte Addressable vs Word Addressable

Extremely important for cache.

Byte Addressable

Each unique address identifies one byte.

Example:

TEXT

```
A d d r e s s   1 0 0   !'   1   b y t e
A d d r e s s   1 0 1   !'   n e x t   b y t e
```

Word Addressable

Each unique address identifies one complete word.

Suppose word size = 32 bits = 4 bytes.

Then:

TEXT

```
A d d r e s s   1 0 0   !'   o n e   3 2 - b i t   w o r d
A d d r e s s   1 0 1   !'   n e x t   3 2 - b i t   w o r d
```

GATE Trap

Suppose:

16-bit address bus, 32-bit word-addressable memory.

Number of addresses:

2^{16}

Each location contains:

$32 \text{ bits} = 4 \text{ bytes}$

Total capacity:

$2^{16} \times 4$

bytes.

$\lceil \frac{2^{18}}{1024} \rceil$ bytes

$\lceil \frac{256}{1024} \rceil$ KB

Do NOT simply answer 64 KB.

0.23 Instruction

An **instruction** is a binary command telling the CPU what operation to perform.

Conceptually:

TEXT

Instruction = Operation + Information about operands

Example:

TEXT

ADD R1, R2

means approximately:

Add values involving registers R1 and R2.

Opcode

The part specifying the operation is the:

Operation Code = Opcode

Examples:

TEXT

ADD

SUB

LOAD

STORE

JUMP

Operand

The data on which the operation works is the **operand**.

Example:

TEXT

ADD R1, R2

ADD !' opcode

R1, R2 !' operands

This becomes the foundation of our Instruction Set chapter.

0.24 Instruction Execution — Big Picture

Suppose an instruction is stored in memory.

CPU broadly performs:

```
TEXT
1. FETCH
   !"
   Get instruction from memory

2. DECODE
   !"
   Understand what it means

3. EXECUTE
   !"
   Perform required operation

4. ACCESS MEMORY
   !"
   if needed

5. WRITE RESULT
```

A commonly discussed pipeline later becomes:

```
TEXT
I F !'   I D !'   E X !'   M E M !'   W B
```

where:

```
TEXT
IF = Instruction Fetch
ID = Instruction Decode
EX = Execute
MEM = Memory Access
WB = Write Back
```

Do not memorize pipeline timing yet. We will build it properly later.

0.25 Important Powers of 2 for COA

You should gradually recognize these instantly:

Power	Value
2^4	16
2^5	32
2^6	64

Power	Value
$\backslash(2^7\backslash)$	128
$\backslash(2^8\backslash)$	256
$\backslash(2^9\backslash)$	512
$\backslash(2^{\{10\}}\backslash)$	1K
$\backslash(2^{\{11\}}\backslash)$	2K
$\backslash(2^{\{12\}}\backslash)$	4K
$\backslash(2^{\{13\}}\backslash)$	8K
$\backslash(2^{\{14\}}\backslash)$	16K
$\backslash(2^{\{15\}}\backslash)$	32K
$\backslash(2^{\{16\}}\backslash)$	64K
$\backslash(2^{\{20\}}\backslash)$	1M
$\backslash(2^{\{30\}}\backslash)$	1G
$\backslash(2^{\{40\}}\backslash)$	1T

Cache becomes dramatically easier if you know these.

GATE EXAMPLE 1 — Address Bits

A byte-addressable memory contains 4 GB. How many address bits are required?

Step 1

$$\backslash[4\text{GB}=4\times 2^{\{30\}}\backslash]$$

Since:

$$\backslash[4=2^2\backslash]$$

$\backslash[4\text{GB}=2^{\{32\}}\backslash\text{text}\{\text{ bytes}\}\backslash]$

Each address identifies one byte.

Therefore number of addresses:

$\backslash 2^{\{32\}} \backslash$

Hence:

$$\boxed{32 \text{ address bits}}$$

Fast GATE Method

TEXT

$$\begin{aligned} 4 \text{ GB} \\ &= 2^2 \times 2^3 \text{ p} \\ &= 2^{32} \text{ bytes} \end{aligned}$$

Therefore:

$$\boxed{32}$$

GATE EXAMPLE 2 — Signed Range

Find the range of a 12-bit 2's complement integer.

Formula:

$$[-2^{n-1} \text{ to } 2^{n-1}-1]$$

Putting $n=12$:

$$[-2^{11} \text{ to } 2^{11}-1]$$

$$\boxed{-2048 \text{ to } 2047}$$

GATE EXAMPLE 3 — Clock Period

Processor frequency:

$$4 \text{ GHz}$$

Find clock period.

$$T = \frac{1}{4 \times 10^9}$$

$$= 0.25 \text{ ns}$$

$$\boxed{0.25 \text{ ns}}$$

Shortcut:

TEXT

1	G H z	!	'	1	n s
2	G H z	!	'	0 . 5	n s
4	G H z	!	'	0 . 2 5	n s

GATE EXAMPLE 4 — Memory Capacity

A memory system has:

- 18 address lines
- each location stores 16 bits

Find total capacity.

Number of locations:

$$2^{18}$$

Each contains:

$16 \text{ bits} = 2 \text{ bytes}$

Therefore:

$2^{18} \times 2$

$= 2^{19} \text{ bytes}$

Since:

$2^{10} = 1 \text{ KB}$

$2^{19} = 512 \text{ KB}$

Answer:

$\boxed{512 \text{ KB}}$

COMMON GATE TRAPS FROM CHAPTER 0

Trap 1 — bits vs bytes

TEXT

8 bits = 1 byte

Do not mix them.

Trap 2 — signed vs unsigned range

Unsigned:

$0 \rightarrow 2^n - 1$

2's complement:

$-2^{n-1} \rightarrow 2^{n-1} - 1$

Trap 3 — Carry "Overflow"

Carry usually matters for unsigned arithmetic.

Overflow matters for signed representation.

Trap 4 — Byte-addressable vs word-addressable

Never calculate memory capacity before identifying what **one address represents**.

Trap 5 — GHz and ns

Remember reciprocal relationship:

$f \uparrow \rightarrow T \downarrow$

Higher frequency means smaller clock period.

CHAPTER 0 — MEMORY TRICKS

Units

K M G T !' 10 20 30 40

Signed range

Signed steals one exponent

$[-2^{n-1} \rightarrow 2^{n-1}-1]$

2's complement

Flip + 1

Overflow

Same signs in, strange sign out

Hexadecimal

1 Hex = 4 Bits

CPU

ACR

ALU Control Unit Registers

CHAPTER 0 SNAPSHOT

1. A bit is 0 or 1.
2. n bits produce 2^n patterns.
3. 1 byte = 8 bits.
4. $1\text{KB} = 2^{10}$ bytes.
5. One hexadecimal digit represents 4 bits.
6. Unsigned n -bit range is $[0]$ to $[2^n-1]$.
7. 2's complement range is $[-2^{n-1}]$ to $[2^{n-1}-1]$.
8. 2's complement = invert + 1.

9. Carry and signed overflow are different.
 10. Registers are tiny fast CPU storage.
 11. Word size depends on the architecture.
 12. $T = 1/f$.
 13. n address bits provide 2^n addresses.
 14. Byte-addressable means one address != one byte.
 15. Instruction = operation + operand information.
 16. CPU broadly performs fetch != decode != execute.
 17. ALU performs arithmetic/logical operations.
 18. Control unit coordinates processor activity.
-

MUST REMEMBER

2^n patterns from n bits

1 byte = 8 bits

Unsigned range = $0 \rightarrow 2^n - 1$

2's complement range = $-2^{n-1} \rightarrow 2^{n-1} - 1$

$T = \frac{1}{f}$

n address bits $\rightarrow 2^n$ addresses

PRACTICE SET

Try these before looking up anything.

Q1 — L0 MCQ

How many different patterns can be represented using 6 bits?

A. 6 B. 12 C. 32 D. 64

Q2 — L1 NAT

What is the maximum unsigned decimal value representable using 10 bits?

Q3 — L1 NAT

Find the minimum value representable using an 8-bit 2's complement number.

Q4 — L1 MCQ

Which is the 2's complement of 00110100 ?

A. 11001011 B. 11001100 C. 00110101 D. 10110011

Q5 — L1 NAT

A processor runs at 5 GHz.

Find its clock period in nanoseconds.

Q6 — L1 NAT

A byte-addressable system has a 24-bit address.

What is its maximum directly addressable memory capacity in MB?

Q7 — L2 NAT

A word-addressable memory has:

- 20 address lines
- word size = 32 bits

Find total memory capacity in MB.

Q8 — L2 Conceptual

For 4-bit signed 2's complement arithmetic:

TEXT

0110 + 0101

Does signed overflow occur?

CHAPTER 0 COMPLETION STATUS

Completed

' Bits and bytes ' Binary basics ' Hexadecimal basics ' Signed/unsigned representation ' 1's complement ' 2's complement ' Numerical ranges ' Binary addition/subtraction fundamentals ' Carry vs overflow ' Boolean operations ' Registers ' Clock cycle ' Word length ' CPU terminology ' Memory terminology ' Instruction basics

Chapter 0: COMPLETE '

COURSE PROGRESS

Modules completed: 1 / 26

Remaining: 25

Approximate course progress:

$\sim 4\%$

We have intentionally spent only one module on foundations. The main GATE COA material begins now.

NEXT

When you type **NEXT**, we begin:

CHAPTER 1 — BASIC COMPUTER ORGANIZATION

We will cover:

CPU !' ALU !' Control Unit !' Registers !' Memory !' I/O !' buses !' PC/IR/MAR/MDR !' fetch-decode-execute cycle !' register-transfer concepts

and then move into the first major GATE topic: **Instruction Set Architecture**.

USER

next

ASSISTANT

CHAPTER 1 — BASIC COMPUTER ORGANIZATION

Classification: [GATE SUPPORT] **Priority:** +HIGH **Goal:** Build the CPU/memory/register foundation needed for instruction sets, control unit, cache and pipelining.

1.1 Basic Functional Units of a Computer

A computer can be viewed as five major functional units:

TEXT

Input

!"

M e m o r y !" C P U !" O u t p u t

%

% % % % %4% % % %

% % % % %

ALU Control Unit

%

Registers

Main components:

1. Input Unit
2. Output Unit
3. Memory Unit
4. Arithmetic and Logic Unit (ALU)
5. Control Unit

The ALU + Control Unit + Registers together form the **CPU**.

$$\boxed{\text{CPU} = \text{ALU} + \text{Control Unit} + \text{Registers}}$$

1.2 CPU — Central Processing Unit

The CPU executes instructions.

Its job can be summarized as:

TEXT

```
Fetch instruction
      !"
Understand instruction
      !"
Get required data
      !"
Perform operation
      !"
Store result
```

The three core internal components are:

TEXT

```
CPU
% % %   A L U
% % %   C o n t r o l   U n i t
% % %   R e g i s t e r s
```

1.3 ALU

ALU = Arithmetic and Logic Unit

It performs arithmetic and logical operations.

Examples:

Arithmetic

TEXT

```
Addition
Subtraction
Increment
Decrement
```

Logical

TEXT

```
AND
OR
XOR
NOT
```

It may also perform:

TEXT

```
Comparison
Shift
Rotate
```

Example:

Suppose:

TEXT

R1 = 5
R2 = 7

Instruction:

TEXT

ADD R1, R2

ALU performs:

[5+7=12]

and produces the result.

We will study ALU design deeply in Chapter 4.

1.4 Control Unit

The **Control Unit (CU)** coordinates operations inside the computer.

Think of CPU components as workers.

The control unit tells them:

- what to do,
- when to do it,
- where data should move.

Example instruction:

TEXT

ADD R1, R2

The control unit may cause operations conceptually like:

TEXT

Read R1
Read R2
Send values to ALU
Tell ALU to ADD
Store result

It generates **control signals**.

Simple analogy

Imagine a kitchen.

TEXT

I n g r e d i e n t s	!	R e g i s t e r s
C o o k	!	A L U
H e a d C h e f	!	C o n t r o l U n i t
R e c i p e	!	I n s t r u c t i o n

The ALU does the operation.

The CU coordinates the operation.

1.5 Registers

Registers are very small and very fast storage locations inside the processor.

They hold temporary information.

Examples:

- instruction address
- current instruction
- operands
- results
- memory addresses

Why are registers needed?

Suppose the CPU had to access RAM for every tiny operation.

That would be much slower.

Registers keep frequently needed values very close to the CPU.

General hierarchy:

```
TEXT
Fastest
Registers
! "
Cache
! "
RAM
! "
SSD/HDD
Slowest
```

1.6 General Purpose vs Special Purpose Registers

General Purpose Registers — GPR

Used for normal computation.

Examples:

```
TEXT
R0
R1
R2
R3
```

Can store:

- integers
- addresses
- intermediate results

Special Purpose Registers

Have specific responsibilities.

Important ones for COA:

- PC
- IR
- MAR
- MDR/MBR
- SP
- Accumulator
- Status/Flag register

Let's understand each carefully.

1.7 Program Counter — PC

Very important

PC = Program Counter

It stores the address of the **next instruction to be fetched**.

Example:

TEXT

PC = 100

means:

Next instruction should be fetched from memory address 100.

After fetching, PC commonly moves to the next instruction.

Example:

TEXT

PC = 100

fetch instruction

PC = 104

if each instruction occupies 4 bytes.

Important GATE Trap

PC generally contains:

address of an instruction

not the instruction itself.

Memory trick:

PC Points to Code

1.8 Instruction Register — IR

IR = Instruction Register

It stores the **currently fetched instruction**.

Example:

Suppose memory address 100 contains:

```
TEXT
ADD R1, R2
```

Then:

```
TEXT
PC = 100
```

CPU fetches the instruction.

Then:

```
TEXT
IR = ADD R1, R2
```

So:

```
TEXT
PC !' address
IR !' instruction
```

Memory trick

P = Pointer

I = Instruction

1.9 Memory Address Register — MAR

MAR = Memory Address Register

Stores the address of the memory location currently being accessed.

Suppose CPU wants to read address:

TEXT

500

Then:

TEXT

MAR = 500

MAR answers:

WHERE in memory?

1.10 Memory Data Register — MDR

Also called:

MBR = Memory Buffer Register

MDR stores the actual data being transferred to/from memory.

MDR answers:

WHAT data?

Example:

Memory location:

TEXT

5 0 0 ! ' 7 5

For a read:

TEXT

MAR = 500

MDR = 75

Therefore:

TEXT

M A R ! ' a d d r e s s
M D R ! ' d a t a

Classic conceptual distinction.

MAR vs MDR

Register	Stores
MAR	Memory address

Register	Stores
MDR	Memory data
PC	Address of next instruction
IR	Current instruction

Memory trick:

A in MAR = Address

D in MDR = Data

1.11 Accumulator — AC

An **Accumulator** is a register traditionally used to store:

- one operand,
- intermediate arithmetic results,
- final ALU results.

Example:

```
TEXT
AC = 5
ADD X
```

Suppose:

```
TEXT
Memory[X] = 3
```

Then:

```
TEXT
AC ← AC + Memory[X]
```

Result:

```
TEXT
AC = 8
```

Accumulator-based machines are especially relevant when we study **1-address instructions**.

1.12 Stack Pointer — SP

SP = Stack Pointer

It stores the address of the top of a stack.

Stack operates on:

$\boxed{\text{LIFO}}$

Last In, First Out.

Example:

```
TEXT
Push A
Push B
Push C
```

Then first value popped is:

```
TEXT
C
```

SP helps locate the current top.

This becomes important in:

- zero-address instructions,
- function calls,
- stack architecture.

1.13 Status / Flag Register

Stores information about ALU results.

Typical flags:

```
TEXT
Z !' Zero flag
C !' Carry flag
S / N !' Sign / Negative flag
V / O !' Overflow flag
```

Example:

If:

$\boxed{5-5=0}$

then:

```
TEXT
Zero flag = 1
```

If signed overflow occurs:

```
TEXT
Overflow flag = 1
```

1.14 Main Memory

Main memory stores:

- program instructions
- data

Think:

TEXT

Address	Contents
1000	LOAD R1, X
1004	ADD R1, R2
1008	STORE R1, Y
...	

The processor accesses memory using addresses.

1.15 Stored Program Concept

Conceptual foundation

In a stored-program computer:

Instructions and data are stored in memory.

So memory can contain:

TEXT

Instructions
+
Data

CPU repeatedly retrieves instructions and executes them.

This leads directly to the fetch-decode-execute cycle.

1.16 Von Neumann Architecture

The classic Von Neumann model stores:

instructions and data in the same memory system.

Simplified:



Von Neumann vs Harvard — Support Concept

You may encounter this comparison.

Von Neumann

Same memory/address pathway conceptually for:

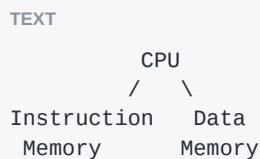
- instructions
- data

Harvard

Separate memories or pathways for:

- instructions
- data

Simplified:



For your GATE COA syllabus, understand the concept but don't spend excessive time here.

Classification: [GATE SUPPORT]

1.17 Von Neumann Bottleneck

Because instructions and data share memory/interface resources, memory transfer can become a performance limitation.

This is called the **Von Neumann bottleneck**.

At a simple level:

CPU may be fast, but it often has to wait for memory.

This problem motivates:

- cache memory,
- memory hierarchy,
- pipelining-related optimizations.

1.18 Input/Output — I/O

I/O devices allow the computer to interact with the external world.

Examples:

Input:

TEXT

Keyboard
Mouse
Sensor
Network input

Output:

TEXT

Monitor
Printer
Speaker
Network output

CPU normally does not directly understand every physical device.

Instead, it communicates through an:

$\boxed{\text{I/O Interface / Controller}}$

We will study this in detail later.

1.19 System Bus

A **bus** is a set of communication lines used to transfer information.

A typical system bus can be thought of as:

TEXT

Address Bus
Data Bus
Control Bus

1.20 Address Bus

Carries the address of the location/device to access.

Example:

CPU wants memory location:

TEXT

0x1000

That address is transmitted using the address bus.

Address Bus Width

If address bus width = n bits:

$\boxed{2^n \text{ addresses}}$

Example:

32-bit address bus:

2^{32}

possible addresses.

If byte-addressable:

$2^{32} \text{ bytes} = 4\text{GB}$

1.21 Data Bus

Carries actual data.

Example:

CPU reads:

TEXT

Memory[1000] = 25

Conceptually:

TEXT

A	d	d	r	e	s	s		b	u	s	!	'	1	0	0	0
D	a	t	a		b	u	s				!	•	2	5		

Data Bus Width

Suppose data bus width = 32 bits.

Then up to 32 bits can be transferred in one bus transfer, depending on the system design.

Do NOT confuse:

TEXT

A	d	d	r	e	s	s		b	u	s	!	'	w	h	e	r	e
D	a	t	a		b	u	s	!	'	w	h	a	t				

1.22 Control Bus

Carries control signals.

Examples:

TEXT

Memory Read
Memory Write

Interrupt Request
Interrupt Acknowledge
Clock-related signals

Memory trick:

TEXT

A d d r e s s ! ' W H E R E
D a t a ! ' W H A T
C o n t r o l ! ' H O W / W H E N

1.23 CPU-Memory Read Operation

Suppose CPU wants:

TEXT

Memory[500]

A simplified sequence:

TEXT

- 1 . M A R ! • 5 0 0
2. CPU sends READ signal
3. Memory reads location 500
4. Data returns into MDR

Symbolically:

$\text{[MAR] } \leftarrow 500$

$\text{[MDR] } \leftarrow \text{M[MAR]}$

where:

[M[x]]

means contents of memory location x .

1.24 CPU-Memory Write Operation

Suppose CPU wants to store:

TEXT

75

at address:

TEXT

500

Then:

TEXT

MAR = 500
MDR = 75
Write signal issued

Conceptually:

$M[MAR] \leftarrow MDR$

GATE Memory Trick

For memory transfer:

TEXT

M A R ! ' W H E R E
M D R ! ' W H A T
R E A D / W R I T E ! ' A C T I O N

1.25 Register Transfer Language — RTL

Register Transfer Language is a notation used to describe operations between registers.

Example:

$R1 \leftarrow R2$

means:

Copy the contents of R2 into R1.

Important:

It does **not** normally mean R2 is destroyed.

After operation:

TEXT

R1 = old R2
R2 = unchanged

Example

Suppose:

TEXT

R1 = 5
R2 = 12

Operation:

$R1 \leftarrow R2$

Result:

TEXT

R1 = 12
R2 = 12

Arithmetic RTL

Example:

$R1 \leftarrow R2 + R3$

Meaning:

TEXT

ALU adds R2 and R3
Result stored in R1

Memory RTL

Read:

$MDR \leftarrow M[MAR]$

Write:

$M[MAR] \leftarrow MDR$

1.26 Micro-operation

A **micro-operation** is a basic operation performed on data stored in registers.

Examples:

TEXT

R1 ← R2
R1 ← R1 + R2
R1 ← R1 AND R2
R1 ← shift-left R1

Typical categories:

1. Register transfer
2. Arithmetic
3. Logical
4. Shift

These become very important in the **Control Unit** chapter.

1.27 Instruction Cycle

The CPU repeatedly executes an **instruction cycle**.

Basic idea:

```
TEXT
FETCH
  ! "
DECODE
  ! "
EXECUTE
  ! "
NEXT INSTRUCTION
```

Let's understand each stage.

1.28 Fetch

CPU obtains the instruction from memory.

Simplified sequence:

```
TEXT
P C ! ' M A R
Memory Read
M D R ! ' I R
PC updated
```

RTL form:

$\text{PC} \leftarrow \text{MAR}$

$\text{MDR} \leftarrow \text{M}[\text{MAR}]$

$\text{IR} \leftarrow \text{MDR}$

$\text{PC} \leftarrow \text{PC} + \text{instruction length}$

This is a simplified model; exact micro-operations depend on architecture.

Memory Trick

Fetch:

PC tells WHERE, memory gives instruction, IR holds it.

1.29 Decode

The control unit examines the instruction in IR.

Suppose:

```
TEXT
IR = ADD R1, R2
```

CPU identifies:

TEXT

Opcode = ADD
Operands = R1, R2

It then knows which operation should occur.

1.30 Execute

The CPU performs the requested operation.

Example:

TEXT

ADD R1, R2

could conceptually mean:

$[R1] \leftarrow R1 + R2$

ALU performs the addition.

1.31 Store / Write Back

If an instruction produces a result, the result may be written to:

- a register,
- memory.

Example:

TEXT

R1 ← result

This is sometimes called **write back**.

Complete Big Picture

TEXT

PC

!"

Instruction address

!"

Memory

!"

Instruction

!"

IR

!"

Decode

!"

Operands

!"

ALU / Memory

!"

Result
!"
Register / Memory

1.32 Example Instruction Execution

Suppose:

TEXT
R1 = 5
R2 = 7

Instruction:

TEXT
ADD R1, R2

Assume it means:

$[R1] \leftarrow R1 + R2$

Execution:

Fetch

Instruction is fetched from memory.

Decode

CPU recognizes:

TEXT
Opcode = ADD

Operand read

TEXT
R1 = 5
R2 = 7

Execute

$[5 + 7 = 12]$

Write back

TEXT
R1 = 12

Final:

TEXT

R1 = 12

R2 = 7

1.33 Clock Cycle vs Instruction Cycle

Do not confuse these.

Clock Cycle

One basic period of the processor clock.

Instruction Cycle

Complete sequence needed to execute one instruction.

An instruction may require:

[1, 2, 5, 10, ...]

clock cycles depending on the architecture.

Therefore:

One instruction cycle is not automatically one clock cycle.

Important for later CPI concepts.

1.34 Machine Cycle — Terminology Caution

Some books use terms such as:

- machine cycle
- processor cycle
- instruction cycle

with slightly different definitions depending on architecture.

For GATE numericals, always follow the definition supplied in the question.

Do not blindly assume all these terms mean the same thing.

1.35 CPU Performance Preview

Later, an important equation will be:

$$\boxed{\text{CPU Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time}}$$

where:

CPI = Cycles Per Instruction

Equivalent:

$$\boxed{\text{CPU Time} = \frac{\text{Instruction Count} \times \text{CPI}}{\text{Clock Rate}}}$$

Do not worry about solving advanced questions yet.

For now remember:

TEXT

Program

```
!' many instructions
!' each instruction requires cycles
!' each cycle takes time
```

1.36 Common Special Registers Summary

Register	Meaning	Stores
PC	Program Counter	Address of next instruction
IR	Instruction Register	Current instruction
MAR	Memory Address Register	Address being accessed
MDR	Memory Data Register	Data moving to/from memory
AC	Accumulator	Operand/intermediate result
SP	Stack Pointer	Address of stack top
Flag Register	Status	Carry, zero, sign, overflow etc.

Learn this table well.

1.37 GATE Example 1 — MAR/MDR

Suppose CPU wants to read memory location 1200 containing value 45.

Which registers hold:

- 1200
- 45

Answer:

TEXT

MAR = 1200
MDR = 45

Because:

TEXT

M A R !' A d d r e s s
M D R !' D a t a

1.38 GATE Example 2 — Address Bus

A byte-addressable computer has 28 address lines.

Maximum addressable memory:

2^{28} bytes

Since:

$2^{28} = 256 \times 2^{20}$

Therefore:

$\boxed{256\text{MB}}$

1.39 GATE Example 3 — Data Bus

A computer has:

- 32-bit data bus
- 24-bit address bus
- byte-addressable memory

The address bus determines maximum directly addressable memory:

$2^{24} \text{ bytes} = 16\text{MB}$

The 32-bit data bus does **not** make it 2^{32} bytes.

☹ Classic trap.

Address Bus vs Data Bus

Address bus decides number of addressable locations.

Data bus primarily decides amount transferred per bus operation.

1.40 GATE Example 4 — Fetch Cycle

Consider:

TEXT

PC = 2000

Memory[2000] = ADD R1, R2

During fetch:

TEXT

M A R ! • P C

SO:

TEXT

MAR = 2000

Memory provides instruction:

TEXT

MDR = ADD R1, R2

Then:

TEXT

IR ← MDR

SO:

TEXT

IR = ADD R1, R2

PC moves to next instruction.

1.41 Control Signal Example

For a memory read:

TEXT

```
CPU
|
| address
! "
Memory
|
| data
! "
CPU
```

But memory also needs to know whether the CPU wants to:

TEXT

READ

OR:

TEXT

WRITE

That information travels through control signals.

1.42 Datapath — Introduction

A **datapath** is the collection of hardware through which data moves and is processed.

Typically includes:

- registers,
- ALU,
- multiplexers,
- buses,
- internal connections.

Simplified:

TEXT

```
Registers
  !"
  ALU
  !"
Registers
```

The **control unit** generates signals controlling the datapath.

So:

TEXT

```
C o n t r o l   U n i t   ! '   t e l l s
D a t a p a t h           ! '   d o e s
```

This distinction becomes important in ALU/control-unit design.

1.43 Control Unit + Datapath

A processor can conceptually be separated into:

TEXT

```
Processor
% % %   D a t a p a t h
%       % % %   R e g i s t e r s
%       % % %   A L U
%       % % %   D a t a   c o n n e c t i o n s
%
% % %   C o n t r o l   U n i t
        % % %   C o n t r o l   s i g n a l s
```

Memory trick:

Datapath carries data.

Control decides direction/actions.

1.44 Bus Organization Inside CPU

Registers need a method to exchange data.

One possibility is a shared internal bus.

Example:

TEXT

```
R 1  % %  
R 2  % %<% % !'   B U S  !'   A L U  
R 3  % %
```

Control signals select:

- which register places data on the bus,
- which register accepts data,
- what ALU operation occurs.

The detailed circuit design is beyond what we need here unless directly useful later.

1.45 Single-Bus vs Multiple-Bus Idea

[LOW PRIORITY / SUPPORT]

A processor may use:

- one internal bus,
- multiple internal buses.

More buses can allow more transfers simultaneously but require more hardware.

You only need the intuition for later datapath discussions.

1.46 CPU-Memory Speed Gap

CPU operations can be much faster than main-memory access.

This produces a performance problem:

TEXT

```
CPU: Ready!  
Memory: Still fetching...
```

Solutions include:

- registers,
- cache,
- memory hierarchy,
- prefetching and architectural techniques.

For GATE, cache is especially important.

1.47 Basic Memory Hierarchy Preview

TEXT

```
      Faster  
      !'  
Registers
```

Cache
RAM
SSD / HDD
! "
Larger

General trends moving upward:

- speed increases,
- cost per bit increases,
- capacity decreases.

Moving downward:

- speed decreases,
- capacity increases,
- cost per bit decreases.

We study this rigorously later.

1.48 Instruction vs Data

An instruction tells the computer what to do.

Example:

```
TEXT
ADD R1, R2
```

Data is the value being processed.

Example:

```
TEXT
5
7
100
```

But ultimately both are represented as bit patterns.

The interpretation depends on how the processor uses those bits.

1.49 Program Flow

Normally instructions execute sequentially:

```
TEXT
Instruction 1
! "  
Instruction 2
! "  
Instruction 3
! "  
Instruction 4
```

PC advances accordingly.

But instructions such as:

```
TEXT
JUMP
BRANCH
CALL
RETURN
```

can change PC.

Example:

```
TEXT
JUMP 500
```

may cause:

```
TEXT
P C !• 5 0 0
```

This becomes important in instruction sets and pipeline control hazards.

1.50 Why PC Matters for Branches

Normally:

$PC \rightarrow PC + \text{next instruction displacement}$

But a branch may instead perform:

$PC \rightarrow \text{target address}$

This simple fact later creates a major pipeline issue:

CPU may already have fetched the wrong next instruction.

That is called a **control hazard**.

We will reach that later.

CHAPTER 1 — GATE CORE CONNECTIONS

Why did we learn all of this?

Because later:

Addressing Modes

need:

- registers,
- PC,
- memory addresses.

Control Unit

needs:

- RTL,
- micro-operations,
- control signals.

Cache

needs:

- address bus,
- memory structure.

DMA

needs:

- CPU-memory-I/O interactions.

Pipeline

needs:

- instruction cycle,
- fetch,
- decode,
- execute,
- memory,
- write back.

So this chapter is foundational rather than filler.

COMMON GATE TRAPS

Trap 1

PC contains next instruction itself.

'LWrong.

PC usually stores the **address** of the next instruction.

Trap 2

IR stores instruction address.

'LWrong.

IR stores the **instruction**.

Trap 3

MAR stores data.

'LWrong.

MAR stores address.

Trap 4

MDR stores only data permanently.

'L Wrong.

MDR is temporary transfer storage for values/instruction bits travelling between CPU and memory.

Trap 5

32-bit data bus means 4 GB address space.

'L Not necessarily.

Address-space size depends primarily on the address structure/address bus and addressability.

Trap 6

One instruction always takes one clock cycle.

'L Wrong.

Architecture determines cycles per instruction.

Trap 7

Registers and cache are the same.

'L Wrong.

Both are fast storage, but registers are directly part of CPU execution state, while cache is a separate level of memory hierarchy.

MEMORY TRICKS

Special registers

PC

Points to Code

IR

Instruction Right now

MAR

Memory Address

MDR

Memory Data

SP

Stack Position

Bus trick

A-D-C

TEXT

A d d r e s s ! ' D e s t i n a t i o n / w h e r e
D a t a ! ' C o n t e n t / w h a t
C o n t r o l ! ' C o m m a n d / a c t i o n

Or simply:

WHERE — WHAT — HOW

Fetch Trick

TEXT

P C ! ' M A R ! ' M e m o r y ! ' M D R ! ' I R

Memory sentence:

PC points, MAR asks, Memory answers, MDR carries, IR keeps.

CHAPTER 1 SNAPSHOT

1. CPU consists mainly of ALU, control unit and registers.
2. ALU performs arithmetic and logical operations.
3. Control unit generates control signals.
4. Registers are fast internal CPU storage.
5. PC stores the address of the next instruction.
6. IR stores the currently executing/fetched instruction.
7. MAR stores memory address.

8. MDR stores data travelling to/from memory.
 9. SP identifies stack top.
 10. Stored-program concept keeps instructions and data in memory.
 11. Von Neumann architecture shares a memory system for instructions/data.
 12. Address bus tells where.
 13. Data bus carries what.
 14. Control bus communicates actions such as read/write.
 15. RTL represents register-level transfers.
 16. Fetch-decode-execute is the basic instruction cycle.
 17. Datapath processes/moves data.
 18. Control unit controls the datapath.
 19. One instruction can require multiple clock cycles.
 20. Branches alter PC and later create pipeline hazards.
-

MUST REMEMBER

$\boxed{\text{CPU} = \text{ALU} + \text{Control Unit} + \text{Registers}}$

$\boxed{\text{PC} \rightarrow \text{next instruction address}}$

$\boxed{\text{IR} \rightarrow \text{current instruction}}$

$\boxed{\text{MAR} \rightarrow \text{memory address}}$

$\boxed{\text{MDR} \rightarrow \text{memory data}}$

Fetch idea:

$\boxed{\text{PC} \rightarrow \text{MAR} \rightarrow \text{Memory} \rightarrow \text{MDR} \rightarrow \text{IR}}$

PRACTICE SET

Try these first.

Q1 — L0 MCQ

Which register normally stores the address of the next instruction?

A. IR B. PC C. MDR D. MAR

Q2 — L0 MCQ

Which register stores the memory address currently being accessed?

A. MDR B. IR C. MAR D. AC

Q3 — L1 MSQ

Which components are usually part of the CPU?

A. ALU B. Registers C. Control Unit D. Hard disk

Q4 — L1 MCQ

What does:

$[R1] \leftarrow R2$

mean?

A. Delete R2 B. Copy R1 into R2 C. Copy R2 into R1 D. Add R1 and R2

Q5 — L1 NAT

A byte-addressable system has 30 address lines.

Maximum addressable memory in GB = ?

Q6 — L1 MCQ

Which bus primarily carries information indicating a memory READ operation?

A. Address bus B. Data bus C. Control bus D. Register bus

Q7 — L2

Suppose:

TEXT

PC = 400

Memory[400] = SUB R1, R2

Immediately after the instruction is fetched into IR, what does IR contain?

Q8 — L2 MSQ

Which statements are correct?

A. PC normally contains an instruction address. B. IR contains the currently fetched instruction. C. MAR contains the actual memory data value. D. MDR may hold data being transferred from memory.

CHAPTER 1 COMPLETION

' Basic functional units ' CPU ' ALU overview ' Control Unit overview ' Registers ' PC ' IR ' MAR ' MDR ' Accumulator ' Stack Pointer ' Flags ' Stored-program concept ' Von Neumann architecture ' I/O overview ' Address/Data/Control buses ' Memory read/write ' RTL ' Micro-operations ' Fetch-decode-execute ' Clock vs instruction cycle ' Datapath/control relationship

CHAPTER 1: COMPLETE '

COURSE PROGRESS

Completed: 2 / 26 modules

Approximate completion:

$\boxed{\sim 8\%}$

The foundation is now ready.

NEXT !' CHAPTER 2 — INSTRUCTION SET ARCHITECTURE

This is our first major [GATE CORE] chapter.

We will cover:

- instruction format,
- opcode and operands,
- instruction length,
- 0-address, 1-address, 2-address and 3-address machines,
- stack/accumulator/register architectures,
- expression evaluation,
- instruction count,
- memory accesses,
- instruction-size numericals,
- opcode-bit calculations,
- GATE traps and shortcuts.

USER

next

ASSISTANT

CHAPTER 2 — INSTRUCTION SET ARCHITECTURE (ISA)

Classification: [GATE CORE] **Priority:** ~~0~~=VERY HIGH **Goal:** Understand how machine instructions are represented, how operands are specified, and how 0/1/2/3-address instruction formats affect code and memory access.

2.1 What is an Instruction Set?

An **Instruction Set** is the collection of machine-level operations a processor can execute.

Examples:

TEXT

ADD
SUB
MUL
LOAD
STORE

```
JUMP
AND
OR
SHIFT
```

A processor cannot directly understand a C statement such as:

```
C
x = a + b;
```

It ultimately executes machine instructions such as:

```
TEXT
LOAD R1, A
LOAD R2, B
ADD R3, R1, R2
STORE X, R3
```

So:

Instruction Set = vocabulary understood by the CPU.

2.2 What is ISA?

ISA = Instruction Set Architecture

ISA defines the programmer-visible behavior of a processor.

It includes things such as:

- supported instructions,
- registers,
- instruction formats,
- operand types,
- addressing modes,
- memory model.

Think of ISA as the **contract between software and hardware**.

```
TEXT
Software
    !"
ISA
    !"
Hardware
```

Programs target the ISA.

Hardware implements the ISA.

2.3 Instruction Structure

A machine instruction generally contains:

TEXT

```
% % % % % % % % % % , % % % % % % % % % % % % % % % % %  
%   O p c o d e       %   O p e r a n d   i n f o   %  
% % % % % % % % % % %4% % % % % % % % % % % % % % % % %
```

Opcode

Specifies **what operation** must be performed.

Examples:

TEXT

```
ADD  
SUB  
LOAD  
STORE  
JUMP
```

Operand

Specifies **on what data** the operation works.

Example:

TEXT

```
ADD R1, R2
```

Here:

- `ADD` = opcode
- `R1` , `R2` = operands

2.4 Instruction Fields

An instruction may contain fields such as:

TEXT

```
% % % % % % % % % % , % % % % % % % % % % , % % % % % % % % % % , % % % % % % % % % %  
%   O p c o d e   %   D e s t   R e g   %   S r c   R e g 1   %   S r c   R e g 2   %  
% % % % % % % % % % %4% % % % % % % % % % %4% % % % % % % % % % %4% % % % % % % % % %
```

Or:

TEXT

```
% % % % % % % % % % , % % % % % % % % % % % % % % % % %  
%   O p c o d e   %   A d d r e s s   %  
% % % % % % % % % % %4% % % % % % % % % % % % % % % % %
```

The exact format depends on the architecture.

2.5 Instruction Length

Instruction length means the total number of bits used to encode one instruction.

Example:

TEXT

Instruction length = 32 bits

It might be divided as:

TEXT

Opcode = 6 bits

Register 1 = 5 bits

Register 2 = 5 bits

Immediate = 16 bits

Total = 32 bits

GATE often asks you to determine:

- opcode bits,
- register field bits,
- address field bits,
- number of possible instructions.

2.6 Number of Opcodes

If opcode field has k bits:

$$\boxed{\text{Maximum opcodes} = 2^k}$$

Example:

Opcode field = 6 bits.

$$2^6 = 64$$

Maximum:

$$\boxed{64 \text{ different opcodes}}$$

2.7 Bits Required for Opcodes

Suppose processor supports 100 instructions.

Need smallest k such that:

$$2^k \geq 100$$

We know:

$$2^6 = 64$$

$$2^7 = 128$$

Therefore:

$$\boxed{7 \text{ opcode bits}}$$

General rule:

$$\lceil \log_2 N \rceil$$

where N = number of possibilities.

2.8 Register Field Bits

Suppose CPU contains 32 registers.

How many bits are required to identify one register?

$$2^5 = 32$$

Therefore:

$$5 \text{ bits}$$

General rule:

For R registers:

$$\lceil \log_2 R \rceil$$

bits are required.

GATE Shortcut

Immediately recognize:

TEXT

8	registers	!	3	bits
16	registers	!	4	bits
32	registers	!	5	bits
64	registers	!	6	bits
128	registers	!	7	bits

2.9 Types of Instructions

A processor normally contains several categories.

1. Data Transfer

Move data.

Examples:

TEXT

LOAD
STORE
MOVE
PUSH
POP

2. Arithmetic

Examples:

```
TEXT
ADD
SUB
MUL
DIV
INC
DEC
```

3. Logical

Examples:

```
TEXT
AND
OR
XOR
NOT
```

4. Shift / Rotate

Examples:

```
TEXT
SHL
SHR
ROL
ROR
```

5. Control Transfer

Change program execution order.

Examples:

```
TEXT
JUMP
BRANCH
CALL
RETURN
```

These become important later for pipeline hazards.

2.10 Number of Address Instructions

~~0-4~~ This is one of the most important ISA concepts.

Architectures can be classified by the number of explicit operand addresses in an instruction:

- 3-address
- 2-address
- 1-address
- 0-address

We will study each.

2.11 Three-Address Instructions

A 3-address instruction explicitly specifies:

TEXT

Destination, Source1, Source2

Example:

TEXT

ADD R1, R2, R3

Meaning:

$[R1] \leftarrow R2 + R3$

So:

TEXT

R1 = destination

R2 = source 1

R3 = source 2

Advantage

One instruction can perform an operation without destroying the source operands.

Example

Expression:

$[X] = A + B$

Possible sequence:

TEXT

ADD X, A, B

Conceptually one instruction.

2.12 Two-Address Instructions

A 2-address instruction generally uses one operand as both:

- source,
- destination.

Example:

```
TEXT
ADD R1, R2
```

often means:

$[R1] \leftarrow R1 + R2$

So original **R1** gets overwritten.

Example

Suppose:

```
TEXT
R1 = 5
R2 = 7
```

After:

```
TEXT
ADD R1, R2
```

we get:

```
TEXT
R1 = 12
R2 = 7
```

2.13 One-Address Instructions

A 1-address architecture usually assumes an implicit **Accumulator (AC)**.

Example:

```
TEXT
ADD X
```

means:

$AC \leftarrow AC + M[X]$

The accumulator is not written explicitly because it is implied.

Typical instructions

TEXT

```
LOAD A
ADD B
STORE X
```

For:

$\boxed{X=A+B}$

Sequence:

TEXT

```
LOAD A
ADD B
STORE X
```

Explanation:

TEXT

```
A C !• A
A C !• A C + B
X !• A C
```

2.14 Zero-Address Instructions

A 0-address architecture normally uses a **stack**.

Operands are implicitly taken from the top of stack.

Example:

TEXT

```
ADD
```

does not explicitly mention operands.

It means roughly:

TEXT

```
Pop operand 1
Pop operand 2
Add them
Push result
```

Therefore it needs:

$\boxed{\text{explicit address fields}}$

Stack Example

Suppose stack contains:

TEXT

Top
!
5
3

Execute:

TEXT

ADD

CPU pops:

TEXT

5
3

computes:

\[5+3=8 \]

pushes:

TEXT

8

2.15 Architecture Summary

Address Count	Main Organization	Implicit Operand
3-address	General registers	Usually none
2-address	General registers	One operand doubles as destination
1-address	Accumulator	AC
0-address	Stack	Stack top

Memory trick:

TEXT

3 !' R e g i s t e r s
2 !' R e g i s t e r s
1 !' A c c u m u l a t o r
0 !' S t a c k

2.16 Expression Evaluation — Core GATE Topic

Consider:

\[X=(A+B)\times(C-D) \]

We will evaluate it using different architectures.

This is a very common conceptual pattern.

2.17 Three-Address Machine

We need temporary registers.

TEXT

```
ADD R1, A, B
SUB R2, C, D
MUL X, R1, R2
```

Meaning:

$[R1 = A + B]$

$[R2 = C - D]$

$[X = R1 \times R2]$

Number of arithmetic instructions:

$\boxed{3}$

2.18 Two-Address Machine

Because destination is also a source:

TEXT

```
MOV R1, A
ADD R1, B

MOV R2, C
SUB R2, D

MUL R1, R2
MOV X, R1
```

Now:

TEXT

```
R1 = A + B
R2 = C - D
R1 = R1 × R2
X = R1
```

More data-movement instructions may be required.

2.19 One-Address Machine

Uses accumulator.

Expression:

$[X = (A + B) \times (C - D)]$

We need a temporary location because accumulator can hold only one working result at a time.

Possible sequence:

```
TEXT
LOAD A
ADD B
STORE T

LOAD C
SUB D
MUL T
STORE X
```

Step by step:

```
TEXT
AC = A
AC = A + B
T = A + B

AC = C
AC = C - D
AC = (C-D)(A+B)

X = AC
```

2.20 Zero-Address / Stack Machine

Stack machines normally use postfix expression form.

Expression:

[(A+B)(C-D)]

Postfix:

```
TEXT
A B + C D - *
```

Instructions:

```
TEXT
PUSH A
PUSH B
ADD

PUSH C
PUSH D
SUB

MUL
POP X
```

Why postfix?

Because stack operation naturally works with postfix expressions.

For:

```
TEXT
A B +
```

we push A, push B, then ADD.

No parentheses are needed.

2.21 Infix vs Postfix

Infix

Operator appears between operands.

```
TEXT
A + B
```

Postfix

Operator appears after operands.

```
TEXT
A B +
```

Stack architectures use postfix very naturally.

2.22 Another Example

Evaluate:

$X = (A - B) + (C \times D)$

Postfix:

```
TEXT
A B - C D * +
```

Stack sequence:

```
TEXT
PUSH A
PUSH B
SUB
PUSH C
PUSH D
MUL
ADD
POP X
```

2.23 Important Operand Order in Stack Machines

0 = 0%TE trap.

Suppose postfix:

```
TEXT
A B -
```

Stack sequence:

```
TEXT
PUSH A
PUSH B
SUB
```

The operation should produce:

$\boxed{A - B}$

When popping:

```
TEXT
first pop = B
second pop = A
```

For subtraction:

$\boxed{\text{second pop} - \text{first pop}}$

Similarly for division:

$\boxed{\text{second pop} / \text{first pop}}$

Do not reverse the operands.

2.24 Memory Accesses — Important Idea

GATE may ask:

How many memory accesses are required to execute an instruction?

You must distinguish:

1. Instruction fetch
2. Operand fetch
3. Result write

For example, suppose:

```
TEXT
ADD X
```

on accumulator machine means:

$\lceil AC \leftarrow AC + M[X] \rceil$

Possible memory accesses:

1. Fetch instruction
2. Read operand $M[X]$

So total:

$\lceil \boxed{2} \rceil$

if instruction itself is stored in memory and no additional indirection is needed.

2.25 Register Operands vs Memory Operands

Compare:

TEXT

ADD R1, R2

If both operands are already in registers:

- instruction fetch requires memory,
- operand data may not require main-memory access.

Compare:

TEXT

ADD R1, X

where X is memory.

Now the CPU must also access memory for operand X.

Therefore:

Do not assume every operand causes a memory access.

Registers are inside the CPU.

2.26 Instruction Size and Address Fields

Suppose:

- instruction length = 32 bits,
- opcode = 8 bits,
- instruction has two equal-sized address fields.

Remaining bits:

$\lceil 32 - 8 = 24 \rceil$

Each address field:

$\lceil 24 / 2 = 12 \rceil$

Therefore:

$$\boxed{12 \text{ bits per address}}$$

Each field can represent:

$$2^{12} = 4096$$

different values.

2.27 GATE Example — Register Fields

A processor has:

- 64 registers
- three-register instruction format
- opcode field = 8 bits

How many bits minimum for the instruction?

Each register:

$$\log_2 64 = 6$$

Three register fields:

$$3 \times 6 = 18$$

Opcode:

$$8$$

Total:

$$18 + 8 = 26$$

So minimum:

$$\boxed{26 \text{ bits}}$$

2.28 GATE Example — Number of Opcodes

Instruction length = 24 bits.

There is:

- one 16-bit address field.

Remaining bits:

$$24 - 16 = 8$$

Opcode bits:

$$8$$

Maximum possible instructions:

$$2^8$$

$$\boxed{256}$$

2.29 GATE Example — Register Selection

A processor has 40 registers.

How many bits are required to select a register?

Need:

$$\lceil 2^5 = 32 < 40 \rceil$$

$$\lceil 2^6 = 64 \geq 40 \rceil$$

Therefore:

$$\boxed{6 \text{ bits}}$$

Not 5.

2.30 Instruction Format Trade-off

Suppose instruction length is fixed.

If you allocate more bits to opcode:

TEXT

More opcode bits

! ' more possible operations

! ' fewer bits left for operands / address

If you allocate more bits to addresses:

TEXT

Larger address range

! ' fewer bits left for opcode

This is an important ISA design trade-off.

2.31 Fixed-Length Instructions

Every instruction has the same size.

Example:

TEXT

Every instruction = 32 bits

Advantages:

- easier decoding,
- simpler pipelining,
- predictable instruction boundaries.

Possible disadvantage:

- some instructions may waste bits.

2.32 Variable-Length Instructions

Different instructions may have different lengths.

Example:

TEXT

```
Instruction A = 1 byte
Instruction B = 3 bytes
Instruction C = 5 bytes
```

Advantages:

- potentially better code density.

Disadvantages:

- decoding becomes more complex.

For your GATE syllabus, understand the idea; do not overfocus.

Classification: [GATE SUPPORT]

2.33 Load/Store Architecture

In a **load/store architecture**, arithmetic operations generally work on registers.

Memory is accessed mainly through explicit:

TEXT

```
LOAD
STORE
```

Example:

TEXT

```
LOAD R1, A
LOAD R2, B
ADD R3, R1, R2
STORE X, R3
```

Here **ADD** works on registers.

2.34 Register-Memory Architecture

Some architectures allow arithmetic with a memory operand.

Example:

TEXT

```
ADD R1, X
```

which might mean:

$[R1] \leftarrow R1 + M[X]$

This can reduce instruction count but makes instruction execution more complex.

2.35 Memory-Memory Architecture

In some architectures, operations may directly access memory operands.

Example conceptually:

```
TEXT
ADD X, A, B
```

meaning:

$M[X] \leftarrow M[A] + M[B]$

This can require several memory accesses.

2.36 RISC and CISC — Only the Relevant Intuition

[PLACEMENT/INTERVIEW + GATE SUPPORT]

RISC

Reduced Instruction Set Computer

Typical characteristics:

- simpler instructions,
- often fixed length,
- many registers,
- load/store design,
- pipeline-friendly.

CISC

Complex Instruction Set Computer

Typical characteristics:

- larger instruction set,
- more complex instructions,
- variable-length instructions common,
- memory operands often allowed.

Do not treat every characteristic as an absolute law.

2.37 RISC vs CISC — Quick Table

Feature	RISC tendency	CISC tendency
Instructions	Simpler	More complex
Length	Often fixed	Often variable
Registers	Usually many	Historically fewer

Feature	RISC tendency	CISC tendency
Memory operations	Load/store	More memory-operating instructions
Pipeline	Easier	More complex
Code size	May be larger	Often denser

This is useful background but not the central target of your listed GATE syllabus.

2.38 Expanding Opcode Concept

+GATE-relevant when asked.

Suppose fixed instruction length supports different instruction formats.

Example:

TEXT

3-address
2-address
1-address
0-address

If some instructions use fewer address fields, the unused bits can sometimes be used to extend the opcode.

This is called **expanding opcode**.

Basic Idea

Suppose instruction length = 16 bits.

Each address field = 4 bits.

For a 3-address instruction:

TEXT

Opcode	A1	A2	A3
4 b	4	4	4

Opcode has only 4 bits:

$2^4=16$

possible patterns.

But if a format uses only two addresses:

TEXT

Extended Opcode	A1	A2
8 b	4	4

Now extra bits can help distinguish additional operations.

Important

Expanding opcode questions can be tricky because some opcode patterns must be reserved to indicate the longer opcode formats.

We will solve these systematically in a dedicated section now.

2.39 Expanding Opcode — GATE Method

Suppose:

- instruction length = 16 bits
- address field = 4 bits
- there are 15 three-address instructions.

3-address format:

TEXT

Opcode: 4 bits
A1: 4
A2: 4
A3: 4

Possible 4-bit opcode patterns:

$\lfloor 2^4=16 \rfloor$

15 are used.

Remaining:

$\lfloor 16-15=1 \rfloor$

pattern can indicate:

this is not a 3-address instruction; continue decoding more bits.

Now the next 4 bits can extend opcode.

Thus:

$\lfloor 1 \times 2^4=16 \rfloor$

possible extended patterns at the next level.

If some are used as 2-address instructions, one or more remaining patterns may continue to 1-address format.

This is the key logic.

2.40 Expanding Opcode Example

Suppose:

- 16-bit instruction
- address field = 4 bits
- 14 three-address instructions
- 15 two-address instructions

Find possible number of one-address instruction opcodes.

Step 1 — 3-address

Opcode length:

$$\lfloor 16-3(4)=4 \rfloor$$

Total 4-bit opcode patterns:

$$\lfloor 16 \rfloor$$

Used:

$$\lfloor 14 \rfloor$$

Remaining prefixes:

$$\lfloor 16-14=2 \rfloor$$

Step 2 — Extend by one address-field width

Each remaining prefix can combine with 4 more bits:

$$\lfloor 2 \times 2^4=32 \rfloor$$

possible extended opcode patterns.

15 are used for two-address instructions.

Remaining:

$$\lfloor 32-15=17 \rfloor$$

Step 3 — Extend again

For one-address instructions, each remaining 2-address opcode pattern can expand through another 4 bits:

$$\lfloor 17 \times 16 \rfloor$$

$$\lfloor \boxed{272} \rfloor$$

possible one-address opcode patterns, assuming all remaining patterns are allocated that way.

GATE Trap

Do not simply calculate:

TEXT

1 6 - b i t i n s t r u c t i o n " o n e 4 - b i t a d d r e s s = 1 2 - b i t o p c o d e
 $2^{12} = 4096$

because some opcode prefixes are already occupied by higher-address formats.

Expanding opcode requires prefix accounting.

2.41 Code Size

Code size depends on:

$\lceil \boxed{\text{Number of instructions}} \times \text{instruction length} \rceil$

For fixed-length instructions.

Example:

100 instructions, each 32 bits:

$\lceil 100 \times 32 = 3200 \text{ bits} \rceil$

$\lceil = 400 \text{ bytes} \rceil$

2.42 Instruction Count vs Instruction Complexity

A more powerful instruction may reduce instruction count.

Example:

Architecture A:

TEXT

MULADD X, A, B, C

one complex instruction.

Architecture B may need:

TEXT

MUL R1, A, B

ADD X, R1, C

two instructions.

But fewer instructions does **not automatically** mean faster execution.

Why?

Because complex instructions may take more cycles.

Later:

$\lceil \text{CPU Time} = IC \times CPI \times T \rceil$

So all three factors matter:

- instruction count,
- CPI,
- clock cycle time.

2.43 Explicit vs Implicit Operands

Explicit Operand

Written in instruction.

Example:

TEXT

ADD R1, R2

R1 and R2 are explicit.

Implicit Operand

Processor assumes it automatically.

Examples:

1-address

TEXT

ADD X

Accumulator is implicit.

0-address

TEXT

ADD

top two stack operands are implicit.

2.44 Source vs Destination Operand

Example:

$[R1 \rightarrow R2 + R3]$

Here:

TEXT

R2, R3 = source operands
R1 = destination

In:

TEXT

ADD R1, R2

if semantics are:

$[R1 \rightarrow R1 + R2]$

then:

- R1 is both source and destination.
- R2 is source.

2.45 Destructive vs Non-Destructive Instructions

A **destructive** instruction overwrites one source.

Example:

```
TEXT
ADD R1, R2
```

with:

$R1 \leftarrow R1 + R2$

old value of R1 is lost.

A 3-address instruction:

```
TEXT
ADD R3, R1, R2
```

can preserve both sources:

$R3 \leftarrow R1 + R2$

2.46 GATE Example — 0/1/2/3 Address

Which architecture typically requires postfix expression evaluation?

A. 3-address B. 2-address C. 1-address D. 0-address

Answer:

$\boxed{D}$

because zero-address machines usually use stacks.

2.47 GATE Example — Accumulator

Which instruction is most naturally associated with a one-address machine?

```
TEXT
ADD X
```

Because accumulator is implicit:

$AC \leftarrow AC + M[X]$

2.48 GATE Example — Memory Access Count

Suppose instruction:

```
TEXT
LOAD X
```

on an accumulator machine means:

$\lceil AC \rceil \rightarrow M[X]$

Assume:

- instruction is one memory word,
- direct addressing,
- instruction and data are in memory.

Memory accesses:

1. Fetch instruction
2. Fetch operand from X

Total:

$\boxed{2}$

2.49 Indirect Addressing Preview

Suppose:

TEXT

LOAD (X)

uses indirect addressing.

Then memory may hold:

TEXT

$M[X] = \text{address } Y$

$M[Y] = \text{actual operand}$

Memory accesses may become:

1. instruction fetch
2. read pointer from X
3. read operand from Y

Total:

$\boxed{3}$

We will study this deeply in Chapter 3.

2.50 GATE Example — Expression Instruction Count

Expression:

$Z = (A+B) - (C+D)$

3-address machine

TEXT

ADD R1, A, B

ADD R2, C, D

SUB Z, R1, R2

Arithmetic instructions:

$\boxed{3}$

Stack machine

Postfix:

TEXT

A B + C D + -

Arithmetic operators:

TEXT

+
+
-

So arithmetic instructions:

$\boxed{3}$

But stack also requires PUSH/POP instructions.

Therefore, when GATE asks "number of instructions," carefully check whether it counts:

- only arithmetic instructions,
- or every instruction including LOAD/PUSH/STORE.

2.51 GATE Trap — Syntax Convention

Different textbooks/architectures may interpret:

TEXT

ADD R1, R2

as either:

$R1 \leftarrow R1 + R2$

or another operand order.

Always follow the semantics given in the question.

Do not rely only on notation.

2.52 GATE Trap — Address "Operand Value

Suppose instruction contains:

TEXT

Address field = 500

That does not necessarily mean operand value = 500.

It may mean:

TEXT

```
operand = M[500]
```

depending on addressing mode.

The distinction becomes crucial in Chapter 3.

2.53 GATE Trap — Number of Address Fields

"Three-address instruction" means three explicit operand/address fields.

It does **not** mean the computer can only address three memory locations.

2.54 GATE Trap — Zero-Address Doesn't Mean No Operands

A zero-address instruction such as:

TEXT

```
ADD
```

still operates on operands.

They are simply **implicit**, usually on the stack.

2.55 High-Yield Comparison

Feature	3-address	2-address	1-address	0-address
Explicit addresses	3	2	1	0
Typical architecture	Registers	Registers	Accumulator	Stack
Instruction length tendency	Larger	Medium	Smaller	Small
Need for extra instructions	Lower	Moderate	Higher	PUSH/POP heavy
Implicit operands	Few/none	One role overlaps	AC	Stack top

2.56 Fast Expression Strategy for GATE

When given an expression:

For 3-address

Create one temporary result per operation.

Example:

```
\[ (A+B)(C-D) \]
```

TEXT

```
ADD T1, A, B
SUB T2, C, D
MUL X, T1, T2
```

For 2-address

Remember:

destination gets destroyed.

Use MOV/LOAD first.

For 1-address

Think:

Everything revolves around AC.

Often:

TEXT

```
LOAD
OP
STORE temporary
```

For 0-address

Convert to postfix.

Then:

TEXT

```
operand !' PUSH
operator !' arithmetic instruction
```

2.57 Postfix Quick Conversion

Expression:

$\backslash [A+B \times C]$

Multiplication first:

$\backslash [A+(B \times C)]$

Postfix:

TEXT

```
A B C * +
```

Expression:

$\boxed{(A+B) \times C}$

Postfix:

TEXT

A B + C *

Expression:

$\boxed{(A-B)/(C+D)}$

Postfix:

TEXT

A B - C D + /

2.58 Placement / Interview Connection

ISA helps you understand why:

- compiler-generated assembly looks the way it does,
- registers are important,
- machine code differs between processor families,
- some architectures use simpler instructions,
- function calls and stacks work differently at machine level.

But for now, keep your focus on GATE numericals.

CHAPTER 2 — FORMULA SHEET

Number of patterns from n bits

$\boxed{2^n}$

Bits needed for N possibilities

$\boxed{\lceil \log_2 N \rceil}$

Fixed-length code size

$\boxed{\text{Code Size} = \text{Instruction Count} \times \text{Instruction Length}}$

CHAPTER 2 — MEMORY TRICKS

Number of addresses

TEXT

```
3 !' 3 e x p l i c i t
2 !' d e s t i n a t i o n   o v e r l a p s   s o u r c e
1 !' A C
0 !' S t a c k
```

Memory sentence:

Three registers, Two overwrite, One accumulates, Zero stacks.

Opcode bits

N choices need ceiling $\log_2 N$ bits.

Zero-address

Zero addresses !' operands hide on Stack.

One-address

One address !' AC is the invisible partner.

CHAPTER 2 — GATE TRAPS

1. 0-address does not mean zero operands.
2. 1-address machines use implicit accumulator.
3. 2-address destination is usually also a source.
4. 3-address instructions generally need larger instruction formats.
5. Postfix operand order matters for subtraction/division.
6. Register operands do not require main-memory operand fetches.
7. Instruction fetch itself is also a memory access unless told otherwise.
8. Bits needed for N choices are $\lceil \log_2 N \rceil$.
9. Expanding opcode requires accounting for already-used opcode prefixes.
10. Fewer machine instructions do not automatically imply less CPU time.

CHAPTER SNAPSHOT

1. ISA defines processor-visible machine behavior.
2. Opcode specifies the operation.
3. Operands specify data/locations.

4. ☐ k opcode bits can represent (2^k) opcodes.
 5. ☐ R registers require $\lceil \log_2 R \rceil$ register-select bits.
 6. 3-address uses explicit destination + 2 sources.
 7. 2-address commonly overwrites one source.
 8. 1-address uses accumulator.
 9. 0-address uses stack.
 10. Stack machines naturally use postfix expressions.
 11. PUSH/POP may increase total instruction count in stack machines.
 12. Instruction fetch must be considered in memory-access questions.
 13. Load/store architectures perform arithmetic mainly on registers.
 14. Fixed-length instructions simplify decoding.
 15. Expanding opcode allows formats with fewer operands to use larger opcode spaces.
-

PRACTICE SET

Do these before checking answers.

Q1 — L0 MCQ

A processor has 128 registers. How many bits are required to identify one register?

A. 6 B. 7 C. 8 D. 128

Q2 — L1 NAT

A machine supports 150 different opcodes.

Minimum number of opcode bits required = ?

Q3 — L1 MCQ

A one-address instruction architecture most commonly uses:

A. Stack B. Accumulator C. Cache D. Program counter

Q4 — L1 MCQ

A zero-address architecture is naturally associated with:

A. Queue B. Heap C. Stack D. Cache

Q5 — L1

Convert:

$[(A+B) \times (C-D)]$

into postfix notation.

Q6 — L2

Write a possible 3-address instruction sequence for:

$[X = (A - B) + (C \times D)]$

Q7 — L2

Write a possible 1-address accumulator sequence for:

$[X = (A + B) - (C + D)]$

You may use a temporary location `T`.

Q8 — L2 NAT

A 32-bit instruction contains:

- 8-bit opcode,
- three equal-size register fields.

How many bits does each register field contain?

Q9 — L2 NAT

A processor has 64 registers. An instruction contains:

- one 8-bit opcode,
- three register fields.

What is the minimum instruction length in bits?

Q10 — L2

A direct-address accumulator instruction:

```
TEXT
ADD X
```

requires one instruction fetch and one data read.

How many total memory accesses?

Q11 — L3

A 16-bit instruction has 4-bit address fields.

There are 15 three-address instructions.

If all remaining opcode capacity is used for two-address instructions, how many different two-address instructions can be encoded?

Q12 — L3

For postfix:

TEXT

A B C - /

what mathematical expression is represented?

Be careful with operand order.

CHAPTER 2 COMPLETION STATUS

' Instruction-set concept ' ISA ' Opcode and operands ' Instruction fields ' Instruction length ' Opcode-bit calculations ' Register-selection bits ' Instruction categories ' 3-address instructions ' 2-address instructions ' 1-address instructions ' 0-address instructions ' Stack architecture ' Accumulator architecture ' Register architecture ' Expression evaluation ' Postfix basics ' Instruction-count reasoning ' Memory-access reasoning ' Fixed/variable instruction formats ' Load/store concept ' RISC/CISC support concepts ' Expanding opcode basics and numerals ' Common GATE traps

CHAPTER 2: COMPLETE '

COURSE PROGRESS

Completed: 3 / 26 modules

Approximate completion:

$\sim 12\%$

The first major ISA block is now complete.

NEXT !' CHAPTER 3 — ADDRESSING MODES

This is ~~Ø~~[✓]**VERY HIGH GATE PRIORITY**.

Next we will master:

- immediate,
- direct,
- indirect,
- register,
- register indirect,
- indexed,
- base,
- PC-relative,
- auto-increment/decrement,
- stack/implied modes,
- **effective address formulas,**

- **memory-access counting**,
- tricky GATE numericals,
- one final comparison table covering all addressing modes.

USER

next

ASSISTANT

CHAPTER 3 — ADDRESSING MODES

Classification: [GATE CORE] **Priority:** ~~Ø=VERY~~ **VERY HIGH** **Goal:** Understand how an instruction locates its operand and solve effective-address + memory-access numericals quickly.

3.1 What is an Addressing Mode?

Suppose CPU executes:

TEXT

LOAD R1, X

Question

USER

next

ASSISTANT

CHAPTER 3 — ADDRESSING MODES

Classification: [GATE CORE] **Priority:** ~~Ø=VERY~~ **VERY HIGH** **Goal:** Learn exactly how the CPU finds an operand and solve **effective-address + memory-access** questions correctly.

3.1 What is an Addressing Mode?

Suppose CPU executes:

TEXT

LOAD R1, X

What does X mean?

It could mean:

- the actual value X ,
- memory location X ,
- a register containing the address,
- an offset added to another register,
- an address relative to PC.

The rule used to interpret the operand field is called an:

$\boxed{\text{Addressing Mode}}$

3.2 Why Do We Need Addressing Modes?

Different situations require different ways of accessing data.

Examples:

Constant value

TEXT

R 1 !• 1 0

Best suited to **Immediate Addressing**.

Variable in memory

TEXT

R 1 !• M [5 0 0]

Can use **Direct Addressing**.

Pointer

C

*p

Naturally relates to **Indirect / Register Indirect Addressing**.

Array

C

A[i]

Naturally relates to **Indexed Addressing**.

Branch

TEXT

jump 20 instructions ahead

Naturally relates to **PC-relative addressing**.

So addressing modes make instruction sets flexible.

3.3 Effective Address — EA

Extremely important term.

The **Effective Address (EA)** is the actual memory address where the operand is located.

Example:

If instruction contains:

TEXT

```
Address field = 500
```

and direct addressing is used:

`[ EA=500 ]`

Operand:

`[ M[500] ]`

Important Notation

Throughout this chapter:

- `A` = address/displacement field in instruction
- `R` = contents of a register
- `PC` = Program Counter
- `EA` = Effective Address
- `M[X]` = contents of memory location `X`

3.4 Addressing Modes Overview

We will cover:

1. Immediate
2. Direct
3. Indirect
4. Register
5. Register Indirect
6. Indexed
7. Base
8. Base + Index
9. PC-relative
10. Auto-increment
11. Auto-decrement
12. Stack
13. Implied

3.5 Immediate Addressing

In **Immediate Addressing**, the operand value is present directly inside the instruction.

Example:

```
TEXT
MOV R1, #25
```

Here:

```
TEXT
25
```

is the actual operand.

So:

$\boxed{\text{Operand}=A}$

There is normally **no effective memory address for the operand** because the value itself is in the instruction.

Intuition

Instruction says:

"Use this number directly."

Example:

```
TEXT
ADD R1, #5
```

means:

$R1 \leftarrow R1 + 5$

Memory Accesses

Assuming instruction itself is in memory:

1. Fetch instruction

No extra memory access for operand.

Therefore:

$\boxed{1 \text{ memory access}}$

for instruction + operand retrieval.

Advantage

Very fast for constants.

Limitation

Operand size is limited by the number of bits available in the instruction.

Memory Trick

Immediate = value is immediately available.

GATE Trap

Suppose:

```
TEXT
MOV R1, #500
```

Do not interpret it as:

$R1 \rightarrow M[500]$

It means:

$R1 \rightarrow 500$

3.6 Direct Addressing

In **Direct Addressing**, the address field itself gives the memory address of the operand.

$EA = A$

Operand:

$M[A]$

Example:

```
TEXT
LOAD R1, 500
```

assuming direct mode:

$EA = 500$

$R1 \rightarrow M[500]$

Example

Suppose:

TEXT

$M[500] = 90$

Instruction:

TEXT

LOAD R1, 500

Then:

$\boxed{R1=90}$

Memory Accesses

Typically:

1. Fetch instruction
2. Fetch operand

Total:

$\boxed{2}$

Advantage

Simple.

Disadvantage

Address range is limited by the instruction's address-field size.

Memory Trick

Direct = instruction directly tells the destination address.

3.7 Indirect Addressing

Important and frequently confused.

In **Indirect Addressing**, the address field points to a memory location that contains the **effective address**.

So:

$\boxed{EA=M[A]}$

Then operand is:

$\boxed{M[EA]}$

Equivalent:

$\text{Operand} = M[M[A]]$

Example

Suppose instruction contains:

```
TEXT
A = 500
```

and:

```
TEXT
M[500] = 800
M[800] = 25
```

Then:

Step 1

$EA = M[500] = 800$

Step 2

$\text{Operand} = M[800] = 25$

Therefore:

$\boxed{\text{Operand} = 25}$

Diagram

```
TEXT
Instruction
|
| A = 500
| "
Memory[500]
|
| 800
| "
Memory[800]
|
| 25
| "
Operand
```

Memory Accesses

Typically:

1. Instruction fetch
2. Read memory location containing EA
3. Read actual operand

Total:

$\boxed{3}$

Memory Trick

Indirect = address of the address.

Major GATE Trap

Direct:

$EA=A$

Indirect:

$EA=M[A]$

Do not confuse these.

3.8 Register Addressing

In **Register Addressing**, the operand is inside a CPU register.

Example:

TEXT

ADD R1, R2

If R2 contains 10:

$\text{Operand}=10$

No main-memory operand access is required.

Formula

$\boxed{\text{Operand}=R}$

No memory EA is needed for the operand.

Memory Accesses

Typically:

1. Instruction fetch only

So:

$\boxed{1}$

main-memory access.

Advantage

Very fast.

Limitation

Number of registers is small compared with memory.

Memory Trick

Register mode = value is inside register.

3.9 Register Indirect Addressing

Very important distinction.

In **Register Indirect Addressing**, the register contains the memory address of the operand.

$\boxed{EA=R}$

Operand:

$\boxed{M[R]}$

Example

Suppose:

```
TEXT
R2 = 500
M[500] = 35
```

Instruction:

```
TEXT
LOAD R1, (R2)
```

Then:

$\boxed{EA=R2=500}$

$\boxed{\text{Operand}=M[500]=35}$

Therefore:

$\boxed{R1=35}$

Compare Register vs Register Indirect

Suppose:

TEXT

R2 = 500
M[500] = 35

Register mode

TEXT

MOV R1, R2

gives:

$\boxed{R1=500}$

Register indirect

TEXT

LOAD R1, (R2)

gives:

$\boxed{R1=35}$

Ø=No distinction.

Memory Accesses

Typically:

1. Instruction fetch
2. Operand memory access

Total:

$\boxed{2}$

Why not 3?

Because the effective address is already in a **register**, not memory.

Memory Trick

Register = value

Register Indirect = register points to value

3.10 Indexed Addressing

Used heavily for:

- arrays,
- tables,
- loops.

Effective address is computed by adding:

- instruction address/displacement field,
- index register.

$\boxed{EA=A+IX}$

where IX is index-register content.

Example

Suppose:

TEXT

A = 1000
IX = 20

Then:

$EA=1000+20$

$\boxed{EA=1020}$

If:

TEXT

M[1020]=75

operand is:

$\boxed{75}$

Array Intuition

Suppose array starts at address 1000.

Index register contains offset 12.

Then:

TEXT

Base address = 1000
Offset = 12

So:

$EA=1012$

Important

If an array element size is greater than 1 byte, the index might need scaling.

For example:

```
C
int A[i]
```

with 4-byte integers:

$\boxed{\text{Address}(A[i]) = \text{Base} + 4i}$

But whether scaling is implicit depends on architecture/question.

Never assume unless stated.

3.11 Base Register Addressing

In **Base Addressing**:

$\boxed{\text{EA} = \text{BR} + \text{A}}$

where:

- **BR** = base-register content
- **A** = displacement/address field.

Example

Suppose:

```
TEXT
BR = 4000
A  = 120
```

Then:

$\boxed{\text{EA} = 4000 + 120}$

$\boxed{4120}$

Why useful?

Base register can represent:

- starting address of a program,
- starting address of a data structure,
- beginning of memory segment.

Instruction supplies small offset.

Indexed vs Base — Subtle Difference

Mathematically they may both look like:

$\text{[Register + displacement]}$

But conceptually:

Indexed

Index register usually changes.

Base address remains relatively fixed.

Base

Base register is usually fixed.

Instruction displacement changes.

Memory Trick

Base = where structure begins

Index = which element

3.12 Base-Indexed Addressing

Some architectures combine:

- base register,
- index register,
- optional displacement.

General form:

$\text{[} \boxed{EA=BR+IX+A} \text{]}$

Example:

TEXT

```
BR = 1000
IX = 40
A  = 8
```

Then:

$\text{[} EA=1000+40+8 \text{]}$

$\text{[} \boxed{1048} \text{]}$

Practical interpretation

Very useful for:

TEXT

`array[index]`

inside a larger data structure.

But keep this as supporting knowledge unless a question explicitly uses it.

3.13 Relative / PC-Relative Addressing

Not important for branches.

Effective address is computed relative to the Program Counter.

$$EA = PC + A$$

where A is usually a signed displacement.

Example

Suppose:

TEXT

PC = 1000
A = 40

Then:

$$EA = 1000 + 40$$

$$EA = 1040$$

Negative Offset

Suppose:

TEXT

PC = 1000
A = -100

Then:

$$EA = 900$$

This allows backward branches used in loops.

Why PC-relative?

You do not need to store a full target address.

Instead:

TEXT

Target = Current PC + small displacement

This can reduce instruction bits and support relocatable code.

GATE Trap — Which PC?

Some questions define whether PC points to:

- current instruction,
- next instruction after fetch.

In real ISAs, PC-relative addressing often uses the already-updated PC.

Therefore:

Always use the exact PC value specified in the question.

Do not blindly add instruction length unless the question's convention requires it.

3.14 Auto-Increment Addressing

In auto-increment:

1. Use register content as EA.
2. Access operand.
3. Increment register.

Initially:

$\boxed{EA=R}$

then:

$\boxed{R \leftarrow R+d}$

where d may be:

- 1 byte,
- word size,
- operand size.

Example

Suppose:

TEXT

R2 = 1000
M[1000] = 50

Instruction uses auto-increment with increment 4.

First:

$\boxed{EA=1000}$

Operand:

$\boxed{50}$

Then:

$\boxed{R2=1004}$

Intuition

Perfect for scanning arrays sequentially.

TEXT

```
read A[0]
move pointer
read A[1]
move pointer
...
```

GATE Trap

Increment happens:

after accessing the operand.

3.15 Auto-Decrement Addressing

In auto-decrement:

1. Decrement register.
2. Use updated register as EA.

$\boxed{R \leftarrow R-d}$

then:

$\boxed{EA=R}$

Example

Initially:

TEXT

```
R2 = 1000
d = 4
```

First:

$\boxed{R2=996}$

Then:

\[EA=996 \]

Operand:

\[M[996] \]

Compare

Auto-Increment

TEXT

U s e ! ' I n c r e m e n t

Auto-Decrement

TEXT

D e c r e m e n t ! ' U s e

Memory trick:

Post-increment, Pre-decrement

3.16 Stack Addressing

In stack addressing, operand locations are implicitly determined by the stack pointer.

Example:

TEXT

ADD

The CPU may use:

- top stack value,
- next stack value.

No address field is required.

This is the addressing behavior of a zero-address machine.

Example

Stack:

TEXT

T O P ! ' 8
5

Execute:

TEXT

ADD

Result:

TEXT

T O P ! ' 1 3

Operands were implicit.

3.17 Implied / Implicit Addressing

In implied addressing, the operand is implied by the opcode itself.

Example:

TEXT

CMA

might mean:

complement accumulator.

No operand needs to be specified because accumulator is implied.

Another example:

TEXT

CLC

may mean clear carry flag.

Difference from Immediate

Do not confuse:

Immediate

Operand appears directly in instruction.

TEXT

ADD #5

Implied

Operand is not explicitly present.

TEXT

CMA

CPU already knows the operand.

3.18 Complete Effective Address Table

Memorize this table.

Addressing Mode	Effective Address / Operand
Immediate	Operand = A
Direct	$\text{EA} = A$
Indirect	$\text{EA} = M[A]$
Register	Operand = R
Register Indirect	$\text{EA} = R$
Indexed	$\text{EA} = A + IX$
Base	$\text{EA} = A + BR$
Base + Index	$\text{EA} = A + BR + IX$
PC-relative	$\text{EA} = PC + A$
Auto-increment	$\text{EA} = R$, then increment R
Auto-decrement	decrement R, then $\text{EA} = R$
Stack	Operand implied by SP
Implied	Operand implied

3.19 Memory Access Table

Assume:

- instruction is stored in memory,
- operand is in memory when applicable,
- no cache effects,
- no additional complications.

Mode	Instruction Fetch	Extra Operand/Pointer Reads	Typical Total
Immediate	1	0	1
Register	1	0	1
Direct	1	1	2
Register Indirect	1	1	2
Indirect	1	2	3
Indexed	1	1	2

Mode	Instruction Fetch	Extra Operand/Pointer Reads	Typical Total
Base	1	1	2
PC-relative	1	often 1 if accessing data	typically 2
Auto-inc/dec	1	1	2

Important Caveat

If an addressing mode is used by a branch instruction, the calculated EA may become a **new PC** rather than an operand-memory address.

Then there may be no data read from that target at the moment.

So never memorize memory-access counts without reading the operation.

3.20 Direct vs Indirect — Fast Comparison

Suppose:

```
TEXT
A = 300
M[300] = 500
M[500] = 80
```

Direct

$\backslash$ EA=300 $\backslash$

Operand:

$\backslash$ M[300]=500 $\backslash$

Answer:

$\backslash$ $\boxed{500}$ $\backslash$

Indirect

$\backslash$ EA=M[300]=500 $\backslash$

Operand:

$\backslash$ M[500]=80 $\backslash$

Answer:

$\backslash$ $\boxed{80}$ $\backslash$

3.21 Register vs Register Indirect

Suppose:

TEXT

R1 = 400
M[400] = 60

Register

Operand:

$\boxed{400}$

Register indirect

$\boxed{EA=400}$

Operand:

$\boxed{60}$

Memory trick:

Register: use what's in the box.

Register indirect: box contains an address.

3.22 GATE Numerical 1 — Direct

Instruction uses direct addressing.

TEXT

Address field = 1200
M[1200] = 75

Find EA and operand.

$\boxed{EA=A}$

$\boxed{EA=1200}$

Operand:

$\boxed{M[1200]=75}$

Therefore:

$\boxed{EA=1200, \text{Operand}=75}$

3.23 GATE Numerical 2 — Indirect

Given:

TEXT

A = 700
M[700] = 1200

$M[1200] = 55$

Indirect mode:

$\{ EA=M[A] \}$

$\{ EA=M[700]=1200 \}$

Operand:

$\{ M[1200]=55 \}$

Answer:

$\{ \boxed{EA=1200, \text{Operand}=55} \}$

3.24 GATE Numerical 3 — Register Indirect

Given:

TEXT

$R3 = 1500$

$M[1500] = 42$

Then:

$\{ EA=R3=1500 \}$

Operand:

$\{ M[1500]=42 \}$

Answer:

$\{ \boxed{EA=1500, \text{Operand}=42} \}$

3.25 GATE Numerical 4 — Indexed

Given:

TEXT

Address field = 500

Index register = 28

$M[528] = 99$

Then:

$\{ EA=500+28 \}$

$\{ \boxed{EA=528} \}$

Operand:

$\{ \boxed{99} \}$

3.26 GATE Numerical 5 — Base

Given:

TEXT

Base register = 4000
Displacement = 250

Then:

$EA = 4000 + 250$

$\boxed{4250}$

3.27 GATE Numerical 6 — PC Relative

Suppose:

TEXT

PC = 2000
Displacement = -80

Then:

$EA = 2000 - 80$

$\boxed{1920}$

3.28 GATE Numerical 7 — Auto-Increment

Suppose:

TEXT

R2 = 1000
Operand size = 4 bytes
 $M[1000] = 25$

Use auto-increment.

Operand address

$EA = 1000$

Operand

25

New R2

$1000 + 4 = 1004$

Answer:

$\boxed{EA=1000, \text{Operand}=25, \text{New R2}=1004}$

3.29 GATE Numerical 8 — Auto-Decrement

Suppose:

```
TEXT
R2 = 1000
Operand size = 4 bytes
```

Auto-decrement:

$\lfloor R2=1000-4=996 \rfloor$

Then:

$\lfloor EA=996 \rfloor$

Answer:

$\lfloor \boxed{\text{New } R2=996, EA=996} \rfloor$

3.30 Memory Access Counting

This deserves careful treatment because it is a common trap.

Suppose instruction is one memory word.

Immediate

```
TEXT
ADD #5
```

1 instruction fetch.

Operand is inside instruction.

$\lfloor \boxed{1} \rfloor$

Register

```
TEXT
ADD R1
```

1 instruction fetch.

Operand in CPU register.

$\lfloor \boxed{1} \rfloor$

Direct

```
TEXT
ADD X
```

1 instruction fetch.

1 operand read.

$\boxed{2}$

Indirect

TEXT

ADD @X

1 instruction fetch.

1 read to get EA.

1 read to get operand.

$\boxed{3}$

Register Indirect

TEXT

ADD (R1)

1 instruction fetch.

Address is already inside register.

1 operand read.

$\boxed{2}$

3.31 If Result Must Be Written to Memory

Suppose instruction is:

TEXT

STORE R1, X

direct addressing.

Possible accesses:

1. instruction fetch
2. memory write

Total:

$\boxed{2}$

There is no operand-memory **read** because the CPU is writing.

Important

Always ask:

Does this instruction READ memory, WRITE memory, or both?

Do not apply a memorized count blindly.

3.32 Multi-Level Indirect Addressing

[LOW PRIORITY but useful]

Suppose instruction uses two levels of memory indirection:

$EA = M[M[A]]$

Then the chain might be:

```
TEXT
A
!"
M[A] = P1
!"
M[P1] = P2
!"
M[P2] = Operand
```

Memory accesses increase with each level.

If there are k indirections:

$\boxed{1 \text{ instruction fetch} + k \text{ pointer reads} + 1 \text{ operand read}}$

for a memory operand.

3.33 Addressing Mode and Pointers

Programming connection:

```
C
int *p;
```

If register contains p , register indirect is conceptually:

```
TEXT
R contains address
!"
memory[address]
```

That is why pointer-heavy code maps naturally to indirect addressing.

3.34 Addressing Mode and Arrays

Suppose:

```
C
A[i]
```

Conceptually:

$$\text{Address of } A[i] = \text{Base}(A) + i \times \text{element size}$$

Example:

- A starts at 1000
- integer size = 4 bytes
- `i = 5`

Then:

$$1000 + 5(4) = 1020$$

Indexed addressing is designed for this kind of computation.

3.35 Addressing Mode and Loops

Consider:

```
C
for(i=0; i<n; i++)
    sum += A[i];
```

A processor may repeatedly:

1. access address in a register,
2. automatically increment pointer,
3. fetch next element.

Auto-increment addressing makes this efficient.

3.36 Addressing Mode and Branches

Suppose instruction is located near its branch target.

Instead of storing full target:

```
TEXT
0x00008020
```

we can store:

```
TEXT
+32
```

and calculate:

$$\text{Target} = \text{PC} + 32$$

That is PC-relative addressing.

3.37 Address Field Size and Reach

Suppose PC-relative displacement has 8 signed bits.

Range:

$$[-2^7 \text{ to } 2^7 - 1]$$

$$[-128 \text{ to } 127]$$

units.

If each unit represents bytes, reach is:

TEXT

-128 to +127 bytes

If each displacement unit represents instructions or words, multiply accordingly.

3.38 GATE Numerical — Relative Reach

Suppose:

- displacement field = 12-bit signed
- byte offset

Range:

$$[-2^{11} \text{ to } 2^{11} - 1]$$

$$[-2048 \text{ to } 2047 \text{ bytes}]$$

3.39 Scaled Indexed Addressing

[PLACEMENT/INTERVIEW + SUPPORT]

Some processors support:

$$[EA = \text{Base} + \text{Index} \times \text{Scale} + \text{Displacement}]$$

Example:

$$[EA = 1000 + 5 \times 4 + 8]$$

$$[EA = 1028]$$

Scale often matches element size:

TEXT

1, 2, 4, 8 bytes

This is useful for arrays.

3.40 GATE Mixed Example

Given:

TEXT

PC = 500
R1 = 100
R2 = 200
A = 20

M[20] = 300
M[100] = 400
M[120] = 450
M[300] = 999

Find EA for each mode.

Direct

\[EA=A=20 \]

Indirect

\[EA=M[20]=300 \]

Register Indirect using R1

\[EA=R1=100 \]

Indexed using R1

\[EA=A+R1=20+100=120 \]

PC Relative

\[EA=PC+A=500+20=520 \]

3.41 Fast Recognition Table

When you see:

TEXT

#5

think:

Immediate.

When you see:

TEXT

500

used as memory address:

Direct.

When you see:

TEXT

M[500] gives another address

Indirect.

When operand comes directly from R1:

Register.

When R1 contains address:

Register Indirect.

When you see:

TEXT

base + offset

Base.

When you see:

TEXT

array base + index

Indexed.

When you see:

TEXT

PC + offset

Relative.

3.42 Common GATE Trap — Immediate vs Direct

Suppose:

TEXT

A = 100

M[100] = 55

Immediate

Operand:

$\boxed{100}$

Direct

Operand:

$\boxed{55}$

This single distinction can completely change the answer.

3.43 Common GATE Trap — Direct vs Indirect

Given:

TEXT

A = 100
M[100] = 500
M[500] = 20

Direct operand:

$\boxed{500}$

Indirect operand:

$\boxed{20}$

3.44 Common GATE Trap — Register vs Register Indirect

Given:

TEXT

R1 = 500
M[500] = 20

Register operand:

$\boxed{500}$

Register-indirect operand:

$\boxed{20}$

3.45 Common GATE Trap — EA vs Operand

Suppose:

$\boxed{EA=500}$

This does NOT necessarily mean:

$\boxed{Operand=500}$

If memory contains:

$\boxed{M[500]=90}$

then:

\[Operand=90 \]

Remember:

EA tells WHERE. Operand tells WHAT.

3.46 Common GATE Trap — Auto Increment Amount

If operand size is 4 bytes, increment may be:

\[+4 \]

not necessarily:

\[+1 \]

Always follow architecture/question.

3.47 Common GATE Trap — PC Relative

Do not blindly use:

\[PC+\text{instruction length}+\text{offset} \]

unless PC in the question still points to the current instruction.

If the question says PC already contains the next-instruction address, use that value directly.

3.48 Common GATE Trap — Memory Access Count

Instruction fetch itself is a memory access unless:

- instruction is assumed already fetched,
- question explicitly asks only operand accesses,
- cache/other assumptions alter the model.

Read wording carefully.

3.49 Addressing Mode Comparison

Mode	Operand Source	Main Benefit
Immediate	Instruction	Constants
Direct	Memory address in instruction	Simplicity
Indirect	Pointer in memory	Large/flexible addressing
Register	CPU register	Speed
Register Indirect	Memory via register pointer	Pointers
Indexed	Base/address + index	Arrays

Mode	Operand Source	Main Benefit
Base	Base register + displacement	Relocation/structures
PC Relative	PC + displacement	Branches
Auto Increment	Register then increment	Sequential traversal
Auto Decrement	decrement then register	Stack/backward traversal
Stack	SP/top of stack	Zero-address machines
Implied	Opcode implies operand	Compact instructions

3.50 GATE-STYLE EXAMPLE — Address and Operand

Suppose:

```

TEXT
Address field = 300
R1 = 100
PC = 500

M[100] = 700
M[300] = 800
M[400] = 900
M[800] = 50

```

Find operand for each.

Immediate

\[Operand=300 \]

Direct

\[Operand=M[300]=800 \]

Indirect

\[EA=M[300]=800 \]

\[Operand=M[800]=50 \]

Register

\[Operand=R1=100 \]

Register Indirect

\[EA=100 \]

\[Operand=M[100]=700 \]

Indexed using R1

\[EA=300+100=400 \]

\[Operand=M[400]=900 \]

3.51 Fast GATE Solving Method

When you see an addressing-mode numerical, make three rows:

TEXT

1. Find EA
2. Find operand
3. Count accesses

Example:

Indirect:

TEXT

A = 500
M[500] = 800
M[800] = 25

Write:

TEXT

EA = M[500] = 800
Operand = M[800] = 25
Access = instruction + pointer + operand = 3

This prevents most mistakes.

3.52 Formula Cheat Sheet

Immediate

$\boxed{\text{Operand}=A}$

Direct

$\boxed{\text{EA}=A}$

Indirect

$\boxed{\text{EA}=\text{M}[A]}$

Register

$\boxed{\text{Operand}=R}$

Register Indirect

$\boxed{\text{EA}=R}$

Indexed

$\boxed{\text{EA}=A+IX}$

Base

$\boxed{\text{EA}=A+BR}$

Base + Index

$\boxed{EA=A+BR+IX}$

PC Relative

$\boxed{EA=PC+A}$

Auto-Increment

$\boxed{EA=R;\quad R\leftarrow R+d}$

Auto-Decrement

$\boxed{R\leftarrow R-d;\quad EA=R}$

CHAPTER 3 — MEMORY TRICKS

Direct

D = Directly use address field

$\boxed{EA=A}$

Indirect

Address points to another address

$\boxed{EA=M[A]}$

Register

Register holds value

Register Indirect

Register holds address

Indexed

Index finds item

Base

Base finds beginning

Relative

Relative to PC

Auto

CHAPTER 3 — MUST REMEMBER

Most important:

$\boxed{\text{Immediate: \text{Operand} = A}}$

$\boxed{\text{Direct: \text{EA} = A}}$

$\boxed{\text{Indirect: \text{EA} = M[A]}}$

$\boxed{\text{Register: \text{Operand} = R}}$

$\boxed{\text{Register \ Indirect: \text{EA} = R}}$

$\boxed{\text{Indexed: \text{EA} = A + IX}}$

$\boxed{\text{PC \ Relative: \text{EA} = PC + A}}$

And:

EA = WHERE

Operand = WHAT

CHAPTER SNAPSHOT

1. Addressing mode tells the CPU how to locate an operand.
2. Effective address is the actual memory address of the operand.
3. Immediate stores the operand directly in the instruction.
4. Direct uses the instruction address field as EA.
5. Indirect uses memory to obtain EA.
6. Register addressing stores the operand directly in a register.
7. Register indirect uses a register as a pointer.
8. Indexed addressing is useful for arrays.
9. Base addressing uses a base + displacement.
10. PC-relative addressing is common for branches.
11. Auto-increment accesses first, then increments.
12. Auto-decrement decrements first, then accesses.
13. Stack/implied modes hide operand addresses.
14. Indirect addressing generally needs one additional memory access compared with direct.
15. Always distinguish EA from operand value.

PRACTICE SET

Q1 — L0 MCQ

Which addressing mode stores the actual operand in the instruction?

A. Direct B. Indirect C. Immediate D. Register indirect

Q2 — L1 NAT

For direct addressing:

TEXT

A = 500
M[500] = 70

Find EA.

Q3 — L1 NAT

For the same data, find the operand.

Q4 — L1 NAT

Indirect mode:

TEXT

A = 500
M[500] = 700
M[700] = 90

Find EA.

Q5 — L1 NAT

For Q4, find the operand.

Q6 — L1 MCQ

Suppose:

TEXT

R1 = 500
M[500] = 40

In register addressing, operand is:

A. 40 B. 500 C. M[40] D. Cannot determine

Q7 — L1 MCQ

Using the same values, in register-indirect addressing the operand is:

A. 40 B. 500 C. 540 D. 0

Q8 — L2 NAT

Indexed addressing:

TEXT

Address field = 1000

Index register = 24

Find EA.

Q9 — L2 NAT

PC-relative addressing:

TEXT

PC = 4000

Displacement = -80

Find target EA.

Q10 — L2 NAT

Auto-increment:

TEXT

R1 = 2000

operand size = 8 bytes

Find:

1. EA used for current access
2. new value of R1

Q11 — L2 NAT

Auto-decrement:

TEXT

R1 = 2000

operand size = 8 bytes

Find EA.

Q12 — L2 MCQ

Assume one-word instructions and no cache. Which mode typically requires the most memory accesses for reading an operand?

A. Immediate B. Register C. Direct D. Indirect

Q13 — L2 NAT

A PC-relative instruction has:

- PC = 10000
- 10-bit signed byte displacement
- maximum positive displacement.

What is the maximum target address?

Q14 — L3

Given:

TEXT

A = 100
R = 30
PC = 500

M[100] = 200
M[130] = 75
M[200] = 90
M[30] = 60

Find the **operand** under:

1. Immediate
2. Direct
3. Indirect
4. Register
5. Register indirect
6. Indexed using R

CHAPTER 3 COMPLETION STATUS

' Addressing mode concept ' Effective address ' Immediate ' Direct ' Indirect ' Register ' Register indirect ' Indexed ' Base ' Base-index ' PC-relative ' Auto-increment ' Auto-decrement ' Stack addressing ' Implied addressing ' Effective-address formulas ' Memory-access counting ' Array/pointer/branch intuition ' Relative-address range ' GATE traps and shortcuts

CHAPTER 3: COMPLETE '

UNIT TEST 1 STATUS

We have now completed the first linked GATE unit:

A mixed revision test will be included after the next logical checkpoint so we keep momentum.

COURSE PROGRESS

Completed: 4 / 26 modules

Approximate progress:

$\sim 15\%$

NEXT !' CHAPTER 4 — ALU DESIGN

Next is another major GATE block:

~~Ø=VERY~~ Arithmetic and Logic Unit

We will cover:

- half adder,
- full adder,
- ripple-carry adder,
- carry propagation,
- carry look-ahead,
- generate/propagate,
- subtraction using 2's complement,
- overflow detection,
- arithmetic circuit,
- logic circuit,
- multiplexers,
- shifts,
- ALU control,
- zero/carry/sign/overflow flags,
- GATE numericals and shortcuts.

USER

next

ASSISTANT

CHAPTER 4 — DESIGN OF ARITHMETIC AND LOGIC UNIT (ALU)

Classification: [GATE CORE] **Priority:** ~~Ø=VERY~~ VERY HIGH **Goal:** Understand how arithmetic and logical operations are implemented inside the CPU and solve GATE questions on adders, overflow, carry, shifts, flags, and ALU selection.

4.1 What is an ALU?

ALU = Arithmetic and Logic Unit

It is the CPU component that performs operations such as:

Arithmetic

```
TEXT
Addition
Subtraction
Increment
Decrement
```

Logical

```
TEXT
AND
OR
XOR
NOT
```

Other operations

```
TEXT
Comparison
Shift left
Shift right
```

Simplified:

```
TEXT
A % % % % %
      ! "
      % % % % %
B % % ! ' % A L U % % % ! ' R e s u l t
      % % % % %
      ! '
Control signals
```

The control unit tells the ALU which operation to perform.

4.2 Why do we need ALU design?

Suppose CPU receives:

```
TEXT
ADD R1, R2
```

Some hardware must actually perform:

$[R1 + R2]$

That hardware is built from digital circuits.

For addition, we start with:

- Half Adder
- Full Adder
- Multi-bit Adders

4.3 Half Adder

A **Half Adder** adds two single-bit inputs.

Inputs:

$\{A, B\}$

Outputs:

- Sum
- Carry

Truth Table

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

From the table:

$\boxed{\text{Sum} = A \oplus B}$

$\boxed{\text{Carry} = A \cdot B}$

Memory Trick

Half Adder:

XOR gives SUM

AND gives CARRY

4.4 Why is Half Adder insufficient?

Consider:

TEXT

1 + 1 + Carry-in

A half adder accepts only:

TEXT

A

B

but higher bit positions may receive a carry from the previous position.

Therefore we need a:

$\boxed{\text{Full Adder}}$

4.5 Full Adder

A Full Adder adds three bits:

- A
- B
- Carry-in (C_{in})

Outputs:

- Sum
- Carry-out (C_{out})

Truth Table

A	B	(C_{in})	Sum	(C_{out})
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Full Adder Sum

$\boxed{S = A \oplus B \oplus C_{in}}$

Carry-out:

$$C_{out} = AB + AC_{in} + BC_{in}$$

Equivalent useful form:

$$C_{out} = AB + (A \oplus B)C_{in}$$

4.6 Building a Multi-Bit Adder

Suppose we want to add:

TEXT

```
A3 A2 A1 A0
B3 B2 B1 B0
```

We connect multiple full adders.

TEXT

```

C 0  !'   F A 0  !'   C 1  !'   F A 1  !'   C 2  !'   F A 2  !'   C 3  !'   F A 3  !'   C 4
      !'           !'           !'           !'
    A0, B0      A1, B1      A2, B2      A3, B3

```

Each stage produces a carry for the next stage.

This is a:

$$\text{Ripple Carry Adder}$$

4.7 Ripple Carry Adder — RCA

In a Ripple Carry Adder, carry propagates from the least significant stage to higher stages.

Example:

TEXT

```

Bit 0 generates C1
C1 required by Bit 1
Bit 1 generates C2
C2 required by Bit 2
...

```

Carry "ripples" through the circuit.

Main Problem

It is slow for large bit-widths.

Why?

The most significant bit may have to wait for every earlier carry.

Example:

TEXT

$c_0, c_1, c_2, c_3, \dots, c_n$

The delay grows approximately with number of bits.

GATE Idea

For an n -bit ripple-carry adder:

$\boxed{\text{Carry propagation delay grows roughly linearly with } n}$

Exact numerical delay depends on the gate delays specified in the question.

4.8 Carry Propagation Example

Add:

TEXT

1111
0001

Rightmost bit:

TEXT

$1 + 1 = 0, \text{ carry } 1$

Carry then moves through every higher position.

Result:

TEXT

1 0000

This is a worst-case carry ripple pattern.

4.9 Carry Look-Ahead Adder — CLA

Important concept.

To avoid waiting for carry to ripple through every stage, we try to calculate carries faster.

This gives the:

$\boxed{\text{Carry Look-Ahead Adder}}$

It uses two concepts:

- Generate
- Propagate

4.10 Generate and Propagate

For bit i :

Generate

The bit pair generates a carry regardless of input carry.

A common definition:

$$\boxed{G_i = A_i B_i}$$

If:

TEXT

$$\begin{aligned} A_i &= 1 \\ B_i &= 1 \end{aligned}$$

a carry is definitely generated.

Propagate

The bit pair propagates an incoming carry.

Common definition:

$$\boxed{P_i = A_i \oplus B_i}$$

Some textbooks use $(A_i + B_i)$ for propagate. Use the definition given in the question.

Then:

$$\boxed{C_{i+1} = G_i + P_i C_i}$$

4.11 Carry Look-Ahead Expansion

For first stage:

$$\boxed{C_1 = G_0 + P_0 C_0}$$

Next:

$$\boxed{C_2 = G_1 + P_1 C_1}$$

Substitute (C_1) :

$$\boxed{C_2 = G_1 + P_1 (G_0 + P_0 C_0)}$$

$$\boxed{C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0}$$

Similarly:

$$\boxed{C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0}$$

The important idea:

Carries can be computed using logic expressions without waiting sequentially for every previous full adder.

4.12 RCA vs CLA

Feature	Ripple Carry	Carry Look-Ahead
Hardware	Simple	More complex
Cost	Lower	Higher
Speed	Slower	Faster
Carry generation	Sequential	Parallel/look-ahead
Large adders	Poorer performance	Better

Memory trick:

Ripple waits. Look-ahead predicts.

4.13 Binary Subtraction

ALU does not need a completely separate complicated subtractor.

We use:

$$[A - B = A + (-B)]$$

In 2's complement:

$$[-B = \overline{B} + 1]$$

Therefore:

$$\boxed{A - B = A + \overline{B} + 1}$$

Example

Calculate:

$$[7 - 3]$$

4-bit:

TEXT

$$7 = 0111$$

$$3 = 0011$$

Complement B:

TEXT

$$1100$$

Add 1:

TEXT

$$1101$$

Now:

TEXT

```
  0111
+ 1101
-----
1 0100
```

Discard extra carry:

TEXT

0100

Answer:

$\boxed{4}$

4.14 Adder-Subtractor Circuit

A clever circuit can perform both addition and subtraction.

Use a control bit M .

For each bit of B :

$B_i' = B_i \oplus M$

and initial carry:

$C_0 = M$

When $M = 0$

$B_i \oplus 0 = B_i$

and:

$C_0 = 0$

So:

$A + B$

When $M = 1$

$B_i \oplus 1 = \overline{B_i}$

and:

$C_0 = 1$

So:

$A + \overline{B} + 1$

which is:

$A - B$

Therefore:

$\boxed{M=0 \rightarrow \text{Add}}$

$\boxed{M=1 \rightarrow \text{Subtract}}$

Memory Trick

Subtract = flip B + start with carry 1

4.15 Increment and Decrement

Increment:

$A+1$

Decrement:

$A-1$

ALUs often support these directly.

Example:

TEXT

0111 + 1 = 1000

4.16 Carry Flag

The Carry Flag indicates a carry out of the most significant position.

Example:

4-bit unsigned:

TEXT

1111
0001

1 0000

The 4-bit result is:

TEXT

0000

carry-out:

TEXT

1

So:

$\boxed{CF=1}$

Carry Interpretation

Carry is particularly useful for **unsigned arithmetic**.

For 4-bit unsigned:

$$\lfloor 15+1=16 \rfloor$$

but maximum representable value is:

$$\lfloor 15 \rfloor$$

The carry indicates result exceeded 4-bit capacity.

4.17 Signed Overflow

~~Extremely~~ extremely important.

For signed 2's complement numbers, **carry out and overflow are different**.

Overflow occurs when mathematical result lies outside signed representable range.

For n bits:

$$\lfloor -2^{n-1} \leq x \leq 2^{n-1}-1 \rfloor$$

Addition Overflow Rule

Overflow can occur only when adding numbers with the same sign.

Positive + Positive != Negative

Overflow.

Negative + Negative != Positive

Overflow.

Positive + Negative

No signed addition overflow.

Example

4-bit signed range:

$$\lfloor -8 \text{ to } 7 \rfloor$$

Calculate:

$$\lfloor 5+4=9 \rfloor$$

Binary:

TEXT

0101

0100

```
----  
1001
```

1001 is negative in 4-bit 2's complement.

So:

positive + positive produced negative

Therefore:

$\boxed{\text{Overflow}=1}$

4.18 Another Overflow Example

$-5 + (-4) = -9$

But 4-bit minimum is:

-8

Representations:

```
TEXT  
-5 = 1011  
-4 = 1100
```

Add:

```
TEXT  
  1011  
+ 1100  
-----  
1 0111
```

4-bit result:

```
TEXT  
0111 = +7
```

Negative + negative produced positive.

Therefore overflow.

4.19 Carry and Overflow Example

4-bit:

```
TEXT  
1111 + 0001
```

Unsigned interpretation:

$15 + 1 = 16$

carry occurs.

Signed interpretation:

TEXT

1111 = -1

0001 = +1

$\lceil -1+1=0 \rceil$

No signed overflow.

So:

$\lceil \boxed{\text{Carry}=1, \text{Overflow}=0} \rceil$

This is a classic GATE concept.

4.20 Overflow Detection Using Carries

For 2's complement addition:

$\lceil \boxed{\text{Overflow} = C_{\text{in to MSB}} \oplus C_{\text{out of MSB}}} \rceil$

That is:

Overflow occurs when carry into sign bit differs from carry out of sign bit.

Important formula.

4.21 Overflow Detection Using Sign Bits

Let:

- (A_s) = sign bit of A
- (B_s) = sign bit of B
- (S_s) = sign bit of result

For addition:

$\lceil \boxed{V = \overline{A_s} \oplus \overline{B_s} \oplus S_s + A_s B_s \overline{S_s}} \rceil$

You usually don't need to memorize the Boolean expression if you remember:

same sign inputs + different sign result = overflow.

4.22 Zero Flag

Zero flag becomes 1 if ALU result is all zeros.

Example:

$\lceil 5-5=0 \rceil$

Therefore:

$\boxed{ZF=1}$

If result is nonzero:

$\boxed{ZF=0}$

4.23 Sign / Negative Flag

Usually the sign flag copies the most significant bit of the result.

For two's complement:

TEXT

```
MSB = 0 !' non-negative
MSB = 1 !' negative
```

So:

$\boxed{SF=MSB(Result)}$

4.24 Overflow Flag

Set when signed arithmetic result cannot be represented.

Example:

TEXT

```
0111 + 0001
```

4-bit signed:

$\boxed{7+1=8}$

8 cannot be represented.

Result:

TEXT

```
1000
```

Therefore:

$\boxed{OF=1}$

4.25 Common ALU Flags

Flag	Meaning
Z / ZF	Result is zero
C / CF	Carry out
S / N	Sign / negative

Flag	Meaning
V / OF	Signed overflow

Do not assume all architectures use identical names.

4.26 Logical Operations

The ALU also performs bitwise logical operations.

Suppose:

TEXT

A = 1010

B = 1100

AND

TEXT

1010

1100

1000

$\boxed{A \text{ AND } B = 1000}$

OR

TEXT

1010

1100

1110

$\boxed{A \text{ OR } B = 1110}$

XOR

TEXT

1010

1100

0110

$\boxed{A \text{ XOR } B = 0110}$

NOT A

TEXT

A = 1010
NOT A = 0101

4.27 Bitwise vs Logical Meaning

At machine level, operations commonly happen bit-by-bit.

Example:

TEXT

1010 AND 1100 = 1000

Each bit position operates independently.

4.28 Multiplexer in ALU

How can one ALU support many operations?

Suppose hardware computes:

TEXT

A + B
A AND B
A OR B
A XOR B

Then a **Multiplexer (MUX)** selects one result.

Simplified:

TEXT

```

                                A D D % % % % %
                                A N D % % % % % $
A , B % % % % % % % % O R % % % % % % < ! ' M U X ! ' A L U R e s u l t
                                X O R % % % % %
                                ! '
                        Select bits
```

The control unit supplies select bits.

4.29 Operation Select Bits

Suppose ALU has four operations:

TEXT

0 0 ! ' A N D
0 1 ! ' O R
1 0 ! ' A D D
1 1 ! ' S U B

Two select bits are enough because:

$$\lceil 2^2=4 \rceil$$

General rule:

For N independently selectable operations:

$$\lceil \log_2 N \rceil$$

select bits are required.

GATE Example

ALU supports 13 functions.

Minimum control bits:

Need:

$$2^3=8 < 13$$

$$2^4=16 \geq 13$$

Therefore:

$$4$$

4.30 Shift Operations

ALU/datapath may support shift operations.

Two major types:

1. Logical shift
2. Arithmetic shift

4.31 Logical Left Shift

Example:

TEXT
00110110

shift left by one:

TEXT
01101100

A zero normally enters from right.

TEXT
00110110
! "
01101100

For unsigned numbers, shifting left by 1 often corresponds to multiplication by 2 **if no significant bit is lost**.

4.32 Logical Right Shift

Example:

```
TEXT
10110010
```

logical right shift:

```
TEXT
01011001
```

A zero enters at left.

For unsigned numbers:

right shift by 1 roughly corresponds to division by 2, discarding remainder.

4.33 Arithmetic Right Shift

For signed two's-complement numbers, we often want to preserve sign.

Example:

```
TEXT
11001000
```

Arithmetic right shift:

```
TEXT
11100100
```

Instead of inserting 0, we replicate the sign bit.

This is called:

$\boxed{\text{Sign Extension}}$

Logical vs Arithmetic Right Shift

Suppose:

```
TEXT
A = 10110000
```

Logical right:

TEXT

01011000

Arithmetic right:

TEXT

11011000

Difference is the new MSB.

4.34 Arithmetic Left Shift

Arithmetic left shift is often bitwise the same hardware operation as logical left shift:

TEXT

discard MSB
shift left
insert 0 at LSB

But signed overflow may occur.

Example:

4-bit:

TEXT

0100 = +4

left shift:

TEXT

1000 = -8

Mathematically $(4 \times 2 = 8)$, but +8 is not representable.

So overflow occurred.

4.35 Shift by k Positions

Unsigned logical left shift by k :

$\lfloor \boxed{x \times 2^k} \rfloor$

if no overflow/lost high bits.

Logical right:

$\lfloor \boxed{\left\lfloor \frac{x}{2^k} \right\rfloor} \rfloor$

for unsigned values.

4.36 Barrel Shifter — Support Concept

[LOW PRIORITY / PLACEMENT]

A barrel shifter can shift by multiple positions in one combinational operation.

Example:

TEXT

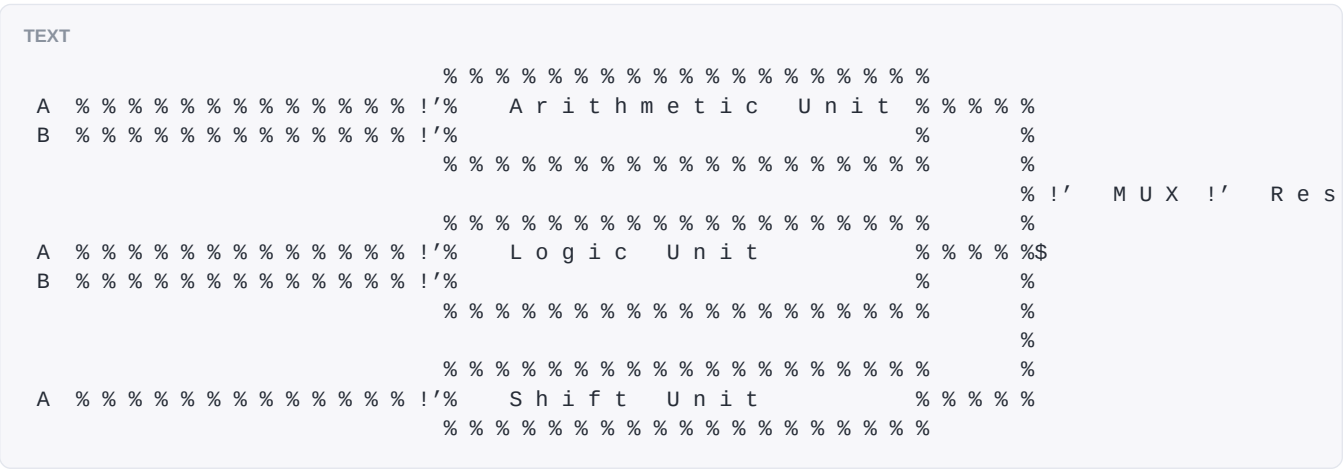
shift by 0,1,2,3,...

without repeatedly performing one-bit shifts.

Understand the concept; detailed design is not necessary for your listed GATE syllabus.

4.37 ALU Block Structure

A simplified ALU might contain:



Control bits choose:

- arithmetic vs logic vs shift,
- exact operation.

4.38 One-Bit ALU Slice

Large ALUs can be constructed from repeated one-bit slices.

Each slice handles:

TEXT

A_i
B_i
Carry-in
operation controls

and produces:

TEXT

Result_i
Carry-out

Then several slices form:

- 4-bit ALU,
- 8-bit ALU,
- 32-bit ALU,
- etc.

4.39 Comparison Using Subtraction

How does an ALU compare numbers?

One common method:

$\lfloor A - B \rfloor$

Then inspect:

- zero flag,
- sign,
- overflow-related conditions.

For equality:

$\lfloor A = B \rfloor$

iff:

$\lfloor A - B = 0 \rfloor$

so:

$\lfloor \boxed{ZF=1} \rfloor$

This is why status flags matter for branch instructions.

4.40 GATE Example — Half Adder

For:

TEXT

A = 1
B = 1

Half adder:

$\lfloor \text{Sum} = 1 \oplus 1 = 0 \rfloor$

$\lfloor \text{Carry} = 1 \cdot 1 = 1 \rfloor$

Answer:

TEXT

Sum = 0
Carry = 1

4.41 GATE Example — Full Adder

Given:

TEXT

```
A = 1
B = 0
Cin = 1
```

Sum:

$1 + 0 + 1 = 0$

Carry:

Two ones are being added, therefore carry = 1.

Answer:

$\boxed{S=0, C_{out}=1}$

4.42 GATE Example — Signed Overflow

4-bit:

TEXT

```
0110
0011
----
1001
```

Interpret:

$6 + 3 = 9$

But signed range:

$-8 \text{ to } 7$

So overflow.

Shortcut:

TEXT

positive + positive != negative

Therefore:

$\boxed{OF=1}$

4.43 GATE Example — No Signed Overflow

TEXT

```
1101
0010
----
1111
```

Interpret:

TEXT

```
1101 = -3
0010 = +2
1111 = -1
```

$\lfloor -3+2=-1 \rfloor$

Different-sign inputs.

Therefore no signed overflow.

$\lfloor \boxed{OF=0} \rfloor$

4.44 GATE Example — Carry vs Overflow

4-bit:

TEXT

```
1000
1000
----
1 0000
```

Unsigned:

$\lfloor 8+8=16 \rfloor$

carry = 1.

Signed:

TEXT

```
1000 = -8
1000 = -8
```

$\lfloor -8+(-8)=-16 \rfloor$

not representable.

Result is 0000 (positive), so overflow = 1.

Therefore:

$\lfloor \boxed{Carry=1, \text{ Overflow}=1} \rfloor$

Both can occur.

4.45 Another Carry vs Overflow Case

TEXT

```
0111
0001
----
1000
```

There is no final carry-out.

So:

$$\boxed{\text{Carry}=0}$$

But:

$$\boxed{7+1=8}$$

not representable as 4-bit signed.

So:

$$\boxed{\text{Overflow}=1}$$

Therefore:

$$\boxed{\text{Carry}=0, \text{Overflow}=1}$$

$\emptyset \neq \mathbb{N}$ proves they are independent concepts.

4.46 GATE Numerical — Control Bits

An ALU supports:

- 8 arithmetic operations
- 4 logic operations
- 4 shift operations

Total selectable functions:

$$\boxed{8+4+4=16}$$

Minimum select bits:

$$\boxed{\log_2 16=4}$$

$$\boxed{4}$$

assuming one encoded control field selects among all 16.

4.47 Ripple Carry Delay — Generic Method

Suppose each full-adder carry propagation takes:

$$\boxed{t_c}$$

For an n -bit ripple chain, worst-case carry delay is approximately:

$$\boxed{nt_c}$$

if the question models one carry-stage delay per bit.

If final sum needs extra delay, add the specified sum-gate delay.

Always use the exact delay model given in the question.

Example

A 4-bit ripple adder has:

$$\boxed{t_c=2\text{ns}}$$

Worst carry propagation:

$$4 \times 2 = 8 \text{ ns}$$

$$\boxed{8 \text{ ns}}$$

under that simplified model.

4.48 GATE Trap — Sum Delay vs Carry Delay

Some questions specify separate delays for:

- generating carry,
- generating sum.

Do not simply multiply all gate delays by number of bits.

Trace the **critical path** — the longest signal path that determines when output becomes valid.

4.49 Critical Path

Critical Path means:

the longest delay path through a combinational circuit.

The overall circuit delay is determined by this longest path.

For ripple carry adder, it usually involves carry propagating through many stages.

This is why CLA improves performance.

4.50 Generate/Propagate Example

Suppose:

TEXT

$$\begin{aligned} A_i &= 1 \\ B_i &= 1 \end{aligned}$$

Then:

$$G_i = 1$$

So the stage generates a carry no matter what C_i is.

Suppose:

TEXT

$$\begin{aligned} A_i &= 1 \\ B_i &= 0 \end{aligned}$$

Then:

$$P_i = 1$$

With incoming carry 1, the stage propagates it.

4.51 Subtraction Overflow Rule

For:

$\lfloor A-B \rfloor$

signed overflow can occur when:

A and B have different signs, and the result sign differs from A.

Examples:

Positive " Negative !' Negative

overflow.

Negative " Positive !' Positive

overflow.

Example

4-bit:

$\lfloor 7-(-2)=9 \rfloor$

not representable.

So overflow.

Memory Trick

Addition:

Same signs can overflow.

Subtraction:

Different signs can overflow.

4.52 Logical Shift GATE Trap

Consider:

TEXT

10000000

logical right shift:

TEXT

01000000

Arithmetic right shift:

TEXT

11000000

Do not confuse them when MSB = 1.

4.53 Arithmetic Right Shift and Negative Numbers

Example 8-bit two's complement:

TEXT

11111000

This is:

$\lfloor -8 \rfloor$

Arithmetic right shift:

TEXT

11111100

which represents:

$\lfloor -4 \rfloor$

So it approximately divides a signed value by 2 while preserving sign, subject to rounding conventions for odd negatives.

4.54 Flags After Logical Operations

Whether carry/overflow flags change after logical operations depends on ISA.

GATE questions will normally specify relevant behavior.

Do not assume every processor updates every flag for every instruction.

4.55 ALU vs Control Unit

A common conceptual confusion:

ALU

performs operation.

Control Unit

selects and coordinates operation.

Example:

Instruction:

```
TEXT
SUB R1, R2
```

Control unit says:

select subtract operation.

ALU performs:

$\lceil R1 - R2 \rceil$

4.56 ALU and Datapath

ALU is part of the processor's **datapath**.

Simplified:

```
TEXT
Registers
    !"
    !"
    ALU
    !"
Result
    !"
Registers
```

Control unit provides:

```
TEXT
Register select
ALU operation select
Write enable
```

4.57 High-Yield Formula Sheet

Half Adder

$$\boxed{S = A \oplus B}$$

$$\boxed{C = AB}$$

Full Adder

$$\boxed{S = A \oplus B \oplus C_{in}}$$

$$\boxed{C_{out} = AB + AC_{in} + BC_{in}}$$

Carry Look-Ahead

$$\boxed{G_i = A_i B_i}$$

$$\boxed{P_i = A_i \oplus B_i}$$

$$\boxed{C_{i+1} = G_i + P_i C_i}$$

Subtraction

$$\boxed{A - B = A + \overline{B + 1}}$$

Signed Overflow

$$\boxed{V = C_{in,MSB} \oplus C_{out,MSB}}$$

4.58 GATE FAST RULES

Rule 1 — Addition overflow

TEXT

Same sign inputs
+
Different sign result
=
Overflow

Rule 2 — Subtraction overflow

TEXT

Different sign operands
+
Result sign differs from A
=
Overflow

Rule 3 — Half Adder

TEXT

S U M !' X O R
C A R R Y !' A N D

Rule 4 — Adder/Subtractor

TEXT

M = 0 !' A d d
M = 1 !' C o m p l e m e n t B + C a r r y 1 !' S u b t r a c t

Rule 5 — Arithmetic Right Shift

4.59 COMMON GATE TRAPS

Trap 1 — Carry = Overflow

'LWrong.

Carry is not equivalent to signed overflow.

Trap 2 — Half Adder accepts carry-in

'LWrong.

Full adder handles carry-in.

Trap 3 — Right shift always inserts 0

'LWrong.

Arithmetic right shift replicates the sign bit.

Trap 4 — Left shift always doubles correctly

'LWrong.

Only if no important high bit is lost and representation remains valid.

Trap 5 — CLA eliminates all delay

'LWrong.

CLA reduces carry propagation delay but still has gate delays.

Trap 6 — (P_i) always defined identically

Different books/questions may define propagate as:

$A_i \oplus B_i$

or:

$A_i + B_i$

Use the definition in the problem.

Trap 7 — Final carry tells signed correctness

'LWrong.

For signed arithmetic, inspect overflow.

CHAPTER 4 — CHAPTER SNAPSHOT

1. ALU performs arithmetic, logic and shifts.
 2. Half adder adds two bits.
 3. Half-adder sum = XOR.
 4. Half-adder carry = AND.
 5. Full adder adds A, B and carry-in.
 6. Ripple carry is simple but slower.
 7. CLA computes carries using generate and propagate logic.
 8. Subtraction is implemented using 2's complement.
 9. $(A-B=A+\overline{B}+1)$.
 10. Carry mainly matters for unsigned arithmetic.
 11. Overflow matters for signed arithmetic.
 12. Same-sign addition producing opposite-sign result means overflow.
 13. Zero flag indicates all-zero result.
 14. Sign flag generally equals result MSB.
 15. ALU operation can be selected using multiplexers/control bits.
 16. Logical shift fills with zero.
 17. Arithmetic right shift preserves sign.
 18. Comparison can be implemented using subtraction and flags.
-

MUST REMEMBER

$\boxed{\text{Half Adder: } S=A \oplus B, C=AB}$

$\boxed{\text{Full Adder: } S=A \oplus B \oplus C_{in}}$

$\boxed{A-B=A+\overline{B}+1}$

$\boxed{C_{i+1}=G_i + P_i C_i}$

$\boxed{\text{Overflow}=C_{in,MSB} \oplus C_{out,MSB}}$

PRACTICE SET

Q1 — L0 MCQ

Half-adder sum output is:

A. AND B. OR C. XOR D. NOT

Q2 — L0 MCQ

A full adder has how many input bits?

A. 1 B. 2 C. 3 D. 4

Q3 — L1

For:

TEXT

A = 1
B = 1
Cin = 0

Find:

- Sum
- Carry-out

Q4 — L1 MCQ

Ripple carry adder is relatively slow mainly because:

A. XOR cannot perform addition B. carry must propagate through successive stages C. registers are too slow D. it cannot add signed numbers

Q5 — L1

In an adder-subtractor using XOR gates on every B bit, what value of mode input normally selects subtraction?

Q6 — L1

Find 4-bit result and carry:

TEXT

1111 + 0001

Q7 — L2

For 4-bit signed arithmetic:

TEXT

0111 + 0001

Find:

- result
- carry flag
- signed overflow flag

Q8 — L2

For 4-bit signed arithmetic:

TEXT

1111 + 0001

Find:

- signed decimal operands
- result
- carry
- overflow

Q9 — L2

Given:

$\{ G_0=1, \text{quad } P_0=0, \text{quad } C_0=1 \}$

find:

$\{ C_1 \}$

using:

$\{ C_1=G_0+P_0C_0 \}$

Q10 — L2

Logical right shift:

TEXT

10110100

by one bit.

Find result.

Q11 — L2

Arithmetic right shift:

TEXT

10110100

by one bit.

Find result.

Q12 — L2 NAT

An ALU supports 25 selectable operations.

Minimum number of operation-selection bits required = ?

Q13 — L3

A 4-bit ripple-carry adder has a carry propagation delay of 3 ns per stage.

Using the simplified stage-delay model, what is the worst-case delay for carry to propagate through all four stages?

Q14 — L3

Determine whether signed overflow occurs in each 4-bit operation:

TEXT

- (a) 0101 + 0010
 - (b) 0101 + 0100
 - (c) 1011 + 1101
 - (d) 1011 + 0011
-

CHAPTER 4 COMPLETION STATUS

' ALU purpose ' Arithmetic operations ' Half adder ' Full adder ' Ripple Carry Adder ' Carry propagation ' Carry Look-Ahead Adder ' Generate/Propagate ' Binary subtraction ' Adder-subtractor circuit ' Increment/decrement ' Carry ' Signed overflow ' Overflow detection ' Logical operations ' Multiplexer-based ALU selection ' Shift operations ' Arithmetic vs logical shift ' Flags ' Comparison via ALU ' GATE traps and numericals

CHAPTER 4: COMPLETE '

COURSE PROGRESS

Completed: 5 / 26 modules

Approximate progress:

$\sim 19\%$

You have now covered:

' Foundations ' Basic Computer Organization ' Instruction Set ' Addressing Modes ' ALU Design

NEXT !' CHAPTER 5 — CONTROL UNIT

Next we cover the complete official syllabus block:

~~Hardwired Control Unit~~ ~~Microprogrammed Control Unit~~

including:

- control signals,
 - micro-operations,
 - instruction sequencing,
 - hardwired control design,
 - control memory,
-

- microinstruction,
- microprogram,
- control word,
- microprogram sequencer,
- horizontal vs vertical microprogramming,
- hardwired vs microprogrammed comparison,
- microinstruction-format numericals and GATE traps.

USER

next

ASSISTANT

CHAPTER 5 — CONTROL UNIT

Classification: [GATE CORE] **Priority:** ~~Ø=VERY~~ HIGH **Official syllabus coverage:** Hardwired and Microprogrammed Control Unit

The Control Unit is one of the most important conceptual parts of COA because it explains:

Who tells the registers, ALU and memory what to do and when?

5.1 What is a Control Unit?

The **Control Unit (CU)** coordinates all operations inside the processor.

Suppose CPU executes:

TEXT

ADD R1, R2

The ALU knows how to add.

Registers know how to store data.

But somebody must tell the hardware:

TEXT

Read R1
Read R2
Select ADD operation
Send operands to ALU
Store result back into R1

That is the job of the:

$\boxed{\text{Control Unit}}$

5.2 Simple CPU View

TEXT

```

Instruction !' % Control Unit %
% % % % % % % %, % % % % % % % %
% Control signals
% % % % % % % % %4% % % % % % % % %
!" !"
Registers ALU
!• !•
Datapath

```

Memory trick:

Control Unit decides. Datapath performs.

5.3 What is a Control Signal?

A **control signal** is a binary signal telling hardware to perform a specific operation.

Examples:

TEXT

```
R1_out = 1
R2_in = 1
MemoryRead = 1
MemoryWrite = 1
ALU_ADD = 1
PC_in = 1
IR_in = 1
```

A signal is commonly:

TEXT

```
0  !'   i n a c t i v e
1  !'   a c t i v e
```

depending on the design.

5.4 Example of Control Signals

Suppose we want:

$$\forall R1 \backslash \text{leftarrow} R2 \ \backslash$$

Hardware might require:

TEXT

```
R2_out = 1
R1_in  = 1
```

Meaning:

1. Put R2 value onto internal bus.
2. Allow R1 to load that value.

5.5 Micro-operation

We introduced this earlier.

A **micro-operation** is a simple operation performed on data stored in registers.

Examples:

$[R1] \rightarrow R2$

$PC \rightarrow PC+1$

$MAR \rightarrow PC$

$[R1] \rightarrow R1+R2$

Each machine instruction is ultimately broken into several micro-operations.

5.6 Instruction vs Micro-operation

Consider:

```
TEXT
LOAD R1, X
```

This is one **machine instruction**.

Internally, it may require several micro-operations:

```
TEXT
MAR !• address X
Memory Read
MDR !• M[MAR]
R1 !• MDR
```

So:

One machine instruction != multiple internal micro-operations.

5.7 Timing Signals

Micro-operations must happen in the proper sequence.

For example:

```
TEXT
T0 : MAR !• PC
T1 : MDR !• M[MAR]
T2 : IR !• MDR
T3: Decode instruction
```

Here:

TEXT

T0, T1, T2, T3

represent timing steps.

The CU combines:

- instruction information,
- timing,
- status conditions,

to generate control signals.

5.8 Inputs to the Control Unit

Conceptually:

TEXT

```
Opcode
Status Flags
Timing / Clock
Interrupt signals
    !"
    Control Unit
    !"
Control signals
```

The CU may use:

1. Opcode from IR
2. Clock/timing
3. ALU flags
4. Interrupt signals
5. Current control state

5.9 Two Main Types of Control Unit

This is the core official syllabus.

$$\boxed{\text{Control Unit} \begin{cases} \text{Hardwired} \\ \text{Microprogrammed} \end{cases}}$$

We will study both carefully.

PART A — HARDWIRED CONTROL UNIT

5.10 What is Hardwired Control?

In a **Hardwired Control Unit**, control signals are generated using fixed hardware logic.

This can include:

- decoders,
- logic gates,
- flip-flops,
- counters,
- finite-state logic.

There is no control program stored in a special control memory.

Intuition

Think of a physical electrical circuit.

Input:

TEXT

Opcode + Timing + Flags

goes through logic circuitry.

Output:

TEXT

Control signals

Diagram

TEXT

```

O p c o d e % % % % % % %
T i m i n g % % % % % % % < % % ! '   C o m b i n a t i o n a l /
F l a g s   % % % % % % % % $           S e q u e n t i a l   L o g i c % % ! '   C o n t r o l   S i g n a l s
S t a t e   % % % % % % % %
  
```

5.11 Hardwired Example

Suppose instruction opcode corresponds to:

TEXT

ADD

At a certain clock step:

TEXT

T4

control logic may activate:

TEXT

R1_out
R2_out
ALU_ADD
R3_in

These are generated directly by hardware conditions.

5.12 Basic Hardwired Components

A hardwired CU may use:

Instruction Decoder

Decodes opcode.

Example:

TEXT

```
0 p c o d e   0 0 0   !'   A D D
0 p c o d e   0 0 1   !'   S U B
0 p c o d e   0 1 0   !'   L O A D
```

Timing Generator

Produces timing steps:

TEXT

T0
T1
T2
T3
...

Control Logic

Combines:

TEXT

```
decoded instruction
+
timing
+
flags
```

to generate control signals.

5.13 Example Control Function

Suppose:

- `D_ADD` = ADD instruction decoded

- T4 = timing step 4

Then perhaps:

$[ALU_ADD = D_ADD] \cdot T_4]$

Meaning:

Enable ADD control only when instruction is ADD and current step is T4.

Real processors are more complicated, but this is the key idea.

5.14 Hardwired Control Sequence

Simplified:

TEXT

```
Instruction fetched
      !"
Opcode decoded
      !"
Timing state determined
      !"
Logic generates control signals
      !"
Datapath performs micro-operation
```

5.15 Advantages of Hardwired Control

1. Fast

Control signals are produced directly by hardware.

No control-memory lookup is generally required.

2. Good for simpler instruction sets

Works well when control behavior is relatively regular.

5.16 Disadvantages of Hardwired Control

1. Difficult to modify

Changing instruction behavior may require redesigning hardware.

2. Complex for large instruction sets

More instructions and complicated control sequences produce complicated logic.

3. Less flexible

Compared with microprogrammed control.

TEXT

```
Microinstruction 0  
Microinstruction 1  
Microinstruction 2  
...
```

Each microinstruction controls one or more micro-operations.

5.19 Microinstruction

A **microinstruction** is a control word stored in control memory.

It specifies things such as:

- which registers output data,
- which register accepts data,
- which ALU operation executes,
- whether memory is read/written,
- where next microinstruction comes from.

Example conceptually:

TEXT

```
R1_out = 1  
R2_out = 1  
ALU_ADD = 1  
R3_in = 1
```

could be encoded in one microinstruction.

5.20 Microprogram

Suppose machine instruction:

TEXT

```
ADD R1, R2
```

requires:

TEXT

```
μ1: Read R1 and R2  
μ2: ALU performs addition  
μ3: Store result
```

This sequence:

TEXT

```
μ 1 !' μ 2 !' μ 3
```

is the microprogram for the machine instruction.

Important Hierarchy

TEXT

```
Program
  !"
Machine Instructions
  !"
Microprograms
  !"
Microinstructions
  !"
Control Signals
  !"
Micro-operations
```

Very useful conceptual chain.

5.21 Control Word

A **Control Word** is the bit pattern representing control signals.

Suppose CPU has four control signals:

TEXT

```
C1 C2 C3 C4
```

A control word:

TEXT

```
1010
```

might mean:

TEXT

```
C1 active
C2 inactive
C3 active
C4 inactive
```

5.22 Control Address Register — CAR

The Control Address Register stores:

address of the next/current microinstruction in control memory.

It is conceptually similar to the Program Counter, but for microinstructions.

So:

TEXT

```
P C      !'   m a c h i n e   i n s t r u c t i o n   a d d r e s s
C A R    !'   m i c r o i n s t r u c t i o n   a d d r e s s
```

Memory trick:

CAR drives through Control Memory.

5.23 Microinstruction Register

Sometimes called:

- Control Buffer Register
- Microinstruction Register

It stores the microinstruction currently read from control memory.

Conceptually:

```
TEXT
CAR
!"
Control Memory
!"
Microinstruction Register
!"
Control signals
```

5.24 Microprogram Sequencer

The **Microprogram Sequencer** decides which microinstruction should execute next.

Possibilities:

1. Next sequential microinstruction
2. Branch to another microinstruction
3. Jump to start of another microprogram

Basic Flow

```
TEXT
CAR
!"
Control Memory
!"
Current Microinstruction
!"
Control Signals
!"
Next-address logic
!"
CAR updated
```

5.25 Microinstruction Format

A general microinstruction may contain:

TEXT

[illegible]

Sometimes also:

TEXT

Condition field

So:

TEXT

Control	Condition	Next Address
---------	-----------	--------------

5.26 Horizontal Microprogramming

Ø=important.

In **Horizontal Microprogramming**, control signals are represented more directly.

Example:

TEXT

C1 C2 C3 C4 C5 C6 C7 C8 ...

Each bit may correspond directly to a control line.

Example

Suppose:

TEXT

```
C1 = R1_out
C2 = R2_out
C3 = ALU_ADD
C4 = R3_in
```

Microinstruction:

TEXT

1111

activates all four simultaneously.

Characteristics of Horizontal Microprogramming

- Wide microinstructions

- Little encoding
- High parallelism
- Fast control signal generation
- Large control memory requirement

Memory Trick

Horizontal = Huge and highly parallel.

5.27 Vertical Microprogramming

In **Vertical Microprogramming**, control signals are encoded.

Instead of one bit per control line, fields specify selected operations.

Example:

TEXT

```
ALU field = 010
Register source = 001
Destination = 101
```

Decoder converts encoded fields into actual control signals.

Characteristics

- Narrower microinstructions
- More encoding
- Less control-memory size
- Requires decoding
- Lower parallelism generally

Memory Trick

Vertical = Very encoded.

5.28 Horizontal vs Vertical

Feature	Horizontal	Vertical
Microinstruction width	Large	Small
Encoding	Little/no encoding	Highly encoded

Feature	Horizontal	Vertical
Parallel operations	More	Less
Control memory	Larger	Smaller
Decoder requirement	Less	More
Speed	Generally faster	Generally slower
Programming complexity	More direct	More compact

Memorize this.

5.29 Why Horizontal is Faster

Suppose a horizontal microinstruction directly contains:

```
TEXT
ALU_ADD = 1
R1_out = 1
R3_in = 1
```

Signals can be activated directly.

Vertical format might store:

```
TEXT
ALU = 010
SRC = 001
DST = 011
```

which needs decoding first.

Therefore:

more decoding != additional delay.

5.30 Why Vertical Saves Memory

Suppose 64 control signals exist.

Horizontal

Potentially around:

$\lceil 64 \text{ control bits} \rceil$

just for direct controls.

Fully encoded selection

If only one of 64 were active in some field:

$\lceil \log_2 64 = 6 \rceil$

bits can identify one choice.

This dramatically reduces microinstruction width.

5.31 Microinstruction Width Numerical

Suppose a horizontal microinstruction must directly control 80 independent control signals and also store a 10-bit next address.

Minimum width:

$$\lfloor 80+10=90 \rfloor$$

$$\lfloor \boxed{90 \text{ bits}} \rfloor$$

assuming no other fields.

5.32 Control Memory Size

Suppose:

- 256 microinstructions
- each microinstruction = 40 bits

Control memory capacity:

$$\lfloor 256 \times 40 \rfloor$$

$$\lfloor =10240 \text{ bits} \rfloor$$

or:

$$\lfloor 1280 \text{ bytes} \rfloor$$

because:

$$\lfloor 10240/8=1280 \rfloor$$

Formula

$$\lfloor \boxed{\text{Control Memory Size} = \text{Number of Microinstructions} \times \text{Microinstruction Width}} \rfloor$$

5.33 Address Bits for Control Memory

If control memory contains N words:

$$\lfloor \boxed{\lceil \log_2 N \rceil} \rfloor$$

address bits required.

Example:

1024 microinstructions.

$$\lfloor \log_2 1024=10 \rfloor$$

So CAR needs:

$\boxed{10 \text{ text{ bits}}}$

to address all entries.

5.34 GATE Numerical — Control Memory

A control memory contains:

2^{12}

microinstructions.

Each is 64 bits wide.

Total capacity:

$2^{12} \times 64$

Since:

$64 = 2^6$

$= 2^{18} \text{ text{ bits}}$

$= 256 \text{ Kb}$

or in bytes:

$2^{15} \text{ text{ bytes}} = 32 \text{ KB}$

Answer depends on requested unit.

5.35 Encoded Control Fields

Suppose there are:

- 16 possible source registers
- 8 possible destination registers
- 8 ALU operations

Bits needed:

Source:

$\log_2 16 = 4$

Destination:

$\log_2 8 = 3$

ALU:

$\log_2 8 = 3$

Total encoded bits:

$4 + 3 + 3 = 10$

Important GATE Trap

If fields are independent and can operate simultaneously:

$\boxed{\text{Add the bits for each field}}$

Do NOT simply add all choices first.

Example

There are:

- 8 source-register choices
- 8 destination-register choices

Need:

$3+3=6$

bits.

Not:

$\log_2(8+8)=4$

because source and destination must be selected independently.

5.36 Mutually Exclusive Control Signals

Suppose 8 control signals are mutually exclusive, meaning only one can be active at a time.

Instead of 8 direct bits, they can be encoded using:

$\lceil \log_2 8 \rceil = 3$

bits.

This is a major idea in vertical microprogramming.

5.37 Parallel Control Groups

Suppose control signals are divided into:

TEXT

Group A: 8 mutually exclusive signals
Group B: 4 mutually exclusive signals
Group C: 16 mutually exclusive signals

Each group can activate independently.

Bits:

$\log_2 8 = 3$

$\log_2 4 = 2$

$\log_2 16 = 4$

Total:

$\boxed{3+2+4=9}$

If a "no operation" choice is needed in each group, account for it:

Example Group A:

TEXT

8 signals + no signal = 9 choices

then:

$\lceil \log_2 9 \rceil = 4$

$2^4 = 16$ is a classic numerical trap.

5.38 Why "No Operation" May Need Encoding

Suppose group has 4 control signals.

If exactly one must always be active:

2^{bits}

is enough.

But if we also need:

none of them active

then choices become:

$4 + 1 = 5$

so:

$\lceil \log_2 5 \rceil = 3$

Always inspect wording.

5.39 Hardwired vs Microprogrammed Control

This table is $2^8 = 256$ as important.

Feature	Hardwired	Microprogrammed
Implementation	Logic circuitry	Control memory + microinstructions
Speed	Faster	Usually slower
Flexibility	Low	High
Modification	Difficult	Easier
Complex instruction handling	Harder	Easier
Control memory	Not central	Required
Design for complex ISA	More difficult	Convenient
Debug/changes	Difficult	Relatively easier

Memory Trick

Hardwired

HF

Hardware **F**ast

Microprogrammed

MF

Memory-based **F**lexible

5.40 Why Microprogrammed is Usually Slower

A microprogrammed CU generally has to:

1. obtain microinstruction from control memory,
2. decode it if encoded,
3. issue control signals.

Hardwired logic directly produces control signals.

Therefore:

$\boxed{\text{Hardwired generally faster}}$

5.41 Why Microprogrammed is Easier to Modify

If control behavior is stored as microcode, changing certain behaviors may involve modifying microprograms rather than redesigning logic circuitry.

Therefore:

$\boxed{\text{Microprogrammed generally more flexible}}$

5.42 RISC vs CISC Connection

Traditional tendency:

RISC

Often associated with:

TEXT

Hardwired control

because instructions are simpler and regular.

CISC

Historically often associated with:

TEXT

Microprogrammed control

because instructions can be complex.

But:

Do not treat this as an absolute law for every modern processor.

For GATE conceptual questions, it is a useful historical tendency.

5.43 Control Store

Another name often used for control memory is:

$\boxed{\text{Control Store}}$

So:

TEXT

C o n t r o l S t o r e "H C o n t r o l M e m o r y

in microprogrammed CU context.

5.44 Firmware Connection

Microcode is sometimes described as a kind of:

$\boxed{\text{Firmware}}$

because it sits between pure hardware and normal software.

[PLACEMENT/INTERVIEW]

Do not overfocus on this for GATE.

5.45 Horizontal Microinstruction Example

Suppose processor has 6 independently activatable control lines:

TEXT

C0 C1 C2 C3 C4 C5

A horizontal control field might be:

TEXT

101101

Active:

TEXT

C0
C2
C3
C5

Multiple signals can be activated simultaneously.

5.46 Vertical Microinstruction Example

Suppose 8 ALU operations:

TEXT

000 ADD
001 SUB
010 AND
011 OR
100 XOR
...

Only:

$\lfloor 3 \text{ bits} \rfloor$

are needed to encode the selected ALU operation.

A decoder converts the 3-bit code into control signals.

5.47 Horizontal Parallelism

Suppose one microinstruction can simultaneously perform:

TEXT

R1_out
R2_out
ALU_ADD
R3_in

Horizontal control makes such concurrent signals easy to specify.

This is called greater **micro-operation parallelism**.

5.48 Vertical Restriction

Highly encoded fields often mean:

only one operation from a field can be selected at once.

Therefore parallelism may be reduced.

5.49 Conditional Microprogram Branching

A microprogram may need decisions.

Example:

TEXT

```
If Zero Flag = 1
    go to microaddress 100
Else
    go to microaddress 101
```

Therefore microinstructions may include:

- condition-select field,
- branch/next-address field.

5.50 Next Address Generation

The next microinstruction may come from:

Sequential

$\text{[CAR} \leftarrow \text{CAR} + 1 \text{]}$

Branch

$\text{[CAR} \leftarrow \text{target]}$

Mapping from machine opcode

At the beginning of a new machine instruction:

TEXT

O p c o d e ! ' s t a r t i n g m i c r o p r o g r a m a d d r e s s

5.51 Mapping Logic

Suppose machine instruction is:

TEXT

MUL

Opcode mapping hardware identifies:

microprogram for MUL starts at control-memory address 400.

Then:

$\text{[CAR} \leftarrow 400 \text{]}$

Microinstructions for MUL execute from there.

5.52 Microprogram Execution Example

Machine instruction:

```
TEXT
LOAD R1, X
```

Possible microprogram:

```
TEXT
μ 0 :  M A R !•  X
μ1: Memory Read
μ 2 :  M D R !•  M [ M A R ]
μ 3 :  R 1 !•  M D R
μ4: End / fetch next instruction
```

Each line is one conceptual microinstruction or step.

5.53 Hardwired Execution of Same Instruction

Hardwired CU would instead generate equivalent control signals using:

```
TEXT
Opcode + timing states + logic circuitry
```

There is no stored microprogram sequence in control memory.

Same required operations; different method of control generation.

5.54 Important Distinction

Do not confuse:

Machine Instruction

Example:

```
TEXT
ADD R1, R2
```

visible to programmer/compiler.

Microinstruction

Internal control instruction used by microprogrammed CU.

Micro-operation

Actual small hardware operation, such as:

$[R1] \leftarrow [R2]$

Memory Trick

Instruction tells CPU what.

Microinstruction tells control hardware how.

Micro-operation is the actual tiny action.

5.55 GATE Example 1 — Type Identification

Which control unit is generally faster?

A. Microprogrammed B. Hardwired

Answer:

$\boxed{\text{Hardwired}}$

because control signals are generated directly by hardware logic.

5.56 GATE Example 2 — Flexibility

Which is easier to modify?

$\boxed{\text{Microprogrammed Control Unit}}$

because behavior is represented using microcode/control memory.

5.57 GATE Example 3 — CAR Bits

Control memory has:

$\boxed{4096}$

microinstructions.

Since:

$\boxed{4096=2^{12}}$

CAR width:

$\boxed{12 \text{ bits}}$

5.58 GATE Example 4 — Control Store Size

Control memory:

- 512 words
- 48 bits per word

Capacity:

512×48

$= 24576 \text{ bits}$

$\boxed{24576 \text{ bits}}$

or:

$3072 \text{ bytes} = 3 \text{ KB}$

5.59 GATE Example 5 — Encoded Fields

A microinstruction has to independently select:

- 32 source registers,
- 32 destination registers,
- 16 ALU operations.

Bits:

Source:

5

Destination:

5

ALU:

4

Total:

$\boxed{14}$

excluding other fields.

5.60 GATE Example 6 — Horizontal Width

Suppose:

- 50 independent control signals,
- 8-bit condition/sequence information,
- 12-bit next-address field.

Horizontal microinstruction width:

$50 + 8 + 12$

$\boxed{70 \text{ bits}}$

5.61 GATE Example 7 — Encoded Groups

Control signals are divided into:

- Group 1: 7 mutually exclusive signals
- Group 2: 15 mutually exclusive signals

Assume each group also permits no signal.

Group 1 choices:

$\lfloor 7+1=8 \rfloor$

bits:

$\lfloor 3 \rfloor$

Group 2 choices:

$\lfloor 15+1=16 \rfloor$

bits:

$\lfloor 4 \rfloor$

Total:

$\lfloor \boxed{7 \text{ bits}} \rfloor$

5.62 GATE Trap — Horizontal Is Not "One Operation Only"

Actually the opposite.

Horizontal microinstructions usually allow:

$\lfloor \boxed{\text{more simultaneous control signals}} \rfloor$

Therefore more parallelism.

5.63 GATE Trap — Vertical Is Not Faster Because It Is Shorter

Shorter microinstructions reduce memory requirements.

But they require decoding.

So:

smaller width does not automatically mean faster control.

Typically horizontal is faster.

5.64 GATE Trap — Hardwired Does Not Mean "No Control"

It still produces control signals.

Difference is only **how those signals are generated**.

5.65 GATE Trap — Microprogram "User Program"

Microprogram is internal processor control code.

A C/Python application is not a microprogram.

5.66 GATE Trap — Control Memory vs Main Memory

Control memory stores:

TEXT

microinstructions

Main memory stores:

TEXT

program instructions + data

They have different roles.

5.67 GATE Trap — PC vs CAR

PC

Points to:

TEXT

machine instruction

CAR

Points to:

TEXT

microinstruction

Do not confuse them.

5.68 GATE Trap — Number of Bits

Whenever a control field must represent N choices:

$$\lceil \log_2 N \rceil$$

But if "no selection" is possible:

$$\lceil \log_2 (N+1) \rceil$$

may be needed.

Read wording carefully.

5.69 Horizontal vs Vertical — Fast GATE Test

If question says:

wider control word, greater parallelism

Answer:

$\boxed{\text{Horizontal}}$

If question says:

highly encoded, compact control memory

Answer:

$\boxed{\text{Vertical}}$

If question says:

less decoding

Answer:

$\boxed{\text{Horizontal}}$

If question says:

lower memory requirement

Answer:

$\boxed{\text{Vertical}}$

5.70 Hardwired vs Microprogrammed — Fast GATE Test

If question says:

maximum speed

think:

$\boxed{\text{Hardwired}}$

If question says:

ease of modification

think:

$\boxed{\text{Microprogrammed}}$

If question says:

control memory

think:

$\boxed{\text{Microprogrammed}}$

If question says:

think:

$\boxed{\text{Hardwired}}$

5.71 Control Unit Complete Comparison

Property	Hardwired	Microprogrammed
Signal generation	Hardware logic	Microinstructions
Control memory	Usually not required	Required
Speed	High	Lower
Flexibility	Low	High
Modification	Hardware redesign	Microcode change easier
Complex ISA	Harder to design	Easier to implement
Control logic	Can become complex	Structured via microprogram
Typical historical association	RISC	CISC

CHAPTER 5 — FORMULA SHEET

Bits for N choices

$\boxed{\lceil \log_2 N \rceil}$

Control memory capacity

$\boxed{\text{Words} \times \text{Bits/Word}}$

CAR bits

For N microinstructions:

$\boxed{\lceil \log_2 N \rceil}$

Encoded independent fields

$\boxed{\text{Total bits} = \sum \lceil \log_2 (\text{choices in field}) \rceil}$

Include an extra "none" choice when required.

CHAPTER 5 — MEMORY TRICKS

Hardwired

Hard = Hardware = High speed

Microprogrammed

Micro = Memory = Modifiable

Horizontal

Horizontal = Huge + High parallelism

Vertical

Vertical = Very encoded

PC vs CAR

PC != Program

CAR != Control

CHAPTER 5 — MUST REMEMBER

$\boxed{\text{Hardwired != faster but less flexible}}$

$\boxed{\text{Microprogrammed != slower but more flexible}}$

$\boxed{\text{Horizontal != wide, parallel, less encoded}}$

$\boxed{\text{Vertical != narrow, encoded, less parallel}}$

And:

TEXT

Machine instruction

!"

Microprogram

!"

Microinstructions

!"

Control signals

!"

Micro-operations

CHAPTER SNAPSHOT

1. Control Unit coordinates processor operations.

2. Control signals activate datapath components.
 3. Machine instructions are implemented using micro-operations.
 4. Hardwired control generates signals using fixed hardware logic.
 5. Hardwired control is generally faster.
 6. Hardwired control is difficult to modify.
 7. Microprogrammed control stores control information in control memory.
 8. A microinstruction specifies control actions.
 9. A microprogram is a sequence of microinstructions.
 10. CAR identifies a microinstruction address.
 11. Microprogram sequencer determines the next microinstruction.
 12. Horizontal microcode uses wide, relatively unencoded words.
 13. Horizontal microcode allows more parallelism.
 14. Vertical microcode is encoded and compact.
 15. Vertical microcode requires more decoding.
 16. Control-memory size = number of words × word width.
 17. Independent encoded control fields require separate bit counts.
-

PRACTICE SET

Q1 — L0 MCQ

Which control unit is generally faster?

- A. Microprogrammed B. Hardwired C. Both always equal D. Cache controlled
-

Q2 — L0 MCQ

Microinstructions are stored in:

- A. General-purpose registers B. Cache only C. Control memory D. ALU
-

Q3 — L1 MCQ

Which is generally easier to modify?

- A. Hardwired control B. Microprogrammed control
-

Q4 — L1 MCQ

Which microprogramming style generally uses wider microinstructions?

- A. Vertical B. Horizontal C. Both identical D. Neither
-

Q5 — L1 NAT

A control memory contains 1024 microinstructions.

Minimum number of CAR bits = ?

Q6 — L1 NAT

A control memory contains:

- 512 words
- 64 bits/word

Find total capacity in bits.

Q7 — L2

A microinstruction must select one of 32 ALU operations.

Minimum bits needed = ?

Q8 — L2

A microinstruction independently selects:

- one of 16 source registers,
- one of 8 destination registers,
- one of 8 ALU operations.

Minimum encoded bits required for these fields?

Q9 — L2

There are 7 mutually exclusive control signals, and the field must also support "none selected."

How many encoded bits are required?

Q10 — L2 MSQ

Which are generally true for horizontal microprogramming?

A. Wider control word B. Greater parallelism C. More encoding D. Larger control-memory requirement

Q11 — L2 MSQ

Which are generally true for hardwired control?

A. High speed B. Easy modification C. Uses logic circuitry to generate control signals D. Usually more flexible than microprogrammed control

Q12 — L3 NAT

A control store has:

2^{11}

microinstructions, each 72 bits long.

Find total control-store size in **KB**, assuming 8 bits per byte and $(1\text{KB}=2^{10})$ bytes.

Q13 — L3

Three independent control groups contain:

- 8 signals,
- 4 signals,
- 15 signals.

Each group may also select "none."

Find minimum total encoded bits.

CHAPTER 5 COMPLETION STATUS

' Control Unit purpose ' Control signals ' Micro-operations ' Timing signals ' Hardwired control ' Instruction decoder ' Timing generator ' Hardwired advantages/disadvantages ' Microprogrammed control ' Control memory ' Microinstruction ' Microprogram ' Control word ' CAR ' Microprogram sequencer ' Microinstruction format ' Horizontal microprogramming ' Vertical microprogramming ' Encoded control fields ' Control-store numerals ' Hardwired vs microprogrammed comparison ' GATE traps

CHAPTER 5: COMPLETE '

OFFICIAL SYLLABUS STATUS

We have now completely covered these official phrases:

' Instruction set ' Addressing modes ' Design of arithmetic and logic unit (ALU) ' Design of control unit – hardwired and microprogrammed

Still remaining:

+ Memory interfacing and hierarchy + Performance + Cache memory mapping + I/O interface + Interrupt + DMA + Instruction pipelining + Pipeline hazards

COURSE PROGRESS

Completed: 6 / 26 modules

Approximate roadmap progress:

$\sim 23\%$

NEXT !' CHAPTER 6 — MEMORY SYSTEM FUNDAMENTALS

Next we begin the largest and one of the highest-yield COA sections:

~~Ø=MEM~~ **MEMORY HIERARCHY**

We will cover:

- registers ! ' cache ! ' RAM ! ' secondary memory,
- why hierarchy is necessary,
- temporal and spatial locality,
- byte vs word addressability,
- memory capacity,
- address-space numericals,
- memory word organization,
- address/data line calculations,

which prepares us for the major GATE topic: **Cache Memory Mapping**.

USER

next

ASSISTANT

CHAPTER 6 — MEMORY SYSTEM FUNDAMENTALS

Classification: [GATE CORE] **Priority:** $\emptyset = \checkmark$ **Official syllabus connection:** Memory interfacing and hierarchy

This chapter builds the foundation for:

- cache mapping,
- memory performance,
- memory interfacing,
- address-bit numericals.

6.1 Why Do We Need a Memory Hierarchy?

The CPU is extremely fast.

Main memory is slower.

Secondary storage is even slower.

If the CPU had to wait for slow storage after every instruction:

most CPU speed would be wasted.

So computers use different levels of memory.

TEXT

Fastest	!
Registers	!
Cache	!"

Main Memory (RAM)	!"
Secondary Storage SSD / HDD	!"
Slowest	!"

This is called:

$\boxed{\text{Memory Hierarchy}}$

6.2 The Fundamental Memory Trade-off

Ideally we want memory that is:

- extremely fast,
- extremely large,
- extremely cheap.

But practically, we cannot maximize all three together.

Generally:

TEXT

Faster memory
!' smaller capacity
!' higher cost per bit

and:

TEXT

Slower memory
!' larger capacity
!' lower cost per bit

Memory Hierarchy Trend

Moving upward:

TEXT

Speed !'
Cost / bit !'
Capacity !"

Moving downward:

TEXT

Speed !"
Cost / bit !"
Capacity !'

6.3 Memory Hierarchy Table

Level	Speed	Capacity	Cost/bit
Registers	Highest	Tiny	Highest
Cache	Very high	Small	High
RAM	Moderate	Large	Moderate
SSD/HDD	Low	Very large	Low

Memory trick:

Fast !' Small !' Expensive

6.4 Registers

Registers are the fastest storage used directly by the CPU.

Examples:

TEXT

R1
R2
PC
IR
MAR
MDR

Characteristics:

- inside CPU,
- extremely fast,
- very limited quantity.

6.5 Cache Memory

Very important.

Cache is a small, fast memory placed between:

TEXT

C P U ! " M a i n M e m o r y

Its purpose:

Keep recently or frequently used data/instructions close to the processor.

Simplified:

TEXT

CPU

!"

Cache

!"

RAM

If required data is in cache:

CPU avoids slower RAM access.

6.6 Main Memory

Main memory generally means RAM.

It stores currently active:

- programs,
- instructions,
- data.

Compared with cache:

TEXT

RAM is:

slower

larger

cheaper per bit

6.7 Secondary Memory

Examples:

- SSD
- HDD

Used for long-term storage.

Much larger than RAM but generally much slower from the CPU's perspective.

6.8 Why Does Cache Actually Work?

This is because programs exhibit:

$\boxed{\text{Locality of Reference}}$

Programs tend to reuse nearby or recently used information.

Two major types:

1. Temporal locality
2. Spatial locality

Extremely important concepts.

6.9 Temporal Locality

Temporal locality means:

If something is accessed now, it is likely to be accessed again soon.

Example:

```
C
for(i = 0; i < 1000; i++)
    sum = sum + A[i];
```

The variable:

```
TEXT
sum
```

is accessed repeatedly.

Therefore it has strong temporal locality.

Memory Trick

Temporal = Time

Same data again soon in time.

6.10 Spatial Locality

Spatial locality means:

If one memory location is accessed, nearby locations are likely to be accessed soon.

Example:

```
C
A[0]
A[1]
A[2]
A[3]
```

Array elements are commonly stored near each other.

So accessing `A[0]` suggests nearby elements may soon be needed.

Memory Trick

Spatial = Space

Nearby addresses are likely to be used.

6.11 Temporal vs Spatial

Locality	Meaning	Example
Temporal	Same location reused soon	loop variable
Spatial	Nearby locations accessed	array traversal

Easy mnemonic:

TEXT

```
T e m p o r a l ! '   A g a i n
S p a t i a l   ! '   A r o u n d
```

6.12 Sequential Locality

Some texts mention **sequential locality**.

Example:

TEXT

```
Instruction 100
Instruction 104
Instruction 108
Instruction 112
```

This is essentially a strong form of spatial locality.

For GATE, focus mainly on:

- temporal,
- spatial.

6.13 Why Cache Stores Blocks

Suppose CPU accesses:

TEXT

```
Address 1000
```

Because of spatial locality, it may soon access:

TEXT

```
1001
1002
1003
```

...

So cache does not usually copy only one tiny value.

It copies a group of nearby memory locations called a:

$\boxed{\text{Block}}$

or:

$\boxed{\text{Cache Line}}$

We study this deeply later.

6.14 Memory Terminology

You must be comfortable with:

- memory location,
- address,
- word,
- address space,
- capacity,
- addressability,
- address lines,
- data lines.

Let's build each carefully.

6.15 Memory Location

A memory location is a uniquely addressable storage unit.

Example:

TEXT	
Address	Content
100	25
101	90
102	14

Each address identifies one memory location.

6.16 Address

An address is the number used to identify a memory location.

If there are n address bits:

$\boxed{2^n \text{ distinct addresses}}$

Example:

10-bit address:

$$\lfloor 2^{10}=1024 \rfloor$$

addresses.

6.17 Memory Address Space

Address space means the set/range of addresses a processor can generate.

With n address bits:

$$\lfloor \boxed{\text{Address space}}=2^n \rfloor$$

But total capacity also depends on:

how much data each address represents.

That is the concept of **addressability**.

6.18 Addressability

Addressability tells us:

How much data is associated with one unique address?

Common types:

1. Byte-addressable
2. Word-addressable

6.19 Byte-Addressable Memory

Each address identifies:

$$\lfloor \boxed{1 \text{ byte}} \rfloor$$

If address width = n :

$$\lfloor 2^n \rfloor$$

and each is 1 byte.

Therefore:

$$\lfloor \boxed{\text{Capacity}}=2^n \rfloor$$

Example

20-bit byte-addressable system:

$$\lfloor 2^{20} \rfloor$$

$$\lfloor \boxed{1\text{MB}} \rfloor$$

6.20 Word-Addressable Memory

Each address identifies:

$$\boxed{1 \text{ word}}$$

Suppose:

- n address bits,
- word size = w bits.

Then:

$$2^n \text{ words}$$

Total bits:

$$2^n \times w$$

Total bytes:

$$2^n \times \frac{w}{8}$$

Example

16-bit address, 32-bit word.

Number of locations:

$$2^{16}$$

Each location:

$$32 \text{ bits} = 4 \text{ bytes}$$

Capacity:

$$2^{16} \times 4$$

$$= 2^{18} \text{ bytes}$$

$$\boxed{256 \text{ KB}}$$

6.21 GATE MASTER FORMULA

If:

- A = number of address bits,
- one address represents X bytes,

then:

$$\boxed{\text{Memory Capacity} = 2^A \times X \text{ bytes}}$$

This one formula solves many memory questions.

6.22 Memory Word

A **word** is a fixed-size unit naturally handled by a machine/memory.

Examples:

TEXT

16-bit word
32-bit word
64-bit word

Do not assume word = byte.

Example:

32-bit word:

$\lceil 32/8=4 \rceil$ bytes

6.23 Memory Organization Notation

A memory chip may be written as:

$N \times M$

where:

- N = number of memory words/locations,
- M = bits per word.

Example:

$1K \times 8$

means:

TEXT

1024 locations
8 bits/location

Total:

$1024 \times 8 = 8192$ bits

$= 1024$ bytes

$1KB$

6.24 Another Example

Memory:

$4K \times 16$

means:

$4K = 2^{12}$ locations

Each:

$16 \text{ bits} = 2 \text{ bytes}$

Total:

$2^{12} \times 2$

$= 2^{13}$ bytes

$\boxed{8KB}$

6.25 Address Lines from Memory Organization

Memory:

$1K \times 8$

Number of locations:

$1K = 2^{10}$

Therefore address lines needed:

$\boxed{10}$

Data lines:

$\boxed{8}$

because each word is 8 bits.

Memory Trick

For:

$N \times M$

think:

TEXT

N !' A d d r e s s s i d e
M !' D a t a s i d e

6.26 Address Lines

If there are:

2^n

memory locations:

$\boxed{n \text{ address lines}}$

Example:

$64K \times 8$

Since:

$64K = 2^{16}$

need:

$\boxed{16 \text{ address lines}}$

6.27 Data Lines

Data lines depend on word width.

For:

$64K \times 8$

data lines:

8

For:

$64K \times 16$

data lines:

16

6.28 GATE Example — Address and Data Lines

Memory chip:

$8K \times 32$

Address lines

$8K = 8 \times 2^{10} = 2^{13}$

Therefore:

13 address lines

Data lines

Word width:

32

Therefore:

32 data lines

6.29 Capacity Calculation

For:

$8K \times 32$

Total bits:

$2^{13} \times 32$

Since:

$32 = 2^5$

$= 2^{18} \text{ bits}$

In bytes:

2^{15} bytes

$32KB$

6.30 Bits, Bytes, KB — Trap

Suppose capacity is:

64Kb

Small **b** usually means bits.

$64\text{Kb} = 64 \times 1024 \text{ bits}$

Whereas:

64KB

means bytes.

Since:

$1\text{B} = 8\text{b}$

$64\text{KB} = 512\text{Kb}$

~~Always~~ Always inspect units.

6.31 Address Range

Suppose an 8-bit address field.

Minimum address:

0

Maximum:

$2^8 - 1 = 255$

So range:

$0 \text{ to } 255$

Similarly, 16-bit:

$0 \text{ to } 65535$

6.32 Hex Address Range

A 16-bit address can be represented using:

$16/4 = 4$

hex digits.

Range:

TEXT

0000H to FFFFH

A 32-bit address:

TEXT

00000000H to FFFFFFFFH

because each hex digit = 4 bits.

6.33 GATE Hex Shortcut

If address has:

TEXT

N hex digits

then address bits:

$\boxed{4N}$

Example:

6 hexadecimal address digits:

$6 \times 4 = 24 \text{ bits}$

If byte-addressable:

$2^{24} \text{ bytes} = 16 \text{ MB}$

6.34 Address Range Size Formula

If address range is:

$A_{\text{start}} \text{ to } A_{\text{end}}$

inclusive, number of locations is:

$A_{\text{end}} - A_{\text{start}} + 1$

$A_{\text{end}} + 1$ is a classic trap.

Example

Addresses:

TEXT

1000 to 1999

Number:

$1999 - 1000 + 1$

$\boxed{1000}$

locations.

6.35 Hex Range Example

Address range:

TEXT

4000H to 7FFFH

Number of addresses:

$\lfloor 7FFF_H - 4000_H + 1 \rfloor$

An easier observation:

TEXT

4 0 0 0 H ! ' 7 F F F H

is a block of:

$\lfloor 4000_H \rfloor$

addresses.

Since:

$\lfloor 4000_H = 4 \times 16^3 \rfloor$

$\lfloor = 2^2 \times 2^{12} \rfloor$

$\lfloor = 2^{14} \rfloor$

Therefore:

$\lfloor \boxed{16K} \text{ locations} \rfloor$

6.36 Memory Capacity Example

A byte-addressable processor has 36-bit addresses.

Capacity:

$\lfloor 2^{36} \text{ bytes} \rfloor$

Since:

$\lfloor 2^{30} = 1\text{GB} \rfloor$

$\lfloor 2^{36} = 64\text{GB} \rfloor$

Therefore:

$\lfloor \boxed{64\text{GB}} \rfloor$

6.37 Reverse Problem — Address Bits

A byte-addressable memory capacity is:

$\lfloor 8\text{GB} \rfloor$

Convert:

$$[8GB=2^3\times 2^{30}=2^{33}\text{ bytes}]$$

Hence:

$$[\boxed{33}\text{ address bits}]$$

6.38 Reverse Problem — Word-Addressable

Memory capacity:

$$[16MB]$$

word size:

$$[32\text{ bits}=4\text{ bytes}]$$

Number of words:

$$[\frac{16MB}{4}]$$

$$[=\frac{2^{24}}{2^2}]$$

$$[=2^{22}]$$

So address bits:

$$[\boxed{22}]$$

because each address identifies one 32-bit word.

6.39 Common GATE Strategy

For memory questions, always write:

TEXT

1. Number of locations
2. Bytes per location
3. Multiply

Example:

TEXT

20 address bits
32-bit word-addressable

Step 1:

$$[2^{20}\text{ locations}]$$

Step 2:

$$[32/8=4\text{ bytes/location}]$$

Step 3:

$$[2^{20}\times 4=2^{22}\text{ bytes}]$$

$$[\boxed{4MB}]$$

6.40 Memory Access Time

Memory Access Time is the time between:

requesting memory access and obtaining/completing the required data operation.

For a read:

TEXT

Address supplied

!"

Read requested

!"

Data becomes available

Time required is access time.

6.41 Memory Cycle Time

Memory Cycle Time is the minimum time between the starts of two consecutive memory operations.

It may include:

- access time,
- recovery/setup time.

Generally:

$$\boxed{\text{Cycle Time}} \geq \text{Access Time}$$

Do not assume they are identical.

6.42 Access Time vs Cycle Time

Term	Meaning
Access time	Time to perform/access one requested item
Cycle time	Minimum interval between memory operations

This distinction occasionally appears conceptually.

6.43 Memory Bandwidth

[SUPPORT / PERFORMANCE]

Bandwidth is the amount of data transferred per unit time.

Example:

If memory transfers:

8 bytes

every:

$\lfloor 4\text{ns} \rfloor$

then theoretical data rate:

$\lfloor \frac{8}{4\text{ns}} = 2 \text{ bytes/ns} \rfloor$

Since:

$\lfloor 1\text{ns} = 10^{-9}\text{s} \rfloor$

$\lfloor 2 \times 10^9 \text{ bytes/s} \rfloor$

approximately:

$\lfloor \boxed{2\text{GB/s}} \rfloor$

under decimal GB convention.

6.44 Latency vs Bandwidth

Very useful distinction:

Latency

How long you wait for one request.

Bandwidth

How much data can be transferred per second.

Analogy:

Latency = time for first bus to arrive.

Bandwidth = number of passengers buses can move per hour.

6.45 Volatile vs Non-Volatile Memory

[SUPPORT]

Volatile

Loses data when power is removed.

Examples:

- registers,
- typical cache,
- RAM.

Non-volatile

Retains data without power.

Examples:

- ROM,
- SSD,

- HDD.

6.46 RAM

RAM = Random Access Memory

"Random access" means any location can be accessed directly rather than sequentially.

Two common technologies:

- SRAM
- DRAM

This distinction is useful for understanding cache vs RAM.

6.47 SRAM

SRAM = Static RAM

Characteristics:

- faster,
- more expensive per bit,
- lower density,
- traditionally used for caches.

It does not require periodic refresh while powered.

6.48 DRAM

DRAM = Dynamic RAM

Characteristics:

- slower than SRAM,
 - cheaper,
 - denser,
 - needs refresh,
 - commonly used for main memory.
-

SRAM vs DRAM

Feature	SRAM	DRAM
Speed	Faster	Slower
Cost/bit	Higher	Lower
Density	Lower	Higher
Refresh	No periodic refresh	Required
Typical use	Cache	Main memory

Memory trick:

S = Speedy SRAM

D = Dense DRAM

6.49 Why Cache Uses SRAM

Cache must be extremely fast.

Therefore SRAM is traditionally appropriate despite higher cost.

Main memory needs large capacity, so DRAM is more economical.

6.50 ROM — Support Concept

ROM = Read Only Memory

Historically used for:

- firmware,
- fixed programs/data.

Variants include:

- PROM,
- EPROM,
- EEPROM.

For your current GATE syllabus, this is low priority unless memory interfacing questions use ROM chips.

6.51 CPU-Memory Interface

Basic communication uses:

TEXT

Address lines
Data lines
Control lines

Conceptually:

TEXT

```

      Address Bus
C P U  % % % % % % % % % % % % % % % % % ! '  M e m o r y

      Data Bus
C P U  ! • % % % % % % % % % % % % % % % % % ! '  M e m o r y

      Control
C P U  % % % % % % % % % % % % % % % % % ! '  M e m o r y
      Read / Write
```

6.52 Read Operation

CPU wants:

`\[ M[500] \]`

Simplified:

TEXT

```
A d d r e s s   b u s   !•   5 0 0
R e a d   s i g n a l   !•   a c t i v e
Memory outputs content
D a t a   b u s   !•   v a l u e
```

6.53 Write Operation

CPU wants:

`\[ M[500]\leftarrow 75 \]`

Simplified:

TEXT

```
A d d r e s s   b u s   !'   5 0 0
D a t a   b u s           !'   7 5
Write signal activated
```

6.54 Memory Chip Select

When multiple memory chips exist, not all should respond to every address.

A **Chip Select (CS)** signal enables the required chip.

Example:

TEXT

```
CPU address
! "
Address decoding
! "
Chip 0 selected
or
Chip 1 selected
or
Chip 2 selected
```

This becomes central in **memory interfacing**.

6.55 Decoder Idea

A decoder converts some high-order address bits into chip-select signals.

Example:

2 input bits:

TEXT

00
01
10
11

can select:

TEXT

Chip 0
Chip 1
Chip 2
Chip 3

A 2-to-4 decoder gives four selection outputs.

6.56 Low-Order vs High-Order Address Bits

When interfacing memory chips:

Low-order bits

Usually select locations **inside a chip**.

High-order bits

Often determine **which chip** is selected.

Example:

A chip has:

$$\lfloor 1K=2^{\{10\}} \rfloor$$

locations.

Therefore 10 low-order address bits are required internally.

Any extra address bits can participate in chip selection.

6.57 GATE Example — Chip Addressing

Suppose chip:

$$\lfloor 2K \times 8 \rfloor$$

Since:

$$\lfloor 2K=2^{\{11\}} \rfloor$$

need:

$$\lfloor \boxed{11} \rfloor$$

address bits to select a location inside the chip.

Data lines:

$\boxed{8}$

6.58 Number of Chips — Preview

Suppose we need:

8×16

memory using:

2×8

chips.

Depth expansion:

$8 / 2 = 4$

Width expansion:

$16 / 8 = 2$

Total chips:

$4 \times 2 = 8$

$\boxed{8}$

We will study this fully in Chapter 8.

6.59 Word Expansion vs Bit Expansion

Preview:

Word expansion / depth expansion

Increase number of memory locations.

Example:

TEXT

$2 \text{ K} \times 8 \rightarrow 8 \text{ K} \times 8$

Need more chips in address depth.

Bit expansion / width expansion

Increase bits per word.

Example:

TEXT

$2 \text{ K} \times 8 \rightarrow 2 \text{ K} \times 16$

Use chips in parallel.

6.60 Memory Hierarchy and Inclusion Idea

[SUPPORT]

A value may conceptually exist simultaneously in:

TEXT

Cache
and
Main memory

Cache holds copies of memory blocks.

This is why later we need:

- cache tags,
- valid bits,
- write policies.

6.61 Cache Hit and Miss — Preview

If CPU asks for data and cache has it:

$\boxed{\text{Hit}}$

If cache does not have it:

$\boxed{\text{Miss}}$

Then lower memory level must be accessed.

6.62 Hit Ratio

$\boxed{\text{Hit Ratio} = \frac{\text{Number of Hits}}{\text{Total Accesses}}}$

Miss ratio:

$\boxed{\text{Miss Ratio} = 1 - \text{Hit Ratio}}$

Detailed performance comes next chapter.

6.63 GATE Example — Hit Ratio

Suppose:

- 1000 memory accesses,
- 950 cache hits.

Hit ratio:

$\frac{950}{1000} = 0.95$

$\boxed{95\%}$

Miss ratio:

$1 - 0.95 = 0.05$

$\boxed{5\%}$

6.64 Why Larger Cache Is Not Automatically Perfect

A larger cache may improve hit rate, but:

- costs more,
- may have longer access,
- cannot be infinitely large.

Computer architecture involves trade-offs.

6.65 Why Huge Blocks Are Not Always Better

Spatial locality suggests bigger blocks help.

But excessively large blocks can:

- waste cache space,
- reduce number of distinct blocks stored,
- increase miss penalty.

So block size has a trade-off.

We will revisit this.

6.66 Memory Hierarchy Principle

The goal is to create the illusion of:

memory almost as fast as the fastest level and almost as large as the larger levels.

That is why hierarchy is powerful.

6.67 GATE NUMERICAL 1 — Memory Organization

Memory:

$[32K \times 16]$

Find:

1. Address lines
2. Data lines
3. Capacity in KB

Address lines

$[32K = 2^{15}]$

So:

$[\boxed{15}]$

Data lines

$\boxed{16}$

Capacity

$2^{15} \times 16 \text{ bits}$

Since:

$16 = 2 \text{ bytes}$

$2^{15} \times 2 = 2^{16} \text{ bytes}$

$\boxed{64 \text{ KB}}$

6.68 GATE NUMERICAL 2 — Byte Addressable

A system has 26 address lines and is byte-addressable.

Capacity:

2^{26} bytes

$= 64 \text{ MB}$

$\boxed{64 \text{ MB}}$

6.69 GATE NUMERICAL 3 — Word Addressable

A system has:

- 24 address bits,
- 64-bit word,
- word-addressable memory.

Capacity:

$2^{24} \times 64 \text{ bits}$

64 bits:

8 bytes

Hence:

$2^{24} \times 2^3 = 2^{27} \text{ bytes}$

$\boxed{128 \text{ MB}}$

6.70 GATE NUMERICAL 4 — Reverse Address Bits

A byte-addressable memory is 2 GB.

$2 \text{ GB} = 2^1 \times 2^{30} = 2^{31} \text{ bytes}$

Therefore:

$\boxed{31 \text{ address bits}}$

6.71 GATE NUMERICAL 5 — Word Address Reverse

Memory:

$[128\text{MB}]$

Word size:

$[64\text{ bits}=8\text{ bytes}]$

Number of word locations:

$[\frac{2^{27}}{2^3}=2^{24}]$

Address bits:

$[\boxed{24}]$

6.72 GATE NUMERICAL 6 — Hex Addresses

A byte-addressable processor uses 5 hex digits for addresses.

Address bits:

$[5 \times 4=20]$

Capacity:

$[2^{20}\text{ bytes}]$

$[\boxed{1\text{MB}}]$

6.73 GATE NUMERICAL 7 — Address Range

Find capacity of byte-addressable range:

TEXT

2000H to 5FFFH

Number of addresses:

$[5\text{FFF}_\text{H}-2000_\text{H}+1]$

Difference corresponds to:

$[4000_\text{H}]$

locations.

$[4000_\text{H}=2^{14}]$

Hence:

$[\boxed{16\text{KB}}]$

6.74 GATE NUMERICAL 8 — Memory Chip

Chip:

$16K \times 4$

Address lines:

$16K = 2^{14}$

2^{14}

Data lines:

2^4

Capacity:

$2^{14} \times 4 \text{ bits}$

$= 2^{16} \text{ bits}$

$= 2^{13} \text{ bytes}$

$8KB$

6.75 GATE NUMERICAL 9 — Locality

Consider:

```
C
for(i=0; i<1000; i++)
    sum += A[i];
```

sum

Repeated frequently.

Temporal locality

A[i], A[i+1], ...

Nearby elements accessed sequentially.

Spatial locality

Both can exist in the same program.

6.76 GATE TRAP — Address Bits vs Data Bits

Suppose:

```
TEXT
20 address lines
32 data lines
```

You cannot say capacity =:

2^{32}

just because data bus is 32 bits.

Address count comes from address lines.

Capacity depends on addressability/word width.

6.77 GATE TRAP — 32-bit Processor

"32-bit processor" does not automatically tell you:

- address space = 4 GB,
- data bus = 32 bits,
- memory word = exactly 32 bits,

unless context specifies what "32-bit" refers to.

Always use explicit information.

6.78 GATE TRAP — KB Meaning

For most GATE memory numericals:

$1\text{KB} = 2^{10}\text{B}$

$1\text{MB} = 2^{20}\text{B}$

$1\text{GB} = 2^{30}\text{B}$

Use binary powers unless question specifies otherwise.

6.79 GATE TRAP — Locations vs Capacity

Example:

$4\text{K} \times 16$

Number of locations:

4K

But capacity is:

8KB

Do not report 4K as total bytes.

6.80 GATE TRAP — Addressability

Suppose 20-bit address.

Without knowing byte vs word addressability, you only know:

2^{20} addresses

You do not necessarily know the total bytes.

6.81 GATE TRAP — Inclusive Range

From:

TEXT

0000H to FFFFH

number of addresses:

$\lfloor \text{FFFF_H} - \text{0000_H} + 1 \rfloor$

$\lfloor = 10000_H \rfloor$

$\lfloor = 2^{\{16\}} \rfloor$

The +1 matters.

6.82 GATE TRAP — SRAM vs DRAM

Do not reverse:

TEXT

C a c h e ! ' t y p i c a l l y S R A M
M a i n m e m o r y ! ' t y p i c a l l y D R A M

Memory trick:

Cache wants speed !' SRAM.

RAM wants density !' DRAM.

6.83 GATE TRAP — Temporal vs Spatial

Repeated same variable:

$\lfloor \boxed{\text{Temporal}} \rfloor$

Nearby addresses:

$\lfloor \boxed{\text{Spatial}} \rfloor$

CHAPTER 6 — MASTER MEMORY CALCULATION METHOD

Whenever you see a capacity problem, write:

Step 1

How many locations?

$\lfloor 2^{\{\text{address bits}\}} \rfloor$

Step 2

How much per location?

TEXT

1 byte
or
word size / 8 bytes

Step 3

Multiply.

$$\boxed{\text{Capacity} = \text{Locations} \times \text{bytes/location}}$$

This solves most basic memory capacity questions.

CHAPTER 6 — FORMULA SHEET

Number of addresses

$$\boxed{2^n}$$

Byte-addressable capacity

$$\boxed{2^n \text{ bytes}}$$

Word-addressable capacity

$$\boxed{2^n \times \text{word size in bytes}}$$

Address bits for N locations

$$\boxed{\lceil \log_2 N \rceil}$$

Memory organization

For:

$$\boxed{N \times M}$$

$$\boxed{\text{Total bits} = N \times M}$$

Address range size

$$\boxed{\text{End} - \text{Start} + 1}$$

Hit ratio

$$\boxed{h = \frac{\text{hits}}{\text{total accesses}}}$$

Miss ratio

$$\boxed{m = 1 - h}$$

CHAPTER 6 — MEMORY TRICKS

Memory hierarchy

R C R S

TEXT

Registers
Cache
RAM
Storage

Trend

Fast = Small + Costly

Locality

Temporal = Again

Spatial = Around

SRAM vs DRAM

S = Speed

D = Density

Memory notation

$N \times M$

TEXT

N !' number of addresses
M !' bits per address

CHAPTER SNAPSHOT

1. Memory hierarchy exists because no memory technology is simultaneously fastest, largest and cheapest.
2. Registers are fastest and smallest.
3. Cache sits between CPU and main memory.
4. RAM is larger but slower than cache.
5. Storage is much larger and slower.
6. Temporal locality = same item reused soon.
7. Spatial locality = nearby locations used soon.
8. n address bits give (2^n) addresses.
9. Addressability determines how much data one address identifies.
10. Byte-addressable means one address = one byte.
11. Word-addressable means one address = one word.
12. $(N \times M)$ means N words of M bits.

13. Address lines depend on number of words.
 14. Data lines depend on word width.
 15. SRAM is typically used for cache.
 16. DRAM is typically used for main memory.
 17. Chip select enables the proper memory chip.
 18. Low-order address bits usually select location inside a chip.
 19. High-order bits often participate in chip selection.
 20. Hit/miss concepts form the basis of cache performance.
-

MUST REMEMBER

$\boxed{\text{address bits}} \rightarrow 2^n \text{ addresses}$

$\boxed{\text{Byte-addressable capacity}} = 2^n \text{ bytes}$

$\boxed{\text{Word-addressable capacity}} = 2^n \times \text{word bytes}$

$\boxed{N \times M = N \text{ locations, } M \text{ bits/location}}$

And:

EA/memory questions become easier when you separate address count from bytes per address.

PRACTICE SET

Q1 — L0 MCQ

Which memory is generally the fastest?

A. RAM B. Cache C. Registers D. SSD

Q2 — L0 MCQ

Repeated access to the same variable mainly demonstrates:

A. Spatial locality B. Temporal locality C. Indirect locality D. Associative locality

Q3 — L0 MCQ

Sequential access to array elements mainly demonstrates:

A. Temporal locality B. Spatial locality C. Control locality D. No locality

Q4 — L1 NAT

A byte-addressable system has 22 address bits.

Maximum capacity in MB = ?

Q5 — L1 NAT

A word-addressable memory has:

- 18 address bits,
- word size = 16 bits.

Find capacity in KB.

Q6 — L1

For memory:

$[8K \times 16]$

find:

1. address lines,
 2. data lines.
-

Q7 — L1 NAT

Find total capacity in KB of:

$[8K \times 16]$

Q8 — L2 NAT

A byte-addressable memory capacity is 32 GB.

How many address bits are needed?

Q9 — L2 NAT

A word-addressable memory has a total capacity of 64 MB and 32-bit words.

How many address bits are required?

Q10 — L2

A memory has address range:

TEXT

4000H to 7FFFH

and is byte-addressable.

Find its capacity in KB.

Q11 — L2

A chip is:

$32\text{K} \times 8$

Find:

- address lines,
- data lines,
- total capacity in KB.

Q12 — L2 MCQ

Which is generally true?

A. SRAM is slower and denser than DRAM. B. DRAM is normally used for CPU registers. C. SRAM is commonly used for cache. D. DRAM does not need refresh.

Q13 — L2 NAT

A processor uses 7 hexadecimal digits for byte addresses.

Maximum addressable memory in MB = ?

Q14 — L3

A word-addressable system has:

- 26-bit addresses,
- 64-bit words.

Find total memory capacity in GB.

Q15 — L3

A program repeatedly executes:

```
c
for (i=0; i<1000; i++) {
    x = x + A[i];
}
```

Identify where temporal locality and spatial locality occur and explain each in one sentence.

CHAPTER 6 COMPLETION STATUS

' Memory hierarchy ' Registers ' Cache overview ' Main memory ' Secondary storage ' Temporal locality ' Spatial locality ' Memory word ' Address ' Address space ' Addressability ' Byte-addressable memory ' Word-addressable memory ' Memory organization $(N \times M)$ ' Address lines ' Data lines ' Capacity calculations ' Address ranges ' Hexadecimal address numerals ' Access time ' Cycle time ' Bandwidth basics ' SRAM vs DRAM ' CPU-memory interface ' Chip select ' Decoder concept ' Cache hit/miss preview ' GATE traps

CHAPTER 6: COMPLETE '

COURSE PROGRESS

Completed: 7 / 26 modules

Approximate roadmap progress:

$\sim 27\%$

The memory foundation is now ready.

NEXT !' CHAPTER 7 — MEMORY PERFORMANCE

This is where numericals become much more important.

Next we will master:

- hit time,
 - miss penalty,
 - hit ratio,
 - miss ratio,
 - effective/average memory access time,
 - **AMAT**,
 - one-level cache,
 - two-level cache,
 - sequential vs parallel lookup assumptions,
 - CPU stall impact,
 - GATE numerical shortcuts and traps.
-

USER

next

ASSISTANT

CHAPTER 7 — MEMORY PERFORMANCE

Classification: [GATE CORE] **Priority:** ~~Ø=VERY~~ HIGH **Official syllabus connection:** Memory hierarchy: performance

This chapter is one of the most important numerical sections in COA.

You must become comfortable with:

- Hit
- Miss
- Hit ratio
- Miss ratio
- Hit time

- Miss penalty
- Effective memory access time
- AMAT
- Multi-level cache calculations

7.1 Why Memory Performance Matters

Suppose CPU needs data.

Possible situations:

TEXT

CPU asks for data

!"

Check Cache

!™

!~

Hit

Miss

!"

!"

Fast

Go to RAM

If most requests are hits, memory appears much faster.

If many are misses, CPU spends more time waiting.

So system performance depends heavily on:

$\boxed{\text{Hit Rate} + \text{Hit Time} + \text{Miss Penalty}}$

7.2 Cache Hit

A **cache hit** occurs when requested data is already present in cache.

Example:

TEXT

CPU requests block X

Cache contains X

Then:

$\boxed{\text{Hit}}$

CPU receives it quickly.

7.3 Cache Miss

A **cache miss** occurs when requested data is not present in cache.

Then:

TEXT

Cache miss

!"

Access next memory level

```
! "  
Bring required block  
! "  
Give data to CPU
```

This costs extra time.

7.4 Hit Ratio / Hit Rate

If:

- total accesses = (N)
- cache hits = (H)

then:

$$h = \frac{H}{N}$$

where h is hit ratio.

Example

1000 accesses, 920 hits.

$$h = \frac{920}{1000}$$

$$h = 0.92 = 92\%$$

7.5 Miss Ratio / Miss Rate

Miss ratio:

$$m = 1 - h$$

Example:

$$h = 0.92$$

then:

$$m = 1 - 0.92$$

$$m = 0.08 = 8\%$$

Memory Trick

$$\text{Hit} + \text{Miss} = 1$$

$$h + m = 1$$

7.6 Hit Time

Hit time is the time required to:

- check cache,
- determine that requested item is present,
- obtain it from cache.

Let:

$T_c = \text{cache access time}$

Example:

$T_c = 2\text{ns}$

7.7 Miss Penalty

Miss penalty is the extra time required when data is not found in cache.

This may include:

- access next memory level,
- transfer block,
- fill cache,
- deliver required data.

Example:

TEXT

Cache hit time = 2 ns

Main memory cost = 80 ns

A miss can impose roughly tens of nanoseconds of extra delay.

Important

Different questions define miss penalty differently.

Some use:

additional time after cache miss.

Others effectively give:

total time required for miss path.

Always inspect wording.

7.8 Average Memory Access Time — AMAT

~~$\text{AMAT} = \text{Hit Time} + \text{Miss Rate} \times \text{Miss Penalty}$~~ MASTER FORMULA

A common form is:

$\boxed{\text{AMAT} = \text{Hit Time} + \text{Miss Rate} \times \text{Miss Penalty}}$

or:

$$\boxed{AMAT = T_c + mP}$$

where:

- T_c = cache hit time
- m = miss rate
- P = miss penalty

7.9 Why This Formula Works

Every access always pays:

$$T_c$$

because cache is checked first.

Only a fraction m of accesses miss.

Those accesses pay additional penalty:

$$P$$

Therefore average:

$$T_c + mP$$

Simple.

7.10 Example 1 — Basic AMAT

Given:

- Cache hit time = 2 ns
- Miss rate = 5%
- Miss penalty = 100 ns

$$AMAT = 2 + 0.05(100)$$

$$= 2 + 5$$

$$\boxed{7\text{ns}}$$

7.11 Equivalent Probability Method

Sometimes it is safer to calculate using separate paths.

Suppose:

- Hit probability = h
- Miss probability = m
- Hit path time = T_h
- Miss path total time = T_m

Then:

$$\boxed{EAT = hT_h + mT_m}$$

This is simply weighted average.

7.12 Example 2 — Weighted Average

Suppose:

- Cache access = 10 ns
- Main memory access = 100 ns
- Hit ratio = 0.9

Assume cache checked first, and on miss main memory is accessed after the cache lookup.

Hit

Time:

$\boxed{10\text{ns}}$

Miss

Time:

$\boxed{10+100=110\text{ns}}$

So:

$\boxed{EAT=0.9(10)+0.1(110)}$

$\boxed{=9+11}$

$\boxed{20\text{ns}}$

7.13 Same Example Using Shortcut

Hit time:

$\boxed{10\text{ns}}$

Miss rate:

$\boxed{0.1}$

Additional miss penalty:

$\boxed{100\text{ns}}$

So:

$\boxed{AMAT=10+0.1(100)}$

$\boxed{20\text{ns}}$

Same answer.

7.14 The Most Common AMAT Trap

Suppose cache = 10 ns and memory = 100 ns.

Students sometimes write:

$$\lfloor 0.9(10)+0.1(100) \rfloor$$

and get:

$$\lfloor 19\text{ns} \rfloor$$

This assumes miss path costs only 100 ns.

But if cache is checked first, miss path is:

$$\lfloor 10+100=110\text{ns} \rfloor$$

Therefore:

$$\lfloor 0.9(10)+0.1(110)=20\text{ns} \rfloor$$

~~0=10~~ Always identify whether main-memory time is:

- additional after cache,
- or
- total miss-path time.

7.15 Two Common Formula Conventions

Convention A — Miss Penalty Is Additional Time

Then:

$$\boxed{AMAT=T_c+mP}$$

Convention B — Miss Cost Includes Entire Miss Path

Then:

$$\boxed{AMAT=hT_h+mT_m}$$

where (T_m) is complete miss time.

Both are valid.

You must know what the question means.

7.16 GATE Safe Method

When unsure, draw the path:

TEXT

Access starts

!"

Cache: 5 ns

!"

H i t ? % y e s ! ' f i n i s h

!" n o

Memory: 50 ns

!"

finish

Then:

Hit path

$$\sqrt{5\text{ns}}$$

Miss path

$$\sqrt{5+50=55\text{ns}}$$

Then weighted average.

This is the safest method.

7.17 Example 3

Given:

- cache = 4 ns
- memory = 80 ns
- hit ratio = 95%

Hit path:

$$\sqrt{4}$$

Miss path:

$$\sqrt{4+80=84}$$

Average:

$$\sqrt{0.95(4)+0.05(84)}$$

$$\sqrt{=3.8+4.2}$$

$$\sqrt{\boxed{8\text{ns}}}$$

Shortcut:

$$\sqrt{4+0.05(80)=8\text{ns}}$$

7.18 Effective Access Time

Many textbooks use:

$$\sqrt{\boxed{\text{EAT = Effective Access Time}}}$$

This is closely related to AMAT.

For GATE questions, interpret formula according to given memory hierarchy and timing assumptions.

7.19 One-Level Cache Formula

For cache followed by main memory:

$$\sqrt{\boxed{\text{AMAT} = T_{L1} + m_{1P_1}}}$$

where:

- (T_{L1}) = L1 hit time
- (m_1) = L1 miss rate
- (P_1) = extra penalty caused by L1 miss

7.20 Multi-Level Cache

Modern systems often have:

```

TEXT
CPU
! "
L1 Cache
! "
L2 Cache
! "
L3 Cache
! "
Main Memory

```

Each level is:

- larger,
- slower,

as we go downward.

7.21 Two-Level Cache AMAT

Let:

- (T_1) = L1 hit time
- (m_1) = L1 miss rate
- (T_2) = L2 hit time
- (m_2) = L2 local miss rate
- (P_2) = main-memory penalty after L2 miss

Then:

$$AMAT = T_1 + m_1(T_2 + m_2P_2)$$

This is one of the most useful formulas.

7.22 Deriving Two-Level AMAT

Every access first costs:

$$T_1$$

Only L1 misses go to L2:

$$m_1$$

For those, we pay:

$$T_2$$

Only L2 misses additionally go to memory:

$$[m_{2P_2}]$$

Hence:

$$[AMAT = T_1 + m_1(T_2 + m_{2P_2})]$$

7.23 Example 4 — Two-Level Cache

Given:

- L1 hit time = 1 ns
- L1 miss rate = 10%
- L2 hit time = 5 ns
- L2 miss rate = 20%
- Main-memory penalty = 100 ns

$$[AMAT = 1 + 0.1(5 + 0.2(100))]$$

$$[= 1 + 0.1(5 + 20)]$$

$$[= 1 + 2.5]$$

$$[\boxed{3.5\text{ns}}]$$

7.24 Local Miss Rate

Suppose L2 miss rate is measured among requests **that actually reach L2**.

That is called:

$$[\boxed{\text{Local Miss Rate}}]$$

Example:

100 total CPU accesses.

10 miss L1.

Of these 10, 2 miss L2.

L2 local miss rate:

$$[\frac{2}{10} = 20\%]$$

7.25 Global Miss Rate

Global miss rate of L2 means fraction of all CPU accesses that miss both L1 and L2.

$$[\boxed{\text{Global L2 Miss Rate} = m_{1m_2} }]$$

Example:

$$[0.1 \times 0.2 = 0.02]$$

So:

$$[\boxed{2\%}]$$

of all references reach main memory.

Memory Trick

Local = among visitors to that level.

Global = among all CPU accesses.

7.26 Example 5 — Global Miss Rate

L1 miss rate:

$$5\%$$

L2 local miss rate:

$$10\%$$

Global miss rate after L2:

$$0.05 \times 0.10$$

$$= 0.005$$

$$\boxed{0.5\%}$$

Only 0.5% of CPU accesses reach main memory.

7.27 Three-Level Cache AMAT

Conceptually:

$$\boxed{AMAT = T_1 + m_1 \left(T_2 + m_2 \left(T_3 + m_3 P \right) \right)}$$

Do not blindly memorize.

Think recursively:

current level time + miss rate $\times$ cost of next level.

Memory Trick

For every level:

$$\boxed{\text{Time here} + \text{miss probability} \times \text{lower-level cost}}$$

7.28 Parallel Cache and Memory Access

Some questions assume cache and main memory are accessed **simultaneously**.

This changes the formula.

Example:

- cache time = 10 ns
- memory time = 100 ns

If accessed in parallel:

Hit

Cache gives answer in:

$\boxed{10\text{ns}}$

Miss

Memory was already being accessed, so data may be ready at:

$\boxed{100\text{ns}}$

not:

$\boxed{110\text{ns}}$

7.29 Parallel Lookup Formula

If:

- hit probability = h
- cache time = T_c
- memory time = T_m
- both begin simultaneously

then:

$\boxed{EAT = hT_c + (1-h)T_m}$

assuming memory time dominates miss path.

Example

$\boxed{h=0.9, T_c=10, T_m=100}$

$\boxed{EAT = 0.9(10) + 0.1(100)}$

$\boxed{= 9 + 10}$

$\boxed{19\text{ns}}$

Compare sequential case:

$\boxed{20\text{ns}}$

GATE Trap

You must determine whether cache and memory operate:

- sequentially,

or

- in parallel.

Do not use the same formula automatically.

7.30 Serial / Sequential Lookup

Typical assumption:

TEXT

```
Cache checked
!"
if miss
!"
Memory checked
```

Miss path:

$\backslash [T_c + T_m]$

Parallel Lookup

TEXT

```
Cache and Memory both start
```

Miss path may be:

$\backslash [T_m]$

if memory is slower and cache result simply determines whether memory output is needed.

7.31 Miss Penalty and Block Transfer

Cache misses often transfer an entire block, not just one byte.

Suppose:

- memory latency = 50 ns
- block contains 4 words
- each additional word takes 10 ns

Then miss penalty may be:

$\backslash [50 + 4(10)]$

or another form depending on whether first-word transfer is included in latency.

Always follow exact timing statement.

7.32 Example 6 — Block Transfer

Suppose a miss requires:

- 40 ns initial memory latency
- block of 4 words

- each word transfer = 5 ns

If all 4 transfer times are additional:

$$P = 40 + 4(5)$$

$$\boxed{60\text{ns}}$$

If wording says first word arrives after 40 ns and every **additional** word takes 5 ns:

$$40 + 3(5) = 55\text{ns}$$

Read wording carefully.

7.33 Average Stall Time Due to Misses

Suppose every memory access has miss rate m .

Average extra delay per memory access:

$$\boxed{m \times \text{MissPenalty}}$$

Example:

$$m = 0.03$$

$$P = 100\text{ns}$$

Average extra delay:

$$3\text{ns}$$

7.34 Cache and CPI Connection

Important later.

Suppose ideal processor CPI:

$$CPI_{\text{ideal}} = 1$$

Cache misses cause stall cycles.

Then:

$$\boxed{CPI_{\text{actual}} = CPI_{\text{ideal}} + \text{Memory stall cycles/instruction}}$$

7.35 Memory Stall Cycles

Suppose:

- Instruction miss rate = m_I
- Data references per instruction = d
- Data miss rate = m_D
- Miss penalty = P cycles

Then:

$$\boxed{\text{StallCycles/Instruction} = m_I P + d m_D P}$$

if same penalty is used.

7.36 Example 7 — CPI with Cache Misses

Given:

- Base CPI = 1
- Instruction-cache miss rate = 2%
- Data-cache miss rate = 4%
- 30% instructions access data
- Miss penalty = 50 cycles

Instruction stall:

$$[0.02 \times 50 = 1]$$

Data stall:

$$[0.30 \times 0.04 \times 50]$$

$$[= 0.6]$$

Total:

$$[1 + 0.6 = 1.6]$$

Actual CPI:

$$[1 + 1.6]$$

$$[\boxed{2.6}]$$

7.37 Why Data Access Fraction Matters

Not every instruction accesses data memory.

For example:

```
TEXT
ADD  R1, R2, R3
```

may use only registers.

But:

```
TEXT
LOAD R1, X
```

accesses data memory.

So if 25% of instructions are loads/stores, then:

$$[d = 0.25]$$

average data-memory references per instruction, under simplified assumptions.

7.38 Instruction Accesses per Instruction

Every instruction generally needs one instruction fetch.

So instruction-cache accesses per instruction:

$$\boxed{1}$$

unless stated otherwise.

This is why instruction misses are multiplied directly by miss rate.

7.39 Unified Cache

A **unified cache** stores both:

- instructions,
- data.

Then total memory references per instruction might be:

$$1 + d$$

Example:

1 instruction fetch + 0.3 data accesses:

$$1.3$$

cache accesses/instruction.

7.40 Split Cache

A **split cache** has:

- I-cache for instructions,
- D-cache for data.

Then miss behavior can be calculated separately.

This is relevant in performance numericals.

7.41 Example 8 — Unified Cache Stall

Suppose:

- 1.25 memory references per instruction
- Cache miss rate = 4%
- Miss penalty = 20 cycles
- Base CPI = 1

Stall cycles:

$$1.25 \times 0.04 \times 20$$

$$= 1$$

Actual CPI:

$$1 + 1$$

$$\boxed{2}$$

7.42 Average Access Time from Counts

Suppose:

- 1000 accesses
- 900 hits
- 100 misses
- Hit path = 2 ns
- Miss path total = 52 ns

Total time:

$$\backslash[900(2)+100(52) \backslash]$$

$$\backslash[=1800+5200 \backslash]$$

$$\backslash[=7000\text{ns} \backslash]$$

Average:

$$\backslash[7000/1000 \backslash]$$

$$\backslash[\boxed{7\text{ns}} \backslash]$$

Same as probability method.

7.43 Hit Ratio from AMAT

Sometimes reverse calculation is asked.

Suppose:

- Cache hit time = 5 ns
- Main-memory additional penalty = 100 ns
- AMAT = 15 ns

Formula:

$$\backslash[15=5+m(100) \backslash]$$

$$\backslash[10=100m \backslash]$$

$$\backslash[m=0.1 \backslash]$$

Therefore:

$$\backslash[h=0.9 \backslash]$$

$$\backslash[\boxed{90\%} \backslash]$$

7.44 Example 9 — Find Hit Rate

Cache = 2 ns.

Main-memory penalty = 60 ns.

AMAT = 5 ns.

$$\backslash[5=2+m(60) \backslash]$$

$$\lceil 3=60m \rceil$$

$$\lceil m=0.05 \rceil$$

Thus:

$$\lceil h=0.95 \rceil$$

$$\lceil \boxed{95\%} \rceil$$

7.45 Speedup Due to Cache

Suppose system without cache takes:

$$\lceil T_{\text{old}} \rceil$$

and with cache:

$$\lceil T_{\text{new}} \rceil$$

Then:

$$\lceil \boxed{\text{Speedup}=\frac{T_{\text{old}}}{T_{\text{new}}}} \rceil$$

Example

Without cache:

$$\lceil 100\text{ns/access} \rceil$$

With cache:

$$\lceil 10\text{ns/access} \rceil$$

Speedup:

$$\lceil 100/10=10 \rceil$$

$$\lceil \boxed{10\times} \rceil$$

7.46 Performance Improvement Percentage

If old time = 100 ns and new = 20 ns:

Reduction:

$$\lceil 100-20=80 \rceil$$

Percentage time reduction:

$$\lceil \frac{80}{100}\times 100 \rceil$$

$$\lceil \boxed{80\%} \rceil$$

Do not confuse with speedup:

$$\lceil 100/20=5\times \rceil$$

7.47 Hit Rate Improvement Is Not Linear Performance Improvement

Increasing hit rate from:

$$\backslash[90\%\rightarrow 95\% \backslash]$$

may have large effect if miss penalty is huge.

Example:

$$\backslash[T_c=1ns, P=100ns \backslash]$$

At 90% hit:

$$\backslash[AMAT=1+0.1(100)=11ns \backslash]$$

At 95% hit:

$$\backslash[AMAT=1+0.05(100)=6ns \backslash]$$

Small hit-rate increase nearly halves access time.

7.48 Why Miss Rate Is Often More Convenient

When hit rate is high:

$$\backslash[h=99\% \backslash]$$

miss rate:

$$\backslash[m=1\% \backslash]$$

AMAT becomes:

$$\backslash[T_c+0.01P \backslash]$$

This makes the miss contribution obvious.

7.49 Local vs Global Hit Rate

If L2 local miss = 20%, then L2 local hit =:

$$\backslash[80\% \backslash]$$

But that does not mean 80% of all CPU accesses hit L2.

Only L1 misses reach L2.

Example:

- L1 miss = 10%
- L2 local hit = 80%

Global L2 hits:

$$\backslash[0.10\times 0.80 \backslash]$$

$$\backslash[\boxed{8\%} \backslash]$$

of all CPU accesses.

7.50 Two-Level Probability Tree

Suppose:

$$\sqrt{h_1=0.9}$$

$$\sqrt{h_2=0.8}$$

Then:

L1 hit

$$\sqrt{0.9}$$

L2 hit

$$\sqrt{0.1 \times 0.8 = 0.08}$$

Main-memory access

$$\sqrt{0.1 \times 0.2 = 0.02}$$

Check:

$$\sqrt{0.9 + 0.08 + 0.02 = 1}$$

Perfect.

7.51 GATE Example 10 — Probability Tree Timing

Given:

- $L1 = 1 \text{ ns}$
- $L2 = 5 \text{ ns}$
- $\text{memory} = 100 \text{ ns}$
- $\sqrt{h_1=0.9}$
- $\sqrt{h_2=0.8}$, local to L2
- sequential access

Paths:

L1 hit

Probability:

$$\sqrt{0.9}$$

Time:

$$\sqrt{1}$$

L2 hit

Probability:

$$\sqrt{0.1(0.8)=0.08}$$

Time:

$$\sqrt{1+5=6}$$

Memory

Probability:

$$\sqrt{0.1(0.2)=0.02}$$

Time:

$$\boxed{1+5+100=106}$$

Average:

$$\boxed{0.9(1)+0.08(6)+0.02(106)}$$

$$\boxed{=0.9+0.48+2.12}$$

$$\boxed{3.5\text{ns}}$$

Same as nested AMAT formula.

7.52 Fast Two-Level Shortcut

$$\boxed{\text{AMAT} = T_1 + m_1(T_2 + m_2P)}$$

For previous values:

$$\boxed{1+0.1(5+0.2(100))}$$

$$\boxed{=3.5\text{ns}}$$

Use probability-tree method if question wording is unusual.

7.53 Average Memory Access with TLB — Support Concept

[LOW PRIORITY / OTHER COA CONTEXT]

Sometimes memory performance questions involve a TLB in virtual memory.

Your current COA syllabus does not explicitly list virtual memory, so we won't expand deeply here.

But remember the same principle:

identify each path, its probability, and its total time.

That method works everywhere.

7.54 Sequential vs Overlapped Block Transfer

Memory systems may support:

- sequential word transfer,
- burst transfer,
- interleaving.

Detailed DRAM organization is outside our main target here.

For GATE cache numericals, use exactly the transfer assumptions supplied.

7.55 Memory Bandwidth Numerical

Suppose a memory system transfers a 64-byte cache block in 20 ns.

Bandwidth:

$$\left[\frac{64 \text{ bytes}}{20 \text{ ns}} \right]$$

$$\left[= 3.2 \text{ bytes/ns} \right]$$

$$\left[= 3.2 \times 10^9 \text{ bytes/s} \right]$$

Approximately:

$$\left[\boxed{3.2 \text{ GB/s}} \right]$$

using decimal rate units.

7.56 Latency vs Block Transfer Time

Miss penalty may contain two parts:

$$\left[\boxed{\text{MissPenalty} = \text{InitialLatency} + \text{TransferTime}} \right]$$

Example:

- latency = 50 ns
- block transfer = 20 ns

Then:

$$\left[\boxed{70 \text{ ns}} \right]$$

7.57 Critical GATE Trap — Percent to Decimal

5% means:

$$\left[\boxed{0.05} \right]$$

not:

$$\left[5 \right]$$

Example:

$$\left[\text{AMAT} = 2 + 0.05(100) = 7 \text{ ns} \right]$$

not:

$$\left[502 \text{ ns} \right]$$

7.58 Critical GATE Trap — Hit Ratio vs Miss Ratio

If:

$$\left[h = 0.98 \right]$$

then:

$$\left[m = 0.02 \right]$$

Do not multiply miss penalty by hit rate.

Formula uses:

$$\left[\boxed{\text{miss rate}} \right]$$

7.59 Critical GATE Trap — Double Counting Cache Time

Suppose miss penalty is explicitly stated as:

120 ns in addition to cache lookup.

Then:

$$[AMAT = T_c + m(120)]$$

Do not add cache time again inside the penalty.

7.60 Critical GATE Trap — Under-Counting Cache Time

Suppose question says:

- cache = 5 ns
- memory = 100 ns
- sequential lookup.

Miss path:

$$[105\text{ns}]$$

not:

$$[100\text{ns}]$$

unless memory time is explicitly defined as the full miss penalty.

7.61 Critical GATE Trap — Local vs Global Miss

Two-level cache:

$$[m_1 = 10\%]$$

$$[m_2 = 20\% \text{ local}]$$

Main memory rate is not 20%.

It is:

$$[0.1 \times 0.2]$$

$$[\boxed{2\%}]$$

7.62 Critical GATE Trap — Data Accesses

If 30% instructions are loads/stores, do not automatically assume:

$$[30\%]$$

of all cache accesses are data.

There is also one instruction fetch per instruction.

Total references:

$$\lfloor 1+0.3=1.3 \rfloor$$

per instruction.

7.63 Critical GATE Trap — Time vs Cycles

Miss penalty may be given in:

- ns,
- cycles.

Do not combine them directly.

Convert first.

Example:

Clock period:

$$\lfloor 2\text{ns} \rfloor$$

Miss penalty:

$$\lfloor 50\text{ns} \rfloor$$

In cycles:

$$\lfloor 50/2=25 \rfloor$$

$$\lfloor \boxed{25\text{ cycles}} \rfloor$$

7.64 GATE Numerical — Time to Cycles

Clock frequency:

$$\lfloor 2\text{GHz} \rfloor$$

Clock period:

$$\lfloor 0.5\text{ns} \rfloor$$

Memory miss penalty:

$$\lfloor 50\text{ns} \rfloor$$

Penalty in cycles:

$$\lfloor 50/0.5=100 \rfloor$$

$$\lfloor \boxed{100\text{ cycles}} \rfloor$$

7.65 GATE Numerical — CPI with Time Conversion

Suppose:

- Base CPI = 1
 - Miss rate = 2%
 - 1.2 cache references/instruction
-

- Miss penalty = 40 ns
- CPU clock = 2 GHz

Clock period:

$$\left[\frac{1}{2 \text{ GHz}} \right]$$

Miss penalty cycles:

$$\left[\frac{40 \text{ ns}}{0.5 \text{ ns}} = 80 \right]$$

Stall cycles/instruction:

$$\left[1.2 \times 0.02 \times 80 \right]$$

$$\left[= 1.92 \right]$$

Actual CPI:

$$\left[1 + 1.92 \right]$$

$$\left[\boxed{2.92} \right]$$

7.66 Fast Exam Decision Tree

When solving memory-performance questions:

Step 1

Write:

TEXT

Hit rate = ?
Miss rate = 1 - hit rate

Step 2

Identify hierarchy:

TEXT

C a c h e ! ' R A M

or:

TEXT

L 1 ! ' L 2 ! ' R A M

Step 3

Ask:

Sequential or parallel?

Step 4

Write total time of every possible path.

Step 5

Use:

$$\boxed{\text{Average} = \sum \text{Probability} \times \text{PathTime}}$$

This method is almost impossible to misuse.

7.67 Shortcut Method

If conventional sequential cache hierarchy is clearly given:

One level

$$\boxed{\text{AMAT} = T_1 + m_1 P_1}$$

Two levels

$$\boxed{\text{AMAT} = T_1 + m_1 (T_2 + m_2 P_2)}$$

Three levels

$$\boxed{\text{AMAT} = T_1 + m_1 (T_2 + m_2 (T_3 + m_3 P_3))}$$

7.68 GATE Example — Reverse Miss Penalty

Given:

- $\text{AMAT} = 6 \text{ ns}$
- Cache hit time = 2 ns
- Miss rate = 4%

Find miss penalty.

$$6 = 2 + 0.04P$$

$$4 = 0.04P$$

$$P = 100 \text{ ns}$$

$$\boxed{100 \text{ ns}}$$

7.69 GATE Example — Reverse L1 Miss Rate

Given:

- $L1 = 1 \text{ ns}$
- $L2 \text{ access} = 5 \text{ ns}$
- assume L2 always hits
- $\text{AMAT} = 1.5 \text{ ns}$

$$1.5 = 1 + m_1(5)$$

$$0.5 = 5m_1$$

$$m_1 = 0.1$$

$$\boxed{10\%}$$

7.70 GATE Example — Three-Level

Given:

- L1 = 1 ns, miss 10%
- L2 = 4 ns, miss 20%
- L3 = 12 ns, miss 25%
- Memory penalty after L3 = 100 ns

$$\boxed{AMAT = 1 + 0.1 \left(4 + 0.2 \left(12 + 0.25(100) \right) \right)}$$

Inside:

$$\boxed{12 + 25 = 37}$$

Then:

$$\boxed{4 + 0.2(37) = 11.4}$$

Then:

$$\boxed{1 + 0.1(11.4)}$$

$$\boxed{2.14\text{ns}}$$

CHAPTER 7 — FORMULA SHEET

Hit ratio

$$\boxed{h = \frac{\text{Hits}}{\text{Total}}}$$

Miss ratio

$$\boxed{m = 1 - h}$$

One-level AMAT

$$\boxed{AMAT = T_c + mP}$$

Weighted average

$$\boxed{EAT = \sum p_i T_i}$$

Two-level AMAT

$$\boxed{AMAT = T_1 + m_1(T_2 + m_2P)}$$

Global miss rate after L2

$$\boxed{m_{\text{global}} = m_1 m_2}$$

CPI with memory stalls

$$\boxed{CPI_{\text{actual}} = CPI_{\text{base}} + \text{StallCycles/Instruction}}$$

Speedup

$$\boxed{\text{Speedup} = \frac{\text{Old Time}}{\text{New Time}}}$$

CHAPTER 7 — MEMORY TRICKS

Hit and Miss

$$H + M = 1$$

AMAT

Fast time always + misses × pain

$$[\boxed{ \text{HitTime} + \text{MissRate} \times \text{MissPenalty} }]$$

Multilevel

Keep nesting downward

TEXT

$$L1 + \text{miss1}(L2 + \text{miss2}(\text{memory}))$$

Local vs Global

Local = among arrivals

Global = among everyone

CHAPTER SNAPSHOT

1. Hit means requested data is found in cache.
 2. Miss means next memory level must be accessed.
 3. Hit rate + miss rate = 1.
 4. Hit time is cache lookup/access time.
 5. Miss penalty is extra cost caused by a miss.
 6. AMAT commonly equals hit time + miss rate × miss penalty.
 7. Weighted path probability is the safest general solution method.
 8. Sequential and parallel lookup produce different miss times.
 9. Multi-level AMAT is calculated recursively.
 10. Local miss rate is measured among accesses reaching that level.
 11. Global miss rate multiplies miss probabilities along the path.
 12. Cache misses add CPU stall cycles.
 13. Instruction fetch occurs once per instruction.
 14. Load/store fraction contributes additional data references.
 15. Time units and cycle units must be converted before combining.
-

MUST REMEMBER

$$\boxed{m=1-h}$$

$$\boxed{AMAT=T_c+mP}$$

$$\boxed{AMAT_{\{2-level\}} = T_1 + m_1(T_2 + m_2P)}$$

$$\boxed{Global\ Miss = m_1m_2}$$

And the safest universal method:

$$\boxed{Average = \sum Probability \times Total\ Path\ Time}$$

PRACTICE SET

Q1 — L0 NAT

Hit ratio = 96%.

Miss ratio = ?

Q2 — L1 NAT

Cache hit time = 2 ns.

Miss rate = 5%.

Additional miss penalty = 80 ns.

Find AMAT.

Q3 — L1 NAT

Cache = 5 ns.

Main memory = 100 ns.

Hit ratio = 90%.

Assume sequential lookup.

Find effective access time.

Q4 — L1 NAT

For Q3, assume cache and memory start in parallel.

Find effective access time.

Q5 — L2 NAT

L1:

- hit time = 1 ns
-

- miss rate = 10%

L2:

- hit time = 5 ns
- local miss rate = 20%

Main-memory penalty = 100 ns.

Find AMAT.

Q6 — L2 NAT

For Q5, what percentage of all CPU accesses reach main memory?

Q7 — L2 NAT

Cache hit time = 3 ns.

AMAT = 7 ns.

Miss penalty = 80 ns.

Find hit rate.

Q8 — L2

A processor has:

- Base CPI = 1
- I-cache miss rate = 2%
- D-cache miss rate = 5%
- 20% instructions access data
- Miss penalty = 40 cycles

Find actual CPI.

Q9 — L2 NAT

A processor runs at 2.5 GHz.

Memory miss penalty = 40 ns.

Find miss penalty in CPU cycles.

Q10 — L2 NAT

There are 1.4 cache references/instruction.

Miss rate = 3%.

Miss penalty = 50 cycles.

Base CPI = 1.

Find actual CPI.

Q11 — L3 NAT

L1:

- 1 ns
- miss rate 8%

L2:

- 6 ns
- local miss rate 25%

Main memory:

- 80 ns additional penalty.

Find AMAT.

Q12 — L3

A cache block miss requires:

- 60 ns latency until first word,
- block size = 8 words,
- each **additional** word requires 5 ns.

Find total block-fill time.

Q13 — L3

A two-level cache has:

- L1 local hit rate = 90%
- L2 local hit rate = 75%

Find:

1. global probability of L1 hit,
 2. global probability of L2 hit,
 3. global probability of main-memory access.
-

CHAPTER 7 COMPLETION STATUS

' Hit ' Miss ' Hit ratio ' Miss ratio ' Hit time ' Miss penalty ' AMAT ' Effective access time ' Weighted path method ' Sequential cache/memory lookup ' Parallel lookup ' Two-level cache ' Three-level cache ' Local miss rate ' Global miss rate ' Block-transfer penalty ' CPI/cache miss connection ' Memory stall cycles ' Speedup ' Reverse AMAT numericals ' GATE traps and shortcuts

CHAPTER 7: COMPLETE '

COURSE PROGRESS

Completed: 8 / 26 modules

Approximate roadmap progress:

$\boxed{\sim 31\%}$

NEXT !' CHAPTER 8 — MEMORY INTERFACING

Next we cover:

- memory chips,
- chip select,
- address decoding,
- word/depth expansion,
- bit/width expansion,
- mixed expansion,
- number of chips,
- address-line calculations,
- decoder size,
- address-range allocation,
- GATE-style memory interfacing numericals.

USER

next

ASSISTANT

CHAPTER 8 — MEMORY INTERFACING

Classification: [GATE CORE] **Priority:** ~~Ø=VERY~~ HIGH **Official syllabus connection:** Memory interfacing

This chapter is mainly about:

- how memory chips are connected to CPU,
- how many chips are required,
- how address lines are used,
- how chip-select works,
- how to expand memory in depth and width,
- how to find address ranges.

8.1 What is Memory Interfacing?

Memory interfacing means connecting memory chips to the processor so that the CPU can:

- select the correct memory chip,
- select the correct location inside the chip,

- read data,
- write data.

Simplified:

TEXT

Address lines

C P U % ! ' M e m o r y

Data lines

C P U ! " % ! ' M e m o r y

Control lines

C P U % ! ' M e m o r y
Read / Write

8.2 Three Important Connections

A memory chip mainly needs:

1. Address lines
2. Data lines
3. Control / Chip-select signals

Think:

TEXT

A d d r e s s ! ' W H E R E ?
D a t a ! ' W H A T ?
C o n t r o l ! ' R E A D o r W R I T E ?

8.3 Memory Chip Notation

A memory chip is often written as:

$[N \times M]$

Example:

$[2K \times 8]$

means:

- $(2K)$ locations,
- 8 bits per location.

Since:

$[2K = 2^{11}]$

this chip needs:

$[\boxed{11} \text{ address lines}]$

and:

$[\boxed{8} \text{ data lines}]$

8.4 Address Lines Inside a Chip

Suppose chip:

$8K \times 16$

Number of locations:

$8K = 2^{13}$

Therefore:

$\boxed{13 \text{ address lines}}$

Word width:

16 bits

Therefore:

$\boxed{16 \text{ data lines}}$

8.5 Chip Select — CS

Suppose the processor has multiple memory chips.

Only one intended chip should respond to a given address.

The **Chip Select** signal decides which chip is active.

Example:

TEXT

CPU Address

! "

Decoder

% % % % % % % %

! " ! " ! " ! "

C0 C1 C2 C3

Exactly one chip may be selected depending on the address.

8.6 Internal Address Bits vs Chip-Select Bits

Suppose:

- CPU has 16 address lines,
- each memory chip has $4K$ locations.

Since:

$4K = 2^{12}$

each chip needs 12 address bits internally.

Therefore:

TEXT

A 0 . . . A 1 1 ! ' s e l e c t l o c a t i o n i n s i d e c h i p

Remaining CPU address bits:

```
TEXT
A12 ... A15
```

can help select which chip or memory region is active.

Memory Trick

Low bits choose location. High bits choose chip.

This is one of the most important rules in memory interfacing.

8.7 Why Low-Order Bits Go Inside the Chip

Suppose a chip contains:

2^{10} locations.

The lower 10 address bits can vary through:

```
TEXT
0000000000
...
1111111111
```

and select all 1024 positions inside that chip.

Higher address bits remain constant over that chip's address range and are useful for selecting the chip.

8.8 Word Expansion / Depth Expansion

is important.

Word expansion or **depth expansion** increases:

number of memory locations

while keeping word width unchanged.

Example:

Available chip:

$2K \times 8$

Required memory:

$8K \times 8$

Width is already 8 bits.

Depth must increase from 2K to 8K.

Number of chips:

$$\lceil \frac{8K}{2K} \rceil = 4$$

Therefore:

$$\boxed{4 \text{ chips}}$$

8.9 Depth Expansion Diagram

TEXT

Required: 8K × 8
Chip: 2K × 8

		% % % % % % % % % %
B a n k	0	% 2 K × 8 %
		% % % % % % % % % %
B a n k	1	% 2 K × 8 %
		% % % % % % % % % %
B a n k	2	% 2 K × 8 %
		% % % % % % % % % %
B a n k	3	% 2 K × 8 %
		% % % % % % % % % %

Each bank provides another 2K addresses.

8.10 Address Lines in Depth Expansion

For each (2K) chip:

$$2K = 2^{11}$$

So 11 low-order address bits go to every chip.

Required total memory:

$$8K = 2^{13}$$

Therefore total address bits required:

$$13$$

Extra bits for bank selection:

$$13 - 11 = 2$$

Two bits can select:

$$2^2 = 4$$

chips.

Exactly what we need.

8.11 Decoder for Depth Expansion

Two selection bits can feed a:

$$\boxed{2\text{-to-4 decoder}}$$

Example:

```

TEXT
A12 A11
    !"
2-to-4 Decoder
    !"    !"    !"    !"
CS0 CS1 CS2 CS3

```

Each output selects one chip.

8.12 Bit Expansion / Width Expansion

Bit expansion or width expansion increases:

number of bits stored at each address

while keeping number of locations unchanged.

Example:

Available chip:

$$\backslash[2K\times 8\backslash]$$

Required:

\[2K\times32 \]

Need:

$$\lfloor 32/8=4 \rfloor$$

chips in parallel.

Therefore:

$$\boxed{4 \text{ chips}}$$

8.13 Width Expansion Diagram

Each chip stores a different portion of the same word:

[illegible]

All four chips receive the same address.

Together they produce:

 $\lfloor 32 \text{ bits} \rfloor$

8.14 Important Difference

Depth expansion

Chips are selected one at a time.

Width expansion

Multiple chips work simultaneously.

Memory trick:

Depth !' one bank at a time

Width !' chips in parallel

8.15 Mixed Expansion

Frequently GATE asks both depth and width expansion together.

Example:

Available chip:

$$\sqrt[2]{2K \times 8}$$

Required memory:

$$\sqrt[2]{8K \times 32}$$

Step 1 — Depth Factor

$$\sqrt[2]{\frac{8K}{2K}} = 4$$

Step 2 — Width Factor

$$\sqrt[2]{\frac{32}{8}} = 4$$

Step 3 — Total Chips

$$4 \times 4 = 16$$

Therefore:

$$\boxed{16 \text{ chips}}$$

8.16 MASTER CHIP FORMULA

If required memory:

$$\sqrt[2]{N_R \times M_R}$$

and available chip:

$$\lfloor N_C \times M_C \rfloor$$

then:

$$\lfloor \boxed{\text{Number of chips}} = \frac{N_R}{N_C} \times \frac{M_R}{M_C} \rfloor$$

provided the ratios are integral.

8.17 Example 1

Construct:

$$\lfloor 64K \times 16 \rfloor$$

using:

$$\lfloor 16K \times 4 \rfloor$$

chips.

Depth factor:

$$\lfloor 64K/16K=4 \rfloor$$

Width factor:

$$\lfloor 16/4=4 \rfloor$$

Total:

$$\lfloor 4 \times 4 \rfloor$$

$$\lfloor \boxed{16} \rfloor$$

chips.

8.18 Example 2

Construct:

$$\lfloor 32K \times 8 \rfloor$$

using:

$$\lfloor 8K \times 8 \rfloor$$

chips.

Depth:

$$\lfloor 32K/8K=4 \rfloor$$

Width:

$$\lfloor 8/8=1 \rfloor$$

Total:

$$\lfloor \boxed{4} \rfloor$$

8.19 Example 3

Construct:

$$\lfloor 4K \times 32 \rfloor$$

using:

$$\lfloor 4K \times 8 \rfloor$$

chips.

Depth:

$$\lfloor 1 \rfloor$$

Width:

$$\lfloor 32/8=4 \rfloor$$

Total:

$$\lfloor \boxed{4} \rfloor$$

chips.

8.20 Address Lines for Required Memory

Suppose required memory:

$$\lfloor 64K \times 16 \rfloor$$

Since:

$$\lfloor 64K=2^{16} \rfloor$$

number of address lines for the entire memory:

$$\lfloor \boxed{16} \rfloor$$

Data lines:

$$\lfloor \boxed{16} \rfloor$$

8.21 Address Lines Per Chip

If chip is:

$$\lfloor 16K \times 4 \rfloor$$

then:

$$\lfloor 16K=2^{14} \rfloor$$

Each chip has:

$$\lfloor \boxed{14} \text{ address inputs} \rfloor$$

Two additional address bits are needed to select among four depth banks.

8.22 Bank Structure in Mixed Expansion

For:

$16K \times 16$

using:

$16K \times 4$

chips:

Width needs 4 chips per bank:

TEXT

16K × 4

16K × 4

16K × 4

16K × 4

16K × 16

Depth needs 4 such banks:

TEXT

Bank 0 !' 16K × 16

Bank 1 !' 16K × 16

Bank 2 !' 16K × 16

Bank 3 !' 16K × 16

Total:

4×16

chips.

8.23 Decoder Size

Suppose depth expansion requires 8 banks.

Need:

$\log_2 8 = 3$

selection bits.

Use:

$\boxed{3 \text{ text-to-8 decoder}}$

General:

If number of banks is:

2^k

then:

$\boxed{k \text{ text-to-} 2^k \text{ decoder}}$

8.24 Example — Decoder

Required:

$16K \times 8$

using:

$2K \times 8$

chips.

Depth banks:

$16K / 2K = 8$

Need 3 selection bits.

Decoder:

$\boxed{3\text{-to-}8}$

8.25 CPU Address Width vs Memory Address Width

Suppose CPU has 20 address bits.

Then CPU can generate:

2^{20}

addresses.

But suppose attached memory is only:

$64K = 2^{16}$

locations.

Then 16 bits choose locations inside memory.

The remaining 4 address bits determine:

- which larger region,
- which chip,
- or whether address maps to another device.

8.26 Address Range Allocation

Suppose a byte-addressable memory chip has:

$4K = 2^{12}$

locations.

If it starts at:

TEXT

0000H

then it occupies 4096 addresses.

Since:

$4096 = 1000_H$

range is:

TEXT

0000H to 0FFFH

Because:

$\lfloor 0FFF_H - 0000_H + 1 = 1000_H \rfloor$

8.27 Next Memory Block

Next 4K block starts at:

TEXT

1000H

and ends at:

TEXT

1FFFH

Then:

TEXT

2000H to 2FFFH

etc.

This pattern is extremely useful.

8.28 Example — 8K Address Range

An 8K byte memory needs:

$\lfloor 8K = 2^{\{13\}} \rfloor$

addresses.

In hexadecimal:

$\lfloor 2000_H \rfloor$

addresses.

If starting address is:

TEXT

4000H

ending address:

$\lfloor 4000_H + 2000_H - 1 \rfloor$

$\lfloor = \boxed{5FFF_H} \rfloor$

8.29 MASTER ADDRESS-RANGE FORMULA

If:

- starting address = S
- memory size in addressable locations = N

then:

$$\boxed{\text{End} = S + N - 1}$$

Conversely:

$$\boxed{N = \text{End} - \text{Start} + 1}$$

8.30 Example — Find Memory Size

Range:

TEXT

8000H to BFFFH

Number of locations:

$$\boxed{\text{BFFF}_H - 8000_H + 1}$$

Difference:

$$\boxed{4000_H}$$

$$\boxed{= 16K}$$

Thus byte-addressable capacity:

$$\boxed{16KB}$$

8.31 High-Order Address Decoding

Suppose 16-bit CPU addresses and each chip is 4K.

A 4K chip uses:

$$\boxed{12}$$

low bits:

TEXT

A0 - A11

High 4 bits:

TEXT

A12 - A15

can define regions.

For example:

TEXT

```
A 1 5   A 1 4   A 1 3   A 1 2   =   0 0 0 0   !'   0 0 0 0 H - 0 F F F H
A 1 5   A 1 4   A 1 3   A 1 2   =   0 0 0 1   !'   1 0 0 0 H - 1 F F F H
A 1 5   A 1 4   A 1 3   A 1 2   =   0 0 1 0   !'   2 0 0 0 H - 2 F F F H
...
```

8.32 Full Address Decoding

Full decoding uses all required high-order address bits to uniquely select the memory region.

Benefit:

each memory location has one unique processor address.

8.33 Partial Address Decoding

[SUPPORT]

If some high-order bits are ignored in chip selection, the same physical memory may appear at multiple addresses.

This is called **address aliasing** or mirroring.

Example idea:

TEXT

0000H and 8000H

might access same physical chip if A15 is ignored.

For GATE, know the concept.

8.34 Address Aliasing

If not all address bits participate in decoding:

multiple logical addresses map to same physical location.

This is usually undesirable unless intentional.

Memory trick:

Ignored high bits !' repeated/mirrored addresses.

8.35 Read and Write Control

Memory interfaces also receive signals such as:

```
TEXT
READ
WRITE
```

or:

```
TEXT
OE = Output Enable
WE = Write Enable
```

Exact signal names depend on chip.

8.36 ROM vs RAM Interfacing

ROM only needs read-related operation in normal usage.

RAM supports:

- read,
- write.

Therefore RAM interface generally includes a write-enable control.

This is supportive knowledge.

8.37 Data Bus Connection in Width Expansion

Suppose CPU data bus is 16 bits and memory chips are 8-bit wide.

Need two chips in parallel.

```
TEXT
C h i p   0   ! '   D 0 - D 7
C h i p   1   ! '   D 8 - D 1 5
```

Both chips use same address and are enabled simultaneously.

Together they supply one 16-bit word.

8.38 Address Bus Connection in Width Expansion

All width-parallel chips receive identical address lines.

Example:

Two $(4K \times 8)$ chips make $(4K \times 16)$.

Both receive:

```
TEXT
A0-A11
```

because both need to access the same word number at the same time.

8.39 Chip Select in Width Expansion

Chips forming the same bank are usually selected together.

Example:

TEXT

CS Bank0

! "

```
% % % % % % % % % % % % % % % % % % % %  
% 4 K x 8 % % 4 K x 8 %  
% % % % % % % % % % % % % % % % % % % %  
low byte high byte
```

Both contribute to one 16-bit word.

8.40 Chip Select in Depth Expansion

Different depth banks are mutually exclusive.

Example:

TEXT

Decoder

! " ! " ! " ! "

B0 B1 B2 B3

Only selected bank responds.

8.41 GATE Numerical — Mixed Expansion

Build:

$\lfloor 256K \times 32 \rfloor$

using:

$\lfloor 64K \times 8 \rfloor$

chips.

Depth factor:

$\lfloor 256K / 64K = 4 \rfloor$

Width factor:

$\lfloor 32 / 8 = 4 \rfloor$

Total chips:

$\boxed{16}$

Each bank contains 4 chips.

There are 4 banks.

8.42 Address Bits in Previous Example

Required:

$$\lceil 256K = 2^{18} \rceil$$

Total address bits:

$$\lceil 18 \rceil$$

Each chip:

$$\lceil 64K = 2^{16} \rceil$$

uses:

$$\lceil 16 \rceil$$

internal address bits.

Extra bits:

$$\lceil 18 - 16 = 2 \rceil$$

Those 2 bits select one of 4 banks.

8.43 Decoder in Previous Example

2 selection bits:

$$\lceil 2^2 = 4 \rceil$$

banks.

Therefore:

$$\lceil \boxed{2\text{-to-}4\text{ decoder}} \rceil$$

8.44 GATE Numerical — More Complex

Required memory:

$$\lceil 1M \times 32 \rceil$$

Available chip:

$$\lceil 256K \times 8 \rceil$$

Depth factor:

$$\lceil 1M / 256K = 2^{20} / 2^{18} = 4 \rceil$$

Width factor:

$$\lceil 32 / 8 = 4 \rceil$$

Total:

$$\lceil \boxed{16\text{-text{ chips}}} \rceil$$

Again 4 banks $\times$ 4 chips/bank.

8.45 GATE Numerical — Nontrivial Width

Required:

$$\lfloor 16K \times 24 \rfloor$$

Chip:

$$\lfloor 4K \times 8 \rfloor$$

Depth:

$$\lfloor 16K / 4K = 4 \rfloor$$

Width:

$$\lfloor 24 / 8 = 3 \rfloor$$

Total:

$$\lfloor 4 \times 3 \rfloor$$

$$\lfloor \boxed{12 \text{ chips}} \rfloor$$

8.46 GATE Numerical — Chip Capacity Check

Required:

$$\lfloor 8K \times 16 \rfloor$$

Total bits:

$$\lfloor 8K \times 16 \rfloor$$

Chip:

$$\lfloor 2K \times 4 \rfloor$$

Total bits/chip:

$$\lfloor 2K \times 4 \rfloor$$

Capacity-ratio method:

$$\lfloor \frac{8K \times 16}{2K \times 4} = 4 \times 4 = 16 \rfloor$$

Same answer.

But depth/width method is safer because it also tells you organization.

8.47 Why Total Capacity Ratio Alone Can Be Dangerous

Suppose required organization and chip organization differ.

Capacity ratio gives number of chips, but it may hide:

- number of banks,
- number of chips in parallel,
- decoder requirements.

For GATE, use:

$$\lfloor \boxed{\text{Depth factor} \times \text{Width factor}} \rfloor$$

8.48 Address Range + Chip Selection Example

A 16-bit address space has 4K memory chips.

Each chip occupies:

$\lfloor 1000_H \rfloor$

addresses.

Suppose chip 3 is mapped to:

TEXT

3000H – 3FFFH

High nibble:

TEXT

0011

can identify that region.

8.49 Decoder Input Calculation

If CPU has:

- 20 address bits,
- each chip has $\lfloor 64K=2^{\{16\}} \rfloor$ locations,

then 16 low-order bits go inside chip.

Remaining:

$\lfloor 20-16=4 \rfloor$

high-order bits are available for chip/region decoding.

At most:

$\lfloor 2^4=16 \rfloor$

such non-overlapping 64K blocks fit into full address space.

8.50 Maximum Number of Identical Chips in Address Space

General:

CPU address bits = $\lfloor A \rfloor$

Chip location bits = $\lfloor C \rfloor$

Then number of chip-sized regions:

$\lfloor \boxed{2^{\{A-C\}}} \rfloor$

assuming same addressability and one chip provides full required word width.

Example

CPU:

$[24\text{-bit address}]$

Chip:

$[1\text{MB}=2^{20}\text{ bytes}]$

Number of 1MB regions:

$[2^{24-20}=16]$

$[16]$

8.51 Data Width Complication

Suppose CPU expects 16-bit words but chip is only 8 bits wide.

Then one logical memory region may require two physical chips in parallel.

So address-space-region count and physical-chip count are not always identical.

8.52 Byte Enable — Support Concept

[LOW PRIORITY]

In wider memories, processors may have signals controlling individual byte lanes.

Example 32-bit word:

TEXT

Byte 0
Byte 1
Byte 2
Byte 3

Byte-enable signals allow writing only selected bytes.

Useful practical concept, but low priority for your GATE syllabus.

8.53 Interleaved Memory — Brief Support

[LOW PRIORITY]

Memory may be split across banks so consecutive addresses are distributed among banks.

This can increase bandwidth.

Example:

TEXT

A d d r e s s	0	!	B a n k	0
A d d r e s s	1	!	B a n k	1
A d d r e s s	2	!	B a n k	2
A d d r e s s	3	!	B a n k	3

Do not spend much time here unless a question explicitly asks it.

8.54 Full Worked Example

CPU has:

- 18 address lines,
- 16-bit data bus.

We want to build full memory using $(16K \times 8)$ chips.

Step 1 — Total addressable words

Assume word-addressed by 16-bit word:

$$2^{18} \text{ words}$$

Required organization:

$$256K \times 16$$

Step 2 — Chip organization

$$16K \times 8$$

Step 3 — Depth

$$256K / 16K = 16$$

Step 4 — Width

$$16 / 8 = 2$$

Step 5 — Total chips

$$16 \times 2$$

$$\boxed{32}$$

Step 6 — Address bits per chip

$$16K = 2^{14}$$

14 low bits select locations.

Extra:

$$18 - 14 = 4$$

bits select one of 16 banks.

Decoder:

$$\boxed{4 \text{ to } 16}$$

8.55 Byte-Addressability Trap in Interfacing

Suppose CPU has:

- 20-bit **byte address**,
- 16-bit word memory.

Do NOT automatically say:

2^{20} words

The CPU addresses bytes.

Total bytes:

2^{20}

If each word = 2 bytes:

2^{19}

words.

Always respect addressability.

8.56 Example — Byte-Addressed Wide Memory

CPU:

- 20-bit byte address
- memory word = 32 bits = 4 bytes

Total bytes:

2^{20}

Total 32-bit words:

$2^{20}/4 = 2^{18}$

So memory organization:

$256K \times 32$

if organized in 32-bit words.

8.57 GATE Trap — Address Lines Required by Chip

Chip:

$8K \times 32$

needs:

13

address lines, not 15 or 32.

Why?

Because address lines depend on number of locations:

$8K = 2^{13}$

32 is data width.

8.58 GATE Trap — Data Lines Required

Chip:

$[8K \times 32]$

needs:

$[32]$

data lines.

Do not use:

$[\log_2 32 = 5]$

Data width is literal number of bits transferred, not number of selectable values.

8.59 GATE Trap — Decoder Inputs

If 8 chips form one width-expanded bank, you may not need 3 selector bits for those 8 chips.

Why?

Because all 8 may be selected simultaneously.

Decoder inputs depend on **number of depth banks**, not total physical chips.

~~0=Very~~ important.

Example

Required:

$[4K \times 64]$

Chip:

$[4K \times 8]$

Need 8 chips.

But:

Depth banks:

$[1]$

All 8 chips work in parallel.

No 3-to-8 decoder is needed merely because there are 8 physical chips.

8.60 GATE Trap — Total Chips vs Banks

Suppose 16 chips are needed:

- 4 chips/bank,
- 4 banks.

Decoder selects:

$\lceil \frac{\text{banks}}{4} \rceil$

Therefore:

$\lceil \frac{\text{to}}{4} \rceil$

not:

$\lceil \frac{\text{to}}{16} \rceil$

unless some different interface arrangement is specified.

8.61 GATE Trap — Start and End Address

Memory size 4K means:

4096

addresses.

If start:

TEXT

2000H

end is:

$2000_H + 1000_H - 1$

$\boxed{2FFF_H}$

not 3000H.

Always subtract 1.

8.62 GATE Trap — Partial Decoding

If a high-order bit is unused in CS logic, address aliasing may occur.

Do not assume a physical memory occupies only one logical range unless full decoding is used.

8.63 FAST GATE SOLVING METHOD

Whenever you see:

"Construct memory X using chips Y"

write only these four lines first:

1. Depth factor

$D = \frac{\text{required locations}}{\text{chip locations}}$

2. Width factor

$W = \frac{\text{required word width}}{\text{chip width}}$

3. Chips

$$\boxed{D \times W}$$

4. Decoder

If depth factor:

$$D = 2^k$$

then:

$$\boxed{k \text{ to } D}$$

decoder.

8.64 Worked Speed Example

Construct:

$$128K \times 32$$

using:

$$32K \times 8$$

chips.

Depth:

$$128/32 = 4$$

Width:

$$32/8 = 4$$

Total:

$$\boxed{16}$$

Banks:

$$4$$

Chips/bank:

$$4$$

Decoder:

$$\boxed{2 \text{ to } 4}$$

Address bits required:

Required:

$$128K = 2^{17}$$

Chip:

$$32K = 2^{15}$$

Low 15 bits go to chips.

High 2 select bank.

Done.

8.65 MASTER SUMMARY TABLE

Requirement	Formula / Idea
Chip address lines	$\lceil \log_2(\text{chip locations}) \rceil$
Required memory address lines	$\lceil \log_2(\text{required locations}) \rceil$
Depth factor	Required locations / chip locations
Width factor	Required width / chip width
Total chips	Depth $\times$ Width
Decoder inputs	$\lceil \log_2(\text{number of banks}) \rceil$
Chips per bank	Width factor
Number of banks	Depth factor
End address	Start + size - 1

CHAPTER 8 — MEMORY TRICKS

Memory construction

$$D \times W = \text{Chips}$$

Depth factor Width factor

Decoder

Decoder chooses banks, not bytes.

Address bits

Low !' Location

High !' Hardware/Bank selection

Range

$$\text{Start} + \text{Size} - 1$$

CHAPTER SNAPSHOT

1. Memory interfacing connects address, data and control signals.

2. Chip select activates the required memory chip/bank.
3. Low-order address bits generally select locations inside a chip.
4. High-order bits commonly select banks/chips.
5. Depth expansion increases number of locations.
6. Width expansion increases bits per memory word.
7. Chips required = depth factor $\times$ width factor.
8. Width-expanded chips operate in parallel.
9. Depth-expanded banks are selected separately.
10. Decoder size depends on number of banks.
11. $(2K)$ locations need 11 address bits.
12. Data lines equal chip word width.
13. Address range uses $\text{End} = \text{Start} + \text{Size} - 1$.
14. Full decoding gives unique address ranges.
15. Partial decoding can create address aliases.
16. Addressability must be considered before deriving memory organization.

MUST REMEMBER

$$\boxed{\text{Depth Factor} = \frac{\text{Required Locations}}{\text{Chip Locations}}}$$

$$\boxed{\text{Width Factor} = \frac{\text{Required Width}}{\text{Chip Width}}}$$

$$\boxed{\text{Total Chips} = \text{Depth} \times \text{Width}}$$

$$\boxed{\text{End} = \text{Start} + \text{Size} - 1}$$

PRACTICE SET

Q1 — L0

A $(4K \times 8)$ chip requires how many address lines?

Q2 — L0

A $(4K \times 8)$ chip requires how many data lines?

Q3 — L1 NAT

Construct:

$$16K \times 8$$

using:

$$4K \times 8$$

chips.

Number of chips = ?

Q4 — L1 NAT

Construct:

$\sqrt[4]{4K \times 32}$

using:

$\sqrt[4]{4K \times 8}$

chips.

Number of chips = ?

Q5 — L2 NAT

Construct:

$\sqrt[4]{32K \times 16}$

using:

$\sqrt[4]{8K \times 4}$

chips.

Find total number of chips.

Q6 — L2

For Q5, determine:

1. number of depth banks,
 2. chips per bank,
 3. decoder required.
-

Q7 — L2

A memory of 8K bytes begins at:

TEXT

4000H

Find ending address.

Q8 — L2

Find the size of the byte-addressable region:

TEXT

8000H – BFFFH

in KB.

Q9 — L2 NAT

A CPU has 20 address bits. Each memory chip contains 64K locations.

How many CPU address bits remain for chip/bank selection?

Q10 — L2

Required memory:

$[128K \times 32]$

Chip:

$[32K \times 8]$

Find:

- total chips,
 - banks,
 - chips per bank,
 - decoder.
-

Q11 — L3

A byte-addressable processor has a 22-bit address space and uses 32-bit memory words.

How many 32-bit words exist in its full address space?

Q12 — L3

A system uses:

$[16K \times 8]$

chips to build:

$[64K \times 32]$

memory.

Find:

1. total chips,
 2. number of banks,
 3. chips per bank,
 4. number of low-order address bits connected to every chip,
 5. number of bank-select bits,
 6. decoder size.
-

Q13 — L3 Conceptual

Why would 8 chips used for width expansion not necessarily require a 3-to-8 decoder?

CHAPTER 8 COMPLETION STATUS

' Memory interface basics ' Address/data/control lines ' Chip select ' Internal vs high-order address bits ' Depth/word expansion ' Width/bit expansion ' Mixed expansion ' Number-of-chip formula ' Banks and chips per bank ' Decoder calculation ' Address-range allocation ' Hexadecimal ranges ' Full decoding ' Partial decoding / aliasing ' Byte-addressability considerations ' GATE traps and shortcuts

CHAPTER 8: COMPLETE '

COURSE PROGRESS

Completed: 9 / 26 modules

Approximate roadmap progress:

$\lfloor \boxed{\sim 35\%} \rfloor$

We have now completed the **memory interfacing foundation**.

NEXT !' CHAPTER 9 — CACHE MEMORY FUNDAMENTALS

Next begins one of the **highest-yield GATE COA topics**:

Ø=100% the block / cache line Ø=100% index and offset Ø=100% number of lines Ø=100% block number Ø=100% address decomposition Ø=100% byte vs word offset Ø=100% the mapping foundation

After that we will separately master:

Direct Mapping !' Fully Associative !' Set Associative.

USER

next

ASSISTANT

CHAPTER 9 — CACHE MEMORY FUNDAMENTALS

Classification: [GATE CORE] **Priority:** Ø=100% VERY HIGH **Official syllabus connection:** Memory hierarchy + cache memory mapping

This chapter is the foundation for the three major mapping techniques:

- Direct Mapping
- Fully Associative Mapping
- Set-Associative Mapping

9.1 Why Do We Need Cache?

CPU is much faster than main memory.

If CPU had to wait for RAM every time:

TEXT

```
C P U ! '   r e q u e s t ! '   R A M ! '   w a i t ! '   d a t a
```

performance would suffer badly.

So we place a small fast memory between CPU and RAM:

TEXT

CPU

! "

Cache

! "

Main Memory

Cache keeps copies of recently/frequently used memory blocks.

9.2 Why Cache Works

Because programs show **locality of reference**.

We already learned:

Temporal locality

Same data may be used again soon.

Spatial locality

Nearby data may be used soon.

Therefore when CPU asks for one location, cache usually brings a **block** of nearby memory.

9.3 What is a Block?

Main memory is divided into fixed-size groups called:

$\boxed{\text{Blocks}}$

Example:

Suppose block size = 16 bytes.

Then:

TEXT

```
B l o c k   0   ! '   b y t e s   0 - 1 5
B l o c k   1   ! '   b y t e s   1 6 - 3 1
B l o c k   2   ! '   b y t e s   3 2 - 4 7
B l o c k   3   ! '   b y t e s   4 8 - 6 3
...
```

9.4 Cache Line

Cache is divided into entries called:

$\boxed{\text{Cache Lines}}$

or cache blocks.

One cache line stores one main-memory block.

Example:

TEXT

Main Memory Block

!"

Cache Line

So:

Memory has blocks. Cache has lines.

In many questions the terms "cache block" and "cache line" are used almost interchangeably.

9.5 Cache Size

Suppose:

- Cache size = 32 KB
- Block size = 64 bytes

Number of cache lines:

$\boxed{\frac{\text{Cache Size}}{\text{Block Size}}}$

Thus:

$\frac{32\text{KB}}{64\text{B}}$

$= \frac{2^{15}}{2^6}$

$= 2^9$

$\boxed{512 \text{ lines}}$

9.6 MASTER FORMULA — Number of Lines

$\boxed{\text{Cache Lines} = \frac{\text{Cache Capacity}}{\text{Block Size}}}$

Memorize this.

9.7 Main Memory Blocks

Suppose:

- Main memory = 1 MB
- Block size = 64 bytes

Number of main-memory blocks:

$$\lceil \frac{2^{20}}{2^6} \rceil$$

$$\lceil 2^{14} \rceil$$

$$\boxed{16384 \text{ blocks}}$$

MASTER FORMULA

$$\boxed{\text{Main Memory Blocks} = \frac{\text{Main Memory Capacity}}{\text{Block Size}}}$$

9.8 Why Mapping Is Needed

Usually:

$$\text{Main-memory blocks} \gg \text{cache lines}$$

Example:

TEXT

```
Main memory blocks = 16384
Cache lines        = 512
```

So many memory blocks must share limited cache lines.

Question:

Which memory block can go into which cache line?

That rule is called:

$$\boxed{\text{Cache Mapping}}$$

9.9 Three Cache Mapping Techniques

We will study:

1. Direct Mapping
2. Fully Associative Mapping
3. Set-Associative Mapping

Simplified:

TEXT

Direct

! ' o n e p o s s i b l e l i n e

Fully Associative

! ' a n y l i n e

Set Associative

! ' a n y l i n e w i t h i n o n e s e t

Memory trick:

Direct = One

Associative = Anywhere

Set = Somewhere in one set

9.10 CPU Address and Cache

Suppose CPU generates a memory address.

Cache must determine:

1. Which cache line/set to inspect?
2. Whether correct memory block is there?
3. Which byte/word inside the block is needed?

Therefore an address is divided into fields.

Typical direct-mapped address:

TEXT

Tag	Index	Offset
4	4	4

Memory trick:

$$\boxed{\text{T-I-O}}$$

Tag !' Index !' Offset

9.11 What is Offset?

The **offset** identifies the required location inside a block.

Example:

Block size:

\[16\text{ bytes} \]

A block contains positions:

TEXT

 $0, 1, 2, \dots, 15$

How many bits are required?

$$\lceil \log_2 16 = 4 \rceil$$

Therefore:

$$\boxed{4 \text{ offset bits}}$$

MASTER FORMULA — Byte Offset

For byte-addressable memory:

$$\boxed{\text{Offset Bits} = \log_2(\text{Block Size in Bytes})}$$

9.12 Example — Offset Bits

Block size = 64 bytes.

$$64 = 2^6$$

Therefore:

$$\boxed{6 \text{ offset bits}}$$

9.13 What is Index?

In direct mapping, the **index** identifies which cache line to check.

Suppose cache has:

$$512 = 2^9$$

lines.

Then:

$$\boxed{9 \text{ index bits}}$$

MASTER FORMULA — Index Bits

For direct mapping:

$$\boxed{\text{Index Bits} = \log_2(\text{Number of Cache Lines})}$$

9.14 What is Tag?

Many main-memory blocks can map to the same cache line.

So the cache must know:

Which one of those blocks is currently stored here?

The **tag** identifies that memory block.

Address:

TEXT

Tag | Index | Offset

Index says:

check this line.

Tag says:

verify whether this line contains my block.

Offset says:

select the exact byte/word.

9.15 Tag Bits Formula

If total address bits = A , then:

$$\boxed{\text{Tag} = A - \text{Index} - \text{Offset}}$$

for direct mapping.

9.16 Complete Direct-Mapped Example

Suppose:

- Main memory = 4 GB
- Byte-addressable
- Cache = 32 KB
- Block size = 64 bytes

Step 1 — Address bits

$$4\text{GB} = 2^{32} \text{ bytes}$$

So:

$$A = 32$$

Step 2 — Cache lines

$$32\text{KB} = 2^{15}$$

Block:

$$64\text{B} = 2^6$$

Lines:

$$2^{15} / 2^6 = 2^9$$

So:

$$\text{Index} = 9$$

Step 3 — Offset

$$\text{Offset} = 6$$

Step 4 — Tag

\[Tag=32-9-6 \]

$$\boxed{17}$$

Address format:

TEXT

[illegible]

9.17 Why Offset Comes from Block Size

Suppose block size = 8 bytes.

Within one block:

TEXT

```
byte 0
byte 1
byte 2
...
byte 7
```

Need:

$$\lfloor 3 \text{ bits} \rfloor$$

because:

 $\backslash [2^3=8 \ \backslash]$

Thus offset tells us exact byte inside the fetched block.

9.18 Block Number

Given byte address **A** and block size **B** bytes:

$$\boxed{\text{Block Number} = \left\lfloor \frac{A}{B} \right\rfloor}$$

and:

$$\boxed{\text{Offset} = A \bmod B}$$

Ø=Important.

Example

Address:

 $\backslash [100 \backslash]$

Block size:

\[16\text{ bytes} \]

Block number:

$$\lfloor \frac{100}{16} \rfloor = 6$$

because:

$$6 \times 16 = 96$$

Offset:

$$100 - 96 = 4$$

So:

$$\boxed{\text{Block}=6, \text{Offset}=4}$$

9.19 Fast Binary Interpretation

If block size:

$$2^b$$

bytes, then lowest b address bits are offset.

Example:

$$\text{Block size} = 16 \text{ bytes} = 2^4.$$

So lowest 4 address bits = offset.

Remaining upper bits represent block number.

9.20 Byte Addressable Cache

Suppose memory is byte-addressable.

Then every CPU address refers to:

$$1 \text{ text{ byte} }$$

Thus if block size = 32 bytes:

$$\text{Offset} = \log_2 32 = 5 \text{ text{ bits} }$$

Straightforward.

9.21 Word-Addressable Cache

Important trap.

Suppose memory is **word-addressable**.

Each address identifies one entire word.

Example:

- word size = 4 bytes
- block contains 8 words

If addresses refer to words, offset only needs to select among:

$$8 \text{ text{ words} }$$

So:

$\lceil \log_2 \text{word-offset bits} \rceil$

not 5.

9.22 Byte-Addressed Memory with Multi-Byte Words

Suppose:

- byte-addressable memory,
- word size = 4 bytes,
- block = 8 words.

Block size:

$8 \times 4 = 32$ bytes

Total block-offset bits:

$\lceil \log_2 32 \rceil = 5$

These can conceptually be split:

TEXT

Word Offset | Byte Offset

Word offset:

$\lceil \log_2 8 \rceil = 3$

Byte offset within each 4-byte word:

$\lceil \log_2 4 \rceil = 2$

Total:

$3 + 2 = 5$

9.23 Word Offset vs Byte Offset

Suppose:

- block has 16 words,
- each word = 4 bytes,
- byte-addressable.

Then:

Word offset:

$\lceil \log_2 16 \rceil = 4$

Byte offset:

$\lceil \log_2 4 \rceil = 2$

Total offset:

$\boxed{6}$

Equivalent block size:

$\lceil 16 \times 4 = 64B \rceil$

$\lceil \log_2 64 = 6 \rceil$

Same result.

9.24 Cache Line Contents

A cache line usually contains more than just data.

Conceptually:

TEXT

```
% % % % % % % %, % % % % % % %, % % % % % % % % % % % % % % % %  
%   v a l i d   %       T a g       %   D a t a   B l o c k       %  
% % % % % % % %4% % % % % % % %4% % % % % % % % % % % % % % % %
```

Write-back caches may also contain:

TEXT

```
Dirty bit
```

We'll study that later.

9.25 Valid Bit

How does cache know whether a line contains meaningful data?

Use a:

$\boxed{\text{Valid Bit}}$

Typically:

TEXT

```
v a l i d   =   1   ! '   l i n e   c o n t a i n s   u s a b l e   c a c h e   d a t a  
v a l i d   =   0   ! '   l i n e   s h o u l d   n o t   b e   t r e a t e d   a s   a   h i t
```

9.26 Why Valid Bit Is Needed

At startup, cache may contain arbitrary old bit patterns.

Tag comparison alone is not enough.

We also require:

$\lceil \text{Valid}=1 \rceil$

for a hit.

9.27 Cache Hit Test

For a direct-mapped cache:

1. Use index to select line.
2. Check valid bit.
3. Compare stored tag with address tag.
4. If both match !' hit.
5. Offset selects required data.

Condition:

$$\boxed{\text{Hit} = (\text{Valid} = 1) \wedge (\text{Tag}_{\text{stored}} = \text{Tag}_{\text{address}})}$$

9.28 Cache Miss Types — Intro

[SUPPORT, but useful]

Three classic cache miss categories:

1. Compulsory miss
2. Capacity miss
3. Conflict miss

9.29 Compulsory Miss

First time a block is ever accessed, it cannot already be in an initially empty cache.

This is:

$$\boxed{\text{Compulsory / Cold Miss}}$$

Memory trick:

First time = compulsory

9.30 Capacity Miss

Cache is too small to contain all blocks required by the program.

Even with ideal mapping, some blocks must be evicted.

This causes:

$$\boxed{\text{Capacity Miss}}$$

9.31 Conflict Miss

Especially in direct/set-associative caches.

Multiple memory blocks compete for the same line/set even though other cache space may exist.

This is:

$$\boxed{\text{Conflict Miss}}$$

Memory trick:

Same destination != conflict

9.32 Mapping and Miss Types

Fully associative mapping eliminates conflict misses caused purely by fixed placement.

But:

- compulsory misses remain,
- capacity misses remain.

Useful conceptual result.

9.33 Cache Metadata

Cache size can mean different things.

Sometimes question says:

cache contains 32 KB of data.

Then tag/valid bits are **not** included in that 32 KB.

Sometimes question asks:

total physical cache storage including tags.

Then metadata must be added.

Read wording carefully.

9.34 Data Capacity vs Total Cache Storage

Suppose:

- 256 lines
- each line stores 16-byte block
- 20 tag bits
- 1 valid bit

Data bits per line:

$$16 \times 8 = 128$$

Metadata:

$$20 + 1 = 21$$

Total per line:

$$128 + 21 = 149 \text{ bits}$$

Total storage:

$$[256 \times 149]$$

This is larger than data capacity alone.

9.35 Example — Total Cache Storage

Cache:

- 128 lines
- 32-byte blocks
- tag = 18 bits
- valid = 1 bit

Data bits/line:

$$[32 \times 8 = 256]$$

Total bits/line:

$$[256 + 18 + 1 = 275]$$

Total:

$$[128 \times 275]$$

$$[= 35200 \text{ bits}]$$

$$[\boxed{35200 \text{ bits}}]$$

if no other bits are included.

9.36 Dirty Bit Preview

In write-back cache, line may contain modified data not yet written to main memory.

A:

$$[\boxed{\text{Dirty Bit}}]$$

indicates that.

Typically:

TEXT

D i r t y = 1 ! ' c a c h e b l o c k m o d i f i e d

We cover this in Chapter 13.

9.37 Replacement Metadata Preview

Set-associative/fully associative caches may need replacement information, e.g.:

- LRU state,
- FIFO state.

This contributes extra metadata if total hardware bits are asked.

Again, only include if stated.

9.38 Direct-Mapped Address Structure

For byte-addressable direct cache:

TEXT

Address

%	%	%	%	%	%	%	%	%	%	%	%	%	%	,	%	%	%	%	%	%	%	%	%	%	%	%	,	%	%	%	%	%	%	%	
%	T a g								%	I n d e x							%	O f f s e t									%								
%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	4%	%	%	%	%	%	%	%	%	%	%	%	%	%	4%	%	%	%	%	%	%

Where:

$$\lfloor \text{Offset} = \log_2(\text{BlockSize}) \rfloor$$
$$\lfloor \text{Index} = \log_2(\text{NumberOfLines}) \rfloor$$

\[Tag=A-Index-Offset\]

9.39 Fully Associative Address Structure Preview

There is no fixed line index.

So address becomes:

TEXT

[illegible]

Why no index?

Because block can go into **any** line.

9.40 Set-Associative Address Structure Preview

Address:

TEXT

[illegible]

Memory trick:

$$\boxed{\text{T-S-O}}$$

Tag! Set! Offset.

9.41 What is Associativity?

Associativity tells us how many cache lines are possible placement choices within one mapping destination.

Examples:

Direct mapped

$\lfloor 1 \text{way} \rfloor$

2-way set associative

2 candidate lines/set.

4-way

4 candidate lines/set.

Fully associative

All cache lines belong effectively to one set.

9.42 Number of Sets

For a k -way set-associative cache:

$$\boxed{\text{Number of Sets}} = \frac{\text{Number of Cache Lines}}{k}$$

Example

Cache has:

512 lines

4-way associative:

$\text{Sets} = 512/4$

$\boxed{128}$

Set bits:

$\log_2 128 = 7$

9.43 Associativity Extremes

Direct mapping:

TEXT

Number of sets = Number of lines
Ways/set = 1

Fully associative:

TEXT

Number of sets = 1
Ways/set = Number of lines

This gives a nice unified view.

9.44 Mapping Overview Example

Suppose cache has 8 lines.

Direct

TEXT

8 sets $\times$ 1 way

2-way

TEXT

4 sets $\times$ 2 ways

4-way

TEXT

2 sets $\times$ 4 ways

Fully associative

TEXT

1 set $\times$ 8 ways

9.45 General Cache Address Formula

For byte-addressable memory:

$$\boxed{\text{AddressBits} = \text{TagBits} + \text{SetIndexBits} + \text{BlockOffsetBits}}$$

For direct:

$$\boxed{\text{SetIndexBits} = \text{LineIndexBits}}$$

For fully associative:

$$\boxed{\text{SetIndexBits} = 0}$$

9.46 Example — 2-Way Set Associative

Given:

- 32-bit addresses
- cache = 16 KB
- block = 32 B
- 2-way associative

Lines

$$\boxed{16\text{KB}/32\text{B} = 2^{14}/2^5 = 2^9 = 512}$$

Sets

$$\lfloor 512/2=256=2^8 \rfloor$$

Set bits:

$$\lfloor 8 \rfloor$$

Offset:

$$\lfloor 5 \rfloor$$

Tag:

$$\lfloor 32-8-5=19 \rfloor$$

$$\lfloor \boxed{\text{Tag}=19, \text{Set}=8, \text{Offset}=5} \rfloor$$

9.47 Example — Fully Associative

Same cache:

- 32-bit address
- block = 32 B

No set/index bits.

Offset:

$$\lfloor 5 \rfloor$$

Tag:

$$\lfloor 32-5 \rfloor$$

$$\lfloor \boxed{27} \rfloor$$

9.48 Why Fully Associative Needs More Hardware

If block can be anywhere, cache must compare requested tag against many/all line tags.

Conceptually:

TEXT

Requested Tag

! " ! " ! " ! "

Tag0 Tag1 Tag2 Tag3 ...

This requires associative comparison hardware.

Therefore:

- more flexible placement,
- fewer conflict misses,
- higher hardware cost.

9.49 Why Direct Mapping Is Simple

Index directly identifies one cache line.

Only one tag needs comparison.

Therefore:

- simple,
 - fast,
 - cheap,
 - but more conflict misses.
-

9.50 Set Associative Is a Compromise

Set associative provides:

- more flexibility than direct,
- less hardware than fully associative.

That's why it is widely used.

9.51 Block Address vs Byte Address

Given byte address:

$\lfloor A \rfloor$

Block address/number:

$\lfloor \boxed{\lfloor A / \text{BlockSize} \rfloor} \rfloor$

Then mapping functions operate mainly on block number.

For direct mapping:

$\lfloor \boxed{\text{Line} = \text{BlockNumber} \bmod \text{NumberOfLines}} \rfloor$

Preview only; next chapter goes deep.

9.52 Example — Find Block and Offset

Block size:

$\lfloor 32B \rfloor$

Address:

$\lfloor 1000 \rfloor$

Block number:

$\lfloor \lfloor 1000 / 32 \rfloor \rfloor = 31$

because:

$\lfloor 31 \times 32 = 992 \rfloor$

Offset:

$\lfloor 1000 - 992 = 8 \rfloor$

Thus:

$\lfloor \boxed{\text{Block}=31, \text{Offset}=8} \rfloor$

9.53 Example — Direct Line Preview

Suppose:

- 16 cache lines
- block number = 45

Line:

$$\lfloor 45 \bmod 16 \rfloor$$

$$\lfloor = 13 \rfloor$$

So block 45 maps to:

$$\lfloor \boxed{\text{Line } 13} \rfloor$$

9.54 Example — Set Preview

Suppose:

- 16 total lines
- 4-way set associative

Sets:

$$\lfloor 16/4=4 \rfloor$$

Block number:

$$\lfloor 45 \rfloor$$

Set:

$$\lfloor 45 \bmod 4 \rfloor$$

$$\lfloor \boxed{1} \rfloor$$

Block may occupy any of 4 ways in set 1.

9.55 Powers of 2 Shortcut

Cache numericals become easy when everything is written as powers of 2.

Example:

TEXT

Cache = 64 KB = 2^{16} bytes

Block = 64 B = 2^6 bytes

Lines:

$$\lfloor 2^{16-6} \rfloor$$

$$\lfloor = 2^{10} \rfloor$$

No long division required.

9.56 Core Shortcut Pattern

Suppose:

- address bits = A
- cache size = (2^C) bytes
- block size = (2^B) bytes
- direct mapped.

Lines:

$$\lfloor 2^{C-B} \rfloor$$

So:

$$\lfloor \text{Index} = C - B \rfloor$$

Offset:

$$\lfloor B \rfloor$$

Tag:

$$\lfloor A - C \rfloor$$

Interesting simplification:

$$\lfloor \text{Tag} = A - (C - B) - B = A - C \rfloor$$

So for direct-mapped **byte-addressable data capacity cache**:

$$\lfloor \boxed{\text{TagBits} = \text{AddressBits} - \log_2(\text{CacheSize})} \rfloor$$

when cache size is data bytes.

This shortcut is safe in that specific scenario.

Example

Address = 32 bits.

Cache = 64 KB:

$$\lfloor 64\text{KB} = 2^{16} \text{B} \rfloor$$

Direct mapped.

Then:

$$\lfloor \text{Tag} = 32 - 16 = 16 \rfloor$$

regardless of block size, while index/offset split changes with block size.

Useful shortcut.

9.57 Why Tag Shortcut Works

Cache data capacity:

$$\lfloor \text{Lines} \times \text{BlockSize} \rfloor$$

Thus:

$$\lfloor \text{IndexBits} + \text{OffsetBits} = \log_2(\text{CacheSize}) \rfloor$$

Therefore:

$$\lfloor \text{Tag} = A - \log_2(\text{CacheSize}) \rfloor$$

Again: direct-mapped, byte-addressable, normal cache-capacity interpretation.

9.58 Set-Associative Tag Shortcut

For k-way associative cache:

$$\lfloor \text{Sets} = \frac{\text{CacheSize}}{\text{BlockSize} \times k} \rfloor$$

So:

$$\lfloor \text{SetBits} = \log_2(\text{CacheSize}) - \log_2(\text{BlockSize}) - \log_2 k \rfloor$$

Offset:

$$\lfloor \log_2 \text{BlockSize} \rfloor$$

Hence:

$$\lfloor \boxed{\text{Tag} = A - \log_2(\text{CacheSize}) + \log_2 k} \rfloor$$

for byte-addressable power-of-two configuration.

We will derive and use this later.

9.59 Fully Associative Tag Shortcut

Fully associative has no index.

Therefore:

$$\lfloor \boxed{\text{Tag} = A - \text{Offset}} \rfloor$$

For byte-addressed block size $\lfloor (B) \rfloor$:

$$\lfloor \boxed{\text{Tag} = A - \log_2 B} \rfloor$$

9.60 Cache Access Sequence

CPU sends address.

Cache:

TEXT

1. Extract index/set
2. Read candidate line(s)
3. Compare tag(s)
4. Check valid bit
5. If hit, use offset
6. If miss, fetch block from lower memory

This sequence connects all address fields.

9.61 Why Tag Is Stored but Index Usually Isn't

Index is obtained directly from CPU address and determines physical cache location/set.

Therefore cache does not need to store index in each line.

Tag must be stored because many memory blocks can map to same line/set.

This is a common conceptual question.

9.62 Why Offset Is Not Stored as Metadata

Offset identifies position within the data block.

The block already contains all those bytes/words.

Thus offset comes from request address and selects one part of block.

No need to store it as tag metadata.

9.63 Tag Comparison Example

Suppose address fields:

TEXT

```
Tag    = 10101
Index  = 011
Offset = 10
```

Index **011** selects line 3.

Stored line has:

TEXT

```
Valid = 1
Tag    = 10101
```

Tags match.

Therefore:

$\boxed{\text{Hit}}$

Offset **10** selects appropriate byte/word.

9.64 Miss Example

Requested:

TEXT

```
Tag = 10101
```

Stored:

TEXT

Valid = 1
Tag = 11100

Different tag.

Therefore:

$\boxed{\text{Miss}}$

even though selected line contains valid data.

It simply contains a different memory block.

9.65 Invalid-Line Example

Requested and stored tags coincidentally match, but:

TEXT

Valid = 0

Then:

$\boxed{\text{Miss}}$

because line is not valid.

9.66 GATE Trap — Cache Size Includes/Excludes Tags

If question says:

64 KB cache with 64-byte blocks

normally this means:

$\boxed{64\text{KB} \text{ data capacity}}$

Do not subtract tag bits from the 64 KB unless explicitly stated.

9.67 GATE Trap — Offset in Words vs Bytes

Suppose block = 8 words.

If memory is word-addressable:

$\boxed{\text{Offset}=3}$

If byte-addressable and word = 4 bytes:

$\boxed{\text{Block}=32\text{B}}$

$\boxed{\text{Offset}=5}$

Huge difference.

9.68 GATE Trap — Cache Lines

Do not use:

$$\lfloor \text{CacheSize} / \text{WordSize} \rfloor$$

unless block size equals one word.

Correct:

$$\lfloor \boxed{\text{CacheSize} / \text{BlockSize}} \rfloor$$

9.69 GATE Trap — Main Memory Size

If CPU address width is explicitly given, you often don't need physical main-memory capacity separately.

Address bits come from the CPU address width.

Example:

32-bit byte address

means address length is 32 regardless of installed RAM size.

9.70 GATE Trap — Tag Bits

Tag is not simply:

$$\lfloor \text{AddressBits} - \text{OffsetBits} \rfloor$$

except fully associative cache.

For direct/set associative, subtract index/set bits too.

9.71 GATE Trap — Block Number vs Offset

For address 100 and block size 16:

TEXT

Block = 6
Offset = 4

Do not reverse them.

9.72 GATE Trap — Set Count

4-way cache does not mean 4 sets.

It means:

$$\lfloor 4 \text{ lines per set} \rfloor$$

Number of sets:

$$\lfloor \frac{\text{Total Lines}}{4} \rfloor$$

9.73 GATE Trap — Fully Associative

Fully associative is not:

TEXT

one line

It is:

TEXT

one set containing all lines

9.74 GATE Numerical 1

Given:

- Cache size = 8 KB
- Block size = 16 B

Find number of cache lines.

$$\sqrt[4]{8\text{KB}=2^{13}}$$

$$\sqrt[4]{16\text{B}=2^4}$$

Lines:

$$\sqrt[4]{2^{13-4}=2^9}$$

$$\sqrt[4]{\boxed{512}}$$

9.75 GATE Numerical 2

Byte-addressable main memory:

$$\sqrt[4]{16\text{MB}}$$

Block:

$$\sqrt[4]{32\text{B}}$$

Number of memory blocks:

$$\sqrt[4]{2^{24}/2^5}$$

$$\sqrt[4]{=2^{19}}$$

$$\sqrt[4]{\boxed{524288}}$$

9.76 GATE Numerical 3

32-bit address.

Direct mapped cache:

- 64 KB
- 64-byte block.

Offset:

$$\lfloor \log_2 64 = 6 \rfloor$$

Lines:

$$\lfloor 2^{16} / 2^6 = 2^{10} \rfloor$$

Index:

$$\lfloor 10 \rfloor$$

Tag:

$$\lfloor 32 - 10 - 6 = 16 \rfloor$$

Answer:

$$\lfloor \boxed{\text{Tag}=16, \text{Index}=10, \text{Offset}=6} \rfloor$$

9.77 GATE Numerical 4

32-bit byte address.

4-way cache:

- cache = 64 KB
- block = 64 B.

Lines:

$$\lfloor 2^{16-6} = 2^{10} \rfloor$$

Sets:

$$\lfloor 2^{10} / 2^2 = 2^8 \rfloor$$

Set bits:

$$\lfloor 8 \rfloor$$

Offset:

$$\lfloor 6 \rfloor$$

Tag:

$$\lfloor 32 - 8 - 6 \rfloor$$

$$\lfloor \boxed{18} \rfloor$$

9.78 GATE Numerical 5

Same cache, fully associative.

Offset:

$$\lfloor 6 \rfloor$$

Index/set bits:

$$\lfloor 0 \rfloor$$

Tag:

$$\lfloor 32-6 \rfloor$$

$$\lfloor \boxed{26} \rfloor$$

9.79 GATE Numerical 6 — Word Addressable

Memory has:

- 24-bit word addresses,
- block = 16 words.

Offset:

$$\lfloor \log_2 16=4 \rfloor$$

If direct cache has 256 lines:

$$\lfloor \text{Index}=8 \rfloor$$

Tag:

$$\lfloor 24-8-4 \rfloor$$

$$\lfloor \boxed{12} \rfloor$$

9.80 GATE Numerical 7 — Byte-Addressed Words

Memory:

- 32-bit byte addresses
- word = 4 bytes
- block = 16 words
- direct cache = 64 KB.

Block bytes:

$$\lfloor 16 \times 4=64B \rfloor$$

Offset:

$$\lfloor 6 \rfloor$$

Lines:

$$\lfloor 64KB/64B = 2^{16-6} = 2^{10} \rfloor$$

Index:

$$\lfloor 10 \rfloor$$

Tag:

$$\lfloor 32-10-6=16 \rfloor$$

Total offset may be viewed as:

TEXT

4-bit word offset
+
2-bit byte offset

CHAPTER 9 — MASTER FORMULAS

Cache lines

$$\boxed{L = \frac{\text{CacheSize}}{\text{BlockSize}}}$$

Memory blocks

$$\boxed{B_M = \frac{\text{MainMemorySize}}{\text{BlockSize}}}$$

Byte offset

$$\boxed{b = \log_2(\text{BlockSize in bytes})}$$

Direct index bits

$$\boxed{i = \log_2(\text{CacheLines})}$$

k-way sets

$$\boxed{\text{Sets} = \frac{\text{CacheLines}}{k}}$$

Set bits

$$\boxed{s = \log_2(\text{Sets})}$$

Address equation

$$\boxed{A = \text{Tag} + \text{Index} + \text{Set} + \text{Offset}}$$

CHAPTER 9 — MEMORY TRICKS

Direct mapped

T-I-O

Tag | Index | Offset

Set associative

T-S-O

Tag | Set | Offset

Fully associative

T-O

Tag | Offset

Cache lines

Cache $\div$ Block

Set count

Lines $\div$ Ways

Offset

Block tells offset

CHAPTER SNAPSHOT

1. Cache stores copies of main-memory blocks.
2. Main memory is divided into blocks.
3. Cache is divided into lines.
4. Number of cache lines = cache size / block size.
5. Cache mapping decides where memory blocks can be placed.
6. Direct mapping gives one possible line.
7. Fully associative allows any line.
8. Set associative allows any way within one set.
9. Offset selects location inside a block.
10. Index/set selects cache line/set.
11. Tag identifies which memory block is present.
12. Valid bit confirms cache-line contents are usable.
13. Byte-addressability changes offset calculations.
14. Block number = address / block size.
15. Offset = address mod block size.
16. 4-way means 4 lines per set, not 4 sets.
17. Fully associative has zero index bits.
18. Cache metadata may include valid, tag, dirty, and replacement state.

MUST REMEMBER

$$\boxed{\text{CacheLines} = \frac{\text{CacheSize}}{\text{BlockSize}}}$$

$$\boxed{\text{OffsetBits} = \log_2(\text{BlockSize})}$$

for byte-addressed memory with block size in bytes.

$$\boxed{\text{Direct: Tag} = \text{A-Index-Offset}}$$

$$\boxed{\text{Sets} = \frac{\text{Lines}}{\text{Ways}}}$$

And:

PRACTICE SET

Q1 — L0 NAT

A 16 KB cache has 64-byte blocks.

How many cache lines?

Q2 — L1 NAT

Block size = 128 bytes.

How many byte-offset bits are required?

Q3 — L1 NAT

A direct-mapped cache has 1024 lines.

How many index bits?

Q4 — L1

A 32-bit byte-addressed system has:

- direct cache = 32 KB
- block = 32 B.

Find:

1. offset bits,
2. index bits,
3. tag bits.

Q5 — L2

A 32-bit byte-addressed system has:

- cache = 32 KB
- block = 32 B
- 4-way associative.

Find:

1. number of lines,
2. number of sets,
3. set bits,
4. tag bits.

Q6 — L2

For the same cache as Q5, if fully associative, find tag bits.

Q7 — L2

Address = 250 decimal.

Block size = 16 bytes.

Find:

1. block number,
 2. byte offset.
-

Q8 — L2

A block contains 8 words, each word = 4 bytes, and memory is byte-addressable.

Find:

1. word-offset bits,
 2. byte-offset bits,
 3. total block-offset bits.
-

Q9 — L2

A word-addressable memory uses 24-bit addresses. Each block contains 32 words.

How many block-offset bits?

Q10 — L2

A cache has:

- 512 total lines,
- 8-way associativity.

Find:

1. sets,
 2. set-index bits.
-

Q11 — L3

A 36-bit byte-addressed machine has:

- 256 KB cache,
- 64-byte blocks,
- 4-way set associativity.

Find:

1. lines,
 2. sets,
 3. offset bits,
 4. set bits,
 5. tag bits.
-

Q12 — L3

A direct cache has:

- 256 lines,
- 16-byte blocks,
- 20-bit tag,
- 1 valid bit.

Find total physical cache storage in bits excluding dirty/replacement bits.

CHAPTER 9 COMPLETION STATUS

' Cache purpose ' Blocks and lines ' Cache-size calculations ' Main-memory block calculations ' Mapping concept ' Direct/associative/set-associative overview ' Tag ' Index ' Set ' Offset ' Byte vs word offset ' Valid bit ' Basic hit detection ' Miss-type introduction ' Associativity ' Number of sets ' Metadata storage ' Address decomposition ' GATE shortcuts and traps

CHAPTER 9: COMPLETE '

COURSE PROGRESS

Completed: 10 / 26 modules

Approximate roadmap progress:

$\boxed{\sim 38\%}$

NEXT !' CHAPTER 10 — DIRECT-MAPPED CACHE

Next we go deep into the first full cache mapping technique:

- exact mapping formula,
 - block !' line mapping,
 - tag derivation,
 - address decomposition,
 - binary/hex cache questions,
 - hit/miss sequences,
 - conflict misses,
 - cache contents after reference strings,
 - total tag-storage calculations,
 - direct-mapped GATE numericals and shortcuts.
-

USER

next

ASSISTANT

CHAPTER 10 — DIRECT-MAPPED CACHE

Classification: [GATE CORE] **Priority:** ~~Ø=VERY~~ VERY HIGH **Official syllabus connection:** Cache memory mapping

Direct mapping is usually the first cache mapping technique GATE expects you to master numerically.

Core idea:

Every main-memory block has exactly one possible cache line.

10.1 What is Direct Mapping?

Suppose cache has 8 lines.

Main-memory blocks map like this:

TEXT

```
B l o c k   0   !'   L i n e   0
B l o c k   1   !'   L i n e   1
B l o c k   2   !'   L i n e   2
...
B l o c k   7   !'   L i n e   7
B l o c k   8   !'   L i n e   0
B l o c k   9   !'   L i n e   1
...
```

So different memory blocks may compete for the same line.

10.2 Master Mapping Formula

If cache has L lines:

$$\boxed{\text{Cache Line} = \text{Main Memory Block Number} \bmod L}$$

~~Ø=VERY~~ This is the direct-mapping formula.

Example

Cache has:

$$8 \text{ lines}$$

Memory block:

$$21$$

Then:

$$21 \bmod 8 = 5$$

Therefore:

$$\boxed{\text{Block } 21} \rightarrow \text{Line } 5$$

10.3 Why Modulo?

Suppose cache has 4 lines.

Mappings:

TEXT			
Block	0	!	0
Block	1	!	1
Block	2	!	2
Block	3	!	3
Block	4	!	0
Block	5	!	1
Block	6	!	2
Block	7	!	3

Pattern repeats every 4 blocks.

That is exactly modulo 4.

10.4 Direct-Mapped Address Format

For a byte-addressable system:

TEXT																																	
%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%
Tag								%	Index								%	Offset															
%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%

Memory trick:

$$\boxed{\text{T-I-O}}$$

Tag! Index! Offset

10.5 Role of Each Field

Offset

Selects byte/word inside the block.

Index

Selects the one cache line where the block must be.

Tag

Confirms which memory block currently occupies that line.

10.6 Full Numerical Example

Given:

- 32-bit byte-addresses
- Cache size = 16 KB
- Block size = 64 bytes
- Direct mapped

Find tag/index/offset bits.

Step 1 — Offset

$$\boxed{64\text{B} = 2^6\text{B}}$$

So:

$$\boxed{\text{Offset} = 6}$$

Step 2 — Number of lines

$$\boxed{16\text{KB} = 2^{14}\text{B}}$$

$$\boxed{\text{Lines} = \frac{2^{14}}{2^6} = 2^8}$$

So:

$$\boxed{\text{Index} = 8}$$

Step 3 — Tag

$$\boxed{\text{Tag} = 32 - 8 - 6}$$

$$\boxed{18}$$

Final:

TEXT

Tag = 18
Index = 8
Offset = 6

10.7 Fast Shortcut for Direct Mapping

If:

- byte-addressable,
- cache data capacity = 2^C bytes,
- address width = A bits,

then:

$$\boxed{\text{Tag} = A - C}$$

Why?

Because:

$$\boxed{\text{Index} + \text{Offset} = C}$$

So:

$\lfloor \text{Tag} = A - C \rfloor$

For previous example:

$\lfloor A = 32, \text{quad } C = 14 \rfloor$

$\lfloor \text{Tag} = 32 - 14 = 18 \rfloor$

10.8 When This Shortcut Is Safe

Use it when:

- direct mapped,
- byte-addressable,
- cache size means data capacity,
- normal power-of-2 organization.

Do not use blindly in unusual word-addressed or metadata-inclusive questions.

10.9 Block Number from Address

For byte address A :

$\lfloor \boxed{\text{BlockNumber} = \left\lfloor \frac{A}{\text{BlockSize}} \right\rfloor} \rfloor$

Offset:

$\lfloor \boxed{\text{Offset} = A \bmod \text{BlockSize}} \rfloor$

Then:

$\lfloor \boxed{\text{Line} = \text{BlockNumber} \bmod \text{NumberOfLines}} \rfloor$

10.10 Worked Example

Given:

- address = 500 decimal
- block size = 32 B
- cache lines = 16

Block number

$\lfloor \lfloor 500 / 32 \rfloor \rfloor = 15$

because:

$\lfloor 15 \times 32 = 480 \rfloor$

Offset

$\lfloor 500 - 480 = 20 \rfloor$

Cache line

$\lfloor 15 \bmod 16 = 15 \rfloor$

Therefore:

$\boxed{\text{Block}=15, \text{Line}=15, \text{Offset}=20}$

10.11 Another Example

Address:

1000

Block size:

64B

Cache lines:

8

Block:

$\lfloor 1000/64 \rfloor = 15$

because:

$15 \times 64 = 960$

Offset:

$1000 - 960 = 40$

Line:

$15 \bmod 8 = 7$

Answer:

$\boxed{\text{Block}=15, \text{Line}=7, \text{Offset}=40}$

10.12 Tag from Block Number

If cache has $L=2^i$ lines:

Block number can conceptually be split into:

TEXT

Tag | Index

The lowest i bits of block number form index.

Remaining upper block-number bits form tag.

Example

Suppose:

- 8 cache lines !' 3 index bits
- block number = 45

Binary:

$\lfloor 45 = 101101_2 \rfloor$

Split lowest 3 bits:

TEXT

101		101
Tag		Index

Index:

$\lfloor 101_2 = 5 \rfloor$

Tag:

$\lfloor 101_2 = 5 \rfloor$

And:

$\lfloor 45 \bmod 8 = 5 \rfloor$

consistent.

10.13 Direct Mapping from Binary Address

Suppose:

- 16-bit byte address
- block size = 16 bytes !' 4 offset bits
- cache = 256 bytes

Number of lines:

$\lfloor 256/16 = 16 = 2^4 \rfloor$

Index = 4 bits.

So address format:

TEXT

Tag: 8 | Index: 4 | Offset: 4

Take address:

TEXT

10110110 0110 1011

Then:

TEXT

Tag	=	10110110
Index	=	0110
Offset	=	1011

Thus:

$\lfloor \text{Index} = 6 \rfloor$

$\lfloor \text{Offset} = 11 \rfloor$

10.14 Hexadecimal Address Shortcut

Because:

[1\ hex\ digit=4\ bits]

hex addresses become easy when field boundaries align with 4 bits.

Example:

16-bit address format:

TEXT

Tag 8 | Index 4 | Offset 4

Address:

TEXT

B65BH

Then:

TEXT

Tag = B6H
Index = 5H
Offset = BH

Very fast.

10.15 If Fields Don't Align with Hex Digits

Suppose:

- offset = 5 bits,
- index = 7 bits.

Then hex boundaries do not match fields cleanly.

Convert only the relevant portion to binary.

Do not force whole-address decimal conversion.

GATE speed tip:

Convert hex !' binary nibble-by-nibble, then slice the fields.

10.16 How a Hit Is Determined

Given CPU address:

TEXT

Tag | Index | Offset

Cache performs:

1. Index selects one cache line.
2. Check valid bit.
3. Compare requested tag with stored tag.
4. If equal and valid = 1 != hit.
5. Offset chooses exact byte/word.

10.17 Hit Condition

$\boxed{\text{Hit} = \text{Valid} \wedge (\text{Tag}_{\text{stored}} = \text{Tag}_{\text{requested}})}$

If either fails:

$\boxed{\text{Miss}}$

10.18 Example — Hit

Suppose requested address gives:

TEXT

Tag = 1010
Index = 011

Line 3 contains:

TEXT

Valid = 1
Tag = 1010

Then:

$\boxed{\text{Hit}}$

10.19 Example — Miss Due to Tag

Requested:

TEXT

Tag = 1010
Index = 011

Line 3:

TEXT

Valid = 1
Tag = 1101

Then:

`\boxed{Miss}`

10.20 Example — Miss Due to Invalid Bit

Requested tag matches, but:

TEXT

Valid = 0

Still:

`\boxed{Miss}`

10.21 Conflict in Direct Mapping

Suppose cache has 4 lines.

Blocks:

TEXT

```
0 !' l i n e 0
4 !' l i n e 0
8 !' l i n e 0
1 2 !' l i n e 0
```

Even if other lines are empty, these blocks all compete for line 0.

This causes:

`\boxed{Conflict\ Misses}`

10.22 Classic Conflict Example

Cache has 4 lines.

Reference string:

TEXT

0, 4, 0, 4

Mappings:

TEXT

```
0 !' l i n e 0
4 !' l i n e 0
```

Simulation:

- Access 0 !' miss, place block 0
- Access 4 !' miss, replace block 0
- Access 0 !' miss

- Access 4 !' miss

Total:

$\boxed{4 \text{ misses}}$

Even though cache has 4 lines.

10.23 Direct Cache Simulation Method

For reference-string questions:

Step 1

Convert each memory address to block number if needed.

Step 2

Find:

$\text{Line} = \text{Block} \bmod \text{NumberOfLines}$

Step 3

Compare stored tag/block.

Step 4

Update line on miss.

Step 5

Count hits and misses.

10.24 Reference String Example

Cache:

- 4 lines
- 1 block per line
- initially empty

Block sequence:

TEXT

0, 1, 2, 3, 0, 4, 0

Let's simulate.

Access 0

Line:

$0 \bmod 4 = 0$

Empty !' miss.

Cache:

TEXT

$L_0=0$

Access 1

Line 1 !' miss.

Access 2

Line 2 !' miss.

Access 3

Line 3 !' miss.

Now:

TEXT

$L_0=0$

$L_1=1$

$L_2=2$

$L_3=3$

Access 0

Line 0 contains 0.

$\boxed{\text{Hit}}$

Access 4

$4 \bmod 4 = 0$

Line 0 contains 0, not 4.

Miss; replace 0 with 4.

Access 0

Maps line 0, now contains 4.

Miss.

Totals:

- Hits = 1
- Misses = 6

Hit rate:

$\boxed{1/7}$

10.25 Tag Calculation from Block Number

For cache with 4 lines:

$\boxed{\text{Line} = \text{Block} \bmod 4}$

Tag can be computed conceptually as:

$\boxed{\text{Tag} = \left\lfloor \frac{\text{BlockNumber}}{\text{NumberOfLines}} \right\rfloor}$

when using block-number interpretation.

Example block 13:

$\text{\[Line=13\bmod4=1 \]}$

$\text{\[Tag=\lfloor 13/4\rfloor=3 \]}$

So:

TEXT

B l o c k 1 3 ! ' T a g 3 , L i n e 1

10.26 Why This Works

Block number can be expressed as:

$\text{\[BlockNumber=Tag\times NumberOfLines+Line \]}$

Thus:

$\text{\[\boxed{Block=Tag\cdot L+Index} \]}$

Useful for reverse questions.

10.27 Reverse Example

Suppose:

- cache has 8 lines
- stored tag = 5
- line index = 3

Which main-memory block is stored?

$\text{\[Block=Tag\times 8+3 \]}$

$\text{\[=40+3 \]}$

$\text{\[\boxed{43} \]}$

10.28 Address Range of a Block

Suppose block number = B , block size = S bytes.

Starting byte address:

$\text{\[\boxed{B\times S} \]}$

Ending byte address:

$\text{\[\boxed{B\times S+S-1} \]}$

Example:

Block 10, block size 16 B.

Start:

$$\lfloor 10 \times 16 = 160 \rfloor$$

End:

$$\lfloor 160 + 15 = 175 \rfloor$$

So block 10 covers:

$$\lfloor \boxed{160 \text{ to } 175} \rfloor$$

10.29 Reverse Address from Tag/Index/Offset

Suppose:

- tag = T
- index = I
- offset = O
- lines = L
- block size = B

Block number:

$$\lfloor \boxed{\text{Block} = T \cdot L + I} \rfloor$$

Byte address:

$$\lfloor \boxed{\text{Address} = \text{Block} \cdot B + O} \rfloor$$

Example

Cache:

- 8 lines
- block size 16 B

Given:

TEXT

Tag = 3
Index = 5
Offset = 7

Block:

$$\lfloor 3 \times 8 + 5 = 29 \rfloor$$

Address:

$$\lfloor 29 \times 16 + 7 \rfloor$$

$$\lfloor = 464 + 7 \rfloor$$

$$\lfloor \boxed{471} \rfloor$$

10.30 Number of Main-Memory Blocks Mapping to One Line

Suppose main memory has:

$\lceil M \rceil$

blocks and cache has:

$\lceil L \rceil$

lines.

Each line receives:

$\lceil \boxed{M/L} \rceil$

memory blocks, assuming power-of-two exact division.

Example

Main memory:

2^{20} blocks

Cache:

2^{10} lines

Blocks mapping per line:

$2^{20}/2^{10}$

$\boxed{2^{10}}$

This also equals number of possible tag values.

10.31 Tag Bits from Mapping Count

If each line can represent one of:

2^t

possible memory blocks, then:

$\boxed{t}$ tag bits

Example:

1024 possible memory blocks per line:

$1024 = 2^{10}$

So tag:

$\boxed{10}$ bits

10.32 Direct-Mapped Cache Total Tag Storage

Suppose:

- number of lines = L
- tag bits per line = t

Then total tag bits:

$\boxed{L \times t}$

If valid bits also included:

$$\boxed{L \times (t+1)}$$

If dirty bits too:

$$\boxed{L \times (t+1+1)}$$

depending on design.

10.33 Example — Tag + Valid Storage

Given:

- 1024 cache lines
- tag = 18 bits
- valid = 1 bit

Metadata:

$$1024(18+1)$$

$$= 1024 \times 19$$

$$\boxed{19456 \text{ bits}}$$

10.34 Total Physical Cache Storage

Suppose:

- 1024 lines
- block size = 64 B
- tag = 18 bits
- valid = 1 bit

Data bits/line:

$$64 \times 8 = 512$$

Total bits/line:

$$512 + 18 + 1 = 531$$

Total:

$$1024 \times 531$$

$$\boxed{543744 \text{ bits}}$$

10.35 Overhead Percentage

Sometimes GATE asks cache overhead.

Metadata overhead:

$$\boxed{\frac{\text{MetadataBits}}{\text{DataBits}} \times 100\%}$$

Example per line:

- Data = 512 bits
- Tag + valid = 19 bits

Overhead:

$$\left[\frac{19}{512} \times 100 \right]$$

$$\left[\approx 3.71\% \right]$$

10.36 Address Bits from Main Memory Size

Suppose:

- main memory = 1 GB
- byte-addressable.

$$\left[1\text{GB} = 2^{30} \text{ bytes} \right]$$

Thus:

$$\left[\boxed{30} \text{ address bits} \right]$$

Do not automatically use 32 bits unless processor address size is explicitly given.

10.37 Worked GATE Example

Given:

- Main memory = 4 MB
- Byte-addressable
- Cache = 16 KB
- Block = 32 B
- Direct mapped

Find tag/index/offset.

Address bits

$$\left[4\text{MB} = 2^{22} \text{ B} \right]$$

So:

$$\left[A = 22 \right]$$

Offset

$$\left[32\text{B} = 2^5 \right]$$

$$\left[\text{Offset} = 5 \right]$$

Lines

$$\left[16\text{KB} / 32\text{B} = 2^{14} / 2^5 = 2^9 \right]$$

$$\left[\text{Index} = 9 \right]$$

Tag

$$\left[22 - 9 - 5 = 8 \right]$$

Answer:

$$\lfloor \boxed{\text{Tag}=8, \text{Index}=9, \text{Offset}=5} \rfloor$$

10.38 Word-Addressable Direct Cache

Suppose:

- 20-bit word address
- block = 8 words
- cache = 1024 words
- direct mapped

Number of lines:

$$\lfloor 1024/8=128=2^7 \rfloor$$

Offset:

$$\lfloor \log_2 8=3 \rfloor$$

Index:

$$\lfloor 7 \rfloor$$

Tag:

$$\lfloor 20-7-3 \rfloor$$

$$\lfloor \boxed{10} \rfloor$$

Notice cache size was given in words and block in words, so units match directly.

10.39 Byte-Addressed Direct Cache with Word Blocks

Suppose:

- 24-bit byte address
- word size = 4 bytes
- block = 8 words
- cache = 8 KB

Block:

$$\lfloor 8 \times 4=32\text{B}=2^5 \rfloor$$

Offset:

$$\lfloor 5 \rfloor$$

Lines:

$$\lfloor 2^{13}/2^5=2^8 \rfloor$$

Index:

$$\lfloor 8 \rfloor$$

Tag:

$$\lfloor 24-8-5 \rfloor$$

$$\lfloor \boxed{11} \rfloor$$

10.40 Word Offset + Byte Offset Split

Same example:

- 8 words/block !' 3 word-offset bits
- 4 bytes/word !' 2 byte-offset bits

Total:

$\lceil 3+2=5 \rceil$

Address:

TEXT

Tag 11 | Index 8 | Word Offset 3 | Byte Offset 2

This split is often useful in GATE.

10.41 Conflict Miss Example with Addresses

Suppose:

- cache = 4 lines
- block size = 16 B

Addresses:

TEXT

0, 64, 0, 64

Block numbers:

$\lceil 0/16=0 \rceil$

$\lceil 64/16=4 \rceil$

Both map:

$\lceil 0 \bmod 4=0 \rceil$

$\lceil 4 \bmod 4=0 \rceil$

Therefore every alternating access can evict the other block.

Classic conflict pattern.

10.42 Compulsory vs Conflict in a Sequence

Sequence:

TEXT

0, 4, 0

4-line direct cache, blocks.

- First 0 !' compulsory miss

- First 4 !' compulsory miss
- Second 0 !' conflict miss because block 4 displaced it

This distinction matters when miss classification is asked.

10.43 Capacity vs Conflict Reminder

If a fully associative cache of the same total capacity would avoid the miss, the miss is likely a **conflict** miss.

If even fully associative cache cannot hold the working set, it is a **capacity** miss.

10.44 Direct Mapping Advantages

- Simple hardware
- Fast lookup
- Only one tag comparison
- Low cost
- Easy indexing

10.45 Direct Mapping Disadvantages

- High conflict misses
- No placement flexibility
- A busy line may repeatedly evict useful blocks

10.46 Why Direct Mapping Can Still Be Fast

Although it may have more misses, hit lookup is simple:

TEXT

I n d e x ! ' o n e l i n e ! ' o n e t a g c o m p a r e

This can reduce hit time.

Architecture always involves trade-offs.

10.47 Direct Cache and Power-of-Two Sizes

Caches are usually designed with powers of 2 because address bits can directly select:

- line,
- offset.

No expensive division/modulo hardware is needed.

Modulo by (2^k) is simply:

take the lowest k bits.

10.48 Modulo Shortcut

If cache has 16 lines:

$$\lfloor 16=2^4 \rfloor$$

Then line number is simply lowest 4 bits of block number.

Example block:

```
TEXT
10110110
```

lowest 4 bits:

```
TEXT
0110
```

So line:

$$\lfloor \boxed{6} \rfloor$$

No arithmetic required.

10.49 Tag Shortcut from Block Binary

Same block:

```
TEXT
1011 0110
```

16 cache lines means 4 index bits.

Split:

```
TEXT
1011 | 0110
Tag   Index
```

Tag:

$$\lfloor 1011_2=11 \rfloor$$

Index:

$$\lfloor 6 \rfloor$$

10.50 Address Decomposition Shortcut

For a byte-address:

1. Lowest **offset bits** !' offset
2. Next **index bits** !' cache line

3. Remaining upper bits != tag

Always work from **right to left**.

Memory trick:

Rightmost = smallest location detail.

10.51 GATE Trap — Index from Cache Size Directly

Wrong:

$\lfloor \text{IndexBits} = \log_2(\text{CacheSize}) \rfloor$

Correct:

$\lfloor \boxed{\text{IndexBits} = \log_2(\text{CacheSize}/\text{BlockSize})} \rfloor$

Because index selects lines, not bytes.

10.52 GATE Trap — Block Size Doesn't Affect Tag?

In direct byte-addressed cache with fixed cache data size and address width, tag bits indeed stay unchanged when block size changes.

But:

- index bits decrease as block size increases,
- offset bits increase equally.

Do not generalize this to set associative or unusual systems without checking.

10.53 GATE Trap — Tag Storage Includes Offset?

No.

Cache stores tag, not offset.

Offset comes from the current address.

10.54 GATE Trap — Replacing One Line

On a direct-mapped miss, you do not choose among multiple candidate lines.

There is exactly one mapped line.

If occupied by another block:

$\lfloor \boxed{\text{replace it}} \rfloor$

No LRU policy is needed for direct mapping.

10.55 Important Consequence

Direct-mapped cache does **not need a replacement algorithm** to choose a victim.

The victim is predetermined by the index.

0-6% Common conceptual question.

10.56 Does Direct Mapping Need LRU?

[\boxed{No}]

There is only one line in the mapped set.

LRU/FIFO choices matter when associativity > 1.

10.57 Valid Bit at Initialization

Initially all valid bits are typically:

TEXT

0

Thus first access to each required block is a miss.

After loading:

TEXT

Valid = 1

10.58 Dirty Bit in Direct Mapping

Direct mapping may still use a dirty bit if write-back policy is used.

So:

- no replacement-choice metadata,
- but valid/tag/dirty metadata may still exist.

10.59 Reference String — Full Table Method

Suppose 4-line cache and blocks:

TEXT

0, 1, 4, 1, 0, 5, 1

You can make:

Access	Line	Previous	Hit/Miss	New
0	0	empty	M	0
1	1	empty	M	1

Access	Line	Previous	Hit/Miss	New
4	0	0	M	4
1	1	1	H	1
0	0	4	M	0
5	1	1	M	5
1	1	5	M	1

Hits:

$\boxed{1}$

Misses:

$\boxed{6}$

This table style is safest for long sequences.

10.60 Hit Ratio from Sequence

For previous sequence:

$\text{Total}=7$

$\text{Hits}=1$

$\text{HitRatio}=\frac{1}{7}$

$\approx 14.29\%$

10.61 GATE Example — Mapping Block to Line

Cache has 256 lines.

Block number:

1000

Line:

$1000 \bmod 256$

Since:

$256 \times 3 = 768$

remainder:

232

$\boxed{\text{Line}=232}$

10.62 Faster Binary Method

256 lines = 2^8 .

Take lowest 8 bits of block number.

This is usually faster than decimal modulo in exam.

10.63 GATE Example — Number of Blocks per Line

Memory:

- 32-bit byte-addressable
- block = 64 B

Number memory blocks:

$$\lfloor 2^{32}/2^6=2^{26} \rfloor$$

Cache:

- 64 KB direct
- block = 64 B

Lines:

$$\lfloor 2^{16}/2^6=2^{10} \rfloor$$

Blocks per line:

$$\lfloor 2^{26}/2^{10} = 2^{16} \rfloor$$

Thus tag bits:

$$\lfloor \boxed{16} \rfloor$$

10.64 Tag/Index/Offset from Exponents

A quick method:

Given:

- Address = $\lfloor A \rfloor$ bits
- Cache = $\lfloor 2^C \rfloor$ bytes
- Block = $\lfloor 2^B \rfloor$ bytes

Direct mapped:

$$\lfloor \boxed{\text{Offset}=B} \rfloor$$

$$\lfloor \boxed{\text{Index}=C-B} \rfloor$$

$$\lfloor \boxed{\text{Tag}=A-C} \rfloor$$

This is one of the fastest GATE formulas in COA.

10.65 Example

36-bit address.

Cache:

$$\lfloor 256\text{KB}=2^{18}\text{B} \rfloor$$

Block:

$$\lfloor \frac{64B}{2^6} \rfloor$$

Then:

$$\lfloor \frac{\text{Offset}}{6} \rfloor$$

$$\lfloor \frac{\text{Index} - 6}{12} \rfloor$$

$$\lfloor \frac{\text{Tag} - 18}{18} \rfloor$$

Answer:

$$\boxed{\text{18|12|6}}$$

10.66 Address Field Sanity Check

Always verify:

$$\text{Tag} + \text{Index} + \text{Offset} = \text{AddressBits}$$

Example:

$$18 + 12 + 6 = 36$$

Correct.

10.67 Direct Mapping Placement Pattern

If there are L lines, then line j can hold blocks:

$$\boxed{j, j+L, j+2L, j+3L, \dots}$$

Example $L = 8$, line 3:

TEXT

3, 11, 19, 27, 35, ...

All compete for line 3.

10.68 GATE Reverse Question

Which blocks map to line 5 in a 16-line cache?

$$\boxed{5, 21, 37, 53, \dots}$$

General:

$$\boxed{5 + 16k}$$

10.69 Direct Mapping and Tag Value Pattern

For line j :

TEXT

$$\text{Block} = \text{Tag} \times L + j$$

So tag increments as blocks progress by one full cache-size-in-blocks interval.

10.70 Cache Size in Address Terms

If:

- L lines
- B bytes/line

then total data cache:

$$\boxed{L \times B}$$

This also represents the address-space span covered by one complete set of line indices, but not a contiguous resident main-memory region necessarily.

CHAPTER 10 — MASTER FORMULAS

Mapping

$$\boxed{\text{Line} = \text{BlockNumber} \bmod \text{NumberOfLines}}$$

Block number

$$\boxed{\text{Block} = \left\lfloor \text{Address} / \text{BlockSize} \right\rfloor}$$

Offset

$$\boxed{\text{Offset} = \text{Address} \bmod \text{BlockSize}}$$

Tag from block number

$$\boxed{\text{Tag} = \left\lfloor \text{Block} / \text{Lines} \right\rfloor}$$

Reverse block

$$\boxed{\text{Block} = \text{Tag} \times \text{Lines} + \text{Index}}$$

Byte address

$$\boxed{\text{Address} = \text{Block} \times \text{BlockSize} + \text{Offset}}$$

Address fields

$$\boxed{\text{AddressBits} = \text{Tag} + \text{Index} + \text{Offset}}$$

CHAPTER 10 — FAST GATE SHORTCUT

If:

TEXT

$$\begin{aligned} \text{Address} &= A \text{ bits} \\ \text{Cache} &= 2^C \text{ bytes} \end{aligned}$$

Block = 2^B bytes

then direct mapped:

$\boxed{\text{Tag} = A - C}$

$\boxed{\text{Index} = C - B}$

$\boxed{\text{Offset} = B}$

Think:

A ! C ! B

TEXT

Tag = A - C

Index = C - B

Offset = B

CHAPTER 10 — GATE TRAPS

1. Index selects **lines**, not bytes.
2. Direct mapping needs no LRU victim selection.
3. Offset bits depend on addressability.
4. `block mod lines` gives line, not tag.
5. Tag must match and valid bit must be 1.
6. Cache capacity usually excludes metadata unless specified.
7. Conflict misses can occur even with empty lines elsewhere.
8. Hex fields only split easily when boundaries align on 4-bit positions.
9. Block size must be converted to bytes for byte-addressed systems.
10. Main-memory physical size and CPU address width are not always the same thing.

CHAPTER SNAPSHOT

1. Direct mapping gives each memory block exactly one possible cache line.
2. Mapping formula is block number mod number of lines.
3. Address fields are Tag–Index–Offset.
4. Offset selects within block.
5. Index selects line.
6. Tag identifies which mapped block is currently stored.
7. Valid bit must also be 1 for a hit.
8. Direct mapping is simple and fast.
9. Direct mapping suffers conflict misses.
10. No replacement algorithm is required because the victim line is predetermined.
11. Tag can be obtained by integer-dividing block number by number of lines.
12. Reverse block = tag $\times$ lines + index.

- 13. Metadata storage can include tag, valid and dirty bits.
 - 14. Binary low-order bits give offset/index quickly.
 - 15. Reference strings should be simulated line-by-line.
-

MUST REMEMBER

$$\lfloor \frac{\text{Line} - \text{Block}}{\text{Lines}} \rfloor$$

$$\lfloor \frac{\text{T} - \text{I} - \text{O}}{\text{Lines}} \rfloor$$

and for standard direct-mapped byte cache:

$$\lfloor \frac{\text{Tag} - \text{A} - \text{C}}{\text{Index} - \text{C} - \text{B}} \rfloor$$

where:

- A = address bits
 - 2^C = cache bytes
 - 2^B = block bytes
-

PRACTICE SET

Q1 — L0 NAT

A direct cache has 16 lines.

Main-memory block 45 maps to which line?

Q2 — L1 NAT

A cache has:

- 32 KB data capacity
- 64-byte blocks.

How many cache lines?

Q3 — L1

A 32-bit byte-addressed system has:

- 32 KB direct cache,
- 64-byte blocks.

Find:

- tag bits,
 - index bits,
 - offset bits.
-

Q4 — L1 NAT

Address = 1000 decimal.

Block size = 32 bytes.

Find block number and offset.

Q5 — L2

A direct cache has 8 lines.

For block number 29, find:

- line,
 - tag.
-

Q6 — L2

A direct cache has 8 lines and 16-byte blocks.

Given:

- Tag = 3
- Index = 5
- Offset = 7

Find decimal byte address.

Q7 — L2

A 24-bit byte-addressed system has:

- 16 KB direct cache,
- block size 32 B.

Find T-I-O bit widths.

Q8 — L2

A 20-bit word-addressed system has:

- block = 8 words,
- cache = 1024 words,
- direct mapping.

Find tag/index/offset bits.

Q9 — L2

A direct cache has four lines and is initially empty.

Block references:

TEXT

0, 4, 0, 4, 1, 5, 1

Find total hits and misses.

Q10 — L2

Which blocks map to line 3 in an 8-line direct cache? Give the first five non-negative block numbers.

Q11 — L3

A 36-bit byte-addressed machine has:

- 256 KB direct-mapped cache,
- 128-byte blocks.

Find:

1. lines,
 2. tag bits,
 3. index bits,
 4. offset bits.
-

Q12 — L3

A direct cache has:

- 2048 lines,
- 32-byte blocks,
- tag = 17 bits,
- valid = 1 bit,
- dirty = 1 bit.

Find total metadata storage in bits.

Q13 — L3

A direct cache has 4 lines and initially empty contents.

Block sequence:

TEXT

0, 1, 4, 5, 0, 1, 4, 5

Find:

- hits,
 - misses,
 - final content of each cache line.
-

CHAPTER 10 COMPLETION STATUS

' Direct mapping definition ' Mapping formula ' Tag-index-offset ' Block/address conversion ' Binary decomposition ' Hexadecimal decomposition ' Hit/miss detection ' Valid-bit logic ' Conflict misses ' Reference-string simulation ' Tag derivation ' Reverse-address calculations ' Metadata storage ' No replacement-algorithm rule ' Word vs byte addressability ' GATE shortcuts and traps

CHAPTER 10: COMPLETE '

COURSE PROGRESS

Completed: 11 / 26 modules

Approximate roadmap progress:

$\sim 42\%$

NEXT !' CHAPTER 11 — FULLY ASSOCIATIVE CACHE

Next we cover:

- why there is **no index field**,
- placement in any cache line,
- tag comparison across all lines,
- associative/CAM lookup intuition,
- replacement requirement,
- LRU/FIFO concepts,
- tag-bit and metadata numericals,
- direct vs fully associative conflict behavior,
- GATE reference-string questions.

USER

next

ASSISTANT

CHAPTER 11 — FULLY ASSOCIATIVE CACHE

Classification: [GATE CORE] **Priority:** ~~Ø=VERY~~ VERY HIGH **Official syllabus connection:** Cache memory mapping

Core idea:

A main-memory block can be placed in any cache line.

This is the opposite extreme of direct mapping.

11.1 What is Fully Associative Mapping?

Suppose cache has 8 lines.

In direct mapping:

TEXT

```
B l o c k   1 3   ! '   e x a c t l y   o n e   s p e c i f i c   l i n e
```

In fully associative mapping:

TEXT

```
B l o c k   1 3   ! '   L i n e   0
                        o r   L i n e   1
                        o r   L i n e   2
                        . . .
                        o r   L i n e   7
```

So:

$\boxed{\text{Any memory block can occupy any cache line}}$

11.2 Why is it Called "Associative"?

Because cache does not first select one line using an index.

Instead, requested tag is compared with stored tags across cache lines.

Conceptually:

TEXT

```
Requested Tag
          %
          % % %   c o m p a r e   w i t h   T a g   0
          % % %
```

USER

next

ASSISTANT

CHAPTER 11 — FULLY ASSOCIATIVE CACHE — CONTINUED

We stopped at the key idea:

In fully associative cache, the requested memory block can be stored in any cache line.

11.2 Why is it Called Associative?

The cache compares the requested tag against all candidate cache-line tags.

Conceptually:

TEXT

```
Requested Tag
%
% % %   c o m p a r e   w i t h   T a g   0
% % %   c o m p a r e   w i t h   T a g   1
% % %   c o m p a r e   w i t h   T a g   2
% % %   c o m p a r e   w i t h   T a g   3
% % %   . . .
```

If one valid line has the matching tag:

$\boxed{\text{Cache Hit}}$

If none matches:

$\boxed{\text{Cache Miss}}$

11.3 Address Format

Very important.

Direct-mapped cache:

TEXT

Tag | Index | Offset

Fully associative cache:

TEXT

Tag | Offset

There is:

$\boxed{\text{No Index Field}}$

because no fixed cache line has to be selected.

Memory trick:

Fully Associative = T-O

Tag !' Offset

11.4 Why No Index?

In direct mapping:

Index tells us the only possible line.

But fully associative allows:

any line.

Therefore there is no need to reserve address bits for selecting one particular line.

11.5 Offset Bits

Offset works exactly as before.

For byte-addressable memory:

$$\boxed{\text{OffsetBits} = \log_2(\text{BlockSize in bytes})}$$

Example:

Block size:

$$64\text{B} = 2^6\text{B}$$

Therefore:

$$\boxed{\text{Offset} = 6 \text{ bits}}$$

11.6 Tag Bits

Because there are no index bits:

$$\boxed{\text{TagBits} = \text{AddressBits} - \text{OffsetBits}}$$

Example:

- 32-bit byte address
- block = 64 B

Offset:

$$6$$

Tag:

$$32 - 6$$

$$\boxed{26}$$

Address:

TEXT

Tag: 26 | Offset: 6

11.7 Fully Associative Mapping Example

Suppose cache has four lines.

Initially:

TEXT

L0 = empty
L1 = empty
L2 = empty
L3 = empty

Access block 10.

It could be stored in any free line.

For example:

TEXT

L0 = 10

Access block 25.

Could use:

TEXT

L1 = 25

There is no modulo restriction such as:

$\lfloor \text{Block} \bmod \text{Lines} \rfloor$

11.8 No Fixed Mapping Formula

For direct mapping:

$\lfloor \text{Line} = \text{Block} \bmod \text{Lines} \rfloor$

For fully associative:

$\lfloor \boxed{\text{No fixed line-mapping formula}} \rfloor$

Any free line can be chosen.

If all lines are occupied, a **replacement policy** must select a victim.

11.9 Replacement Policy

Because several lines are possible candidates, when cache is full we need to decide:

Which existing block should be removed?

Common policies:

- LRU
- FIFO
- Random

These are much more relevant here than in direct mapping.

11.10 LRU

LRU = Least Recently Used

Replace the block that has not been used for the longest time.

Example cache:

TEXT

A, B, C, D

Suppose usage recency is:

TEXT

M o s t r e c e n t ! ' D , B , A , C ! ' L e a s t r e c e n t

A new block arrives.

LRU victim:

$\boxed{C}$

11.11 FIFO

FIFO = First In First Out

Replace the block that entered cache earliest.

Important:

FIFO tracks:

insertion order

not recent usage.

11.12 Random

Choose a victim randomly.

Simple hardware, but less predictable.

11.13 LRU vs FIFO

Suppose blocks enter:

TEXT

A, B, C, D

Then A is oldest.

But later A is accessed repeatedly.

FIFO

A may still be replaced because it entered first.

LRU

A becomes recent, so some less-recent block may be replaced instead.

Do not confuse insertion age with usage age.

11.14 Associative Lookup

Fully associative cache needs to search for a tag across multiple lines.

Special hardware conceptually performs:

$\bigvee [\text{RequestedTag} == \text{StoredTag}]$

for many lines.

This is sometimes associated with:

$\bigvee [\text{Content Addressable Memory (CAM)}]$

11.15 What is CAM?

CAM = Content Addressable Memory

Normal memory:

give address !' get content.

CAM:

give content/key !' find matching entry.

This is useful for tag matching.

[SUPPORT]

You mainly need the intuition, not deep CAM circuit design.

11.16 Hardware Cost

Suppose cache has 1024 lines.

Fully associative lookup may need comparison against many tags.

This means:

- more comparators,
- more wiring,
- more cost,
- more complexity.

Therefore fully associative caches are expensive for large caches.

11.17 Advantage

Placement flexibility is maximum.

Any block can use any line.

Therefore:

$\boxed{\text{Conflict misses are minimized/eliminated}}$

More precisely:

fully associative placement eliminates misses caused purely by fixed line/set conflicts.

11.18 Remaining Misses

Fully associative mapping does **not** eliminate:

Compulsory misses

First access to a block.

Capacity misses

Cache is simply too small.

So:

TEXT

```
C o n f l i c t   m i s s e s ! '   e s s e n t i a l l y   e l i m i n a t e d
C o m p u l s o r y   m i s s e s ! '   r e m a i n
C a p a c i t y   m i s s e s ! '   r e m a i n
```

11.19 Direct vs Fully Associative

Feature	Direct	Fully Associative
Placement	One line	Any line
Index bits	Present	0
Tag comparisons	One	Many/all
Hardware cost	Low	High
Hit lookup simplicity	High	Lower
Conflict misses	More	Eliminated by placement flexibility
Replacement policy	Not needed for choice	Required when full

11.20 Numerical Example

Given:

- 36-bit byte addresses
- block size = 32 B
- fully associative cache

Offset:

$$\lceil \log_2 32 = 5 \rceil$$

$$\lceil \text{Offset} = 5 \rceil$$

Tag:

$$\lceil 36 - 5 \rceil$$

$$\lceil \boxed{31} \rceil$$

Notice cache capacity does not affect tag bits in fully associative mapping.

11.21 Why Cache Size Does Not Affect FA Tag Width

Address structure:

TEXT

Tag | Offset

There is no index.

So:

$$\lceil \text{Tag} = \text{AddressBits} - \text{Offset} \rceil$$

Increasing number of cache lines changes:

- number of available placements,
- total hardware,

but not the address-field split.

11.22 Word-Addressable Example

Suppose:

- 24-bit word addresses
- block = 16 words
- fully associative

Offset:

$$\lceil \log_2 16 = 4 \rceil$$

Tag:

$$\lceil 24 - 4 \rceil$$

$$\lceil \boxed{20} \rceil$$

11.23 Byte-Addressed Word Example

Suppose:

- 32-bit byte address
- word = 4 bytes
- block = 8 words

Block bytes:

$8 \times 4 = 32\text{B}$

Offset:

5

Tag:

$32 - 5$

$\boxed{27}$

Offset can also be split:

TEXT

Word offset = 3
Byte offset = 2

Total:

5

11.24 Tag Storage

Suppose:

- fully associative cache has 256 lines
- tag = 27 bits
- valid = 1 bit

Metadata per line:

$27 + 1 = 28$

Total:

256×28

$\boxed{7168 \text{ bits}}$

excluding replacement metadata.

11.25 Replacement Metadata

Fully associative caches need information for victim selection.

For LRU, some extra state must track recency.

Exact number of LRU bits depends on implementation.

Therefore:

Do not assume LRU metadata size unless the problem gives a representation or asks for a specific theoretical encoding.

11.26 Fully Associative Hit Test

Address:

TEXT
Tag | Offset

For each line:

1. check valid bit,
2. compare stored tag.

If any line satisfies both:

$\{ \text{Valid}=1 \}$

and

$\{ \text{StoredTag}=\text{RequestedTag} \}$

then hit.

11.27 Example

Cache contains:

TEXT
Line 0: V=1, Tag=00110
Line 1: V=1, Tag=11100
Line 2: V=0, Tag=10101
Line 3: V=1, Tag=01011

Request:

TEXT
Tag=11100

Line 1 matches and is valid.

Therefore:

$\{ \boxed{\text{Hit}} \}$

11.28 Miss Example

Request:

TEXT

Tag=10101

Line 2 has matching bits, but:

TEXT

Valid=0

Therefore:

$\boxed{\text{Miss}}$

11.29 Reference String with Fully Associative Cache

Suppose cache has 3 lines and uses LRU.

Block sequence:

TEXT

1, 2, 3, 1, 4, 2

Initially empty.

Let's simulate.

Access 1

Miss.

Cache:

TEXT

1

Access 2

Miss.

TEXT

1, 2

Access 3

Miss.

TEXT

1, 2, 3

Usage order:

TEXT

Most recent: 3,2,1 : least recent

Access 1

Hit.

Now recency becomes:

TEXT

1, 3, 2

So 2 is least recently used.

Access 4

Miss.

Cache full.

Replace LRU block 2.

Cache now:

TEXT

1, 3, 4

Recency:

TEXT

4, 1, 3

Access 2

Miss.

LRU is 3.

Replace 3.

Final cache:

TEXT

1, 4, 2

Totals:

\[Hits=1 \]

\[Misses=5 \]

11.30 Same Sequence with FIFO

Sequence:

TEXT

1, 2, 3, 1, 4, 2

3-line cache.

Insertion order initially:

TEXT

1, 2, 3

Access 1 is a hit, but FIFO order stays unchanged.

Next new block 4 arrives.

Oldest = 1.

So FIFO replaces 1, unlike LRU.

This shows the policies can produce different results.

11.31 GATE Trap — Hit Does Not Change FIFO Order

FIFO cares only about insertion time.

On a hit:

block does not become "newly inserted."

So FIFO queue usually remains unchanged.

11.32 GATE Trap — LRU Hit Changes Recency

LRU cares about usage.

On a hit:

accessed block becomes most recently used.

This must update recency information.

11.33 Associative Cache Block Number

For fully associative cache, block number is essentially represented by the tag when offset is removed.

For byte-addressable memory:

$$\lfloor \text{BlockNumber} = \left\lfloor \frac{\text{Address}}{\text{BlockSize}} \right\rfloor \rfloor$$

The block number bits become tag bits.

11.34 Example

32-bit byte address.

Block size:

$$\lfloor 64B = 2^6 \rfloor$$

Block number requires:

$$\lfloor 32 - 6 = 26 \rfloor$$

bits.

Those 26 bits are exactly the fully associative tag.

11.35 Reverse Address Calculation

Given:

- tag = block number
- offset = O
- block size = B

Then:

$$\lfloor \boxed{\text{Address} = \text{Tag} \times B + \text{Offset}} \rfloor$$

because there is no index field.

Example

Block/tag:

$$\lfloor 25 \rfloor$$

Block size:

$$\lfloor 16B \rfloor$$

Offset:

$$\lfloor 7 \rfloor$$

Address:

$$\lfloor 25 \times 16 + 7 \rfloor$$

$$\lfloor = 400 + 7 \rfloor$$

$$\lfloor \boxed{407} \rfloor$$

11.36 Main Memory Block Count

Suppose:

- address width = 30 bits
- byte-addressable
- block = 64 B

Memory blocks:

$$\lfloor 2^{30}/2^6 \rfloor$$

$$\lfloor 2^{24} \rfloor$$

Therefore tag:

$$\lfloor \boxed{24 \text{ bits}} \rfloor$$

for fully associative cache.

11.37 Cache Capacity Example

Suppose:

- 512 lines
- 64 bytes/line

Data capacity:

$$\lfloor 512 \times 64 \rfloor$$

$$\lfloor 2^9 \times 2^6 \rfloor$$

$$\lfloor 2^{15} \text{ B} \rfloor$$

$$\lfloor \boxed{32 \text{ KB}} \rfloor$$

Again, this capacity does not enter FA tag width directly.

11.38 Hardware Comparator Count — Simplified Concept

If cache has L lines and does a fully parallel lookup, conceptually up to:

$$\lfloor \boxed{L} \rfloor$$

tag comparisons may occur.

This is why hardware cost increases substantially.

Do not interpret this as a universal implementation rule unless the question frames it that way.

11.39 Fully Associative and LRU

Because all lines are in one set, LRU must conceptually choose among all cache lines.

For large caches, exact LRU becomes expensive.

This is one practical reason fully associative organization is rarely used for large caches.

11.40 Where Fully Associative Structures Are Useful

[PLACEMENT/INTERVIEW]

Fully associative lookup is useful where structures are relatively small and fast matching is valuable.

Examples conceptually include:

- TLBs,
- small lookup buffers.

But your GATE focus here is mapping logic.

11.41 Direct vs Associative Conflict Example

Cache has 2 lines.

Block sequence:

```
TEXT
0, 2, 0, 2
```

Direct

Both map to line 0.

Result:

```
TEXT
M M M M
```

Fully Associative

First:

0 !' miss 2 !' miss

Both can coexist.

Then:

0 !' hit 2 !' hit

Result:

```
TEXT
M M H H
```

This perfectly shows why associativity reduces conflict misses.

11.42 Capacity Miss Example

Fully associative cache has 2 lines.

Sequence:

```
TEXT
0, 1, 2, 0
```

After 0 and 1, cache full.

Access 2 forces one block out.

Later 0 may miss.

This is not a direct-mapping conflict issue.

Cache simply cannot hold all three blocks.

11.43 Compulsory Miss Example

Even with infinite associativity:

First access to block 10:

$\boxed{\text{Miss}}$

because cache has never loaded it.

Associativity cannot eliminate first-touch misses.

11.44 Formula Sheet

Offset

For byte-addressed cache:

$\boxed{\text{OffsetBits} = \log_2(\text{BlockSizeBytes})}$

Fully associative tag

$\boxed{\text{TagBits} = \text{AddressBits} - \text{OffsetBits}}$

Cache data capacity

$\boxed{\text{Lines} \times \text{BlockSize}}$

Metadata without replacement state

$\boxed{\text{Lines} \times (\text{TagBits} + \text{ValidBits} + \text{DirtyBits} \dots)}$

11.45 Fast GATE Method

Given:

- A address bits
- block size $(=2^B)$ bytes

Immediately write:

TEXT

Fully associative:

Offset = B

Index = 0

Tag = $A - B$

That's all for address decomposition.

11.46 GATE Example 1

Given:

- 40-bit byte addresses
- block = 128 B
- fully associative

$$\lfloor 128=2^7 \rfloor$$

Offset:

$$\lfloor 7 \rfloor$$

Tag:

$$\lfloor 40-7=33 \rfloor$$

Answer:

$$\lfloor \boxed{\text{Tag}=33, \text{Offset}=7} \rfloor$$

11.47 GATE Example 2

Fully associative cache:

- 1024 lines
- 32 B/line
- 36-bit byte addresses
- valid bit = 1

Offset:

$$\lfloor 5 \rfloor$$

Tag:

$$\lfloor 31 \rfloor$$

Metadata per line:

$$\lfloor 31+1=32 \rfloor$$

Total metadata:

$$\lfloor 1024 \times 32 \rfloor$$

$$\lfloor \boxed{32768 \text{ bits}} = 4\text{KB} \rfloor$$

11.48 GATE Example 3 — Reverse Address

Fully associative:

- block size 32 B
- tag/block number = 100
- offset = 7

Address:

$$\lfloor 100 \times 32 + 7 \rfloor$$

$\lfloor =3207 \rfloor$

$\lfloor \boxed{3207} \rfloor$

11.49 GATE Example 4 — LRU

Cache has 3 lines.

Sequence:

TEXT

A, B, C, A, D, B

LRU:

- A !' M
- B !' M
- C !' M
- A !' H
- D !' M, replace B
- B !' M

Hits:

$\lfloor \boxed{1} \rfloor$

Misses:

$\lfloor \boxed{5} \rfloor$

11.50 GATE Example 5 — No Index

Which field is absent from a fully associative cache address?

$\lfloor \boxed{\text{Index/Set field}} \rfloor$

Only:

TEXT

Tag | Offset

11.51 GATE Traps

Trap 1

Fully associative means every block has a fixed line.

'L Wrong.

It can go anywhere.

Trap 2

Fully associative has index bits.

'LWrong.

\[Index=0 \]

Trap 3

Fully associative needs no replacement policy.

'LWrong.

When full, some line must be chosen.

Trap 4

Fully associative eliminates every miss.

'LWrong.

Compulsory and capacity misses remain.

Trap 5

FIFO and LRU are same.

'LWrong.

FIFO tracks insertion.

LRU tracks recent use.

Trap 6

Cache size determines fully associative tag width.

Not directly.

Tag width depends on:

- address width,
 - block offset.
-

Trap 7

On an LRU hit, recency doesn't change.

'LWrong.

It does.

CHAPTER 11 — MASTER COMPARISON

Property	Direct	Fully Associative
Address	Tag + Index + Offset	Tag + Offset
Placement choices/block	1	All lines
Index	Yes	No
Tag comparators	1	Many
Replacement choice	None	Required
Hardware	Simple	Expensive
Conflict misses	More	Eliminated by unrestricted placement
Lookup	Simple	Associative search

MEMORY TRICKS

Fully Associative

FA = Free Anywhere

A block is **free to go anywhere**.

Address

FA !' T-O

Tag !' Offset.

Replacement

Anywhere placement means replacement choice matters.

LRU

Least Recently Used = forgotten for longest

FIFO

First entered = first removed

CHAPTER SNAPSHOT

1. Fully associative cache allows any block in any line.

2. There is no index field.
3. Address format is Tag + Offset.
4. Tag contains all block-number bits.
5. All relevant tags must be searched/compared.
6. Associative lookup requires more hardware.
7. Fully associative mapping minimizes conflict misses.
8. Compulsory and capacity misses remain.
9. Replacement policy is needed when cache is full.
10. LRU tracks recency of use.
11. FIFO tracks insertion order.
12. Hits change LRU order but usually not FIFO order.
13. Cache size does not directly determine FA tag width.
14. Metadata includes at least tag and valid bits.
15. Fully associative hardware is costly for large caches.

MUST REMEMBER

$\boxed{FA = \text{Tag} + \text{Offset}}$

$\boxed{\text{IndexBits} = 0}$

$\boxed{\text{TagBits} = \text{AddressBits} - \log_2(\text{BlockSize})}$

for byte-addressable memory.

And:

Direct = one place. Fully associative = anywhere.

PRACTICE SET

Q1 — L0 MCQ

In fully associative mapping, a main-memory block can be placed:

- A. Only in line 0 B. Only in one predetermined line C. In any cache line D. Only in even-numbered lines

Q2 — L0 NAT

A fully associative cache has 32-bit byte addresses and 64-byte blocks.

How many offset bits?

Q3 — L1 NAT

For Q2, how many tag bits?

Q4 — L1 MCQ

Which address field is absent in fully associative cache?

A. Tag B. Offset C. Index D. Valid

Q5 — L1

A fully associative cache has 256 lines, 16-byte blocks, and 24-bit byte addresses.

Find tag bits.

Q6 — L2 NAT

The cache in Q5 has one valid bit per line.

Find total tag + valid storage in bits.

Q7 — L2

A fully associative cache has 3 lines and uses LRU.

Reference sequence:

TEXT

1, 2, 3, 1, 4, 2

Find hits and misses.

Q8 — L2

For the same sequence, repeat using FIFO.

Q9 — L2

A fully associative cache uses 36-bit byte addresses and 128-byte blocks.

Find tag and offset widths.

Q10 — L2

Block size = 16 B.

Tag/block number = 50.

Offset = 9.

Find byte address.

Q11 — L3

A fully associative cache has:

- 1024 lines,
- 64-byte blocks,
- 40-bit byte addresses,
- 1 valid bit,
- 1 dirty bit.

Find total metadata bits excluding replacement information.

Q12 — L3

A 2-line fully associative cache uses LRU.

Block sequence:

TEXT

0, 1, 0, 2, 1, 0

Find:

- hits,
- misses,
- final cache contents.

CHAPTER 11 COMPLETION STATUS

' Fully associative mapping ' No-index address format ' Tag calculation ' Offset calculation ' Associative/CAM lookup intuition ' Hit/miss detection ' Replacement need ' LRU ' FIFO ' Random replacement overview ' Conflict vs capacity vs compulsory misses ' Metadata calculations ' Reference-string simulation ' Reverse-address calculation ' Direct vs fully associative comparison ' GATE traps

CHAPTER 11: COMPLETE '

COURSE PROGRESS

Completed: 12 / 26 modules

Approximate roadmap progress:

$\boxed{\sim 46\%}$

NEXT !' CHAPTER 12 — SET-ASSOCIATIVECACHE

This is the most important compromise between direct and fully associative mapping.

Next we will master:

- k-way associativity,
- number of sets,

- block !' set mapping,
- **Tag-Set-Offset**,
- 2-way / 4-way / 8-way numericals,
- replacement within a set,
- LRU inside sets,
- hit/miss reference strings,
- tag storage,
- direct vs set vs fully associative comparison,
- fast GATE shortcuts.

USER

next

ASSISTANT

CHAPTER 12 — SET-ASSOCIATIVECACHE

Classification: [GATE CORE] **Priority:** $\emptyset = \text{VERY HIGH}$ **Official syllabus connection:** Cache memory mapping

Set-associative mapping is the middle ground between:

- Direct Mapping
- Fully Associative Mapping

Core idea:

A memory block maps to exactly one set, but can occupy any line inside that set.

12.1 Why Set-Associative Mapping?

Recall:

Direct Mapping

One memory block has exactly one possible cache line.

Advantage:

- simple,
- fast.

Problem:

- many conflict misses.

Fully Associative

A block can go anywhere.

Advantage:

- very flexible,
- avoids fixed-placement conflict.

Problem:

- expensive associative comparison across many lines.

Set-associative mapping combines both ideas:

$\boxed{\text{One set selected, multiple lines available inside it}}$

12.2 What Does k-Way Mean?

Suppose cache is:

$4 \text{ text{-}way set associative}$

This means:

each set contains 4 cache lines.

Not:

cache has 4 sets.

$\emptyset = \text{Very common trap.}$

12.3 Number of Sets

If:

- Total cache lines = (L)
- Associativity = (k)

Then:

$\boxed{\text{Number of Sets} = \frac{L}{k}}$

Example

Cache has:

1024 lines

and is 4-way.

Then:

$\text{Sets} = \frac{1024}{4} = 256$

$\boxed{256 \text{ sets}}$

Each set has:

$\boxed{4 \text{ lines}}$

$$\boxed{\text{SetNumber} = \text{BlockNumber} \bmod S}$$

Compare:

Direct

$$\boxed{\text{Line} = \text{Block} \bmod \text{Lines}}$$

Set Associative

$$\boxed{\text{Set} = \text{Block} \bmod \text{Sets}}$$

12.8 Example

Suppose:

- 4 sets
- 2-way set associative

Block:

$$\boxed{13}$$

Then:

$$\boxed{13 \bmod 4 = 1}$$

Therefore:

$$\boxed{\text{Block } 13 \rightarrow \text{Set } 1}$$

Inside Set 1, it may occupy either of the two ways.

12.9 Placement Example

2-way cache:

TEXT

Set	0	Way	0		Way	1
Set	1	Way	0		Way	1
Set	2	Way	0		Way	1
Set	3	Way	0		Way	1

Suppose block 5:

$$\boxed{5 \bmod 4 = 1}$$

So block 5 can occupy:

TEXT

Set 1, Way 0
or
Set 1, Way 1

But it cannot go to:

- Set 0,
- Set 2,
- Set 3.

12.10 Number of Cache Lines

Still:

$$\boxed{\text{Lines} = \frac{\text{CacheSize}}{\text{BlockSize}}}$$

Associativity changes how those lines are grouped.

12.11 Set Bits

If number of sets is:

$$2^s$$

then:

$$s = \log_2(\text{NumberOfSets})$$

set-index bits.

12.12 Offset Bits

For byte-addressed memory:

$$\boxed{\text{OffsetBits} = \log_2(\text{BlockSizeBytes})}$$

12.13 Tag Bits

If address width = A :

$$\boxed{\text{Tag} = A - \text{SetBits} - \text{OffsetBits}}$$

12.14 Complete Example — 2-Way Cache

Given:

- 32-bit byte addresses
- Cache size = 16 KB
- Block size = 32 B
- 2-way set associative

Step 1 — Lines

$$16\text{KB} = 2^{14}\text{B}$$

$$32\text{B} = 2^5\text{B}$$

$$\text{Lines} = 2^{14-5} = 2^9 = 512$$

Step 2 — Sets

2-way means:

$$\text{Sets} = \frac{512}{2} = 256 = 2^8$$

So:

$$\boxed{\text{SetBits}=8}$$

Step 3 — Offset

$$32B=2^5$$

$$\boxed{\text{Offset}=5}$$

Step 4 — Tag

$$\text{Tag}=32-8-5$$

$$\boxed{19}$$

Final:

$$\boxed{\text{Tag}=19, \text{Set}=8, \text{Offset}=5}$$

12.15 Complete Example — 4-Way

Same system, but 4-way.

Lines:

$$512$$

Sets:

$$512/4=128=2^7$$

So:

$$\text{SetBits}=7$$

Offset:

$$5$$

Tag:

$$32-7-5$$

$$\boxed{20}$$

Notice:

increasing associativity decreases set bits and increases tag bits.

12.16 Complete Example — 8-Way

Same cache:

$$512 \text{ lines}$$

8-way:

$$\text{Sets}=512/8=64=2^6$$

Set bits:

$\lfloor 6 \rfloor$

Offset:

$\lfloor 5 \rfloor$

Tag:

$\lfloor 32-6-5 \rfloor$

$\lfloor \boxed{21} \rfloor$

12.17 Pattern with Increasing Associativity

For fixed:

- cache size,
- block size,
- address width.

As associativity increases:

TEXT

```
N u m b e r   o f   s e t s   ! "
S e t   b i t s   ! "
T a g   b i t s   ! '

```

At fully associative:

TEXT

```
Sets = 1
Set bits = 0

```

12.18 Fast Tag Formula

For a standard byte-addressed $\lfloor k \rfloor$ -way cache:

If:

- address width = A
- cache size = $\lfloor 2^C \rfloor$ bytes
- associativity = $\lfloor k=2^w \rfloor$

Then:

$\lfloor \boxed{\text{Tag} = A - C + w} \rfloor$

because:

$\lfloor \text{SetBits} = C - \text{BlockBits} - w \rfloor$

and:

$\lfloor \text{Offset} = \text{BlockBits} \rfloor$

So:

$\lfloor \text{Tag} = A - (C - B - w) - B \rfloor$

$$\boxed{\text{Tag}=\text{A}-\text{C}+w}$$

Example

32-bit addresses.

Cache:

$$64\text{KB}=2^{16}$$

4-way:

$$w=2$$

Then:

$$\text{Tag}=32-16+2$$

$$\boxed{18}$$

Very fast.

12.19 When This Shortcut Is Safe

Use when:

- byte-addressed,
- cache size = data capacity,
- power-of-two cache/block/ways,
- conventional set associative cache.

Otherwise use full method.

12.20 Mapping from Block Number

Suppose:

- 8 total sets
- block number = 29

Then:

$$29 \bmod 8 = 5$$

So:

$$\boxed{\text{Set}=5}$$

12.21 Tag from Block Number

If there are S sets:

$$\boxed{\text{Tag}=\left\lfloor \frac{\text{BlockNumber}}{S} \right\rfloor}$$

Example:

$$\text{Block}=29, \quad S=8$$

$\lfloor \text{Tag} = \lfloor \text{floor} 29/8 \rfloor \text{floor} = 3 \rfloor$

Thus:

TEXT

Block 29 !' Tag 3 , Set 5

because:

$\lfloor 29 = 3(8) + 5 \rfloor$

12.22 Reverse Formula

Given tag T and set number S_i :

$\lfloor \boxed{\text{BlockNumber} = T \times \text{NumberOfSets} + \text{SetIndex}} \rfloor$

Example:

- 8 sets
- Tag = 4
- Set = 3

$\lfloor \text{Block} = 4(8) + 3 \rfloor$

$\lfloor \boxed{35} \rfloor$

12.23 Address from Tag, Set, Offset

Suppose:

- Number of sets = S
- Block size = B
- Tag = T
- Set index = I
- Offset = O

Then:

$\lfloor \text{Block} = T \cdot S + I \rfloor$

and:

$\lfloor \boxed{\text{Address} = \text{Block} \cdot B + O} \rfloor$

Example

- 4 sets
- block size = 16 B
- Tag = 5
- Set = 2
- Offset = 7

Block:

$$\lfloor 5(4)+2=22 \rfloor$$

Address:

$$\lfloor 22(16)+7 \rfloor$$

$$\lfloor 352+7 \rfloor$$

$$\lfloor \boxed{359} \rfloor$$

12.24 How a Set-Associative Lookup Works

Suppose 4-way cache.

CPU address gives:

TEXT

Tag | Set | Offset

Step 1

Set bits select one set.

Step 2

Requested tag is compared with all 4 tags in that set.

Step 3

If any valid tag matches:

$$\lfloor \boxed{\text{Hit}} \rfloor$$

Step 4

Offset selects required data.

12.25 Comparator Count Intuition

For a k-way cache, a hit lookup generally needs up to k tag comparisons in the selected set.

Conceptually:

TEXT

Requested Tag

! " ! " ! " ! "

Way0 Way1 Way2 Way3

For 4-way:

$$\lfloor \boxed{4 \text{ candidate tag comparisons}} \rfloor$$

This is more hardware than direct mapping but far less than full associativity across all lines.

12.26 Why Set Associative Is a Compromise

Compared with Direct

- fewer conflict misses,
- more hardware.

Compared with Fully Associative

- less flexible,
- cheaper,
- fewer tag comparisons.

Thus:

$\boxed{\text{Set associative balances performance and cost}}$

12.27 Replacement Is Only Within the Selected Set

This is crucial.

Suppose block maps to Set 3.

If Set 3 is full:

choose a victim only among the ways of Set 3.

You cannot replace a line in Set 0 or Set 7.

12.28 Replacement Policies

Common:

- LRU
- FIFO
- Random

These operate:

$\boxed{\text{within each set}}$

12.29 Example — 2-Way LRU

Cache has:

- 2 sets
- 2 ways/set

Block mapping:

$\text{Set} = \text{Block} \bmod 2$

Reference sequence:

TEXT

0, 2, 0, 4

All are even:

$\lfloor 0 \bmod 2 = 0 \rfloor$

$\lfloor 2 \bmod 2 = 0 \rfloor$

$\lfloor 4 \bmod 2 = 0 \rfloor$

So all use Set 0.

Simulation

Access 0

Miss.

Set 0:

TEXT

[0, empty]

Access 2

Miss.

TEXT

[0, 2]

Access 0

Hit.

0 becomes most recently used.

So 2 becomes LRU.

Access 4

Miss.

Set full.

Replace block 2.

Final Set 0:

TEXT

[0, 4]

Total:

$\lfloor \boxed{1 \text{ hit}, 3 \text{ misses}} \rfloor$

12.30 Compare with Direct Mapping

Suppose same total cache has 4 lines direct mapped.

Sequence:

TEXT

0, 2, 0, 4

Different behavior can occur depending on line mapping.

The key is:

set associativity reduces collision pressure by giving each mapped set multiple ways.

12.31 Reference String Example

Cache:

- 4 total lines
- 2-way
- therefore 2 sets
- LRU
- initially empty

Block sequence:

TEXT

0, 2, 1, 3, 0, 4, 2

Mapping:

TEXT

E	v	e	n		b	l	o	c	k	s	!	'		S	e	t		0
O	d	d			b	l	o	c	k	s	!	'		S	e	t		1

Access 0

Set 0 !' miss.

TEXT

Set0: [0, -]

Access 2

Set 0 !' miss.

TEXT

Set0: [0, 2]

2 most recent, 0 LRU.

Access 1

Set 1 !' miss.

TEXT

Set1: [1, -]

Access 3

Set 1 !' miss.

TEXT

Set1: [1, 3]

Access 0

Set 0 contains 0.

$\boxed{\text{Hit}}$

Now 2 becomes LRU in Set 0.

Access 4

Maps Set 0.

Set 0 full.

Replace LRU = 2.

TEXT

Set0: [0, 4]

Access 2

Maps Set 0.

Not present.

Replace LRU = 0.

Final:

TEXT

Set0: [4, 2]

Set1: [1, 3]

Totals:

$\boxed{\text{Hits}=1}$

$\lfloor \boxed{\text{Misses}=6} \rfloor$

12.32 LRU Must Be Maintained Per Set

A block in Set 1 does not affect recency order in Set 0.

Each set maintains its own replacement state.

This is a common conceptual trap.

12.33 FIFO Within Set

FIFO also works independently per set.

Example:

Set 2 contains:

TEXT

A entered first
B entered second

Even if A was just accessed, FIFO may still remove A when a new block arrives.

12.34 Direct Mapping as 1-Way Set Associative

Direct mapped cache is simply:

$\lfloor \boxed{1 \text{ text{-way set associative}}} \rfloor$

Because each set has exactly one line.

Thus no replacement choice.

12.35 Fully Associative as One-Set Cache

Fully associative cache is:

$\lfloor \boxed{\text{One set with all cache lines}} \rfloor$

This unifies all three techniques.

12.36 General Mapping Rule

For k-way cache:

$\lfloor \boxed{\text{Set} = \text{Block} \bmod \text{NumberOfSets}} \rfloor$

Then choose one of the k ways within that set.

Special cases:

Direct

$$\forall (k=1)$$

Fully associative

Number of sets = 1.

Then:

$$\forall \text{ Block } \bmod 1 = 0$$

Every block maps to the only set.

12.37 Example — 4-Way Mapping

Cache has:

$$64 \text{ lines}$$

4-way.

Sets:

$$64/4 = 16$$

Block 100:

$$100 \bmod 16 = 4$$

Therefore:

$$\boxed{\text{Set}=4}$$

Block can occupy any of four ways in Set 4.

12.38 Address Bits Example

Given:

- 36-bit byte addresses
- Cache = 256 KB
- Block = 64 B
- 4-way

Cache:

$$256 \text{ KB} = 2^{18} \text{ B}$$

Block:

$$64 \text{ B} = 2^6$$

Lines:

$$2^{18-6} = 2^{12}$$

Sets:

$$2^{12}/4 = 2^{10}$$

Set bits:

$$10$$

Offset:

$$\lfloor 6 \rfloor$$

Tag:

$$\lfloor 36-10-6 \rfloor$$

$$\lfloor \boxed{20} \rfloor$$

So:

$$\lfloor \boxed{20|10|6} \rfloor$$

12.39 Same Cache as 8-Way

Ways:

$$\lfloor 8=2^3 \rfloor$$

Sets:

$$\lfloor 2^{12}/2^3=2^9 \rfloor$$

Set bits:

$$\lfloor 9 \rfloor$$

Tag:

$$\lfloor 36-9-6 \rfloor$$

$$\lfloor \boxed{21} \rfloor$$

12.40 Same Cache Fully Associative

Sets:

$$\lfloor 1 \rfloor$$

Set bits:

$$\lfloor 0 \rfloor$$

Tag:

$$\lfloor 36-6 \rfloor$$

$$\lfloor \boxed{30} \rfloor$$

12.41 Comparison for Same Cache

Organization	Set/Index Bits	Tag Bits
Direct	12	18
2-way	11	19
4-way	10	20

Organization	Set/Index Bits	Tag Bits
8-way	9	21
Fully associative	0	30

with 36-bit addresses, 256 KB cache, 64 B blocks.

Pattern:

each doubling of associativity removes 1 set bit and adds 1 tag bit.

Useful shortcut.

12.42 Why Doubling Ways Changes One Bit

Suppose ways:

$k \rightarrow 2k$

Number of sets halves.

Therefore:

$\text{SetBits} \rightarrow \text{SetBits} - 1$

Total address width fixed, so:

$\text{TagBits} \rightarrow \text{TagBits} + 1$

12.43 Block Distribution Across Sets

If cache has 8 sets:

Set 0 receives:

TEXT

0, 8, 16, 24, ...

Set 1 receives:

TEXT

1, 9, 17, 25, ...

Set 2:

TEXT

2, 10, 18, 26, ...

General for set j :

$\boxed{j, j+S, j+2S, \dots}$

where S = number of sets.

12.44 Number of Main-Memory Blocks Mapping to a Set

Suppose main memory has M blocks and cache has S sets.

Then blocks mapping to each set:

$$\boxed{\frac{M}{S}}$$

assuming regular power-of-two organization.

12.45 Tag Values Per Set

Number of possible tags for each set equals:

$$\frac{\text{MainMemoryBlocks}}{\text{NumberOfSets}}$$

If this is:

$$2^t$$

then tag requires:

$$t$$

bits.

12.46 Example

Main memory:

- 32-bit byte addresses
- block = 64 B

Memory blocks:

$$2^{32-6}=2^{26}$$

Cache:

- 4-way
- 64 KB
- 64 B block

Lines:

$$2^{16-6}=2^{10}$$

Sets:

$$2^{10}/4=2^8$$

Blocks per set:

$$2^{26}/2^8=2^{18}$$

Tag:

$$\boxed{18 \text{ bits}}$$

Same as address decomposition.

12.47 Tag Storage in Set-Associative Cache

Suppose:

- Lines = L
- Tag = t bits
- Valid = 1 bit/line

Total tag+valid storage:

$$L(t+1)$$

Associativity does not change the fact that every line needs its own tag.

12.48 Example — Metadata

Cache:

- 512 lines
- tag = 20 bits
- valid = 1 bit
- dirty = 1 bit

Metadata:

$$512(20+1+1)$$

$$= 512 \times 22$$

$$11264 \text{ bits}$$

excluding replacement bits.

12.49 Replacement Metadata — LRU

For a 2-way set, exact LRU can often be represented by roughly one bit per set:

which way was least recently used.

For higher associativity, exact LRU requires more state.

But do not assume a universal formula unless the question specifies encoding.

12.50 2-Way LRU Bit Intuition

For each set with two ways:

TEXT

Way 0

Way 1

Only need to know which one is older.

Therefore one bit can represent:

TEXT

0	!	'	W	a	y	0	i	s	L	R	U
1	!	'	W	a	y	1	i	s	L	R	U

This may appear in storage-overhead problems.

12.51 Total Storage Example with LRU

Suppose:

- 128 sets
- 2 ways/set
- tag = 18 bits
- valid = 1 bit/line
- one LRU bit per set

Total lines:

$$\lfloor 128 \times 2 = 256 \rfloor$$

Tag+valid:

$$\lfloor 256(18+1) = 4864 \rfloor$$

LRU:

$$\lfloor 128 \rfloor$$

Total metadata:

$$\lfloor 4864 + 128 \rfloor$$

$$\lfloor \boxed{4992} \text{ bits} \rfloor$$

assuming no dirty bits.

12.52 Byte vs Word Addressability

Suppose block has:

$$\lfloor 16 \text{ words} \rfloor$$

Word-addressable

Offset:

$$\lfloor \log_2 16 = 4 \rfloor$$

Byte-addressable with 4-byte words

Block size:

$$\lfloor 16 \times 4 = 64 \text{ B} \rfloor$$

Offset:

$$\lfloor \log_2 64 = 6 \rfloor$$

Again:

always determine what one address refers to.

12.53 Example — Word Addressable Set Cache

Given:

- 24-bit word addresses
- cache = 2048 words
- block = 8 words
- 4-way

Lines:

$$\lfloor 2048/8=256=2^8 \rfloor$$

Sets:

$$\lfloor 256/4=64=2^6 \rfloor$$

Offset:

$$\lfloor \log_2 8=3 \rfloor$$

Set:

$$\lfloor 6 \rfloor$$

Tag:

$$\lfloor 24-6-3 \rfloor$$

$$\lfloor \boxed{15} \rfloor$$

12.54 Example — Byte Addressed Word Cache

Given:

- 32-bit byte addresses
- 4-byte words
- 8 words/block
- cache = 32 KB
- 4-way

Block size:

$$\lfloor 8 \times 4=32B=2^5 \rfloor$$

Lines:

$$\lfloor 2^{15}/2^5=2^{10} \rfloor$$

Sets:

$$\lfloor 2^{10}/4=2^8 \rfloor$$

Offset:

$\lceil \frac{5}{8} \rceil$

Set:

$\lceil \frac{8}{8} \rceil$

Tag:

$\lceil \frac{32-8-5}{8} \rceil$

$\lceil \frac{19}{8} \rceil$

12.55 Word/Byte Offset Split

Same example:

- 8 words/block !' 3 word-offset bits
- 4 bytes/word !' 2 byte-offset bits

Address can be shown:

TEXT

Tag 19 | Set 8 | Word Offset 3 | Byte Offset 2

Total:

$\lceil 19+8+3+2=32 \rceil$

12.56 Conflict Misses in Set-Associative Cache

Set associative reduces conflict misses but does not necessarily eliminate them.

Example:

2-way cache, 4 sets.

Blocks:

TEXT

0, 4, 8

all map to:

$\lceil \text{Set0} \rceil$

Set 0 holds only 2 lines.

When all three are active, one must be evicted.

Thus conflict can still occur.

12.57 Higher Associativity Reduces Conflict

Same blocks:

TEXT

0, 4, 8

With 4-way Set 0:

all three can coexist.

Thus associativity can reduce misses.

12.58 But Higher Associativity Has Costs

Increasing associativity usually means:

- more tag comparisons,
- more muxing,
- more replacement metadata,
- potentially longer hit time,
- more complex hardware.

So:

$\boxed{\text{Higher associativity is not free}}$

12.59 Direct vs 2-Way Example

Suppose cache has 4 total lines.

Reference blocks:

TEXT

0, 4, 0, 4

Direct

4 sets $\times$ 1 way.

Both:

$0 \bmod 4 = 0$

$4 \bmod 4 = 0$

They conflict constantly:

TEXT

M M M M

2-way

2 sets $\times$ 2 ways.

Both:

$0 \bmod 2 = 0$

$4 \bmod 2 = 0$

But Set 0 has 2 ways.

Sequence:

TEXT

0	!	'	M
4	!	'	M
0	!	'	H
4	!	'	H

Thus:

$\boxed{M, M, H, H}$

12.60 Set-Associative Hit Condition

Suppose k ways.

Hit occurs if:

$\exists \text{ way } j: \text{Valid}_j = 1 \text{ \& } \text{Tag}_j = \text{RequestedTag}$

within selected set.

12.61 GATE Numerical — Mapping

Cache:

- 16 sets
- 4-way

Block number:

123

Set:

$123 \bmod 16$

Since:

$16 \times 7 = 112$

remainder:

11

$\boxed{\text{Set}=11}$

Tag:

$\lfloor 123 / 16 \rfloor$

$\boxed{7}$

12.62 Binary Shortcut

If number of sets:

$16 = 2^4$

then lowest 4 bits of block number = set index.

Remaining upper block bits = tag.

No modulo division required.

12.63 Hexadecimal Address Example

Suppose:

- 16-bit byte addresses
- block = 16 B !' 4 offset bits
- 2-way cache with 16 total lines

Sets:

$\lfloor 16/2=8 \rfloor$

Set bits:

$\lfloor 3 \rfloor$

Tag bits:

$\lfloor 16-3-4=9 \rfloor$

Field boundaries are not all on nibble boundaries.

So convert address hex to binary before slicing.

12.64 GATE Strategy for Hex Cache Questions

If field sizes are multiples of 4:

split hex directly.

If not:

1. convert each hex digit to 4 bits,
2. group bits from right:
 - offset,
 - set,
 - tag.

This avoids decimal conversion.

12.65 Direct vs Set vs Fully Address Fields

Direct

$\lfloor \boxed{\text{Tag}|\text{Line}|\text{Offset}} \rfloor$

k-Way Set Associative

$\lfloor \boxed{\text{Tag}|\text{Set}|\text{Offset}} \rfloor$

Fully Associative

$\boxed{\text{Tag}|\text{Offset}}$

12.66 Complete Comparison

Feature	Direct	Set Associative	Fully Associative
Placement	One line	One set, any way	Any line
Ways	1	k	All lines
Sets	Lines	Lines/k	1
Index/set bits	Most	Fewer	0
Tag bits	Smallest	Larger	Largest
Comparisons	1	k	Many/all
Replacement	No choice	Within set	Across cache
Conflict misses	Highest	Lower	Lowest
Hardware cost	Lowest	Medium	Highest

12.67 Relationship to Replacement Algorithms

Direct

Replacement policy unnecessary.

Set associative

Replacement policy needed **inside selected set**.

Fully associative

Replacement policy operates across all lines.

12.68 GATE Trap — k-Way Means k Sets

'LWrong.

k-way means:

$\boxed{k \text{ lines per set}}$

12.69 GATE Trap — Set Formula Uses Total Lines

Correct:

$\text{Sets} = \frac{\text{Lines}}{\text{Ways}}$

Not:

$\lfloor \text{CacheSize/Ways} \rfloor$

unless you've handled block size correctly.

12.70 GATE Trap — Mapping Uses Number of Sets

For set associative:

$\lfloor \boxed{\text{Set} = \text{Block} \bmod \text{Sets}} \rfloor$

Not:

$\lfloor \text{Block} \bmod \text{Lines} \rfloor$

The latter is for direct mapping.

12.71 GATE Trap — Replacement Across Entire Cache

'L Wrong.

If block maps to Set 3, victim must come from Set 3.

12.72 GATE Trap — Higher Associativity Eliminates All Misses

'L Wrong.

Compulsory and capacity misses remain.

Conflict misses may also remain until fully associative placement.

12.73 GATE Trap — Fully Associative Is Very High-Way Cache

Conceptually yes:

$\lfloor \text{Ways} = \text{NumberOfLines} \rfloor$

and:

$\lfloor \text{Sets} = 1 \rfloor$

This is a useful unification.

12.74 GATE Trap — Tag Bits Decrease with Associativity

Actually, for fixed cache size/block/address width:

$\lfloor \boxed{\text{Tag bits increase as associativity increases}} \rfloor$

because set bits decrease.

12.75 GATE Trap — Offset Changes with Associativity

For fixed block size and addressability:

$\boxed{\text{Offset bits stay unchanged}}$

Associativity only changes set/tag split.

12.76 FAST GATE SOLVING METHOD

Given:

- address bits = A
- cache size = C bytes
- block size = B bytes
- k-way associative

Write:

Step 1

$\boxed{\text{Lines} = C/B}$

Step 2

$\boxed{\text{Sets} = \text{Lines}/k}$

Step 3

$\boxed{\text{Offset} = \log_2 B}$

Step 4

$\boxed{\text{SetBits} = \log_2 \text{Sets}}$

Step 5

$\boxed{\text{Tag} = A - \text{SetBits} - \text{Offset}}$

Done.

12.77 Faster Power-of-2 Method

Suppose:

$\boxed{\text{Cache} = 2^C}$

$\boxed{\text{Block} = 2^B}$

$\boxed{\text{Ways} = 2^W}$

Then:

$\boxed{\text{Lines} = 2^{C-B}}$

$\boxed{\text{Sets} = 2^{C-B-W}}$

$\boxed{\text{Offset} = B}$

$\boxed{\text{SetBits} = C - B - W}$

$\boxed{\text{Tag} = A - C + W}$

~~Ø = 0~~ Extremely useful.

12.78 Worked Speed Example

Given:

- 40-bit byte addresses
- Cache = 1 MB = (2^{20}) B
- Block = 64 B = (2^6)
- 8-way = (2^3)

Then:

Lines:

$$\lfloor 2^{20-6} \rfloor = 2^{14}$$

Sets:

$$\lfloor 2^{14-3} \rfloor = 2^{11}$$

Offset:

$$\lfloor 6 \rfloor$$

Set bits:

$$\lfloor 11 \rfloor$$

Tag:

$$\lfloor 40-20+3 \rfloor$$

$$\lfloor \boxed{23} \rfloor$$

Check:

$$\lfloor 23+11+6=40 \rfloor$$

Correct.

12.79 Reference-String Exam Method

For every block:

1. Compute set:

$$\lfloor \text{Block} \bmod \text{Sets} \rfloor$$

2. Look only inside that set.

3. If block exists !' hit.

4. If free way !' miss and fill.

5. If full !' apply replacement policy within set.

6. Update LRU/FIFO state correctly.

This method is reliable.

12.80 LRU Sequence — Full Example

2-way cache.

2 sets.

Sequence:

TEXT

0, 2, 4, 0, 2, 4

All map to Set 0.

0

Miss !' [0, -]

2

Miss !' [0, 2]

LRU = 0.

4

Miss !' replace 0.

Set becomes [4, 2].

LRU = 2.

0

Miss !' replace 2.

Set [4, 0].

2

Miss !' replace 4.

Set [2, 0].

4

Miss !' replace 0.

All accesses miss.

Why?

Three active blocks compete for only two ways.

12.81 Important Insight

Associativity helps only if number of simultaneously competing blocks does not exceed the ways available in that set.

12.82 Cache Set Storage

For k-way cache:

Each set stores:

TEXT

Way 0: Valid | Tag | Data

Way 1: Valid | Tag | Data

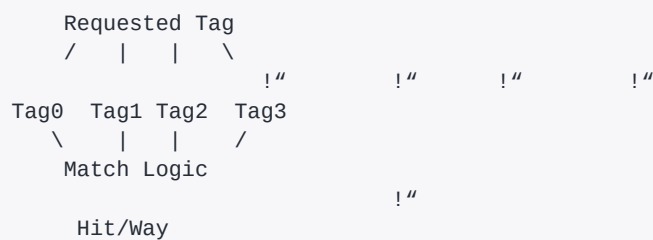
...
Way k-1

Replacement state is associated with the set.

12.83 Set Comparator Diagram

For 4-way:

TEXT



Set index already selected this group.

12.84 Which Way Supplies Data?

If one tag matches, the corresponding way is selected.

A multiplexer chooses that way's data.

Thus higher associativity adds:

- more tag comparisons,
- larger data-selection mux.

12.85 Cache Hit Time Trade-off

Direct cache:

TEXT

1 candidate

4-way:

TEXT

4 candidates

Fully associative:

TEXT

all candidates

Higher associativity can therefore increase hit-path complexity.

12.86 Associativity and Miss Rate Trade-off

General tendency:

$\lceil \text{Associativity} \rceil \uparrow$

$\lceil \text{ConflictMisses} \rceil \downarrow$

but:

$\lceil \text{HardwareComplexity} \rceil \uparrow$

Hence architecture balances both.

12.87 GATE Numerical — Metadata

Given:

- 32-bit byte address
- cache = 16 KB
- block = 32 B
- 4-way
- valid bit = 1 per line

We found:

Lines:

$\lceil 512 \rceil$

Sets:

$\lceil 128 \rceil$

Offset:

$\lceil 5 \rceil$

Set bits:

$\lceil 7 \rceil$

Tag:

$\lceil 20 \rceil$

Metadata:

$\lceil 512(20+1) \rceil$

$\lceil =512 \times 21 \rceil$

$\lceil \boxed{10752} \text{ bits} \rceil$

excluding replacement information.

12.88 Add Dirty Bits

If dirty bit = 1 per line:

$\lceil 512(20+1+1) \rceil$

$$\lceil 512 \times 22 \rceil$$

$$\lceil \boxed{11264 \text{ bits}} \rceil$$

12.89 Add 2-Way LRU Metadata Example

Suppose:

- 128 sets
- 2 ways
- tag+valid+dirty storage = X
- one LRU bit per set

Then add:

$$\lceil \boxed{128 \text{ bits}} \rceil$$

not one per line, under this 2-way LRU representation.

12.90 GATE Numerical — Main Memory Mapping Count

Suppose:

- 30-bit byte address
- block = 64 B
- cache has 128 sets.

Memory blocks:

$$\lceil 2^{30-6} = 2^{24} \rceil$$

Blocks mapping per set:

$$\lceil 2^{24} / 2^7 \rceil$$

$$\lceil = 2^{17} \rceil$$

Thus tag:

$$\lceil \boxed{17 \text{ bits}} \rceil$$

12.91 NAT Unit Trap

If answer asks:

number of sets

give an integer like:

$$\lceil 256 \rceil$$

If asks:

number of set-index bits

give:

$\lfloor \frac{8}{1} \rfloor$

Do not interchange these.

12.92 MCQ Trap

Statement:

A 4-way set-associative cache requires four set-index bits.

'L Not necessarily.

"4-way" tells number of lines per set.

Set-index bits depend on total number of sets.

12.93 MSQ Pattern

Possible true statements about increasing associativity:

- conflict misses generally decrease '
- number of sets decreases '
- tag bits increase '
- offset bits necessarily increase '
- hardware complexity generally increases '

This is a typical conceptual MSQ pattern.

CHAPTER 12 — FORMULA SHEET

Cache lines

$$\lfloor \boxed{L = \frac{\text{CacheSize}}{\text{BlockSize}}} \rfloor$$

Sets

$$\lfloor \boxed{S = \frac{L}{k}} \rfloor$$

Set mapping

$$\lfloor \boxed{\text{Set} = \text{Block} \bmod S} \rfloor$$

Offset

$$\lfloor \boxed{\text{OffsetBits} = \log_2(\text{BlockSize})} \rfloor$$

for byte-addressable memory and byte block size.

Set bits

$$\lfloor \boxed{\text{SetBits} = \log_2 S} \rfloor$$

Tag

$$\boxed{\text{TagBits} = \text{A} - \text{SetBits} - \text{OffsetBits}}$$

Block from tag/set

$$\boxed{\text{Block} = \text{Tag} \times \text{S} + \text{Set}}$$

CHAPTER 12 — POWER-OF-2 SHORTCUT

If:

$$\boxed{\text{Cache} = 2^C, \quad \text{Block} = 2^B, \quad k = 2^W}$$

then:

$$\boxed{\text{Lines} = 2^{C-B}}$$

$$\boxed{\text{Sets} = 2^{C-B-W}}$$

$$\boxed{\text{SetBits} = C - B - W}$$

$$\boxed{\text{Offset} = B}$$

$$\boxed{\text{Tag} = A - C + W}$$

MEMORY TRICKS

Set associative

One Set, Many Ways

Address

T-S-O

Tag | Set | Offset

k-way

k = lines per set

Mapping

Block MOD Sets

Replacement

Replace only inside the selected set

CHAPTER SNAPSHOT

1. Set associative maps each block to one set.

2. A block may use any way inside its set.
3. k-way means k lines per set.
4. Number of sets = lines / k.
5. Address format is Tag–Set–Offset.
6. Mapping formula is block mod number of sets.
7. Tag = block divided by number of sets, conceptually.
8. Replacement occurs only inside selected set.
9. LRU/FIFO state is maintained per set.
10. Direct mapping is 1-way set associative.
11. Fully associative is one set containing all lines.
12. Increasing associativity reduces number of sets.
13. Increasing associativity increases tag width.
14. Offset depends on block size, not associativity.
15. Higher associativity generally reduces conflict misses.
16. Higher associativity increases lookup complexity.

MUST REMEMBER

$$\boxed{\text{Sets} = \frac{\text{Lines}}{\text{Ways}}}$$

$$\boxed{\text{Set} = \text{Block} \bmod \text{Sets}}$$

$$\boxed{\text{T-S-O}}$$

and:

4-way means 4 lines in each set — NOT 4 sets.

PRACTICE SET

Q1 — L0

A cache has 128 lines and is 4-way set associative.

How many sets?

Q2 — L1

A cache has 512 lines and is 8-way.

Find:

1. sets,
2. set-index bits.

Q3 — L1

A 32-bit byte-addressed system has:

- cache = 32 KB,
- block = 64 B,
- 2-way associativity.

Find:

- lines,
 - sets,
 - tag bits,
 - set bits,
 - offset bits.
-

Q4 — L2

Same system as Q3, but 8-way.

Find T-S-O widths.

Q5 — L2

A 4-way cache has 16 sets.

Block number = 123.

Find:

- set,
 - tag.
-

Q6 — L2

A cache has:

- 8 sets,
- block size = 16 B.

Given:

- Tag = 4
- Set = 3
- Offset = 5

Find decimal byte address.

Q7 — L2

A 2-way cache has 2 sets and uses LRU.

Reference sequence:

TEXT

0, 2, 0, 4, 2

Find hits and misses.

Q8 — L2

A cache has 64 lines.

Give:

- number of sets in direct mapping,
 - number of sets in 2-way,
 - number of sets in 4-way,
 - number of sets in fully associative mapping.
-

Q9 — L2 MSQ

Increasing associativity while keeping cache size/block size fixed generally:

A. decreases number of sets B. increases tag bits C. changes block-offset bits D. may reduce conflict misses

Q10 — L3

A 40-bit byte-addressed processor has:

- 1 MB cache,
- 128-byte blocks,
- 8-way associativity.

Find:

1. lines,
 2. sets,
 3. offset bits,
 4. set-index bits,
 5. tag bits.
-

Q11 — L3

A 4-way cache has:

- 256 sets,
- 64-byte blocks,
- 36-bit byte addresses,
- valid = 1 bit/line,
- dirty = 1 bit/line.

Find:

1. total cache lines,
-

2. cache data capacity,
3. tag bits,
4. total tag+valid+dirty metadata bits.

Q12 — L3

A 2-way cache has 4 sets and initially empty lines. It uses LRU.

Reference sequence:

TEXT

0, 4, 1, 5, 0, 8, 4, 1, 9

Find:

- hits,
- misses,
- final contents of every set.

CHAPTER 12 COMPLETION STATUS

' k-way associativity ' Number of lines ' Number of sets ' Tag-Set-Offset ' Block-to-set mapping ' Tag derivation ' Reverse block/address calculation ' 2-way / 4-way / 8-way cache ' LRU within sets ' FIFO within sets ' Reference-string simulation ' Direct/Set/Fully unified view ' Conflict behavior ' Metadata storage ' Byte vs word addressing ' Power-of-2 shortcuts ' GATE traps

CHAPTER 12: COMPLETE '

COURSE PROGRESS

Completed: 13 / 26 modules

Approximate roadmap progress:

$\boxed{\sim 50\%}$

You have now completed the **three major cache mapping techniques**:

' Direct Mapping ' Fully Associative Mapping ' Set-Associative Mapping

NEXT !' CHAPTER 13 — CACHE REPLACEMENT + WRITE POLICIES

Next we cover:

- LRU,
- FIFO,
- Random,
- replacement-policy traps,
- **write-through**,
- **write-back**,
- dirty bit,

- write allocate,
- no-write allocate,
- write miss behavior,
- write-buffer idea,
- complete comparison tables,
- GATE-style hit/miss/write-policy questions.

USER

next

ASSISTANT

CHAPTER 13 — CACHE REPLACEMENT + WRITE POLICIES

Classification: [GATE CORE] **Priority:** +HIGH PRIORITY **Official syllabus connection:** Cache memory mapping + cache behavior

You already know **where** a block can be placed. Now we answer two new questions:

1. If the allowed cache location/set is full, **which block should be removed?**
2. When CPU writes data, **when should main memory be updated?**

PART A — CACHE REPLACEMENT POLICIES

13.1 Why Do We Need a Replacement Policy?

Suppose a 2-way set contains:

TEXT

```
W a y    0    !'    B l o c k    A
W a y    1    !'    B l o c k    B
```

A new block **C** maps to the same set.

The set is full.

One existing block must leave.

The rule used to select it is called:

$\boxed{\text{Replacement Policy}}$

Common policies:

- LRU
- FIFO
- Random

13.2 Where Is Replacement Needed?

Direct-Mapped Cache

Each block has exactly one destination line.

Therefore:

$\boxed{\text{No victim-selection policy is needed}}$

The mapped line is automatically replaced.

Set-Associative Cache

Victim chosen only among lines in the selected set.

Fully Associative Cache

Victim may be chosen from any cache line.

13.3 LRU — Least Recently Used

LRU removes the block that has not been accessed for the longest time.

Memory trick:

LRU = Longest time since Used

13.4 LRU Example

3-line fully associative cache:

TEXT

Reference: A, B, C, A, D

After:

TEXT

A B C

recency:

TEXT

M o s t r e c e n t ! ' C , B , A ! ' l e a s t r e c e n t

Access **A**:

TEXT

M o s t r e c e n t ! ' A , C , B ! ' l e a s t r e c e n t

Now **D** arrives.

Cache full.

LRU victim:

$\boxed{B}$

because B has gone unused for the longest time.

13.5 LRU Important Rule

On **every hit**, recency changes.

Example:

TEXT

Cache: A B C

Access: A

A becomes most recently used.

This is extremely important in reference-string questions.

13.6 FIFO — First In First Out

FIFO removes the block that entered the cache/set earliest.

Memory trick:

First entered !' first removed

FIFO does **not** care how recently a block was used.

13.7 FIFO Example

Cache receives:

TEXT

A, B, C

Insertion order:

TEXT

o l d e s t ! ' A , B , C ! ' n e w e s t

Now access **A** again.

This is a hit.

But FIFO order remains:

TEXT

A, B, C

A is still the oldest inserted block.

When D arrives:

$\boxed{A \text{ text is replaced}}$

13.8 LRU vs FIFO

Property	LRU	FIFO
Tracks	Recent usage	Insertion order
Hit changes state?	Yes	Usually no
Victim	Least recently accessed	Oldest inserted
Hardware complexity	Higher	Simpler

13.9 Random Replacement

Choose any candidate line randomly.

Advantages:

- simple,
- low tracking overhead.

Disadvantage:

- may remove a useful block.

For GATE, usually know the basic idea.

13.10 Replacement Happens Only on a Miss When Space Is Needed

Suppose a 4-way set contains:

TEXT

A, B, -, -

New block C maps here.

No victim is needed.

Use an empty way.

Replacement policy matters only when:

$\boxed{\text{selected set is full}}$

13.11 Set-Associative LRU Example

2-way cache with 2 sets.

Mapping:

$\lfloor \text{Set} = \text{Block} \bmod 2 \rfloor$

Reference:

TEXT

0, 2, 0, 4

All map to Set 0.

0

Miss !' [0, -]

2

Miss !' [0, 2]

0

Hit.

Now block 2 is LRU.

4

Miss.

Replace 2.

Final:

TEXT

Set 0 = [0, 4]

Hits:

$\lfloor 1 \rfloor$

Misses:

$\lfloor 3 \rfloor$

13.12 FIFO on Same Sequence

Sequence:

TEXT

0, 2, 0, 4

After loading:

TEXT

0, 2

FIFO oldest = 0.

Hit on 0 does not change insertion order.

Then 4 arrives.

FIFO replaces:

$\boxed{0}$

Thus LRU and FIFO behave differently.

13.13 Replacement Metadata

Replacement policies require extra state.

Examples:

2-way LRU

Often one bit per set is sufficient to identify which way is least recent.

FIFO

May require pointer/counter information.

Higher-way exact LRU

Requires more state.

Important:

Do not assume a particular bit count unless the question specifies the implementation or encoding.

PART B — CACHE WRITE POLICIES

13.14 Why Are Writes More Complicated?

For a read:

CPU just needs the correct data.

For a write:

TEXT

C P U c h a n g e s X f r o m 1 0 ! ' 2 0

Now potentially:

TEXT

Cache copy = 20

Main memory copy = ?

We must keep the memory hierarchy consistent.

This leads to two major write policies:

$\boxed{\text{Write-Through}}$

and

`\[ \boxed{\text{Write-Back}} \]`

13.15 Write-Through

In **Write-Through**, every cache write also updates the next memory level immediately.

Conceptually:

```
TEXT
CPU writes
    !"
Cache updated
    !"
Main Memory updated
```

So:

`\[ \boxed{\text{Cache and memory are kept synchronized immediately}} \]`

13.16 Write-Through Example

Initially:

```
TEXT
Cache X = 10
Memory X = 10
```

CPU writes:

```
TEXT
X = 25
```

Write-through:

```
TEXT
Cache X = 25
Memory X = 25
```

immediately.

13.17 Advantages of Write-Through

- Simpler consistency
 - Main memory usually stays up to date
 - Dirty bit generally not required for this purpose
-

13.18 Disadvantages of Write-Through

Every write can generate lower-level memory traffic.

If CPU writes frequently:

$\boxed{\text{Memory bandwidth demand increases}}$

Main memory is slower than cache, so direct waiting would hurt performance.

13.19 Write Buffer

A **write buffer** helps write-through caches.

Instead of forcing CPU to wait for every memory update:

TEXT

```
C P U  !'   C a c h e
        !"
Write Buffer
        !"
Main Memory
```

CPU can often continue while buffer handles memory write.

[SUPPORT]

13.20 Write-Back

In **Write-Back**, CPU updates the cache first.

Main memory is **not immediately updated**.

Example:

TEXT

```
Cache X = 25
Memory X = 10
```

This temporary difference is allowed.

Main memory is updated later when the modified block is evicted or otherwise written back.

13.21 Dirty Bit

How does cache know whether a block differs from memory?

Use a:

$\boxed{\text{Dirty Bit}}$

Typically:

TEXT

```
Dirty = 0 !'   cache block not modified relative to lower
Dirty = 1 !'   cache block modified
```

13.22 Write-Back Example

Initially:

TEXT

```
Cache X = 10  
Memory X = 10  
Dirty = 0
```

CPU writes X = 25.

After write:

TEXT

```
Cache X = 25  
Memory X = 10  
Dirty = 1
```

Later block is evicted.

Because:

\[Dirty=1 \]

cache writes block back to memory.

Then memory becomes:

TEXT

```
Memory X = 25
```

13.23 Clean vs Dirty Block

Clean block

\[Dirty=0 \]

It matches lower memory.

Can usually be discarded without writing it back.

Dirty block

\[Dirty=1 \]

It contains modified data.

Must generally be written to lower memory before replacement.

13.24 Write-Through vs Write-Back

Feature	Write-Through	Write-Back
Cache updated on CPU write	Yes	Yes
Lower memory updated immediately	Yes	No

Feature	Write-Through	Write-Back
Dirty bit required	Usually no	Yes
Memory traffic	Higher	Lower
Replacement of modified block	No special write-back needed	Dirty block must be written back
Complexity	Lower	Higher
Main memory freshness	More immediate	May be stale temporarily

Memory Trick

Through = goes through to memory now

Back = goes back later

PART C — WRITE MISS POLICIES

13.25 What Is a Write Miss?

CPU wants to write to address X, but X's block is not in cache.

This is:

$\boxed{\text{Write Miss}}$

Now there are two major choices:

1. Write Allocate
2. No-Write Allocate

13.26 Write Allocate

Also called:

$\boxed{\text{Fetch on Write}}$

On a write miss:

1. Fetch the block into cache.
2. Perform the write in cache.

Conceptually:

```
TEXT
Write miss
    !
Bring block to cache
    !
```

13.27 Why Write Allocate?

Reasoning:

If CPU writes this block now, it may access it again soon.

This exploits temporal locality.

13.28 No-Write Allocate

Also called:

$\boxed{\text{Write Around}}$

On a write miss:

Do not bring block into cache.

Instead write directly to lower memory.

Conceptually:

TEXT

Write miss

!"

Write lower memory

!"

Cache remains without block

13.29 Common Pairings

Typical design tendency:

Write-Back

often paired with:

$\boxed{\text{Write Allocate}}$

Write-Through

often paired with:

$\boxed{\text{No-Write Allocate}}$

Why?

Write-back benefits from keeping future writes in cache.

Write-through can simply send a miss write directly downward.

But:

These are common pairings, not absolute laws.

GATE questions may specify any combination.

13.30 Four Possible Combinations

Conceptually possible:

1. Write-through + write-allocate
2. Write-through + no-write-allocate
3. Write-back + write-allocate
4. Write-back + no-write-allocate

Do not assume a pairing unless stated or clearly implied.

13.31 Read Miss vs Write Miss

Read miss

Usually:

fetch block into cache, then provide requested data.

Write miss

Behavior depends on:

- write allocate,
- no-write allocate.

This distinction is very important.

13.32 Write Hit Behavior

Suppose block is already in cache.

Write-Through Hit

Update:

TEXT

Cache + Main Memory

Write-Back Hit

Update:

TEXT

Cache only
Dirty = 1

13.33 Write Miss Behavior — Write Allocate

Suppose write misses.

Then:

1. fetch block from lower level,
2. place it in cache,
3. update requested data.

If write-back:

```
TEXT
Dirty = 1
```

after modification.

13.34 Write Miss Behavior — No-Write Allocate

On miss:

```
TEXT
Write lower memory
Do not place block into cache
```

So a later access may still miss.

13.35 Example — Write Allocate

Cache initially lacks block X.

CPU executes:

```
TEXT
WRITE X = 100
```

With write allocate:

```
TEXT
Fetch X's block
!"
Install in cache
!"
Modify X to 100
```

Future access to X may hit.

13.36 Example — No-Write Allocate

Same situation.

CPU writes $X = 100$.

No-write allocate:

TEXT

Memory gets 100
Cache still has no X

Later read X :

likely cache miss.

13.37 Write-Back Replacement Penalty

Suppose incoming block causes eviction.

Two cases:

Victim clean

No write-back required.

Victim dirty

Must first write dirty block to lower memory.

Then fetch new block.

Therefore dirty replacement may have larger miss penalty.

13.38 Example

Main-memory block transfer time:

$\boxed{100\text{ns}}$

On miss with clean victim:

$\boxed{100\text{ns}}$

for block fetch, ignoring other overhead.

On miss with dirty victim:

1. write victim = 100 ns
2. fetch new block = 100 ns

Total:

$\boxed{200\text{ns}}$

under simple sequential assumptions.

13.39 Average Miss Penalty with Dirty Blocks

Suppose:

- normal block-fetch penalty = 80 ns
- dirty write-back penalty = 80 ns
- 30% of evicted blocks are dirty

Average penalty:

$$[80 + 0.30(80)]$$

$$[=80+24]$$

$$[\boxed{104\text{ns}}]$$

This type of numerical is very GATE-relevant.

13.40 General Formula

If:

- Fetch penalty = (F)
- Probability victim dirty = (d)
- Write-back cost = (W)

Then:

$$[\boxed{\text{AverageMissPenalty} = F + dW}]$$

assuming sequential independent costs.

13.41 Combine with AMAT

Suppose:

- hit time = 2 ns
- miss rate = 5%
- fetch penalty = 80 ns
- 25% victims dirty
- dirty write-back = 80 ns

Average miss penalty:

$$[P = 80 + 0.25(80)]$$

$$[=100\text{ns}]$$

Then:

$$[\text{AMAT} = 2 + 0.05(100)]$$

$$[\boxed{7\text{ns}}]$$

13.42 Memory Traffic in Write-Through

Suppose there are:

$$[1000]$$

CPU writes and every write-through update reaches memory.

Ignoring write combining/buffering:

$\boxed{\text{lower-level writes}}$

This is why write-through can create heavy traffic.

13.43 Memory Traffic in Write-Back

Suppose CPU writes same cached block 100 times.

Write-back may modify cache 100 times but eventually send only one block write when evicted.

Thus write-back can significantly reduce lower-level traffic.

13.44 But Write-Back Has Complexity

Need:

- dirty bit,
- eviction write-back,
- more consistency handling,
- potentially larger miss penalty on dirty replacement.

So again:

speed/bandwidth benefit comes with complexity.

13.45 Write-Through and Main Memory Consistency

Because every cache write propagates downward immediately, lower memory is generally more current.

This simplifies certain system behaviors.

But a write buffer may mean the lower write is pending briefly, depending on precise architecture.

For GATE conceptual questions, think:

$\boxed{\text{Write-through keeps lower memory updated promptly}}$

13.46 Valid Bit vs Dirty Bit

Do not confuse them.

Valid bit

Answers:

Is this cache-line content meaningful?

Dirty bit

Answers:

Has this valid cache block been modified since loading?

A line can be:

TEXT

```
Valid = 1
Dirty = 0
```

or:

TEXT

```
Valid = 1
Dirty = 1
```

If:

TEXT

```
Valid = 0
```

dirty state is typically irrelevant.

13.47 Tag vs Valid vs Dirty

Cache line:

TEXT

```
% % % % % % % %, % % % % % % %, % % % % % % %, % % % % % % % % % % % %
%   v a l i d   %   D i r t y   %   T a g   %   D a t a   B l o c k   %
% % % % % % % %4% % % % % % %4% % % % % % %4% % % % % % % % % % % % % % % % %
```

Valid

Can this line be used?

Tag

Which memory block is this?

Dirty

Has it changed?

13.48 Replacement Flow in Write-Back Cache

On a miss:

TEXT

```
Select victim
    !"
Dirty?
    !™    !~
No      Yes
```

```

! "      ! "
discard  write old block to memory
  \      /
      ! "
Fetch new block
      ! "
Install new block

```

This flow is worth memorizing.

13.49 Write-Through Flow

On write hit:

```

TEXT
CPU Write
  ! "
Cache update
  ! "
Lower-memory write

```

No dirty eviction is required later because lower memory was already updated.

13.50 Write-Back Flow

On write hit:

```

TEXT
CPU Write
  ! "
Cache update
  ! "
Dirty = 1

```

No immediate lower-memory write.

13.51 GATE Example — Policy Identification

Statement:

Main memory is updated only when a modified cache block is replaced.

Answer:

$\boxed{\text{Write-Back}}$

13.52 GATE Example — Dirty Bit

Which policy typically requires a dirty bit?

$\boxed{\text{Write-Back}}$

13.53 GATE Example — Write Miss

Statement:

On a write miss, block is loaded into cache before performing the write.

Answer:

$\boxed{\text{Write Allocate}}$

13.54 GATE Example — Write Around

Statement:

On a write miss, data is sent to lower memory without loading block into cache.

Answer:

$\boxed{\text{No-Write Allocate}}$

13.55 GATE Numerical — Average Dirty Penalty

Given:

- miss penalty to fetch block = 60 cycles
- dirty victim probability = 40%
- writing dirty victim = 60 cycles

Average miss penalty:

$60 + 0.4(60)$

$60 + 24$

$\boxed{84 \text{ cycles}}$

13.56 GATE Numerical — AMAT

Given:

- hit time = 1 cycle
- miss rate = 2%
- clean fetch = 50 cycles
- dirty victim probability = 20%
- write-back cost = 50 cycles

Average miss penalty:

$50 + 0.2(50) = 60$

AMAT:

$\lceil 1+0.02(60) \rceil$

$\lceil \boxed{2.2 \text{ cycles}} \rceil$

13.57 GATE Numerical — Metadata

Cache:

- 1024 lines
- tag = 18 bits
- valid = 1 bit
- dirty = 1 bit

Metadata/line:

$\lceil 20 \text{ bits} \rceil$

Total:

$\lceil 1024 \times 20 \rceil$

$\lceil \boxed{20480 \text{ bits}} \rceil$

If write-through, dirty bit may not be required, so storage could be smaller.

13.58 Replacement-Policy Reference String

Fully associative cache with 3 lines.

Sequence:

TEXT

1, 2, 3, 1, 4, 2

LRU

After 1,2,3:

TEXT

L R U ! ' 1

Access 1 updates recency.

Now 2 becomes LRU.

4 replaces 2.

Then 2 misses.

Hits:

$\lceil 1 \rceil$

Misses:

$\lceil 5 \rceil$

13.59 Same Sequence — FIFO

Insertion:

TEXT

1, 2, 3

Access 1 does not change FIFO order.

4 replaces 1.

Then access 2 hits.

So FIFO may produce different hit count.

You must simulate the exact policy.

13.60 Belady's Anomaly — LOW PRIORITY

[LOW PRIORITY]

FIFO in paging can exhibit **Belady's anomaly**, where increasing number of frames can sometimes increase page faults.

This is mainly an OS paging concept, not a major cache topic for this syllabus.

Do not spend preparation time here.

13.61 Optimal Replacement — SUPPORT

[LOW PRIORITY]

An ideal policy would evict the block whose next use is farthest in the future.

This is often called **Optimal/MIN**.

Useful as a theoretical benchmark.

Not generally implementable perfectly because future accesses are unknown.

13.62 LRU vs Optimal

LRU uses:

past behavior.

Optimal uses:

future knowledge.

So:

$\boxed{\text{LRU approximates locality using recent history}}$

13.63 Replacement Only Matters Among Eligible Lines

This is crucial.

Direct

Eligible lines = 1.

k-way

Eligible lines = k lines of selected set.

Fully associative

Eligible lines = all lines.

Replacement policy cannot choose outside those placement constraints.

13.64 Write Policy vs Replacement Policy

These solve different problems.

Replacement policy

Which block leaves?

Write policy

When/how does modified data reach lower memory?

Do not mix them.

13.65 Write Allocation vs Write Propagation

There are also two separate write decisions:

Allocation decision

On a write miss:

bring block into cache or not?

Propagation decision

On a write hit:

update lower memory immediately or later?

Thus:

TEXT

Write Allocate / No-Write Allocate

is separate from:

TEXT

Write-Through / Write-Back

Important distinction.

13.66 Complete Write Policy Matrix

Policy	Write Hit	Write Miss
Write-through + Write-allocate	Cache + memory updated; on miss bring block then write	Fetch block, then update cache/memory
Write-through + No-write-allocate	Cache + memory updated on hit	Write directly to memory, don't cache
Write-back + Write-allocate	Cache updated, dirty=1	Fetch block, modify cache, dirty=1
Write-back + No-write-allocate	Cache updated, dirty=1 on hit	Write lower memory directly, don't cache

Again, common pairings are not mandatory.

13.67 Write Buffer Stall Trap

With write-through + write buffer, CPU may avoid immediate stall if buffer has space.

But if buffer becomes full:

$\boxed{\text{CPU may need to stall}}$

Only calculate this if explicitly described.

13.68 Block Write-Back vs Word Write

Write-back typically writes an entire modified block to lower memory on eviction.

Write-through may propagate individual writes, depending on implementation.

Use exact problem wording if transfer costs are specified.

13.69 GATE Trap — Dirty Bit with Write-Through

Usually unnecessary for normal write-through semantics because lower memory is already updated.

So if a conceptual question asks which policy relies on dirty bits:

$\boxed{\text{Write-Back}}$

13.70 GATE Trap — Write Allocate Means Write-Back

'L Wrong.

Write allocation and write propagation are independent concepts.

13.71 GATE Trap — No-Write Allocate Means No Cache Write Ever

'L Wrong.

It affects **write misses**.

If the block is already present and there is a write hit, cache can still be updated according to write policy.

13.72 GATE Trap — Write-Back Updates Memory on Every Write

'L Wrong.

That is write-through behavior.

13.73 GATE Trap — Clean Victim Needs Write-Back

Normally:

$\boxed{\text{No}}$

If lower memory already has the same block, simply discard it.

13.74 GATE Trap — Dirty Victim Can Be Discarded

'L Wrong.

You would lose modifications.

It must generally be written to lower memory first.

13.75 GATE Trap — LRU and FIFO Hits

On an LRU hit:

$\boxed{\text{recency updates}}$

On a FIFO hit:

$\boxed{\text{insertion order generally stays}}$

13.76 GATE Trap — Replacement in Direct Mapping

No LRU/FIFO choice is needed.

The incoming block has exactly one destination.

13.77 High-Yield Comparison: Replacement

Policy	Decision Basis	Main Strength	Main Weakness
LRU	Time since last access	Uses temporal locality	More state/complexity
FIFO	Arrival time	Simple	Ignores recent reuse
Random	Random choice	Very simple tracking	Can evict useful block

13.78 High-Yield Comparison: Writes

Property	Write-Through	Write-Back
Lower memory updated	Every write	On eviction/later
Dirty bit	Usually unnecessary	Required
Lower-memory traffic	Higher	Lower
Complexity	Lower	Higher
Dirty eviction penalty	No	Yes

13.79 Fast GATE Recognition Rules

If you see:

"memory updated on every cache write"

think:

$\boxed{\text{Write-Through}}$

If you see:

"modified block written when replaced"

think:

$\boxed{\text{Write-Back}}$

If you see:

"fetch block on write miss"

think:

$\boxed{\text{Write Allocate}}$

If you see:

"write directly to memory on miss"

think:

$\boxed{\text{No-Write Allocate}}$

13.80 Numerical Strategy for Dirty Miss Penalty

If question gives:

- normal miss/fetch cost = F
- dirty probability = d
- dirty write-back cost = W

write:

$\boxed{P = F + dW}$

Then use:

$\boxed{AMAT = \text{HitTime} + \text{MissRate} \times P}$

unless timing assumptions differ.

13.81 Worked Example

Given:

- L1 hit time = 2 ns
- miss rate = 4%
- block fetch = 80 ns
- 25% evictions dirty
- write-back cost = 80 ns

Average miss penalty:

$\boxed{80 + 0.25(80)}$

$\boxed{= 100 \text{ ns}}$

AMAT:

$\boxed{2 + 0.04(100)}$

$\boxed{6 \text{ ns}}$

13.82 Multi-Level Write-Back Insight

[SUPPORT]

In multi-level caches, a dirty block may be written to the next lower cache level rather than immediately to DRAM.

For your GATE calculations:

follow the exact hierarchy and penalties stated.

13.83 Replacement and Cache Miss Types

Compulsory miss

Replacement cannot avoid first access.

Capacity miss

Better replacement may help somewhat, but cache size remains limiting.

Conflict miss

Higher associativity can reduce it.

Replacement policy affects which competing block survives.

CHAPTER 13 — FORMULA SHEET

Average dirty miss penalty

$$P = \text{FetchCost} + P(\text{Dirty}) \times \text{WriteBackCost}$$

AMAT

$$\text{AMAT} = \text{HitTime} + \text{MissRate} \times \text{MissPenalty}$$

Metadata

$$\text{Metadata} = \text{Lines} \times (\text{Tag} + \text{Valid} + \text{Dirty} + \dots) + \text{ReplacementState}$$

CHAPTER 13 — MEMORY TRICKS

LRU

Last used longest ago !' Leave

FIFO

First In !' First Out

Write-Through

Through to memory NOW

Write-Back

Back to memory LATER

Write Allocate

No-Write Allocate

CHAPTER SNAPSHOT

1. Replacement policy chooses the victim when eligible cache lines are full.
2. Direct-mapped cache does not need victim selection.
3. LRU removes the least recently accessed block.
4. FIFO removes the oldest inserted block.
5. LRU state changes on hits; FIFO insertion order generally does not.
6. Write-through updates lower memory on each write.
7. Write-back delays lower-memory update.
8. Write-back requires dirty-state tracking.
9. Dirty blocks must be written back before eviction.
10. Clean blocks can generally be discarded directly.
11. Write allocate loads block into cache on write miss.
12. No-write allocate bypasses cache on write miss.
13. Write allocation and write propagation are separate decisions.
14. Dirty victims can increase miss penalty.
15. Common tendency: write-back + write-allocate; write-through + no-write-allocate.
16. Replacement is always restricted by the mapping scheme.

MUST REMEMBER

$\boxed{\text{LRU} = \text{least recently accessed}}$

$\boxed{\text{FIFO} = \text{oldest inserted}}$

$\boxed{\text{Write-Through} = \text{memory now}}$

$\boxed{\text{Write-Back} = \text{memory later}}$

$\boxed{\text{Dirty bit belongs mainly to write-back}}$

and:

$\boxed{P_{\text{miss}} = \text{Fetch} + d \times \text{WriteBack}}$

when dirty-victim probability is given.

PRACTICE SET

Q1 — L0 MCQ

Which replacement policy removes the block that has not been used for the longest time?

A. FIFO B. LRU C. Random D. Write-back

Q2 — L0 MCQ

Which write policy normally updates lower memory on every cache write?

A. Write-back B. Write-through C. FIFO D. LRU

Q3 — L1 MCQ

Which policy typically requires a dirty bit?

A. Write-through B. Write-back C. No-write allocate only D. FIFO

Q4 — L1 MCQ

On a write miss, bringing the block into cache is called:

A. Write allocate B. Write-through C. FIFO D. Dirty replacement

Q5 — L1

A 3-line fully associative cache uses LRU.

Reference sequence:

TEXT

1, 2, 3, 1, 4

Which block is evicted when 4 arrives?

Q6 — L2

Repeat Q5 using FIFO.

Which block is evicted?

Q7 — L2 NAT

Block-fetch penalty = 100 cycles.

30% of victims are dirty.

Dirty write-back cost = 100 cycles.

Find average miss penalty.

Q8 — L2 NAT

Hit time = 1 cycle.

Miss rate = 5%.

Average miss penalty = 120 cycles.

Find AMAT in cycles.

Q9 — L2 MSQ

Which are generally true for write-back?

A. Main memory is updated on every hit B. Dirty bit is useful C. Lower-memory traffic may be reduced D. Dirty replacement can add miss penalty

Q10 — L2 MSQ

Which statements are correct?

A. Write allocate concerns behavior on a write miss. B. Write-through determines cache placement. C. No-write allocate may bypass cache on a write miss. D. Write-back and write allocate mean the same thing.

Q11 — L3

A 2-way set-associative cache has one set under consideration and uses LRU.

Reference sequence for that set:

TEXT

A, B, A, C, B

Find:

- hits,
 - misses,
 - victim on C,
 - final two blocks.
-

Q12 — L3

Cache:

- hit time = 2 ns
- miss rate = 3%
- block fetch = 90 ns
- dirty-victim probability = 40%
- dirty write-back cost = 90 ns

Find average miss penalty and AMAT.

Q13 — L3

A cache has:

- 2048 lines
- 19-bit tag
- 1 valid bit
- write-back policy with 1 dirty bit
- 2-way associativity
- 1 LRU bit per set

Find total metadata bits.

CHAPTER 13 COMPLETION STATUS

' Need for replacement ' Direct vs associative replacement behavior ' LRU ' FIFO ' Random ' Replacement metadata ' Write-through ' Write-back ' Dirty bit ' Clean vs dirty eviction ' Write buffer concept ' Write allocate ' No-write allocate ' Common write-policy pairings ' Dirty-victim miss penalty ' AMAT integration ' GATE traps and numericals

CHAPTER 13: COMPLETE '

COURSE PROGRESS

Completed: 14 / 26 modules

Approximate roadmap progress:

$\boxed{\sim 54\%}$

You have now covered almost the entire conceptual cache subsystem.

NEXT !' CHAPTER 14 — CACHE PERFORMANCE NUMERICALS

This will be the **heavy numerical consolidation chapter**:

- AMAT with cache mapping,
 - hit/miss reference strings,
 - direct/set/fully associative comparison,
 - LRU/FIFO numericals,
 - dirty write-back penalties,
 - multi-level caches,
 - tag overhead,
 - CPI + cache misses,
 - effective memory access time,
 - mixed GATE-style NAT/MCQ/MSQ traps.
-

USER

next

ASSISTANT

CHAPTER 14 — CACHE PERFORMANCE NUMERICALS

Classification: [GATE CORE] **Priority:** ~~0~~ = VERY HIGH **Goal:** Combine everything learned so far into the exact numerical patterns GATE likes to ask.

This chapter combines:

- Cache mapping
- Hit/miss sequences
- Direct / Set / Fully Associative
- LRU / FIFO
- Write-back penalties
- AMAT
- Multi-level cache
- CPI and stall cycles
- Tag/metadata overhead

14.1 The Master Cache Numerical Workflow

Whenever a cache problem looks complicated, solve it in this order:

TEXT

1. Determine addressability
2. Convert cache/block sizes to powers of 2
3. Find number of lines
4. Find number of sets
5. Find Tag / Index(Set) / Offset
6. Map each requested block/address
7. Determine hits and misses
8. Apply replacement/write policy
9. Calculate miss rate
10. Calculate AMAT/CPI/traffic if asked

This prevents most mistakes.

14.2 Important Formula Collection

Number of cache lines

$$\boxed{L = \frac{\text{CacheSize}}{\text{BlockSize}}}$$

Number of sets

For k -way associativity:

$$\boxed{S = \frac{L}{k}}$$

Direct mapping

$$\boxed{\text{Line} = \text{Block} \bmod L}$$

Set associative

$$\boxed{\text{Set} = \text{Block} \bmod S}$$

Byte-address offset

$$\boxed{\text{OffsetBits} = \log_2(\text{BlockSizeBytes})}$$

AMAT

$$\boxed{\text{AMAT} = \text{HitTime} + \text{MissRate} \times \text{MissPenalty}}$$

14.3 Numerical Type 1 — Address Decomposition

Given:

- 32-bit byte-address
- Cache = 64 KB
- Block = 64 B
- Direct mapped

Find Tag, Index, Offset.

Cache:

$$\boxed{64\text{KB} = 2^{16}\text{B}}$$

Block:

$$\boxed{64\text{B} = 2^6\text{B}}$$

Lines:

$$\boxed{2^{16-6} = 2^{10}}$$

Therefore:

$$\boxed{\text{Offset} = 6}$$

$$\boxed{\text{Index} = 10}$$

$$\boxed{\text{Tag} = 32 - 10 - 6}$$

$$\boxed{\text{Tag} = 16}$$

Final:

$$\boxed{16|10|6}$$

14.4 Same Cache — 4-Way

Lines:

$$2^{10}$$

4-way:

$$4 = 2^2$$

Sets:

$$2^{10-2} = 2^8$$

Thus:

$$\text{Set} = 8$$

$$\text{Offset} = 6$$

$$\text{Tag} = 32 - 8 - 6 = 18$$

Answer:

$$\boxed{18|8|6}$$

14.5 Same Cache — Fully Associative

Set/index bits:

$$0$$

Offset:

$$6$$

Tag:

$$32 - 6$$

$$\boxed{26}$$

Address:

$$\boxed{26|6}$$

14.6 Fast Comparison

For the same cache:

Mapping	Tag	Index/Set	Offset
Direct	16	10	6
2-way	17	9	6
4-way	18	8	6
8-way	19	7	6
Fully associative	26	0	6

Pattern:

14.7 Numerical Type 2 — Address to Block

Suppose:

- Byte address = 1000
- Block size = 64 B

Block number:

$$\lfloor \left\lfloor \frac{1000}{64} \right\rfloor \rfloor$$

$$\lfloor =15 \rfloor$$

Offset:

$$\lfloor 1000 - 15(64) \rfloor$$

$$\lfloor =40 \rfloor$$

Thus:

$$\lfloor \boxed{\text{Block}=15, \text{Offset}=40} \rfloor$$

14.8 Direct-Mapped Line

If cache has 8 lines:

$$\lfloor \text{Line} = 15 \bmod 8 \rfloor$$

$$\lfloor \boxed{7} \rfloor$$

14.9 Set-Associative Set

If same cache has:

- 8 lines
- 2-way

Sets:

$$\lfloor 8/2=4 \rfloor$$

Then:

$$\lfloor \text{Set} = 15 \bmod 4 \rfloor$$

$$\lfloor \boxed{3} \rfloor$$

14.10 Numerical Type 3 — Direct-Mapped Hit/Miss Sequence

Cache:

- 4 lines
- initially empty

- reference sequence is in **block numbers**

TEXT

0, 1, 4, 0, 1, 5, 4

Mapping:

$\lfloor \text{Line} = \text{Block} \bmod 4 \rfloor$

Let's simulate.

Access	Line	Before	Result	After
0	0	empty	M	0
1	1	empty	M	1
4	0	0	M	4
0	0	4	M	0
1	1	1	H	1
5	1	1	M	5
4	0	0	M	4

Hits:

$\lfloor \boxed{1} \rfloor$

Misses:

$\lfloor \boxed{6} \rfloor$

Hit rate:

$\lfloor \frac{1}{7} \rfloor$

$\lfloor \boxed{\approx 14.29\%} \rfloor$

14.11 Why So Many Misses?

Blocks:

TEXT

0 and 4

fight for line 0.

Blocks:

TEXT

1 and 5

fight for line 1.

These are:

$\boxed{\text{Conflict misses}}$

14.12 Numerical Type 4 — 2-Way LRU

Same total cache:

- 4 lines
- 2-way
- therefore 2 sets
- LRU
- sequence:

TEXT

0, 1, 4, 0, 1, 5, 4

Mapping:

$\text{Set} = \text{Block} \bmod 2$

Even blocks !' Set 0.

Odd blocks !' Set 1.

Access 0

Set 0:

TEXT

[0, -]

Miss.

Access 1

Set 1:

TEXT

[1, -]

Miss.

Access 4

Set 0:

TEXT

[0, 4]

Miss.

Access 0

0 exists.

Hit.

0 becomes most recent.

Access 1

1 exists.

Hit.

Access 5

Set 1 has free way:

TEXT

[1, 5]

Miss.

Access 4

Set 0 contains 0 and 4.

Hit.

Totals:

$\boxed{\text{Hits}=3}$

$\boxed{\text{Misses}=4}$

Compared with direct:

TEXT

Direct: 1 hit

2-way: 3 hits

Associativity reduced conflict misses.

14.13 Numerical Type 5 — Fully Associative LRU

Same 4-line cache.

Sequence:

TEXT

0, 1, 4, 0, 1, 5, 4

Initially empty.

First:

TEXT

0	!	'	M
1	!	'	M
4	!	'	M

Cache has room.

Then:

TEXT

0	!	'	H
1	!	'	H

Then:

TEXT

5	!	'	M
---	---	---	---

Now all four blocks fit:

TEXT

0, 1, 4, 5

Final:

TEXT

4	!	'	H
---	---	---	---

Totals:

$\boxed{\text{Hits}=3, \text{Misses}=4}$

Here 2-way and fully associative happen to produce same count.

This will not always happen.

14.14 Important Exam Lesson

Higher associativity does **not guarantee** a different answer for every reference string.

It only increases placement flexibility.

Actual benefit depends on access pattern.

14.15 Numerical Type 6 — LRU vs FIFO

3-line fully associative cache.

Sequence:

TEXT

1, 2, 3, 1, 4, 2

LRU

After:

TEXT

1, 2, 3

access 1 makes 1 recent.

So 2 becomes LRU.

When 4 arrives:

2\text{ replaced}

Then final access 2 misses.

Hits:

1

Misses:

5

FIFO

Insertion order:

TEXT

1, 2, 3

Access 1 does not change FIFO age.

When 4 arrives:

1\text{ replaced}

Then final 2 is still present.

So:

Hits=2

Misses=4

Thus replacement policy can change performance.

14.16 Numerical Type 7 — AMAT Basic

Given:

- Hit time = 2 ns
- Hit rate = 96%

- Miss penalty = 100 ns

Miss rate:

$$\sqrt[1]{1-0.96=0.04}$$

AMAT:

$$\sqrt[1]{2+0.04(100)}$$

$$\sqrt[1]{\boxed{6\text{ns}}}$$

14.17 Sequential Access Trap

Given:

- cache time = 5 ns
- memory access time = 80 ns
- hit rate = 90%
- cache checked before memory

Hit path:

$$\sqrt[1]{5\text{ns}}$$

Miss path:

$$\sqrt[1]{5+80=85\text{ns}}$$

Average:

$$\sqrt[1]{0.9(5)+0.1(85)}$$

$$\sqrt[1]{4.5+8.5}$$

$$\sqrt[1]{\boxed{13\text{ns}}}$$

Shortcut:

$$\sqrt[1]{5+0.1(80)=13\text{ns}}$$

14.18 Parallel Access Trap

Same values, but cache and memory start simultaneously.

Hit:

$$\sqrt[1]{5\text{ns}}$$

Miss:

$$\sqrt[1]{80\text{ns}}$$

Average:

$$\sqrt[1]{0.9(5)+0.1(80)}$$

$$\sqrt[1]{4.5+8}$$

$$\sqrt[1]{\boxed{12.5\text{ns}}}$$

Different assumption !' different answer.

14.19 Numerical Type 8 — Two-Level AMAT

Given:

- L1 = 1 ns
- L1 miss = 10%
- L2 = 6 ns
- L2 local miss = 20%
- Memory penalty after L2 = 100 ns

$$\boxed{AMAT = 1 + 0.1(6 + 0.2(100))}$$

$$\boxed{= 1 + 0.1(26)}$$

$$\boxed{= 1 + 2.6}$$

$$\boxed{3.6 \text{ ns}}$$

14.20 Probability-Tree Check

Paths:

L1 hit

Probability:

$$\boxed{0.9}$$

Time:

$$\boxed{1}$$

L2 hit

Probability:

$$\boxed{0.1(0.8) = 0.08}$$

Time:

$$\boxed{1 + 6 = 7}$$

Memory

Probability:

$$\boxed{0.1(0.2) = 0.02}$$

Time:

$$\boxed{1 + 6 + 100 = 107}$$

Average:

$$\boxed{0.9(1) + 0.08(7) + 0.02(107)}$$

$$\boxed{0.9 + 0.56 + 2.14}$$

$$\boxed{3.6 \text{ ns}}$$

14.21 Global Miss Rate

For previous example:

$$m_{\text{global}} = 0.10 \times 0.20$$

$$\boxed{0.02 = 2\%}$$

Only 2% of CPU memory references reach main memory.

14.22 Numerical Type 9 — Three-Level AMAT

Given:

- L1: 1 ns, miss 10%
- L2: 5 ns, miss 20%
- L3: 15 ns, miss 25%
- DRAM penalty = 120 ns

$$\text{AMAT} = 1 + 0.1 \left(5 + 0.2 \left(15 + 0.25(120) \right) \right)$$

First:

$$15 + 30 = 45$$

Then:

$$5 + 0.2(45) = 14$$

Then:

$$1 + 0.1(14) = 2.4$$

$$\boxed{2.4 \text{ ns}}$$

14.23 Numerical Type 10 — Dirty Write-Back Penalty

Given:

- Clean miss fetch = 80 ns
- 25% victim blocks dirty
- Dirty block write-back = 80 ns

Average miss penalty:

$$80 + 0.25(80)$$

$$\boxed{100 \text{ ns}}$$

14.24 Combine Dirty Penalty with AMAT

Hit time:

$$2 \text{ ns}$$

Miss rate:

$$4\%$$

Average penalty:

$$\boxed{100\text{ns}}$$

Then:

$$\text{AMAT} = 2 + 0.04(100)$$

$$\boxed{6\text{ns}}$$

14.25 Dirty Victim Probability Trap

Do not multiply dirty probability by miss rate twice.

Correct:

$$\text{AveragePenalty} = \text{Fetch} + dW$$

then:

$$\text{AMAT} = \text{HitTime} + m(\text{AveragePenalty})$$

Equivalent expanded form:

$$\text{HitTime} + mF + mdW$$

14.26 Numerical Type 11 — CPI with Cache Misses

Given:

- Base CPI = 1
- I-cache miss rate = 2%
- D-cache miss rate = 5%
- 30% instructions access data
- Miss penalty = 40 cycles

Instruction stalls:

$$0.02(40) = 0.8$$

Data stalls:

$$0.30(0.05)(40)$$

$$= 0.6$$

Total stall:

$$1.4$$

Actual CPI:

$$1 + 1.4$$

$$\boxed{2.4}$$

14.27 Why Instruction Miss Rate Is Not Multiplied by 30%

Every instruction itself needs an instruction fetch.

Thus:

$\text{1.25 cache accesses/instruction}$

But only 30% instructions in this example access data memory.

14.28 Unified Cache CPI Example

Suppose:

- Base CPI = 1
- 1.25 cache accesses/instruction
- miss rate = 3%
- miss penalty = 50 cycles

Stall cycles:

$1.25(0.03)(50)$

$= 1.875$

Actual CPI:

$1 + 1.875$

$\boxed{2.875}$

14.29 Numerical Type 12 — Convert ns to Cycles

CPU frequency:

2GHz

Clock period:

$\frac{1}{2\text{GHz}} = 0.5\text{ns}$

Memory miss:

50ns

Penalty in cycles:

$50/0.5$

$\boxed{100\text{ cycles}}$

14.30 Frequency Shortcut

For GHz:

$\boxed{\text{ClockPeriod(ns)} = \frac{1}{\text{Frequency(GHz)}}}$

Examples:

$1\text{GHz} \rightarrow 1\text{ns}$

$2\text{GHz} \rightarrow 0.5\text{ns}$

$4\text{GHz} \rightarrow 0.25\text{ns}$

14.31 Numerical Type 13 — CPI with ns Penalty

Given:

- Base CPI = 1
- CPU = 2.5 GHz
- cache accesses/instruction = 1.2
- miss rate = 2%
- miss penalty = 40 ns

Clock:

$$T = \frac{1}{2.5} = 0.4 \text{ ns}$$

Penalty cycles:

$$40 / 0.4 = 100$$

Stalls:

$$1.2(0.02)(100)$$

$$= 2.4$$

CPI:

$$1 + 2.4$$

$$\boxed{3.4}$$

14.32 Numerical Type 14 — Tag Storage

Given:

- 32-bit byte address
- cache = 64 KB
- block = 64 B
- direct
- valid bit = 1
- dirty bit = 1

We found:

$$\text{Tag} = 16$$

Lines:

$$1024$$

Metadata per line:

$$16 + 1 + 1 = 18$$

Total:

$$1024(18)$$

$$\boxed{18432 \text{ bits}}$$

14.33 Total Physical Cache Size

Data capacity:

$$\lfloor 64\text{KB}=65536\text{B} \rfloor$$

Data bits:

$$\lfloor 65536 \times 8 \rfloor$$

$$\lfloor =524288 \text{ bits} \rfloor$$

Add metadata:

$$\lfloor 524288+18432 \rfloor$$

$$\lfloor \boxed{542720 \text{ bits}} \rfloor$$

14.34 Metadata Overhead

$$\lfloor \frac{18432}{524288} \times 100 \rfloor$$

$$\lfloor \boxed{\approx 3.52\%} \rfloor$$

if overhead is defined relative to data bits.

14.35 Numerical Type 15 — Set-Associative Metadata

Given:

- 36-bit byte address
- cache = 256 KB
- block = 64 B
- 4-way
- valid = 1
- dirty = 1

Cache:

$$\lfloor 2^{18}\text{B} \rfloor$$

Block:

$$\lfloor 2^6\text{B} \rfloor$$

Lines:

$$\lfloor 2^{12}=4096 \rfloor$$

Sets:

$$\lfloor 4096/4=1024=2^{10} \rfloor$$

Tag:

$$\lfloor 36-10-6=20 \rfloor$$

Metadata per line:

$$\lfloor 20+1+1=22 \rfloor$$

Total:

$$\lfloor 4096(22) \rfloor$$

$$\lfloor \boxed{90112 \text{ bits}} \rfloor$$

excluding LRU metadata.

14.36 Add 4-Way Replacement Bits Carefully

If a question explicitly says:

2 bits per set are used for replacement state

then additional bits:

$$\lfloor 1024 \times 2 = 2048 \rfloor$$

Total:

$$\lfloor 90112 + 2048 \rfloor$$

$$\lfloor \boxed{92160 \text{ bits}} \rfloor$$

Do not invent the replacement-bit count.

14.37 Numerical Type 16 — Reverse Hit Rate

Given:

- hit time = 2 ns
- miss penalty = 100 ns
- AMAT = 7 ns

$$\lfloor 7 = 2 + m(100) \rfloor$$

$$\lfloor m = 0.05 \rfloor$$

So hit rate:

$$\lfloor h = 0.95 \rfloor$$

$$\lfloor \boxed{95\%} \rfloor$$

14.38 Reverse Miss Penalty

Given:

- Hit time = 1 ns
- miss rate = 4%
- AMAT = 5 ns

$$\lfloor 5 = 1 + 0.04P \rfloor$$

$$\lfloor 4 = 0.04P \rfloor$$

$$\boxed{P=100\text{ns}}$$

14.39 Reverse L2 Miss Rate

Given:

$$\boxed{AMAT=2\text{ns}}$$

L1:

$$\boxed{T_1=1\text{ns}, \text{quad } m_1=0.1}$$

L2:

$$\boxed{T_2=5\text{ns}}$$

Memory penalty:

$$\boxed{100\text{ns}}$$

Find m_2 .

$$\boxed{2=1+0.1(5+100m_2)}$$

$$\boxed{1=0.5+10m_2}$$

$$\boxed{0.5=10m_2}$$

$$\boxed{m_2=0.05}$$

$$\boxed{5\%}$$

14.40 Numerical Type 17 — Block Fill Time

Suppose:

- first word arrives after 60 ns
- block has 8 words
- each **additional** word requires 5 ns

Total:

$$\boxed{60+7(5)}$$

$$\boxed{95\text{ns}}$$

14.41 Different Wording

If question instead says:

- initial latency = 60 ns
- each of 8 words requires 5 ns transfer

Then:

$$\boxed{60+8(5)}$$

$$\boxed{100\text{ns}}$$

Read the wording carefully.

14.42 Numerical Type 18 — Memory Bandwidth

A 64-byte cache line transfers in 16 ns.

Bandwidth:

$$\left[\frac{64}{16} = 4 \text{ bytes/ns} \right]$$

$$\left[= 4 \times 10^9 \text{ bytes/s} \right]$$

$$\left[\boxed{4 \text{ GB/s}} \right]$$

using decimal rate units.

14.43 Numerical Type 19 — Hit/Miss from Byte Addresses

Direct cache:

- 4 lines
- block size = 16 B
- initially empty

Address sequence:

TEXT

0, 4, 16, 0, 64, 4

First convert addresses to blocks:

$$\left[\text{Block} = \left\lfloor \frac{\text{Address}}{16} \right\rfloor \right]$$

So:

TEXT

0	!	'	B	0
4	!	'	B	0
16	!	'	B	1
0	!	'	B	0
64	!	'	B	4
4	!	'	B	0

Sequence:

TEXT

0, 0, 1, 0, 4, 0

Now simulate.

0

M

0

H

1

M

0

H

4

Maps line 0, replaces block 0 !' M

0

M

Totals:

$\boxed{\text{Hits}=2, \text{Misses}=4}$

14.44 Critical Trap — Addresses Are Not Always Block Numbers

If reference string contains byte addresses, first convert to blocks.

If the question explicitly says:

block reference string

do not divide again.

14.45 Numerical Type 20 — Spatial Locality

Block size = 4 words.

Access sequence:

TEXT

0, 1, 2, 3

If these are word addresses and all belong to one block:

First access:

$\boxed{\text{Miss}}$

Remaining:

$\boxed{\text{Hit}, \text{Hit}, \text{Hit}}$

assuming block remains cached.

This shows spatial locality.

14.46 Larger Block Can Improve Hits

Sequential access:

TEXT

0, 1, 2, 3, 4, 5, 6, 7

If block = 1 word:

potentially many first-time misses.

If block = 4 words:

TEXT

0-3 loaded together

4-7 loaded together

fewer compulsory block fetches for sequential access.

14.47 But Larger Block Can Hurt

For fixed cache capacity:

$\lceil \text{BlockSize} \rceil$

means:

$\lfloor \text{NumberOfLines} \rfloor$

This can:

- reduce number of distinct blocks held,
- increase conflicts/capacity pressure,
- increase transfer penalty.

Thus optimal block size is a trade-off.

14.48 Numerical Type 21 — Direct Cache Conflict

Cache:

- 8 lines
- block sequence:

TEXT

1, 9, 17, 1

All:

$1 \bmod 8 = 1$

$9 \bmod 8 = 1$

$17 \bmod 8 = 1$

So:

TEXT

M, M, M, M

because each replaces previous line-1 block.

14.49 Same Sequence in 4-Way Set Associative

Suppose total cache = 8 lines, 4-way.

Sets:

$$\lfloor 8/4=2 \rfloor$$

All odd blocks map Set 1.

Set 1 has four ways.

Blocks:

TEXT

1, 9, 17

can coexist.

Sequence:

TEXT

1	!	'	M
9	!	'	M
1	7	!	' M
1	!	'	H

Thus:

$$\lfloor \boxed{3M, 1H} \rfloor$$

14.50 Important Associativity Insight

If n simultaneously active blocks map to the same set:

- associativity $(k < n)$!' replacement/conflict possible
- associativity $(k \geq n)$!' they can coexist

assuming no other blocks interfere.

14.51 Numerical Type 22 — Direct Tag from Address

Suppose:

- 16-bit byte address
- cache = 256 B
- block = 16 B

Offset:

$$\lfloor 4 \rfloor$$

Lines:

$$\lfloor 256/16=16 \rfloor$$

Index:

$\lfloor 4 \rfloor$

Tag:

$\lfloor 8 \rfloor$

Address:

```
TEXT
ABCD_H
```

Since fields align:

```
TEXT
AB | C | D
```

So:

$\lfloor \boxed{\text{Tag=AB_H, \text{Index=C_H, \text{Offset=D_H}}} \rfloor$

14.52 Hex Non-Aligned Example

Suppose:

- 16-bit address
- Offset = 5 bits
- Set = 3 bits

Then:

$\lfloor \text{Tag=8} \rfloor$

Address:

```
TEXT
B6D3_H
```

Convert binary:

```
TEXT
B   6   D   3
1011 0110 1101 0011
```

Split:

```
TEXT
Tag      | Set | Offset
10110110 | 110 | 10011
```

Thus:

Tag:

$\lfloor 10110110_2=\text{B6_H} \rfloor$

Set:

$\lfloor 110_2 = 6 \rfloor$

Offset:

$\lfloor 10011_2 = 19 \rfloor$

14.53 Numerical Type 23 — Cache Access Count

Suppose 1 million memory references.

Hit rate:

$\lfloor 95\% \rfloor$

Hits:

$\lfloor 0.95(10^6) = 950000 \rfloor$

Misses:

$\lfloor 50000 \rfloor$

If every miss causes one block fetch:

$\lfloor \boxed{50000} \text{ block fetches} \rfloor$

ignoring write-back traffic.

14.54 Add Dirty Eviction Traffic

Suppose:

- 50,000 misses
- 20% miss replacements evict dirty blocks.

Dirty writes:

$\lfloor 50000(0.2) \rfloor$

$\lfloor \boxed{10000} \rfloor$

So lower memory sees approximately:

- 50,000 block reads
- 10,000 block write-backs

under the simplified assumptions.

14.55 Write-Through Traffic Numerical

Suppose:

- 1,000,000 instructions
- 20% are stores
- write-through cache
- every store causes one lower-memory write

Lower-level writes:

$$\lfloor 10^6(0.2) \rfloor$$

$$\lfloor \boxed{200000} \rfloor$$

ignoring buffering/coalescing.

14.56 Write-Back Traffic Benefit

Suppose those 200,000 stores repeatedly modify only 20,000 cache blocks, each eventually written back once.

Approximate lower-memory block writes:

$$\lfloor 20000 \rfloor$$

rather than 200,000 individual writes.

This illustrates why write-back can reduce traffic.

14.57 Numerical Type 24 — Speedup

Without cache:

$$\lfloor 100\text{ns/access} \rfloor$$

With cache AMAT:

$$\lfloor 5\text{ns/access} \rfloor$$

Speedup:

$$\lfloor 100/5 \rfloor$$

$$\lfloor \boxed{20\text{times}} \rfloor$$

14.58 Percentage Time Reduction

Same case:

$$\lfloor \frac{100-5}{100} \times 100 \rfloor$$

$$\lfloor \boxed{95\%} \rfloor$$

Do not confuse:

- speedup $\lfloor (=20\text{times}) \rfloor$
 - time reduction $\lfloor (=95\%) \rfloor$
-

14.59 Numerical Type 25 — Required Hit Rate

Suppose:

- Cache time = 2 ns
- memory penalty = 98 ns
- target AMAT = 6.9 ns

$$\lfloor 6.9 = 2 + m(98) \rfloor$$

$$\lfloor 4.9 = 98m \rfloor$$

$$\lceil m=0.05 \rceil$$

Therefore:

$$\lceil h=0.95 \rceil$$

$$\lceil \boxed{95\%} \rceil$$

14.60 Common GATE Cache Traps

Trap 1: k -way means k sets.

Wrong.

$$k = \text{lines per set}$$

Trap 2: Offset always uses words.

Wrong.

Depends on addressability.

Trap 3: Direct cache uses LRU.

No victim selection is needed.

Trap 4: Fully associative has index bits.

$$\boxed{\text{Index}=0}$$

Trap 5: Hit ratio and miss ratio are independent.

No.

$$h+m=1$$

Trap 6: Main-memory access time is always miss penalty.

Not necessarily. It depends on whether cache lookup/transfer time is included.

Trap 7: LRU hit doesn't change order.

Wrong.

Trap 8: FIFO hit changes insertion order.

Usually no.

Trap 9: Dirty block can be discarded.

Not in ordinary write-back semantics.

Trap 10: Cache size always includes tag bits.

Normally stated cache capacity means data capacity unless otherwise specified.

14.61 GATE Fast Formula Sheet

For standard byte-addressed cache:

If:

$$\boxed{\text{Cache} = 2^C \text{ B}}$$

$$\boxed{\text{Block} = 2^B \text{ B}}$$

$$\boxed{\text{Ways} = 2^W}$$

$$\boxed{\text{Address} = A \text{ bits}}$$

then:

Lines

$$\boxed{2^{C-B}}$$

Sets

$$\boxed{2^{C-B-W}}$$

Offset

$$\boxed{B}$$

Set bits

$$\boxed{C-B-W}$$

Tag

$$\boxed{A-C+W}$$

14.62 Mapping Formula Sheet

Direct

$$\boxed{\text{Line} = \text{Block} \bmod \text{Lines}}$$

Set associative

$$\boxed{\text{Set} = \text{Block} \bmod \text{Sets}}$$

Fully associative

$$\boxed{\text{Any line}}$$

14.63 Performance Formula Sheet

Miss rate

$$\boxed{m = 1 - h}$$

AMAT

$$\boxed{\text{AMAT} = T_h + mP}$$

Two-level

$$\boxed{AMAT = T_1 + m_1(T_2 + m_2P)}$$

Global L2 miss rate

$$\boxed{m_1 m_2}$$

Actual CPI

$$\boxed{CPI = CPI_{\text{base}} + \text{MemoryStallCycles}}$$

14.64 Write-Back Formula

If dirty victim probability is d :

$$\boxed{\text{AverageMissPenalty} = \text{Fetch} + d(\text{WriteBack})}$$

Then:

$$\boxed{AMAT = \text{HitTime} + m(\text{AverageMissPenalty})}$$

14.65 Universal Safe Method

If you forget every cache formula, use this:

Draw every possible path and calculate probability $\times$ time.

For performance:

$$\boxed{\text{Average} = \sum p_i T_i}$$

For mapping:

Address !' Block !' Set/Line !' Compare Tag !' Hit/Miss

These two methods can reconstruct almost everything.

CHAPTER 14 — MIXED GATE PRACTICE

Do not look for answers immediately. Solve first.

Q1 — L1 NAT

A 32-bit byte-addressed processor has:

- 128 KB direct-mapped cache
- 64 B blocks.

Find tag bits.

Q2 — L1 NAT

For Q1, find:

- cache lines,
 - index bits,
 - offset bits.
-

Q3 — L2 NAT

A 36-bit byte-addressed processor has:

- 256 KB cache,
- 64 B block,
- 8-way associativity.

Find tag bits.

Q4 — L2

A direct cache has 8 lines and is initially empty.

Block sequence:

TEXT

0, 8, 1, 9, 0, 8, 1, 9

Find hits and misses.

Q5 — L2

Same total cache, but 2-way set associative with 4 sets using LRU.

Use the same sequence:

TEXT

0, 8, 1, 9, 0, 8, 1, 9

Find hits and misses.

Q6 — L2 NAT

Cache hit time = 1.5 ns.

Hit rate = 97%.

Additional miss penalty = 100 ns.

Find AMAT.

Q7 — L2 NAT

L1:

- 1 ns
-

- miss = 8%

L2:

- 6 ns
- local miss = 25%

Main memory penalty = 80 ns.

Find AMAT.

Q8 — L2 NAT

Base CPI = 1.

I-cache miss = 1%.

D-cache miss = 4%.

25% of instructions access data.

Miss penalty = 50 cycles.

Find actual CPI.

Q9 — L2 NAT

CPU = 4 GHz.

Cache miss penalty = 25 ns.

Find miss penalty in cycles.

Q10 — L3 NAT

Cache:

- 4096 lines
- tag = 20 bits
- valid = 1 bit
- dirty = 1 bit.

Find total metadata bits.

Q11 — L3 NAT

Hit time = 2 ns.

Miss rate = 4%.

Block fetch = 90 ns.

30% of evicted blocks are dirty.

Dirty write-back = 90 ns.

Find:

1. average miss penalty,
2. AMAT.

Q12 — L3

A 2-way set-associative cache has 2 sets and uses LRU.

Reference string:

TEXT

0, 2, 4, 0, 2, 6, 4, 0

Find:

- hits,
- misses,
- final contents of each set.

Q13 — L3 MSQ

For fixed cache size and block size, increasing associativity generally:

A. decreases number of sets B. increases tag bits C. decreases offset bits D. reduces conflict misses E. may increase hit-path hardware complexity

Q14 — L3 NAT

A 32-bit byte-addressed cache contains 64 KB data and uses 32-byte blocks with 4-way associativity.

Each line has:

- 1 valid bit,
- 1 dirty bit.

Assume no replacement metadata.

Find total metadata storage in bits.

CHAPTER 14 — LAST-MINUTE CACHE CHEAT SHEET

Direct

TEXT

Tag | Index | Offset

$\lfloor \text{Line} = \text{Block} \backslash \text{bmod Lines} \rfloor$

Set Associative

TEXT

Tag | Set | Offset

\[Set=Block\bmod Sets \]

Fully Associative

TEXT

Tag | Offset

Any line.

LRU

replace least recently used.

FIFO

replace oldest inserted.

Write-through

update memory now.

Write-back

update memory later; dirty bit.

Write allocate

bring block into cache on write miss.

No-write allocate

bypass cache on write miss.

CHAPTER 14 COMPLETION STATUS

' Address decomposition numericals ' Direct mapping sequences ' Set-associative sequences ' Fully associative sequences ' LRU vs FIFO ' Conflict-miss analysis ' AMAT ' Sequential vs parallel timing ' Two-level cache ' Three-level cache ' Dirty write-back penalty ' CPI + cache stalls ' Time-to-cycle conversion ' Tag/metadata overhead ' Block-transfer numericals ' Reverse AMAT numericals ' Memory-traffic numericals ' Speedup numericals ' Mixed GATE traps

CHAPTER 14: COMPLETE '

CACHE UNIT STATUS

You have now completed:

' Cache fundamentals ' Direct mapping ' Fully associative mapping ' Set-associative mapping ' Replacement policies ' Write policies ' Cache performance numericals

So the official GATE phrase:

“cache memory mapping”

is now covered in depth.

COURSE PROGRESS

Completed: 15 / 26 modules

Approximate roadmap progress:

$\sim 58\%$

NEXT !' CHAPTER 15 — I/O ORGANIZATION & I/O INTERFACE

Next we move to the official syllabus block:

~~I/O~~ interface

We will cover:

- why I/O interfacing is needed,
- CPU vs device speed mismatch,
- I/O modules/controllers,
- data/status/control registers,
- programmed I/O,
- memory-mapped I/O,
- isolated/port-mapped I/O,
- polling preview,
- handshaking,
- synchronous vs asynchronous transfer,
- GATE comparison questions.

Then Chapter 16 will go deep into **Interrupts**, and Chapter 17 into **DMA**.

USER

next

ASSISTANT

CHAPTER 15 — I/O ORGANIZATION & I/O INTERFACE

Classification: [GATE CORE + GATE SUPPORT] **Priority:** +HIGH PRIORITY **Official syllabus connection:** I/O interface

The CPU is extremely fast, while I/O devices such as keyboards, disks, printers, sensors, and network interfaces operate at very different speeds.

So the CPU usually does **not** communicate with a raw device directly.

It communicates through an:

$\boxed{\text{I/O Interface / I/O Controller}}$

15.1 What is I/O?

I/O = Input / Output

Examples:

Input devices

TEXT

Keyboard
Mouse
Microphone
Camera
Sensors
Network receiver

Output devices

TEXT

Monitor
Printer
Speaker
Network transmitter

Storage devices

TEXT

SSD
HDD
USB storage

15.2 The Main I/O Problem

CPU and peripherals differ greatly in:

- speed,
- data format,
- transfer size,
- timing,
- control requirements.

Example:

TEXT

CPU: billions of operations/second

Keyboard:
human typing speed

It makes no sense for the CPU to communicate with the keyboard as though it were another CPU register.

Hence we need an interface.

15.3 I/O Interface

An **I/O interface** acts as an intermediary between:

TEXT

C P U ! " P e r i p h e r a l

Conceptually:

TEXT

CPU
! "
I/O Controller
! "
Device

It can handle:

- data buffering,
- status information,
- control commands,
- timing,
- protocol conversion.

15.4 Why Buffering Is Needed

Suppose device produces data slowly while CPU is very fast.

The controller can temporarily store data in a:

$\boxed{\text{Buffer}}$

Then CPU accesses the buffer when appropriate.

Example:

TEXT

Keyboard
! "
I/O buffer
! "
CPU

This helps handle speed mismatch.

15.5 Basic I/O Interface Registers

A typical I/O controller may contain:

1. Data Register
 2. Status Register
 3. Control Register
-

15.6 Data Register

Stores actual input/output data.

Example:

Keyboard controller receives:

```
TEXT
'A'
```

It may place the character in the data register.

CPU reads that register.

15.7 Status Register

Contains information about device state.

Example bits:

```
TEXT
Ready
Busy
Error
Interrupt Pending
Data Available
```

Suppose:

```
TEXT
Ready = 1
```

means device can perform the requested transfer.

15.8 Control Register

CPU writes commands/settings here.

Examples:

TEXT

Start device
Stop device
Reset
Enable interrupt
Set transfer mode

Memory Trick

DSC

TEXT

D a t a ! ' w h a t
S t a t u s ! ' c o n d i t i o n
C o n t r o l ! ' c o m m a n d

15.9 Simplified I/O Controller

TEXT

```

CPU
      %
% % % % % % %4% % % % % % %
%   I / O   I n t e r f a c e %
%                                     %
%   D a t a   R e g i s t e r   %
%   S t a t u s   R e g .       %
%   C o n t r o l   R e g .     %
% % % % % % % , % % % % % % % %
%                                     %
Device
```

15.10 CPU–I/O Communication Methods

Three major techniques are important:

1. Programmed I/O / Polling
2. Interrupt-Driven I/O
3. DMA

We will cover interrupts and DMA deeply in Chapters 16 and 17.

This chapter builds the interface foundation.

15.11 Programmed I/O

In **Programmed I/O**, CPU itself executes instructions to control the I/O transfer.

Typical flow:

TEXT

```
CPU checks status
!"
Device ready?
!"
CPU transfers data
```

CPU is directly involved in each transfer.

15.12 Polling

Polling means CPU repeatedly asks:

"Are you ready yet?"

Example:

```
C
while (device_ready == 0) {
    // keep checking
}
read_data();
```

This repeated checking is called:

`\boxed{\text{Busy Waiting}}`

15.13 Polling Flow

TEXT

```

% % % % % % % % % % % % %
%   C h e c k   S t a t u s %
% % % % % % % % , % % % % % %
%                               !"
Device Ready?
!™                               !~
No           Yes
!~                               !"
Check again   Transfer data
```

15.14 Main Problem with Polling

CPU wastes cycles repeatedly checking the device.

Suppose device becomes ready only after:

`\[ 1ms \]`

while CPU executes instructions in nanoseconds.

Huge amount of CPU time may be wasted.

15.15 Advantage of Polling

Polling is:

- simple,
- easy to implement,
- predictable.

It may be acceptable when:

- device is very fast,
- waiting time is tiny,
- system is simple.

15.16 Polling Memory Trick

Polling = CPU keeps asking.

15.17 Interrupt Preview

Instead of CPU repeatedly asking:

"Are you ready?"

the device can tell CPU:

"I am ready!"

That signal is called an:

$\boxed{\text{Interrupt}}$

Then CPU can do other work meanwhile.

We study this fully next chapter.

15.18 DMA Preview

For large data transfers, even interrupts can create too much CPU involvement.

DMA allows data transfer:

TEXT

I / O D e v i c e ! " M a i n M e m o r y

with much less CPU participation.

We study this in Chapter 17.

15.19 Three I/O Methods — Big Picture

TEXT

Polling:
CPU checks repeatedly

Interrupt:
Device alerts CPU

DMA:
Controller transfers block largely without CPU

Memory trick:

Ask !' Alert !' Automate

TEXT

Polling = Ask
Interrupt = Alert
DMA = Automate

15.20 I/O Addressing

CPU must identify which I/O register/device it wants to access.

Two major approaches:

1. Memory-Mapped I/O
2. Isolated I/O / Port-Mapped I/O

0-100% important conceptual topic.

15.21 Memory-Mapped I/O

In **Memory-Mapped I/O**, I/O registers are assigned addresses from the normal memory address space.

Example:

TEXT

```
0 0 0 0 0 0 0 0 0 H - 7 F F F F F F F H !' R A M
8 0 0 0 0 0 0 0 0 H !' K e y b o a r d S t a t u s
8 0 0 0 0 0 0 0 4 H !' K e y b o a r d D a t a
8 0 0 0 0 0 0 0 8 H !' D i s p l a y C o n t r o l
```

CPU uses ordinary memory-style instructions to access them.

15.22 Core Idea

Suppose instruction:

TEXT

```
LOAD R1, [80000004H]
```

If that address belongs to an I/O register, CPU reads device data instead of RAM.

Thus:

$\boxed{\text{Memory and I/O share the address space}}$

15.23 Advantages of Memory-Mapped I/O

- Same instructions can often access memory and I/O.
- Same addressing modes may be usable.
- Programming model can be simpler.

15.24 Disadvantage

I/O consumes part of the memory address space.

If CPU has:

2^{32}

possible byte addresses and some are assigned to devices, those particular addresses cannot simultaneously address ordinary RAM under that mapping.

15.25 Isolated I/O

Also called:

$\boxed{\text{Port-Mapped I/O}}$

Memory and I/O have separate address spaces.

Conceptually:

TEXT

```
Memory Address Space  
and  
I/O Port Address Space
```

Special I/O instructions may be used.

Example conceptual instructions:

TEXT

```
IN  
OUT
```

15.26 Isolated I/O Example

Suppose:

TEXT

Memory address 50

and:

TEXT

I/O port 50

can both exist because the spaces are separate.

CPU knows which space is intended from instruction/control signals.

15.27 Memory-Mapped vs Isolated I/O

Feature	Memory-Mapped I/O	Isolated I/O
Address space	Shared with memory	Separate
Instructions	Normal memory instructions	Often special I/O instructions
Addressing modes	Usually broader	Often limited
Memory address space consumed	Yes	No
Hardware/control distinction	Address decoding	Separate I/O control often used

Memory Trick

Memory-Mapped

Memory and devices live on the same map.

Isolated

I/O has its own island.

15.28 GATE Example — Address Space

Suppose CPU has 16-bit addresses.

Total addressable locations:

$2^{16} = 65536$

If 4 KB of memory-mapped address range is reserved for I/O:

Available address locations for normal memory mapping:

$65536 - 4096$

$= 61440$

assuming every address corresponds to one byte/location in the stated model.

15.29 Isolated I/O Advantage in Same Situation

If I/O has a completely separate port address space:

the normal 16-bit memory address space need not be reduced by I/O ports.

15.30 I/O Data Transfer Direction

I/O operations can be:

Input

[Device $\rightarrow$ CPU]

Example keyboard.

Output

[CPU $\rightarrow$ Device]

Example printer.

15.31 CPU Reads an Input Device

Simplified:

TEXT

1. CPU reads status register
2. Checks Ready/Data Available
3. Reads data register
4. Controller/device updates status

This is typical programmed I/O logic.

15.32 CPU Writes an Output Device

Example printer:

TEXT

1. Check printer ready
2. Write character to data register
3. Controller marks device busy
4. Printer processes character
5. Ready becomes 1 again

15.33 Ready and Busy Flags

Common device status indicators:

Ready = 1

Device can accept/provide data.

Busy = 1

Device is currently working and may not accept another operation.

Do not assume exact polarity unless stated.

15.34 Handshaking

Useful concept.

When two devices operate asynchronously, they may exchange control signals to coordinate transfer.

This is called:

$\boxed{\text{Handshaking}}$

Example:

TEXT

```
Sender !' Data Ready
Receiver !' Acknowledge
```

15.35 Why Handshaking?

Suppose sender and receiver do not share exactly the same timing.

Sender needs to know:

Has receiver accepted my data?

Receiver needs to know:

Is the data valid now?

Handshaking solves this.

15.36 Simple Handshake Sequence

TEXT

```
Sender places data
!"
Sender asserts DATA_READY
!"
Receiver reads data
!"
Receiver asserts ACK
!"
Sender removes DATA_READY
!"
Receiver removes ACK
```

This coordinates asynchronous transfer safely.

15.37 Synchronous Transfer

In **synchronous I/O transfer**, components coordinate using a shared/common timing reference such as a clock.

Conceptually:

TEXT

Transfer occurs at known clock instants

15.38 Asynchronous Transfer

In **asynchronous transfer**, sender and receiver need not share the same clock.

They coordinate using:

- handshaking,
- request/acknowledge signals,
- other timing protocols.

15.39 Synchronous vs Asynchronous

Feature	Synchronous	Asynchronous
Common timing	Yes	Not necessarily
Control	Clock based	Handshake/event based
Timing relation	Predictable	Flexible
Different device speeds	Less flexible	Better suited

15.40 Strobe Control — Support

[LOW PRIORITY]

Another asynchronous transfer idea is a **strobe signal** indicating when data is valid.

But handshaking is safer because both sides acknowledge the transfer.

For the official syllabus, understand the broad idea without over-investing.

15.41 Data Transfer Width

An I/O interface may transfer:

- byte,
- word,

- block.

Example:

8-bit interface:

8 bits/transfer

32-bit interface:

32 bits/transfer

Transfer width matters for bandwidth.

15.42 I/O Bandwidth

If device transfers:

$D \text{ bytes}$

every:

$T \text{ seconds}$

then:

$\boxed{\text{Bandwidth} = \frac{D}{T}}$

15.43 Example

Device transfers 4 bytes every $2 \frac{1}{4} \text{ s}$.

$\text{Bandwidth} = \frac{4}{2 \frac{1}{4} \text{ s}}$

$= 2 \times 10^6 \text{ bytes/s}$

$\boxed{2 \text{ MB/s}}$

using decimal rate units.

15.44 Transfer Rate Example

A device sends:

1000

bytes every millisecond.

Per second:

1000×1000

$= 10^6 \text{ bytes/s}$

$\boxed{1 \text{ MB/s}}$

15.45 CPU Cost of Polling

Suppose CPU checks a device every:

$$\sqrt{10\mu s}$$

Each polling operation consumes:

$$\sqrt{100\text{ CPU cycles}}$$

Device requires 1000 polls/second.

CPU cycles spent:

$$\sqrt{1000 \times 100}$$

$$\sqrt{\boxed{100000\text{ cycles/s}}}$$

CPU utilization can be calculated if CPU cycles/s are known.

15.46 Polling Overhead Numerical

Suppose:

- CPU = 1 GHz
- each poll = 100 cycles
- 1 million polls/s

CPU cycles available/sec:

$$\sqrt{10^9}$$

Polling cycles:

$$\sqrt{10^6 \times 100 = 10^8}$$

Fraction:

$$\sqrt{\frac{10^8}{10^9} = 0.1}$$

$$\sqrt{\boxed{10\%}}$$

CPU time spent polling.

15.47 Why Interrupts Can Help

If device generates only occasional events, polling continuously is wasteful.

Interrupt-based system:

TEXT

```
CPU does useful work
!"
Device becomes ready
!"
Interrupt
!"
CPU handles device
!"
Resume work
```

This can reduce wasted CPU checking.

15.48 But Interrupts Also Have Overhead

Interrupts are not free.

CPU may need to:

- finish/currently suspend work,
- save state,
- identify interrupt,
- run handler,
- restore state.

Thus for extremely frequent transfers, interrupt overhead can also become significant.

This motivates DMA for bulk transfers.

15.49 When Polling Can Be Better

Polling is not always bad.

If device response is expected almost immediately:

TEXT

```
check once or twice !' ready
```

polling may be simpler and cheaper than interrupt setup/handling.

So choice depends on workload.

15.50 I/O Controller vs CPU

I/O controller can perform low-level tasks such as:

- device protocol,
- buffering,
- error status,
- converting device signals,
- controlling physical operation.

CPU sees a simpler register-based interface.

15.51 Device Controller vs Device Driver

[PLACEMENT/INTERVIEW]

Do not confuse:

Device Controller

Hardware.

Device Driver

Software—usually OS code used to control the device/controller.

Memory trick:

Controller = hardware

Driver = software

15.52 I/O Interface and Bus

CPU, memory and I/O may communicate over system buses.

Recall:

Address bus

Select destination/register.

Data bus

Transfers actual data.

Control bus

Read/write/interrupt/etc.

15.53 I/O Read

Conceptually:

TEXT

```
CPU places I/O address
!"
I/O Read control active
!"
Controller places data on bus
!"
CPU receives data
```

15.54 I/O Write

Conceptually:

TEXT

```
CPU places I/O address
!"
CPU places data
!"
I/O Write control active
!"
Controller accepts data
```

15.55 Device Selection

Just like memory-chip select, address decoding can select an I/O device/register.

Example:

TEXT

```
Port 00 !' Keyboard
Port 01 !' Display
Port 10 !' Disk Controller
Port 11 !' Network Controller
```

A decoder/controller determines which interface responds.

15.56 Multiple Registers per Device

One physical device may occupy multiple addresses.

Example:

TEXT

```
0x1000 !' Data
0x1004 !' Status
0x1008 !' Control
```

All may belong to the same device controller.

15.57 Memory-Mapped I/O Numerical

A 32-bit byte-addressed system reserves address range:

TEXT

```
FFFF0000H - FFFFFFFFH
```

for I/O.

How many byte addresses?

Use:

$\lfloor \text{End} - \text{Start} + 1 \rfloor$

$\lfloor \text{FFFFFFFH} - \text{FFFF0000H} + 1 \rfloor$

$\lfloor = 10000 \rfloor$

$\lfloor = 2^{16} \rfloor$

Therefore:

$\lfloor \boxed{64\text{KB}} \rfloor$

of address space is reserved for I/O.

15.58 Address Range Trap

Always use:

$\lceil \text{End} - \text{Start} + 1 \rceil$

because both endpoints are included.

15.59 Port Address Bits

Suppose isolated I/O supports:

256

ports.

Minimum port-address bits:

$\lceil \log_2 256 \rceil$

8

15.60 Another Example

System supports 1024 I/O port addresses.

$1024 = 2^{10}$

So:

$\lceil 10 \rceil$ port-address bits

15.61 Programmed I/O Data Transfer

Suppose one transfer requires:

TEXT

Read status

Check ready

Read/write data

CPU participates explicitly.

This differs strongly from DMA, where a controller can transfer a whole block.

15.62 Programmed I/O vs Interrupt-Driven I/O

Feature	Programmed/Polling	Interrupt-Driven
Device ready detection	CPU repeatedly checks	Device signals CPU
Busy waiting	Yes	Usually avoided
CPU overhead while waiting	High	Lower
Implementation	Simple	More complex

Feature	Programmed/Polling	Interrupt-Driven
Good for rare/slow events	Poorer	Better

15.63 Programmed I/O vs DMA Preview

Feature	Programmed I/O	DMA
CPU moves each data item	Yes	No, controller does
CPU involvement	High	Low
Bulk transfer	Inefficient	Efficient
Hardware complexity	Lower	Higher

15.64 Three Method Spectrum

Think of CPU involvement:

TEXT

Polling	Interrupt	DMA
High	Low	

As we move toward DMA:

[\boxed{\text{CPU involvement generally decreases}}]

15.65 Interrupt vs DMA Important Preview

Interrupt does **not** automatically mean data bypasses CPU.

Interrupt merely tells CPU:

service the device.

In interrupt-driven I/O, CPU may still move the data.

DMA actually lets a DMA controller perform memory-device transfers.

Very important distinction.

15.66 I/O Transfer Granularity

Typical tendency:

Programmed I/O

Small/individual transfers.

Interrupt-driven

Moderate asynchronous events.

DMA

Large blocks/high-speed transfers.

Not an absolute law, but useful intuition.

15.67 GATE Example — Memory-Mapped I/O

Which is true?

A. I/O has a separate address space. B. I/O registers share normal memory address space. C. CPU cannot use memory-style instructions. D. Memory addresses increase automatically.

Correct concept:

$\boxed{B}$

15.68 GATE Example — Isolated I/O

Which is generally associated with isolated I/O?

$\boxed{\text{Separate I/O address space}}$

and often:

$\boxed{\text{Special I/O instructions}}$

15.69 GATE Example — Polling

If CPU continuously tests device's ready bit, method is:

$\boxed{\text{Polling}}$

15.70 GATE Example — Handshake

If sender asserts Request and receiver responds with Acknowledge:

$\boxed{\text{Handshaking}}$

15.71 GATE Trap — Memory-Mapped Means Device is RAM

'L Wrong.

It only means device registers are assigned addresses in the memory address space.

The device is still an I/O device.

15.72 GATE Trap — Isolated I/O Uses No Addresses

'LWrong.

It still uses I/O/port addresses.

They are simply in a separate address space.

15.73 GATE Trap — Polling Means Interrupt

'LOpposite idea.

Polling:

CPU checks device.

Interrupt:

device alerts CPU.

15.74 GATE Trap — Interrupt Removes CPU from Data Transfer

Not necessarily.

Interrupt-driven I/O can still require CPU to execute instructions that move data.

DMA is the mechanism designed to reduce CPU involvement in bulk transfer.

15.75 GATE Trap — DMA Means CPU Does Nothing

Not exactly.

CPU typically:

- initializes DMA,
- gives source/destination/count/control,
- then DMA performs transfer,
- CPU may receive completion interrupt.

We cover this fully later.

15.76 GATE Trap — Busy Waiting Is Always Wrong

No.

It wastes CPU when waits are long, but can be appropriate for very short or predictable waits.

15.77 GATE Trap — Data and Status Register Are Same

Usually logically different:

- Data register stores transferred data.
-

- Status register stores state bits.

15.78 GATE Trap — Shared Clock in Asynchronous Transfer

Not required.

Asynchronous communication commonly uses handshaking rather than fixed common timing.

15.79 Fast GATE Recognition Table

Phrase in Question	Think
CPU repeatedly checks status	Polling
Device signals CPU	Interrupt
Device !" Memory block transfer	DMA
I/O shares memory address space	Memory-mapped
Separate port address space	Isolated I/O
Request + acknowledgement	Handshaking
Shared timing/clock	Synchronous
No common timing requirement	Asynchronous

15.80 Memory-Mapped vs Isolated — Fast Trick

Memory-Mapped

M = Merge

Memory + I/O addresses merged.

Isolated

I = Independent

Separate I/O address space.

15.81 I/O Performance Numerical

Suppose:

- device generates 5000 events/s,
- handling each event via polling consumes 400 cycles,
- CPU = 2 GHz.

Cycles spent:

$$\lfloor 5000 \times 400 = 2,000,000 \rfloor$$

Total cycles/sec:

$$\lfloor 2 \times 10^9 \rfloor$$

Fraction:

$$\lfloor \frac{2 \times 10^6}{2 \times 10^9} = 0.001 \rfloor$$

$$\lfloor \boxed{0.1\%} \rfloor$$

under the stated simplified assumptions.

15.82 Another CPU Utilization Example

Suppose polling loop takes:

$$\lfloor 100 \rfloor$$

cycles and executes:

$$\lfloor 5 \times 10^6 \rfloor$$

times per second.

CPU = 1 GHz.

Polling cycles:

$$\lfloor 100(5 \times 10^6) = 5 \times 10^8 \rfloor$$

Total:

$$\lfloor 10^9 \rfloor$$

CPU fraction:

$$\lfloor 0.5 \rfloor$$

$$\lfloor \boxed{50\%} \rfloor$$

This illustrates how polling can become expensive.

15.83 Throughput Numerical

Device transfers:

$$\lfloor 512B \rfloor$$

every:

$$\lfloor 100 \mu s \rfloor$$

Transfers per second:

$$\lfloor \frac{1}{100 \times 10^{-6}} = 10000 \rfloor$$

Bandwidth:

$$\lfloor 512 \times 10000 \rfloor$$

$$\lfloor = 5,120,000B/s \rfloor$$

$\boxed{5.12\text{MB/s}}$

using decimal MB/s.

15.84 Address-Decoding Numerical

Memory-mapped I/O device occupies:

TEXT

8000H – 80FFH

Number of addresses:

$\boxed{80\text{FF}_\text{H} - 8000_\text{H} + 1}$

$\boxed{=100_\text{H}}$

$\boxed{=256}$

If byte-addressed:

$\boxed{256 \text{ bytes of address space}}$

15.85 Port-Mapped Numerical

An isolated I/O architecture has 12-bit port addresses.

Maximum number of distinct port addresses:

$\boxed{2^{12}}$

$\boxed{4096}$

CHAPTER 15 — FORMULA SHEET

Number of I/O addresses

$\boxed{2^n}$

for n port/address bits.

Inclusive address range

$\boxed{\text{End} - \text{Start} + 1}$

Bandwidth

$\boxed{\text{Bandwidth} = \frac{\text{Data Transferred}}{\text{Time}}}$

CPU polling utilization

$\boxed{\text{Utilization} = \frac{\text{Polling Cycles/sec}}{\text{Total CPU Cycles/sec}}}$

where:

$\boxed{\text{Polling Cycles/sec} = \text{Polls/sec} \times \text{Cycles/Poll}}$

CHAPTER 15 — MEMORY TRICKS

Interface Registers

DSC

Data Status Control

Three I/O methods

Ask !' Alert !' Automate

Polling !' Interrupt !' DMA

Memory-Mapped

Merge addresses

Isolated I/O

Independent address spaces

Handshake

Request !' Acknowledge

CHAPTER SNAPSHOT

1. I/O interface connects CPU to peripherals.
2. I/O controllers handle speed/timing/protocol differences.
3. Data register stores transferred data.
4. Status register reports device state.
5. Control register receives commands.
6. Programmed I/O requires explicit CPU involvement.
7. Polling repeatedly checks device status.
8. Polling can cause busy waiting.
9. Interrupt-driven I/O lets the device signal the CPU.
10. DMA reduces CPU involvement in bulk transfer.
11. Memory-mapped I/O shares the normal memory address space.
12. Isolated I/O uses a separate port address space.
13. Handshaking coordinates asynchronous transfers.
14. Synchronous transfer relies on common timing.
15. Asynchronous transfer need not share a common clock.
16. Device controller is hardware; device driver is software.
17. Interrupt and DMA are not the same mechanism.

MUST REMEMBER

$\boxed{\text{Polling = CPU asks}}$

$\boxed{\text{Interrupt = device alerts}}$

$\boxed{\text{DMA = controller transfers}}$

and:

$\boxed{\text{Memory-Mapped I/O = shared address space}}$

$\boxed{\text{Isolated I/O = separate address space}}$

PRACTICE SET

Q1 — L0 MCQ

Which register usually stores actual I/O data?

A. Status register B. Data register C. PC D. Control register

Q2 — L0 MCQ

Repeatedly checking a device's ready bit is called:

A. DMA B. Interrupt C. Polling D. Caching

Q3 — L1 MCQ

In memory-mapped I/O:

A. I/O has no addresses B. I/O shares the normal memory address space C. I/O can only use interrupts D. Main memory is not addressable

Q4 — L1 MCQ

Isolated I/O generally uses:

A. One shared memory/I/O address space B. Separate I/O address space C. Cache tags D. Page tables

Q5 — L1 NAT

An isolated I/O system uses 10-bit port addresses.

How many port addresses are possible?

Q6 — L1 NAT

A memory-mapped I/O device occupies:

How many byte addresses are reserved?

Q7 — L2

Explain the difference between:

- data register,
 - status register,
 - control register.
-

Q8 — L2 MSQ

Which statements about polling are generally true?

A. CPU repeatedly checks device status B. It may cause busy waiting C. Device always transfers directly to memory D. It is simple to implement

Q9 — L2 MSQ

Which are generally advantages of memory-mapped I/O?

A. Ordinary memory-style instructions may access devices B. I/O requires no addresses C. Rich memory addressing modes may be usable D. It consumes no memory address range

Q10 — L2 NAT

A CPU runs at 2 GHz. It performs 2 million polls per second, each requiring 200 cycles.

What percentage of CPU cycles are spent polling?

Q11 — L2 NAT

A device transfers 64 bytes every $8 \frac{1}{4}$ s .

Find transfer rate in MB/s using $(1\text{MB}=10^6\text{B})$.

Q12 — L3

A 16-bit byte-addressed machine reserves 8 KB of its memory address space for memory-mapped I/O.

How many byte addresses remain available for ordinary memory mapping?

Q13 — L3 Conceptual

Why can interrupt-driven I/O reduce busy waiting but still require significant CPU work for large transfers?

CHAPTER 15 COMPLETION STATUS

' Need for I/O interface ' CPU/device speed mismatch ' I/O controller ' Data/status/control registers ' Programmed I/O ' Polling ' Busy waiting ' Interrupt preview ' DMA preview ' Memory-mapped I/O ' Isolated/port-mapped I/O ' Address-space numericals ' Handshaking ' Synchronous transfer ' Asynchronous transfer ' I/O bandwidth ' CPU polling overhead ' GATE traps and shortcuts

CHAPTER 15: COMPLETE '

COURSE PROGRESS

Completed: 16 / 26 modules

Approximate roadmap progress:

$\sim 62\%$

NEXT !' CHAPTER 16 — INTERRUPTS

Next we go deep into one of the explicit official GATE topics:

Interrupt request and interrupt handling ISIRI / interrupt service routine Interrupt cycle Context saving maskable vs non-maskable Hardware vs software interrupts Vectored vs non-vectored interrupts Interrupt priority Nested interrupts Polling vs interrupt comparison Interrupt overhead and CPU-utilization numericals.

USER

next

ASSISTANT

CHAPTER 16 — INTERRUPTS

Classification: [GATE CORE] **Priority:** VERY HIGH **Official syllabus connection:** I/O interface — Interrupt

Interrupts are one of the central ideas of I/O organization.

Core idea:

Instead of the CPU continuously checking whether a device needs attention, the device can signal the CPU when service is required.

$\text{Interrupt} = \text{Request for CPU attention}$

16.1 Polling vs Interrupt — First Intuition

With polling:

TEXT

CPU: Ready?
Device: No.

CPU: Ready?
Device: No.

CPU: Ready?
Device: Yes.

CPU wastes time repeatedly checking.

With interrupts:

TEXT

CPU does useful work
!"
Device becomes ready
!"
Device sends interrupt
!"
CPU temporarily services device
!"
CPU returns to previous work

Memory trick:

Polling = CPU asks

Interrupt = Device tells

16.2 What Exactly Is an Interrupt?

An interrupt is an event that causes the CPU to temporarily suspend its normal instruction sequence and execute special code called an:

$\boxed{\text{Interrupt Service Routine}}$

or:

$\boxed{\text{ISR}}$

After servicing the event, CPU usually resumes the interrupted program.

16.3 Basic Interrupt Flow

TEXT

Normal Program
!"
Instruction
!"
Interrupt request arrives
!"
CPU acknowledges interrupt


```
! "  
Save necessary state  
! "  
Execute ISR  
! "  
Restore state  
! "  
Resume interrupted program
```

16.4 Interrupt Service Routine — ISR

An **ISR** is the routine that handles the interrupt.

Examples:

- read keyboard character,
- acknowledge disk completion,
- process timer event,
- handle network packet arrival.

The ISR must eventually return control to the interrupted program.

16.5 Interrupt Cycle

The CPU instruction cycle can conceptually be expanded:

```
TEXT  
Fetch  
! "  
Decode  
! "  
Execute  
! "  
Check Interrupt  
! "  
Next instruction
```

If interrupt is pending and enabled:

```
TEXT  
Save current state  
! "  
Load ISR address  
! "  
Execute ISR  
! "  
Return
```

16.6 When Does CPU Accept an Interrupt?

In simplified GATE models, CPU commonly checks interrupts:

after completing the current instruction.

So an interrupt usually does not stop an instruction halfway through unless the architecture/question explicitly says otherwise.

This preserves a clean processor state.

16.7 Interrupt Latency

Interrupt latency is the time between:

interrupt request occurring

and:

CPU beginning meaningful service of that interrupt.

Latency may include:

- finishing current instruction,
- recognizing interrupt,
- saving state,
- locating ISR.

Lower interrupt latency is important for time-critical devices.

16.8 Context

The CPU's current execution state is often called its:

$\boxed{\text{Context}}$

It can include:

- PC
- status/flag register
- general registers
- other architectural state

Not every interrupt saves every register automatically; details depend on architecture.

For GATE, understand the general principle.

16.9 Why Context Must Be Saved

Suppose CPU is executing:

TEXT

R1 = 10

R2 = 20

PC = 500

Interrupt occurs.

ISR uses R1 and R2.

If old values are not preserved, returning to the original program could corrupt execution.

Therefore necessary state is saved before or during ISR processing.

16.10 PC Saving

The most important piece of state is the return address.

CPU must know:

Where should execution continue after ISR?

Therefore current/next PC is saved.

After ISR:

$\boxed{PC \leftarrow \text{SavedReturnAddress}}$

16.11 Status/Flag Saving

The processor status may include:

- Zero
- Carry
- Sign
- Overflow
- Interrupt enable

If ISR changes these flags, the interrupted program may behave incorrectly.

Hence relevant status information is usually preserved.

16.12 Interrupt Return

ISR ends with a special return operation, conceptually:

TEXT

Restore processor state

!"

Restore PC

!"

Resume program

This may be implemented by a special instruction such as an interrupt return instruction.

Exact instruction name depends on ISA.

16.13 Hardware Interrupt

A **hardware interrupt** is generated by hardware external or internal to CPU execution.

Examples:

- keyboard input,
- timer,
- network controller,
- disk controller.

Conceptually:

```
TEXT
Device !' Interrupt Request Line !' CPU
```

16.14 Software Interrupt

A **software interrupt** is triggered by executing a special instruction or software-generated event.

Historically these can be used to:

- request OS services,
- enter privileged software,
- implement system-call-like mechanisms.

Exact terminology varies by architecture.

16.15 Hardware vs Software Interrupt

Feature	Hardware Interrupt	Software Interrupt
Trigger	Device/hardware event	Program instruction/software event
Timing	Often asynchronous to program	Occurs at defined instruction point
Example	Keyboard ready	Software trap/system call mechanism

16.16 Exceptions vs Interrupts — Support

[SUPPORT]

Architectures often distinguish:

Interrupt

Usually external and asynchronous.

Exception

Usually caused by current instruction.

Examples:

- divide by zero,
- invalid opcode,

- page fault.

Terminology varies, so do not overgeneralize.

For your official syllabus, the important idea is interrupt-based I/O.

16.17 Maskable Interrupt

A **maskable interrupt** can be temporarily disabled or ignored by processor control.

For example, CPU may use an interrupt-enable bit.

If disabled:

TEXT

```
Interrupt request arrives
!"
CPU postpones/ignores it
```

depending on architecture.

16.18 Non-Maskable Interrupt — NMI

A **Non-Maskable Interrupt** generally cannot be disabled through the normal interrupt-mask mechanism.

Used for high-priority critical events.

Examples conceptually:

- serious hardware fault,
- power failure warning.

Memory trick:

NMI = No Masking Normally

16.19 Maskable vs Non-Maskable

Property	Maskable	Non-Maskable
Can normal masking disable it?	Yes	Generally no
Priority	Normal	Typically very high
Typical role	I/O events	Critical events

16.20 Interrupt Enable Flag

Many processors have a state bit such as:

TEXT

IE = Interrupt Enable

Conceptually:

TEXT

```
IE = 1 !' maskable interrupts accepted
IE = 0 !' maskable interrupts disabled
```

Exact polarity depends on processor.

16.21 Vectored Interrupt

Important.

In a **vectored interrupt**, the CPU can obtain the ISR address directly or indirectly from a predefined vector mechanism.

Each interrupt source may have an associated:

$\boxed{\text{Interrupt Vector}}$

which identifies the handler location.

16.22 Interrupt Vector Table

A collection of interrupt vectors can be stored in an:

$\boxed{\text{Interrupt Vector Table}}$

Conceptually:

TEXT

```
Interrupt 0 !' ISR address A
Interrupt 1 !' ISR address B
Interrupt 2 !' ISR address C
...
```

16.23 Vectored Interrupt Example

Suppose:

TEXT

```
Timer !' vector 5
Keyboard !' vector 9
Disk !' vector 12
```

Keyboard interrupt occurs.

CPU uses vector 9 to locate keyboard ISR.

No need to test every device in software to determine the source.

16.24 Non-Vectored Interrupt

In a **non-vectored interrupt**, a fixed/common ISR entry or other mechanism is used, and software may need to identify which device caused the interrupt.

Example conceptual flow:

TEXT

```
Interrupt occurs
!"
Enter common handler
!"
Check device A
!"
Check device B
!"
Check device C
!"
Find requester
```

16.25 Vectored vs Non-Vectored

Feature	Vectored	Non-Vectored
ISR location	Vector provides handler information	Common/fixed mechanism
Source identification	Faster/direct	May require polling
Hardware support	More	Simpler
Service latency	Often lower	Can be higher

Memory Trick

Vectored = address direction is given

Non-vectored = find the source

16.26 Interrupt Priority

Suppose multiple devices request interrupts simultaneously.

Which one should CPU service first?

We need:

$\boxed{\text{Interrupt Priority}}$

Example:

TEXT

```
P o w e r   f a i l u r e   ! '   h i g h e s t
D i s k                   ! '   m e d i u m
K e y b o a r d           ! '   l o w e r
```

16.27 Why Priority Is Needed

Different devices have different urgency.

A high-speed device might lose data if not serviced quickly.

A keyboard can usually wait longer.

So priority determines service order.

16.28 Fixed Priority

Each device has a predetermined priority.

Example:

TEXT

```
Device A > Device B > Device C
```

Simple but low-priority devices can potentially wait a long time.

16.29 Dynamic Priority

Priority can change over time.

This can help fairness and reduce starvation.

[SUPPORT]

For GATE, know the basic distinction.

16.30 Daisy-Chaining

One of the most important classic interrupt-priority mechanism.

Devices are connected serially for interrupt acknowledgement.

Conceptually:

TEXT

```
CPU Interrupt Acknowledge
      ! "
Device 1
      ! "
Device 2
      ! "
Device 3
```


The first requesting device encountered gets priority.

16.31 Daisy-Chain Priority

If arrangement is:

TEXT

C P U ! ' D 1 ! ' D 2 ! ' D 3 ! ' D 4

priority:

$$\boxed{D1 > D2 > D3 > D4}$$

assuming acknowledgement passes in that order.

Nearest device to CPU has highest priority.

16.32 Daisy-Chain Problem

A low-priority device may experience:

$$\boxed{\text{Starvation}}$$

if higher-priority devices continuously request service.

16.33 Parallel Priority

Another approach uses priority logic/encoder to determine highest-priority interrupt among simultaneous requests.

Conceptually:

TEXT

D 0 % %
D 1 % %\$
D 2 % %< ! ' P r i o r i t y E n c o d e r ! ' C P U
D 3 % %

More hardware but faster priority resolution.

16.34 Daisy Chain vs Parallel Priority

Feature	Daisy Chain	Parallel Priority
Hardware	Simpler	More complex
Priority	Physical chain order	Priority logic
Speed	Slower through chain	Faster
Flexibility	Lower	Higher

16.35 Nested Interrupts

Suppose CPU is already executing one ISR.

Another higher-priority interrupt occurs.

If CPU allows it, second ISR can interrupt the first ISR.

This is:

`\[ \boxed{\text{Nested Interrupt}} \]`

16.36 Nested Interrupt Example

TEXT

```
Main program
    !"
Keyboard interrupt
    !"
Keyboard ISR
    !"
Critical timer interrupt
    !"
Timer ISR
    !"
Return to Keyboard ISR
    !"
Return to Main program
```

16.37 Why Nesting Is Useful

A low-priority ISR should not block a much more urgent event.

Therefore high-priority interrupts may be allowed during low-priority ISR execution.

16.38 Context Stack in Nested Interrupts

Each nesting level must preserve enough state to return correctly.

Conceptually:

TEXT

```
Main context saved
    !"
ISR1 context saved
    !"
ISR2 executes
    !"
Return ISR1
    !"
Return Main
```

A stack is naturally useful for this.

16.39 Interrupt Masking During ISR

Some systems temporarily disable interrupts when ISR begins to prevent unwanted nesting.

Others selectively mask lower/equal-priority interrupts but allow higher-priority ones.

Exact details depend on architecture.

16.40 Interrupt Request and Acknowledge

Common conceptual signals:

INTR / IRQ

Device asks CPU for service.

INTA / Acknowledge

CPU indicates it accepted the interrupt.

Exact names differ by system.

16.41 Interrupt Handshake

Conceptually:

TEXT

```
D e v i c e ! '   I n t e r r u p t   R e q u e s t
C P U           ! '   I n t e r r u p t   A c k n o w l e d g e
D e v i c e ! '   V e c t o r   /   i n f o r m a t i o n
C P U           ! '   I S R
```

This can be part of a vectored interrupt system.

16.42 Interrupt Polling

Even with interrupts, CPU may need to identify the requesting device.

One method is **software polling** inside the interrupt handler.

Example:

TEXT

```
i f   d e v i c e 1 _ p e n d i n g ! '   s e r v i c e   D 1
e l s e   i f   d e v i c e 2 _ p e n d i n g ! '   s e r v i c e   D 2
...

```

This is different from continuous programmed-I/O polling during normal execution.

16.43 Priority Polling

If software checks devices in fixed order:

TEXT

D1
D2
D3

then that order itself establishes priority.

First checked device has highest effective priority.

16.44 Interrupt Overhead

Interrupt servicing costs CPU time.

Suppose one interrupt requires:

- 50 cycles to enter ISR
- 150 cycles to service
- 50 cycles to return

Total:

$$[50+150+50]$$

$$[\boxed{250 \text{ cycles/interrupt}}]$$

16.45 CPU Time Used by Interrupts

If:

- interrupts per second = (N)
- cycles per interrupt = (C)
- CPU frequency = (f) cycles/s

then:

$$[\boxed{\text{CPU Fraction} = \frac{NC}{f}}]$$

Percentage:

$$[\boxed{\text{CPU}\% = \frac{NC}{f} \times 100}]$$

16.46 GATE Numerical 1 — Interrupt Overhead

CPU:

$$[1\text{GHz} = 10^9 \text{ cycles/s}]$$

Device generates:

$$[10000 \text{ interrupts/s}]$$

Each requires:

$$[500 \text{ cycles}]$$

Interrupt cycles/sec:

$$[10000 \times 500 = 5 \times 10^6]$$

Fraction:

$$\left[\frac{5 \times 10^6}{10^9} = 0.005 \right]$$

Therefore:

$$\left[\boxed{0.5\%} \right]$$

CPU time.

16.47 GATE Numerical 2

CPU:

$$\left[2\text{GHz} \right]$$

Interrupt rate:

$$\left[100000/\text{s} \right]$$

Interrupt cost:

$$\left[1000 \text{ cycles} \right]$$

Cycles consumed:

$$\left[10^5 \times 10^3 = 10^8 \right]$$

Total cycles:

$$\left[2 \times 10^9 \right]$$

Fraction:

$$\left[0.05 \right]$$

$$\left[\boxed{5\%} \right]$$

16.48 Maximum Interrupt Rate

Suppose:

- CPU = 500 MHz
- ISR cost = 1000 cycles
- CPU could theoretically spend 100% time servicing interrupts.

Maximum:

$$\left[\frac{500 \times 10^6}{1000} \right]$$

$$\left[\boxed{500000 \text{ interrupts/s}} \right]$$

This leaves zero useful CPU time, so practical maximum must be lower.

16.49 Interrupt Data Transfer Overhead

Suppose each interrupt transfers only one byte.

If device produces 1 MB/s:

$$\left[10^6 \text{ bytes/s} \right]$$

Then approximately:

10^6 interrupts/s

would be needed if every byte causes an interrupt.

This could be extremely expensive.

This motivates transferring larger chunks or using DMA.

16.50 Interrupt per Block

Suppose device transfers blocks of:

1024 bytes

at:

10MB/s

Approximate blocks/s:

$\frac{10^7}{1024}$

≈ 9766

interrupts/s if one interrupt per block.

Far lower than one interrupt per byte.

16.51 Polling vs Interrupt CPU Cost

Suppose:

Polling

CPU spends 10 million cycles/s checking.

Interrupt

Actual interrupt handling costs 1 million cycles/s.

Then interrupts save:

9 million cycles/s

for useful work.

But if interrupts occur extremely frequently, interrupt overhead can become large too.

16.52 Interrupt Throughput Formula

If:

- event rate = R interrupts/s
- CPU cycles/interrupt = C

then service load:

$R \times C$ cycles/s

This should be compared against processor clock rate.

16.53 Response-Time Concept

Suppose interrupt occurs at time t_0 .

CPU begins ISR at t_1 .

Then:

$\boxed{\text{Latency} = t_1 - t_0}$

ISR completion at t_2 .

Full interrupt response/service time may include:

$t_2 - t_0$

depending on wording.

Do not confuse latency with total service time.

16.54 Interrupt Priority Numerical Logic

Suppose requests arrive simultaneously from:

TEXT

D1, D2, D3

Priority:

$D2 > D3 > D1$

Then D2 serviced first.

After completion, assuming pending requests remain:

D3 next, then D1.

Simple, but GATE may combine this with masking/nesting.

16.55 Nested Priority Example

Priorities:

$A > B > C$

CPU servicing C.

Then B arrives.

If higher-priority nesting enabled:

$B \text{ interrupts } C$

Then A arrives while B ISR runs:

$A \text{ interrupts } B$

Return order:

TEXT

```
A completes
!"
resume B
!"
B completes
!"
resume C
!"
C completes
```

16.56 Same-Priority Interrupts

Exact behavior depends on system:

- masking,
- arbitration,
- arrival order,
- fixed secondary priority.

Do not invent a universal rule.

Use question conditions.

16.57 Edge-Triggered vs Level-Triggered — Support

[SUPPORT / LOW PRIORITY]

Interrupt inputs may be detected as:

Edge-triggered

Triggered by signal transition.

Level-triggered

Active as long as interrupt signal stays at a particular level.

Understand concept only unless explicitly asked.

16.58 Interrupt Controller — Support

Systems with many devices may use an interrupt controller to:

- collect interrupt requests,
- prioritize them,
- mask selected interrupts,
- provide vectors.

This reduces complexity at CPU pins.

16.59 Interrupt Vector Size Numerical

Suppose vector field has 8 bits.

Possible vectors:

$$\lfloor 2^8 \rfloor$$

$$\lfloor \boxed{256} \rfloor$$

If each vector-table entry occupies 4 bytes, table size:

$$\lfloor 256 \times 4 \rfloor$$

$$\lfloor \boxed{1024\text{B}=1\text{KB}} \rfloor$$

16.60 Interrupt Vector Table Numerical

Suppose:

- 64 interrupt types
- each vector entry = 8 bytes.

Table size:

$$\lfloor 64 \times 8 \rfloor$$

$$\lfloor \boxed{512\text{B}} \rfloor$$

16.61 Vector Address Numerical

Suppose vector table begins at address 1000.

Each entry:

$$\lfloor 4\text{B} \rfloor$$

Interrupt type:

$$\lfloor 20 \rfloor$$

Its entry begins at:

$$\lfloor 1000 + 20(4) \rfloor$$

$$\lfloor \boxed{1080} \rfloor$$

assuming vector numbering begins from 0.

16.62 Priority Encoder Numerical

If priority hardware must distinguish among:

$$\lfloor 8 \rfloor$$

interrupt sources, encoded source ID requires:

$$\lfloor \log_2 8 \rfloor$$

$$\lfloor \boxed{3 \text{ bits}} \rfloor$$

If a separate "no interrupt" state must also be encoded in the same field:

$$\lfloor 8 + 1 = 9 \rfloor$$

needs:

$\boxed{4 \text{ bits}}$

Same encoding principle from Control Unit chapter.

16.63 Polling Priority Example

CPU checks:

TEXT

Device A
then B
then C
then D

If all request simultaneously and CPU services first found requester:

$\boxed{A > B > C > D}$

Priority follows polling order.

16.64 Daisy-Chain Example

Physical chain:

TEXT

C P U ! ' D 3 ! ' D 1 ! ' D 4 ! ' D 2

Highest priority:

$\boxed{D3}$

then:

$\boxed{D1 > D4 > D2}$

Priority depends on physical chain position, not device number.

16.65 Interrupt and DMA Relationship

Important preview:

DMA often generates an interrupt:

after completing a block transfer.

So DMA and interrupts are not mutually exclusive.

Typical:

TEXT

CPU configures DMA
!"

```
DMA transfers block
!"
DMA completion interrupt
!"
CPU handles completion
```

16.66 Why Interrupt Is Better than Polling for Slow Devices

If device becomes ready only occasionally, CPU can do useful computation instead of busy-waiting.

This usually improves utilization.

16.67 Why Interrupt May Be Poor for Extremely High Event Rates

Every interrupt may require:

- context save,
- handler execution,
- return.

Millions of interrupts per second can overwhelm CPU.

For high-throughput block I/O:

`\[ \boxed{\text{DMA is usually more appropriate}} \]`

16.68 Interrupt vs Polling Comparison

Feature	Polling	Interrupt
Who initiates attention?	CPU	Device
Busy waiting	Yes	Usually no
CPU waiting overhead	High	Lower
Event response	Depends on polling frequency	Often faster
Hardware/software complexity	Lower	Higher
Good for infrequent events	Poor	Good

16.69 Interrupt vs DMA Preview

Feature	Interrupt I/O	DMA
CPU notified	Yes	Usually at setup/completion
CPU transfers each item	Often yes	No
Main memory access	CPU-mediated	DMA controller

Feature	Interrupt I/O	DMA
Best for	Event-driven/moderate transfers	Large blocks
CPU involvement	Medium	Low

16.70 GATE Trap — Interrupt = DMA

'L Wrong.

Interrupt:

CPU stops and handles event.

DMA:

controller transfers data between device and memory.

DMA may use interrupt only for completion.

16.71 GATE Trap — Interrupt Always Occurs Immediately

Not necessarily.

CPU may:

- finish current instruction,
- have interrupts masked,
- be handling higher-priority interrupt.

Thus latency exists.

16.72 GATE Trap — Maskable Means Low Priority Always

Not strictly.

"Maskable" means it can be disabled through normal masking mechanisms.

Priority is a related but separate concept.

16.73 GATE Trap — Non-Maskable Means It Can Never Be Delayed

Not a safe universal interpretation.

NMI generally cannot be disabled by normal mask control, but hardware may still have defined recognition/priority rules.

For GATE:

NMI = critical, normally non-maskable.

16.74 GATE Trap — Vectored Means ISR Is Stored in Vector Table

Usually the vector table stores:

- ISR address,
- or information used to reach ISR.

The ISR itself is generally elsewhere.

16.75 GATE Trap — Non-Vectored Means No ISR

'L Wrong.

There is still an ISR.

CPU just does not directly obtain a unique handler from an interrupt vector in the same way.

16.76 GATE Trap — ISR Can Freely Destroy Registers

No.

Any architectural state needed by interrupted code must be preserved according to calling/interrupt conventions.

16.77 GATE Trap — LRU Cache Priority and Interrupt Priority

Completely unrelated concepts.

One selects cache victims; the other chooses interrupt service order.

16.78 GATE Trap — Software Polling Priority vs Daisy Chain

Software polling

Priority comes from order in software.

Daisy chain

Priority comes from physical acknowledgement chain.

16.79 Fast Recognition Table

Phrase	Answer
CPU repeatedly checks device	Polling
Device signals CPU	Interrupt
Handler routine	ISR
Can be disabled normally	Maskable

Phrase	Answer
Critical, not normally maskable	NMI
Handler identified using vector	Vectored
Common handler checks devices	Non-vectored / polling identification
Priority from physical chain	Daisy chain
Higher-priority ISR interrupts lower ISR	Nested interrupt

16.80 GATE Numerical — CPU Utilization

Processor:

4GHz

Interrupt rate:

$2 \times 10^5/\text{s}$

Cost:

$400\text{ cycles/interrupt}$

Interrupt cycles:

$2 \times 10^5 \times 400 = 8 \times 10^7$

Total CPU cycles:

4×10^9

Fraction:

0.02

$\boxed{2\%}$

16.81 GATE Numerical — Maximum Sustainable Rate

Suppose CPU:

2GHz

You allow maximum 10% CPU time for interrupt handling.

ISR cost:

500 cycles

Available interrupt cycles/s:

$0.10(2 \times 10^9) = 2 \times 10^8$

Maximum interrupt rate:

$\frac{2 \times 10^8}{500}$

$\boxed{400000/\text{s}}$

16.82 GATE Numerical — Service Rate

If ISR requires 2500 cycles and CPU runs at 1 GHz:

Time/interrupt:

$$\left[\frac{2500}{10^9} \text{s} \right]$$

$$\left[=2.5\mu\text{s} \right]$$

Maximum theoretical service rate:

$$\left[\frac{1}{2.5\mu\text{s}} \right]$$

$$\left[\boxed{400000 \text{ interrupts/s}} \right]$$

assuming CPU does nothing else.

16.83 GATE Numerical — Interrupts per Data Rate

Device rate:

$$\left[8\text{MB/s} \right]$$

Each interrupt handles:

$$\left[4\text{KB} \right]$$

Using binary units:

$$\left[8\text{MB}=8192\text{KB} \right]$$

Interrupts/s:

$$\left[8192/4 \right]$$

$$\left[\boxed{2048/\text{s}} \right]$$

16.84 GATE Numerical — CPU Cost of Previous Transfer

If each interrupt costs:

$$\left[2000 \text{ cycles} \right]$$

then:

$$\left[2048 \times 2000 = 4,096,000 \right]$$

cycles/s.

At 1 GHz:

$$\left[\frac{4.096 \times 10^6}{10^9} \right]$$

$$\left[=0.004096 \right]$$

$$\left[\boxed{0.4096\%} \right]$$

CPU usage.

CHAPTER 16 — FORMULA SHEET

CPU fraction due to interrupts

$$U = \frac{\text{InterruptRate} \times \text{CyclesPerInterrupt}}{\text{CPUClockRate}}$$

Percentage

$$100U$$

Maximum interrupt rate

If fraction U of CPU can be used:

$$R_{\max} = \frac{U \times \text{CPUClockRate}}{\text{CyclesPerInterrupt}}$$

Vector-table size

$$\text{NumberOfVectors} \times \text{BytesPerEntry}$$

CHAPTER 16 — MEMORY TRICKS

Polling vs interrupt

Polling = Ask

Interrupt = Alert

ISR

Interrupt !' Service !' Return

Maskable

Mask can block

NMI

Normally Must Interrupt

Vector

Vector gives direction to handler

Daisy chain

Closest in chain wins

CHAPTER SNAPSHOT

1. Interrupt lets a device request CPU attention.
 2. ISR handles the interrupt.
 3. CPU saves necessary context before servicing.
 4. PC/return address must be preserved.
 5. Interrupts are usually recognized at instruction boundaries in simplified models.
 6. Interrupt latency is delay from request to service.
 7. Hardware and software interrupts have different trigger sources.
 8. Maskable interrupts can normally be disabled.
 9. NMI is reserved for critical events.
 10. Vectored interrupts provide handler-location information.
 11. Non-vectored systems may use common handling/polling.
 12. Priority decides which simultaneous interrupt is serviced first.
 13. Daisy chaining creates priority from physical order.
 14. Parallel priority uses dedicated priority logic.
 15. Nested interrupts allow higher-priority service during lower-priority ISR.
 16. Interrupt handling consumes CPU cycles.
 17. Excessive interrupt rates can hurt performance.
 18. DMA can reduce per-item CPU interrupt overhead for bulk data.
-

MUST REMEMBER

$\boxed{\text{Interrupt = Device requests CPU attention}}$

$\boxed{\text{ISR = Interrupt Service Routine}}$

$\boxed{\text{Vectored = handler address/vector available}}$

$\boxed{\text{Maskable can be disabled; NMI generally cannot}}$

and:

$\boxed{\text{CPU Utilization} = \frac{\text{InterruptRate} \times \text{InterruptCost}}{\text{CPUClockRate}}}$

PRACTICE SET

Q1 — L0 MCQ

The routine executed to handle an interrupt is called:

A. DMA B. ISR C. Cache line D. Decoder

Q2 — L0 MCQ

Which interrupt can normally be disabled through interrupt masking?

A. Maskable interrupt B. NMI C. DMA transfer D. Cache miss

Q3 — L1 MCQ

In a vectored interrupt system:

A. CPU never runs an ISR B. interrupt provides information for locating the ISR C. device cannot have priority D. CPU must continuously poll all devices

Q4 — L1

Why is the Program Counter usually saved during interrupt handling?

Q5 — L1 MCQ

In a daisy chain:

TEXT

C P U ! ' A ! ' B ! ' C ! ' D

which device has the highest priority under normal chain ordering?

A. A B. B C. C D. D

Q6 — L2 NAT

CPU = 1 GHz.

Interrupt rate = 20,000/s.

Cost = 500 cycles/interrupt.

What percentage of CPU cycles are spent on interrupt handling?

Q7 — L2 NAT

A CPU runs at 2 GHz.

Each ISR costs 1000 cycles.

If no more than 5% CPU time should be used for this interrupt source, what is the maximum interrupt rate per second?

Q8 — L2 MSQ

Which statements are generally true?

A. Interrupt-driven I/O reduces continuous busy waiting. B. Interrupt handling has zero overhead. C. CPU may need to save context. D. DMA and interrupts are identical.

Q9 — L2 MSQ

Which statements about vectored interrupts are generally true?

A. They can reduce source-identification overhead. B. They provide information for reaching the appropriate ISR. C. They eliminate the need for an ISR. D. Different sources may have different vectors.

Q10 — L2

Explain the difference between:

- maskable interrupt,
- non-maskable interrupt.

Q11 — L2

Priorities are:

$P_1 > P_2 > P_3$

CPU is servicing P_3 . P_2 arrives, then P_1 arrives. Assume full priority nesting is enabled.

Write the service/resume order.

Q12 — L3 NAT

A device transfers 16 MB/s. It generates one interrupt per 8 KB block.

How many interrupts per second occur? Use binary KB/MB.

Q13 — L3 NAT

For Q12, if each interrupt costs 2500 CPU cycles and CPU frequency is 2 GHz, calculate CPU percentage spent on interrupt handling.

Q14 — L3

A vector table contains 256 entries, each 8 bytes.

Find table size in KB.

CHAPTER 16 COMPLETION STATUS

' Interrupt concept ' ISR ' Interrupt cycle ' Context saving/restoration ' Interrupt latency ' Hardware/software interrupts ' Maskable interrupts ' Non-maskable interrupts ' Interrupt-enable concept ' Vectored interrupts ' Non-vectored interrupts ' Interrupt vector table ' Interrupt priority ' Fixed/dynamic priority ' Daisy chaining ' Parallel priority ' Nested interrupts ' Software polling for interrupt identification ' CPU-utilization numericals ' Vector-table numericals ' GATE traps

CHAPTER 16: COMPLETE '

COURSE PROGRESS

Completed: 17 / 26 modules

Approximate roadmap progress:

$\boxed{\sim 65\%}$

NEXT !' CHAPTER 17 — DMA (DIRECT MEMORY ACCESS)

Next we finish the official I/O block with:

DMA controller CPU–DMA–memory interaction DMA registers bus arbitration cycle stealing burst/block mode transparent DMA CPU slowdown due to DMA transfer-rate numericals interrupt vs DMA vs polling comparison GATE DMA numerical traps.

USER

next

ASSISTANT

CHAPTER 17 — DMA (DIRECT MEMORY ACCESS)

Classification: [GATE CORE] **Priority:** VERY HIGH **Official syllabus connection:** I/O interface — DMA

DMA is used when transferring large amounts of data between an I/O device and memory without forcing the CPU to move every individual byte/word.

Core idea:

$\boxed{\text{DMA} = \text{Direct Memory Access}}$

17.1 Why Do We Need DMA?

Suppose a disk needs to transfer:

1 MB

to main memory.

If CPU manually transfers every word:

TEXT

Read device

!"

Move data to CPU register

!"

Write memory

!"

Repeat thousands of times

CPU wastes a huge amount of time doing data movement.

Instead:

TEXT

```
CPU configures DMA
!"
DMA controller transfers data
D e v i c e !" M e m o r y
!"
CPU does other work
!"
DMA interrupts CPU when finished
```

This reduces CPU involvement.

17.2 DMA Architecture

Basic structure:

TEXT

```

% % % % % % % % % % %
%           C P U           %
% % % % % % % , % % % % % %
%
%           S y s t e m   B u s
% % % % % % % % % % % % % %
%   D M A   C o n t r o l l e r %
% % % % % % % , % % % % % % %
%
% % % % % % % % % % % % % %
!"                               !"
Main Memory   I/O Device
```

The DMA controller can become a bus master temporarily.

17.3 What Does "Direct" Mean?

Without DMA:

TEXT

```
D e v i c e !'   C P U !'   M e m o r y
```

With DMA:

TEXT

```
D e v i c e !'   M e m o r y
```

or:

TEXT

```
M e m o r y !'   D e v i c e
```

without CPU copying each data item.

The CPU still controls setup and completion.

17.4 CPU's Role in DMA

The CPU usually initializes DMA with:

1. starting memory address,
2. transfer count,
3. transfer direction,
4. device/channel information,
5. control information.

Then DMA performs the transfer.

17.5 DMA Registers

A DMA controller commonly has:

Address Register

Stores current memory address.

Count Register

Stores number of bytes/words remaining.

Control Register

Stores information such as:

- read/write direction,
- transfer mode,
- enable,
- device selection.

Memory trick:

A-C-C

Address !' Count !' Control

17.6 Example Setup

Suppose CPU wants disk to transfer 1024 bytes into memory starting at:

TEXT
0x4000

CPU may configure:

TEXT
DMA Address = 0x4000
DMA Count = 1024
Direction = Device !' Memory
Start = 1

Then DMA controller handles the transfer.

17.7 What Happens After Each Transfer?

Conceptually:

TEXT

```
Memory Address !• Memory Address + transfer size
Count          !• Count - 1
```

When:

$\lceil \text{Count} = 0 \rceil$

transfer is complete.

DMA may then:

$\lceil \boxed{\text{Interrupt CPU}} \rceil$

17.8 DMA Transfer Sequence

Typical flow:

TEXT

1. CPU programs DMA controller
2. Device becomes ready
3. DMA requests bus
4. CPU/bus arbiter grants bus
5. DMA transfers data
6. DMA updates address and count
7. Repeat until complete
8. DMA interrupts CPU

17.9 Bus Master

Normally CPU controls the system bus.

During DMA:

DMA controller may temporarily control the bus.

A component currently controlling bus transactions is often called the:

$\lceil \boxed{\text{Bus Master}} \rceil$

17.10 Bus Arbitration

What happens if CPU and DMA both want the bus?

Some mechanism must decide who gets control.

This is called:

17.11 DMA Request and Grant

Conceptually:

TEXT

```
D M A ! ' B u s R e q u e s t ! ' C P U / A r b i t e r
C P U ! ' B u s G r a n t ! ' D M A
```

Then DMA performs memory transfer.

Exact signal names vary.

17.12 CPU and DMA Competition

CPU needs memory for:

- instruction fetch,
- load/store.

DMA also needs memory.

Therefore DMA can temporarily reduce memory bandwidth available to CPU.

This can cause CPU slowdown.

17.13 DMA Transfer Modes

Three important modes:

1. Burst / Block Mode
2. Cycle Stealing
3. TransparentDMA

17.14 Burst Mode

In **Burst Mode**, DMA obtains the bus and transfers an entire block or many consecutive data units before releasing it.

Conceptually:

TEXT

```
DMA gets bus
!"
Transfer
Transfer
Transfer
Transfer
...
!"
Release bus
```


17.15 Burst Mode Advantage

Very efficient for high-throughput data transfer.

Less bus-arbitration overhead per transferred item.

17.16 Burst Mode Disadvantage

CPU may be blocked from memory/bus for a relatively long interval.

Therefore:

$\boxed{\text{High DMA throughput, potentially larger CPU delay}}$

17.17 Cycle Stealing

Not important.

In **Cycle Stealing**, DMA takes the bus for a small number of memory cycles, often one transfer at a time, then releases it.

Conceptually:

TEXT

```
CPU cycle
DMA cycle
CPU cycle
CPU cycle
DMA cycle
...
```

DMA "steals" memory/bus cycles from CPU.

17.18 Why Called Cycle Stealing?

Because the DMA controller temporarily uses a cycle that CPU could otherwise have used.

CPU may have to wait.

Hence:

$\boxed{\text{DMA steals memory/bus cycles}}$

17.19 Cycle Stealing Advantage

CPU is not blocked for a long continuous burst.

Interleaving can make system more responsive.

17.20 Cycle Stealing Disadvantage

CPU repeatedly loses individual bus cycles.

Overall CPU performance may decrease.

17.21 Transparent DMA

In **Transparent DMA**, DMA transfers only when CPU is not using the bus.

Conceptually:

TEXT

CPU needs bus !' CPU uses it
CPU idle from bus !' DMA uses it

17.22 Transparent DMA Advantage

Minimal interference with CPU.

17.23 Transparent DMA Disadvantage

DMA transfer may take longer because it must wait for idle bus periods.

17.24 DMA Modes Comparison

Mode	DMA Behavior	CPU Impact	DMA Speed
Burst	Holds bus for block	High temporary blocking	High
Cycle stealing	Takes individual cycles	Moderate repeated slowdown	Medium/high
Transparent	Uses bus only when CPU doesn't	Lowest interference	Potentially slower

Memory trick:

- Burst = Block
- Steal = Share by taking cycles
- Transparent = Take only idle time

17.25 DMA vs Interrupt-Driven I/O

Interrupt-driven I/O:

TEXT

```
Device ready
!"
Interrupt CPU
!"
CPU runs ISR
!"
CPU transfers data
```

DMA:

TEXT

```
CPU configures DMA
!"
DMA transfers data
!"
Completion interrupt
```

Key distinction:

$\boxed{\text{Interrupt-driven I/O still often uses CPU for data movement}}$

$\boxed{\text{DMA performs bulk data movement itself}}$

17.26 Polling vs Interrupt vs DMA

Feature	Polling	Interrupt	DMA
CPU waits/checks	High	Low	Low
CPU transfers each data item	Yes	Usually yes	No
Hardware complexity	Low	Medium	Higher
Best for bulk transfer	Poor	Moderate	Excellent
CPU involvement	Highest	Medium	Lowest

17.27 Important Hierarchy

Memory trick:

P-I-D

TEXT

```
P o l l i n g      !'   C P U   c h e c k s
I n t e r r u p t !'   C P U   s e r v i c e s
D M A              !'   C o n t r o l l e r   t r a n s f e r s
```

17.28 DMA Read vs DMA Write

Terminology can be confusing.

A DMA transfer may be:

Device !' Memory

Memory is being written.

Memory !' Device

Memory is being read.

Always identify the actual data direction rather than relying blindly on the phrase "DMA read/write."

17.29 Device-to-Memory Example

Suppose network card receives a packet.

DMA can copy packet:

TEXT

```
Network Controller
      ! "
      Memory
```

CPU later processes it from memory.

17.30 Memory-to-Device Example

Suppose disk/controller needs data to write externally.

DMA reads main memory and sends block to the device.

TEXT

```
Memory
      ! "
DMA
      ! "
Device
```

17.31 DMA Address Increment

Suppose memory starts at:

\[1000 \]

and DMA transfers 4-byte words.

After each word:

TEXT

```
1000
1004
1008
1012
```

...

So increment depends on transfer size and addressing convention.

17.32 DMA Count Register

Suppose block contains:

$\lfloor 256 \rfloor$

words.

Count starts:

$\lfloor 256 \rfloor$

After each word transfer:

$\lfloor \text{Count} \leftarrow \text{Count} - 1 \rfloor$

After final transfer:

$\lfloor \text{Count} = 0 \rfloor$

Transfer complete.

17.33 GATE Numerical — Transfer Count

Block size:

$\lfloor 8\text{KB} \rfloor$

Each DMA transfer:

$\lfloor 4\text{B} \rfloor$

Number of transfers:

$\lfloor \frac{8\text{KB}}{4\text{B}} = \frac{8192}{4} \rfloor$

$\lfloor \boxed{2048} \rfloor$

17.34 Transfer Rate

If DMA transfers N bytes in time T :

$\lfloor \boxed{\text{TransferRate} = \frac{N}{T}} \rfloor$

Example:

1 MB transferred in 10 ms:

Using decimal units:

$\lfloor \frac{10^6}{0.01} = 10^8\text{B/s} \rfloor$

$\lfloor \boxed{100\text{MB/s}} \rfloor$

17.35 DMA Time Numerical

Device rate:

$\backslash[50\text{MB/s} \backslash]$

File size:

$\backslash[100\text{MB} \backslash]$

Ideal transfer time:

$\backslash[\frac{100}{50} \backslash]$

$\backslash[\boxed{2\text{s}} \backslash]$

ignoring setup/bus constraints.

17.36 Bus Cycle Numerical

Suppose:

- one DMA transfer moves 4 bytes,
- one bus cycle = 50 ns.

Maximum transfer rate:

$\backslash[\frac{4\text{B}}{50\text{ns}} \backslash]$

$\backslash[=80\times 10^6\text{B/s} \backslash]$

$\backslash[\boxed{80\text{MB/s}} \backslash]$

using decimal MB/s.

17.37 Another Bus Bandwidth Example

Bus width:

$\backslash[64\text{ bits}=8\text{B} \backslash]$

Bus performs one transfer every:

$\backslash[10\text{ns} \backslash]$

Bandwidth:

$\backslash[\frac{8}{10\text{ns}} = 0.8\times 10^9\text{B/s} \backslash]$

$\backslash[\boxed{800\text{MB/s}} \backslash]$

17.38 CPU Slowdown Due to Cycle Stealing

Suppose CPU normally uses memory bus for all cycles.

DMA steals:

$\backslash[10\% \backslash]$

of available memory cycles.

In a simple model, CPU may lose roughly:

$$\boxed{10\%}$$

of those bus opportunities.

Exact CPU slowdown depends on whether CPU actually needed every stolen cycle.

Read question assumptions carefully.

17.39 GATE Numerical — DMA Cycle Stealing

CPU memory bus supports:

$$10^8$$

cycles/s.

DMA needs:

$$2 \times 10^7$$

cycles/s.

Fraction stolen:

$$\frac{2 \times 10^7}{10^8}$$

$$= 0.2$$

$$\boxed{20\%}$$

of bus cycles.

17.40 CPU Utilization Under Simplified Assumption

If CPU is assumed to require bus continuously and DMA steals 20% cycles:

CPU gets:

$$80\%$$

of bus cycles.

Under the simplified memory-limited model:

$$\boxed{\text{CPU effective bus utilization} = 80\%}$$

Do not generalize this to overall processor performance unless stated.

17.41 DMA Block + Completion Interrupt

Suppose DMA transfers:

$$4096B$$

per block.

Device rate:

$$40MB/s$$

Using decimal MB:

Blocks/sec:

$$\left[\frac{40 \times 10^6}{4096} \right]$$

$$\left[\approx 9765.625 \right]$$

So about:

$$\left[\boxed{9766 \text{ completion interrupts/s}} \right]$$

if one interrupt per block.

17.42 Compare One-Byte Interrupts

If every byte generated one interrupt:

$$\left[40 \times 10^6 \right]$$

interrupts/s.

With 4096-byte DMA blocks:

$$\left[\approx 9766 \right]$$

interrupts/s.

This is a dramatic reduction in CPU overhead.

17.43 DMA Setup Overhead

DMA is not free.

CPU must spend some cycles programming DMA.

Suppose setup cost:

$$\left[1000 \text{ cycles} \right]$$

and DMA transfers 1 MB.

That setup cost is tiny per byte.

But if each DMA block were only 4 bytes, setup overhead could dominate.

Thus DMA is especially useful for:

$$\left[\boxed{\text{large/bulk transfers}} \right]$$

17.44 GATE Numerical — CPU Setup Fraction

CPU:

$$\left[2 \text{ GHz} \right]$$

DMA performs:

$$\left[10000 \right]$$

blocks/s.

Each setup + completion handling costs:

$$\lfloor 500 \rfloor$$

CPU cycles.

CPU cycles used:

$$\lfloor 10000 \times 500 = 5 \times 10^6 \rfloor$$

Fraction:

$$\lfloor \frac{5 \times 10^6}{2 \times 10^9} \rfloor$$

$$\lfloor = 0.0025 \rfloor$$

$$\lfloor \boxed{0.25\%} \rfloor$$

17.45 Cycle Stealing Transfer Rate

Suppose bus cycle:

$$\lfloor 20\text{ns} \rfloor$$

DMA steals one cycle every:

$$\lfloor 10 \rfloor$$

bus cycles.

Each DMA cycle transfers:

$$\lfloor 4\text{B} \rfloor$$

Bus cycles/sec:

$$\lfloor \frac{1}{20\text{ns}} = 50 \times 10^6 \rfloor$$

DMA cycles/sec:

$$\lfloor 5 \times 10^6 \rfloor$$

Data rate:

$$\lfloor 5 \times 10^6 \times 4 \rfloor$$

$$\lfloor \boxed{20\text{MB/s}} \rfloor$$

17.46 GATE Shortcut for Cycle-Stealing Rate

If DMA steals fraction f of bus cycles:

$$\lfloor \boxed{\text{DMA Bandwidth} = f \times \text{BusCyclesPerSecond} \times \text{BytesPerCycle}} \rfloor$$

17.47 Example

Bus:

- 100 million cycles/s
 - 8 B/cycle
 - DMA gets 25% cycles
-

DMA bandwidth:

$$\left[0.25(100 \times 10^6)(8) \right]$$

$$\left[=200 \times 10^6 \text{B/s} \right]$$

$$\left[\boxed{200 \text{MB/s}} \right]$$

17.48 Burst Transfer Numerical

Suppose DMA receives bus and transfers 16 words.

Each word transfer:

$$\left[20 \text{ns} \right]$$

Ignore arbitration overhead.

Burst duration:

$$\left[16 \times 20 \right]$$

$$\left[\boxed{320 \text{ns}} \right]$$

During this interval CPU may be unable to use bus, depending on architecture.

17.49 Add Arbitration Overhead

Suppose:

- bus request/grant overhead = 40 ns,
- 16 transfers $\times$ 20 ns.

Total DMA burst:

$$\left[40 + 16(20) \right]$$

$$\left[40 + 320 \right]$$

$$\left[\boxed{360 \text{ns}} \right]$$

17.50 Effective Burst Bandwidth

16 words.

Each word:

$$\left[4 \text{B} \right]$$

Total:

$$\left[64 \text{B} \right]$$

Time:

$$\left[360 \text{ns} \right]$$

Bandwidth:

$$\left[\frac{64}{360 \text{ns}} \right]$$

$$\left[\approx 177.78 \times 10^6 \text{B/s} \right]$$

$\approx 177.8 \text{ MB/s}$

17.51 Why Bigger Bursts Can Improve Efficiency

If arbitration overhead occurs once per burst:

OverheadPerByte

decreases as burst size increases.

But large bursts block CPU for longer.

Trade-off:

throughput vs CPU responsiveness.

17.52 DMA and Cache Coherence — Support

[PLACEMENT/INTERVIEW / SUPPORT]

Suppose DMA modifies main memory directly while CPU cache holds an old copy.

Potential problem:

TEXT

Cache X = old value

Memory X = DMA-updated value

Systems need mechanisms to maintain coherence, such as:

- cache invalidation,
- flushing,
- coherent interconnects.

This is useful practical knowledge but not a major focus of your listed syllabus.

17.53 DMA and Virtual Memory — Low Priority

[LOW PRIORITY]

Real systems may need address translation/IOMMU support for DMA.

Not required for this GATE syllabus.

Do not spend time here now.

17.54 DMA Controller vs I/O Controller

An I/O controller handles device-specific operations.

DMA controller handles direct block movement involving memory.

Sometimes both functions may exist in the same hardware device/controller.

For GATE, distinguish concepts rather than assuming a specific physical layout.

17.55 Bus Arbitration Types — Support

Possible schemes include:

- centralized arbitration,
- distributed arbitration,
- fixed priority,
- rotating priority.

Detailed bus-arbitration design is not explicitly in your syllabus.

Know only what is needed to understand DMA ownership of the bus.

17.56 CPU Halt During Burst

In burst mode, CPU may be unable to access the system bus during DMA ownership.

However, CPU may still execute operations that do not need that bus depending on processor design.

For simplified GATE problems, follow stated assumptions.

17.57 Cycle Stealing Does Not Mean Whole CPU Cycle Always Lost

It specifically means DMA gets a bus/memory cycle.

Whether CPU completely stalls depends on whether CPU needed the shared resource.

Important wording distinction.

17.58 Transparent DMA Example

Suppose CPU uses bus:

TEXT

```
C y c l e   1   !'   y e s
C y c l e   2   !'   n o
C y c l e   3   !'   y e s
C y c l e   4   !'   n o
```

Transparent DMA may use:

TEXT

```
Cycle 2
Cycle 4
```

CPU's bus requests are not delayed.

17.59 Transparent DMA Limitation

If CPU uses bus almost continuously:

DMA gets very few opportunities.

Therefore DMA throughput may become poor.

17.60 DMA Request from Device

Typical conceptual sequence:

TEXT

```
Device needs transfer
!"
Device requests DMA service
!"
DMA requests system bus
!"
DMA receives bus
!"
Transfer occurs
```

Some systems have dedicated DMA request/acknowledge signals.

17.61 DMA Completion

After transfer:

1. DMA count reaches zero.
2. DMA releases bus.
3. Status indicates completion.
4. DMA may send interrupt.

CPU can then process completed data or issue next operation.

17.62 Polling DMA Completion

Although DMA commonly interrupts CPU on completion, CPU could theoretically poll DMA status instead.

Again:

DMA and interrupts solve different aspects of the problem.

DMA = data movement.

Interrupt = notification.

17.63 GATE Trap — DMA Transfers Instructions

DMA generally transfers data blocks between I/O and memory.

It is not executing CPU program instructions.

17.64 GATE Trap — DMA Never Uses CPU

Wrong.

CPU generally performs:

- setup,
- control,
- completion handling.

Correct statement:

`\[ \boxed{\text{DMA greatly reduces CPU involvement}} \]`

17.65 GATE Trap — DMA Means Device Directly Controls RAM

Usually DMA controller/device's DMA engine coordinates access via bus arbitration.

Do not interpret "direct" as "no control mechanism."

17.66 GATE Trap — DMA Removes Interrupts Entirely

Wrong.

DMA often uses an interrupt at block completion.

17.67 GATE Trap — Cycle Stealing Means DMA Takes Entire Block at Once

Wrong.

That describes burst/block mode more closely.

Cycle stealing takes smaller bus opportunities, often one transfer/cycle at a time.

17.68 GATE Trap — Burst Mode Causes No CPU Interference

Wrong.

Burst mode can block CPU bus access for the burst duration.

17.69 GATE Trap — Transparent DMA Is Fastest

Not necessarily.

It minimizes CPU interference but may have fewer opportunities to transfer.

Burst mode often has higher raw transfer efficiency.

17.70 GATE Trap — DMA Memory Address Stays Constant

Normally address increments during sequential memory transfer.

But some special device/transfer modes may differ.

Use question assumptions.

17.71 GATE Trap — Count Always Means Bytes

Count could represent:

- bytes,
- words,
- transfers.

Always check unit.

17.72 GATE Trap — Bus Bandwidth and Device Bandwidth Are Same

Actual transfer rate is constrained by the bottleneck.

If:

$\text{Device} = 50\text{MB/s}$

and:

$\text{Bus} = 200\text{MB/s}$

maximum transfer cannot exceed:

$\boxed{50\text{MB/s}}$

ignoring other effects.

17.73 Bottleneck Formula

Ideal maximum throughput:

$\boxed{\text{Throughput} = \min(\text{DeviceRate}, \text{BusRate}, \text{MemoryRate})}$

Useful conceptually.

17.74 GATE Numerical — Bottleneck

Device:

80MB/s

Bus:

120MB/s

Memory:

100MB/s

Maximum sustained rate:

$$\boxed{80\text{MB/s}}$$

because device is slowest.

17.75 DMA Transfer Time with Setup

Suppose:

- setup = 10^{-4} s
- transfer size = 1 MB
- DMA transfer rate = 100 MB/s

Transfer time:

$$\frac{1}{100}\text{s} = 0.01\text{s} = 10\text{ms}$$

Total:

$$10\text{ms} + 10^{-4}\text{s}$$

$$= 10.01\text{ms}$$

$$\boxed{10.01\text{ms}}$$

using decimal MB.

17.76 Small Block Problem

Suppose:

- setup = 10^{-4} s
- transfer = only 100 bytes
- bus rate = 100 MB/s

Data transfer itself:

$$\frac{100}{100 \times 10^6} = 1\mu\text{s}$$

Total:

$$11\mu\text{s}$$

Setup dominates.

Thus DMA is less attractive for tiny transfers.

17.77 Polling vs Interrupt vs DMA — Final Comparison

Property	Polling	Interrupt I/O	DMA
Device-ready detection	CPU checks	Device interrupts	DMA/device coordination
CPU waits actively	Yes	No	No
CPU moves each item	Yes	Usually yes	No
Bulk efficiency	Low	Medium	High

Property	Polling	Interrupt I/O	DMA
Hardware complexity	Low	Medium	High
Completion notification	CPU sees status	Interrupt	Often interrupt
Best use	Very simple/short waits	Sporadic events	Large/high-rate transfers

17.78 Fast Recognition Rules

If you see:

CPU repeatedly checks status

$\boxed{\text{Polling}}$

If you see:

Device requests CPU service

$\boxed{\text{Interrupt}}$

If you see:

Device !”memory block transfer without CPU moving each item

$\boxed{\text{DMA}}$

If you see:

DMA temporarily takes one memory cycle

$\boxed{\text{Cycle\ Stealing}}$

If you see:

DMA holds bus for an entire block

$\boxed{\text{Burst\ Mode}}$

If you see:

DMA uses bus only when CPU is idle

$\boxed{\text{Transparent\ DMA}}$

17.79 GATE Numerical — Block Count

A DMA controller transfers:

$\boxed{64\text{KB}}$

using:

$\lfloor 32 \text{ text{-bit words}} \rfloor$

Each word:

$\lfloor 4B \rfloor$

Number of DMA word transfers:

$\lfloor \frac{64 \times 1024}{4} \rfloor$

$\lfloor \boxed{16384} \rfloor$

17.80 GATE Numerical — Final Address

Starting address:

$\lfloor 1000 \rfloor$

DMA transfers 100 words.

Each word = 4 bytes.

Assume byte addressing and address register increments after each word.

After 100 transfers, next address is:

$\lfloor 1000 + 100(4) \rfloor$

$\lfloor \boxed{1400} \rfloor$

Last transferred word begins at:

$\lfloor 1000 + 99(4) \rfloor$

$\lfloor \boxed{1396} \rfloor$

Be careful whether question asks last used address or next address.

17.81 GATE Numerical — Count Register

If count initially represents number of transfers and begins at:

$\lfloor 500 \rfloor$

after 123 transfers:

$\lfloor 500 - 123 \rfloor$

$\lfloor \boxed{377} \rfloor$

17.82 GATE Numerical — Cycle Stealing

Memory bus supports:

$\lfloor 50 \times 10^6 \rfloor$

transfers/s.

DMA requires:

$$\left[5 \times 10^6 \right]$$

transfers/s.

Fraction stolen:

$$\left[5/50 \right]$$

$$\left[\boxed{10\%} \right]$$

17.83 GATE Numerical — DMA Bandwidth

Each DMA transfer moves:

$$\left[8B \right]$$

DMA gets:

$$\left[10^7 \right]$$

bus cycles/sec.

Bandwidth:

$$\left[8 \times 10^7 B/s \right]$$

$$\left[\boxed{80MB/s} \right]$$

17.84 GATE Numerical — Burst Efficiency

Each burst:

- arbitration overhead = 100 ns
- 20 transfers
- each transfer = 10 ns
- 4 bytes/transfer.

Data:

$$\left[20 \times 4 = 80B \right]$$

Time:

$$\left[100 + 20(10) \right]$$

$$\left[= 300ns \right]$$

Bandwidth:

$$\left[\frac{80}{300ns} \right]$$

$$\left[\approx 266.67MB/s \right]$$

$$\left[\boxed{\approx 266.7MB/s} \right]$$

CHAPTER 17 — FORMULA SHEET

Number of transfers

$$\left[\boxed{N = \frac{\text{TotalData}}{\text{DataPerTransfer}}} \right]$$

Transfer rate

$$\boxed{\text{Rate} = \frac{\text{Data}}{\text{Time}}}$$

Cycle-stealing fraction

$$\boxed{f = \frac{\text{DMA Bus Cycles}}{\text{Total Bus Cycles}}}$$

DMA bandwidth from stolen cycles

$$\boxed{\text{BW} = f \times \text{Bus Cycles/s} \times \text{Bytes/Cycle}}$$

Burst time

$$\boxed{T_{\text{burst}} = \text{Arbitration Overhead} + N \times \text{Transfer Time}}$$

CHAPTER 17 — MEMORY TRICKS

DMA registers

A-C-C

Address Count Control

DMA modes

B-C-T

Burst Cycle stealing Transparent

CPU involvement

Polling > Interrupt > DMA

DMA lifecycle

Setup ! Transfer ! Interrupt

CHAPTER SNAPSHOT

1. DMA transfers data between I/O and memory with little CPU involvement.
2. CPU configures DMA before transfer.
3. DMA commonly uses address, count and control registers.
4. DMA temporarily needs control of the memory/system bus.
5. Bus arbitration resolves CPU–DMA bus competition.
6. Burst mode transfers many items before releasing the bus.
7. Cycle stealing takes individual bus cycles.
8. Transparent DMA uses bus only when CPU does not.
9. DMA can transfer device-to-memory or memory-to-device.
10. DMA updates address/count during transfer.

11. Completion often produces an interrupt.
 12. DMA greatly reduces per-item CPU overhead.
 13. DMA does not eliminate CPU involvement entirely.
 14. Cycle stealing can slow CPU by consuming shared bus cycles.
 15. Burst mode can maximize transfer efficiency but block CPU longer.
 16. Transparent DMA minimizes CPU interference but may transfer more slowly.
 17. Actual throughput is limited by the slowest component.
-

MUST REMEMBER

$\boxed{\text{DMA = Device/Controller ! " Memory"}}$

$\boxed{\text{CPU mainly performs setup + completion handling}}$

$\boxed{\text{Cycle stealing = DMA steals bus cycles}}$

$\boxed{\text{Burst = DMA holds bus for many transfers}}$

$\boxed{\text{Transparent = DMA uses idle bus cycles}}$

PRACTICE SET

Q1 — L0 MCQ

The main purpose of DMA is to:

- A. Increase cache tag size B. Allow I/O-memory transfer with reduced CPU involvement C. Replace interrupts completely D. Execute ALU operations
-

Q2 — L0 MCQ

Which DMA register stores the number of transfers remaining?

- A. Address register B. Count register C. PC D. IR
-

Q3 — L1 MCQ

In cycle-stealing DMA:

- A. DMA permanently owns the bus B. DMA takes selected bus/memory cycles C. DMA never affects CPU D. CPU moves every data item
-

Q4 — L1 MCQ

Which DMA mode usually holds the bus for a series of consecutive transfers?

- A. Transparent B. Burst C. Polling D. Interrupt-only
-

Q5 — L1 NAT

A DMA block is 16 KB and each transfer moves 4 bytes.

How many transfers are required?

Q6 — L2 NAT

A bus can perform 10^8 transfers/s. DMA receives 20% of bus cycles and transfers 8 bytes/cycle.

Find DMA bandwidth in MB/s using decimal MB.

Q7 — L2 NAT

A DMA controller transfers 64 words beginning at byte address 2000. Each word is 4 bytes.

Find the next memory address after the transfer completes.

Q8 — L2 MSQ

Which are generally true for DMA?

A. CPU normally configures the transfer. B. DMA may become bus master. C. CPU must move every byte through a general-purpose register. D. DMA may interrupt CPU when block transfer completes.

Q9 — L2 MSQ

Which statements are generally true?

A. Burst DMA may block CPU bus access for longer continuous intervals. B. Cycle stealing takes some bus cycles from CPU. C. Transparent DMA normally uses bus only when CPU is not using it. D. Transparent DMA must always have the highest throughput.

Q10 — L2 NAT

A DMA burst transfers 32 words. Each word transfer requires 20 ns. Bus arbitration overhead is 100 ns.

Find total burst time.

Q11 — L3 NAT

A CPU runs at 2 GHz. DMA completion occurs 20,000 times/s. CPU spends 800 cycles handling setup/completion per block.

What percentage of CPU cycles is consumed?

Q12 — L3 NAT

A DMA engine transfers 8-byte words. Bus cycle = 25 ns. DMA steals one out of every five bus cycles.

Find ideal DMA bandwidth in MB/s.

Q13 — L3

Explain in one line each:

- Polling
 - Interrupt-driven I/O
 - DMA
 - Burst DMA
 - Cycle stealing
 - TransparentDMA
-

CHAPTER 17 COMPLETION STATUS

' DMA motivation ' DMA controller ' Address/count/control registers ' CPU setup ' Device !" memory transfer ' Bus master concept ' Bus arbitration ' Burst/block DMA ' Cycle stealing ' TransparentDMA ' DMA completion interrupts ' DMA vs polling ' DMA vs interrupts ' CPU slowdown concepts ' Transfer-count numericals ' Bandwidth numericals ' Burst-overhead numericals ' Bottleneck concept ' GATE traps

CHAPTER 17: COMPLETE '

OFFICIAL I/O SYLLABUS STATUS

The official phrase:

"I/O interface (interrupt and DMA)"

is now covered:

' I/O interface ' Programmed I/O / polling ' Memory-mapped vs isolated I/O ' Interrupts ' Interrupt priority / vectored handling ' DMA ' DMA modes and performance

COURSE PROGRESS

Completed: 18 / 26 modules

Approximate roadmap progress:

$\sim 69\%$

NEXT !' CHAPTER 18 — INSTRUCTION PIPELINING BASICS

Next begins the final major official COA block:

What pipelining means IF-ID-EX-MEM-WB stages $\text{non-pipelined vs pipelined execution}$ $\text{latency vs throughput}$ $\text{pipeline timing diagrams}$ ideal speedup $\text{flush and drain time}$ $\text{(K)-stage pipeline formulas}$ GATE numericals

After that we will move into **pipeline hazards**, one of the highest-yield COA topics.

USER

next

ASSISTANT

CHAPTER 18 — INSTRUCTION PIPELINING BASICS

Classification: [GATE CORE] **Priority:** ~~0~~ = VERY HIGH **Official syllabus connection:** Instruction pipelining

This is one of the most important COA topics for GATE.

Core idea:

Instead of completing one instruction fully before starting the next, different instructions are allowed to overlap in different stages.

18.1 Why Do We Need Pipelining?

Suppose every instruction needs 5 steps:

TEXT

1. Fetch instruction
2. Decode instruction
3. Execute
4. Access memory
5. Write result

Without pipelining:

TEXT

```
I n s t r u c t i o n   1   ! '   a l l   5   s t a g e s
I n s t r u c t i o n   2   ! '   a l l   5   s t a g e s
I n s t r u c t i o n   3   ! '   a l l   5   s t a g e s
```

Only one instruction progresses at a time.

This wastes hardware because while ALU is working, instruction-fetch hardware may be idle.

Pipelining tries to use different hardware stages simultaneously.

18.2 Assembly-Line Intuition

Imagine a car factory.

A car goes through:

TEXT

```
S t a g e   1   ! '   F r a m e
S t a g e   2   ! '   E n g i n e
S t a g e   3   ! '   P a i n t
```


Without pipelining:

Finish the entire first car before starting the second.

With pipelining:

TEXT

```
C a r   1   !'   P a i n t
C a r   2   !'   E n g i n e
C a r   3   !'   F r a m e
```

Several cars are being processed at once.

CPU pipelining works similarly.

18.3 Classic 5-Stage Instruction Pipeline

A commonly used simplified pipeline is:

$\boxed{IF \rightarrow ID \rightarrow EX \rightarrow MEM \rightarrow WB}$

IF — Instruction Fetch

Fetch instruction from memory.

ID — Instruction Decode

Decode opcode and usually read registers.

EX — Execute

ALU operation / effective-address calculation / branch comparison depending on design.

MEM — Memory Access

Read/write data memory if required.

WB — Write Back

Write result into register file.

Memory Trick

TEXT

```
I  !'  I F
D  !'  I D
E  !'  E X
M  !'  M E M
B  !'  W B
```

18.4 Non-Pipelined Execution

Suppose each instruction takes:

5 stages.

Each stage requires:

1 ns

Without pipelining:

One instruction:

5 ns

For 4 instructions:

20 ns

Timeline:

TEXT

```
Time !' 1 2 3 4 5 6 7 8 9 10 ...
I1 IF ID EX MEM WB
I2 IF ID EX MEM WB
I3 ...
```

18.5 Pipelined Execution

Now overlap them:

TEXT

```
Cycle !' 1 2 3 4 5 6 7 8
I1 IF ID EX MEM WB
I2 IF ID EX MEM WB
I3 IF ID EX MEM WB
I4 IF ID EX MEM WB
```

First instruction still needs 5 cycles to finish.

But after pipeline fills:

ideally one instruction completes every cycle.

18.6 Pipeline Latency

Latency means:

time taken by one instruction from entry to completion.

For a 5-stage ideal pipeline:

$\boxed{\text{Latency} = 5 \text{ cycles}}$

if each stage takes one cycle.

Pipelining does not necessarily reduce individual instruction latency.

It mainly improves:

$\boxed{\text{Throughput}}$

18.7 Pipeline Throughput

Throughput means:

number of instructions completed per unit time.

After pipeline fills, ideal pipeline may complete:

$\boxed{1 \text{ instruction/cycle}}$

So ideal throughput:

$\boxed{\frac{1}{T_{\text{clock}}}}$

instructions per second.

18.8 Latency vs Throughput

Very important.

Latency

Time for one instruction.

Throughput

How many instructions finish per second.

Pipelining mainly improves:

$\boxed{\text{Throughput}}$

not necessarily latency.

18.9 k-Stage Pipeline

Suppose there are:

k

pipeline stages.

Each stage takes one clock cycle.

For the first instruction:

k cycles

Then each additional instruction ideally finishes one cycle later.

For:

n

instructions:

$k+n-1$

cycles.

Master formula.

18.10 Why $k+n-1$?

First instruction requires:

k

cycles.

Remaining:

$n-1$

instructions each add one cycle.

Therefore:

$k+(n-1)$

$k+n-1$

18.11 Example

5-stage pipeline.

10 instructions.

Ideal cycles:

$5+10-1$

14 cycles

18.12 Non-Pipelined Time

If every instruction requires k stages of equal duration t :

$$T_{\text{nonpipe}} = nkt$$

18.13 Pipelined Time

If each pipeline stage clock is t :

$$T_{\text{pipe}} = (k+n-1)t$$

assuming:

- equal stage times,
 - no hazards,
 - no stalls,
 - no register overhead.
-

18.14 Pipeline Speedup

$$\text{Speedup} = \frac{T_{\text{nonpipe}}}{T_{\text{pipe}}}$$

For equal stages:

$$\text{Speedup} = \frac{nk}{k+n-1}$$

18.15 Example

5 stages, 10 instructions.

Non-pipelined:

$$10 \times 5 = 50$$

cycle-equivalent units.

Pipelined:

$$5 + 10 - 1 = 14$$

Speedup:

$$\frac{50}{14}$$

$$3.57 \times$$

approximately.

18.16 Maximum Ideal Speedup

For very large n :

$$\text{Speedup} = \frac{nk}{k+n-1}$$

As:

$$n \rightarrow \infty$$

$\lceil \text{Speedup} \rceil \rightarrow k$

Therefore maximum ideal speedup:

$\boxed{k}$

for a k -stage balanced pipeline.

18.17 Important Interpretation

A 5-stage pipeline does **not** mean every workload gets exactly:

$5\times$

speedup.

For finite instruction counts:

$\text{Speedup} < 5$

because pipeline must first fill.

Real hazards reduce speedup further.

18.18 Pipeline Fill Time

Before the first instruction finishes, pipeline must be filled.

For k stages:

$\boxed{k-1}$

cycles of initial fill before steady completion begins.

First completion occurs at end of cycle:

k

18.19 Pipeline Drain

At the end, after no more instructions enter, remaining instructions still need to move through later stages.

This is called:

$\boxed{\text{Pipeline Drain}}$

Fill and drain overhead are why finite-workload speedup is less than ideal k .

18.20 Timing Diagram Example

4-stage pipeline:

TEXT

Stages: A B C D

Cycle: 1 2 3 4 5 6 7

I1	A	B	C	D			
I2		A	B	C	D		
I3			A	B	C	D	
I4				A	B	C	D

For:

$$[k=4, n=4]$$

cycles:

$$[4+4-1]$$

$$[\boxed{7}]$$

18.21 Pipeline Efficiency

Efficiency measures how close achieved speedup is to maximum theoretical speedup.

$$[\boxed{\text{Efficiency} = \frac{\text{Speedup}}{k}}]$$

For ideal equal-stage pipeline:

$$[\boxed{\text{Efficiency} = \frac{n}{k+n-1}}]$$

18.22 Example

5-stage pipeline, 10 instructions.

Speedup:

$$[3.57]$$

Efficiency:

$$[\frac{3.57}{5}]$$

$$[\approx 0.714]$$

$$[\boxed{71.4\%}]$$

18.23 Another Efficiency Formula

Since maximum possible active stage work is:

$$[k(k+n-1)]$$

stage-slots, and actual useful stage work:

$$[nk]$$

Efficiency:

$$[\frac{nk}{k(k+n-1)}]$$

$$[=\boxed{\frac{n}{k+n-1}}]$$

Same result.

18.24 Balanced Pipeline

A pipeline is **balanced** if all stages take approximately the same time.

Example:

TEXT

```
Stage 1 = 2 ns
Stage 2 = 2 ns
Stage 3 = 2 ns
Stage 4 = 2 ns
```

Clock can be:

$\sqrt{2\text{ns}}$

ignoring register overhead.

18.25 Unbalanced Pipeline

Suppose:

TEXT

```
Stage 1 = 1 ns
Stage 2 = 3 ns
Stage 3 = 2 ns
Stage 4 = 1.5 ns
```

All stages must use one common pipeline clock.

Clock must be at least as long as the slowest stage.

Thus:

$\boxed{T_{\text{clock}} = \max(\text{Stage Delays})}$

ignoring pipeline-register overhead.

Here:

$\boxed{T_{\text{clock}} = 3\text{ns}}$

18.26 Why Slowest Stage Controls Clock

Every stage must finish its work before next clock boundary.

If clock were:

$\sqrt{2\text{ns}}$

Stage 2 needs:

$\sqrt{3\text{ns}}$

and would fail to complete.

Therefore pipeline clock is determined by bottleneck stage.

18.27 Pipeline Register Overhead

Real pipelines place registers between stages.

These registers add overhead such as:

- setup time,
- clock-to-Q,
- skew allowance.

If pipeline register overhead is:

$$t_r$$

then simplified clock period:

$$T_{\text{clock}} = \max(t_i) + t_r$$

where t_i are stage logic delays.

18.28 Example

Stage delays:

$$2, 3, 4, 2 \text{ ns}$$

Pipeline-register overhead:

$$0.5 \text{ ns}$$

Clock:

$$4 + 0.5$$

$$\boxed{4.5 \text{ ns}}$$

18.29 Non-Pipelined Delay for Unequal Stages

Suppose instruction stages take:

$$1, 2, 3, 2 \text{ ns}$$

If non-pipelined instruction executes them sequentially:

$$T_{\text{instruction}} = 1 + 2 + 3 + 2$$

$$\boxed{8 \text{ ns}}$$

For 10 instructions:

$$80 \text{ ns}$$

18.30 Pipelined Delay for Same Stages

Slowest stage:

$$3 \text{ ns}$$

Ignoring register overhead:

$$T_{\text{clock}} = 3\text{ns}$$

For:

$$k=4, n=10$$

Pipeline cycles:

$$4+10-1=13$$

Time:

$$13(3)$$

$$\boxed{39\text{ns}}$$

Speedup:

$$80/39$$

$$\boxed{\approx 2.05\text{times}}$$

Not (4times) because stages are unbalanced.

18.31 General Pipeline Time with Unequal Stages

If stage delays are:

$$t_1, t_2, \dots, t_k$$

and register overhead is t_r :

$$\boxed{T_{\text{clock}} = \max(t_i) + t_r}$$

Then:

$$\boxed{T_{\text{pipe}} = (k+n-1)T_{\text{clock}}}$$

assuming no stalls.

18.32 General Non-Pipelined Time

If a non-pipelined instruction uses all stage operations sequentially:

$$\boxed{T_{\text{nonpipe}} = n \sum_{i=1}^k t_i}$$

ignoring other overhead.

Then speedup:

$$\boxed{S = \frac{n \sum t_i}{(k+n-1)(\max t_i + t_r)}}$$

18.33 GATE Numerical

Stages:

$$2, 3, 4, 3, 2\text{ ns}$$

Register overhead:

$$1\text{ ns}$$

Number instructions:

$$\lfloor 100 \rfloor$$

Clock:

$$\lfloor 4+1=5\text{ns} \rfloor$$

Pipeline cycles:

$$\lfloor 5+100-1=104 \rfloor$$

Time:

$$\lfloor 104(5) \rfloor$$

$$\lfloor \boxed{520\text{ns}} \rfloor$$

18.34 Non-Pipeline Time for Previous Example

One instruction:

$$\lfloor 2+3+4+3+2=14\text{ns} \rfloor$$

100:

$$\lfloor 1400\text{ns} \rfloor$$

Speedup:

$$\lfloor 1400/520 \rfloor$$

$$\lfloor \boxed{\approx 2.69\text{times}} \rfloor$$

Notice much less than 5.

18.35 Throughput of Pipeline

Once filled:

$$\lfloor \boxed{\text{Throughput} = \frac{1}{T_{\text{clock}}}} \rfloor$$

Example:

$$\lfloor T_{\text{clock}} = 2\text{ns} \rfloor$$

Throughput:

$$\lfloor \frac{1}{2\text{ns}} = 500 \times 10^6 \rfloor$$

$$\lfloor \boxed{500\text{ million instructions/s}} \rfloor$$

under ideal one-completion-per-cycle conditions.

18.36 Pipeline Latency with Register Overhead

If:

$$\lfloor k=5 \rfloor$$

and:

$$T_{\text{clock}} = 2\text{ns}$$

instruction latency:

$$5(2)$$

$$\boxed{10\text{ns}}$$

Again, throughput after fill:

$$1/2\text{ns}$$

not one instruction every 10 ns.

18.37 Latency Can Increase After Pipelining

Suppose original combinational instruction delay:

$$8\text{ns}$$

Split into 5 stages with:

$$2\text{ns/clock}$$

Latency:

$$5(2) = 10\text{ns}$$

One instruction now takes longer.

Yet throughput can improve greatly.

Important conceptual question:

Pipelining can improve throughput even if individual instruction latency does not improve.

18.38 Pipeline Depth

Number of stages is called:

$$\boxed{\text{Pipeline Depth}}$$

Example:

5-stage CPU:

$$\text{Depth} = 5$$

Deeper pipeline can permit smaller stage logic delays and higher clock rate, but:

- register overhead increases,
- hazards can cost more cycles,
- branch penalties may increase.

We study hazards later.

18.39 Ideal CPI of a Pipeline

After pipeline is filled:

$\boxed{CPI_{ideal}=1}$

because ideally one instruction completes every cycle.

This does **not** mean each instruction uses only one stage.

Each instruction still passes through all stages.

18.40 Real Pipeline CPI

Hazards/stalls increase CPI.

Conceptually:

$\boxed{CPI_{actual} = CPI_{ideal} + StallCyclesPerInstruction}$

Usually:

$CPI_{ideal} \approx 1$

for a simple scalar pipeline.

We will use this heavily in hazard numericals.

18.41 Pipeline Stall

A **stall** means an instruction cannot advance normally for one or more cycles.

A stall inserts empty work into the pipeline.

Often called:

$\boxed{\text{Bubble}}$

18.42 Bubble Intuition

Normal:

TEXT

I 1 !' I 2 !' I 3 !' I 4

With stall:

TEXT

I 1 !' I 2 !' [b u b b l e] !' I 3

Bubbles reduce throughput.

Full hazard study comes next.

18.43 Pipeline Hazard Preview

Three broad hazard classes:

1. Structural Hazard

- 2. Data Hazard
- 3. Control Hazard

Memory trick:

SDC

Structural Data Control

18.44 Structural Hazard Preview

Two instructions need the same hardware resource at the same time.

Example:

instruction fetch and data-memory access both need one single memory.

18.45 Data Hazard Preview

An instruction depends on a result that an earlier instruction has not produced yet.

Example:

TEXT

```
ADD R1, R2, R3
SUB R4, R1, R5
```

SUB needs R1 produced by ADD.

18.46 Control Hazard Preview

Branch/jump changes the instruction flow.

CPU may not immediately know which instruction should be fetched next.

Example:

TEXT

```
BEQ R1, R2, L1
```

Full hazard treatment comes in Chapters 20–22.

18.47 Pipeline Timing with a Stall

Suppose 4-stage pipeline and one stall occurs.

Without stall, 5 instructions:

$\lceil 4 \times 5 - 1 = 8 \rceil$

cycles.

With one stall:

$$\boxed{9 \text{ cycles}}$$

General simplified:

$$\boxed{\text{TotalCycles} = k + n - 1 + \text{StallCycles}}$$

18.48 Master Formula with Stalls

$$\boxed{T = (k + n - 1 + S)T_{\text{clock}}}$$

where:

- k = stages,
 - n = instructions,
 - S = total stall cycles.
-

18.49 Example

5-stage pipeline.

20 instructions.

3 total stall cycles.

Clock = 2 ns.

Cycles:

$$\boxed{5 + 20 - 1 + 3 = 27}$$

Time:

$$\boxed{27(2)}$$

$$\boxed{54\text{ns}}$$

18.50 Speedup with Stalls

If non-pipeline time:

$$\boxed{200\text{ns}}$$

pipeline time:

$$\boxed{54\text{ns}}$$

Speedup:

$$\boxed{200/54}$$

$$\boxed{3.70\text{times}}$$

18.51 Pipeline Completion Rate

Once steady-state is reached with no stalls:

TEXT

Cycle	5	!	I 1	completes
Cycle	6	!	I 2	completes
Cycle	7	!	I 3	completes
Cycle	8	!	I 4	completes

Thus:

$\boxed{1 \text{ completion/cycle}}$

18.52 First Instruction Completion

In a k -stage pipeline:

First instruction finishes after:

$\boxed{k \text{ cycles}}$

not after one cycle.

18.53 Last Instruction Completion

For n instructions:

Last instruction completes at:

$\boxed{k+n-1}$

th cycle under ideal conditions.

18.54 GATE Trap — n Instructions Need n Cycles

Wrong.

Only after pipeline is already full can completions happen one/cycle.

Starting from empty:

$\boxed{k+n-1}$

cycles.

18.55 GATE Trap — Pipeline Makes One Instruction k Times Faster

Wrong.

Pipelining improves total throughput, not necessarily single-instruction latency.

18.56 GATE Trap — Clock = Average Stage Delay

Wrong.

Clock is governed by:

$\boxed{\text{slowest stage}}$

plus pipeline-register overhead.

18.57 GATE Trap — More Stages Always Better

Wrong.

More stages can:

- reduce combinational delay per stage,
- potentially raise frequency,

but also:

- add register overhead,
- increase hazard penalties,
- increase branch penalty,
- increase complexity.

18.58 GATE Trap — Ideal Speedup Always Equals k

Only asymptotically for a balanced pipeline and large number of instructions.

Exact:

$\boxed{S = \frac{nk}{n+k-1}}$

for equal stage times.

18.59 GATE Trap — CPI = Number of Pipeline Stages

Wrong.

Latency may be k cycles, but ideal steady-state CPI:

$\boxed{1}$

18.60 GATE Trap — Throughput = $1 / \text{Instruction Latency}$

Not for a pipeline.

If:

- latency = 10 ns,
- clock = 2 ns,

pipeline throughput after fill can be:

$\boxed{1/2\text{ns}}$

not:

$\boxed{1/10\text{ns}}$

18.61 GATE Numerical — Basic

6-stage pipeline.

50 instructions.

Each stage = 2 ns.

Ideal pipeline time:

$$\lceil (6+50-1)(2) \rceil$$

$$\lceil 55(2) \rceil$$

$$\lceil \boxed{110\text{ns}} \rceil$$

18.62 Non-Pipeline Time

One instruction:

$$\lceil 6(2)=12\text{ns} \rceil$$

50:

$$\lceil 600\text{ns} \rceil$$

Speedup:

$$\lceil 600/110 \rceil$$

$$\lceil \boxed{5.45\text{times}} \rceil$$

Maximum theoretical:

$$\lceil 6\text{times} \rceil$$

18.63 GATE Numerical — Efficiency

Using previous:

$$\lceil \text{Efficiency} = \frac{5.45}{6} \rceil$$

$$\lceil \approx 0.909 \rceil$$

$$\lceil \boxed{90.9\%} \rceil$$

18.64 GATE Numerical — Unequal Stages

Stages:

$$\lceil 1, 2, 4, 3 \text{ ns} \rceil$$

No register overhead.

Clock:

$$\lceil 4\text{ns} \rceil$$

20 instructions:

$$\lceil (4+20-1)(4) \rceil$$

$$\lceil 23(4) \rceil$$

$$\lceil \boxed{92\text{ns}} \rceil$$

18.65 Non-Pipeline for Previous

One instruction:

$$\lceil 1+2+4+3=10\text{ns} \rceil$$

20:

$$\lceil 200\text{ns} \rceil$$

Speedup:

$$\lceil 200/92 \rceil$$

$$\lceil \boxed{\approx 2.17\text{times}} \rceil$$

18.66 Add Register Overhead

Same stages, register overhead:

$$\lceil 0.5\text{ns} \rceil$$

Clock:

$$\lceil 4.5\text{ns} \rceil$$

Pipeline:

$$\lceil 23(4.5) \rceil$$

$$\lceil \boxed{103.5\text{ns}} \rceil$$

Speedup:

$$\lceil 200/103.5 \rceil$$

$$\lceil \boxed{\approx 1.93\text{times}} \rceil$$

Register overhead matters.

18.67 GATE Numerical — Required Number of Instructions

Suppose a 5-stage ideal pipeline needs total:

$$\lceil 24 \rceil$$

cycles from empty.

$$\lceil 5+n-1=24 \rceil$$

$$\lceil n+4=24 \rceil$$

$$\lceil \boxed{n=20} \rceil$$

18.68 GATE Numerical — Find Number of Stages

A pipeline processes 100 instructions in 107 cycles without stalls.

$$k + 100 - 1 = 107$$

$$k + 99 = 107$$

$$\boxed{k=8}$$

18.69 GATE Numerical — Find Stall Cycles

5-stage pipeline executes 50 instructions in 60 cycles.

Ideal:

$$5 + 50 - 1 = 54$$

Actual:

$$60$$

Stalls:

$$60 - 54$$

$$\boxed{6}$$

18.70 GATE Numerical — CPI Approximation

Suppose:

- 1000 instructions,
- 5-stage pipeline,
- 100 stall cycles.

Total:

$$5 + 1000 - 1 + 100 = 1104$$

Average cycles/instruction:

$$1104 / 1000$$

$$\boxed{1.104}$$

For very long programs, fill/drain overhead becomes negligible, so approximately:

$$CPI \approx 1 + \frac{\text{Stalls}}{\text{Instructions}}$$

$$= 1.1$$

18.71 Large-n CPI Approximation

For long instruction stream:

$$\boxed{CPI \approx 1 + \text{AverageStallCyclesPerInstruction}}$$

This formula becomes very important in hazard chapters.

18.72 Throughput Numerical

Pipeline clock:

$$\sqrt[5]{500\text{MHz}}$$

Ideal CPI:

$$\sqrt[5]{1}$$

Ideal throughput:

$$\sqrt[5]{500 \times 10^6}$$

$$\sqrt[5]{\boxed{500 \text{ MIPS}}}$$

if one instruction completion per cycle and ignoring other complications.

18.73 Actual Throughput with CPI

If:

$$\sqrt[5]{\text{ClockRate}=2\text{GHz}}$$

and:

$$\sqrt[5]{\text{CPI}=1.25}$$

Instruction rate:

$$\sqrt[5]{\frac{2 \times 10^9}{1.25}}$$

$$\sqrt[5]{=1.6 \times 10^9}$$

$$\sqrt[5]{\boxed{1.6 \text{ billion instructions/s}}}$$

18.74 Pipeline and CPU Performance Formula

Recall:

$$\sqrt[5]{\text{CPUTime} = \text{InstructionCount} \times \text{CPI} \times \text{ClockCycleTime}}$$

or:

$$\sqrt[5]{\boxed{\text{CPUTime} = \frac{\text{IC} \times \text{CPI}}{\text{ClockRate}}}}$$

Pipelining aims to reduce effective CPI/raise throughput, often also enabling higher clock rate.

18.75 Pipeline Registers

Between stages, special registers hold intermediate values.

For example:

TEXT

IF/ID

ID/EX

EX/MEM

MEM/WB

These preserve data/control information between stages.

[SUPPORT]

You should recognize these names if they appear in a question.

18.76 Why Pipeline Registers Are Needed

At one instant:

- I1 may be in MEM,
- I2 in EX,
- I3 in ID,
- I4 in IF.

Each stage needs its own intermediate values.

Pipeline registers separate stages and retain values until next cycle.

18.77 Pipeline Stage Utilization

In early cycles, many stages are empty.

Example 5-stage:

Cycle 1:

TEXT

IF busy
others empty

Cycle 2:

TEXT

IF + ID busy

Eventually all 5 may be busy.

This explains fill inefficiency.

18.78 Steady State

After initial filling and before draining:

$\boxed{\text{Steady State}}$

means pipeline can ideally have one instruction in every stage.

Example:

TEXT

W B	!	I 1
M E M	!	I 2
E X	!	I 3
I D	!	I 4

18.79 Scalar Pipeline

[SUPPORT]

A simple scalar pipeline ideally issues/completes at most about one instruction per cycle.

More advanced superscalar processors can handle multiple instructions per cycle, but that is outside the core requirement here.

Do not mix superscalar execution with basic GATE pipeline formulas unless specified.

18.80 Fast GATE Recognition Rules

If question says:

k stages, n instructions, equal stage delay

think:

$$\boxed{k+n-1}$$

cycles.

If asks ideal speedup:

$$\boxed{\frac{n}{n+k-1}}$$

If asks maximum speedup:

$$\boxed{k}$$

If stage delays differ:

$$\boxed{T_{\text{clock}} = \max(\text{stage})}$$

If register delay given:

$$\boxed{T_{\text{clock}} = \max(\text{stage}) + \text{register delay}}$$

If stalls given:

$$\boxed{k+n-1+\text{stalls}}$$

CHAPTER 18 — MASTER FORMULA SHEET

Ideal pipeline cycles

$$\boxed{C_{\text{pipe}} = k+n-1}$$

With stalls

$$\boxed{C_{\text{pipe}} = k+n-1+S}$$

Pipeline time

$$T_{\text{pipe}} = (k+n-1)T_{\text{clock}}$$

Clock period

$$T_{\text{clock}} = \max(t_i) + t_r$$

Equal-stage non-pipelined time

$$T_{\text{nonpipe}} = nkt$$

General non-pipeline time

$$T_{\text{nonpipe}} = n \sum t_i$$

Speedup

$$\text{Speedup} = \frac{T_{\text{nonpipe}}}{T_{\text{pipe}}}$$

Equal-stage ideal speedup

$$S = \frac{nk}{n+k-1}$$

Efficiency

$$E = \frac{S}{k} = \frac{n}{n+k-1}$$

CHAPTER 18 — MEMORY TRICKS

Five stages

I Don't Eat Much Bread

IF ! ID ! EX ! MEM ! WB

Pipeline cycles

First takes k, rest add one

$$k+n-1$$

Clock

Slowest stage sets the pace

Pipeline benefit

Throughput, not necessarily latency

Hazards

SDC

Structural Data Control

CHAPTER SNAPSHOT

1. Pipelining overlaps execution of multiple instructions.
2. Classic stages are IF–ID–EX–MEM–WB.
3. Pipelining mainly improves throughput.
4. First instruction still needs all stages.
5. Ideal k-stage pipeline completes n instructions in $(k+n-1)$ cycles.
6. Maximum ideal speedup approaches (k) .
7. Finite workloads have fill/drain overhead.
8. Ideal CPI after filling is approximately 1.
9. Pipeline clock is determined by slowest stage.
10. Pipeline registers add overhead.
11. Unbalanced stages reduce achievable speedup.
12. Pipeline efficiency approaches 100% for large (n) under ideal conditions.
13. Stalls increase total cycles.
14. Structural, data, and control hazards cause pipeline disruption.
15. More pipeline stages are not automatically better.

MUST REMEMBER

$$\boxed{k+n-1}$$

$$\boxed{T_{\text{clock}} = \max(\text{StageDelay}) + \text{RegisterOverhead}}$$

$$\boxed{\text{Speedup} = \frac{T_{\text{nonpipe}}}{T_{\text{pipe}}}}$$

and:

Pipelining improves instruction throughput, not automatically single-instruction latency.

PRACTICE SET

Q1 — L0 NAT

A 5-stage ideal pipeline executes 20 instructions.

How many cycles are required?

Q2 — L1 NAT

A 6-stage pipeline executes 100 instructions without stalls.

How many cycles?

Q3 — L1 NAT

A 4-stage pipeline has stage delay 2 ns each.

Find time for 10 instructions.

Q4 — L1 NAT

For Q3, find non-pipelined time and speedup.

Q5 — L2 NAT

A 5-stage pipeline executes 50 instructions and suffers 8 total stall cycles.

Find total cycles.

Q6 — L2

Stage delays are:

$[2, 5, 3, 4] \text{ ns}$

Pipeline-register overhead = 1 ns.

Find pipeline clock period.

Q7 — L2 NAT

For Q6, find pipelined execution time for 25 instructions.

Q8 — L2 NAT

For the same stage delays, assume non-pipelined instruction delay equals the sum of stage delays.

Find non-pipelined execution time for 25 instructions and pipeline speedup.

Q9 — L2 MCQ

Pipelining primarily improves:

A. Single-instruction latency always B. Throughput C. Main-memory capacity D. Address width

Q10 — L2 MSQ

For a fixed instruction stream, increasing pipeline depth may:

A. allow a shorter logic delay per stage B. increase pipeline-register overhead C. increase branch misprediction penalty D. guarantee infinite speedup

Q11 — L3 NAT

A pipeline executes 200 instructions in 214 cycles. It has 8 stages.

How many total stall cycles occurred?

Q12 — L3 NAT

A 5-stage pipeline with clock period 2 ns executes 1000 instructions with 150 stall cycles.
Find total execution time.

Q13 — L3 NAT

A processor runs at 2.4 GHz with actual CPI 1.2.
Find instruction throughput in billion instructions/s.

CHAPTER 18 COMPLETION STATUS

' Pipeline motivation ' IF-ID-EX-MEM-WB ' Non-pipelined vs pipelined execution ' Latency vs throughput ' Timing diagrams ' k-stage pipeline ' Fill and drain ' Ideal cycle formula ' Speedup ' Maximum speedup ' Efficiency ' Balanced/unbalanced pipelines ' Slowest-stage clock rule ' Pipeline-register overhead ' CPI concept ' Stalls/bubbles preview ' Structural/data/control hazard preview ' GATE numericals and traps

CHAPTER 18: COMPLETE '

COURSE PROGRESS

Completed: 19 / 26 modules
Approximate roadmap progress:

NEXT !' CHAPTER 19 — PIPELINE PERFORMANCE NUMERICALS

Next we go deeper into the numerical side:
Ø=stage-delay optimization Ø=throughput and latency Ø=pipe speedup Ø=efficiency Ø=CPI Ø=stalls Ø=inst. completion timing Ø=uneven stage delays Ø=reg. overhead Ø=pipe frequency Ø=large GATE-style mixed numericals
Then Chapter 20 starts the major topic: PIPELINE HAZARDS.

USER

next

ASSISTANT

CHAPTER 19 — PIPELINE PERFORMANCE NUMERICALS

Classification: [GATE CORE] **Priority:** ~~Ø=VERY~~ HIGH **Official syllabus connection:** Instruction pipelining — performance

This chapter is the numerical consolidation of pipeline basics.

You must become comfortable with:

- pipeline clock period,
- execution time,
- speedup,
- throughput,
- latency,
- efficiency,
- stalls,
- CPI,
- pipeline-register overhead,
- unequal stage delays.

19.1 Master Pipeline Numerical Workflow

Whenever a pipeline numerical appears, solve in this order:

1. Find number of stages (k) .
2. Find pipeline clock period.
3. Find number of instructions (n) .
4. Add any stalls.
5. Find pipelined time.
6. Find non-pipelined time if needed.
7. Calculate speedup

USER

next

ASSISTANT

CHAPTER 19 — PIPELINE PERFORMANCE NUMERICALS — CONTINUED

We stopped at the general workflow. Now we complete the full numerical toolkit.

19.2 Pipeline Clock Period

If stage delays are:

$[t_1, t_2, \dots, t_k]$

and pipeline-register overhead is (t_r) ,

then:

$$\boxed{T_{\text{clock}} = \max(t_i) + t_r}$$

This is the first formula to write in unequal-stage questions.

19.3 Example — Clock Period

Stage delays:

$$[2, 3, 5, 4, 2] \text{ ns}$$

Register overhead:

$$[0.5 \text{ ns}]$$

Slowest stage:

$$[5 \text{ ns}]$$

Therefore:

$$T_{\text{clock}} = 5 + 0.5$$

$$\boxed{5.5 \text{ ns}}$$

19.4 Pipeline Frequency

Once clock period is known:

$$\boxed{f = \frac{1}{T_{\text{clock}}}}$$

If:

$$T_{\text{clock}} = 5 \text{ ns}$$

then:

$$f = \frac{1}{5 \times 10^{-9}}$$

$$[= 200 \text{ MHz}]$$

$$\boxed{200 \text{ MHz}}$$

19.5 Fast GHz/ns Rule

If period is in ns:

$$\boxed{f(\text{GHz}) = \frac{1}{T(\text{ns})}}$$

Examples:

$$[1 \text{ ns} \rightarrow 1 \text{ GHz}]$$

$$[2 \text{ ns} \rightarrow 0.5 \text{ GHz}]$$

$$[0.5 \text{ ns} \rightarrow 2 \text{ GHz}]$$

19.6 Ideal Pipeline Cycles

For:

- k stages
- n instructions
- no stalls

$$\boxed{\text{Cycles} = k + n - 1}$$

If total stall cycles S occur:

$$\boxed{\text{Cycles} = k + n - 1 + S}$$

19.7 Example

Pipeline:

$$k = 6$$

Instructions:

$$n = 40$$

Stalls:

$$S = 5$$

Cycles:

$$6 + 40 - 1 + 5$$

$$\boxed{50}$$

19.8 Pipelined Execution Time

$$\boxed{T_{\text{pipe}} = (k + n - 1 + S)T_{\text{clock}}}$$

For previous example, if clock = 2 ns:

$$T_{\text{pipe}} = 50(2)$$

$$\boxed{100\text{ns}}$$

19.9 Non-Pipelined Time — Equal Stages

If each of k stages takes t :

$$\boxed{T_{\text{nonpipe}} = nkt}$$

Example:

- 5 stages
- each = 2 ns
- 100 instructions

$$T_{\text{nonpipe}} = 100(5)(2)$$

$$\boxed{1000\text{ns}}$$

19.10 Non-Pipelined Time — Unequal Stages

If one instruction passes sequentially through:

$$[t_1, t_2, \dots, t_k]$$

then one instruction takes:

$$[\sum t_i]$$

For (n) instructions:

$$[\boxed{T_{\text{nonpipe}} = n \sum_{i=1}^k t_i}]$$

19.11 Example

Stage delays:

$$[1, 2, 4, 3 \text{ ns}]$$

One non-pipelined instruction:

$$[1+2+4+3=10\text{ns}]$$

For 20 instructions:

$$[\boxed{200\text{ns}}]$$

19.12 Pipeline Time for Same Example

Slowest stage:

$$[4\text{ns}]$$

Assume zero register overhead.

$$[T_{\text{clock}}=4\text{ns}]$$

For $(k=4, n=20)$:

$$[\text{Cycles}=4+20-1=23]$$

$$[T_{\text{pipe}}=23(4)]$$

$$[\boxed{92\text{ns}}]$$

19.13 Speedup

$$[\boxed{\text{Speedup}=\frac{T_{\text{nonpipe}}}{T_{\text{pipe}}}}]$$

Previous example:

$$[S=\frac{200}{92}]$$

$$[\boxed{2.17\text{times}}]$$

approximately.

19.14 Ideal Equal-Stage Speedup

For equal stage delay and zero stalls:

$$\boxed{\text{Speedup} = \frac{nk}{n+k-1}}$$

As n becomes very large:

$$\boxed{\text{Speedup} \rightarrow k}$$

19.15 Example — Finite Instructions

5-stage pipeline, 20 instructions.

$$S = \frac{20(5)}{20+5-1}$$

$$= \frac{100}{24}$$

$$\boxed{4.17 \times}$$

not exactly $5\times$.

19.16 Pipeline Efficiency

$$\boxed{\text{Efficiency} = \frac{\text{Speedup}}{k}}$$

For equal stages:

$$\boxed{\text{Efficiency} = \frac{n}{n+k-1}}$$

Previous:

$$E = \frac{20}{24}$$

$$\boxed{83.33\%}$$

19.17 Effect of More Instructions

For fixed k :

$$n \uparrow$$

then:

$$\text{Efficiency} \uparrow$$

because fill/drain overhead becomes less significant.

19.18 Throughput

Ideal steady-state throughput:

$$\boxed{\text{Throughput} = \frac{1}{T_{\text{clock}}}}$$

If clock:

$$2\text{ns}$$

then:

$$\text{Throughput} = 500 \times 10^6$$

$$\boxed{500 \text{ million instructions/s}}$$

for ideal CPI = 1.

19.19 Throughput Using CPI

More generally:

$$\boxed{\text{InstructionRate} = \frac{\text{ClockRate}}{\text{CPI}}}$$

Example:

- Clock = 3 GHz
- CPI = 1.5

$$\frac{3}{1.5}$$

$$\boxed{2 \text{ billion instructions/s}}$$

19.20 Latency

For a k -stage pipeline:

$$\boxed{\text{Latency} = kT_{\text{clock}}}$$

assuming one cycle per stage.

Example:

$$k=5, \quad T_{\text{clock}}=2\text{ns}$$

$$\boxed{\text{Latency}=10\text{ns}}$$

19.21 Throughput vs Latency Trap

Suppose:

$$\boxed{\text{Latency}=10\text{ns}}$$

This does **not** mean one instruction completes every 10 ns.

After filling, one may complete every:

$$\boxed{2\text{ns}}$$

if:

$$\boxed{T_{\text{clock}}=2\text{ns}}$$

19.22 First Completion Time

For k -stage pipeline:

$$\boxed{\text{FirstInstructionCompletion} = kT_{\text{clock}}}$$

19.23 Completion Time of the m -th Instruction

Assuming pipeline starts empty and no stalls:

$$\boxed{T_m = (k+m-1)T_{\text{clock}}}$$

19.24 Example

5-stage pipeline, clock = 3 ns.

When does 7th instruction finish?

$$(5+7-1)(3)$$

$$= 11(3)$$

$$\boxed{33\text{ns}}$$

19.25 Completion Interval

After pipeline is full:

$$\boxed{\text{CompletionInterval} = T_{\text{clock}}}$$

So if clock = 3 ns:

TEXT

I1 completes at 15 ns

I2 completes at 18 ns

I3 completes at 21 ns

...

19.26 Find Number of Instructions from Total Cycles

No stalls:

$$C = k + n - 1$$

Therefore:

$$\boxed{n = C - k + 1}$$

Example:

8-stage pipeline completes workload in 107 cycles.

$$n = 107 - 8 + 1$$

$$\boxed{100}$$

19.27 Find Number of Stages

$$C = k + n - 1$$

So:

$$\boxed{k = C - n + 1}$$

Example:

100 instructions require 105 cycles.

$$\lfloor k=105-100+1 \rfloor$$

$$\lfloor \boxed{6} \rfloor$$

19.28 Find Stall Cycles

Actual cycles:

$$\lfloor C_{\text{actual}} \rfloor$$

Ideal:

$$\lfloor k+n-1 \rfloor$$

Therefore:

$$\lfloor \boxed{S=C_{\text{actual}}-(k+n-1)} \rfloor$$

19.29 Example

5-stage pipeline.

100 instructions.

Actual cycles:

$$\lfloor 120 \rfloor$$

Ideal:

$$\lfloor 5+100-1=104 \rfloor$$

Stalls:

$$\lfloor 120-104 \rfloor$$

$$\lfloor \boxed{16} \rfloor$$

19.30 Average CPI from Total Cycles

For $\lfloor n \rfloor$ instructions:

$$\lfloor \boxed{CPI_{\text{avg}}=\frac{\text{TotalCycles}}{n}} \rfloor$$

Previous example:

$$\lfloor CPI=\frac{120}{100} \rfloor$$

$$\lfloor \boxed{1.2} \rfloor$$

19.31 Long-Program CPI Approximation

For large $\lfloor n \rfloor$:

$$\lfloor k-1 \rfloor$$

fill/drain cycles become negligible.

Thus:

$$\boxed{CPI \approx 1 + \frac{\text{StallCycles}}{\text{Instructions}}}$$

19.32 Example

1000 instructions.

200 stalls.

$$CPI \approx 1 + \frac{200}{1000}$$

$$\boxed{1.2}$$

Exact cycles for a 5-stage pipeline:

$$5 + 1000 - 1 + 200 = 1204$$

Exact CPI:

$$1.204$$

Difference is small.

19.33 Pipeline Stall Percentage

Suppose ideal CPI = 1 and actual CPI = 1.3.

Average stall:

$$1.3 - 1$$

$$\boxed{0.3 \text{ stall cycles/instruction}}$$

19.34 Stall Fraction of Total Cycles

If:

- total cycles = 1200
- stall cycles = 200

stall fraction:

$$\frac{200}{1200}$$

$$\boxed{16.67\%}$$

Do not confuse this with:

$$200/1000 = 0.2$$

stall cycles per instruction.

19.35 Pipeline Frequency with Register Overhead

Stage delays:

$$1.5, 2, 2.5, 2 \text{ ns}$$

Register overhead:

$$\lfloor 0.5\text{ns} \rfloor$$

Clock:

$$\lfloor 2.5 + 0.5 = 3\text{ns} \rfloor$$

Frequency:

$$\lfloor \frac{1}{3\text{ns}} \rfloor$$

$$\lfloor \boxed{333.33\text{MHz}} \rfloor$$

19.36 Register Overhead Can Limit Speedup

Suppose original logic delay:

$$\lfloor 10\text{ns} \rfloor$$

Split perfectly into 5 stages:

$$\lfloor 2\text{ns}/\text{stage} \rfloor$$

But pipeline-register overhead:

$$\lfloor 0.5\text{ns} \rfloor$$

Clock:

$$\lfloor 2.5\text{ns} \rfloor$$

Maximum steady-state speedup compared with 10 ns non-pipelined instruction interval:

$$\lfloor \frac{10}{2.5} \rfloor$$

$$\lfloor \boxed{4\times} \rfloor$$

not 5×.

19.37 Ideal Stage Balancing

For fixed total logic delay:

$$\lfloor D \rfloor$$

and $\lfloor k \rfloor$ equal stages:

$$\lfloor \text{StageDelay} \approx \frac{D}{k} \rfloor$$

With register overhead $\lfloor r \rfloor$:

$$\lfloor \boxed{T_{\text{clock}} \approx \frac{D}{k} + r} \rfloor$$

As $\lfloor k \rfloor$ increases:

- logic per stage decreases,
 - register overhead becomes more significant.
-

19.38 Very Deep Pipeline Insight

As:

$$\lfloor k \rightarrow \infty \rfloor$$

$\lceil D/k \rceil$

but:

$T_{\text{clock}} \nrightarrow$

because register overhead remains.

Therefore pipeline depth cannot produce infinite clock rate.

19.39 Stage Splitting Numerical

Suppose a stage takes:

8 ns

while other stages are:

$4\text{ ns}, 4\text{ ns}, 4\text{ ns}$

Pipeline clock:

8 ns

If the 8 ns stage is split into two 4 ns stages, new pipeline:

TEXT

4, 4, 4, 4 ns

Clock becomes:

4 ns

But stage count increases from 4 to 5.

For long programs, throughput nearly doubles.

For short programs, fill overhead matters.

19.40 Compare Before and After Splitting

100 instructions.

Before

4 stages, 8 ns clock:

$(4+100-1)(8)$

$103(8)=824\text{ ns}$

After

5 stages, 4 ns clock:

$104(4)$

416 ns

Speedup due to redesign:

$824/416$

$$\boxed{1.98\times}$$

19.41 But Splitting with Register Overhead

Suppose each pipeline boundary adds:

$$1\text{ ns}$$

Before:

$$\text{Clock}=8+1=9\text{ ns}$$

After:

$$\text{Clock}=4+1=5\text{ ns}$$

Times:

$$103(9)=927\text{ ns}$$

$$104(5)=520\text{ ns}$$

Improvement:

$$927/520$$

$$\boxed{1.78\times}$$

Still useful, but less ideal.

19.42 Pipeline with Different Functional Delays

Suppose stages:

TEXT

$$\text{IF} = 250 \text{ ps}$$

$$\text{ID} = 350 \text{ ps}$$

$$\text{EX} = 150 \text{ ps}$$

$$\text{MEM} = 300 \text{ ps}$$

$$\text{WB} = 200 \text{ ps}$$

Pipeline-register overhead:

$$50\text{ ps}$$

Clock:

$$350+50$$

$$\boxed{400\text{ ps}}$$

Frequency:

$$\frac{1}{400\text{ ps}}$$

$$\boxed{2.5\text{ GHz}}$$

19.43 Non-Pipelined Single-Cycle Processor Comparison

If same logic is performed in one long cycle:

$$\lfloor 250+350+150+300+200 \rfloor$$

$$\lfloor =1250\text{ps} \rfloor$$

Single-cycle frequency:

$$\lfloor \frac{1}{1.25\text{ns}} \rfloor$$

$$\lfloor \boxed{800\text{MHz}} \rfloor$$

19.44 Long-Stream Throughput Speedup

Pipeline interval:

$$\lfloor 400\text{ps} \rfloor$$

Non-pipeline interval:

$$\lfloor 1250\text{ps} \rfloor$$

Approximate long-stream throughput speedup:

$$\lfloor 1250/400 \rfloor$$

$$\lfloor \boxed{3.125\times} \rfloor$$

not $5\times$ because stages are unbalanced and register overhead exists.

19.45 Time for n Instructions in Previous Pipeline

For 100 instructions:

$$\lfloor (5+100-1)(400\text{ps}) \rfloor$$

$$\lfloor 104(400) = 41600\text{ps} \rfloor$$

$$\lfloor \boxed{41.6\text{ns}} \rfloor$$

19.46 Non-Pipeline Time

$$\lfloor 100(1250\text{ps}) \rfloor$$

$$\lfloor =125000\text{ps} \rfloor$$

$$\lfloor \boxed{125\text{ns}} \rfloor$$

Speedup:

$$\lfloor 125/41.6 \rfloor$$

$$\lfloor \boxed{3.00\times} \rfloor$$

approximately.

19.47 GATE Pattern — Given Frequency, Find Pipeline Time

Suppose:

- 5 stages
-

- 100 instructions
- clock = 2 GHz
- no stalls

Clock period:

$$\boxed{0.5\text{ns}}$$

Cycles:

$$\boxed{104}$$

Time:

$$\boxed{104(0.5)}$$

$$\boxed{52\text{ns}}$$

19.48 GATE Pattern — Given Pipeline Time, Find Frequency

Suppose 4-stage pipeline executes 97 instructions in:

$$\boxed{200\text{ns}}$$

No stalls.

Cycles:

$$\boxed{4+97-1=100}$$

Clock:

$$\boxed{200/100}$$

$$\boxed{2\text{ns}}$$

Frequency:

$$\boxed{500\text{MHz}}$$

19.49 GATE Pattern — Speedup Target

Suppose an ideal 5-stage equal pipeline processes n instructions.

Find n for speedup 4.

$$4 = \frac{5n}{n+4}$$

$$4n+16=5n$$

$$\boxed{n=16}$$

19.50 General Formula for Required n

For ideal k -stage pipeline and desired speedup S :

$$S = \frac{nk}{n+k-1}$$

Solve:

$$S(n+k-1) = nk$$

$$\lceil S_n + S(k-1) = nk \rceil$$

$$\lceil n(k-S) = S(k-1) \rceil$$

Therefore:

$$\lceil \boxed{n = \frac{S(k-1)}{k-S}} \rceil$$

provided $(S < k)$.

19.51 GATE Pattern — Efficiency Target

For ideal equal-stage pipeline:

$$\lceil E = \frac{n}{n+k-1} \rceil$$

Suppose:

$$\lceil k=5 \rceil$$

desired efficiency:

$$\lceil 80\% = 0.8 \rceil$$

$$\lceil 0.8 = \frac{n}{n+4} \rceil$$

$$\lceil 0.8n + 3.2 = n \rceil$$

$$\lceil 0.2n = 3.2 \rceil$$

$$\lceil \boxed{n=16} \rceil$$

19.52 General n for Efficiency E

$$\lceil E = \frac{n}{n+k-1} \rceil$$

$$\lceil E(n+k-1) = n \rceil$$

$$\lceil n - En = E(k-1) \rceil$$

$$\lceil n(1-E) = E(k-1) \rceil$$

$$\lceil \boxed{n = \frac{E(k-1)}{1-E}} \rceil$$

19.53 GATE Pattern — Fractional Stage Times

Suppose:

$$\lceil 1.2, 1.8, 2.1, 1.4 \text{ ns} \rceil$$

register overhead:

$$\lceil 0.2 \text{ ns} \rceil$$

Clock:

$$\lceil 2.1 + 0.2 \rceil$$

$$\lceil \boxed{2.3 \text{ ns}} \rceil$$

Don't round too early.

19.54 Rounding Trap

If final answer asks NAT, preserve intermediate precision.

Example:

$$\text{Speedup} = \frac{1000}{303}$$

Do not prematurely approximate 303 as 300.

Calculate final value at end.

19.55 Pipeline with Bubbles

Suppose timing:

TEXT

Cycle: 1 2 3 4 5 6 7 8

I1: F D E M W

I2: F D - E M W

The - represents one stall/bubble.

This increases completion time by one cycle.

19.56 Bubble Propagation

A bubble behaves like an empty pipeline slot.

It moves through later stages, reducing useful completion density.

Conceptually:

TEXT

Instruction

Instruction

Bubble

Instruction

19.57 Total Stall Cycles vs Number of Stall Events

One hazard event may create more than one stall cycle.

Example:

- 3 hazard events
- each introduces 2 stalls

Total:

$$\boxed{6 \text{ stall cycles}}$$

Always use total cycles, not number of hazard events.

19.58 Branch Stall Average

Suppose:

- 20% instructions are branches
- each branch causes 2 stall cycles

Average stall/instruction:

$$\lfloor 0.2(2) \rfloor$$

$$\lfloor 0.4 \rfloor$$

Thus:

$$\lfloor \text{CPI} = 1 + 0.4 \rfloor$$

$$\lfloor \boxed{1.4} \rfloor$$

This is a preview for control-hazard numericals.

19.59 Branch Misprediction Version

Suppose:

- 20% branches
- prediction accuracy = 90%
- misprediction penalty = 3 cycles

Misprediction fraction/instruction:

$$\lfloor 0.2(0.1) = 0.02 \rfloor$$

Average stall:

$$\lfloor 0.02(3) = 0.06 \rfloor$$

CPI:

$$\lfloor \boxed{1.06} \rfloor$$

We will study this deeply later.

19.60 Data Hazard Average CPI Preview

Suppose:

- 30% instructions experience one dependency stall
- each creates 1 cycle stall

Average stall:

$$\lfloor 0.3 \rfloor$$

CPI:

$$\lfloor \boxed{1.3} \rfloor$$

19.61 Multiple Stall Sources

Suppose:

- data hazards add 0.2 cycles/instruction
- branches add 0.1
- cache stalls add 0.4

Then:

$$[CPI = 1 + 0.2 + 0.1 + 0.4]$$

$$[\boxed{1.7}]$$

assuming independent/additive average penalties as stated.

19.62 CPU Time with Pipeline CPI

Suppose:

- $IC = 10^9$
- $CPI = 1.25$
- Clock = 2.5 GHz

$$[CPUTime = \frac{10^9(1.25)}{2.5 \times 10^9}]$$

$$[\boxed{0.5s}]$$

19.63 Compare Two Pipelines

Processor A:

- 2 GHz
- $CPI = 1$

Processor B:

- 3 GHz
- $CPI = 1.8$

Instruction count equal.

Time is proportional to:

$$[\frac{CPI}{ClockRate}]$$

A:

$$[1/2 = 0.5]$$

B:

$$[1.8/3 = 0.6]$$

Therefore:

$$[\boxed{A \text{ is faster}}]$$

despite lower clock rate.

Important lesson:

19.64 Speedup Between CPUs

$$\boxed{\text{Speedup}_{A \text{ over } B} = \frac{\text{Time}_B}{\text{Time}_A}}$$

Using previous:

$$\frac{0.6}{0.5}$$

$$\boxed{1.2 \times}$$

A is 20% faster.

19.65 Clock Rate vs Pipeline Depth

Deeper pipelines may increase clock rate but can also increase:

- branch penalties,
- register overhead,
- hazard sensitivity.

Therefore compare:

$$\boxed{\frac{\text{CPI}}{\text{ClockRate}}}$$

not clock rate alone.

19.66 GATE Mixed Example

Processor P1:

- 5-stage pipeline
- clock = 2 GHz
- CPI = 1.1

Processor P2:

- 10-stage pipeline
- clock = 3 GHz
- CPI = 1.6

Same IC.

P1 time factor:

$$1.1/2 = 0.55$$

P2:

$$1.6/3 \approx 0.5333$$

Therefore P2 is slightly faster.

Speedup:

$$0.55/0.5333$$

$\boxed{\approx 1.031 \times}$

Only about 3.1% faster.

19.67 GATE Trap — Use $k+n$ Instead of $k + n - 1$

Wrong.

Correct:

$\boxed{k+n-1}$

Reason:

first instruction needs k cycles, each remaining one adds 1.

19.68 GATE Trap — Ignore Pipeline Register Delay

If explicitly given, it must be added:

$\boxed{\max(\text{stage}) + \text{register overhead}}$

19.69 GATE Trap — Sum Stage Delays for Pipeline Clock

Wrong.

Sum is relevant to non-pipelined single-instruction delay.

Pipeline clock uses:

$\boxed{\max(\text{stage})}$

plus overhead.

19.70 GATE Trap — Speedup = Number of Stages

Only ideal asymptotic limit.

Exact finite-workload speedup can be much smaller.

19.71 GATE Trap — CPI Is Always 1

Only ideal steady state with no hazards/stalls.

Real:

$\boxed{CPI > 1}$

often.

19.72 GATE Trap — Fill Overhead Ignored for Small n

For short sequences, fill/drain can dominate.

Example:

10-stage pipeline, only 2 instructions:

$\lfloor 10+2-1=11 \rfloor$

cycles.

Not close to 2 cycles.

19.73 GATE Trap — Stalls Added Once Per Hazard Type

Wrong.

Use actual total stall cycles or expected average stalls.

19.74 GATE Trap — Throughput and Latency Same

They are different:

- latency = time/instruction
- throughput = instructions/time

19.75 GATE Trap — Clock Frequency Convert Incorrectly

Remember:

$\lfloor 1\text{GHz} \rightarrow 1\text{ns} \rfloor$

$\lfloor 2\text{GHz} \rightarrow 0.5\text{ns} \rfloor$

$\lfloor 4\text{GHz} \rightarrow 0.25\text{ns} \rfloor$

19.76 FAST GATE SOLVING TEMPLATE

When you see a pipeline numerical:

A. Clock

$\lfloor T_c = \max(\text{stage}) + \text{reg} \rfloor$

B. Cycles

$\lfloor C = k + n - 1 + \text{stalls} \rfloor$

C. Time

$\lfloor T = C \times T_c \rfloor$

D. Non-pipeline

$\lfloor T_{\text{np}} = n \times \text{stage} \rfloor$

E. Speedup

$\lfloor S = T_{\text{np}} / T_p \rfloor$

F. CPI

$\backslash$ CPI=cycles/instructions $\backslash$

19.77 MASTER MIXED NUMERICAL

Stages:

$\backslash$ 2, \ 4, \ 3, \ 5, \ 2 \ ns $\backslash$

Register overhead:

$\backslash$ 1ns $\backslash$

Instructions:

$\backslash$ 50 $\backslash$

Stalls:

$\backslash$ 6 $\backslash$

Pipeline clock

$\backslash$ 5+1=6ns $\backslash$

Cycles

$\backslash$ 5+50-1+6 = 60 $\backslash$

Pipeline time

$\backslash$ 60(6) = 360ns $\backslash$

Non-pipeline one instruction

$\backslash$ 2+4+3+5+2 = 16ns $\backslash$

Non-pipeline 50 instructions

$\backslash$ 800ns $\backslash$

Speedup

$\backslash$ 800/360 $\backslash$

$\backslash$ $\boxed{2.22\times}$ $\backslash$

Exact CPI

$\backslash$ 60/50 $\backslash$

$\backslash$ $\boxed{1.2}$ $\backslash$

19.78 Another Mixed Numerical

Pipeline:

- 8 stages
- 1000 instructions
- clock = 400 ps

- 92 stall cycles

Cycles:

$$\lfloor 8 + 1000 - 1 + 92 \rfloor$$

$$\lfloor = 1099 \rfloor$$

Time:

$$\lfloor 1099(400\text{ps}) \rfloor$$

$$\lfloor 439600\text{ps} \rfloor$$

$$\lfloor \boxed{439.6\text{ns}} \rfloor$$

CPI:

$$\lfloor 1099/1000 \rfloor$$

$$\lfloor \boxed{1.099} \rfloor$$

19.79 Steady-State CPI Approximation

Previous:

$$\lfloor 1 + \frac{92}{1000} = 1.092 \rfloor$$

Exact:

$$\lfloor 1.099 \rfloor$$

Difference:

$$\lfloor 0.007 \rfloor$$

comes from fill overhead:

$$\lfloor k-1=7 \rfloor$$

cycles.

19.80 Why Large-n Formula Works

Exact cycles:

$$\lfloor n + (k-1) + S \rfloor$$

Divide by n:

$$\lfloor \text{CPI} = 1 + \frac{k-1}{n} + \frac{S}{n} \rfloor$$

For large n :

$$\lfloor \frac{k-1}{n} \approx 0 \rfloor$$

Thus:

$$\lfloor \boxed{\text{CPI} \approx 1 + \frac{S}{n}} \rfloor$$

CHAPTER 19 — FORMULA SHEET

Clock

$$T_c = \max(t_i) + t_r$$

Frequency

$$f = \frac{1}{T_c}$$

Ideal cycles

$$k + n - 1$$

With stalls

$$k + n - 1 + S$$

Pipeline time

$$T_p = (k + n - 1 + S) T_c$$

Non-pipeline

$$T_{np} = \sum t_i$$

Speedup

$$\text{Speedup} = \frac{T_{np}}{T_p}$$

Exact CPI

$$\text{CPI} = \frac{\text{TotalCycles}}{n}$$

Large-stream CPI

$$\text{CPI} \approx 1 + \frac{\text{Stalls}}{\text{Instructions}}$$

Instruction throughput

$$\text{Rate} = \frac{\text{ClockRate}}{\text{CPI}}$$

CHAPTER 19 — MEMORY TRICKS

Clock

Slowest + Storage overhead

Cycles

$k + n - 1 + \text{stalls}$

Speedup

Old time ÷ New time

CPI

Cycles ÷ Instructions

Long program

$$\text{CPI} = 1 + \text{average stalls}$$

CHAPTER SNAPSHOT

1. Pipeline clock is determined by slowest stage plus register overhead.
2. Ideal (n) -instruction time is $(k+n-1)$ cycles.
3. Stalls add directly to total cycles.
4. Throughput depends on clock period and CPI.
5. Latency depends on number of stages $\times$ clock period.
6. Speedup must use execution-time ratio.
7. Register overhead prevents unlimited benefit from deeper pipelines.
8. Unbalanced stages waste pipeline potential.
9. Finite workloads suffer fill/drain overhead.
10. Exact CPI includes fill overhead.
11. Large-stream CPI approaches $(1 + \text{average stall cycles/instruction})$.
12. Higher clock rate alone does not guarantee a faster processor.
13. Stage splitting can improve throughput but adds pipeline overhead.
14. Branch/data/cache stalls can all increase CPI.

MUST REMEMBER

$$T_c = \max(\text{stage}) + \text{register overhead}$$

$$\text{Cycles} = k + n - 1 + \text{stalls}$$

$$\text{CPI} = \frac{\text{Cycles}}{\text{Instructions}}$$

$$\text{CPUTime} = \frac{\text{IC} \times \text{CPI}}{\text{ClockRate}}$$

PRACTICE SET

Q1 — L1 NAT

A 7-stage ideal pipeline processes 100 instructions.

How many cycles?

Q2 — L1 NAT

Stage delays:

$$2, 3, 6, 4, 3 \text{ ns}$$

Register overhead = 1 ns.

Find clock period.

Q3 — L1 NAT

For Q2, find clock frequency in MHz.

Q4 — L2 NAT

For Q2, pipeline 50 instructions with 5 total stall cycles.

Find execution time.

Q5 — L2 NAT

For Q4, assume non-pipelined instruction delay equals sum of stage delays.

Find speedup.

Q6 — L2 NAT

A 5-stage pipeline executes 200 instructions in 220 cycles.

Find total stall cycles.

Q7 — L2 NAT

For Q6, find exact average CPI.

Q8 — L2 NAT

A 6-stage pipeline with 1 ns clock executes 1000 instructions with 100 stalls.

Find total execution time.

Q9 — L2

A processor runs at 4 GHz with CPI 1.25.

Find instruction throughput in billion instructions/s.

Q10 — L3 NAT

A 5-stage pipeline has equal stage delay 2 ns and no register overhead.

How many instructions are required to achieve exactly 4× speedup?

Q11 — L3 NAT

A 6-stage ideal pipeline must operate at 90% efficiency.

Find the minimum number of instructions required.

Q12 — L3

Processor A:

- 2 GHz
- CPI 1.0

Processor B:

- 3 GHz
- CPI 1.6

Same instruction count.

Which is faster and by what factor?

Q13 — L3 NAT

Stage delays:

$\{150, 300, 200, 250, 180\}$ ps

Pipeline register overhead:

$\{50\}$ ps

Find:

1. clock period,
 2. clock frequency,
 3. latency,
 4. ideal steady-state throughput.
-

CHAPTER 19 COMPLETION STATUS

' Pipeline clock numerical problems ' Frequency calculations ' Ideal and stalled execution time ' Speedup ' Efficiency ' Throughput ' Latency ' Completion timing ' CPI ' Fill/drain overhead ' Unequal stage delays ' Pipeline-register overhead ' Stage splitting ' Deep-pipeline trade-offs ' CPU performance comparison ' GATE traps and reverse numericals

CHAPTER 19: COMPLETE '

COURSE PROGRESS

Completed: 20 / 26 modules

Approximate roadmap progress:

$\sim 77\%$

NEXT !' CHAPTER 20 — PIPELINE HAZARDS

This is now the most important remaining pipeline topic:

Structural hazards Data hazards RAW / WAR / WAW Control hazards Dependency vs hazard
exact pipeline timing diagrams stall insertion Identifying whether a dependency actually causes a hazard
GATE conceptual and numerical traps

Then Chapter 21 will focus entirely on data-hazard solutions: forwarding, stalling and scheduling.

USER

next

ASSISTANT

CHAPTER 20 — PIPELINE HAZARDS

Classification: [GATE CORE] Priority: VERY HIGH Official syllabus connection: Pipeline hazards

This is one of the most important COA topics.

Core idea:

A pipeline hazard is a situation that prevents the next instruction from executing in its intended pipeline cycle.

Three major categories:

[Structural, Data, Control]

Memory trick:

SDC

Structural Data Control

20.1 What Is a Hazard?

In an ideal 5-stage pipeline:

TEXT

Cycle	1	2	3	4	5	6	7
I1	IF	ID	EX	MEM	WB		
I2		IF	ID	EX	MEM	WB	
I3			IF	ID	EX	MEM	WB

Every instruction moves one stage per cycle.

A hazard can force:

- a stall,
- a bubble,
- forwarding,
- instruction reordering,

- flushing.

20.2 Hazard vs Dependency

These are related but not identical.

Dependency

A logical relationship between instructions.

Example:

TEXT

```
I1: ADD R1, R2, R3
I2: SUB R4, R1, R5
```

I2 depends on the value produced by I1.

Hazard

The dependency actually creates a timing problem in a particular pipeline.

Thus:

Dependency is in the program. Hazard depends on implementation/timing.

Very important.

20.3 Three Hazard Classes

1. Structural Hazard

Two operations need the same hardware resource at the same time.

2. Data Hazard

An instruction needs data that is not yet safely available.

3. Control Hazard

CPU does not yet know the correct next instruction because of branch/jump/control flow.

PART A — STRUCTURAL HAZARDS

20.4 Structural Hazard

A **structural hazard** occurs when hardware resources are insufficient for overlapping instructions.

Example:

Suppose the processor has only one memory for both:

- instruction fetch,

- data access.

Then one cycle may require:

TEXT

```
I 1 !'  M E M   s t a g e   w a n t s   m e m o r y
I 4 !'  I F   s t a g e   w a n t s   m e m o r y
```

Both need the same memory simultaneously.

Conflict.

20.5 Structural Hazard Example

5-stage pipeline:

TEXT

```
I F   !'   I D   !'   E X   !'   M E M   !'   W B
```

Suppose:

TEXT

```
I1: LOAD R1, 0(R2)
```

In cycle 4:

- I1 is in MEM and needs memory.
- I4 may be in IF and also needs memory.

If only one memory port exists:

$\boxed{\text{Structural hazard}}$

20.6 Structural Hazard Solution 1 — Stall

One instruction waits.

Example:

TEXT

```
Cycle 4:
MEM gets memory
IF stalls
```

This inserts a bubble / delays fetch.

20.7 Structural Hazard Solution 2 — Duplicate Resources

Use separate:

- instruction memory,
- data memory.

This is similar to Harvard-style separation.

Then:

TEXT

```
I F !'   I n s t r u c t i o n   M e m o r y
M E M !'   D a t a   M e m o r y
```

No conflict.

20.8 Structural Hazard Solution 3 — Multi-Port Resource

A multi-ported register file or memory may allow multiple simultaneous accesses.

More hardware cost, fewer structural hazards.

20.9 Structural Hazard Memory Trick

Structural = Same Structure needed twice

PART B — DATAHAZARDS

20.10 What Is a Data Hazard?

A data hazard occurs when overlapping instructions access the same operand/register in a problematic order.

Three classical types:

1. RAW
2. WAR
3. WAW

20.11 RAW — Read After Write

Most important in basic pipelines.

Instruction 2 wants to **read** a value after Instruction 1 is supposed to **write** it.

Example:

TEXT

```
I1: ADD R1, R2, R3
I2: SUB R4, R1, R5
```

I1 writes:

[R1]

I2 reads:

[R1]

Dependency:

$\boxed{W \rightarrow R}$

Hence:

$\boxed{RAW}$

20.12 RAW Meaning

Correct program order requires:

TEXT

```
I1 writes R1
then
I2 reads new R1
```

If pipeline lets I2 read too early, it may read the old value.

That creates a hazard.

20.13 RAW Is a True Dependency

RAW corresponds to a true data dependence.

Also called:

$\boxed{\text{Flow Dependency}}$

Because data flows from producer to consumer.

20.14 RAW Memory Trick

RAW = Read After Write

or:

Producer writes, consumer reads

20.15 WAR — Write After Read

Example:

TEXT

```
I1: SUB R4, R1, R5
I2: ADD R1, R2, R3
```

I1 must read old $R1$.

I2 later writes new $R1$.

Correct order:

TEXT

I1 reads R1 first
I2 writes R1 later

Dependency:

$R \rightarrow W$

Therefore:

$\boxed{WAR}$

20.16 WAR Is an Anti-Dependency

WAR is called:

$\boxed{\text{Anti-dependency}}$

It occurs because both instructions reuse the same register name.

It is not true data flow from I1 to I2.

20.17 WAW — Write After Write

Example:

TEXT

I1: ADD R1, R2, R3
I2: MUL R1, R4, R5

Both write:

$R1$

Correct final value should come from I2 because I2 is later in program order.

If I1 writes after I2 due to out-of-order timing, final value becomes wrong.

Thus:

$\boxed{WAW}$

20.18 WAW Is Output Dependency

WAW is called:

$\boxed{\text{Output dependency}}$

because both instructions produce output into the same destination.

20.19 RAW/WAR/WAW Summary

Hazard	Earlier Instruction	Later Instruction	Dependency
RAW	Write X	Read X	True/Flow
WAR	Read X	Write X	Anti
WAW	Write X	Write X	Output

Memory trick:

Look at the later operation.

- later **Reads** after earlier Write !' RAW
- later **Writes** after earlier Read !' WAR
- later **Writes** after earlier Write !' WAW

20.20 Fast Hazard Identification

Example:

TEXT

I1: R1 = R2 + R3
I2: R4 = R1 + R5

I1:

TEXT

Write R1

I2:

TEXT

Read R1

Therefore:

$\boxed{\text{RAW on } R1}$

20.21 Example — WAR

TEXT

I1: R5 = R1 + R2
I2: R1 = R3 + R4

I1 reads R1.

I2 writes R1.

$\boxed{\text{WAR on } R1}$

20.22 Example — WAW

TEXT

```
I1: R1 = R2 + R3
I2: R1 = R4 - R5
```

Both write R1:

$\boxed{\text{WAW}}$

20.23 Multiple Hazards Can Exist

Example:

TEXT

```
I1: R1 = R2 + R3
I2: R2 = R1 + R4
```

On R1:

TEXT

```
I1 writes
I2 reads
```

RAW.

On R2:

TEXT

```
I1 reads
I2 writes
```

WAR.

Therefore same pair can have:

$\boxed{\text{more than one dependency}}$

20.24 Important Basic-Pipeline Fact

In the simple classic 5-stage **in-order** pipeline:

TEXT

```
IF ID EX MEM WB
```

instructions usually:

- read operands in ID,
- write results in WB,
- proceed in program order.

Therefore RAW hazards are the main register data hazard.

WAR and WAW typically do **not** occur in this basic in-order pipeline.

Ø=Very important GATE concept.

20.25 Why WAR Usually Doesn't Occur in Simple In-Order Pipeline

All instructions read registers in the ID stage.

Earlier instruction reaches ID before later instruction.

So earlier read happens before later write.

Thus later instruction normally cannot write before earlier read.

No WAR timing hazard.

20.26 Why WAW Usually Doesn't Occur

All register writes occur in WB and instructions reach WB in program order for a fixed-length simple pipeline.

So later instruction cannot normally write before earlier one.

No WAW hazard.

20.27 When WAR/WAW Can Occur

They become relevant when:

- instructions execute out of order,
- different instructions have different execution latencies,
- multiple functional units complete at different times.

Example:

TEXT

I1: DIV R1, ...

I2: ADD R1, ...

If ADD finishes before DIV, WAW ordering can become problematic.

20.28 True vs Name Dependencies

RAW

True dependency.

Cannot simply remove by renaming because later instruction truly needs earlier result.

WAR/WAW

Name dependencies.

Can often be removed using:

$\boxed{\text{Register Renaming}}$

[SUPPORT / interview-relevant]

20.29 Register Renaming Example — WAR

Original:

TEXT

```
I1: R5 = R1 + R2
I2: R1 = R3 + R4
```

Rename I2 destination:

TEXT

```
I1: R5 = R1 + R2
I2: R10 = R3 + R4
```

WAR disappears if future uses are updated accordingly.

PART C — RAW TIMING IN 5-STAGE PIPELINE

20.30 Basic Pipeline Assumption

Use:

TEXT

```
I F !' I D !' E X !' M E M !' W B
```

Assume:

- source registers read in ID,
- ALU result generated in EX,
- register written in WB.

Consider:

TEXT

```
I1: ADD R1, R2, R3
I2: SUB R4, R1, R5
```

20.31 Timing Without Handling

TEXT

```
C y c l e !' 1 2 3 4 5 6
```

```
I1      IF  ID  EX  MEM WB
I2      IF  ID  EX  MEM WB
```

I2 reads R1 in:

\[Cycle\ 3 \]

I1 writes R1 in:

[Cycle\ 5]

So I2 reads too early.

[\boxed{\text{RAW hazard}}]

20.32 Naive Stall Solution

If no forwarding and register must be written before consumer reads:

I2 must delay its ID/register read until the value becomes available.

Conceptually:

TEXT

I1: IF ID EX MEM WB

I2: IF ID ST ST EX MEM WB

Exact number of stalls can depend on register-file timing assumptions.

This is crucial.

20.33 Register File Same-Cycle Timing

Many textbook pipelines assume:

- register write occurs in first half of cycle,
- register read occurs in second half.

Then consumer may read a value in ID during the same cycle producer writes it in WB.

If so:

TEXT

Producer WB = cycle 5

Consumer ID = cycle 5

is allowed.

This affects stall counts.

~~0-1~~ Always follow question's timing convention.

20.34 Without Same-Cycle Read/Write

If a problem assumes read and write cannot safely occur in same cycle, consumer ID must happen after WB.

That can add one extra stall.

Never memorize one universal number blindly.

20.35 Forwarding Preview

Instead of waiting for WB, the ALU result may be sent directly from a later pipeline register to a consumer's EX stage.

This is:

$\boxed{\text{Forwarding / Bypassing}}$

Chapter 21 will cover this deeply.

20.36 Why Forwarding Helps

For:

TEXT

```
I1: ADD R1, R2, R3
I2: SUB R4, R1, R5
```

I1 produces ALU result at end of EX.

I2 needs it in its EX stage one cycle later.

Forwarding path:

TEXT

```
I1 EX result
    !"
I2 EX input
```

can avoid waiting until WB.

20.37 Load-Use Hazard Preview

This is one of the most important special cases.

TEXT

```
I1: LW R1, 0(R2)
I2: ADD R3, R1, R4
```

For a load:

- address calculated in EX,
- data arrives from memory at end of MEM.

But consumer needs R1 in EX too early.

Even with ordinary forwarding, one stall is commonly needed in classic 5-stage pipeline.

We will derive this in Chapter 21.

PART D — CONTROL HAZARDS

20.38 What Is a Control Hazard?

A control hazard occurs when the processor does not know which instruction should be fetched next.

Usually caused by:

- conditional branch,
- jump,
- return,
- exception.

Most GATE questions focus on branches.

20.39 Branch Example

TEXT

```
BEQ R1, R2, TARGET
ADD R3, R4, R5
SUB R6, R7, R8
...
TARGET:
MUL ...
```

Until CPU knows whether branch is taken, it may not know whether next PC should be:

- sequential PC,
- target PC.

20.40 Why a Branch Causes Trouble

Pipeline keeps fetching instructions before branch result may be known.

Suppose branch decision is known in EX:

TEXT

```
C y c l e ! ' 1      2      3

Branch  IF  ID  EX
Next    IF  ID
Next2   IF
```

When branch reaches EX, two younger instructions may already have entered pipeline.

If branch taken, they may be wrong-path instructions.

20.41 Wrong-Path Instructions

Instructions fetched before correct branch direction is known are sometimes called speculative/wrong-path instructions.

If branch turns out taken and sequential instructions are wrong:

$\boxed{\text{Flush them}}$

20.42 Pipeline Flush

A **flush** discards incorrectly fetched instructions.

Conceptually:

TEXT

```
W r o n g   i n s t r u c t i o n   ! '   r e m o v e  
W r o n g   i n s t r u c t i o n   ! '   r e m o v e
```

Their work is lost.

This creates a branch penalty.

20.43 Control Hazard Solutions

Major techniques:

1. Stall until branch resolved.
2. Predict branch not taken.
3. Predict branch taken.
4. Dynamic branch prediction.
5. Delayed branching (historical support).
6. Resolve branch earlier.

20.44 Stall Until Branch Resolved

Simplest:

TEXT

```
Branch fetched  
!"  
Stop fetching useful new instructions  
!"  
Wait for branch decision  
!"  
Continue correct path
```

Safe but slow.

20.45 Predict Not Taken

Assume:

branch will not be taken.

Continue fetching sequential instructions.

If correct:

$\boxed{\text{No/low penalty}}$

If wrong:

$\boxed{\text{Flush + penalty}}$

20.46 Predict Taken

Assume branch is taken and begin fetching target path.

Requires target address availability.

Useful when branches are frequently taken.

20.47 Dynamic Prediction Preview

Processor learns from previous branch behavior.

Examples include:

- 1-bit predictors,
- 2-bit saturating counters.

Not always required in basic GATE COA, but control-hazard performance numericals often use prediction accuracy.

Chapter 22 covers the numerical side.

20.48 Branch Penalty

Branch penalty = number of cycles lost due to handling branch/control hazard.

Example:

- branch resolved in EX,
- two younger instructions must be flushed.

Penalty may be:

$\lfloor 2 \text{cycles} \rfloor$

depending on pipeline/assumptions.

Do not assume penalty solely from stage count; use timing described.

20.49 Average Branch Stall

If:

- branch frequency = $\lfloor f_b \rfloor$,
- fraction mispredicted = $\lfloor m \rfloor$,
- misprediction penalty = $\lfloor P \rfloor$,

then:

$\lfloor \boxed{\text{AverageBranchStall} = f_b \times m \times P} \rfloor$

Then:

$\lfloor \boxed{\text{CPI} = \text{CPI}_{\text{base}} + f_b m P} \rfloor$

20.50 Example

20% instructions are branches.

Prediction accuracy:

90%

Misprediction rate:

$10\% = 0.1$

Penalty:

3

Average stall:

$0.2(0.1)(3)$

$= 0.06$

If base CPI = 1:

$\boxed{CPI = 1.06}$

PART E — STRUCTURAL HAZARD TIMING EXAMPLE

20.51 Single-Memory Pipeline

Suppose IF and MEM share one memory.

Instructions:

TEXT

I1: LW
I2: ADD
I3: SUB
I4: OR

Timeline before considering conflict:

TEXT

	Cycle 1	Cycle 2	Cycle 3	Cycle 4	Cycle 5
I1	IF	ID	EX	MEM	WB
I2		IF	ID	EX	MEM
I3			IF	ID	EX
I4				IF	ID

Cycle 4:

- I1 needs memory in MEM.
- I4 wants memory for IF.

Conflict.

Thus IF must stall or resource must be duplicated.

20.52 Why ALU Instructions May Not Cause That Specific Conflict

For an ADD:

TEXT

ADD R1, R2, R3

MEM stage may simply pass through without using data memory.

So a unified-memory structural conflict may specifically occur when older instruction is load/store.

Use actual resource requirements.

PART F — DATA DEPENDENCY ANALYSIS

20.53 Read Set and Write Set Method

For each instruction, identify:

$\{ R(I) = \text{registers read} \}$

$\{ W(I) = \text{registers written} \}$

For pair (I_1) before (I_2) :

RAW

$\{ W(I_1) \cap R(I_2) \neq \emptyset \}$

WAR

$\{ R(I_1) \cap W(I_2) \neq \emptyset \}$

WAW

$\{ W(I_1) \cap W(I_2) \neq \emptyset \}$

$\emptyset = \text{Excellent exam method.}$

20.54 Example Using Sets

TEXT

I1: ADD R1, R2, R3

I2: SUB R2, R1, R4

I1:

$\{ R(I_1) = \{R2, R3\} \}$

$\{ W(I_1) = \{R1\} \}$

I2:

$\{ R(I_2) = \{R1, R4\} \}$

$\{ W(I_2) = \{R2\} \}$

RAW

$\{ W(I_1) \cap R(I_2) = \{R1\} \}$

RAW on R1.

WAR

$\mathbb{R}(I_1) \cap W(I_2) = \{R2\}$

WAR on R2.

WAW

empty.

20.55 Store Instruction Dependency Trap

Consider:

TEXT

```
SW R1, 0(R2)
```

A store usually:

- reads R1 as data,
- reads R2 as base/address register,
- writes memory,
- does not write a general-purpose register.

Thus:

$\mathbb{R} = \{R1, R2\}$

$\mathbb{W}_{\text{reg}} = \emptyset$

Be careful.

20.56 Load Instruction

TEXT

```
LW R1, 0(R2)
```

Typically:

- reads R2,
- writes R1.

Thus:

$\mathbb{R} = \{R2\}$

$\mathbb{W} = \{R1\}$

20.57 Branch Dependency

Example:

TEXT

```
BEQ R1, R2, L
```


Branch reads:

`\[ R1,R2 \]`

and affects PC/control flow.

It usually does not write a normal general-purpose destination register.

If previous instruction produces R1, branch can have RAW dependency too.

20.58 Dependency Distance

Suppose:

TEXT

```
I1: ADD R1, ...  
I2: independent  
I3: independent  
I4: SUB ..., R1, ...
```

Consumer is farther away.

By the time I4 needs R1, producer may already have written or forwarded it.

Thus dependency may exist but no stall hazard remains.

This is why:

dependency "automatic stall.

20.59 Example — Dependency but No Hazard

Classic 5-stage pipeline, no forwarding, write in first half/read second half.

TEXT

```
I1: ADD R1, R2, R3  
I2: independent  
I3: independent  
I4: SUB R4, R1, R5
```

I1 WB occurs before/at time I4 reads operands.

So RAW dependency exists, but it may not require a stall.

20.60 Critical Exam Question

"Is there a dependency?" and "Does it create a hazard?" are different questions.

Always check timing.

PART G — PIPELINE STALLS AND BUBBLES

20.61 What Happens During a Stall?

If instruction cannot proceed:

- it may remain in same stage,
- earlier stages may also freeze,
- a bubble can be inserted into later stage.

Exact control depends on pipeline.

20.62 Bubble

A bubble is an empty slot that occupies pipeline resources but performs no useful instruction work.

Example:

TEXT

C y c l e ! ' 1 2 3 4 5 6 7

I1 F D E M W

I2 F D - E M W

- represents a bubble/stall.

20.63 Stall Cost

If an instruction stream would ideally take:

$k+n-1$

cycles and hazards add S stalls:

$\boxed{\text{Cycles} = k+n-1+S}$

20.64 Average CPI with Hazards

For a long pipeline:

$\boxed{\text{CPI} \approx 1 + \text{Average Stall Cycles/Instruction}}$

If:

- data hazards = 0.2 stalls/instruction
- control hazards = 0.1
- structural = 0.05

Then:

$\boxed{\text{CPI} = 1 + 0.2 + 0.1 + 0.05}$

$\boxed{1.35}$

assuming additive averages.

PART H — HAZARD IDENTIFICATION EXAMPLES

20.65 Example 1

TEXT

I1: ADD R1, R2, R3
I2: SUB R4, R1, R5

I1 writes R1.

I2 reads R1.

$\boxed{\text{RAW}}$

20.66 Example 2

TEXT

I1: ADD R1, R2, R3
I2: SUB R2, R4, R5

I1 reads R2.

I2 writes R2.

$\boxed{\text{WAR}}$

20.67 Example 3

TEXT

I1: ADD R1, R2, R3
I2: MUL R1, R4, R5

Both write R1.

$\boxed{\text{WAW}}$

20.68 Example 4

TEXT

I1: LW R1, 0(R2)
I2: ADD R3, R1, R4

I1 writes R1.

I2 reads R1.

$\boxed{\text{RAW}}$

Specifically:

$\boxed{\text{Load-use dependency}}$

20.69 Example 5

TEXT

```
I1: ADD R1, R2, R3
I2: SW R1, 0(R4)
```

Store reads R1 as store data.

Therefore:

$\boxed{\text{RAW on R1}}$

Do not think store has no dependencies simply because it has no destination register.

20.70 Example 6

TEXT

```
I1: ADD R1, R2, R3
I2: BEQ R1, R4, L
```

Branch reads R1.

Therefore:

$\boxed{\text{RAW dependency}}$

and branch itself also creates a potential control hazard.

So one instruction pair can involve both:

- data hazard,
 - control hazard.
-

PART I — CONTROL HAZARD PERFORMANCE

20.71 Always-Stall Branch Strategy

Suppose:

- branches = 20% of instructions,
- branch resolution requires 2 stall cycles.

Then:

$\boxed{\text{AverageBranchStalls} = 0.2(2)=0.4}$

Thus:

$\boxed{\text{CPI}=1+0.4}$

$\boxed{1.4}$

20.72 Prediction Strategy

Suppose:

- branch fraction = 20%
- accuracy = 90%
- penalty when wrong = 3 cycles.

$$\backslash[\text{MispredictionFraction} = 0.2(0.1) = 0.02 \backslash]$$

Average penalty:

$$\backslash[0.02(3) = 0.06 \backslash]$$

$$\backslash[\boxed{\text{CPI} = 1.06} \backslash]$$

Huge improvement compared with always stalling.

20.73 Taken-Branch Fraction

Sometimes question says:

- 20% instructions are branches,
- 60% of branches are taken,
- predict not taken,
- taken-branch penalty = 2 cycles.

Prediction is wrong only for taken branches.

Average penalty:

$$\backslash[0.2(0.6)(2) \backslash]$$

$$\backslash[= 0.24 \backslash]$$

CPI:

$$\backslash[\boxed{1.24} \backslash]$$

20.74 Predict Taken Example

Suppose:

- branch frequency = 25%
- 70% branches taken
- predict taken
- wrong penalty = 2 cycles.

Wrong when branch is not taken:

$$\backslash[30\% = 0.3 \backslash]$$

Average:

$$\backslash[0.25(0.3)(2) \backslash]$$

$$\backslash[= 0.15 \backslash]$$

$$\backslash[\boxed{\text{CPI} = 1.15} \backslash]$$

PART J — HAZARD SOLUTIONS OVERVIEW

20.75 Structural Hazard Solutions

- Stall
 - Duplicate hardware
 - Multi-port resources
 - Separate instruction/data memory
-

20.76 RAW Hazard Solutions

- Stall/interlock
 - Forwarding/bypassing
 - Instruction scheduling
 - Compiler reordering
-

20.77 WAR/WAW Solutions

Especially in more advanced/out-of-order pipelines:

- register renaming
 - ordered completion
 - scheduling control
-

20.78 Control Hazard Solutions

- stall
 - branch prediction
 - speculative execution
 - flush on misprediction
 - earlier branch resolution
 - delayed branch
-

20.79 Interlock

A hardware mechanism that detects a hazard and automatically stalls pipeline is often called a:

$\boxed{\text{Pipeline Interlock}}$

Useful term to recognize.

PART K — GATE TRAPS

20.80 Trap — Every Dependency Is a Hazard

'LWrong.

A dependency creates a hazard only if pipeline timing makes it unsafe.

20.81 Trap — RAW/WAR/WAW All Occur in Basic 5-Stage Pipeline

Usually false.

In a simple in-order fixed-stage pipeline:

[\boxed{RAW} is the main register data hazard]

WAR/WAW generally require more complex execution timing.

20.82 Trap — Store Writes Its Source Register

'LWrong.

Store generally **reads** its data register and base register.

It writes memory.

20.83 Trap — Load Data Is Ready After EX

Usually not.

For a load:

- EX computes address.
- MEM reads memory.

Loaded data becomes available after MEM.

This is why load-use hazards are special.

20.84 Trap — Structural Hazard = Data Hazard

No.

Structural:

resource conflict.

Data:

operand ordering/value conflict.

20.85 Trap — Branch Hazard Only When Branch Is Taken

A control hazard arises because direction/target is uncertain.

However actual penalty may depend on prediction and whether branch is taken/mispredicted.

20.86 Trap — Flush and Stall Are Same

Not exactly.

Stall

Pause progress / insert bubbles.

Flush

Discard instructions already fetched/executed along wrong path.

20.87 Trap — Forwarding Fixes All Hazards

No.

Forwarding mainly helps RAW data hazards.

It does not solve:

- structural hazards,
 - control hazards,
 - every load-use timing case.
-

20.88 Trap — Register Renaming Fixes RAW

No.

RAW is true dependence.

Renaming cannot remove the actual need for produced value.

Renaming mainly removes:

$\backslash$ WAR, $\backslash$ WAW $\backslash$

name dependencies.

20.89 Trap — More Hardware Always Removes Every Structural Hazard

It can remove a specific resource conflict, but new resource bottlenecks may still exist.

20.90 Trap — Branch Penalty = Pipeline Depth

Not necessarily.

Penalty depends on:

- stage where branch resolves,
 - fetch policy,
 - prediction,
 - architecture.
-

20.91 Fast Dependency Recognition Method

For two instructions $I_1 \rightarrow I_2$:

Check same register X.

Earlier writes, later reads

$\boxed{\text{RAW}}$

Earlier reads, later writes

$\boxed{\text{WAR}}$

Both write

$\boxed{\text{WAW}}$

20.92 Read/Write Table Example

TEXT

I1: ADD R1, R2, R3

I2: SUB R2, R1, R4

Instruction	Reads	Writes
I1	R2,R3	R1
I2	R1,R4	R2

Therefore:

- RAW on R1
- WAR on R2

20.93 GATE-Style Multi-Instruction Analysis

Instructions:

TEXT

I1: R1 = R2 + R3

I2: R4 = R1 + R5

I3: R2 = R6 + R7

I4: R1 = R8 + R9

Dependencies:

I1 ! I2

I1 writes R1, I2 reads R1.

$\boxed{\text{RAW R1}}$

I1 !' I3

I1 reads R2, I3 writes R2.

\[WAR\ R2 \]

I1 !' I4

Both write R1.

\[WAW\ R1 \]

I2 !' I4

I2 reads R1, I4 writes R1.

\[WAR\ R1 \]

Good practice pattern.

20.94 Hazard Distance Example

TEXT

```
I1: ADD R1, R2, R3
I2: OR  R6, R7, R8
I3: AND R9, R10, R11
I4: SUB R4, R1, R5
```

There is RAW dependency I1 !' I4 .

But in a classic 5-stage pipeline, spacing may allow I1 result to be available by time I4 needs it.

Therefore likely:

\[\boxed{\text{dependency but possibly no stall}} \]

depending on register-file timing/forwarding.

20.95 Exact Timing Always Wins

Never answer stall count from dependency name alone.

You need:

- producer stage when value becomes available,
- consumer stage when value is needed,
- forwarding paths,
- read/write timing,
- pipeline stage definitions.

Chapter 21 will train this exactly.

PART L — MIXED PERFORMANCE NUMERICALS

20.96 Structural Hazard CPI

Suppose:

- 30% instructions are loads/stores,
- each such instruction causes one IF structural stall because unified memory,
- no overlap complications given.

Average stalls:

$$\boxed{0.30(1)=0.30}$$

CPI:

$$\boxed{1.30}$$

under simplified assumptions.

20.97 Data + Branch Hazards

Suppose:

- data hazards: 20% instructions, 1 stall each
- branches: 25%, 10% mispredicted, 3-cycle penalty

Data penalty:

$$\boxed{0.2}$$

Branch:

$$\boxed{0.25(0.1)(3)=0.075}$$

Total:

$$\boxed{CPI=1+0.2+0.075}$$

$$\boxed{1.275}$$

20.98 All Three Hazard Types

Suppose:

- structural = 0.1 cycles/instruction
- data = 0.25
- control = 0.15

Then:

$$\boxed{CPI=1.50}$$

assuming penalties are additive as supplied.

20.99 CPU Time with Hazard CPI

Instruction count:

$$\boxed{10^9}$$

CPI:

$$\boxed{1.5}$$

Clock:

$\backslash [3\text{GHz} \backslash]$

$\backslash [\text{CPUTime} = \frac{10^9(1.5)}{3 \times 10^9} \backslash]$

$\backslash [\boxed{0.5\text{s}} \backslash]$

CHAPTER 20 — MASTER COMPARISON

Hazard	Cause	Typical Example	Main Solutions
Structural	Resource conflict	IF and MEM need same memory	Stall, duplicate resource
RAW	Consumer needs unfinished value	<code>ADD! SUB using result</code>	Forward, stall
WAR	Later write before earlier read	Name dependency	Renaming
WAW	Writes complete out of order	Two writes same reg	Renaming/order
Control	Unknown next PC	Branch/jump	Predict, stall, flush

CHAPTER 20 — MEMORY TRICKS

Hazard classes

SDC

Structural Data Control

Data hazards

RAW = true

WAR = anti

WAW = output

Structural

Same structure wanted simultaneously

Control

Control changes PC

Rename

CHAPTER SNAPSHOT

1. A hazard prevents ideal pipeline progress.
2. Dependency and hazard are not identical.
3. Structural hazards arise from resource conflicts.
4. Data hazards are RAW, WAR, WAW.
5. RAW is a true/flow dependence.
6. WAR is an anti-dependence.
7. WAW is an output dependence.
8. Basic in-order 5-stage pipelines mainly suffer RAW register hazards.
9. WAR/WAW become important in more advanced/out-of-order pipelines.
10. Store instructions read their source data/base registers.
11. Load-use RAW hazards are especially important.
12. Forwarding can eliminate many RAW stalls.
13. Control hazards come from branches/jumps.
14. Wrong-path instructions may need flushing.
15. Branch prediction reduces control-hazard penalty.
16. Structural hazards can be reduced by duplicating resources.
17. Stall/bubble increases CPI.
18. Exact stall count depends on timing and forwarding assumptions.

MUST REMEMBER

$\boxed{\text{RAW} = \text{Write} \rightarrow \text{Read}}$

$\boxed{\text{WAR} = \text{Read} \rightarrow \text{Write}}$

$\boxed{\text{WAW} = \text{Write} \rightarrow \text{Write}}$

and:

$\boxed{\text{Basic 5-stage in-order pipeline: RAW is the main data hazard}}$

PRACTICE SET

Q1 — L0

Identify the hazard:

TEXT

I1: ADD R1, R2, R3

I2: SUB R4, R1, R5

Q2 — L0

Identify:

TEXT

```
I1: ADD R5, R1, R2
I2: SUB R1, R3, R4
```

Q3 — L0

Identify:

TEXT

```
I1: ADD R1, R2, R3
I2: MUL R1, R4, R5
```

Q4 — L1 MSQ

Which are true?

A. RAW is a true dependency. B. WAR is an anti-dependency. C. WAW is an output dependency. D. RAW can always be removed by register renaming.

Q5 — L1

For:

TEXT

```
I1: LW R1, 0(R2)
I2: ADD R3, R1, R4
```

identify the dependency and explain why it is called a load-use dependency.

Q6 — L1

For:

TEXT

```
I1: ADD R1, R2, R3
I2: SW R1, 0(R4)
```

is there a register dependency? If yes, which type?

Q7 — L2

Find all dependencies:

TEXT

I1: $R1 = R2 + R3$
I2: $R2 = R1 + R4$

Q8 — L2 MCQ

In a classic in-order 5-stage pipeline with all register writes in WB and reads in ID, which hazard is the primary register data hazard?

A. RAW B. WAR C. WAW D. None

Q9 — L2

A unified memory is used for both instruction fetch and load/store data access. Explain one possible structural hazard.

Q10 — L2 NAT

20% of instructions are branches. A static predictor is 90% accurate. Misprediction penalty is 4 cycles.

Find average control-hazard stall cycles per instruction.

Q11 — L2 NAT

Base CPI = 1.

Data hazards contribute 0.2 cycles/instruction.

Structural hazards contribute 0.1 cycles/instruction.

Control hazards contribute 0.08 cycles/instruction.

Find actual CPI.

Q12 — L3

Find all RAW, WAR and WAW dependencies:

TEXT

I1: ADD R1, R2, R3
I2: SUB R4, R1, R2
I3: MUL R2, R5, R6
I4: OR R1, R7, R8

Q13 — L3 Conceptual

Why can a RAW dependency exist without requiring a pipeline stall?

Q14 — L3

A processor predicts every branch as not taken.

- 25% instructions are branches.
- 60% branches are actually taken.
- Wrong prediction penalty = 3 cycles.

Find CPI if base CPI = 1.

CHAPTER 20 COMPLETION STATUS

' Hazard definition ' Dependency vs hazard ' Structural hazards ' Resource conflicts ' RAW ' WAR ' WAW ' True/anti/output dependencies ' Basic 5-stage hazard behavior ' Read/write set method ' Load/store dependency analysis ' Branch dependencies ' Control hazards ' Pipeline flush ' Branch prediction basics ' Stalls/bubbles ' Hazard CPI numericals ' GATE traps

CHAPTER 20: COMPLETE '

COURSE PROGRESS

Completed: 21 / 26 modules

Approximate roadmap progress:

$$\boxed{\sim 81\%}$$

NEXT ! CHAPTER 21 — DATA HAZARD SOLUTIONS

Next is the most numerical part of hazards:

0=forwarding / bypassing 0=0% E forwarding 0=1% M! Forwarding 0=100% no-use hazard 0=8% exact stall counts 0=1%
no-forwarding vs forwarding timing diagrams 0=100% register-file same-cycle read/write assumption 0=100% compiler instruction
scheduling 0=100% NOP insertion 0=99% RAW dependency chains 0=6% BTE pipeline-stall numericals.

USER

next

ASSISTANT

CHAPTER 21 — DATA HAZARD SOLUTIONS

Classification: [GATE CORE] **Priority:** $\emptyset = \checkmark$ **VERY HIGH** **Official syllabus connection:** Pipeline hazards — data hazards

This chapter focuses on solving the most important hazard in a classic pipeline:

$$\boxed{\text{RAW — Read After Write}}$$

Main solutions:

- Stalling
- Forwarding / bypassing
- Compiler scheduling
- NOP insertion

21.1 Base Pipeline Model

We will use the classic:

$\boxed{IF \rightarrow ID \rightarrow EX \rightarrow MEM \rightarrow WB}$

Assume:

- source registers are read in ID,
- ALU operations produce result at end of EX,
- loads obtain data after MEM,
- registers are normally written in WB.

21.2 The Main RAW Problem

Example:

TEXT

```
I1: ADD R1, R2, R3
I2: SUB R4, R1, R5
```

I1 produces R1.

I2 immediately consumes R1.

Without special handling:

TEXT

	C	y	c	l	e	!	1	2	3	4	5	6
I1		IF	ID	EX	MEM	WB						
I2				IF	ID	EX	MEM	WB				

I2 needs the correct R1 for EX in cycle 4.

But normal register write occurs only at WB in cycle 5.

So:

$\boxed{\text{RAW hazard}}$

21.3 Solution 1 — Stall

Simplest solution:

Make the consumer wait until the producer's result becomes available.

This may insert:

$\boxed{\text{Bubble / Stall cycle}}$

21.4 No Forwarding Case

Suppose:

TEXT

I1: ADD R1, R2, R3
I2: SUB R4, R1, R5

and there is **no forwarding**.

If I2 must read R1 from register file only after I1 writes it in WB, I2's ID must be delayed.

Conceptually:

TEXT

	C	y	c	l	e		1		2		3		4		5		6		7
I1							IF	ID	EX	MEM	WB								
I2								IF	ID	ST	ST	EX	MEM	WB					

The exact stall count depends on register-file timing convention.

21.5 Same-Cycle Register Write/Read Assumption

A very common textbook assumption:

- WB writes register in first half of clock cycle.
- ID reads register in second half.

Then:

TEXT

I1 WB = cycle 5
I2 ID = cycle 5

is allowed.

So immediate ALU dependency may need:

$\boxed{2\text{ stalls}}$

without forwarding in that classic model.

21.6 If Same-Cycle Read/Write Is Not Allowed

Then consumer must read in cycle 6.

This may require:

$\boxed{3\text{ stalls}}$

for the same pair.

Therefore:

Never memorize a stall count without checking assumptions.

21.7 Solution 2 — Forwarding

Forwarding, also called **bypassing**, sends a produced result directly to a later instruction without waiting for register-file write-back.

Core idea:

```
TEXT
Producer result
      !"
Forwarding path
      !"
Consumer ALU input
```

21.8 Why Forwarding Works

For:

```
TEXT
I1: ADD R1, R2, R3
I2: SUB R4, R1, R5
```

I1's result becomes available after its EX stage.

I2 needs that result in its own EX stage one cycle later.

So:

```
TEXT
I 1   E X   r e s u l t   ! '   I 2   E X   i n p u t
```

can solve the hazard.

21.9 EX-to-EX Forwarding

Timeline:

```
TEXT
C y c l e   ! '   1       2       3       4       5       6

I1          IF  ID  EX  MEM  WB
I2          IF  ID  EX  MEM  WB
```

I1 calculates result at end of cycle 3.

I2 uses ALU at cycle 4.

Forward:

$\boxed{\text{EX/MEM} \rightarrow \text{EX}}$

No stall needed.

21.10 ALU-to-ALU Dependency

Example:

TEXT

```
I1: ADD R1, R2, R3
I2: SUB R4, R1, R5
```

With normal forwarding:

$\boxed{0 \text{ stalls}}$

in a classic 5-stage pipeline.

This is a high-yield result.

21.11 Forwarding from Later Stages

Consider:

TEXT

```
I1: ADD R1, R2, R3
I2: independent
I3: SUB R4, R1, R5
```

By the time I3 reaches EX, I1 may be in WB.

Result can be forwarded from a later pipeline register such as:

$\boxed{\text{MEM/WB} \rightarrow \text{EX}}$

depending on design.

21.12 Two Common Forwarding Paths

Classic conceptual forwarding paths:

$\boxed{\text{EX/MEM} \rightarrow \text{EX}}$

and:

$\boxed{\text{MEM/WB} \rightarrow \text{EX}}$

These let consumer ALU receive recently computed results directly.

21.13 Forwarding Unit

Hardware checks register numbers.

Example:

If:

TEXT

EX/MEM.RegisterRd == ID/EX.RegisterRs

then forwarding may be required.

You do not need detailed logic equations unless specifically asked.

Know the idea:

compare destination registers of older instructions against source registers of current EX instruction.

21.14 Priority of Forwarding

Suppose both an older and a newer instruction can provide the same register.

Example:

TEXT

I1: ADD R1, ...
I2: SUB R1, ...
I3: AND R4, R1, R5

I3 must use the **latest** value of R1, produced by I2.

Therefore the more recent producer must take priority.

Conceptually:

$\boxed{\text{EX/MEM forwarding often has priority over older MEM/WB}}$

if both match.

21.15 Forwarding Does Not Mean Register File Is Updated Early

Forwarding bypasses the register file for immediate use.

The architectural register may still be written normally in WB.

So:

TEXT

Forwarding path "early permanent register update"

21.16 Load-Use Hazard

Extremely important.

TEXT

I1: LW R1, 0(R2)
I2: ADD R3, R1, R4

I1 loads data from memory.

Timeline:

TEXT										
C y c l e		!	'	1	2	3	4	5		
I1		IF	ID	EX	MEM	WB				
I2			IF	ID	EX	MEM				

Problem:

- I2 needs R1 at start of EX in cycle 4.
- Load data becomes available only after I1's MEM stage in cycle 4.

Too late for I2 EX in same cycle under classic assumptions.

Therefore even with forwarding:

$\boxed{1\text{ stall}}$

is commonly required.

21.17 Load-Use Timing with One Stall

TEXT										
C y c l e		!	'	1	2	3	4	5	6	7
I1		IF	ID	EX	MEM	WB				
I2			IF	ID	ST	EX	MEM	WB		

Now:

- I1 memory data available after MEM.
- I2 EX occurs next cycle.
- Data can be forwarded to I2 EX.

Thus one bubble solves it.

21.18 Important Result Table

Classic 5-stage pipeline with forwarding:

Dependency	Typical Stalls
ALU !' next ALU	0
ALU !' instruction after one gap	0
LOAD !' immediate consumer	1
LOAD !' consumer after one independent instruction	often 0

~~Memorize~~ Memorize conceptually, but still verify timing assumptions.

21.19 Load with One Instruction Gap

TEXT

```
I1: LW  R1, 0(R2)
I2: ADD R6, R7, R8
I3: SUB R3, R1, R4
```

Timeline:

TEXT

```
I1: IF ID EX MEM WB
I2:   IF ID EX MEM WB
I3:   IF ID EX MEM WB
```

I3 EX occurs one cycle later than immediate consumer.

Load data is available in time for forwarding.

Thus:

$\boxed{0 \text{ stalls}}$

with standard forwarding.

21.20 Why Load Is Different from ADD

For ALU instruction:

$\boxed{\text{Result available after EX}}$

For load:

$\boxed{\text{Result available after MEM}}$

That one-stage delay causes the classic load-use bubble.

Memory trick:

ALU gives early, LOAD gives late.

21.21 Store Data Forwarding

Consider:

TEXT

```
I1: ADD R1, R2, R3
I2: SW  R1, 0(R4)
```

Store needs R1 as data to write into memory.

Depending on pipeline design, forwarding may send the result directly to store-data input.

So this dependency may not require waiting until WB.

Exact timing depends on when store data is needed.

21.22 Store Address Dependency

Example:

TEXT

```
I1: ADD R1, R2, R3
I2: SW R5, 0(R1)
```

Here R1 is used as base address.

Address calculation usually happens in EX.

So normal ALU forwarding can feed R1 into I2 EX.

Often:

$\lfloor \boxed{0 \text{ stalls}} \rfloor$

for ALU! store - a ~~data~~ ^{data} hazard standard forwarding pipeline.

21.23 Load !' Store Address

TEXT

```
I1: LW R1, 0(R2)
I2: SW R5, 0(R1)
```

Store needs R1 for EX address calculation immediately after load.

This behaves like a load-use dependency.

Typical:

$\lfloor \boxed{1 \text{ stall}} \rfloor$

with classic forwarding.

21.24 Load !' Store Data

TEXT

```
I1: LW R1, 0(R2)
I2: SW R1, 0(R3)
```

This is subtler.

Store may not need data until MEM stage.

Some pipelines can forward load data directly to store's MEM stage without stall.

Others may stall depending on forwarding support.

Therefore:

Use the specified forwarding paths and stage where store data is consumed.

Do not apply the one-stall load-use rule blindly.

21.25 Branch Dependency

Suppose branch compares registers in ID:

TEXT

```
I1: ADD R1, R2, R3
I2: BEQ R1, R4, L
```

If branch needs R1 in ID, ordinary EX-stage forwarding may not be enough.

Additional forwarding to ID or stalls may be required.

So branch dependency timing depends strongly on branch-resolution stage.

21.26 Key Rule

To solve any data-hazard stall problem, ask:

1. When does producer generate the value?
2. When does consumer need the value?
3. Is there a forwarding path between those stages?

This is the safest method.

21.27 Producer Availability Table

For classic 5-stage pipeline:

ALU instruction

Result generated:

$\boxed{\text{end of EX}}$

Load

Loaded data generated:

$\boxed{\text{end of MEM}}$

21.28 Consumer Need Table

Typically:

ALU operands

Needed in:

$\boxed{\text{EX}}$

Load/store base address

Needed in:

$\boxed{\text{EX}}$

Store data

Often needed in:

$\boxed{\text{MEM}}$

Branch operands

Could be ID or EX depending on design.

21.29 Availability vs Need Method

If producer value is ready before consumer needs it, forwarding can solve it.

If not:

$\boxed{\text{stall until timing becomes safe}}$

21.30 No Forwarding — General Method

Suppose registers read in ID and written in WB.

Then consumer cannot safely read until producer has written WB.

So shift consumer's ID until:

$\text{ConsumerID} \geq \text{ProducerWB}$

under same-cycle write-before-read convention.

21.31 Adjacent ALU Instructions — No Forwarding

TEXT

I1: ADD R1, ...
I2: SUB ..., R1, ...

Producer WB:

$\text{Cycle } 5$

Consumer normal ID:

$\text{Cycle } 3$

Need delay from cycle 3 to cycle 5.

Thus:

$\boxed{2 \text{ stalls}}$

with same-cycle WB/ID allowed.

21.32 One Instruction Gap — No Forwarding

TEXT

I1: ADD R1, ...
I2: independent
I3: SUB ..., R1, ...

Normal I3 ID:

\[Cycle\ 4 \]

Producer WB:

\[Cycle\ 5 \]

Need one delay:

\[\boxed{1\ stall} \]

21.33 Two Instruction Gap — No Forwarding

TEXT

I1: ADD R1, ...
I2: independent
I3: independent
I4: SUB ..., R1, ...

I4 ID naturally occurs in cycle 5.

Same cycle as I1 WB.

If write-first/read-second assumption:

\[\boxed{0\ stalls} \]

21.34 No-Forwarding Distance Rule

For adjacent producer/consumer in classic 5-stage:

TEXT

```
d i s t a n c e   1   ! '   2   s t a l l s
d i s t a n c e   2   ! '   1   s t a l l
d i s t a n c e   3 +   ! '   0   s t a l l s
```

under same-cycle WB! allowed.

This is a useful shortcut.

21.35 But Don't Use Distance Rule Blindly

It assumes:

- result written in WB,
- operands read in ID,
- fixed 5-stage pipeline,
- write-first/read-second register file.

Different problem assumptions != different result.

PART B — NOP INSERTION

21.36 What Is a NOP?

NOP = No Operation

It occupies an instruction slot but performs no useful computation.

Compiler/software can insert NOPs to create enough distance between dependent instructions.

21.37 Example Without Forwarding

Original:

```
TEXT
ADD R1, R2, R3
SUB R4, R1, R5
```

May be transformed conceptually:

```
TEXT
ADD R1, R2, R3
NOP
NOP
SUB R4, R1, R5
```

if two empty cycles are needed.

21.38 Disadvantage of NOPs

NOPs waste:

- instruction slots,
- code space,
- pipeline cycles.

Better approach:

move useful independent instructions into the gap.

This is instruction scheduling.

PART C — COMPILER SCHEDULING

21.39 Instruction Scheduling

Compiler rearranges independent instructions to reduce stalls while preserving program correctness.

Example:

TEXT

```
LW  R1, 0(R2)
ADD R3, R1, R4
SUB R5, R6, R7
```

Immediate load-use causes one stall.

Reorder independent SUB:

TEXT

```
LW  R1, 0(R2)
SUB R5, R6, R7
ADD R3, R1, R4
```

Now there is one instruction gap.

Load data becomes available in time.

Potential stall:

$\boxed{0}$

with standard forwarding.

21.40 Scheduling Must Preserve Semantics

Compiler cannot move instructions arbitrarily.

It must respect:

- true dependencies,
- memory dependencies,
- control dependencies.

Example:

TEXT

```
I1: R1 = ...
I2: use R1
```

I2 cannot move before I1.

RAW represents actual data flow.

21.41 Scheduling Memory Trick

Don't waste the bubble—fill it with independent work.

PART D — MULTIPLE DEPENDENCIES

21.42 Dependency Chain

TEXT

```
I1: ADD R1, R2, R3
I2: SUB R4, R1, R5
I3: AND R6, R4, R7
I4: OR R8, R6, R9
```

Every instruction uses result of previous instruction.

With standard ALU forwarding:

- I 1! I forwarded
- I 2! I forwarded
- I 3! I forwarded

Potentially:

$\boxed{0}$ stalls

despite many RAW dependencies.

Important:

many RAW dependencies do not automatically mean many stalls.

21.43 Load Followed by Chain

TEXT

```
I1: LW R1, 0(R2)
I2: ADD R3, R1, R4
I3: SUB R5, R3, R6
I4: AND R7, R5, R8
```

With forwarding:

- I 1! I load-use = 1 stall
- I 2! I ALU = 0
- I 3! I ALU = 0

Total:

$\boxed{1}$ stall

21.44 Multiple Producers for Same Register

TEXT

```
I1: ADD R1, R2, R3
I2: SUB R1, R4, R5
I3: AND R6, R1, R7
```

I3 must consume value from I2, not I1.

This is called using the:

$\boxed{\text{youngest / most recent producer}}$

Forwarding logic must select correct source.

21.45 Dependency Between Nonadjacent Instructions

TEXT

```
I1: ADD R1, ...
I2: independent
I3: SUB ..., R1, ...
```

With forwarding:

$\boxed{0 \text{ stalls}}$

Without forwarding under classic assumptions:

$\boxed{1 \text{ stall}}$

This is a common comparison question.

PART E — EXACT TIMING TABLE

21.46 Standard 5-Stage Availability

Let's label:

TEXT

IF | ID | EX | MEM | WB

For ALU producer:

TEXT

result available after EX

For load:

TEXT

result available after MEM

Consumer ALU:

TEXT

needs operand at EX

This alone solves many GATE questions.

21.47 ALU's ALU with Forwarding

Producer:

TEXT

I 1 E X ! ' r e s u l t

Consumer:

TEXT

I2 EX one cycle later

Hence:

$\boxed{0 \text{ stalls}}$

21.48 L O A D ! ' A with Forwarding

Producer:

TEXT

I 1 M E M ! ' r e s u l t

Consumer's immediate EX occurs same cycle.

Too early.

Delay consumer by one cycle.

Hence:

$\boxed{1 \text{ stall}}$

21.49 L O A D ! ' A with One Gap

Independent instruction shifts consumer EX one cycle later.

Now:

$\text{ProducerMEM} \rightarrow \text{ConsumerEX}$

works.

Hence:

$\boxed{0 \text{ stalls}}$

PART F — HAZARD DETECTION HARDWARE

21.50 Hazard Detection Unit

Hardware may examine:

- source register numbers in ID,
- destination of preceding load,
- control signals.

Classic load-use detection idea:

If instruction in EX is a load and its destination matches a source of instruction in ID:

$\boxed{\text{stall}}$

21.51 Conceptual Load-Use Condition

Something like:

$ID/EX.MemRead=1$

and:

$ID/EX.Rt=IF/ID.Rs$

or:

$ID/EX.Rt=IF/ID.Rt$

depending on instruction semantics.

Then hazard unit inserts bubble.

You only need the logic concept unless detailed MIPS pipeline is asked.

21.52 What Happens During Load-Use Stall?

Typical control:

- freeze PC,
- freeze IF/ID register,
- insert bubble into EX.

Conceptually:

TEXT

Producer continues
Consumer waits
Next fetch waits
Bubble enters EX

PART G — FORWARDING AND MEMORY

21.53 Forwarding Cannot Travel Back in Time

This is the central limitation.

A value must exist **before** or at the consumer's need time.

If value is generated after consumer needs it:

$\boxed{\text{forwarding alone cannot solve it}}$

Hence load-use stall.

21.54 Forwarding Is Not Magic

Question:

Why not forward load value from MEM to consumer EX in same cycle?

Because in classic timing:

- load data is ready only near end of MEM cycle,
- consumer needs operand at beginning of EX cycle.

The consumer needs it too early.

PART H — STALL COUNT NUMERICALS

21.55 Example — No Forwarding

Classic assumptions.

TEXT

```
I1: ADD R1, R2, R3
I2: SUB R4, R1, R5
I3: AND R6, R1, R7
```

I2 depends on I1.

I3 also depends on I1.

Once I2 is stalled, later instructions may also be held behind it.

Do not independently add stalls for every dependency without drawing timing.

21.56 Why Independent Addition Can Overcount

If one stall delays the entire front of pipeline, that same delay may already make later consumers safe.

Thus:

Always simulate the sequence or track availability.

21.57 Example with Forwarding

TEXT

```
I1: LW R1, 0(R2)
I2: ADD R3, R1, R4
I3: SUB R5, R1, R6
```

I2 immediate load-use !' 1 stall.

That stall also shifts I3 one cycle.

I3 may then receive load value without additional stall.

Total:

$\boxed{1}$

not 2.

21.58 Example — Two Independent Load-Use Hazards

TEXT

```
I1: LW   R1, 0(R2)
I2: ADD  R3, R1, R4
```

```
I3: LW   R5, 0(R6)
I4: SUB  R7, R5, R8
```

With forwarding:

- I1' I2 stall
- I3' I4 stall

Total:

$\boxed{2 \text{ stalls}}$

21.59 Example — Scheduling Removes Both?

If suitable independent work exists, compiler might interleave instructions:

TEXT

```
LW   R1, 0(R2)
LW   R5, 0(R6)
ADD  R3, R1, R4
SUB  R7, R5, R8
```

Now each load has one instruction gap before use.

Potential stalls:

$\boxed{0}$

with standard forwarding.

21.60 GATE Numerical — Pipeline Cycles

5-stage pipeline.

100 instructions.

There are 20 load-use pairs, each causing one stall.

Assume no overlap reduction and no other hazards.

Ideal:

$\boxed{5+100-1=104}$

Stalls:

$\lfloor 20 \rfloor$

Total:

$\lfloor \boxed{124 \text{ cycles}} \rfloor$

21.61 Long-Program CPI

20% instructions cause one load-use stall.

Average:

$\lfloor 0.2(1)=0.2 \rfloor$

$\lfloor \boxed{\text{CPI} \approx 1.2} \rfloor$

21.62 Without Forwarding CPI Example

Suppose 25% instructions immediately depend on previous ALU instruction.

Without forwarding, 2 stalls each.

Average:

$\lfloor 0.25(2)=0.5 \rfloor$

Thus:

$\lfloor \boxed{\text{CPI} \approx 1.5} \rfloor$

under classic assumptions.

21.63 With Forwarding Same Workload

Immediate ALU dependencies need zero stalls.

Thus contribution:

$\lfloor 0 \rfloor$

Potential CPI:

$\lfloor \boxed{1} \rfloor$

ignoring other hazards.

This shows forwarding's benefit.

21.64 Mixed Workload

Suppose:

- 20% immediate ALU RAW dependencies !' 0 stalls with forwarding
- 10% immediate load-use dependencies !' 1 stall
- branches contribute 0.15 cycles/instruction

Then:

$\text{CPI} = 1 + 0 + 0.10 + 0.15$

$\boxed{1.25}$

PART I — FORWARDING PATH QUESTIONS

21.65 Example

TEXT

I1: ADD R1, R2, R3
I2: SUB R4, R1, R5

Required:

$\boxed{\text{EX/MEM} \rightarrow \text{EX}}$

because newest result comes from previous ALU instruction.

21.66 Example with One Gap

TEXT

I1: ADD R1, R2, R3
I2: OR R6, R7, R8
I3: SUB R4, R1, R5

I1 result is farther ahead.

Forward may come from:

$\boxed{\text{MEM/WB} \rightarrow \text{EX}}$

depending on exact timing.

21.67 Load-Use After Stall

TEXT

I1: LW R1, 0(R2)
I2: ADD R3, R1, R4

After one stall, loaded value may be forwarded:

$\boxed{\text{MEM/WB} \rightarrow \text{EX}}$

or equivalent memory-result path depending on architecture diagram.

PART J — GATE TRAPS

21.68 Trap — Forwarding Eliminates Every RAW Stall

'L Wrong.

Classic immediate load-use still usually needs one stall.

21.69 Trap — Every Load Followed by Any Instruction Stalls

Wrong.

Only if the following instruction actually needs the loaded value early enough.

Example:

TEXT

```
LW R1, 0(R2)
ADD R5, R6, R7
```

No dependency.

No load-use stall.

21.70 Trap — One Stall per Dependency

Wrong.

Dependencies and stalls are not one-to-one.

Timing may allow:

- zero stalls,
- shared stalls,
- multiple stalls.

21.71 Trap — Store Doesn't Read Registers

Wrong.

Store usually reads:

- base register,
- data register.

21.72 Trap — Forwarding Removes WAR/WAW

Forwarding is mainly for RAW true dependencies.

WAR/WAW are name dependencies usually addressed by renaming/order controls.

21.73 Trap — NOP and Stall Are Completely Different Performance-Wise

Both can create an empty useful-work slot.

NOP is explicit instruction/software approach; stall/bubble is often hardware-generated.

21.74 Trap — Reordering Instructions Is Always Safe

No.

Compiler must preserve dependencies and program semantics.

21.75 Trap — ALU Result Available Only in WB

Wrong for forwarding purposes.

ALU result exists after EX and can often be bypassed earlier.

21.76 Trap — Load Data Available After EX

Wrong.

EX computes address.

Actual load data generally appears after MEM.

21.77 Trap — Branch Data Hazard Uses Same Timing as ALU Consumer

Not necessarily.

If branch compares in ID, its operands are needed earlier than an ALU instruction's EX operands.

21.78 Fast GATE Decision Tree

When two instructions share a register:

Step 1

Is it RAW?

If no !' ordinary forwarding won't be relevant.

Step 2

What instruction produces value?

- ALU !' ready after EX
- LOAD !' ready after MEM

Step 3

When does consumer need it?

- ALU/base address !' EX
- store data !' often MEM
- branch !' ID or EX depending design

Step 4

Is forwarding path available?

If yes and timing works !' 0 stall.

If value arrives too late !' stall.

21.79 Fast Classic Result Sheet

Under standard 5-stage MIPS-like assumptions with full ordinary forwarding:

TEXT									
A L U	!	A L U	i m m e d i a t e	:	0	s t a l l s			
A L U	!	A L U	w i t h g a p	:	0				
L O A D	!	i m m e d i a t e	A L U	:	1				
L O A D	!	A L U	w i t h o n e g a p	:	0				

Without forwarding, same-cycle WB-read allowed:

TEXT									
P r o d u c e r	!	n e x t	c o n s u m e r	:	2	s t a l l s			
P r o d u c e r	!	o n e - g a p	c o n s u m e r	:	1				
P r o d u c e r	!	t w o - g a p	c o n s u m e r	:	0				

21.80 Full Timing Example — Forwarding

Instructions:

TEXT									
I1:	ADD	R1,R2,R3							
I2:	SUB	R4,R1,R5							
I3:	AND	R6,R4,R7							

Timeline:

TEXT									
C y c l e	!	1	2	3	4	5	6	7	
I1	IF	ID	EX	MEM	WB				
I2		IF	ID	EX	MEM	WB			
I3			IF	ID	EX	MEM	WB		

Forward:

TEXT									
I 1	E X	!	I 2	E X					
I 2	E X	!	I 3	E X					

No stalls.

21.81 Full Timing Example — Load Use

TEXT									
I1:	LW	R1,0(R2)							
I2:	ADD	R3,R1,R4							


```
I3: SUB R5, R6, R7
```

With forwarding:

TEXT

Cycle	1	2	3	4	5	6	7	8
I1	IF	ID	EX	MEM	WB			
I2		IF	ID	ST	EX	MEM	WB	
I3			IF	ST	ID	EX	MEM	WB

One stall delays consumer and following fetch/decode.

Total extra cycles:

$\boxed{1}$

21.82 Full Timing Example — Scheduled

Reorder:

TEXT

```
I1: LW R1, 0(R2)
I2: SUB R5, R6, R7
I3: ADD R3, R1, R4
```

Timeline:

TEXT

I1:	IF	ID	EX	MEM	WB		
I2:		IF	ID	EX	MEM	WB	
I3:			IF	ID	EX	MEM	WB

No stall.

CHAPTER 21 — MASTER FORMULAS

Long-stream CPI

$\boxed{CPI \approx 1 + \sum_i f_i P_i}$

where:

- f_i = frequency of hazard type,
- P_i = stall cycles per occurrence.

Pipeline total cycles

$\boxed{Cycles = k + n - 1 + \text{TotalStalls}}$

Execution time

$\boxed{T = Cycles \times T_{\text{clock}}}$

CHAPTER 21 — MEMORY TRICKS

Forwarding

Don't wait for WB — take result directly.

Load-use

LOAD is one stage late.

Scheduling

Fill bubbles with useful independent work.

No forwarding

Wait until register file has the result.

Safe method

Ready when? Needed when? Path exists?

CHAPTER SNAPSHOT

1. Stalling delays a consumer until its operand is safe.
2. Forwarding bypasses register write-back.
3. ALU result becomes available after EX.
4. Load data becomes available after MEM.
5. Immediate ALU RAW often needs zero stalls with forwarding.
6. Immediate load usually needs one stall.
7. A one-instruction gap often removes load-use stall.
8. Without forwarding, consumers may need to wait until producer WB.
9. Same-cycle WB write/ID read convention affects stall count.
10. NOPs can create spacing but waste slots.
11. Compiler scheduling can fill hazard gaps with useful instructions.
12. Forwarding logic must choose the newest matching producer.
13. Store and branch timing may differ from normal ALU consumers.
14. Stalls must be counted from exact timing, not number of dependencies.
15. One stall may simultaneously make multiple later dependencies safe.

MUST REMEMBER

$\boxed{\text{ALU result ready after EX}}$

$\boxed{\text{LOAD result ready after MEM}}$

and under classic full forwarding:

$\boxed{\text{A! 0 Stalls}}$

$\boxed{\text{LOAD! use m1 stall}}$

PRACTICE SET

Q1 — L0

With full forwarding:

TEXT

I1: ADD R1, R2, R3
I2: SUB R4, R1, R5

How many stalls in a classic 5-stage pipeline?

Q2 — L0

With full forwarding:

TEXT

I1: LW R1, 0(R2)
I2: ADD R3, R1, R4

How many stalls?

Q3 — L1

Why does Q2 need a stall even though forwarding exists?

Q4 — L1

With full forwarding:

TEXT

I1: LW R1, 0(R2)
I2: ADD R5, R6, R7
I3: SUB R3, R1, R4

How many stalls?

Q5 — L1

Without forwarding, assume WB writes in first half and ID reads in second half.

TEXT

I1: ADD R1, R2, R3
I2: SUB R4, R1, R5

How many stalls?

Q6 — L1

Same assumptions:

TEXT

I1: ADD R1, R2, R3
I2: independent
I3: SUB R4, R1, R5

How many stalls?

Q7 — L2

Identify appropriate forwarding source conceptually:

TEXT

I1: ADD R1, R2, R3
I2: SUB R4, R1, R5

Q8 — L2

Sequence:

TEXT

I1: LW R1, 0(R2)
I2: ADD R3, R1, R4
I3: SUB R5, R3, R6

With full forwarding, find total stalls.

Q9 — L2

Sequence:

TEXT

I1: LW R1, 0(R2)
I2: LW R5, 0(R6)
I3: ADD R3, R1, R4
I4: SUB R7, R5, R8

With full forwarding, find total stalls.

Q10 — L2

Can the following be rescheduled to remove a load-use stall?

TEXT

```
LW  R1, 0(R2)
ADD  R3, R1, R4
SUB  R5, R6, R7
```

If yes, write a safe reordered sequence.

Q11 — L2 NAT

A 5-stage pipeline executes 200 instructions. There are 30 load-use hazards, each causing exactly one stall.

Find total cycles from an empty pipeline.

Q12 — L3 NAT

For Q11, find exact CPI.

Q13 — L3

A workload has:

- 15% immediate load-use dependencies, 1 stall each,
- 25% immediate ALU dependencies, fully solved by forwarding,
- control hazards contributing 0.12 cycles/instruction.

Find approximate CPI.

Q14 — L3 Conceptual

Why can forwarding solve an A D D! ' A D dependency but not an immediate L O A D! ' A D dependency in the standard pipeline?

CHAPTER 21 COMPLETION STATUS

' Stalling ' Bubbles ' No-forwarding timing ' Register-file same-cycle timing ' Forwarding/bypassing ' E X / M E M! ' E X forwarding ' M E M / W B! ' Forwarding ' Newest-producer priority ' Load-use hazard ' One-cycle load-use stall ' Store timing considerations ' Branch timing considerations ' NOP insertion ' Compiler scheduling ' Dependency chains ' Hazard detection ' CPI/stall numericals ' GATE traps

CHAPTER 21: COMPLETE '

COURSE PROGRESS

Completed: 22 / 26 modules

Approximate roadmap progress:

$\sim 85\%$

NEXT !' CHAPTER 22 — CONTROL HAZARD & BRANCH NUMERICALS

Next we complete the hazard section with:

Branch resolution stage Pipeline flushes Predict taken / not taken Prediction accuracy Misprediction penalty Branch frequency CPI due to branches Delayed branching Branch-target timing Exact GATE control-hazard numericals.

USER

next

ASSISTANT

CHAPTER 22 — CONTROL HAZARDS & BRANCH NUMERICALS

Classification: [GATE CORE] **Priority:** +HIGH PRIORITY / Numerically important **Official syllabus connection:** Pipeline hazards — control hazards

Control hazards mainly arise because of:

- branches,
- jumps,
- returns,
- exceptions.

For GATE, branches are the major focus.

Core problem:

The processor may not know the correct next PC early enough.

22.1 Control Hazard Intuition

Consider:

TEXT

```
BEQ R1, R2, TARGET
ADD R3, R4, R5
SUB R6, R7, R8
...
TARGET:
MUL R9, R10, R11
```

When `BEQ` is fetched, CPU may not yet know:

- whether branch is taken,
- branch target,

- whether next sequential instruction should execute.

But the pipeline wants to keep fetching every cycle.

That uncertainty creates a:

$\boxed{\text{Control Hazard}}$

22.2 Classic 5-Stage Pipeline

Recall:

$IF \rightarrow ID \rightarrow EX \rightarrow MEM \rightarrow WB$

Suppose branch decision is known in **EX**.

Timeline:

TEXT				
C y c l e ! ' 1 2 3				
Branch	IF	ID	EX	
I2		IF	ID	
I3			IF	

By the time branch is resolved in EX, two younger instructions may already be in pipeline.

If they belong to wrong path, they must be discarded.

22.3 Pipeline Flush

Discarding wrong-path instructions is called:

$\boxed{\text{Flushing}}$

Example:

TEXT				
Branch taken				
!"				
I 2	w r o n g	!"	F L U S H	
I 3	w r o n g	!"	F L U S H	
!"				
Fetch TARGET				

These lost cycles contribute to:

$\boxed{\text{Branch Penalty}}$

22.4 Branch Penalty

Branch penalty means:

number of useful pipeline cycles lost because of a branch/control hazard.

It depends on:

- where branch is resolved,
- prediction method,
- branch direction,
- target availability,
- pipeline organization.

Do **not** memorize one universal penalty.

22.5 Stall Until Resolved

Simplest technique:

Stop fetching until branch outcome is known.

Suppose branch decision takes 2 cycles after fetch.

Then every branch may introduce:

$\boxed{2 \text{ stall cycles}}$

If branch fraction is f_b :

$\boxed{\text{AverageBranchStall} = f_b P}$

where P is stall cycles/branch.

22.6 Example — Always Stall

20% instructions are branches.

Each branch creates 2 cycles of stall.

$\text{AverageStall} = 0.2(2)$

$= 0.4$

If ideal CPI = 1:

$\boxed{\text{CPI} = 1.4}$

22.7 Static Branch Prediction

Instead of stopping, CPU can guess branch behavior.

Two simple strategies:

1. Predict Not Taken
2. Predict Taken

22.8 Predict Not Taken

Assume branch will continue sequentially.

So CPU keeps fetching:

TEXT

PC+4

PC+8

...

If branch really is not taken:

$\boxed{\text{Prediction correct}}$

If branch is taken:

$\boxed{\text{Misprediction}}$

and wrong sequential instructions are flushed.

22.9 Predict Taken

Assume branch will jump to target.

If branch really is taken:

$\boxed{\text{Correct}}$

If not:

$\boxed{\text{Misprediction}}$

Correct target address must also be available soon enough for full benefit.

22.10 Prediction Accuracy

If:

$\text{Accuracy} = a$

then:

$\boxed{\text{MispredictionRate} = 1 - a}$

Example:

Accuracy:

90%

Misprediction:

$\boxed{10\%}$

22.11 Master Branch CPI Formula

Let:

- f_b = fraction of instructions that are branches
- m = misprediction rate among branches
- P = cycles lost per misprediction

Then:

$$\boxed{\text{BranchStallCycles/Instruction} = f_{\text{bmP}}}$$

and:

$$\boxed{\text{CPI} = \text{CPI}_{\text{base}} + f_{\text{bmP}}}$$

One of the most important pipeline formulas.

22.12 Example

Branch fraction:

$$20\% = 0.2$$

Prediction accuracy:

$$90\%$$

Thus:

$$m = 0.1$$

Penalty:

$$3$$

Then:

$$\text{BranchStall} = 0.2(0.1)(3)$$

$$= 0.06$$

If base CPI = 1:

$$\boxed{\text{CPI} = 1.06}$$

22.13 Why Multiply Three Factors?

Because penalty occurs only when:

1. instruction is a branch,
2. branch is mispredicted,
3. each misprediction loses (P) cycles.

Therefore:

$$\boxed{f_{\text{b}} \times m \times P}$$

22.14 Predict-Not-Taken Formula

Suppose:

- fraction branches = (f_{b})
- fraction of branches actually taken = (t)
- predict not taken
- taken branch penalty = (P)

Then wrong predictions happen for taken branches.

$$\boxed{CPI = CPI_{\text{base}} + f_{\text{bt}}P}$$

22.15 Example

25% instructions are branches.

60% branches taken.

Predict not taken.

Penalty:

$$2$$

Average branch penalty:

$$0.25(0.60)(2)$$

$$= 0.30$$

Thus:

$$\boxed{CPI = 1.30}$$

22.16 Predict-Taken Formula

If fraction of branches taken is t , then not-taken fraction:

$$1 - t$$

If always predict taken:

$$\boxed{CPI = CPI_{\text{base}} + f_b(1 - t)P}$$

assuming only incorrect predictions incur the stated penalty.

22.17 Example

Branch fraction:

$$0.25$$

Taken:

$$0.70$$

Predict taken.

Wrong fraction among branches:

$$0.30$$

Penalty:

$$2$$

$$\text{Stall} = 0.25(0.30)(2)$$

$$= 0.15$$

$$\boxed{CPI = 1.15}$$

22.18 Which Static Prediction Is Better?

If branches are taken more than 50% of the time:

predict taken tends to be better.

If branches are mostly not taken:

predict not taken tends to be better.

But branch-target timing may affect actual performance.

22.19 Compare Static Predictors

Suppose:

- branches = 20%
- 80% are taken
- penalty = 3 cycles.

Predict Not Taken

$$\lfloor 0.2(0.8)(3) = 0.48 \rfloor$$

$$\lfloor \text{CPI} = 1.48 \rfloor$$

Predict Taken

Wrong fraction:

$$\lfloor 0.2 \rfloor$$

$$\lfloor 0.2(0.2)(3) = 0.12 \rfloor$$

$$\lfloor \text{CPI} = 1.12 \rfloor$$

Predict taken is much better here.

22.20 Dynamic Branch Prediction

Modern processors can predict based on past branch behavior.

Basic idea:

TEXT

Past outcomes

!"

Predict next outcome

For GATE, you may encounter:

- prediction accuracy,
 - 1-bit predictor,
-

- 2-bit predictor.

The most important numerical tool remains:

$\boxed{\text{branch frequency} \times \text{misprediction rate} \times \text{penalty}}$

22.21 1-Bit Predictor — Basic Idea

A 1-bit predictor remembers last branch outcome.

If branch was taken last time:

predict taken next time.

If not taken last time:

predict not taken.

This works for repetitive behavior but can fail twice around loop transitions.

[SUPPORT]

22.22 2-Bit Saturating Counter

A common predictor uses four states:

TEXT

Strongly Not Taken
Weakly Not Taken
Weakly Taken
Strongly Taken

Two consecutive opposite outcomes are generally needed to strongly change direction.

This makes predictor less sensitive to one unusual outcome.

[SUPPORT]

For your listed syllabus, knowing the concept is enough unless a question gives explicit state transitions.

22.23 Branch Target Buffer — Support

A **BTB** can cache branch target information.

Why?

Predicting "taken" is useful only if CPU also knows where to fetch from.

BTB helps provide target address quickly.

[SUPPORT / PLACEMENT]

22.24 Earlier Branch Resolution

Suppose originally branch resolves in EX.

If redesign resolves it in ID, fewer wrong instructions enter pipeline.

Thus branch penalty can decrease.

Example:

TEXT

```
R e s o l v e d   i n   E X   ! '   m a y b e   2   w r o n g   y o u n g e r   i n s t r u c t i o n s
R e s o l v e d   i n   I D   ! '   m a y b e   1   w r o n g   y o u n g e r   i n s t r u c t i o n
```

Exact number depends on pipeline.

22.25 Branch Penalty from Resolution Stage

Simplified intuition:

If branch resolves late:

$\boxed{\text{Penalty larger}}$

If resolves early:

$\boxed{\text{Penalty smaller}}$

But derive from timing diagram rather than blindly using stage number.

22.26 Example — Branch Resolved in EX

Classic:

TEXT

```
Cycle 1: Branch IF
Cycle 2: Branch ID, next instruction IF
Cycle 3: Branch EX, next in ID, another in IF
```

If branch taken, the two younger sequential instructions are wrong.

Potential flush:

$\boxed{2 \text{ instructions}}$

Often interpreted as:

$\boxed{2 \text{ cycle misprediction penalty}}$

under simple assumptions.

22.27 Example — Branch Resolved in ID

TEXT

```
Cycle 1: Branch IF
Cycle 2: Branch ID, next instruction IF
```

At end of ID, only one younger instruction may need flushing.

Possible penalty:

$\boxed{1 \text{ cycle}}$

again under simplified assumptions.

22.28 Delayed Branching

[LOW PRIORITY / SUPPORT]

Some older architectures define one or more **branch delay slots**.

Instruction immediately after branch executes regardless of whether branch is taken.

Example:

TEXT

```
BEQ R1, R2, L
    ADD R3, R4, R5      !• delay slot
```

Compiler tries to place useful instruction in this slot.

22.29 Branch Delay Slot

If delay slot can be filled with a useful independent instruction:

$\boxed{\text{branch penalty can be reduced}}$

If no useful instruction exists:

TEXT

NOP

may be inserted.

Historical concept; not worth major study time.

22.30 Flush vs Misprediction Penalty

Suppose branch prediction is wrong and 3 younger instructions are discarded.

If each lost pipeline slot corresponds to one cycle:

$\boxed{\text{Penalty}=3 \text{ cycles}}$

But exact timing can be more subtle.

Always use given penalty when explicitly stated.

22.31 Branch Frequency vs Taken Rate

Do not confuse:

Branch frequency

Fraction of **all instructions** that are branches.

Example:

$$\boxed{20\%}$$

Taken rate

Fraction of **branches** that are taken.

Example:

$$\boxed{60\%}$$

Fraction of all instructions that are taken branches:

$$0.20 \times 0.60$$

$$\boxed{12\%}$$

22.32 Example

1,000,000 instructions.

20% are branches.

So branches:

$$200000$$

60% taken:

$$120000$$

If predict not taken:

$$120000$$

mispredictions.

If penalty = 2:

$$240000$$

lost cycles.

22.33 CPI from Counts

Base cycles approximately:

$$1,000,000$$

Added branch cycles:

$$240000$$

Approximate total:

$$1,240,000$$

$$CPI = \frac{1,240,000}{1,000,000}$$

$$\boxed{1.24}$$

Same result as formula:

$$\sqrt[1+0.2(0.6)(2)=1.24]{}$$

22.34 Misprediction Count

For:

- (N) instructions
- branch frequency (f_b)
- misprediction rate (m)

$$\boxed{\text{NumberOfMispredictions} = N f_b m}$$

22.35 Lost Cycles

$$\boxed{\text{LostCycles} = N f_b m P}$$

22.36 Example

$$(N = 10^8)$$

Branches:

$$15\%$$

Accuracy:

$$92\%$$

Misprediction:

$$8\%$$

Penalty:

$$4$$

Mispredictions:

$$10^8 (0.15)(0.08)$$

$$= 1.2 \times 10^6$$

Lost cycles:

$$1.2 \times 10^6 (4)$$

$$\boxed{4.8 \times 10^6}$$

22.37 Approximate CPI

$$1 + 0.15(0.08)(4)$$

$$= 1.048$$

$$\boxed{1.048}$$

22.38 CPU Time Due to Branches

Suppose:

- $IC = 10^9$
- $CPI = 1.12$
- $\text{Clock} = 2.8 \text{ GHz}$

$$CPUTime = \frac{10^9(1.12)}{2.8 \times 10^9}$$

$$\boxed{0.4s}$$

22.39 Compare Two Predictors

Predictor A:

$$95\% \text{ accuracy}$$

Predictor B:

$$90\% \text{ accuracy}$$

Branch fraction:

$$20\%$$

Penalty:

$$4$$

A

$$CPI = 1 + 0.2(0.05)(4)$$

$$= 1.04$$

B

$$CPI = 1 + 0.2(0.10)(4)$$

$$= 1.08$$

A is better.

22.40 Speedup from Better Predictor

Same clock and IC.

Old CPI:

$$1.08$$

New CPI:

$$1.04$$

Speedup:

$$\frac{1.08}{1.04}$$

$$\boxed{\approx 1.0385 \times}$$

about 3.85%.

22.41 Why Small Accuracy Gains Matter

Only branches are affected, but in deep pipelines:

$$P$$

can be large.

So even a small reduction in misprediction rate may save many cycles.

22.42 Branch Penalty and Pipeline Depth

Deeper pipeline often resolves branches after more pipeline stages.

Potentially:

$$\text{PipelineDepth} \uparrow \rightarrow \text{MispredictionPenalty} \uparrow$$

This is one trade-off of deep pipelines.

22.43 Structural + Data + Control CPI

Suppose:

- Base CPI = 1
- data stalls = 0.15/instruction
- structural stalls = 0.05
- branch frequency = 20%
- branch misprediction = 10%
- penalty = 3

Branch:

$$0.2(0.1)(3)=0.06$$

Total:

$$1+0.15+0.05+0.06$$

$$\boxed{1.26}$$

22.44 Cache + Branch CPI

Suppose:

- Base CPI = 1
- cache stalls = 0.4
- branch stalls = 0.1

Then:

$$\boxed{\text{CPI}=1.5}$$

assuming additive average penalties.

This connects pipeline + cache questions.

22.45 Branch Resolution Probability Trap

Sometimes problem says:

30% instructions are branches; 70% branches taken; only taken branches incur 2-cycle penalty.

Then:

$$\text{Penalty} = 0.3(0.7)(2)$$

not:

$$0.7(2)$$

because only 30% of all instructions are branches.

22.46 Accuracy Trap

If predictor accuracy = 85%:

$$m = 1 - 0.85 = 0.15$$

Do not use 0.85 as penalty probability.

22.47 Percent Conversion Trap

$$20\% = 0.20$$

$$5\% = 0.05$$

If both multiply:

$$0.20(0.05) = 0.01$$

which means only:

$$1\%$$

of all instructions are mispredicted branches.

22.48 Example

Branch frequency:

$$20\%$$

Misprediction:

$$5\%$$

Then mispredicted branch instructions as fraction of all instructions:

$$0.20(0.05)$$

$$\boxed{1\%}$$

If penalty 4:

$$\lfloor 0.04 \text{ cycles/instruction} \rfloor$$

22.49 Branch Target Known Later Than Direction

Sometimes branch direction may be known but target not yet available.

Then even correct "taken" prediction may not fully eliminate penalty.

For GATE:

Follow exactly what the question says about direction and target availability.

Do not assume prediction automatically means zero cost.

22.50 Correct Prediction Penalty

Some questions may state:

- correct prediction costs 1 cycle,
- misprediction costs 4 cycles.

Then average branch penalty is not just misprediction formula.

Use weighted average:

$$\lfloor \boxed{f_b [aP_c + (1-a)P_m] } \rfloor$$

where:

- $\lfloor a \rfloor$ = prediction accuracy
- $\lfloor P_c \rfloor$ = correct-prediction cost
- $\lfloor P_m \rfloor$ = wrong-prediction cost

22.51 Example

Branch fraction:

$$\lfloor 0.2 \rfloor$$

Accuracy:

$$\lfloor 0.9 \rfloor$$

Correct prediction penalty:

$$\lfloor 1 \rfloor$$

Wrong prediction penalty:

$$\lfloor 4 \rfloor$$

Average branch penalty:

$$\lfloor 0.2[0.9(1)+0.1(4)] \rfloor$$

$$\lfloor =0.2(1.3) \rfloor$$

$$\lfloor =0.26 \rfloor$$

CPI:

$$\lfloor \boxed{1.26} \rfloor$$

22.52 Universal Weighted Formula

If different branch outcomes have different costs:

$$\lfloor \boxed{\text{AveragePenalty} = \sum_i \text{Probability}_i \times \text{Cost}_i} \rfloor$$

Then add to base CPI as appropriate.

This is safer than memorizing special formulas.

22.53 Example — Predict Not Taken with Different Costs

Branches:

$$\lfloor 20\% \rfloor$$

Taken:

$$\lfloor 60\% \rfloor$$

Not taken:

$$\lfloor 40\% \rfloor$$

Suppose:

- correct not-taken branch costs 0
- taken branch costs 3

Then:

$$\lfloor \text{BranchStall} = 0.2[0.4(0) + 0.6(3)] \rfloor$$

$$\lfloor =0.36 \rfloor$$

$$\lfloor \boxed{\text{CPI} = 1.36} \rfloor$$

22.54 Example — Predict Taken, Target Delay

Suppose:

- branch frequency = 20%
- taken = 70%
- correct taken branch still costs 1 cycle for target fetch
- incorrect not-taken costs 3 cycles

Average:

$$\lfloor 0.2[0.7(1) + 0.3(3)] \rfloor$$

$$\lfloor =0.2(1.6) \rfloor$$

$$\lfloor =0.32 \rfloor$$

$\boxed{CPI=1.32}$

22.55 Pipeline Flush Count

If branch resolves in stage r , a simplistic pipeline diagram may show how many younger instructions entered before resolution.

Do not blindly use:

$r-1$

unless stage numbering/timing supports it.

Draw:

TEXT

Branch
Next1
Next2
...

and count wrong-path instructions explicitly.

22.56 Exact Timing Method

For a control hazard question:

1. Draw branch across stages.
2. Mark stage when direction becomes known.
3. Mark instructions fetched behind branch.
4. Identify which are wrong on misprediction.
5. Count lost cycles/flushes.

This is the safest way.

22.57 Example — Decision in MEM

Suppose branch is resolved only in MEM stage:

TEXT

	C	y	c	l	e	!	1	2	3	4
B						IF	ID	EX	MEM	
I2							IF	ID	EX	
I3								IF	ID	
I4									IF	

At resolution, up to three younger instructions may be present.

A taken/mispredicted branch may require flushing:

$\boxed{3}$

under this simple model.

22.58 Earlier Resolution Benefit

If branch moved from MEM to EX:

Potential wrong instructions decrease from:

$$3 \rightarrow 2$$

If moved to ID:

$$2 \rightarrow 1$$

This directly lowers misprediction cost.

22.59 Branch Frequency from Counts

Suppose program has:

- 1,000 instructions
- 150 branches.

$$f_b = \frac{150}{1000}$$

$$\boxed{0.15}$$

If 30 branch mispredictions:

$$m = \frac{30}{150}$$

$$\boxed{0.20}$$

Do not calculate misprediction rate over all instructions unless specifically desired.

22.60 Overall Misprediction Fraction

Overall fraction of instructions that are mispredicted branches:

$$f_{bm}$$

Previous:

$$0.15(0.20)$$

$$\boxed{0.03}$$

3% of all instructions.

22.61 GATE Numerical — CPI

Given:

- Base CPI = 1
- 15% branches
- 80% prediction accuracy
- 4-cycle misprediction penalty

$$m = 0.2$$

$$\text{BranchPenalty} = 0.15(0.2)(4)$$

$$= 0.12$$

$$\boxed{\text{CPI} = 1.12}$$

22.62 GATE Numerical — Predict Not Taken

Branches:

$$30\%$$

Taken:

$$40\%$$

Predict not taken.

Penalty:

$$2$$

$$\text{Stall} = 0.3(0.4)(2)$$

$$= 0.24$$

$$\boxed{\text{CPI} = 1.24}$$

22.63 GATE Numerical — Predict Taken

Same workload:

Branches:

$$0.3$$

Not taken:

$$0.6$$

Penalty:

$$2$$

$$\text{Stall} = 0.3(0.6)(2)$$

$$= 0.36$$

$$\boxed{\text{CPI} = 1.36}$$

Thus predict not taken is better because most branches are not taken.

22.64 GATE Numerical — Execution Time

Program:

$$5 \times 10^8$$

instructions.

Branch-adjusted CPI:

$$\sqrt[1.2]{}$$

Clock:

$$\sqrt[3]{\text{GHz}}$$

$$T = \frac{5 \times 10^8 (1.2)}{3 \times 10^9}$$

$$\boxed{0.2s}$$

22.65 GATE Numerical — Predictor Improvement

Old accuracy:

$$\sqrt[80]{\%}$$

New accuracy:

$$\sqrt[95]{\%}$$

Branch fraction:

$$\sqrt[25]{\%}$$

Penalty:

$$\sqrt[4]{}$$

Old CPI

$$\sqrt[1+0.25(0.20)(4)]{}$$

$$\sqrt[=1.20]{}$$

New CPI

$$\sqrt[1+0.25(0.05)(4)]{}$$

$$\sqrt[=1.05]{}$$

Speedup:

$$\sqrt[\frac{1.20}{1.05}]{}$$

$$\sqrt[\boxed{\approx 1.143 \times}]{}$$

assuming same clock/IC.

22.66 GATE Numerical — Required Accuracy

Suppose:

- base CPI = 1
- branch fraction = 20%
- penalty = 5
- target CPI = 1.1

Let misprediction rate = m .

$$\sqrt[1.1=1+0.2m(5)]{}$$

$$\sqrt[0.1=m]{}$$

Thus accuracy:

$$1 - 0.1$$

$$\boxed{90\%}$$

22.67 General Required Accuracy

$$CPI_{\text{target}} = CPI_{\text{base}} + f_b(1-a)P$$

Thus:

$$a = 1 - \frac{CPI_{\text{target}} - CPI_{\text{base}}}{f_b P}$$

when only mispredictions add penalty.

22.68 GATE Numerical — Find Penalty

Given:

- CPI = 1.18
- base CPI = 1
- branch fraction = 20%
- accuracy = 85%

$$0.18 = 0.2(0.15)P$$

$$0.18 = 0.03P$$

$$\boxed{P = 6 \text{ cycles}}$$

22.69 GATE Numerical — Find Branch Fraction

Given:

- base CPI = 1
- actual CPI = 1.08
- misprediction = 10%
- penalty = 4

$$0.08 = f_b(0.1)(4)$$

$$0.08 = 0.4f_b$$

$$\boxed{f_b = 0.20 = 20\%}$$

22.70 Branch Penalty + Data Hazard

Suppose:

- Load-use stalls = 0.10 cycles/instruction
- Branches = 20%
- accuracy = 90%
- penalty = 3

Branch:

$$\lfloor 0.2(0.1)(3)=0.06 \rfloor$$

Actual CPI:

$$\lfloor 1+0.10+0.06 \rfloor$$

$$\lfloor \boxed{1.16} \rfloor$$

22.71 Add Cache Stall

Suppose cache contributes:

$$\lfloor 0.50 \rfloor$$

cycles/instruction.

Then:

$$\lfloor \text{CPI}=1+0.10+0.06+0.50 \rfloor$$

$$\lfloor \boxed{1.66} \rfloor$$

This kind of combined performance question is common.

22.72 GATE Trap — Penalty Applied to Every Instruction

Wrong.

Multiply by branch frequency.

22.73 GATE Trap — Prediction Accuracy Applied to Every Instruction

Wrong.

Accuracy refers to branches, unless explicitly stated otherwise.

22.74 GATE Trap — Taken Rate = Branch Frequency

Wrong.

Example:

TEXT

20% instructions are branches
60% branches are taken

Taken branch fraction of all instructions:

$$\lfloor 12\% \rfloor$$

22.75 GATE Trap — Correct Prediction Always Zero Cost

Not necessarily.

If problem specifies target delay or fixed branch handling cost, include it.

22.76 GATE Trap — Flush Means Re-execute Branch

Usually no.

Branch itself is valid.

You discard younger wrong-path instructions.

22.77 GATE Trap — Static Predictor Learns History

No.

Static prediction uses fixed policy such as always taken/not taken.

Dynamic prediction uses runtime history.

22.78 GATE Trap — More Accurate Predictor Always Gives Same Percentage Speedup

No.

Benefit depends on:

- branch frequency,
 - penalty,
 - base CPI,
 - clock rate.
-

22.79 GATE Trap — Deep Pipeline Always Faster

Control-hazard penalty may increase with deeper pipelines.

Clock rate improvement must outweigh additional CPI penalties.

22.80 Fast Solving Template

For branch prediction numerical:

Step 1

Convert branch frequency:

$\lfloor f_b \rfloor$

Step 2

Find wrong prediction probability among branches:

$\lfloor m \rfloor$

Step 3

Multiply:

$$\boxed{f_{bmP}}$$

Step 4

Add to base CPI:

$$\boxed{CPI = CPI_{\text{base}} + f_{bmP}}$$

22.81 Predict-Not-Taken Shortcut

$$\boxed{m = \text{TakenFraction}}$$

so:

$$\boxed{CPI = CPI_{\text{base}} + f_b(\text{TakenFraction})P}$$

22.82 Predict-Taken Shortcut

$$\boxed{m = \text{NotTakenFraction}}$$

so:

$$\boxed{CPI = CPI_{\text{base}} + f_b(\text{NotTakenFraction})P}$$

22.83 Weighted-Cost Safe Method

If correct and incorrect predictions have different costs:

$$\boxed{\text{BranchPenalty} = f_b[aP_{\text{correct}} + (1-a)P_{\text{wrong}}]}$$

This is universally safer.

CHAPTER 22 — FORMULA SHEET

Misprediction rate

$$\boxed{m = 1 - \text{accuracy}}$$

Branch stall/instruction

$$\boxed{f_{bmP}}$$

CPI

$$\boxed{CPI = CPI_{\text{base}} + f_{bmP}}$$

Mispredictions

$$\boxed{N_{\text{mispred}} = N f_{bm}}$$

Lost cycles

$$\boxed{\text{LostCycles} = N f_{bmP}}$$

Predict not taken

$\boxed{m = \text{TakenFraction}}$

Predict taken

$\boxed{m = 1 - \text{TakenFraction}}$

CHAPTER 22 — MEMORY TRICKS

Control hazard

Branch changes the road.

Flush

Wrong road !' throw instructions away.

Prediction

Guess first, pay only if wrong.

Formula

$\text{Branch} \times \text{Wrong} \times \text{Penalty}$

$\boxed{BWP}$

Predict Not Taken

Penalty on Taken

Predict Taken

Penalty on Not Taken

CHAPTER SNAPSHOT

1. Control hazards arise because next PC is uncertain.
2. Branches are the main GATE control-hazard source.
3. Wrong-path instructions may need flushing.
4. Branch penalty depends on resolution/prediction design.
5. Stalling until resolution is simple but costly.
6. Static prediction can assume taken or not taken.
7. Dynamic predictors use execution history.
8. Misprediction rate = $1 - \text{accuracy}$.

9. Average branch stall = branch frequency $\times$ misprediction rate $\times$ penalty.
 10. Predict-not-taken is wrong for taken branches.
 11. Predict-taken is wrong for not-taken branches.
 12. Earlier branch resolution usually reduces penalty.
 13. Deep pipelines can suffer larger misprediction penalties.
 14. Correct predictions may still have a stated cost in some questions.
 15. Weighted probability $\times$ cost is the safest general method.
 16. Branch penalties can be added with data/cache stalls to calculate CPI.
-

MUST REMEMBER

$$CPI = CPI_{\text{base}} + \text{BranchFrequency} \times \text{MispredictionRate} \times \text{Penalty}$$

and:

Branch $\times$ Wrong $\times$ Penalty

PRACTICE SET

Q1 — L0 NAT

20% instructions are branches.

Prediction accuracy = 90%.

Penalty = 3 cycles.

Find average branch stalls per instruction.

Q2 — L1 NAT

For Q1, if base CPI = 1, find actual CPI.

Q3 — L1 NAT

25% instructions are branches.

60% branches are taken.

Processor always predicts not taken.

Penalty = 2 cycles.

Find CPI if base CPI = 1.

Q4 — L1 NAT

Same as Q3, but always predict taken.

Find CPI.

Q5 — L2

Which predictor is better in Q3–Q4 and why?

Q6 — L2 NAT

A program executes 10^8 instructions.

15% are branches.

Predictor accuracy = 92%.

Find number of mispredictions.

Q7 — L2 NAT

For Q6, penalty = 4 cycles.

Find total lost cycles.

Q8 — L2 NAT

Base CPI = 1.

Branch frequency = 20%.

Penalty = 5 cycles.

Required total CPI = 1.1.

Find minimum prediction accuracy.

Q9 — L2 NAT

Base CPI = 1.

Actual CPI = 1.18.

20% instructions are branches.

Prediction accuracy = 85%.

Find misprediction penalty.

Q10 — L2

A branch is resolved in EX in a simple 5-stage pipeline. Draw the pipeline positions of the two following instructions when branch reaches EX and explain why they may need flushing.

Q11 — L3 NAT

A processor has:

- Base CPI = 1
- 25% branches
- 95% prediction accuracy
- 5-cycle misprediction penalty
- data-hazard stalls = 0.10 cycles/instruction
- cache stalls = 0.35 cycles/instruction

Find total CPI.

Q12 — L3 NAT

Predictor A has 90% accuracy.

Predictor B has 97% accuracy.

Branches = 20%.

Penalty = 6 cycles.

Base CPI = 1.

Find CPI for both and speedup of B over A.

Q13 — L3 NAT

20% of instructions are branches. Predictor accuracy is 90%.

Correctly predicted branches cost 1 cycle; mispredicted branches cost 4 cycles.

Find CPI if base CPI = 1.

CHAPTER 22 COMPLETION STATUS

' Control hazard definition ' Branch uncertainty ' Pipeline flushing ' Branch penalty ' Stall-until-resolved ' Predict taken ' Predict not taken ' Prediction accuracy ' Dynamic prediction overview ' 1-bit/2-bit intuition ' Early branch resolution ' Delayed branch concept ' Branch-frequency numericals ' Misprediction numericals ' CPI calculations ' Reverse numericals ' Combined hazard CPI ' GATE traps

CHAPTER 22: COMPLETE '

PIPELINE HAZARD UNIT STATUS

You have now completed:

' Structural hazards ' RAW / WAR / WAW ' Forwarding ' Load-use hazards ' Stalling ' Instruction scheduling ' Control hazards ' Branch prediction ' Pipeline CPI numericals

COURSE PROGRESS

Completed: 23 / 26 modules

Approximate roadmap progress:

$\boxed{\sim 88\%}$

NEXT !' CHAPTER 23 — CPU PERFORMANCE, CPI & EXECUTION-TIME NUMERICALS

Next we consolidate the performance formulas that connect everything:

Ø = CPU execution time Ø = IC CPI, clock rate Ø = average CPI for instruction classes Ø = MIPS and its traps Ø = speedup
Ø = Amdahl-style improvement intuition Ø = cache + pipeline + branch CPI integration Ø = comparing two processors Ø = Y%
GATE mixed performance numericals.

USER

next

ASSISTANT

CHAPTER 23 — CPU PERFORMANCE, CPI & EXECUTION-TIME NUMERICALS

Classification: [GATE CORE + GATE SUPPORT] **Priority:** Ø = VERY HIGH **Connection:** This chapter connects **pipeline + cache + hazards + instruction execution** into one performance model.

The central formula is:

$$\boxed{\text{CPU Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time}}$$

or equivalently:

$$\boxed{\text{CPU Time} = \frac{\text{Instruction Count} \times \text{CPI}}{\text{Clock Rate}}}$$

If you master this formula and understand what changes **IC, CPI, and clock rate**, many COA numericals become straightforward.

23.1 The Three Performance Factors

CPU execution time depends on:

1. Instruction Count — IC

How many machine instructions execute?

2. CPI

Average:

$$\boxed{\text{Cycles Per Instruction}}$$

3. Clock Cycle Time / Clock Rate

How long is each cycle?

Thus:

$$[\text{CPUTime} = IC \times \text{CPI} \times T_{\text{clock}}]$$

23.2 Clock Rate Form

Since:

$$[T_{\text{clock}} = \frac{1}{\text{ClockRate}}]$$

we get:

$$[\boxed{\text{CPUTime} = \frac{IC \times \text{CPI}}{\text{ClockRate}}}]$$

Memorize this.

23.3 Simple Example

Program executes:

$$[10^9]$$

instructions.

Average CPI:

$$[2]$$

Clock:

$$[2\text{GHz}]$$

Then:

$$[\text{CPUTime} = \frac{10^9(2)}{2 \times 10^9}]$$

$$[\boxed{1\text{s}}]$$

23.4 Total CPU Cycles

Another useful relation:

$$[\boxed{\text{CPU Cycles} = IC \times \text{CPI}}]$$

Previous example:

$$[10^9 \times 2 = 2 \times 10^9]$$

cycles.

At 2 GHz:

$$[\frac{2 \times 10^9}{2 \times 10^9} = 1\text{s}]$$

23.5 CPI Meaning

Suppose:

$$[\text{CPI} = 1.5]$$

This does **not** mean every instruction takes exactly 1.5 cycles.

It is an average across all executed instructions.

Some may take:

- 1 cycle equivalent,
- 2 cycles,
- more because of stalls.

23.6 Average CPI from Instruction Classes

Suppose a program has:

- Class A: 50%, CPI 1
- Class B: 30%, CPI 2
- Class C: 20%, CPI 4

Average CPI:

$$CPI_{avg} = 0.5(1) + 0.3(2) + 0.2(4)$$

$$= 0.5 + 0.6 + 0.8$$

$$\boxed{1.9}$$

23.7 General Weighted CPI Formula

If instruction class i has:

- fraction f_i ,
- CPI C_i ,

then:

$$\boxed{CPI_{avg} = \sum_i f_i C_i}$$

where:

$$\sum_i f_i = 1$$

23.8 Using Instruction Counts Instead of Fractions

If class counts are I_i :

$$\boxed{CPI_{avg} = \frac{\sum_i I_i C_i}{\sum_i I_i}}$$

23.9 Example

Instruction counts:

- 400 arithmetic, CPI 1
- 300 load/store, CPI 2
- 100 branches, CPI 3

Total instructions:

$$800$$

Total cycles:

$$\lfloor 400(1)+300(2)+100(3) \rfloor$$

$$\lfloor =400+600+300 \rfloor$$

$$\lfloor =1300 \rfloor$$

Average CPI:

$$\lfloor 1300/800 \rfloor$$

$$\lfloor \boxed{1.625} \rfloor$$

23.10 CPU Time from Average CPI

If previous program runs at:

$$\lfloor 2\text{GHz} \rfloor$$

then:

$$\lfloor \text{CPUTime} = \frac{800(1.625)}{2 \times 10^9} \rfloor$$

$$\lfloor = \frac{1300}{2 \times 10^9} \rfloor$$

$$\lfloor \boxed{650\text{ns}} \rfloor$$

23.11 Clock Period Conversion

Important:

$$\lfloor \boxed{T_{\text{clock}} = \frac{1}{f}} \rfloor$$

Fast GHz !" ns:

$$\lfloor 1\text{GHz} \rightarrow 1\text{ns} \rfloor$$

$$\lfloor 2\text{GHz} \rightarrow 0.5\text{ns} \rfloor$$

$$\lfloor 4\text{GHz} \rightarrow 0.25\text{ns} \rfloor$$

23.12 Example

Clock rate:

$$\lfloor 2.5\text{GHz} \rfloor$$

Clock period:

$$\lfloor \frac{1}{2.5} \rfloor$$

$$\lfloor \boxed{0.4\text{ns}} \rfloor$$

23.13 Execution Time Using Period

Suppose:

- $\text{IC} = (5 \times 10^8)$
-

- CPI = 1.2
- clock period = 0.5 ns

CPU cycles:

$$[5 \times 10^8 (1.2) = 6 \times 10^8]$$

Time:

$$[6 \times 10^8 (0.5 \text{ ns})]$$

$$[= 3 \times 10^8 \text{ ns}]$$

$$[\boxed{0.3 \text{ s}}]$$

23.14 Performance vs Execution Time

Performance is inversely proportional to execution time.

$$[\boxed{\text{Performance} \propto \frac{1}{\text{ExecutionTime}}}]$$

So a processor that finishes sooner has higher performance.

23.15 Speedup

If old execution time is (T_{old}) and new time is (T_{new}) :

$$[\boxed{\text{Speedup} = \frac{T_{\text{old}}}{T_{\text{new}}}}]$$

23.16 Example

Old:

$$[10 \text{ s}]$$

New:

$$[8 \text{ s}]$$

Speedup:

$$[10/8]$$

$$[\boxed{1.25 \times}]$$

New system is 25% faster in performance terms.

23.17 Percentage Time Reduction

Same example:

$$[\frac{10-8}{10} \times 100]$$

$$[\boxed{20\%}]$$

Notice:

- speedup = 1.25×

- execution time reduction = 20%

Do not confuse them.

23.18 "X% Faster" Trap

If A is 25% faster than B:

$$\text{Performance}_A = 1.25 \text{Performance}_B$$

Since time is inverse:

$$T_A = \frac{T_B}{1.25}$$

$$\boxed{0.8 T_B}$$

So A takes 20% less time, not 25% less.

0 = 0% Classic exam trap.

23.19 Comparing Two CPUs

For same program:

$$T = \frac{IC \times CPI}{f}$$

So if instruction count is identical:

$$T \propto \frac{CPI}{f}$$

23.20 Example

CPU A:

- 2 GHz
- CPI = 1

CPU B:

- 3 GHz
- CPI = 1.8

Compare:

$$A: \frac{1}{2} = 0.5$$

$$B: \frac{1.8}{3} = 0.6$$

Therefore:

$$\boxed{A \text{ is faster}}$$

despite lower clock rate.

23.21 Speedup A over B

$$\text{Speedup}_{A/B} = \frac{T_B}{T_A}$$

$$= \frac{0.6}{0.5}$$

$$\boxed{1.2 \times}$$

23.22 Different ISAs / Compilers

If processors execute different instruction counts, you cannot cancel IC.

You must use:

$$\boxed{T = \frac{IC \times CPI}{ClockRate}}$$

for each.

23.23 Example — Different IC

Machine A:

- $IC = 10^9$
- $CPI = 1.2$
- 2 GHz

Machine B:

- $IC = 0.8 \times 10^9$
- $CPI = 1.5$
- 2.5 GHz

A:

$$T_A = \frac{10^9(1.2)}{2 \times 10^9}$$

$$= 0.6s$$

B:

$$T_B = \frac{0.8 \times 10^9(1.5)}{2.5 \times 10^9}$$

$$= 0.48s$$

Thus B faster.

Speedup:

$$0.6/0.48$$

$$\boxed{1.25 \times}$$

23.24 Why Clock Rate Alone Is Misleading

A higher clock can come with:

- higher CPI,
- deeper pipeline,
- more branch penalty,
- different instruction count.

Therefore performance must be judged with all three:

$$\boxed{IC, CPI, ClockRate}$$

Memory trick:

ICC

Instruction Count CPI Clock

23.25 MIPS

MIPS means:

$\boxed{\text{Million Instructions Per Second}}$

Formula:

$\boxed{\text{MIPS} = \frac{\text{ClockRate}}{\text{CPI} \times 10^6}}$

if clock rate is in Hz.

23.26 Example

Clock:

2GHz

CPI:

2

$\text{MIPS} = \frac{2 \times 10^9}{2 \times 10^6}$

$\boxed{1000\text{ MIPS}}$

23.27 MIPS from Instruction Count and Time

$\boxed{\text{MIPS} = \frac{\text{InstructionCount}}{\text{ExecutionTime} \times 10^6}}$

Example:

100 million instructions in 0.05 s:

$\frac{100 \times 10^6}{0.05 \times 10^6}$

$\boxed{2000\text{ MIPS}}$

23.28 MIPS Trap

Higher MIPS does **not always mean** a program finishes faster.

Why?

Different ISAs may require different numbers of instructions.

Example:

Processor A:

- 1000 MIPS

- program needs 2 billion instructions.

Time:

$\boxed{2s}$

Processor B:

- 800 MIPS
- program needs 1 billion instructions.

Time:

$\boxed{1.25s}$

B is faster despite lower MIPS.

23.29 Therefore

$\boxed{\text{Execution time is the safest performance metric}}$

MIPS is useful only with caution.

23.30 CPI from MIPS

From:

$\boxed{MIPS = \frac{\text{ClockRate}}{CPI \times 10^6}}$

we get:

$\boxed{CPI = \frac{\text{ClockRate}}{MIPS \times 10^6}}$

23.31 Example

Clock = 3 GHz.

MIPS = 1500.

$\boxed{CPI = \frac{3 \times 10^9}{1500 \times 10^6}}$

$\boxed{2}$

23.32 Instruction Rate

Instruction throughput:

$\boxed{\text{InstructionRate} = \frac{\text{ClockRate}}{CPI}}$

MIPS is simply this rate divided by (10^6) .

23.33 Pipeline CPI Integration

For an ideal scalar pipeline:

$\boxed{CPI_{\text{base}} \approx 1}$

Real:

$$\boxed{\text{CPI} = 1 + \text{StallCyclesPerInstruction}}$$

for a long instruction stream.

23.34 Example

Data hazards:

$$\boxed{0.15}$$

cycles/instruction.

Control hazards:

$$\boxed{0.08}$$

Cache stalls:

$$\boxed{0.40}$$

Then:

$$\boxed{\text{CPI} = 1 + 0.15 + 0.08 + 0.40}$$

$$\boxed{1.63}$$

23.35 Branch Contribution

From Chapter 22:

$$\boxed{\text{CPI}_{\text{branch}} = f_{\text{bmP}}}$$

Example:

- branches = 20%
- misprediction = 10%
- penalty = 4

$$\boxed{0.2(0.1)(4) = 0.08}$$

23.36 Cache Contribution

Suppose:

- 1.2 cache references/instruction
- miss rate = 2%
- penalty = 50 cycles

Cache stalls:

$$\boxed{1.2(0.02)(50)}$$

$$\boxed{1.2 \text{ cycles/instruction}}$$

23.37 Combined CPI Example

Base pipeline CPI:

$$\sqrt[1]{1}$$

Load-use:

$$\sqrt[0.10]{0.10}$$

Branches:

$$\sqrt[0.08]{0.08}$$

Cache:

$$\sqrt[1.20]{1.20}$$

Total:

$$\sqrt[1+0.10+0.08+1.20]{1+0.10+0.08+1.20}$$

$$\sqrt[\boxed{2.38}]{2.38}$$

23.38 CPU Time from Combined CPI

Suppose:

- $IC = 5 \times 10^8$
- $CPI = 2.38$
- $\text{Clock} = 2 \text{ GHz}$

$$\sqrt[T = \frac{5 \times 10^8 (2.38)}{2 \times 10^9}]{T = \frac{5 \times 10^8 (2.38)}{2 \times 10^9}}$$

$$\sqrt[\boxed{0.595s}]{0.595s}$$

23.39 Instruction Mix + Cache Stalls

Sometimes base CPI depends on instruction type first.

Example:

Type	Fraction	Base CPI
ALU	50%	1
Load	20%	2
Store	10%	2
Branch	20%	3

Base average:

$$\sqrt[0.5(1)+0.2(2)+0.1(2)+0.2(3)]{0.5(1)+0.2(2)+0.1(2)+0.2(3)}$$

$$\sqrt[=0.5+0.4+0.2+0.6]{=0.5+0.4+0.2+0.6}$$

$$\sqrt[\boxed{1.7}]{1.7}$$

Then if cache adds:

$$\sqrt[0.3]{0.3}$$

stall cycles/instruction:

$$\boxed{\text{CPI}=2.0}$$

23.40 Be Careful About Double Counting

If stated CPI already includes memory stalls, do not add them again.

Example:

"Measured CPI including cache effects = 2.0"

Do not then add cache miss penalty unless question explicitly asks to modify it.

Not important.

23.41 CPU Performance Equation with Instruction Classes

Total cycles:

$$\boxed{\text{Cycles} = \sum_i \text{IC}_i \text{CPI}_i}$$

Execution time:

$$\boxed{T = \frac{\sum_i \text{IC}_i \text{CPI}_i}{\text{ClockRate}}}$$

This is sometimes faster than calculating average CPI first.

23.42 Example

Instruction classes:

- 2 million at CPI 1
- 1 million at CPI 3
- 1 million at CPI 2

Total cycles:

$$\boxed{2\text{M}(1) + 1\text{M}(3) + 1\text{M}(2)}$$

$$\boxed{= 7\text{M}}$$

At 1 GHz:

$$\boxed{T = \frac{7 \times 10^6}{10^9}}$$

$$\boxed{7\text{ms}}$$

23.43 Reverse Numerical — Find CPI

Given:

- execution time = 0.5 s
- $\text{IC} = 10^9$
- clock = 3 GHz

From:

$$T = \frac{IC \times CPI}{f}$$

$$0.5 = \frac{10^9 CPI}{3 \times 10^9}$$

$$0.5 = \frac{CPI}{3}$$

$$\boxed{CPI = 1.5}$$

23.44 Reverse — Find Clock Rate

Given:

- $IC = 2 \times 10^9$
- $CPI = 1.5$
- $\text{time} = 1 \text{ s}$

$$f = \frac{IC \times CPI}{T}$$

$$= \frac{2 \times 10^9 (1.5)}{1}$$

$$\boxed{3 \text{ GHz}}$$

23.45 Reverse — Find Instruction Count

Given:

- $\text{CPU time} = 2 \text{ s}$
- $CPI = 2$
- $\text{clock} = 3 \text{ GHz}$

$$IC = \frac{T \times f}{CPI}$$

$$= \frac{2 (3 \times 10^9)}{2}$$

$$\boxed{3 \times 10^9}$$

instructions.

23.46 Percentage CPI Improvement

Old CPI:

$$2$$

New CPI:

$$1.5$$

Same IC/clock.

Speedup:

$$2/1.5$$

$$\boxed{1.333 \times}$$

Execution time falls by:

$$\left[\frac{2-1.5}{2} \times 100 \right]$$

$$\left[\boxed{25\%} \right]$$

23.47 Clock Increase vs CPI Increase

Suppose old processor:

- 2 GHz
- CPI 1.2

New:

- 3 GHz
- CPI 1.6

Old time factor:

$$\left[1.2/2=0.6 \right]$$

New:

$$\left[1.6/3=0.5333 \right]$$

New processor faster by:

$$\left[0.6/0.5333 \right]$$

$$\left[\boxed{1.125 \times} \right]$$

Even though CPI worsened, higher frequency compensates.

23.48 Optimization Trade-Off

An optimization may:

- reduce IC,
- increase CPI,
- change clock frequency.

Never evaluate only one quantity.

Use the full CPU-time equation.

23.49 Example — Compiler A vs Compiler B

Compiler A generates:

- (1.0×10^9) instructions
- CPI = 1.2

Compiler B:

- (0.8×10^9) instructions
- CPI = 1.4

Same 2 GHz CPU.

A:

$$T_A = \frac{1.0(1.2)}{2} = 0.6s$$

B:

$$T_B = \frac{0.8(1.4)}{2} = 0.56s$$

B faster.

Speedup:

$$0.6/0.56$$

$$\boxed{1.0714 \times}$$

23.50 Amdahl's Law — Important Performance Principle

[SUPPORT / HIGH VALUE]

Suppose fraction (F) of execution time can be improved by factor (S) .

Overall speedup:

$$\boxed{\text{Speedup} = \frac{1}{(1-F) + \frac{F}{S}}}$$

This is Amdahl's Law.

Useful whenever only part of system is improved.

23.51 Amdahl Intuition

If only 10% of program can be improved, even making that part infinitely fast cannot make whole program 10× faster.

Because remaining 90% is unchanged.

23.52 Example

40% of execution time is improved by 2×.

$$F = 0.4$$

$$S = 2$$

New normalized time:

$$0.6 + \frac{0.4}{2}$$

$$= 0.8$$

Overall speedup:

$$1/0.8$$

$$\boxed{1.25 \times}$$

23.53 Infinite Improvement Limit

If improved portion becomes infinitely fast:

$$S \rightarrow \infty$$

then:

$$\left[\frac{F}{S} \right]$$

So maximum speedup:

$$\left[\boxed{\text{Speedup}_{\max} = \frac{1}{1-F}} \right]$$

23.54 Example

20% of runtime is enhanced.

Even infinite improvement gives:

$$\left[\frac{1}{0.8} \right]$$

$$\left[\boxed{1.25 \times} \right]$$

Maximum.

23.55 Amdahl Memory Trick

You are limited by what you did NOT improve.

23.56 Cache Optimization via Amdahl

Suppose memory stalls are 40% of execution time.

A new cache makes memory-stall portion 4× faster.

Overall:

$$\left[\text{Speedup} = \frac{1}{0.6 + \frac{0.4}{4}} \right]$$

$$\left[= \frac{1}{0.7} \right]$$

$$\left[\boxed{1.429 \times} \right]$$

23.57 Pipeline Improvement via Amdahl

Suppose branch stalls consume 20% of execution time.

Perfect branch prediction eliminates them completely.

Then improved fraction effectively becomes infinitely fast.

Maximum:

$$\left[\frac{1}{0.8} \right]$$

$$\left[\boxed{1.25 \times} \right]$$

So even perfect prediction cannot produce more than 25% speedup in this scenario.

23.58 Relative Performance

If A takes 4 s and B takes 5 s:

$$\frac{\text{Performance}_A}{\text{Performance}_B} = \frac{T_B}{T_A}$$

$$= \frac{5}{4}$$

$$\boxed{1.25}$$

A has 25% higher performance.

23.59 "A Is X Times Faster" vs "X Times As Fast"

Language can be ambiguous in informal writing.

For GATE, rely on formulas/numbers and interpret "speedup of X" as:

$$\frac{\text{OldTime}}{\text{NewTime}} = X$$

23.60 MIPS with Mixed CPI

Suppose:

- clock = 2 GHz
- average CPI = 1.6

$$\text{MIPS} = \frac{2 \times 10^9}{1.6 \times 10^6}$$

$$\boxed{1250 \text{ MIPS}}$$

23.61 MIPS Reverse Numerical

CPU reports:

$$2000 \text{ MIPS}$$

CPI:

$$1.5$$

Required clock:

$$\text{Clock} = \text{MIPS} \times \text{CPI} \times 10^6$$

$$= 2000(1.5)(10^6)$$

$$\boxed{3 \text{ GHz}}$$

23.62 MIPS and Program Time

Suppose:

- program IC = 500 million
- machine = 1000 MIPS

Time:

$$\left[\frac{500 \times 10^6}{1000 \times 10^6} \right]$$

$$\left[\boxed{0.5s} \right]$$

23.63 CPI from Stall Probabilities

Base CPI:

$$\left[1 \right]$$

Suppose:

- 10% instructions cause 1-cycle load-use stall
- 20% branches, 5% mispredicted, penalty 4
- average cache stall = 0.3

Load contribution:

$$\left[0.1 \right]$$

Branch:

$$\left[0.2(0.05)(4)=0.04 \right]$$

Total:

$$\left[1+0.1+0.04+0.3 \right]$$

$$\left[\boxed{1.44} \right]$$

23.64 CPU Time for Previous

IC:

$$\left[2 \times 10^9 \right]$$

Clock:

$$\left[3\text{GHz} \right]$$

$$\left[T = \frac{2 \times 10^9 (1.44)}{3 \times 10^9} \right]$$

$$\left[\boxed{0.96s} \right]$$

23.65 Comparing Cache Designs Through CPI

Cache A:

- processor base CPI = 1
- cache stalls = 0.5
- clock = 2 GHz

Total:

$$\left[\text{CPI}_A = 1.5 \right]$$

Cache B requires slower clock:

- stalls = 0.2

- clock = 1.8 GHz

$$\lceil \text{CPI}_B = 1.2 \rceil$$

Time factor:

A:

$$\lceil 1.5/2 = 0.75 \rceil$$

B:

$$\lceil 1.2/1.8 = 0.6667 \rceil$$

Thus B is faster despite lower frequency.

23.66 Speedup of B over A

$$\lceil 0.75/0.6667 \rceil$$

$$\lceil \boxed{\approx 1.125 \times} \rceil$$

23.67 Comparing Pipeline Designs

Pipeline A:

- 2 GHz
- branch CPI penalty = 0.1
- data penalty = 0.1

$$\lceil \text{CPI}_A = 1.2 \rceil$$

Pipeline B:

- 3 GHz
- deeper pipeline
- branch penalty = 0.4
- data penalty = 0.1

$$\lceil \text{CPI}_B = 1.5 \rceil$$

Time factors:

$$\lceil A = 1.2/2 = 0.6 \rceil$$

$$\lceil B = 1.5/3 = 0.5 \rceil$$

B still faster:

$$\lceil 0.6/0.5 \rceil$$

$$\lceil \boxed{1.2 \times} \rceil$$

23.68 When Deeper Pipeline Loses

Suppose B's CPI becomes:

$$\lceil 2 \rceil$$

at 3 GHz.

$\lfloor 2/3=0.6667 \rfloor$

A remains:

$\lfloor 0.6 \rfloor$

Now A is faster.

Again:

Frequency alone is insufficient.

23.69 GATE Trap — Lower CPI Always Means Faster

'L Wrong.

Clock and instruction count matter too.

23.70 GATE Trap — Higher Clock Always Means Faster

'L Wrong.

CPI/IC may increase.

23.71 GATE Trap — Fewer Instructions Always Means Faster

'L Wrong.

Those instructions may have larger CPI or processor may have slower clock.

23.72 GATE Trap — Average CPI Is Simple Arithmetic Mean

Usually no.

If instruction frequencies differ, use weighted mean:

$\lfloor \sum f_i C_i \rfloor$

23.73 GATE Trap — MIPS Is Universal Comparison Metric

'L Wrong.

Different ISAs can do different amounts of work per instruction.

Use execution time.

23.74 GATE Trap — Speedup 2 Means 200% Time Reduction

Impossible.

2× speedup means:

$$T_{\text{new}} = \frac{1}{2} T_{\text{old}}$$

so time reduction:

$$50\%$$

23.75 GATE Trap — 50% Faster Means Half the Time

No.

50% faster:

$$Performance_{\text{new}} = 1.5 Performance_{\text{old}}$$

so:

$$T_{\text{new}} = \frac{T_{\text{old}}}{1.5}$$

$$\approx 0.667 T_{\text{old}}$$

which is about:

$$33.3\%$$

less time.

23.76 GATE Trap — Add Percentage Improvements Directly

Suppose two independent parts improve by percentages.

Do not simply add them.

Compute actual execution times or use Amdahl's law.

23.77 Fast Performance Comparison Method

For each processor calculate:

$$\boxed{\frac{IC \times CPI}{\text{ClockRate}}}$$

You can often omit common units/constants.

Smallest value = fastest.

23.78 If IC Is Same

Use:

$$\boxed{\frac{CPI}{\text{ClockRate}}}$$

23.79 If Clock Is Same

Use:

$$\boxed{IC \times CPI}$$

23.80 If CPI Is Same

Use:

$$\boxed{\frac{IC}{\text{ClockRate}}}$$

23.81 Mixed GATE Numerical 1

Processor:

- $IC = 10^8$
- 50% instructions CPI 1
- 30% CPI 2
- 20% CPI 4
- Clock = 2 GHz

Average CPI:

$$0.5 + 0.6 + 0.8 = 1.9$$

Cycles:

$$1.9 \times 10^8$$

Time:

$$\frac{1.9 \times 10^8}{2 \times 10^9}$$

$$\boxed{0.095\text{s}}$$

23.82 Mixed GATE Numerical 2

Base CPI:

$$1$$

- 15% load-use hazards, 1 stall
- 20% branches
- predictor accuracy 95%
- branch penalty 4
- cache stall = 0.25 cycles/instruction

Load:

$$0.15$$

Branch:

$$0.2(0.05)(4) = 0.04$$

Total:

$$1 + 0.15 + 0.04 + 0.25$$

$$\boxed{1.44}$$

23.83 Mixed GATE Numerical 3

For previous CPI:

- $IC = 10^9$
- clock = 2.4 GHz

$$T = \frac{10^9(1.44)}{2.4 \times 10^9}$$

$$\boxed{0.6s}$$

23.84 Mixed GATE Numerical 4 — Improvement

Suppose cache redesign reduces cache stall contribution:

$$0.25 \rightarrow 0.10$$

New CPI:

$$1 + 0.15 + 0.04 + 0.10$$

$$= 1.29$$

Old:

$$1.44$$

Same clock and IC.

Speedup:

$$1.44 / 1.29$$

$$\boxed{\approx 1.116 \times}$$

23.85 Mixed GATE Numerical 5 — Clock Trade-Off

Same cache redesign increases clock period, reducing frequency:

Old:

$$2.4\text{GHz}, \text{CPI}=1.44$$

New:

$$2.1\text{GHz}, \text{CPI}=1.29$$

Old time factor:

$$1.44 / 2.4 = 0.6$$

New:

$$1.29 / 2.1 \approx 0.6143$$

So new design is actually slower.

Performance improvements must be evaluated end-to-end.

23.86 Amdahl Reverse Numerical

Overall speedup = 1.25.

Enhanced fraction = 50%.

Find enhancement factor $\backslash(S\backslash)$.

$$\backslash[1.25 = \frac{1}{0.5 + \frac{0.5}{S}} \backslash]$$

Invert:

$$\backslash[0.8 = 0.5 + \frac{0.5}{S} \backslash]$$

$$\backslash[0.3 = \frac{0.5}{S} \backslash]$$

$$\backslash[S = \frac{0.5}{0.3} \backslash]$$

$$\backslash[\boxed{1.667 \text{ times}} \backslash]$$

23.87 Maximum Possible Improvement

Suppose 30% of runtime can be optimized.

Maximum overall speedup:

$$\backslash[\frac{1}{0.7} \backslash]$$

$$\backslash[\boxed{1.429 \text{ times}} \backslash]$$

even if that 30% becomes infinitely fast.

23.88 CPU Performance Master Chain

You can think:

``text id="ze6j59" Instruction Mix !" Instruction Count + Base CPI !" Pipeline stalls !" Cache stalls !" Branch penalties !" Actual CPI !" Clock Rate !" CPU Execution Time

This connects nearly the whole COA syllabus.

23.89 Final Master Formula Set

CPU cycles

$$\backslash[\boxed{\text{Cycles} = \text{IC} \times \text{CPI}} \backslash]$$

CPU time

$$\backslash[\boxed{T = \text{IC} \times \text{CPI} \times T_{\text{clock}}} \backslash]$$

or:

$$\backslash[\boxed{\text{CPI} = \frac{\text{Cycles}}{\text{IC}}} \backslash]$$

$$T = \frac{IC \times CPI}{\text{ClockRate}}$$

Average CPI

$$CPI = \frac{\sum_i f_i CPI_i}{\sum_i f_i}$$

MIPS

$$MIPS = \frac{\text{ClockRate}}{CPI \times 10^6}$$

Speedup

$$\text{Speedup} = \frac{\text{OldTime}}{\text{NewTime}}$$

Amdahl

$$\text{Speedup} = \frac{1}{(1-F) + F/S}$$

CHAPTER 23 – MEMORY TRICKS

Performance equation

> ****ICC****

Instruction Count × CPI ÷ Clock Rate

Average CPI

> ****Fraction × Cost, then add****

Speedup

> ****Old ÷ New****

MIPS

> ****Clock ÷ CPI****

Amdahl

> ****Unimproved part limits everything****

CHAPTER SNAPSHOT

1. CPU time depends on IC, CPI and clock rate.
2. Total CPU cycles = IC × CPI.
3. Average CPI is a weighted average.
4. Lower CPI alone does not guarantee lower execution time.
5. Higher clock rate alone does not guarantee better performance.
6. Fewer instructions alone does not guarantee better performance.
7. Execution time is the safest comparison metric.
8. Speedup = old time/new time.
9. Performance is inverse of execution time.
10. MIPS must be used carefully across different ISAs.
11. Pipeline/cache/branch stalls contribute to CPI.
12. CPU time can combine all these penalties.
13. Amdahl's law limits benefit when only part of program is improved.
14. Deep pipelines can improve clock but hurt CPI.
15. End-to-end execution time decides the winner.

MUST REMEMBER

```
\[
\boxed{
CPUTime=\frac{IC\times CPI}{ClockRate}
}
\]
```

```
\[
\boxed{
CPI_{avg}=\sum f_{iC_i}
}
\]
```

```
\[
\boxed{
Speedup=\frac{T_{old}}{T_{new}}
}
\]
```

and:

```
\[
\boxed{
Speedup_{Amdahl}
=
\frac{1}{(1-F)+F/S}
}
\]
```

PRACTICE SET

Q1 – L0 NAT

A program executes 10^9 instructions with CPI 2 on a 4 GHz CPU.

Find execution time.

Q2 – L1 NAT

Instruction mix:

- 50% CPI 1

- 30% CPI 2
- 20% CPI 3

Find average CPI.

Q3 – L1 NAT

A processor has:

- Clock = 2 GHz
- CPI = 1.25

Find MIPS.

Q4 – L1 NAT

A program needs 500 million instructions on a 1000 MIPS machine.

Find execution time.

Q5 – L2

CPU A:

- IC = 10^9
- CPI = 1.2
- 2 GHz

CPU B:

- IC = 0.8×10^9
- CPI = 1.5
- 2.5 GHz

Which is faster and by what factor?

Q6 – L2 NAT

Base CPI = 1.

- Load-use stalls = 0.12/instruction
- Branch frequency = 20%
- prediction accuracy = 90%
- branch penalty = 3
- cache stalls = 0.40/instruction

Find actual CPI.

Q7 – L2 NAT

For Q6, program has 2×10^9 instructions and CPU frequency 3 GHz.

Find execution time.

Q8 – L2 NAT

A program runs in 2 seconds on CPU A and 1.6 seconds on CPU B.

Find speedup of B over A.

Q9 – L2 Conceptual

Why is MIPS not always a valid way to compare processors with different ISAs?

Q10 – L3 NAT

40% of execution time is improved by a factor of 5.

Find overall speedup using Amdahl's Law.

Q11 – L3 NAT

25% of execution time can be optimized.

What is the maximum theoretical overall speedup?

Q12 – L3 NAT

A CPU runs at 3 GHz and executes a program in 0.4 s. The program has 6×10^8 instructions.

Find average CPI.

Q13 – L3

Processor A:

- 2.5 GHz
- CPI 1.2

Processor B:

- 3.2 GHz
- CPI 1.6

Same instruction count.

Determine which is faster and the speedup.

Q14 – L3 NAT

Base CPI = 1.

- 10% instructions suffer 1 load-use stall,
- 25% are branches,
- prediction accuracy 96%,
- misprediction penalty 5,
- unified-cache stalls = 0.35/instruction.

Find total CPI.

CHAPTER 23 COMPLETION STATUS

```
' CPU performance equation
' IC/CPI/clock relationship
' Average CPI
' Instruction-mix numericals
' Clock-rate conversions
' CPU cycles
' Speedup
' Percentage-performance traps
' Comparing processors
' Different instruction counts
' MIPS
' MIPS traps
' Pipeline CPI integration
' Branch + cache + data-stall CPI
' Amdahl's law
' Optimization trade-offs
' Reverse numericals
' GATE mixed performance problems
```

CHAPTER 23: COMPLETE '

COURSE PROGRESS

****Completed: 24 / 26 modules****

Approximate roadmap progress:

\[
\boxed{\sim 92\%}
\]

NEXT !' CHAPTER 24 – MIXED COA NUMERICALS

This will combine the entire syllabus into GATE-style mixed problems:

```
Ø=Ý% ISA + instruction formats
Ø=Ý% addressing modes
Ø=Ý% ALU/overflow
Ø=Ý% control-unit calculations
Ø=Ý% memory interfacing
Ø=Ý% direct/set/fully associative cache
Ø=Ý% AMAT + dirty write-back
Ø=Ý% interrupts + DMA
Ø=Ý% pipeline + forwarding + hazards
Ø=Ý% CPI + branch prediction
Ø=Ý% multi-concept NAT/MSQ traps
```

After that, only ****Chapter 25 – Final COA Revision & Complete Syllabus Audit**** remains.

USER

next

CHAPTER 24 — MIXED COA NUMERICALS

Classification: [GATE CORE] **Priority:** ~~Ø=VERY~~ HIGH **Purpose:** Full-syllabus numerical integration before final revision.

This chapter mixes:

- ISA
- Addressing modes
- ALU
- Control unit
- Memory
- Cache
- I/O
- Interrupts
- DMA
- Pipelining
- Hazards
- CPI/performance

The goal is not just to know formulas separately. It is to recognize **which topic a mixed question is actually testing**.

24.1 MASTER MIXED-PROBLEM STRATEGY

When you see a large COA question:

Step 1 — Identify the block

Ask:

Is this ISA, memory, cache, I/O, pipeline, or performance?

Step 2 — Write only the relevant formula

Do not start calculating immediately.

Step 3 — Check units

Especially:

- bits vs bytes
- KB/MB
- ns vs cycles
- hit vs miss %
- address vs block number

Step 4 — Draw a tiny diagram if needed

Especially for:

- addressing modes

- cache mapping
- pipeline timing
- DMA
- memory interfacing

Step 5 — Calculate.

PART A — ISA + INSTRUCTION FORMAT

24.2 Problem 1 — Opcode Bits

An ISA supports 64 instructions.

Minimum opcode bits:

$$\lceil \log_2 64 \rceil = 6$$

$$\boxed{6}$$

24.3 Problem 2 — Register Field

Processor has:

$$32$$

general-purpose registers.

Bits required for one register field:

$$\lceil \log_2 32 \rceil$$

$$\boxed{5}$$

If instruction contains 3 register operands:

$$3(5) = 15$$

register-selection bits.

24.4 Problem 3 — Instruction Size

Instruction format contains:

- opcode = 6 bits
- three register operands
- 32 registers

Register bits:

$$3(5) = 15$$

Total:

$$6 + 15 = 21$$

Minimum instruction width:

$$\boxed{21 \text{ bits}}$$

if no other fields/padding are required.

24.5 Problem 4 — Maximum Opcodes

Instruction has a 7-bit opcode.

Maximum distinct opcode patterns:

$$2^7$$

$$2^{128}$$

24.6 Problem 5 — Expanding Opcode

16-bit instruction.

Three-address instructions use:

- three 4-bit address fields.

Address bits:

$$12$$

Opcode bits:

$$16 - 12 = 4$$

Maximum 3-address instructions:

$$2^4 = 16$$

Suppose 14 are used.

Unused primary opcode patterns:

$$16 - 14 = 2$$

Each unused 4-bit prefix can be extended by another 4-bit field for 2-address instructions.

Patterns:

$$2 \times 2^4$$

$$32$$

possible 2-address opcodes, under this expanding-opcode structure.

PART B — ADDRESSING MODES

24.7 Problem 6 — Direct Addressing

Instruction address field:

$$A = 500$$

Memory:

$$M[500] = 900$$

Direct:

$\boxed{EA=A}$

$\boxed{EA=500}$

Operand:

$M[500]=900$

$\boxed{\text{Operand}=900}$

24.8 Problem 7 — Indirect Addressing

Given:

$A=500$

$M[500]=900$

$M[900]=40$

Indirect:

$EA=M[500]=900$

Operand:

$M[900]$

$\boxed{40}$

24.9 Problem 8 — Indexed Addressing

Address field:

1000

Index register:

$R=60$

$EA=1000+60$

$\boxed{1060}$

24.10 Problem 9 — PC Relative

Current architectural PC used for addressing:

2000

Signed displacement:

-100

$EA=2000-100$

$\boxed{1900}$

Be careful: some questions define PC as already incremented. Use the exact convention provided.

24.11 Problem 10 — Auto-Increment

Register:

[R=1000]

word size:

[4B]

Auto-increment operand access:

[EA=1000]

then:

[R=1004]

Answer:

$\text{[\boxed{EA=1000, \text{NewR}=1004}]}$

PART C — ALU + NUMBER REPRESENTATION

24.12 Problem 11 — 8-Bit Signed Range

$\text{[} -2^7 \text{]}$

to:

$\text{[} 2^{7-1} \text{]}$

$\text{[\boxed{-128 \text{ to } 127}]}$

24.13 Problem 12 — 2's Complement

Find 8-bit representation of:

$\text{[} -25 \text{]}$

$\text{[} 25=00011001 \text{]}$

Invert:

````text id="fm0ugf" 11100110`

Add 1:

````text id="50p98d"  
11100111`

$\text{[\boxed{11100111_2}]}$

24.14 Problem 13 — Signed Overflow

8-bit signed:

$$\text{\[100+50 \]}$$

Both positive.

True result:

$$\text{\[150 \]}$$

but maximum representable:

$$\text{\[127 \]}$$

Therefore:

$$\text{\[\boxed{\text{Signed overflow occurs}} \]}$$

Shortcut:

same-sign inputs + opposite-sign stored result != overflow.

24.15 Problem 14 — Carry vs Overflow

4-bit:

$$\text{\[1111+0001 \]}$$

Unsigned:

$$\text{\[15+1=16 \]}$$

stored 4-bit result:

$$\text{\[0000 \]}$$

Carry out:

$$\text{\[1 \]}$$

But signed:

$$\text{\[-1+1=0 \]}$$

which is valid.

So:

$$\text{\[\boxed{\text{Carry=1, \ Overflow=0}} \]}$$

This proves carry and signed overflow are different.

24.16 Problem 15 — Full Adder

Given:

$$\text{\[A=1, \ B=1, \ C_{in}=0 \]}$$

Sum:

$$\text{\[A\oplus B\oplus C_{in} \]}$$

$$\text{\[1\oplus 1\oplus 0=0 \]}$$

Carry:

$$\lceil AB + (A \oplus B)C_{in} \rceil$$

$$\lceil 1 + 0 = 1 \rceil$$

$$\boxed{\text{Sum} = 0, \text{Carry} = 1}$$

PART D — CONTROL UNIT

24.17 Problem 16 — Control Memory Size

Microprogrammed control unit has:

- 1024 microinstructions
- each 40 bits wide.

Control memory:

$$\lceil 1024 \times 40 \rceil$$

$$\boxed{40960 \text{ bits}}$$

In bytes:

$$\lceil 40960 / 8 \rceil$$

$$\boxed{5120 \text{ B} = 5 \text{ KB}}$$

24.18 Problem 17 — CAR Bits

Control memory contains:

$$\lceil 2048 \rceil$$

microinstructions.

Control Address Register bits:

$$\lceil \log_2 2048 \rceil$$

$$\boxed{11}$$

24.19 Problem 18 — Encoded Control Field

A field must select one of 10 mutually exclusive micro-operations, plus "none."

Choices:

$$\lceil 11 \rceil$$

Bits:

$$\lceil \lceil \log_2 11 \rceil \rceil$$

$$\boxed{4}$$

Trap: do not use $\lceil \log_2 10 \rceil$ if "none" must be encoded.

24.20 Problem 19 — Hardwired vs Microprogrammed

Question:

Which is generally faster?

$\boxed{\text{Hardwired control}}$

Which is easier to modify?

$\boxed{\text{Microprogrammed control}}$

PART E — MEMORY FUNDAMENTALS + INTERFACING

24.21 Problem 20 — Memory Capacity

Memory:

$4K \times 16$

Locations:

$4K = 4096 = 2^{12}$

Each location:

$16 \text{ bits} = 2B$

Capacity:

$4096(2)$

$\boxed{8192B = 8KB}$

24.22 Address and Data Lines

For:

$4K \times 16$

Address lines:

$\boxed{12}$

Data lines:

$\boxed{16}$

24.23 Problem 21 — Address Space

32-bit byte-addressed processor:

2^{32} bytes

$\boxed{4GB}$

address space.

24.24 Problem 22 — Word Addressable

20 address bits.

Memory is word-addressable.

Word size:

$$\lfloor 32 \text{ bits} \rfloor = 4 \text{ B}$$

Capacity:

$$\lfloor 2^{20} \times 4 \text{ B} \rfloor$$

$$\lfloor 2^{22} \text{ B} \rfloor$$

$$\boxed{4 \text{ MB}}$$

24.25 Problem 23 — Number of Chips

Build:

$$\lfloor 64 \text{ K} \times 16 \rfloor$$

using:

$$\lfloor 16 \text{ K} \times 4 \rfloor$$

chips.

Depth expansion:

$$\lfloor 64 \text{ K} / 16 \text{ K} = 4 \rfloor$$

Width expansion:

$$\lfloor 16 / 4 = 4 \rfloor$$

Total:

$$\lfloor 4 \times 4 \rfloor$$

$$\boxed{16 \text{ chips}}$$

24.26 Problem 24 — Decoder Size

Previous design has:

$$\lfloor 4 \rfloor$$

banks.

Bank-select bits:

$$\lfloor \log_2 4 = 2 \rfloor$$

Need:

$$\boxed{2 \text{-to-} 4 \text{ decoder}}$$

Important:

24.27 Problem 25 — Address Range

A 4 KB memory block starts at:

8000H

Size:

```
\[
4KB=1000_H
\]
```

End:

```
\[
8000_H+1000_H-1
\]
```

```
\[
\boxed{8FFF_H}
\]
```

PART F — CACHE MAPPING

24.28 Problem 26 — Direct-Mapped Fields

Given:

- 32-bit byte address
- Cache = 32 KB
- Block = 64 B
- direct mapped

Cache:

```
\[
32KB=2^{15}B
\]
```

Block:

```
\[
2^6B
\]
```

Lines:

```
\[
2^{15-6}=2^9
\]
```

Thus:

```
\[
Offset=6
\]
```

\[
Index=9
\]

\[
Tag=32-9-6
\]

\[
\boxed{17}
\]

Final:

\[
\boxed{17|9|6}
\]

24.29 Problem 27 – 4-Way Cache

Same cache:

\[
512\text{ lines}
\]

4-way:

\[
 $512/4=128=2^7$
\]

Set bits:

\[
7
\]

Offset:

\[
6
\]

Tag:

\[
32-7-6
\]

\[
\boxed{19}
\]

24.30 Problem 28 – Fully Associative

Same 32-bit address, 64 B block.

Offset:

\[
6
\]

\]

No index/set field.

Tag:

\[

32-6

\]

\[

\boxed{26}

\]

24.31 Problem 29 – Block to Direct Line

Direct cache has:

\[

16

\]

lines.

Memory block:

\[

100

\]

Line:

\[

$100 \bmod 16$

\]

\[

\boxed{4}

\]

Tag:

\[

$\left\lfloor 100/16 \right\rfloor$

\]

\[

\boxed{6}

\]

24.32 Problem 30 – Set Mapping

Cache:

- 16 lines

- 4-way.

Sets:

\[

$16/4=4$

\]

Block:

$\lfloor$
100
 $\rfloor$

Set:

$\lfloor$
 $100 \bmod 4$
 $\rfloor$

$\lfloor$
 $\boxed{0}$
 $\rfloor$

Set-relative tag:

$\lfloor$
 $\left\lfloor 100/4 \right\rfloor$
 $\rfloor$

$\lfloor$
 $\boxed{25}$
 $\rfloor$

PART G – CACHE HIT/MISS

24.33 Problem 31 – Direct Cache Sequence

Direct cache:

- 4 lines
- initially empty.

Block sequence:

```text id="3ljtz8"  
0,4,0,4,1,5,1

Mapping:

```text id="613yev" 0 !' line 0 4 !' line 0 1 !' line 1 5 !' line 1

Simulate:

```text id="kyc9ic"  
0 M  
4 M  
0 M  
4 M  
1 M  
5 M  
1 M

$\lfloor \boxed{0} \text{ hits, } 7 \text{ misses} \rfloor$

Conflict-heavy sequence.

## 24.34 Problem 32 — Same Cache, 2-Way

4 total lines, 2-way:

`\[ \text{sets} \]`

0 and 4 both map set 0 but can coexist.

1 and 5 both set 1 and can coexist.

Sequence:

```text id="1wjkoj" 0 M 4 M 0 H 4 H 1 M 5 M 1 H`

```
\[
\boxed{3\ hits,\ 4\ misses}
\]
```

24.35 Problem 33 — LRU

Fully associative cache with 3 lines.

Sequence:

```
``text id="8k781b"
1,2,3,1,4,2
```

LRU result:

`\[ \boxed{1\ hit,\ 5\ misses} \]`

because after 1 is reused, 2 becomes LRU and gets evicted by 4.

24.36 Problem 34 — FIFO

Same sequence:

```text id="53vuab" 1,2,3,1,4,2`

FIFO does not update order on hit.

``4` evicts `1`.`

Final ``2`` hits.

```
\[
\boxed{2\ hits,\ 4\ misses}
\]
```

---

# PART H — CACHE PERFORMANCE

# 24.37 Problem 35 — AMAT

Hit time:

```
\[
```

1ns

\]

Miss rate:

\[

5\%

\]

Miss penalty:

\[

80ns

\]

\[

AMAT=1+0.05(80)

\]

\[

$\boxed{5\text{ns}}$

\]

---

# 24.38 Problem 36 – Two-Level Cache

L1:

\[

$T_1=1\text{ns}, m_1=0.1$

\]

L2:

\[

$T_2=5\text{ns}, m_2=0.2$

\]

Memory:

\[

100ns

\]

\[

AMAT=

$1+0.1(5+0.2(100))$

\]

\[

$=1+0.1(25)$

\]

\[

$\boxed{3.5\text{ns}}$

\]

---

# 24.39 Problem 37 – Global Miss Rate

Using previous:

\[

$m_{\text{global}}=0.1(0.2)$

\]

\[  
\boxed{2\%}  
\]

---

# 24.40 Problem 38 – Dirty Write-Back

Fetch cost:

\[  
100ns  
\]

Dirty-victim probability:

\[  
30\%  
\]

Write-back cost:

\[  
100ns  
\]

Average miss penalty:

\[  
 $100 + 0.3(100)$   
\]

\[  
\boxed{130ns}  
\]

---

# 24.41 AMAT with Dirty Victim

Hit time:

\[  
2ns  
\]

Miss rate:

\[  
4\%  
\]

\[  
 $AMAT = 2 + 0.04(130)$   
\]

\[  
 $2 + 5.2$   
\]

\[  
\boxed{7.2ns}  
\]

---

#### # 24.42 Problem 39 — Metadata

Cache:

- 1024 lines
- tag = 18 bits
- valid = 1
- dirty = 1.

Metadata:

\[  
1024(20)  
\]

\[  
\boxed{20480\ bits}  
\]

---

#### # PART I — I/O + INTERRUPTS

#### # 24.43 Problem 40 — Memory-Mapped I/O Range

Address range:

```text id="5b7o6m"  
F000H - FFFFH

Number of addresses:

\[FFFF_H-F000_H+1 \]

\[=1000_H \]

\[=4096 \]

If byte-addressed:

\[\boxed{4KB} \]

24.44 Problem 41 — Port Address Bits

System supports:

\[512 \]

I/O ports.

Bits:

\[\log_2 512 \]

\[\boxed{9} \]

24.45 Problem 42 — Polling Utilization

CPU:

$\backslash 2\text{GHz} \backslash$

Polls:

$\backslash 10^6/\text{s} \backslash$

Cost:

$\backslash 200 \backslash \text{cycles/poll} \backslash$

Polling cycles:

$\backslash 2 \times 10^8 \backslash$

CPU cycles:

$\backslash 2 \times 10^9 \backslash$

Fraction:

$\backslash 0.1 \backslash$

$\backslash \boxed{10\%} \backslash$

24.46 Problem 43 — Interrupt Utilization

CPU:

$\backslash 1\text{GHz} \backslash$

Interrupt rate:

$\backslash 20000/\text{s} \backslash$

Cost:

$\backslash 1000 \backslash \text{cycles} \backslash$

Cycles consumed:

$\backslash 2 \times 10^7 \backslash$

Fraction:

$\backslash 0.02 \backslash$

$\backslash \boxed{2\%} \backslash$

24.47 Problem 44 — Maximum Interrupt Rate

CPU:

$\backslash 2\text{GHz} \backslash$

Maximum 5% CPU time for interrupts.

Cost:

$\backslash 500 \backslash \text{cycles/interrupt} \backslash$

Available cycles:

$\backslash 0.05(2 \times 10^9) = 10^8 \backslash$

Rate:

$$\lfloor 10^8/500 \rfloor$$

$$\lfloor \boxed{200000/s} \rfloor$$

24.48 Problem 45 — Vector Table

256 vectors.

Each entry:

$$\lfloor 8B \rfloor$$

Total:

$$\lfloor 256(8) \rfloor$$

$$\lfloor \boxed{2048B=2KB} \rfloor$$

PART J — DMA

24.49 Problem 46 — DMA Transfer Count

Block:

$$\lfloor 32KB \rfloor$$

Each transfer:

$$\lfloor 8B \rfloor$$

Transfers:

$$\lfloor \frac{32 \times 1024}{8} \rfloor$$

$$\lfloor \boxed{4096} \rfloor$$

24.50 Problem 47 — DMA Bandwidth

Bus cycle:

$$\lfloor 20ns \rfloor$$

Each DMA cycle transfers:

$$\lfloor 4B \rfloor$$

If DMA gets every bus cycle:

$$\lfloor \frac{4}{20ns} \rfloor$$

$$\lfloor =200 \times 10^6B/s \rfloor$$

$$\lfloor \boxed{200MB/s} \rfloor$$

24.51 Problem 48 — Cycle Stealing

DMA steals:

$\sqrt{25\%}$

of bus cycles.

Bus maximum:

$\sqrt{200\text{MB/s}}$

DMA bandwidth:

$\sqrt{0.25(200)}$

$\sqrt{\boxed{50\text{MB/s}}}$

under proportional-transfer assumptions.

24.52 Problem 49 — Burst Mode

DMA transfers 16 words.

Each transfer:

$\sqrt{10\text{ns}}$

Arbitration overhead:

$\sqrt{40\text{ns}}$

Burst time:

$\sqrt{40+16(10)}$

$\sqrt{\boxed{200\text{ns}}}$

24.53 Problem 50 — Burst Bandwidth

Each word:

$\sqrt{4\text{B}}$

Total:

$\sqrt{16(4)=64\text{B}}$

Time:

$\sqrt{200\text{ns}}$

Bandwidth:

$\sqrt{64/(200\text{ns})}$

$\sqrt{\boxed{320\text{MB/s}}}$

using decimal rate units.

PART K — PIPELINE BASICS

24.54 Problem 51 — Ideal Pipeline Cycles

5 stages.

100 instructions.

$$\lfloor 5+100-1 \rfloor$$

$$\lfloor \boxed{104 \text{ cycles}} \rfloor$$

24.55 Problem 52 — Pipeline with Stalls

Same pipeline.

20 stall cycles.

$$\lfloor 104+20 \rfloor$$

$$\lfloor \boxed{124 \text{ cycles}} \rfloor$$

24.56 Problem 53 — Pipeline Clock

Stages:

$$\lfloor 2,3,5,4,2ns \rfloor$$

Register overhead:

$$\lfloor 0.5ns \rfloor$$

Clock:

$$\lfloor 5+0.5 \rfloor$$

$$\lfloor \boxed{5.5ns} \rfloor$$

24.57 Problem 54 — Pipeline Time

For:

$$\lfloor k=5, \text{quad } n=100, \text{quad } S=10 \rfloor$$

Clock:

$$\lfloor 5.5ns \rfloor$$

Cycles:

$$\lfloor 5+100-1+10 \rfloor$$

$$\lfloor 114 \rfloor$$

Time:

$$\lfloor 114(5.5) \rfloor$$

$$\lfloor \boxed{627ns} \rfloor$$

PART L — DATA HAZARDS

24.58 Problem 55 — RAW Identification

```
```text id="f372um" ADD R1,R2,R3 SUB R4,R1,R5
```

First writes R1.

Second reads R1.

```
\[
\boxed{RAW}
\]
```

---

# 24.59 Problem 56 — WAR

```
```text id="63alcp"
ADD R5,R1,R2
SUB R1,R3,R4
```

First reads R1.

Second writes R1.

```
\[ \boxed{WAR} \]
```

24.60 Problem 57 — WAW

```
```text id="be25h3" ADD R1,R2,R3 MUL R1,R4,R5
```

Both write R1.

```
\[
\boxed{WAW}
\]
```

---

# 24.61 Problem 58 — ALU Forwarding

```
```text id="t49xzx"
I1: ADD R1,R2,R3
I2: SUB R4,R1,R5
```

With standard full forwarding:

```
\[ \boxed{0\ stalls} \]
```

24.62 Problem 59 — Load Use

```
```text id="jv777j" I1: LW R1,0(R2) I2: ADD R3,R1,R4
```

Standard 5-stage pipeline with forwarding:

```
\[
```

```
\boxed{1\ stall}
\]
```

---

# 24.63 Problem 60 – Scheduling

Original:

```
```text id="np2fbn"
LW R1,0(R2)
ADD R3,R1,R4
SUB R5,R6,R7
```

Reorder:

```
```text id="ilpyy9" LW R1,0(R2) SUB R5,R6,R7 ADD R3,R1,R4
```

Independent SUB fills the load-use gap.

Potential stall:

```
\[
\boxed{0}
\]
```

---

# PART M – CONTROL HAZARDS

# 24.64 Problem 61 – Branch CPI

Branches:

```
\[
20\%
\]
```

Prediction accuracy:

```
\[
90\%
\]
```

Penalty:

```
\[
3
\]
```

Misprediction:

```
\[
10\%
\]
```

Branch penalty/instruction:

```
\[
0.2(0.1)(3)
\]
```

```
\[
```

```

\boxed{0.06}
\]

If base CPI = 1:

\[
\boxed{1.06}
\]

24.65 Problem 62 – Predict Not Taken

Branches:

\[
25\%
\]

60% taken.

Penalty:

\[
2
\]

Wrong predictions:

\[
\text{taken}
\]

Average:

\[
0.25(0.6)(2)
\]

\[
=0.30
\]

\[
\boxed{\text{CPI}=1.30}
\]

24.66 Problem 63 – Required Accuracy

Base CPI:

\[
1
\]

Branch fraction:

\[
0.2
\]

Penalty:

\[

```

4  
\\]

Target CPI:

\\[  
1.08  
\\]

Let misprediction  $m$ :

\\[  
 $1.08 = 1 + 0.2m(4)$   
\\]

\\[  
 $0.08 = 0.8m$   
\\]

\\[  
 $m = 0.1$   
\\]

Accuracy:

\\[  
 $\boxed{90\%}$   
\\]

---

# PART N – CPU PERFORMANCE

# 24.67 Problem 64 – CPU Time

IC:

\\[  
 $10^9$   
\\]

CPI:

\\[  
1.5  
\\]

Clock:

\\[  
3GHz  
\\]

\\[  
 $T = \frac{10^9(1.5)}{3 \times 10^9}$   
\\]

\\[  
 $\boxed{0.5s}$   
\\]

---

# 24.68 Problem 65 – Weighted CPI



Instruction mix:

- 40% CPI 1
- 40% CPI 2
- 20% CPI 4

\[  
CPI=  
 $0.4(1)+0.4(2)+0.2(4)$   
\]

\[  
 $=0.4+0.8+0.8$   
\]

\[  
 $\boxed{2.0}$   
\]

---

# 24.69 Problem 66 – MIPS

Clock:

\[  
3GHz  
\]

CPI:

\[  
1.5  
\]

\[  
MIPS=  
 $\frac{3 \times 10^9}{1.5 \times 10^6}$   
\]

\[  
 $\boxed{2000 \text{ MIPS}}$   
\]

---

# 24.70 Problem 67 – Compare CPUs

CPU A:

- 2 GHz
- CPI 1.1

CPU B:

- 3 GHz
- CPI 1.8

Same IC.

Compare:

\[  
A:  $\frac{1.1}{2} = 0.55$   
\]

$$\begin{aligned} & \backslash[ \\ & B:\frac{1.8}{3}=0.60 \\ & \backslash] \end{aligned}$$

Smaller is faster.

Therefore:

$$\begin{aligned} & \backslash[ \\ & \boxed{A \text{ is faster}} \\ & \backslash] \end{aligned}$$

Speedup:

$$\begin{aligned} & \backslash[ \\ & 0.60/0.55 \\ & \backslash] \end{aligned}$$

$$\begin{aligned} & \backslash[ \\ & \boxed{\approx 1.091 \text{ times}} \\ & \backslash] \end{aligned}$$

---

# PART 0 – FULLY MIXED GATE-STYLE PROBLEMS

# 24.71 Mixed Problem 1 – Cache + CPI

Processor:

- Base CPI = 1
- 1 instruction-cache access/instruction
- 0.3 data accesses/instruction
- unified cache miss rate = 2%
- miss penalty = 50 cycles

References/instruction:

$$\begin{aligned} & \backslash[ \\ & 1+0.3=1.3 \\ & \backslash] \end{aligned}$$

Cache stalls:

$$\begin{aligned} & \backslash[ \\ & 1.3(0.02)(50) \\ & \backslash] \end{aligned}$$

$$\begin{aligned} & \backslash[ \\ & =1.3 \\ & \backslash] \end{aligned}$$

Actual CPI:

$$\begin{aligned} & \backslash[ \\ & 1+1.3 \\ & \backslash] \end{aligned}$$

$$\begin{aligned} & \backslash[ \\ & \boxed{2.3} \\ & \backslash] \end{aligned}$$

---

# 24.72 Add Branches

Suppose additionally:

- 20% branches
- accuracy = 90%
- penalty = 3

Branch contribution:

$$\begin{aligned} & \backslash[ \\ & 0.2(0.1)(3)=0.06 \\ & \backslash] \end{aligned}$$

Total CPI:

$$\begin{aligned} & \backslash[ \\ & 2.3+0.06 \\ & \backslash] \end{aligned}$$

$$\begin{aligned} & \backslash[ \\ & \boxed{2.36} \\ & \backslash] \end{aligned}$$

---

#### # 24.73 Add Load-Use Stalls

Suppose:

- 10% instructions suffer one load-use stall.

Contribution:

$$\begin{aligned} & \backslash[ \\ & 0.10 \\ & \backslash] \end{aligned}$$

Total:

$$\begin{aligned} & \backslash[ \\ & 2.36+0.10 \\ & \backslash] \end{aligned}$$

$$\begin{aligned} & \backslash[ \\ & \boxed{2.46} \\ & \backslash] \end{aligned}$$

This is the kind of integration you should expect.

---

#### # 24.74 Mixed Problem 2 – CPU Time

Using:

$$\begin{aligned} & \backslash[ \\ & \text{CPI}=2.46 \\ & \backslash] \end{aligned}$$

IC:

$$\begin{aligned} & \backslash[ \\ & 5\times 10^8 \\ & \backslash] \end{aligned}$$

Clock:

\[  
2.5GHz  
\]

\[  
T=  
\frac{5\times10^8(2.46)}{2.5\times10^9}  
\]

\[  
\boxed{0.492s}  
\]

---

#### # 24.75 Mixed Problem 3 – Cache + Dirty Write-Back

Given:

- cache hit time = 1 ns
- miss rate = 3%
- fetch = 80 ns
- 25% victims dirty
- write-back = 80 ns

Average miss penalty:

\[  
80+0.25(80)  
\]

\[  
=100ns  
\]

AMAT:

\[  
1+0.03(100)  
\]

\[  
\boxed{4ns}  
\]

---

#### # 24.76 Mixed Problem 4 – AMAT to Cycles

CPU:

\[  
2GHz  
\]

Clock period:

\[  
0.5ns  
\]

AMAT:

\[

4ns

\]

Equivalent average memory access time:

\[

4/0.5

\]

\[

\boxed{8\text{ cycles}}

\]

Be careful: that is not automatically "8 stall cycles/instruction"; you still need access frequency and baseline assumptions.

---

# 24.77 Mixed Problem 5 – Interrupt vs DMA

Device rate:

\[

8MB/s

\]

If one interrupt per byte:

\[

$8 \times 10^6$

\]

interrupts/s using decimal MB.

If DMA transfers 4 KB/block:

\[

$\frac{8 \times 10^6}{4096}$

\]

\[

approx 1953

\]

completion interrupts/s.

Huge difference.

Core conclusion:

\[

\boxed{\text{DMA greatly reduces CPU event overhead for bulk transfers}}

\]

---

# 24.78 Mixed Problem 6 – Pipeline + Branch + Cache

Processor:

- Base CPI = 1
- 15% load-use, 1 stall
- 20% branches
- 95% predictor accuracy
- branch penalty 4
- cache stalls = 0.5 cycles/instruction

Load penalty:

```
\[
0.15
\]
```

Branch:

```
\[
0.2(0.05)(4)=0.04
\]
```

Cache:

```
\[
0.5
\]
```

Total:

```
\[
1+0.15+0.04+0.5
\]
```

```
\[
\boxed{1.69}
\]
```

---

# 24.79 Mixed Problem 7 – New Clock Design

Old:

```
\[
CPI=1.69,\ Clock=2GHz
\]
```

New pipeline:

- clock = 3 GHz
- deeper branch penalty causes CPI = 2.2

Compare:

Old factor:

```
\[
1.69/2=0.845
\]
```

New:

```
\[
2.2/3\approx0.7333
\]
```

New is faster.

Speedup:

```
\[
0.845/0.7333
\]
```

$\boxed{\approx 1.152 \times}$

---

#### # 24.80 Mixed Problem 8 – Address + Cache Mapping

System:

- 16-bit byte address
- direct cache = 256 B
- block = 16 B
- address = ``0xABCD``

Cache:

$256 = 2^8$

Block:

$16 = 2^4$

Index:

$8 - 4 = 4$

Tag:

$16 - 4 - 4 = 8$

Thus:

```text id="u8fg3z"  
AB | C | D
Tag Index Offset

$\boxed{\text{Tag}=\text{AB}_H, \text{Index}=\text{C}_H, \text{Offset}=\text{D}_H}$

24.81 Mixed Problem 9 — Memory Chip Construction

Need:

$32\text{K} \times 32$

Using:

$8\text{K} \times 8$

Depth factor:

$32\text{K} / 8\text{K} = 4$

Width factor:

$32 / 8 = 4$

Total chips:

$$\boxed{16}$$

Banks:

$$4$$

Chips per bank:

$$4$$

Bank-select decoder:

$$\boxed{2^{\text{to-}4}}$$

24.82 Mixed Problem 10 — Pipeline Stall Count

5-stage pipeline.

100 instructions.

- 10 load-use stalls
- 5 structural stalls
- 6 branch misprediction penalties, each 2 cycles

Branch stalls:

$$6(2)=12$$

Total stalls:

$$10+5+12=27$$

Ideal cycles:

$$104$$

Actual:

$$104+27$$

$$\boxed{131}$$

Exact CPI:

$$131/100$$

$$\boxed{1.31}$$

24.83 Mixed Problem 11 — Amdahl

40% of runtime is memory stalls.

New cache makes memory-stall portion 5× faster.

$$\text{Speedup} = \frac{1}{0.6 + \frac{0.4}{5}}$$

$$= \frac{1}{0.68}$$

$$\boxed{\approx 1.471 \text{ times}}$$

24.84 Mixed Problem 12 — Control Memory + Opcode

A processor has:

- 128 machine instructions
- microprogrammed control
- 2048 control-memory words
- control word = 48 bits.

Opcode bits:

$\lceil \log_2 128 \rceil$

$\boxed{7}$

CAR bits:

$\lceil \log_2 2048 \rceil$

$\boxed{11}$

Control store:

$2048(48)$

$= 98304$ bits

$\boxed{12\text{KB}}$

because:

$98304/8 = 12288\text{B} = 12\text{KB}$

PART P — HIGH-YIELD GATE TRAPS ACROSS COA

24.85 Trap 1 — Bits vs Bytes

If memory says:

$1\text{K} \times 16$

it contains:

1024×16 bits

not 16 KB.

Convert correctly.

24.86 Trap 2 — Address Bits vs Data Bits

Address lines depend on:

$\boxed{\text{number of locations}}$

Data lines depend on:

$\boxed{\text{bits per location}}$

24.87 Trap 3 — EA vs Operand

Direct addressing:

$\boxed{EA=A}$

Operand:

$\boxed{M[A]}$

Do not return memory value when question asks only EA.

24.88 Trap 4 — Carry vs Overflow

Carry is mainly unsigned.

Overflow is signed.

They can differ.

24.89 Trap 5 — Cache Block Number vs Byte Address

If given byte address:

$\boxed{Block = \left\lfloor \frac{Address}{BlockSize} \right\rfloor}$

Do not map the raw address modulo lines unless block size is 1.

24.90 Trap 6 — k-Way Means k Lines per Set

Not k sets.

$\boxed{Sets = \frac{Lines}{k}}$

24.91 Trap 7 — Fully Associative Index

$\boxed{0 \text{ bits}}$

24.92 Trap 8 — Write Allocate

Write allocate describes:

$\boxed{\text{write miss behavior}}$

not write-hit behavior.

24.93 Trap 9 — Dirty Bit

Dirty means:

cache block has changed relative to lower memory.

It does not mean "invalid" or "recently used."

24.94 Trap 10 — Polling vs Interrupt vs DMA

``text id="aby96i" Polling !' CPU checks Interrupt !' device alerts CPU DMA !' controller moves data

24.95 Trap 11 — DMA Doesn't Mean Zero CPU Work

CPU still typically:

- configures DMA,
- handles completion.

24.96 Trap 12 — RAW/WAR/WAW

Always analyze earlier instruction relative to later instruction.

\[
W\to R = RAW
\]

\[
R\to W = WAR
\]

\[
W\to W = WAW
\]

24.97 Trap 13 — Forwarding Solves Immediate Load-Use

Not fully in classic 5-stage pipeline.

Usually:

\[
\boxed{1\ stall}
\]

24.98 Trap 14 — Pipeline Speedup = Number of Stages

Only asymptotic ideal maximum for balanced pipeline.

24.99 Trap 15 — Higher Clock Means Faster

Not necessarily.

Use:

\[
\boxed{\frac{IC \times CPI}{ClockRate}}
\]

24.100 Trap 16 – Branch Accuracy

If accuracy:

\[
95\%
\]

misprediction:

\[
\boxed{5\%}
\]

24.101 Trap 17 – Taken Rate Among Branches

If:

- 20% instructions are branches
- 60% branches taken

then taken branches among all instructions:

\[
 $0.2(0.6)=0.12$
\]

\[
\boxed{12\%}
\]

24.102 Trap 18 – Miss Penalty Wording

Some problems define miss penalty as:

- additional time after cache lookup,

others as:

- total miss-path time.

If unclear, build a path-time weighted average.

24.103 Trap 19 – Same-Cycle WB/ID Timing

Exact no-forwarding stall count can change depending on whether:

- WB write first half,
- ID read second half.

Always inspect assumption.

24.104 Trap 20 – Decoder Count in Memory Interfacing

Depth expansion determines banks.

Width expansion means parallel chips inside one bank.

Decoder generally chooses banks.

24.105 Trap 21 – MIPS

Higher MIPS across different ISAs does not necessarily mean lower execution time.

PART Q – RAPID-FIRE CONCEPT CHECK

24.106

Which cache mapping has no conflict misses caused solely by placement restrictions?

\[
\boxed{\text{Fully associative}}
\]

Capacity misses can still occur.

24.107

Which cache mapping needs no victim-selection algorithm?

\[
\boxed{\text{Direct mapped}}
\]

24.108

Which policy generally needs a dirty bit?

\[
\boxed{\text{Write-back}}
\]

24.109

Which addressing mode is most natural for arrays?

\[
\boxed{\text{Indexed}}
\]

24.110

Which addressing mode is natural for relocatable branches?

\[
\boxed{\text{PC-relative}}
\]

24.111

Which control unit is generally faster?

```
\[
\boxed{\text{Hardwired}}
\]
```

24.112

Which control unit is easier to modify?

```
\[
\boxed{\text{Microprogrammed}}
\]
```

24.113

What determines the clock of an unbalanced pipeline?

```
\[
\boxed{\text{Slowest stage + register overhead}}
\]
```

24.114

What is the main data hazard in a basic in-order 5-stage pipeline?

```
\[
\boxed{\text{RAW}}
\]
```

24.115

What does forwarding primarily reduce?

```
\[
\boxed{\text{RAW stalls}}
\]
```

24.116

What does branch prediction primarily reduce?

```
\[
\boxed{\text{Control-hazard penalty}}
\]
```

24.117

What does DMA primarily reduce?

```
\[
```

```
\boxed{\text{CPU involvement in bulk I/O transfer}}
\]
```

PART R – MIXED PRACTICE SET

Try these without looking back.

Q1 – L1 NAT

A processor supports 256 opcodes and 64 registers.

How many bits are needed for:

1. opcode,
2. one register field?

Q2 – L1 NAT

An 18-bit byte-addressed system has how much address space in KB?

Q3 – L1 NAT

A memory chip is $(8K \times 16)$.

Find:

1. address lines,
2. data lines,
3. total capacity in KB.

Q4 – L2 NAT

Build $(64K \times 32)$ memory using $(16K \times 8)$ chips.

Find total chips.

Q5 – L2 NAT

A 32-bit byte-addressed system has 64 KB direct-mapped cache with 64-byte blocks.

Find:

1. offset bits,
2. index bits,
3. tag bits.

Q6 – L2 NAT

Same cache as Q5, but 4-way set associative.

Find set bits and tag bits.

Q7 — L2

A direct cache has 8 lines.

Reference block sequence:

```
```text id="a92cbc"
0,8,0,1,9,1
```

Find hits and misses.

---

## Q8 — L2 NAT

Hit time = 2 ns.

Miss rate = 4%.

Fetch penalty = 80 ns.

25% of victims are dirty and write-back cost = 80 ns.

Find AMAT.

---

## Q9 — L2 NAT

CPU = 2 GHz.

Interrupt rate = 50,000/s.

Each interrupt requires 1000 cycles.

Find CPU percentage used by interrupts.

---

## Q10 — L2 NAT

DMA transfers 64 KB using 8-byte transfers.

How many DMA transfers occur?

---

## Q11 — L2 NAT

A 5-stage pipeline processes 100 instructions and suffers 15 total stall cycles.

Find total cycles.

---

## Q12 — L2

With full forwarding:

```
```text id="j65k92" LW R1,0(R2) ADD R3,R1,R4 SUB R5,R3,R6
```

Find total data-hazard stall cycles.

Q13 – L2 NAT

20% instructions are branches.

Predictor accuracy = 95%.

Penalty = 4 cycles.

Find branch stall cycles/instruction.

Q14 – L2 NAT

Base CPI = 1.

Load-use stalls = 0.1 cycles/instruction.

Branch stalls = answer from Q13.

Cache stalls = 0.4.

Find total CPI.

Q15 – L3 NAT

A program executes 10^9 instructions on a 2.5 GHz CPU with CPI from Q14.

Find execution time.

Q16 – L3 MSQ

Which statements are correct?

- A. RAW is a true dependency.
- B. WAR can usually be removed by register renaming.
- C. Fully associative cache has index bits.
- D. Write-back normally requires dirty tracking.
- E. DMA can use a completion interrupt.

Q17 – L3 MSQ

Which may reduce pipeline stalls?

- A. Forwarding
- B. Instruction scheduling
- C. Branch prediction
- D. Increasing every cache tag bit arbitrarily
- E. Duplicating a contended hardware resource

Q18 – L3

A 4-stage pipeline has stage delays:

\[
2, \ 3, \ 5, \ 4ns

\]

Register overhead = 1 ns.

For 50 instructions and 5 stalls, find:

1. pipeline clock,
2. pipeline execution time,
3. non-pipelined execution time,
4. speedup.

Q19 – L3

Processor A:

- $\text{IC} = 10^9$
- CPI 1.2
- 2 GHz

Processor B:

- $\text{IC} = 0.9 \times 10^9$
- CPI 1.5
- 3 GHz

Find execution time for both and speedup of faster over slower.

Q20 – L3

40% of runtime is improved 4×.

Find overall speedup using Amdahl's law.

CHAPTER 24 – FINAL MIXED FORMULA SHEET

ISA

\[
Bits = $\lceil \log_2 N \rceil$
\]

Memory

\[
Capacity = Locations $\times$ BitsPerLocation
\]

Chips

\[
N_{chips} = $\frac{\text{RequiredDepth}}{\text{ChipDepth}} \times \frac{\text{RequiredWidth}}{\text{ChipWidth}}$
\]

Cache

\[

```

Lines=\frac{Cache}{Block}
\]

\[
Sets=\frac{Lines}{Ways}
\]

\[
Set=Block\bmod Sets
\]

### AMAT

\[
AMAT=HitTime+MissRate\times MissPenalty
\]

### Dirty penalty

\[
P=Fetch+dw
\]

### Interrupt utilization

\[
U=\frac{Rate\times Cost}{ClockRate}
\]

### Pipeline

\[
Cycles=k+n-1+Stalls
\]

### Branch

\[
CPI_{\{branch\}}=f_{bmP}
\]

### CPU time

\[
CPUTime=
\frac{IC\times CPI}{ClockRate}
\]

### Amdahl

\[
Speedup=
\frac{1}{1-F+F/S}
\]

---

# CHAPTER 24 – 10 LAST-MINUTE RULES

1. **Address != block != set/line != tag.**
2. **EA is not the same as operand.**
3. **Carry is unsigned; overflow is signed.**
4. **Direct mapping has no victim choice.**
5. **Write-back uses dirty state.**
6. **Polling asks; interrupt alerts; DMA transfers.**
7. **RAW is the main classic pipeline data hazard.**

```

```

8.  **ALU!' ALU forwarding = usually 0 stalls; load!' use =
9.  **Branch x wrong x penalty gives branch CPI cost.**
10. **CPU performance always comes back to IC x CPI / clock rate.**

```

```
---
```

CHAPTER 24 COMPLETION STATUS

```

'  I S A   m i x e d   n u m e r i c a l s
'  A d d r e s s i n g - m o d e   n u m e r i c a l s
'  A L U   /   o v e r f l o w
'  C o n t r o l - u n i t   c a l c u l a t i o n s
'  M e m o r y   c a p a c i t y
'  M e m o r y   i n t e r f a c i n g
'  C a c h e   m a p p i n g
'  C a c h e   r e p l a c e m e n t
'  C a c h e   A M A T
'  D i r t y   w r i t e - b a c k
'  I / O   a d d r e s s i n g
'  P o l l i n g   o v e r h e a d
'  I n t e r r u p t   o v e r h e a d
'  D M A   t r a n s f e r s
'  P i p e l i n e   e x e c u t i o n   t i m e
'  D a t a   h a z a r d s
'  F o r w a r d i n g / l o a d - u s e
'  B r a n c h   p r e d i c t i o n
'  C P I   i n t e g r a t i o n
'  C P U   p e r f o r m a n c e
'  A m d a h l ' s   l a w
'  C r o s s - t o p i c   G A T E   t r a p s

```

```
# CHAPTER 24: COMPLETE '
```

```
---
```

COURSE PROGRESS

```
**Completed: 25 / 26 modules**
```

```

\[
\boxed{\approx 96\%}
\]

```

Only one module remains.

```
# NEXT !' CHAPTER 25 - FINAL COA REVISION + OFFICIAL SYLL
```

In the final chapter I will give you:

- complete official-syllabus checklist,
- addressing-mode EA cheat sheet,
- ALU/overflow cheat sheet,
- hardwired vs microprogrammed table,
- memory-interfacing formulas,
- complete cache cheat sheet,
- polling vs interrupt vs DMA table,
- pipeline formula sheet,
- RAW/WAR/WAW + forwarding cheat sheet,
- branch prediction formulas,
- top GATE traps,
- last-day revision notes,
- final mixed self-test,
- and then verify line-by-line whether **every official COA syllabus item is genuinely complete**.

USER

next

ASSISTANT

CHAPTER 25 — FINAL COA REVISION + OFFICIAL SYLLABUS AUDIT

Classification: [FINAL REVISION] **Priority:** 0-1% **ESSENTIAL** **Status after this chapter:** 26 / 26 modules

This chapter is your **last-day COA revision sheet**.

The aim is not to reteach everything from zero. It is to compress the complete subject into formulas, comparisons, traps, and recognition rules you can revise quickly.

PART 1 — OFFICIAL GATE COA SYLLABUS

Your official syllabus was:

Instruction set and addressing modes. Design of arithmetic and logic unit (ALU). Design of control unit – hardwired and microprogrammed. Memory interfacing and hierarchy: performance, cache memory mapping. I/O interface (interrupt and DMA). Instruction pipelining, pipeline hazards.

We now audit it line by line.

| Official topic | Status |
|-------------------------|------------|
| Instruction set | ' COMPLETE |
| Addressing modes | ' COMPLETE |
| ALU design | ' COMPLETE |
| Hardwired control | ' COMPLETE |
| Microprogrammed control | ' COMPLETE |
| Memory hierarchy | ' COMPLETE |
| Memory performance | ' COMPLETE |
| Memory interfacing | ' COMPLETE |
| Cache memory mapping | ' COMPLETE |
| I/O interface | ' COMPLETE |
| Interrupts | ' COMPLETE |
| DMA | ' COMPLETE |
| Instruction pipelining | ' COMPLETE |
| Pipeline hazards | ' COMPLETE |

PART 2 — ISA ULTRA-FASTREVISION

25.1 Instruction Structure

Typical instruction:

$\boxed{\text{Opcode} + \text{Operand fields}}$

Opcode tells:

What operation?

Operands tell:

On what data?

25.2 Opcode Bits

If processor supports (N) instructions:

$\boxed{\text{OpcodeBits} = \lceil \log_2 N \rceil}$

Example:

$N = 64$

$\boxed{6 \text{ bits}}$

25.3 Register Field Bits

For (R) registers:

$\boxed{\text{RegisterBits} = \lceil \log_2 R \rceil}$

32 registers:

$\boxed{5 \text{ bits/register}}$

25.4 Instruction Types

Remember:

- Data transfer
 - Arithmetic
 - Logical
 - Shift
 - Control/branch
-

25.5 Number of Explicit Addresses

3-address

TEXT

ADD R1, R2, R3

$[R1 = R2 + R3]$

2-address

TEXT

ADD R1, R2

$[R1 = R1 + R2]$

Destination usually overwritten.

1-address

Accumulator implicit.

0-address

Stack operands implicit.

25.6 ISA Trap

Zero-address instruction does not mean zero operands.

Operands are implicit on stack.

PART 3 — ADDRESSING MODES CHEAT SHEET

Use:

- $\backslash(A)$ = address/displacement field
- $\backslash(R)$ = register
- $\backslash(IX)$ = index register
- $\backslash(BR)$ = base register
- $\backslash(PC)$ = program counter

25.7 Immediate

$[\boxed{\text{Operand} = A}]$

Example:

TEXT

MOV R1, #10

10 itself is data.

No separate memory access to fetch operand.

25.8 Direct

$\boxed{EA=A}$

$\boxed{Operand=M[A]}$

25.9 Memory Indirect

$\boxed{EA=M[A]}$

$\boxed{Operand=M[M[A]]}$

Extra memory access required.

25.10 Register

$\boxed{Operand=R}$

Fast.

25.11 Register Indirect

$\boxed{EA=R}$

$\boxed{Operand=M[R]}$

Pointer-like behavior.

25.12 Indexed

$\boxed{EA=A+IX}$

Think:

arrays.

25.13 Base/Displacement

$\boxed{EA=BR+A}$

Useful for:

- relocation,
 - structures,
 - stack frames.
-

25.14 PC Relative

$\boxed{EA = PC + \text{SignedDisplacement}}$

Think:

branches.

25.15 Auto-Increment

$EA = R_{\text{old}}$

then:

$R \leftarrow R + d$

25.16 Auto-Decrement

First:

$R \leftarrow R - d$

then:

$\boxed{EA = R_{\text{new}}}$

25.17 Addressing Memory Trick

Immediate = value Direct = address Indirect = address of address Register indirect = pointer
Indexed = array PC-relative = branch

PART 4 — NUMBER REPRESENTATION+ ALU CHEAT SHEET

25.18 Unsigned n-Bit Range

$\boxed{0 \text{ to } 2^{n-1}}$

25.19 Signed 2's Complement Range

$\boxed{-2^{n-1} \text{ to } 2^{n-1}-1}$

25.20 2's Complement

To obtain negative:

25.21 Half Adder

$$\boxed{S=A\oplus B}$$

$$\boxed{C=AB}$$

25.22 Full Adder

$$\boxed{S=A\oplus B\oplus C_{in}}$$

$$\boxed{C_{out}=AB+(A\oplus B)C_{in}}$$

Equivalent:

$$\boxed{AB+AC_{in}+BC_{in}}$$

25.23 Ripple Carry Adder

Carry propagates stage by stage.

Main problem:

$$\boxed{\text{Carry delay}}$$

Delay approximately grows with number of bits.

25.24 Carry Look-Ahead

Common convention:

$$\boxed{G_i=A_iB_i}$$

$$\boxed{P_i=A_i\oplus B_i}$$

Then:

$$\boxed{C_{i+1}=G_i+P_iC_i}$$

Remember the problem may define P_i differently.

Use the definition given.

25.25 Subtraction

$$\boxed{A-B=A+\overline{B}+1}$$

Adder/subtractor:

$$\boxed{B_i'=B_i\oplus M}$$

and:

$\lceil C_0 = M \rceil$

M = 0

Addition.

M = 1

Subtraction.

25.26 Signed Overflow

Addition shortcut:

Same-sign inputs producing opposite-sign result.

Example:

TEXT

P o s i t i v e + P o s i t i v e ! ' N e g a t i v e

overflow.

Or:

TEXT

N e g a t i v e + N e g a t i v e ! ' P o s i t i v e

overflow.

25.27 Overflow Formula

$\lceil \boxed{V = C_{\text{intoMSB}} \oplus C_{\text{outMSB}}} \rceil$

25.28 Carry vs Overflow

Carry

Important for:

$\lceil \boxed{\text{unsigned arithmetic}} \rceil$

Overflow

Important for:

$\lceil \boxed{\text{signed arithmetic}} \rceil$

They are not the same.

25.29 Common Flags

Z

Zero.

C

Carry.

N/S

Negative/sign.

V

Overflow.

PART 5 — CONTROL UNIT CHEAT SHEET

25.30 Main Function

Control unit decides:

which micro-operation occurs and when.

Datapath performs work.

25.31 Hardwired Control

Implemented using:

- logic,
- gates,
- decoders,
- counters,
- finite-state control.

Advantages:

$\boxed{\text{Fast}}$

Disadvantages:

$\boxed{\text{Harder to modify}}$

25.32 Microprogrammed Control

Control signals stored as microinstructions in:

$\boxed{\text{Control Memory}}$

Advantages:

- flexible,
- easier to design/modify.

Disadvantage:

- generally slower than hardwired.

25.33 Hardwired vs Microprogrammed

| Feature | Hardwired | Microprogrammed |
|-----------------------------|-----------|-----------------|
| Speed | Higher | Lower |
| Flexibility | Lower | Higher |
| Modification | Hard | Easier |
| Complex instruction support | Harder | Easier |
| Control storage | Logic | Control memory |

25.34 Microprogram Hierarchy

TEXT

Machine instruction

!"

Microprogram

!"

Microinstructions

!"

Control signals

!"

Micro-operations

25.35 Horizontal Microprogramming

- wide control word
- little encoding
- more parallel control
- larger memory
- less decoding

25.36 Vertical Microprogramming

- narrower
- encoded
- less parallel
- smaller control memory
- more decoding

25.37 Control Memory Size

$\boxed{\text{ControlMemoryBits} = \text{NumberOfWords} \times \text{BitsPerWord}}$

25.38 CAR Bits

If control memory has (N) microinstructions:

$\boxed{\text{CARBits} = \lceil \log_2 N \rceil}$

PART 6 — MEMORY FUNDAMENTALS

25.39 Hierarchy

Fastest !' slowest:

```
TEXT
Registers
    !"
Cache
    !"
Main Memory
    !"
SSD/HDD
```

Going upward:

- faster
- smaller
- more expensive per bit

25.40 Locality

Temporal

Recently used data likely reused.

Spatial

Nearby locations likely accessed.

Memory trick:

Temporal = Time Spatial = Space

25.41 Address Space

With (A) address bits:

$\boxed{2^A \text{ addresses}}$

25.42 Byte-Addressable Memory

$$\boxed{\text{Capacity} = 2^A \text{ bytes}}$$

25.43 Word-Addressable Memory

If each word is (W) bytes:

$$\boxed{\text{Capacity} = 2^A \times W \text{ bytes}}$$

25.44 Memory Organization

$$N \times M$$

means:

- (N) locations
- (M) bits/location

Total:

$$\boxed{N \times M \text{ bits}}$$

Address lines:

$$\boxed{\lceil \log_2 N \rceil}$$

Data lines:

$$\boxed{M}$$

25.45 Address Range

Inclusive:

$$\boxed{\text{Size} = \text{End} - \text{Start} + 1}$$

and:

$$\boxed{\text{End} = \text{Start} + \text{Size} - 1}$$

PART 7 — MEMORY INTERFACING CHEAT SHEET

25.46 Number of Chips

$$\boxed{\text{Chips} = \frac{\text{RequiredDepth}}{\text{ChipDepth}} \times \frac{\text{RequiredWidth}}{\text{ChipWidth}}}$$

25.47 Depth Expansion

Example:

$$\boxed{64K/16K = 4}$$

means 4 banks.

25.48 Width Expansion

Example:

$$\lfloor 32/8=4 \rfloor$$

means four chips operate in parallel per bank.

25.49 Decoder

If there are B banks:

$$\lfloor \log_2 B \rfloor$$

bank-select bits.

Example:

4 banks:

$$\lfloor 2^{\text{to-}4} \text{ decoder} \rfloor$$

25.50 Memory Interfacing Rule

Low address bits select location inside chip.

High address bits select chip/bank.

25.51 Partial Decoding

Not decoding all high address bits may cause:

$$\boxed{\text{Address aliasing / mirroring}}$$

Same physical memory may respond to multiple addresses.

PART 8 — CACHE MASTER CHEAT SHEET

25.52 Number of Cache Lines

$$\lfloor L = \frac{\text{CacheSize}}{\text{BlockSize}} \rfloor$$

25.53 Number of Sets

For k -way:

$$\boxed{S = \frac{L}{k}}$$

25.54 Direct-Mapped Address

TEXT

Tag | Index | Offset

$$\boxed{\text{Line} = \text{Block} \bmod L}$$

25.55 Set-Associative

TEXT

Tag | Set | Offset

$$\boxed{\text{Set} = \text{Block} \bmod S}$$

25.56 Fully Associative

TEXT

Tag | Offset

No index/set bits.

Block can occupy:

$$\boxed{\text{any line}}$$

25.57 Offset Bits

For byte-addressed cache:

$$\boxed{\text{Offset} = \log_2(\text{BlockSizeBytes})}$$

25.58 Direct-Mapped Bits

$$\boxed{\text{Index} = \log_2 L}$$

$$\boxed{\text{Tag} = A - \text{Index} - \text{Offset}}$$

25.59 Set-Associative Bits

$$\boxed{\text{SetBits} = \log_2 S}$$

$$\boxed{\text{Tag} = A - \text{SetBits} - \text{Offset}}$$

25.60 Power-of-Two Shortcut

If:

- $\text{address} = \lfloor A \rfloor$ bits
- $\text{cache} = \lfloor 2^C \rfloor$ bytes
- $\text{block} = \lfloor 2^B \rfloor$ bytes
- $\text{associativity} = \lfloor 2^W \rfloor$

then:

$$\lfloor \boxed{\text{Offset} = B} \rfloor$$

$$\lfloor \boxed{\text{SetBits} = C - B - W} \rfloor$$

$$\lfloor \boxed{\text{Tag} = A - C + W} \rfloor$$

25.61 Direct-Mapped Shortcut

For direct map:

$$\lfloor \boxed{\text{Tag} = A - C} \rfloor$$

because:

$$\lfloor \text{Index} + \text{Offset} = C \rfloor$$

25.62 Block Number

$$\lfloor \boxed{\text{Block} = \left\lfloor \frac{\text{Address}}{\text{BlockSize}} \right\rfloor} \rfloor$$

Offset:

$$\lfloor \boxed{\text{Address} \bmod \text{BlockSize}} \rfloor$$

25.63 Direct-Mapped Conflict

Blocks:

$$\lfloor j, j+L, j+2L, \dots \rfloor$$

map to same cache line.

25.64 Set Mapping

For $\lfloor S \rfloor$ sets:

$$\lfloor j, j+S, j+2S, \dots \rfloor$$

map to same set.

25.65 Cache Miss Types

Compulsory

First access.

Capacity

Cache too small.

Conflict

Placement restriction.

Fully associative removes placement-based conflict misses.

PART 9 — CACHE REPLACEMENT

25.66 LRU

Evict:

$\boxed{\text{least recently accessed}}$

Hit changes recency.

25.67 FIFO

Evict:

$\boxed{\text{oldest inserted}}$

Hit normally does not change insertion order.

25.68 Random

Select random eligible victim.

25.69 Direct Mapping Trap

Direct mapping has no victim selection.

Incoming block has exactly one target line.

PART 10 — CACHE WRITE POLICIES

25.70 Write-Through

On write hit:

$\boxed{\text{Cache + lower memory updated immediately}}$

Pros:

- simpler
-

- memory more current.

Cons:

- more traffic.

Memory trick:

Write Through = Write Twice

25.71 Write-Back

On hit:

$\boxed{\text{Update cache only}}$

Set:

$\boxed{\text{Dirty}=1}$

Write lower memory when dirty block is evicted/flushed.

Memory trick:

Write Back = Back later

25.72 Write Allocate

On write miss:

$\boxed{\text{Bring block into cache, then write}}$

25.73 No-Write Allocate

On write miss:

$\boxed{\text{Write around cache}}$

Do not allocate block.

25.74 Common Pairings

Often:

$\boxed{\text{Write-back} + \text{Write-allocate}}$

and:

$\boxed{\text{Write-through} + \text{No-write-allocate}}$

But these are not mandatory combinations.

25.75 Dirty Victim Penalty

If:

- fetch cost = (F)
- dirty probability = (d)
- write-back cost = (W)

then:

$$\boxed{\text{AverageMissPenalty} = F + dW}$$

under serial-cost assumption.

PART 11 — CACHE PERFORMANCE

25.76 Miss Rate

$$\boxed{m = 1 - h}$$

where (h) = hit rate.

25.77 AMAT

$$\boxed{\text{AMAT} = \text{HitTime} + \text{MissRate} \times \text{MissPenalty}}$$

25.78 Two-Level Cache

$$\boxed{\text{AMAT} = T_1 + m_1(T_2 + m_2P)}$$

where (m_2) is local L2 miss rate.

25.79 Global L2 Miss Rate

$$\boxed{m_1 m_2}$$

25.80 Universal Timing Method

If definitions are confusing:

$$\boxed{\text{AverageTime} = \sum p_i T_i}$$

Draw all paths and average them.

PART 12 — I/O MASTER REVISION

25.81 I/O Interface Registers

Memory trick:

DSC

Data

actual data.

Status

ready/busy/error.

Control

commands/settings.

25.82 Polling

CPU repeatedly asks device:

Ready?

Can cause:

$\boxed{\text{Busy waiting}}$

25.83 Interrupt

Device tells CPU:

I need service.

CPU runs ISR.

25.84 DMA

Controller performs bulk:

$\boxed{\text{Device} \rightarrow \text{Memory}}$

transfer.

25.85 Three-Method Memory Trick

Ask !' Alert !' Automate

Polling !' Interrupt !' DMA

25.86 Memory-Mapped I/O

Memory and I/O share:

$\boxed{\text{one address space}}$

Normal memory-style instructions can often be used.

25.87 Isolated I/O

Separate:

$\boxed{\text{I/O port address space}}$

Often special I/O instructions.

25.88 Handshaking

Typical:

TEXT

R e q u e s t ! ' A c k n o w l e d g e

Used for asynchronous coordination.

PART 13 — INTERRUPT CHEAT SHEET

25.89 ISR

$\boxed{\text{Interrupt Service Routine}}$

handles interrupt.

25.90 Interrupt Flow

TEXT

```
Interrupt request
! "
Finish/current boundary
! "
Save state
! "
Run ISR
! "
Restore state
! "
Resume program
```

25.91 Maskable

Can normally be disabled/masked.

25.92 NMI

Non-maskable interrupt.

Usually critical/high priority.

25.93 Vectored Interrupt

Vector helps locate correct ISR.

25.94 Non-Vectored

Common/fixed entry may require source identification.

25.95 Daisy Chain

Physical order creates priority.

TEXT

C P U ! ' D 1 ! ' D 2 ! ' D 3

Priority:

$\boxed{D1 > D2 > D3}$

25.96 Interrupt CPU Utilization

If:

- interrupt rate = $\lambda(R)$
- cost = $\lambda(C)$ cycles
- CPU clock = $\lambda(f)$

then:

$\boxed{U = \frac{RC}{f}}$

Percentage:

$\boxed{100U}$

PART 14 — DMA CHEAT SHEET

25.97 DMA Registers

Memory trick:

ACC

- Address
- Count
- Control

25.98 DMA Cycle

TEXT

```
CPU programs DMA
! "
DMA requests bus
! "
DMA transfers
! "
Address/count updated
! "
Completion
! "
Interrupt CPU
```

25.99 Burst Mode

DMA holds bus for multiple transfers.

Pros:

- high transfer efficiency.

Cons:

- CPU may wait longer.

25.100 Cycle Stealing

DMA takes individual bus cycles.

$\boxed{\text{CPU loses some bus opportunities}}$

25.101 Transparent DMA

DMA uses bus only when CPU is not using it.

Lowest CPU interference, potentially slower transfer.

25.102 Number of DMA Transfers

$\boxed{N = \frac{\text{TotalData}}{\text{BytesPerTransfer}}}$

25.103 DMA Bandwidth

$$\boxed{BW = \text{Transfers/sec} \times \text{Bytes/Transfer}}$$

or:

$$\boxed{BW = \frac{\text{Data}}{\text{Time}}}$$

PART 15 — PIPELINING MASTER CHEAT SHEET

25.104 Classic Pipeline

$$\boxed{IF \rightarrow ID \rightarrow EX \rightarrow MEM \rightarrow WB}$$

Memory trick:

I Don't Eat Much Bread

25.105 Ideal Pipeline Cycles

For k stages and n instructions:

$$\boxed{k+n-1}$$

25.106 With Stalls

$$\boxed{\text{Cycles} = k+n-1+S}$$

25.107 Clock Period

For unequal stages:

$$\boxed{T_c = \max(t_i) + t_r}$$

where t_r is pipeline-register overhead.

25.108 Pipeline Time

$$\boxed{T_p = (k+n-1+S)T_c}$$

25.109 Non-Pipelined Time

For unequal stage delays:

$$\boxed{T_{np} = n \sum t_i}$$

25.110 Speedup

$$\boxed{\text{Speedup} = \frac{T_{np}}{T_p}}$$

For ideal equal-stage pipeline:

$$\boxed{\text{Speedup} = \frac{nk}{n+k-1}}$$

As $n \rightarrow \infty$:

$$\boxed{\text{Speedup} \rightarrow k}$$

25.111 Efficiency

Ideal equal-stage:

$$\boxed{E = \frac{n}{n+k-1}}$$

25.112 Throughput vs Latency

Latency

Time for one instruction to cross pipeline.

Throughput

Instructions completed per time.

Pipelining mainly improves:

$$\boxed{\text{Throughput}}$$

25.113 Ideal CPI

For a long scalar pipeline:

$$\boxed{CPI_{ideal} \approx 1}$$

PART 16 — HAZARDS MASTER REVISION

25.114 Hazard Types

Memory trick:

SDC

- Structural
- Data
- Control

25.115 Structural Hazard

Two instructions need same hardware resource simultaneously.

Example:

one memory needed by IF and MEM.

Solutions:

- stall,
- duplicate resource,
- multi-port resource.

25.116 RAW

Earlier:

\[Write\ X \]

Later:

\[Read\ X \]

\[\boxed{RAW} \]

True dependency.

25.117 WAR

Earlier:

\[Read\ X \]

Later:

\[Write\ X \]

\[\boxed{WAR} \]

Anti-dependency.

25.118 WAW

Earlier and later both:

\[Write\ X \]

\[\boxed{WAW} \]

Output dependency.

25.119 Fast Dependency Table

| Earlier | Later | Type |
|---------|-------|------|
| Write | Read | RAW |
| Read | Write | WAR |
| Write | Write | WAW |

25.120 Classic 5-Stage In-Order Pipeline

Main register hazard:

$\boxed{\text{RAW}}$

WAR/WAW typically do not become timing hazards because reads/writes occur in fixed order.

PART 17 — FORWARDING CHEAT SHEET

25.121 ALU Result Availability

$\boxed{\text{After EX}}$

25.122 Load Result Availability

$\boxed{\text{After MEM}}$

25.123 Consumer ALU Needs Operand

$\boxed{\text{In EX}}$

25.124 Full Forwarding Typical Results

ALU !' immediate ALU

$\boxed{0 \text{ stalls}}$

LOAD !' immediate ALU

$\boxed{1 \text{ stall}}$

LOAD !' ALU after one independent instruction

$\boxed{0 \text{ stalls}}$

25.125 No Forwarding Typical Rule

Classic WB-write-first / ID-read-second convention:

TEXT

| | | |
|-------------|--------------------|------------|
| Producer !' | immediate consumer | = 2 stalls |
| Producer !' | one-gap consumer | = 1 stall |
| Producer !' | two-gap consumer | = 0 stalls |

Only use this shortcut when assumptions match.

25.126 Safe Data-Hazard Method

Ask:

Ready when? Needed when? Forwarding path exists?

If producer value arrives too late:

$\boxed{\text{stall}}$

PART 18 — CONTROL HAZARD CHEAT SHEET

25.127 Control Hazard

Branch makes next PC uncertain.

Solutions:

- stall,
- predict taken,
- predict not taken,
- dynamic prediction,
- flush if wrong,
- resolve branch earlier.

25.128 Misprediction Rate

$\boxed{m=1-\text{Accuracy}}$

25.129 Branch CPI Penalty

If:

- branch fraction = f_b
- misprediction rate = m
- penalty = P

then:

$\boxed{\text{BranchStall}=f_b m P}$

and:

$$\boxed{CPI = CPI_{\text{base}} + f_{\text{bmP}}}$$

Memory trick:

Branch × Wrong × Penalty

25.130 Predict Not Taken

Wrong when branch is:

$$\boxed{\text{Taken}}$$

So:

$$\boxed{\text{Penalty} = f_b(\text{TakenFraction})P}$$

25.131 Predict Taken

Wrong when branch is:

$$\boxed{\text{Not taken}}$$

25.132 Branch Trap

Branch frequency and taken rate are different.

If:

- 20% instructions branches
- 60% of branches taken

then taken branches among all instructions:

$$0.2(0.6)$$

$$\boxed{12\%}$$

PART 19 — CPU PERFORMANCE FINAL CHEAT SHEET

25.133 CPU Cycles

$$\boxed{\text{Cycles} = IC \times CPI}$$

25.134 CPU Time

$$\boxed{\text{CPUTime} = IC \times CPI \times \text{ClockCycleTime}}$$

or:

$$\boxed{\text{CPUTime} = \frac{IC \times CPI}{\text{ClockRate}}}$$

25.135 Average CPI

$$\boxed{CPI_{avg} = \sum_i f_i CPI_i}$$

25.136 MIPS

$$\boxed{MIPS = \frac{ClockRate}{CPI \times 10^6}}$$

But:

Do not use MIPS blindly to compare different ISAs.

25.137 Speedup

$$\boxed{Speedup = \frac{OldTime}{NewTime}}$$

25.138 Amdahl's Law

If fraction (F) is improved by factor (S) :

$$\boxed{OverallSpeedup = \frac{1}{(1-F) + F/S}}$$

25.139 Combined CPI

For a long pipeline:

$$\boxed{CPI = CPI_{base} + DataStalls + ControlStalls + StructuralStalls + CacheStalls}$$

when average penalties are additive as given.

PART 20 — TOP 30 GATE COA TRAPS

These are worth revising repeatedly.

Trap 1

$$\boxed{1KB = 2^{10}B}$$

unless problem explicitly uses decimal units.

Trap 2

1 byte:

$$\boxed{8 \text{ bits}}$$

Trap 3

One hexadecimal digit:

$$\boxed{4 \text{ bits}}$$

Trap 4

Effective address "" operand.

Trap 5

Immediate mode has no memory EA for operand.

Trap 6

Memory indirect usually requires an extra memory access.

Trap 7

Carry "" signed overflow.

Trap 8

Positive + positive != negative means overflow.

Trap 9

Hardwired generally faster.

Trap 10

Microprogrammed generally more flexible.

Trap 11

Memory address lines depend on number of locations, not total bit capacity directly.

Trap 12

Byte-addressed and word-addressed memory formulas differ.

Trap 13

Width expansion does not itself need additional bank decoding.

Trap 14

Cache capacity usually refers to data capacity unless stated otherwise.

Trap 15

k-way means k lines per set.

Trap 16

Fully associative cache has zero index bits.

Trap 17

Direct map does not use LRU/FIFO victim choice.

Trap 18

Block number "" raw byte address.

Trap 19

Write-through and write-allocate are different decisions.

Trap 20

Write-back does eventually update memory.

Trap 21

Dirty bit means modified, not invalid.

Trap 22

Polling " interrupt.

Trap 23

Interrupt " DMA.

Trap 24

DMA does not mean CPU is completely uninvolved.

Trap 25

Pipeline clock uses slowest stage, not average stage.

Trap 26

Pipelining mainly improves throughput, not necessarily latency.

Trap 27

Dependency " guaranteed stall.

Trap 28

Forwarding does not eliminate classic immediate load-use stall.

Trap 29

Prediction accuracy = 95% means misprediction = 5%.

Trap 30

Higher clock frequency alone does not guarantee faster execution.

PART 21 — 5-MINUTE LAST-DAY FORMULA SHEET

If you have only five minutes, revise this section.

Addressing

$EA_{\text{direct}} = A$

$EA_{\text{indirect}} = M[A]$

$EA_{\text{regind}} = R$

$EA_{\text{indexed}} = A + IX$

$EA_{\text{base}} = A + BR$

$EA_{\text{PC}} = PC + d$

Signed range

$\boxed{-2^{n-1} \text{ to } 2^{n-1}-1}$

Full adder

$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = AB + (A \oplus B)C_{in}$$

Memory

$$Capacity = 2^A \times BytesPerAddress$$

Chips

$$Chips = DepthRatio \times WidthRatio$$

Cache

$$Lines = Cache / Block$$

$$Sets = Lines / Ways$$

$$Set = Block \bmod Sets$$

$$Offset = \log_2 Block$$

AMAT

$$AMAT = HitTime + MissRate \times MissPenalty$$

Dirty miss penalty

$$P = Fetch + dW$$

Interrupt utilization

$$U = \frac{InterruptRate \times CyclesPerInterrupt}{ClockRate}$$

DMA transfers

$$N = \frac{DataSize}{DataPerTransfer}$$

Pipeline

$$Cycles = k + n - 1 + Stalls$$

$$T_c = \max(StageDelay) + RegisterOverhead$$

Branch

$$CPI_{branch} = f_{bm}P$$

CPU

$$CPUTime = \frac{IC \times CPI}{ClockRate}$$

PART 22 — 1-MINUTE CACHE SHEET

TEXT

DIRECT

Tag | Index | Offset
Line = Block mod Lines

SET ASSOCIATIVE

Tag | Set | Offset
Set = Block mod Sets

FULLY ASSOCIATIVE

Tag | Offset
Any line

Remember:

TEXT

L R U !' least recently used
F I F O !' oldest inserted

W T !' memory now
W B !' memory later + dirty

W A !' bring block on write miss
N W A !' bypass cache on write miss

PART 23 — 1-MINUTE PIPELINE SHEET

TEXT

I F !' I D !' E X !' M E M !' W B

$\backslash$ Cycles=k+n-1+Stalls $\backslash$

$\backslash$ Clock=SlowestStage+RegisterOverhead $\backslash$

TEXT

S t r u c t u r a l !' resource conflict

R A W !' Write then Read
W A R !' Read then Write
W A W !' Write then Write

With full classic forwarding:

TEXT

A L U !' A L U = 0 stall
L O A D !' U S E = 1 stall

Branch:

$\backslash$ $\boxed{\text{Branch}}$ times Wrongtimes Penalty $\backslash$

PART 24 — 1-MINUTE I/O SHEET

TEXT

Polling:
CPU asks repeatedly.

Interrupt:
Device alerts CPU.

DMA:
Controller transfers data to/from memory.

DMA:

TEXT

Burst !' whole block / long bus ownership
Cycle steal !' individual bus cycles
Transparent !' idle bus cycles

PART 25 — FINAL MIXED SELF-TEST

Try these without notes.

Q1

A 32-bit byte-addressed machine has a 64 KB, 4-way cache with 64 B blocks.

Find:

- lines,
- sets,
- offset bits,
- set bits,
- tag bits.

Q2

A $(64K \times 32)$ memory is built from $(16K \times 8)$ chips.

Find:

- total chips,
- banks,
- chips per bank,
- decoder.

Q3

Find EA:

$[A=200, \text{quad } IX=30]$

for indexed addressing.

Q4

For an 8-bit signed number, determine whether:

$\lfloor 100+50 \rfloor$

overflows.

Q5

A cache has:

- hit time 1 ns,
- miss rate 4%,
- block fetch 100 ns,
- 20% dirty victims,
- write-back 100 ns.

Find AMAT.

Q6

CPU runs at 2 GHz.

Interrupt rate is 100,000/s.

Interrupt cost is 500 cycles.

Find CPU interrupt-utilization percentage.

Q7

A DMA transfer moves 64 KB using 4-byte units.

How many transfers?

Q8

A 5-stage pipeline runs 100 instructions with 10 stalls.

Find total cycles.

Q9

With full forwarding:

TEXT

LW R1, 0(R2)

ADD R3, R1, R4

How many stalls?

Q10

20% instructions are branches.

Predictor accuracy = 90%.

Misprediction penalty = 4 cycles.

Base CPI = 1.

Find CPI.

Q11

A processor executes 10^9 instructions with CPI = 1.5 at 3 GHz.

Find execution time.

Q12

Identify all dependencies:

TEXT

I1: ADD R1, R2, R3

I2: SUB R2, R1, R4

PART 26 — FINAL ANSWERS TO SELF-TEST

Q1

Cache:

$$64\text{KB} = 2^{16}$$

Block:

$$64\text{B} = 2^6$$

Lines:

$$2^{16-6} = 2^{10} = 1024$$

4-way:

$$\text{Sets} = 1024 / 4 = 256 = 2^8$$

Therefore:

$$\boxed{\text{Offset} = 6}$$

$$\boxed{\text{Set} = 8}$$

$$\boxed{\text{Tag} = 32 - 8 - 6 = 18}$$

Q2

Depth:

$$\lfloor 64K/16K=4 \rfloor$$

Width:

$$\lfloor 32/8=4 \rfloor$$

Total:

$$\lfloor \boxed{16 \text{ chips}} \rfloor$$

Banks:

$$\lfloor \boxed{4} \rfloor$$

Chips/bank:

$$\lfloor \boxed{4} \rfloor$$

Decoder:

$$\lfloor \boxed{2 \text{ to } 4} \rfloor$$

Q3

$$\lfloor EA=200+30 \rfloor$$

$$\lfloor \boxed{230} \rfloor$$

Q4

8-bit signed range:

$$\lfloor -128 \text{ to } 127 \rfloor$$

$$\lfloor 100+50=150 \rfloor$$

not representable.

$$\lfloor \boxed{\text{Overflow}} \rfloor$$

Q5

Dirty-adjusted miss penalty:

$$\lfloor 100+0.2(100) \rfloor$$

$$\lfloor =120\text{ns} \rfloor$$

AMAT:

$$\lfloor 1+0.04(120) \rfloor$$

$$\lfloor 1+4.8 \rfloor$$

$$\lfloor \boxed{5.8\text{ns}} \rfloor$$

Q6

Cycles/sec used:

$$\lfloor 100000(500)=5\times 10^7 \rfloor$$

Available:

$$\lfloor 2\times 10^9 \rfloor$$

$$\lfloor U=0.025 \rfloor$$

$$\lfloor \boxed{2.5\%} \rfloor$$

Q7

$$\lfloor 64\text{KB}=65536\text{B} \rfloor$$

$$\lfloor 65536/4 \rfloor$$

$$\lfloor \boxed{16384} \rfloor$$

Q8

$$\lfloor 5+100-1+10 \rfloor$$

$$\lfloor \boxed{114\text{ cycles}} \rfloor$$

Q9

Classic full forwarding:

$$\lfloor \boxed{1\text{ stall}} \rfloor$$

Q10

Misprediction:

$$\lfloor 0.1 \rfloor$$

Penalty:

$$\lfloor 0.2(0.1)(4)=0.08 \rfloor$$

CPI:

$$\lfloor \boxed{1.08} \rfloor$$

Q11

$$\lfloor T=\frac{10^9(1.5)}{3\times 10^9} \rfloor$$

$$\lfloor \boxed{0.5\text{s}} \rfloor$$

Q12

I1:

- reads R2,R3
- writes R1

I2:

- reads R1,R4
-

- writes R2

Therefore:

$\boxed{RAW \text{ on } R1}$

and:

$\boxed{WAR \text{ on } R2}$

FINAL OFFICIAL SYLLABUS AUDIT

Now compare against every official phrase.

1. Instruction set and addressing modes

Covered:

' Instruction format ' Opcode/operand fields ' 0/1/2/3 address instructions ' Instruction categories ' Expanding opcode ' Immediate ' Direct ' Indirect ' Register ' Register indirect ' Indexed ' Base/displacement ' PC relative ' Auto-increment/decrement ' Stack/implied addressing ' EA calculations ' Memory-access counting

STATUS: COMPLETE

2. Design of arithmetic and logic unit

Covered:

' Half adder ' Full adder ' Ripple carry ' Carry look-ahead ' Generate/propagate ' Addition/subtraction ' Increment/decrement ' Boolean operations ' Shifts ' ALU selection ' Flags ' Signed overflow ' Carry vs overflow ' Comparator intuition

STATUS: COMPLETE

3. Control unit — hardwired and microprogrammed

Covered:

' Control signals ' Micro-operations ' Timing ' Hardwired control ' Microprogrammed control ' Control memory ' Microinstruction ' CAR ' Microprogram sequencing ' Horizontal control ' Vertical control ' Control-store numerals

STATUS: COMPLETE

4. Memory interfacing and hierarchy: performance, cache memory mapping

Covered:

' Memory hierarchy ' Locality ' SRAM/DRAM ' Address space ' Byte/word addressing ' Memory organization ' Access time/cycle time/bandwidth ' Memory interfacing ' Depth/width expansion ' Chip selection ' Decoding ' Address ranges ' Direct cache ' Fully associative cache ' Set-associative cache ' Tag/index/set/offset ' Cache misses ' LRU/FIFO/random ' Write-through/back ' Write allocate/no-write allocate ' Dirty bit ' AMAT ' Multi-level cache ' Cache CPI ' Metadata ' Dirty write-back penalty

STATUS: COMPLETE

5. I/O interface — interrupt and DMA

Covered:

' I/O controller ' Data/status/control registers ' Programmed I/O ' Polling ' Memory-mapped I/O ' Isolated I/O ' Handshaking ' Interrupts ' ISR ' Context saving ' Maskable/NMI ' Vectored/non-vectored ' Priority ' Daisy chain ' Nested interrupts ' Interrupt overhead ' DMA ' DMA registers ' Bus arbitration ' Burst DMA ' Cycle stealing ' Transparent DMA ' DMA bandwidth/performance

STATUS: COMPLETE

6. Instruction pipelining, pipeline hazards

Covered:

' IF-ID-EX-MEM-WB ' Pipeline timing diagrams ' Latency ' Throughput ' Speedup ' Efficiency ' Pipeline clock ' Register overhead ' Stalls/bubbles ' Structural hazards ' RAW/WAR/WAW ' Dependency vs hazard ' Forwarding ' Load-use hazard ' NOP/scheduling ' Control hazard ' Branch prediction ' Flush ' Branch penalty ' CPI with hazards ' Combined CPU performance

STATUS: COMPLETE

FINAL COURSE STATUS

\[\boxed{\text{26 / 26 MODULES COMPLETE}} \]

' GATE COA SYLLABUS COVERAGE: COMPLETE

You have now covered the full syllabus from:

Instruction Set

all the way through:

Pipeline Hazards

with prerequisite foundations, formulas, numericals, traps, memory tricks, and mixed questions.

Your final COA revision order should be:

Addressing modes !' ALU/overflow !' Control Unit !' Memory Interfacing !' Cache mapping + AMAT !' Interrupt/DMA !' Pipeline formulas !' RAW/load-use !' Branch CPI !' CPU performance.

That sequence gives you the highest-value concepts with the least revision time.

USER

next

COA COURSE COMPLETE — NOW STARTING TEST SERIES

You finished the **entire GATE COA syllabus**.

So from here, **NEXT** will move into the planned **exam-practice phase** rather than adding unnecessary new theory.

TEST 1 — ISA + ADDRESSING MODES

Coverage:

- Instruction formats
- Opcode/register-bit calculations
- 0/1/2/3-address instructions
- Expanding opcode
- Effective address
- Immediate/direct/indirect
- Register/register-indirect
- Indexed/base/PC-relative
- Auto-increment/decrement
- Memory-access counting

Level: GATE mixed L0 ! L3 **Questions:** 20 **Types:** MCQ + MSQ + NAT

Try them before checking answers. I will give the **full solutions in the next response**.

Q1 — MCQ [L0]

A processor supports 128 different instructions.

Minimum number of opcode bits required is:

A. 6 B. 7 C. 8 D. 128

Q2 — NAT [L1]

A processor contains 64 general-purpose registers.

How many bits are required to identify one register?

Q3 — NAT [L1]

A 32-bit instruction contains:

- 8-bit opcode
- 3 register fields
- each register field selects one of 32 registers

How many bits remain for other fields?

Q4 — MCQ [L1]

Which instruction organization normally uses operands implicitly from the top of a stack?

A. Three-address B. Two-address C. One-address D. Zero-address

Q5 — MCQ [L1]

Consider:

TEXT

ADD R1, R2

with the convention:

$\backslash R1 \leftarrow R1 + R2 \backslash$

This is normally a:

A. 0-address instruction B. 1-address instruction C. 2-address instruction D. 3-address instruction

ADDRESSING MODES

Use the following values for Q6–Q9:

$\backslash A = 500 \backslash$

$\backslash R1 = 800 \backslash$

$\backslash IX = 40 \backslash$

$\backslash BR = 1000 \backslash$

$\backslash M[500] = 700 \backslash$

$\backslash M[700] = 90 \backslash$

Q6 — NAT [L1]

For **direct addressing**, find the effective address.

Q7 — NAT [L1]

For **memory-indirect addressing**, find the effective address.

Q8 — NAT [L1]

For **register-indirect addressing using R1**, find the effective address.

Q9 — NAT [L1]

For **indexed addressing**, using $\backslash A \backslash$ and $\backslash IX \backslash$, find effective address.

Q10 — NAT [L1]

For base/displacement addressing:

[BR=1000, displacement=-120]

Find effective address.

Q11 — NAT [L2]

A PC-relative branch has:

[PC=4000]

and signed displacement:

[-200]

Assume the given PC is the architectural PC value used in the calculation.

Find target effective address.

AUTO ADDRESSING

Q12 — NAT [L2]

A register contains:

[R=2000]

The machine is byte-addressed and operand size is 4 bytes.

An instruction uses **auto-increment register-indirect addressing**.

Find:

1. effective address used,
2. value of R after access.

Give your answer as:

TEXT

EA, NewR

Q13 — NAT [L2]

Same initial state:

[R=2000]

operand size = 4 bytes.

Now use **auto-decrement** addressing.

Find:

MEMORY ACCESS COUNTING

Assume:

- instruction itself occupies one memory word,
- instruction fetch counts as one memory access,
- ignore cache,
- instruction reads an operand but does not write a result to memory.

Q14 — MCQ [L2]

For an **immediate-addressing** instruction, total memory accesses are typically:

A. 0 B. 1 C. 2 D. 3

Q15 — MCQ [L2]

For **direct addressing**, total memory accesses are typically:

A. 1 B. 2 C. 3 D. 4

Q16 — MCQ [L2]

For **memory-indirect addressing**, total memory accesses are typically:

A. 1 B. 2 C. 3 D. 4

CONCEPTUAL TRAPS

Q17 — MSQ [L2]

Which statements are correct?

A. In immediate addressing, the address field itself represents the operand value.

B. In direct addressing,

[EA=A]

C. In register-indirect addressing, the register contains the operand itself.

D. In memory-indirect addressing,

$\text{[EA=M[A]}]$

E. PC-relative addressing is useful for branches.

Q18 — MSQ [L2]

Which statements are generally correct?

- A. Indexed addressing is useful for arrays.
- B. Base/displacement addressing is useful for relocation and structured data.
- C. Register addressing usually requires a memory access for the operand.
- D. Auto-increment uses the current register value as EA and then increments the register.
- E. Auto-decrement normally decrements the register before forming EA.

EXPANDING OPCODE

Q19 — NAT [L3]

A 16-bit instruction contains three 4-bit address fields for a 3-address format.

How many maximum 3-address opcodes are possible if the remaining bits are used entirely as opcode?

Q20 — NAT [L3]

For the same 16-bit architecture:

- three-address instructions use 3×4 -bit address fields,
- 12 of the possible primary opcode patterns are assigned to 3-address instructions,
- each unused primary opcode can be extended by one additional 4-bit field to encode 2-address instructions.

How many 2-address instruction opcodes can be formed from the unused primary opcode patterns?

BONUS GATE TRAP

A programmer writes:

TEXT

```
LOAD R1, #500
```

and:

TEXT

```
LOAD R2, 500
```

Assume `#` denotes immediate addressing and the second instruction uses direct addressing.

Which statement is correct?

- A. Both load the value stored at memory location 500. B. R1 gets 500; R2 gets $\text{\textbackslash}(M[500])$. C. R1 gets $\text{\textbackslash}(M[500])$; R2 gets 500. D. Both load 500.

TEST 1 QUICK ANSWER SHEET

Write your answers like this:

TEXT

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.
- 8.
- 9.
- 10.
- 11.
- 12.
- 13.
- 14.
- 15.
- 16.
- 17.
- 18.
- 19.
- 20.
- Bonus .

TEST SERIES PROGRESS

Test 1: ISA + Addressing !• CURRENT **Test 2:** ALU + Control Unit **Test 3:** Memory + Cache **Test 4:** I/O + Interrupt + DMA **Test 5:** Pipelining + Hazards **Final Test:** Full COA mixed mock

NEXT !' TEST 1 FULL SOLUTIONS + SHORTCUTS

I'll solve every question step-by-step, show the fast GATE method, explain traps, and calculate your conceptual score pattern.